Reference textbook: Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Algorithms

Full course handout

A4 · Two slides per page · Lin Cing

Contents

1. Analysis: growth, probability, and amortized costs

01 · The role of algorithms in computing 3
02 · Getting started 13
03 · Growth of functions 28
05 · Probabilistic analysis and randomized algorithms 41
17 · Amortized analysis 61

2. Toolbox: sorting and data structures

06 · Heapsort 71
08 · Sorting in linear time 84
10 · Elementary data structures 94
11 · Hash tables 104
12 · Binary search trees 116
13 · Red-black trees 129
14 · Augmenting data structures 152
18 · B-trees 163
19 · Fibonacci heaps 173
20 · van Emde Boas trees 183

3. Recurrence and divide-and-conquer

04 · Divide and conquer 194
07 · Quicksort 215
09 · Medians and order statistics 226
30 · Polynomials and FFT 237

4. Design: dynamic programming and greedy algorithms

15 · Dynamic programming 250
16 · Greedy algorithms 268

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algorithms

Full course handout

A4 · Two slides per page · Lin Cing

Contents

5. Graphs: traversal, connectivity, shortest paths, and flow

22 · Elementary graph algorithms 280
21 · Disjoint sets 298
23 · Minimum spanning trees 315
24 · Single-source shortest paths 331
25 · All-pairs shortest paths 348
26 · Maximum flow 363

6. Computational geometry

33 · Computational geometry 388

7. Further methods: parallelism, matrices, optimization, and strings

27 · Multithreaded algorithms 403
28 · Matrix operations 428
29 · Linear programming 441
31 · Number-theoretic algorithms 460
32 · String matching 483

8. Theory: intractability and approximation

34 · NP-completeness 497
35 · Approximation algorithms 523

Reference appendices: sums, sets, probability, and matrices

A · Summations 543
B · Sets, relations, functions, graphs, and trees 557
C · Counting and probability 575
D · Matrices 599

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The role of algorithms in computing

Part I Foundations

Scope

Specify legal inputs, acceptable outputs, and termination before comparing algorithms, representations, and hardware under one cost model. Duplicate-key sorting, paths and tours, and a numerical crossover distinguish general correctness, randomized guarantees, and finite testing.

Prerequisites

Basic functions, sets, program execution, and discrete input encodings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Distinguish a problem, its input domain, its output relation, and an algorithm specification.
- Describe algorithm quality through correctness, termination, and resource growth.
- Distinguish abstract efficiency from hardware-dependent constants.
- 1.1 Algorithms
- 1.2 Algorithms as a technology

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Algorithms

A computational problem specifies a legal input domain D and a set $R(x)$ of acceptable outputs for each $x \in D$. The same keys may admit several sorted outputs, so a specification is generally a relation rather than a uniquely valued formula. An instance is one particular x. An algorithm is an unambiguous executable procedure that transforms inputs into outputs.

Total correctness of a deterministic algorithm A requires termination and $A(x) \in R(x)$ for **every** $x \in D$. Partial correctness requires only that a terminating execution produce a legal result. Termination needs a separate proof. The behavior outside the legal domain, such as rejection, an undefined result, or a separate return value, must also be specified.

A finite program can describe an infinite loop. Finitely many pseudocode lines do not prove finite execution. Tests on selected inputs do not establish the universal quantifier in the specification.

Sorting preserves order and every occurrence

Let $A = [(3, a), (1, b), (3, c)]$, with records represented by (key, identity). Keys have a total order, and the comparison cost is specified separately. A legal sorted output must have nondecreasing keys and be a permutation of the same input records, with neither losses nor copies. The output $[(1, b), (3, a), (3, c)]$ satisfies both conditions. The ordered output $[(1, b), (3, a)]$ loses record c and is invalid.

Stability additionally preserves input order among equal keys, requiring a before c. The output $[(1, b), (3, c), (3, a)]$ satisfies ordering and preservation but fails stability. An empty array returns an empty array and satisfies both basic conditions.

If keys include values without ordinary total-order semantics, define a comparison policy first. Incomparable values cannot silently count as ordered. Record identities make preservation and equal-key order directly observable.

A completely provable algorithm

For an array of length $n \geq 0$, find the smallest index i with $A[i] = v$, or return NIL if none exists. Indices start at 1. Logical `and` uses left-to-right short-circuit evaluation.

```
FIRST-MATCH(A, v)
    n = A.length
    i = 1
    while i <= n and A[i] != v
        i = i + 1
    if i <= n
        return i
    return NIL
```

Before each loop test, $A[1..i-1]$ contains no occurrence of v. The empty initial prefix satisfies this invariant. Each iteration confirms $A[i] \neq v$ before advancing, preserving it. At exit, if $i \leq n$, the current value equals v and all earlier values differ, so this is the smallest matching index. Otherwise $i = n + 1$, and no array element matches. The variant $n - i + 1$ is nonnegative inside the loop and decreases by 1 per iteration. At most n iterations occur, proving termination.

Success, failure, and empty input

For $A = [4, 9, 4]$, searching for 4 starts with $i = 1$. The bound $1 \leq 3$ holds and $A[1] = 4$, so the loop stops and returns 1. The second occurrence at index 3 does not change the smallest index.

Searching the same array for 7 gives the complete trace below.

Index before test	$i \leq n$	Key comparison	Next state
1	true	$4 \neq 7$	$i = 2$
2	true	$9 \neq 7$	$i = 3$
3	true	$4 \neq 7$	$i = 4$
4	false	do not read $A[4]$	return NIL

For $n = 0$, the first bound test is false and $A[1]$ is never read. Bounds and short-circuit semantics are part of implementable pseudocode.

Similar problems can have different legal answers

Same input, different output contracts

Same complete undirected graph. Crossing diagonals do not add a vertex. Only a tour must visit all vertices and return.

In this complete undirected graph, the four perimeter edges have weight 1 and both diagonals have weight 2. A shortest path from 0 to 2 costs 2, using either $0 \to 2$ or $0 \to 1 \to 2$; visiting 3 is unnecessary. TSP optimization requires visiting every vertex exactly once and returning to the start. The minimum tour $0 \to 1 \to 2 \to 3 \to 0$ costs 4.

Similar problems can have different legal answers (continued)

Fixing start 0 and identifying a tour with its reversal leaves three tours. Their sequences and costs are 01230 with cost 4, 01320 with cost 6, and 02130 with cost 6. Exhaustive enumeration proves optimality for **this four-vertex instance**. It is not a polynomial-time algorithm for general TSP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Representation determines feasibility and search space

LCS asks for a subsequence that preserves order but need not be contiguous. For $X = ABA$ and $Y = BAA$, both AA and BA are common subsequences of length 2. A length-3 subsequence would have to equal the whole string, but the strings differ, so the optimum is 2. A common substring additionally requires contiguous positions and defines a different problem.

The eight position subsets of X produce $\varepsilon, A, B, A, AB, AA, BA, ABA$. Repeated characters let different subsets produce the same string. Thus 2^n counts position choices, not necessarily distinct strings.

The dependency edges $a \to c, b \to c, c \to d$ require both a, b before c, and c before d. Orders a, b, c, d and b, a, c, d are valid; a, c, b, d is not. Adding $d \to b$ forces $b < c < d < b$, so no topological order exists. A graph representation makes such infeasibility explicit.

Quantifiers in hardness and approximation

A decision problem asks yes or no; an optimization problem asks for the best value or solution. TSP decision asks whether some tour costs at most B; optimization finds a minimum-cost tour. NP-complete usually describes the decision version; the optimization version is NP-hard. State which version is meant.

The reduction property of NP-completeness says that a polynomial-time algorithm for one NP-complete decision problem would yield polynomial-time algorithms for all NP problems. The label alone does not prove that a polynomial-time algorithm is impossible; that conclusion requires the appropriate complexity separation premise.

An approximation guarantee needs a feasible output and an explicit objective bound, such as $\mathrm{cost}(output) \leq \rho\, OPT$ for minimization. A plausible-looking answer does not establish this bound. Heuristics, exact exponential algorithms, restricted input classes, and approximation algorithms address different requirements, with distinct time and correctness assumptions.

State the source of randomness

Fix a legal input x and let r be the internal random bits. A Monte Carlo guarantee may state bounded running time and $\Pr_r[A(x;r) \text{ is correct}] \geq 2/3$. This is not a claim that random testing passes on two thirds of inputs. A guarantee for every x cannot be replaced by an unspecified average over inputs.

Three independent runs of the same binary decision followed by majority vote have error probability at most

$$\binom{3}{2}(1/3)^2(2/3) + (1/3)^3 = 7/27,$$

so success is at least $20/27$. Independence is essential. Sharing a random choice may make all three runs fail together. Search problems can have several distinct correct outputs, so literal string majority does not automatically apply.

Zero-error randomized algorithms need a separate termination and expected-time specification. Finite expected running time implies almost-sure termination, but need not give a uniform finite bound for every infinite random-bit sequence. These quantifiers differ from deterministic total correctness.

Data structures and parallel costs

An arbitrary array lookup may scan n elements. A sorted array permits binary search, but insertion may move $\Theta(n)$ elements. Hash-table expectations depend on the hash and load model. No representation unconditionally makes every operation constant-time.

Consider four independent unit tasks followed, after all four finish, by a chain of four unit tasks. Work is $W = 8$; span is $S = 5$. Completion times on 1, 2, and 4 processors are 8, 6, and 5. For every P,

$$T_P \geq \max(W/P, S), \qquad W/T_P \leq W/S = 1.6.$$

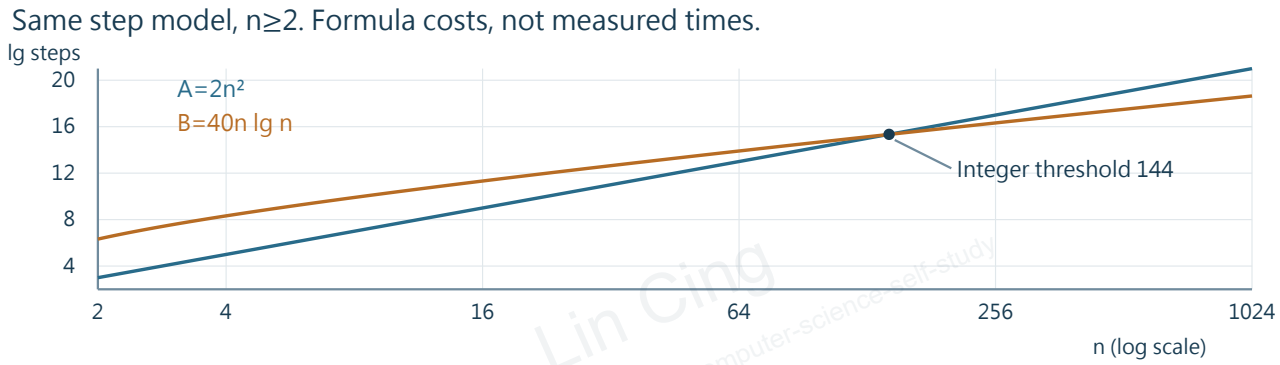
The ratio W/S bounds speedup. It does not mean that processors beyond 1.6 cannot help. Here $P = 2 > 1.6$, yet increasing to 4 reduces time from 6 to 5. Achievable times also depend on the DAG and the integer processor count.

1.2 Algorithms as a technology

Algorithms and hardware jointly determine time. If the operation count is $I(n)$ and the effective processing rate is R, a simplified model gives $T(n) = I(n)/R$. Comparisons must use consistent input sizes, operation definitions, rates, and units. Memory, I/O, and synchronization can invalidate a single-rate model.

Let $I_A(n) = 2n^2$ and $I_B(n) = 40n \lg n$, for $n \geq 2$, using identical abstract steps and the same R. Algorithm A has a smaller constant and quadratic growth; B has $n \log n$ growth. These are explicit models, not fixed exact instruction counts for every insertion-sort or merge-sort implementation.

Check both sides of a crossover



The inequality $B < A$ is equivalent to $g(n) = n - 20 \lg n > 0$. Since $g'(x) = 1 - 20/(x \ln 2)$, the function is strictly increasing after $20/\ln 2$. We have $g(143) < 0$ and $g(144) > 0$, so B wins for every integer $n \geq 144$. The initial decrease followed by increase rules out a missed earlier interval for $n \geq 2$.

.

Check both sides of a crossover (continued)

n	A steps	B steps	Fewer steps
16	512	2560	A
128	32768	35840	A
256	131072	81920	B
1024	2097152	409600	B

The figure uses logarithmic axes for this same model. A continuous intersection need not be the integer threshold. Strict inequality also requires checking possible equality.

Convert hardware rates to the same time unit

Now give A rate $R_A = 10^9$ steps/s and B rate $R_B = 10^7$ steps/s. A has 100 times the raw throughput, while the operation formulas remain unchanged. At $n = 10^6$, A uses 2×10^{12} steps and takes 2000 s. B uses about 7.9726274×10^8 steps and takes 79.7263 s. Despite the slower processor, B finishes about 25.086 times as fast in this model.

At $n = 1000$, A takes .002 s and B about .0398631 s, so A wins. The time crossover now solves $n - 2000 \lg n > 0$; the equal-rate threshold 144 no longer applies.

Big-O denotes an upper-bound class and cannot directly compare implementations with unknown constants. The limit $n/\log n \to \infty$ supports eventual dominance of the lower growth rate when constants are fixed.

Input length and available memory

A positive integer N has binary length $L = \lfloor \lg N \rfloor + 1$. A loop with N iterations is $\Theta(N)$ in the numerical value but can be $\Theta(2^L)$ in input bits. It is not generally linear in encoded input size. An array of n keys of w bits occupies $\Theta(nw)$ bits. Whether a comparison costs $O(1)$ depends on the word and cost model.

If memory holds only M records, an input with $n > M$ requires a different model. External I/O and buffering may dominate. Parallelism, caches, and networks are not automatically negligible constants; choose the input scale and cost boundary first.

Algorithms appear in applications, compilers, runtimes, operating systems, networks, and hardware. The transferable ability is to formulate contracts, choose representations, and prove correctness and cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Getting started

Part I Foundations

Scope and prerequisites

Stable insertion and merging with explicit boundaries establish loop and induction proofs. Track the hole, record preservation, inversions, and RAM costs, then derive merge-sort time and peak space for arbitrary lengths. Prerequisites are arrays, loop invariants, elementary recursion, and logarithms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- 2.1 Insertion sort. Prove correctness with loop invariants.
- 2.2 Analyzing algorithms. Derive worst-case bounds from line counts in the RAM model.
- 2.3 Designing algorithms. Formulate and solve the merge-sort recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 Insertion sort

Records have comparable keys and immutable identities. Keys form a total order; one comparison and one record move each cost a constant. Output keys must be nondecreasing, with each original record occurring exactly once. Stability additionally preserves equal-key order. Indices start at 1; empty intervals are never read.

An incremental algorithm maintains a sorted prefix and inserts the next record. In-place means constant extra words; it does not mean that every intermediate array is sorted or has no temporary duplicate values. Logical `and` short-circuits from left to right. The variable `key` stores a record; if records are object references, sorting moves references without changing the objects. Inputs of length at most 1 return immediately, so the initialization does not assume an existing first record for empty input.

Stable in-place insertion

```
INSERTION-SORT(A)
    n = A.length
    if n <= 1: return
    j = 2
    while j <= n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
        j = j + 1
```

Shift only on strict greater-than, so equal keys never pass the new record. At $i = 0$, short-circuit evaluation prevents an out-of-bounds read. Each inner iteration decreases i, giving at most $j - 1$ iterations. Each outer iteration increases j, giving at most $n - 1$. Both loops terminate.

The outer invariant proves order and preservation

Let S be a copy of the initial input. Before each outer test, $A[1..j-1]$ is the **stable sorting** of $S[1..j-1]$, and $A[j..n] = S[j..n]$. Identities distinguish equal-key occurrences.

Initially $j = 2$, so the one-record prefix is sorted and stable, with unchanged suffix. The inner loop shifts exactly the suffix whose keys are strictly greater than key. Inserting key to its left preserves the old prefix's order and leaves every equal key before the new record. The extended prefix is the stable sorting of $S[1..j]$; the suffix is unchanged. At $j = n + 1$, the invariant gives the required stable sorting of the entire input. The early return separately handles $n \leq 1$. The invariant establishes the result; the iteration bounds establish that it is reached.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Preservation means array minus hole plus key

Fix an insertion j, and let B be the starting $A[1..j]$, whose first $j-1$ records are sorted. Treat $h = i + 1$ as a logical hole. Its old memory value is excluded from the representation.

Before each inner test,

$$A[1..i] = B[1..i], \qquad A[i+2..j] = B[i+1..j-1].$$

Every key in the right interval exceeds the saved $key = B[j]$. The nonhole positions together with key preserve exactly all records of B. Initially $i = j - 1$, and the right interval is empty.

If $A[i] > key$, copying $A[i]$ into the hole and decreasing i moves the hole left and preserves both equalities. At $i = 0$ or $A[i] \leq key$, all left keys are at most key, and all right keys exceed it. Filling the hole gives a sorted permutation. Claiming that the physical array remains a permutation after every shift is false; the logical hole excludes the duplicate created by copying.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete insertion trace

Input is [(7,a),(2,b),(5,c),(2,d),(1,e)], abbreviated 7a and so on. Identity is not a secondary sorting key.

j, saved record	Key comparisons from right to left	Shifts h_j	Array after filling hole
2, 2b	$7 > 2$ true; boundary	1	2b,7a,5c,2d,1e
3, 5c	$7 > 5$ true; $2 > 5$ false	1	2b,5c,7a,2d,1e
4, 2d	$7 > 2, 5 > 2$ true; $2 > 2$ false	2	2b,2d,5c,7a,1e
5, 1e	$7 > 1, 5 > 1, 2d > 1, 2b > 1$ true; boundary	4	1e,2b,2d,5c,7a

Record 2b always precedes 2d. There are 8 shifts, 10 key comparisons, and 12 complete inner-guard evaluations. Two guard evaluations stop at $i = 0$ without comparing keys. These counts are distinct.

Every state within one insertion

For $j = 4$, let $B = [2b, 5c, 7a, 2d]$ and save 2d. The square denotes a logical hole, not an extra array write.

i before test	Nonhole representation	Action
3	[2b,5c,7a,□] + 2d	$7 > 2$, write 7a to $A[4]$
2	[2b,5c,□,7a] + 2d	$5 > 2$, write 5c to $A[3]$
1	[2b,□,5c,7a] + 2d	$2 > 2$ false; stop
1	[2b,2d,5c,7a]	fill $A[2]$ with saved record

Every state within one insertion (continued)

One insertion, j=4: save key, shift, then fill the hole

i=3: move 7a into the right hole	2b	5c	7a	□	saved key=2d
i=2: move 5c into the right hole	2b	5c	□	7a	saved key=2d
i=1: 2b≤2d, stop	2b	□	5c	7a	saved key=2d
Fill with 2d; 2b stays first	2b	2d	5c	7a	Stable; all records preserved

□ marks an excluded logical slot. Its old memory value is not a second logical record.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Parameters, aliases, and control semantics

Arguments are passed by value. An array value is a reference to storage, so assigning $A[i]$ changes storage visible to the caller. `B=A` copies the reference and aliases the same storage; `B=copy(A)` creates an independent array. Rebinding local `A` to another array does not rebind the caller's variable.

For $A = [3, 1]$, after `B=A`, assigning $B[1] = 8$ also makes $A = [8, 1]$. With `B=copy(A)`, that assignment leaves A unchanged. Record identity differs from a memory address; sorting may rearrange references without reconstructing objects.

Indentation defines blocks; `return` ends the current call. NIL means no object, not a legal array position. The final failed while guard also costs time. A description such as “sort this set” cannot count as one constant step if it hides a variable-length computation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.2 Analyzing algorithms

A single-processor word-RAM executes loads, stores, comparisons, branches, and bounded-word arithmetic sequentially. Array access costs a constant. A word must represent indices, typically $w = \Theta(\lg(n+1))$ bits. Arbitrarily long integers cannot be placed in one word with every operation declared constant-time.

The model does not directly charge cache misses, page faults, external I/O, or parallel work. Implementations and hardware determine physical time. Comparing long strings can require reading many characters, changing the constant-comparison assumption. Arithmetic exceeding one word needs an explicit multiword representation and cost.

Input size depends on the problem. Sorting uses record count n, an integer problem may use bit length L, and graph problems commonly use both V, E. Numerical value N and encoded length $\lg N$ are not interchangeable. Counts below measure primitive operations, not seconds.

Count each line

For $n \geq 2$, let h_j be insertion j's shift count, $t_j = h_j + 1$ its inner-guard count, and $I = \sum h_j$. Initialization of $j = 2$ occurs once; the outer guard executes n times.

Operation	Count
Set n; test early return; initialize j	1 each
Outer guard	n
Save key; initialize i; fill hole; increment j	$n - 1$ each
Inner guard	$I + n - 1$
Shift; decrement i	I each

With fixed positive operation costs independent of n,

$$T(A) = c_0 + c_1 n + c_2(n-1) + c_3(I + n - 1) + c_4 I = \Theta(n + I).$$

Count each line (continued)

Here c_2 combines four operations and c_4 combines two. Including empty and singleton inputs gives $\Theta(1 + n + I)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shifts count inversions exactly

An inversion of the original input S is a pair (u, v) with $u < v$ and $S[u].key > S[v].key$. Equal keys do not form inversions.

Before insertion j, the sorted prefix preserves the first $j - 1$ records. Those with keys greater than $S[j]$ form precisely one suffix. Each shifts once; no other record shifts. Therefore h_j counts exactly inversions with right endpoint j. Every inversion has one right endpoint, giving $\sum h_j = I(S)$ for all inputs.

The example has counts 1,1,2,4 and $I = 8$. Sorted input has $I = 0$ and time $\Theta(n)$. Distinct reverse-sorted keys have $I = n(n - 1)/2$ and time $\Theta(n^2)$. Few inversions yield nearly linear time, an adaptive property.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparisons, writes, best case, and worst case

Insertion j performs $h_j + 1$ key comparisons when $h_j < j - 1$, including a final false comparison; otherwise it performs h_j, stopping by short-circuit at zero. If b insertions reach the far left,

$$C_{key} = I + n - 1 - b, \quad C_{guard} = I + n - 1, \quad W_{array} = I + n - 1.$$

Saving `key` is not an array write. Sorted input has $h_j = b = 0$ and $C_{key} = n - 1$. Distinct reverse order has $h_j = j - 1$, $b = n - 1$, and $C_{key} = n(n-1)/2$. Equal lengths can have different costs.

Define $T_{best}(n) = \min_{|x|=n} T(x)$ and $T_{worst}(n) = \max_{|x|=n} T(x)$. Best and worst specify a function; O, Ω, and Θ specify bounds on that function. O does not mean worst case, nor Ω best case.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Average costs require a distribution

Assume all $n!$ permutations of **distinct** keys are equally likely. For $u < v$, let X_{uv} indicate $S[u] > S[v]$. Swapping the two positions is a bijection between either relative order, so $\Pr(X_{uv} = 1) = 1/2$. Linearity gives

$$I = \sum_{u<v} X_{uv}, \qquad E[I] = n(n-1)/4.$$

The indicators need not be independent; shared positions generally create dependence. Since $T(A) = \Theta(1 + n + I)$ with uniform constants across inputs, $E[T] = \Theta(n^2)$.

This is not a claim about every real dataset. A distribution concentrated on sorted inputs has expectation $\Theta(n)$. Random-input expectations and expectations over an algorithm's internal randomness on fixed input use different probability spaces.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

From exact costs to growth

With fixed RAM costs, worst-case time has form $an^2 + bn + c$, with $a > 0$. For sufficiently large n, $|b|n + |c| \leq (a/2)n^2$, yielding

$$(a/2)n^2 \leq T_{worst}(n) \leq (3a/2)n^2.$$

This proves the Θ bound. Constants and lower-order terms still affect finite inputs. In the same units, $1000n$ exceeds n^2 for $1 < n < 1000$, but is smaller for $n > 1000$. Ignoring finitely many sizes asymptotically does not excuse incorrect empty or singleton behavior. A constant-time early-return branch does not make every other input cheaper.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Designing algorithms

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \leq p \leq r \leq n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \geq 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MERGE without an extra key value

Assume $1 \le p \le q < r \le n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.

Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing k preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all m records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R.

i,j before test	Comparison	Output prefix
1,1	$1a \leq 2d$	1a
2,1	$4b > 2d$	1a,2d
2,2	$4b \leq 4e$	1a,2d,4b
3,2	$4c \leq 4e$	1a,2d,4b,4c
4,2	L exhausted; no key comparison	append 4e,6f

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. Θ(m) work does not mean exactly m key comparisons.

Complete merge with duplicate keys (continued)

Consume one, emit one; output writes cannot corrupt copied inputs

	L			R			
Start	1a	4b	4c	2d	4e	6f	Output: empty
After 1a,2d	—	4b	4c	—	4e	6f	Output: 1a,2d
Take left on ties	—	—	—	—	4e	6f	Output: 1a,2d,4b,4c

L exhausted: append 4e,6f. Equal-key records 4b,4c,4e retain their original order.

Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting

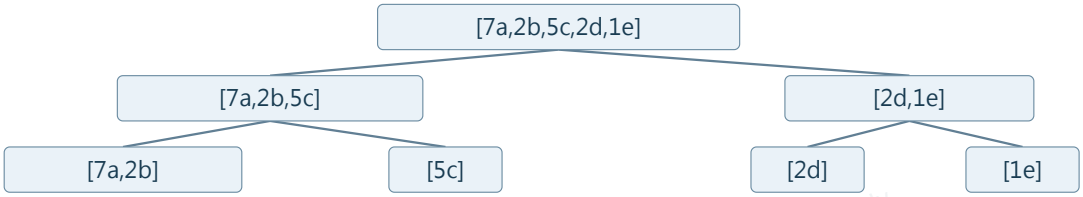
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting (continued)

Five records: split downward, return stable sorted results



Bottom up: [2b,7a] → [2b,5c,7a]; right half → [1e,2d]

Root merge: [1e,2b,2d,5c,7a]; key-2 tie: left 2b before right 2d

Child lengths 3 and 2; singletons return. The final split of [7a,2b] into singletons is omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Include merging in the recurrence

Midpoint computation costs Θ(1). Copying and outputting m records costs Θ(m), giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has 2^ℓ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Arbitrary lengths and a six-record cost table

At depth ℓ, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

Depth for m=6	Sizes	Total internal size
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1; two earlier leaves	0

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving Θ(n) peak auxiliary record space plus Θ(log n) stack space, hence Θ(n) overall for $n \geq 2$.

Cumulative allocation may be Θ(n log n), but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees Θ(n log n) time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Growth of functions

Part I Foundations

Scope and prerequisites

Use full quantifiers, constants, and counterexamples to establish five asymptotic relations. Distinguish input costs from function bounds, then derive conditions for floor and mod, fixed-base growth comparisons, factorials, log-star, and Fibonacci numbers. Prerequisites are real numbers, logarithms, polynomials, and elementary inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- 3.1 Asymptotic notation. Define O, Ω, and Θ with precise quantifiers and prove polynomial leading-term bounds.
- 3.2 Standard notations and common functions. Distinguish logarithm bases, exponents, factorials, and iteration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Asymptotic notation

Choose a function and limit direction first. Here n is a natural number tending to infinity, f is eventually nonnegative, and the comparison function g is eventually positive. If g can vanish, use inequality definitions rather than division. A threshold can exclude finitely many small sizes from an asymptotic statement, but legal small program inputs still need correct handling.

O(g), Ω(g), and Θ(g) are **sets of functions**. The convention f=O(g) means membership, not numerical equality. The function may measure time, space, comparisons, or a mathematical quantity; the notation does not specify best, worst, or average case.

Insertion sort has worst-case $\Theta(n^2)$ and best-case $\Theta(n)$. A common $O(n^2)$ bound on all inputs does not give a common $\Omega(n^2)$ bound. Specify both the function and the input quantifiers.

Quantifiers and constants

Relation	Definition, holding for every integer $n \geq n_0$
$f \in O(g)$	some $c > 0, n_0$ give $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	some $c > 0, n_0$ give $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	some $c_1, c_2 > 0, n_0$ give $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

Constants may depend on the functions and a fixed algorithm, but not on n. Choosing c=n does not prove n^2=O(n). A threshold need not be smallest; any larger threshold works. Tight bounds require both sides, not a ratio limit or equal physical running times. Θ(g) is a subset of both O(g) and Ω(g). The true statement n=$O(n^2)$ is loose; confusing O with Θ loses this distinction.

A two-sided proof with explicit constants

For $f(n) = 3n^2 - 8n + 5$, at $n \geq 4$,

$$f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1.$$

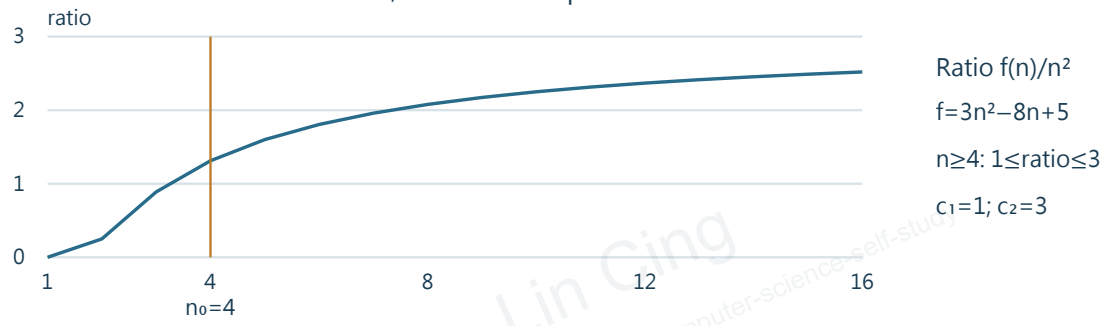
For $n \geq 1$, $-8n + 5 \leq 0$, so $f(n) \leq 3n^2$. Thus $c_1 = 1, c_2 = 3, n_0 = 4$ prove Θ(n²). At n=4, $16 \leq 21 \leq 48$; at n=8, $64 \leq 133 \leq 192$. The inequalities prove the entire tail; a finite table alone does not.

If f=O(n), some fixed c would give f≤cn eventually. Combining with f≥n² gives n≤c. An integer $n > \max(n_0, 4, c)$ contradicts this. A disproof must defeat **every candidate constant**, rather than one guessed constant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-sided proof with explicit constants (continued)

Bounds must cover the entire tail, not selected points



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Θ is the intersection of O and Ω

A Θ bound immediately supplies its upper constant for O and lower constant for Ω. Conversely, from $f \le c_U g$ after N_U and $f \ge c_L g$ after N_L, choose $n_0 = \max(N_U, N_L)$ to obtain both bounds simultaneously.

For eventually nonnegative f,g,

$$\max(f,g) \le f + g \le 2\max(f,g),$$

so $f + g = \Theta(\max(f,g))$. Nonnegativity matters; f=n and g=−n cancel to zero.

For $p(n) = a_d n^d + \sum_{i<d} a_i n^i$, with $a_d > 0, d \ge 1$, put $M = \sum_{i<d} |a_i|$. At $n \ge \max(1, 2M/a_d)$, the lower-order absolute sum is at most $Mn^{d-1} \le a_d n^d/2$, giving

$$(a_d/2)n^d \le p(n) \le (3a_d/2)n^d.$$

For d=0, a positive constant is Θ(1).

Little-o and little-ω quantify over every constant

Membership $f \in o(g)$ means that **for every** c>0, a threshold $n_0(c)$ makes $0 \le f(n) < cg(n)$ for all later n. Membership $f \in \omega(g)$ means that every c>0 eventually gives $0 \le cg(n) < f(n)$. The threshold may depend on c; the chosen c may not depend on n.

For $7n = o(n^2)$, given c choose an integer threshold greater than 7/c. In contrast, $7n^2$ is not $o(n^2)$, since c=7 would require strict self-inequality. Also $n^2 = \omega(7n)$, using n>7c. With positive denominators, little-o is equivalent to ratio limit zero, and little-ω to ratio limit infinity.

Although o(g) is contained in O(g), a loose O bound need not be little-o. A function equal to n on even n and n^2 on odd n belongs to $O(n^2)$, but neither $\Theta(n^2)$ nor $o(n^2)$: its ratio repeatedly returns to 1.

What limits establish

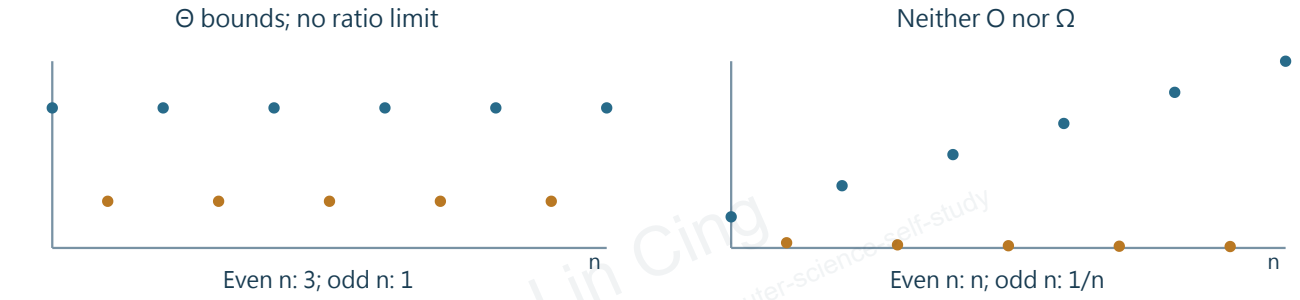
For eventually positive g, if $L = \lim f/g$ exists, L=0 gives o(g), finite positive L gives Θ(g), and L=∞ with eventually positive f gives ω(g). For finite positive L, epsilon=L/2 gives $L/2 < f/g < 3L/2$ eventually, proving both bounds.

A nonexistent ratio limit need not mean incomparability. For $f = n(2 + (-1)^n), g = n$, the ratio alternates 1 and 3, yet $n \leq f \leq 3n$, giving Θ(n). A genuinely incomparable example has f=n² on even n and f=1 on odd n, with g=n. The even ratios n preclude O(g); the odd ratios 1/n preclude Ω(g). Each subsequence defeats every fixed candidate constant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

What limits establish (continued)

Plots of f/g: bounded between 1 and 3 on the left; separated subsequences on the right



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algebraic properties

For eventually positive functions, O, Ω, and Θ are reflexive and transitive; Θ is symmetric. We have f∈O(g) exactly when g∈Ω(f), and f∈o(g) exactly when g∈ω(f). For example, f≤ag and g≤bh after their common threshold give f≤abh, proving O transitivity. Reverse inequalities prove Ω transitivity; applying both proves Θ transitivity.

For little-o transitivity, given c>0 use the first little-o bound with constant 1 and the second with c. Beyond the common threshold, f<g<ch. Inverting little-o relations proves little-ω transitivity. Neither strict relation is reflexive, and their intersection is empty: taking c=1 in both would require f<g and g<f eventually.

These relations are not a total order. The parity example is neither O nor Ω relative to g, so arbitrary functions need not fall into smaller, equal-order, or larger categories.

Anonymous functions and one-way equalities

The expression $4n^2 + 6n + 2 = 4n^2 + \Theta(n)$ selects the particular member h=6n+2. The expression $4n^2 + \Theta(n) = \Theta(n^2)$ means that **every** left-hand choice of h yields a right-hand member. It does not assert equality of the sets or justify reversal.

Two O terms may select different anonymous functions. O(n)−O(n) does not guarantee Θ(n), since n−n=0. For signed errors, explicitly write $|e(n)| \leq cn$; this chapter's nonnegative O class does not automatically include negative functions.

If a single h satisfies h(i)=O(i), its constants work uniformly for all sufficiently large i. The finite prefix contributes a constant, so $\sum_{i=1}^{n} h(i) = O(n^2)$. Choosing a different constant depending on n for each term is invalid. The family $h_n(i) = ni$ can instead sum to Θ(n^3).

Uniform bounds over inputs

Let $T(x) \geq 0$, $T_{max}(n) = \max_{|x|=n} T(x)$, and $T_{min}(n) = \min_{|x|=n} T(x)$, assuming nonempty input sets and attained extrema. A common Θ(g) bound means fixed c_L, c_U, n_0 such that **every** input x of every size n≥n₀ satisfies $c_L g(n) \leq T(x) \leq c_U g(n)$.

This is equivalent to $T_{max} \in O(g)$ and $T_{min} \in \Omega(g)$. Common bounds give the extrema bounds; conversely use $T_{min} \leq T(x) \leq T_{max}$ after a common threshold. A worst-case Θ(g) bound alone is insufficient because its lower bound applies only to some slow inputs.

For two parameters n,m, define the tail domain. Here it means both n≥n₀ **and** m≥m₀. A stronger domain where either is sufficiently large must explicitly use “or”; these quantifiers are not interchangeable.

3.2 Standard notations and common functions

Nondecreasing means $m \leq n \Rightarrow f(m) \leq f(n)$; strictly increasing requires a strict conclusion for m<n. Decreasing notions reverse the inequalities. Nondecreasing does not imply strictly increasing.

For nondecreasing f,g, their sum is nondecreasing. Composition is too when g's range lies in f's domain. If both are nonnegative, $f(m)g(m) \leq f(n)g(m) \leq f(n)g(n)$, so their product is nondecreasing. Without nonnegativity the product claim can fail.

Floor is the greatest integer at most x; ceiling is the least integer at least x. Thus $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$. For −2.3, floor is −3 and ceiling −2. Truncation toward zero is not general floor.

Integer splits, nested rounding, and mod

For integer n, $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$: even n=2k gives k+k; odd n=2k+1 gives (k+1)+k. For positive integers a,b and x≥0,

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor .$$

For integer k, $kb \le \lfloor x/a \rfloor$ is equivalent to $kab \le x$, so both sides choose the same greatest k. The ceiling identity similarly follows by choosing the least integer with reversed inequalities.

For positive integers a,b, $\lceil a/b \rceil \le (a+b-1)/b$ and $\lfloor a/b \rfloor \ge (a-b+1)/b$. Write a=qb+r, 0≤r<b, and separate r=0 from r>0.

For integer u and positive modulus m,

$$u \bmod m = u - m\lfloor u/m \rfloor \in \{0, \ldots, m-1\}.$$

Thus −17 mod 5=−17−5(−4)=3, while 17 mod 5=2. A language remainder following the dividend's sign may differ. Congruence u≡v modulo m means m divides u−v.

Powers, exponentials, and fixed constants

For a>0 and real u,v, $a^{u+v} = a^u a^v$, $(a^u)^v = a^{uv}$, $a^0 = 1$, and $a^{-u} = 1/a^u$. A **fixed** a>1 gives increasing a^n, while 0<a<1 gives decreasing a^n. For n>0, n^b increases when b>0 and decreases when b<0.

Polynomially bounded means $O(n^k)$ for some fixed k, not necessarily a polynomial expression. The function $n^{\lg n}$ is not polynomially bounded, since for every fixed k its ratio to n^k is $n^{\lg n - k} \to \infty$. Conversely $2^{\lg n} = n$ is polynomial, while 2^n is exponential. A visible base 2 alone does not classify growth.

When a combinatorial formula needs 0^0, this chapter uses the empty-product value 1, not a universal value for all limits. Real-base identities cannot be applied unconditionally to negative bases.

Fixed-base exponentials dominate polynomials

Let a=1+δ>1 and b be fixed. For b≤0, $n^b/a^n \le 1/a^n \to 0$. For b>0, choose an integer k>b. The binomial expansion gives $a^n \ge \binom{n}{k}\delta^k$. At n≥2k, every numerator factor of the binomial coefficient is at least n/2, so

$$a^n \ge \frac{\delta^k n^k}{2^k k!}, \qquad 0 \le \frac{n^b}{a^n} \le \frac{2^k k!}{\delta^k} n^{b-k} \to 0.$$

Thus $n^b=o(a^n)$. The choice k depends on b, not on n.

For fixed ε>0 and fixed b, put t=lg n. Then $(\lg n)^b/n^\varepsilon = t^b/(2^\varepsilon)^t \to 0$. Every fixed polylogarithm is smaller than every positive power. A base approaching 1 with n violates the fixed-base premise; for example $(1+1/n)^n \to e$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Logarithm bases, powers, and composition

Use $\lg n = \log_2 n$, $\ln n = \log_e n$, $\lg^k n = (\lg n)^k$, $\lg\lg n = \lg(\lg n)$, and $\lg^{(k)} n$ for k iterations. A logarithm applies to its adjacent argument; lg n+q is not lg(n+q). Real log arguments must be positive, and bases positive and different from 1.

Taking log base c of $b^{\log_b x} = x$ gives $\log_b x = \log_c x / \log_c b$. Fixed b,c>1 differ by a positive constant, giving equal growth orders. Variable b=n gives $\log_n n = 1$, unlike lg n.

Products and powers obey $\log_b(xy) = \log_b x + \log_b y$ and $\log_b(x^r) = r \log_b x$. Also $a^{\log_b c} = c^{\log_b a}$, since both natural logarithms equal $(\ln a \ln c)/\ln b$. For example $3^{\lg 8} = 27 = 8^{\lg 3}$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bounds and errors for e and ln

The series is $e^x = \sum_{k\ge 0} x^k/k!$. Convexity gives $e^x \ge 1 + x$ for every real x; the tangent at zero has slope 1. Taylor's remainder gives $e^x = 1 + x + (e^\xi/2)x^2$, with ξ between 0 and x. For $|x| \le 1$,

$$e^{-1}x^2/2 \le e^x - 1 - x \le ex^2/2,$$

proving a $\Theta(x^2)$ remainder as x tends to zero.

For $x > -1$, $x/(1+x) \le \ln(1+x) \le x$. For the upper bound, $h = x - \ln(1+x)$ has $h(0) = 0$ and derivative $x/(1+x)$, giving $h \ge 0$ on either side of zero. For the lower bound, $q = \ln(1+x) - x/(1+x)$ has derivative $x/(1+x)^2$ and the same argument applies. For $|x| < 1$,

$$\ln(1+x) = \sum_{k\ge 1} (-1)^{k+1} x^k/k.$$

Fix x and put $u = x/n$. Once $1 + u > 0$, the logarithmic bounds squeeze $n \ln(1 + x/n) \to x$. Exponentiation yields $(1 + x/n)^n \to e^x$. Domains and errors distinguish these limits from identities.

Factorial bounds from products

For integer $n \ge 0$, $0! = 1$ and $n! = n(n-1)!$. Every factor is at most n, so $n! \le n^n$. The first $\lfloor n/2 \rfloor$ factors are at most $n/2$, giving

$$n!/n^n \le 2^{-\lfloor n/2 \rfloor} \to 0,$$

and hence $n! = o(n^n)$. At least $\lfloor n/2 \rfloor$ final factors are at least $n/2$, so

$$n! \ge (n/2)^{\lfloor n/2 \rfloor}.$$

Taking logarithms bounds $\lg(n!)$ between $\lfloor n/2 \rfloor \lg(n/2)$ and $n \lg n$, proving $\Theta(n \log n)$. Subtracting $n \lg a$ from the lower bound tends to infinity for every fixed $a > 0$. Thus $n!/a^n \to \infty$, including $n! = \omega(2^n)$.

These proofs do not assume Stirling. Numerically, $6! = 720$, $2^6 = 64$, and $6^6 = 46656$; finite examples do not replace the general limits.

Stirling precision and constants in exponents

The standard Stirling estimate states, for n≥1,

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n}.$$

Thus the relative factor is $1 + \Theta(1/n)$, and

$$\lg(n!) = n\lg n - (\lg e)n + \tfrac{1}{2}\lg(2\pi n) + \Theta(1/n).$$

At n=6 the main term $\sqrt{12\pi}(6/e)^6$ is about 710.078; multiplying by factors between e^(1/73) and e^(1/72) brackets 720.

Fixed log bases differ by constants, but exponential bases cannot be ignored. We have $2^{n+3} = 8 \cdot 2^n = \Theta(2^n)$, while $2^{3n}/2^n = 4^n \to \infty$. If f=Θ(n), the safe conclusion is $2^f = 2^{\Theta(n)}$, not necessarily Θ(2^n).

Functional iteration and log-star stopping

Define $f^{(0)}(n) = n$ and $f^{(k+1)}(n) = f(f^{(k)}(n))$. For f(n)=3n, iteration gives $3^k n$, not $(3n)^k$.

For n>0,

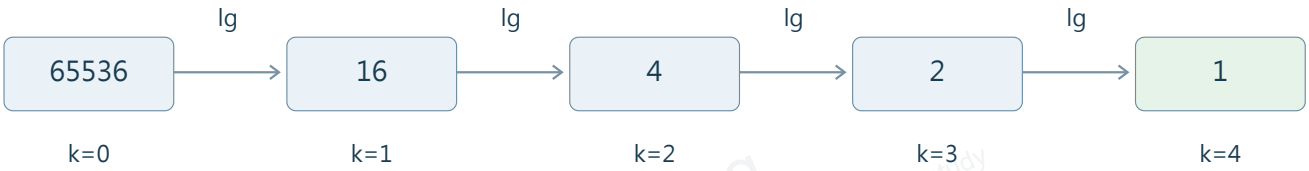
$$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

Take another logarithm only while the current value exceeds 1, avoiding nonpositive arguments. For n≤1, the answer is 0. The chain 65536→16→4→2→1 gives log-star 4. Taking two logarithms gives 4, whereas $(\lg 65536)^2$=256.

Set tower thresholds a_0=1 and $a_{k+1} = 2^{a_k}$. For $n \in (a_{k-1}, a_k]$, log-star equals k. Thresholds for k=0 through 4 are 1,2,4,16,65536. Growth is slow but unbounded: for every finite k, some n exceeds a_k.

Functional iteration and log-star stopping (continued)

Iterate lg once per arrow; stop at 1



Continue only above 1. Iteration 4 reaches 1, so lg*65536=4.

Two lg iterations give 4; (lg65536)²=256. Iteration differs from a power.

Proving the Fibonacci closed form

Let F₀=0, F₁=1, and $F_n = F_{n-1} + F_{n-2}$. Both $\varphi = (1+\sqrt{5})/2$ and $\psi = (1-\sqrt{5})/2$ satisfy z²=z+1. Define $G_n = (\varphi^n - \psi^n)/\sqrt{5}$. Its first values are 0 and 1. Both root-power sequences satisfy the same recurrence, so G does too. Induction proves F=G.

Since |ψ|<1,

$$|F_n - \varphi^n/\sqrt{5}| = |\psi|^n/\sqrt{5} < 1/2,$$

including n=0. Therefore $F_n = \lfloor \varphi^n/\sqrt{5} + 1/2 \rfloor$. At n=6, the terms are approximately 8.024922 and .024922, whose difference is 8.

For sufficiently large n, the error bound gives $\varphi^n/(2\sqrt{5}) \le F_n \le 2\varphi^n/\sqrt{5}$, proving Θ(ϕⁿ). A constant-size real expression is not a constant-time exact floating-point algorithm for arbitrary n. The output itself has Θ(n) bits.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probabilistic analysis and randomized algorithms

Part I Foundations

Scope and prerequisites

Fix the sample space and dependence assumptions first. Hiring indicators and a general uniform-shuffle proof lead to birthday collisions, bin waiting times, overlapping streaks, and online selection. Random recurrences and Quicksort distinguish expectations, event probabilities, worst cases, and tails. Prerequisites are discrete probability, expectation, random permutations, and elementary bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Distinguish input-distribution assumptions from internal algorithmic randomness.
- Use indicators and linearity without unjustified independence.
- Analyze uniform permutations and randomized expected costs.
- Derive expectations and data-structure costs from random splits and comparison indicators.

Sections 5.1–5.4 cover hiring, indicators, randomized algorithms, and further probabilistic applications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1 The hiring problem

The n candidates have distinct comparable ranks, with larger better. After each interview, replace the current employee if the candidate is better. Initialize with a dummy worse than every candidate. The quantity of interest includes replacement cost, not just comparisons.

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]; hire best; hires += 1
    return (best, hires)
```

After round i, best is the greatest of the i interviewed candidates, immediately by induction. With interview fee c_i per candidate and hiring fee c_h per replacement, cost is nc_i+Xc_h, where X counts hires. At n=0 return the dummy with X=0; at $n \geq 1$, $1 \leq X \leq n$.

Worst inputs, input distributions, and random bits

Ranks [1,2,…,n] set a record every time, so X=n. Ranks [n,n−1,…,1] give X=1. Deterministic HIRE has a fixed cost on a fixed input.

Average-case analysis needs a distribution. Initially assume all n! orders of distinct ranks are equally likely, a joint condition stronger than equal position marginals. An internally uniform shuffle converts every fixed input into this distribution. Expectations then concern internal randomness instead of an unknown input distribution.

This does not eliminate bad executions: shuffling can still produce increasing order. Randomization guarantees expected cost for each fixed input, not small worst-case cost for every random outcome.

5.2 Indicator random variables

For an event E, indicator I_E is 1 on E and 0 otherwise, so $E[I_E] = P(E)$. For $X = \sum_i X_i$, exchanging finite sums on a finite sample space gives

$$E[X] = \sum_{\omega} P(\omega) \sum_i X_i(\omega) = \sum_i \sum_{\omega} P(\omega) X_i(\omega) = \sum_i E[X_i].$$

This is linearity of expectation, requiring no independence. Independence may be needed to calculate individual probabilities or factor intersections. If X_1=X_2 indicate heads on the same fair coin, $E[X_1+X_2]$=1 still holds, but $P(X_1=X_2=1)$=1/2 rather than 1/4.

The harmonic hiring expectation

Let X_i indicate hiring candidate i. Conditional on the set of the first i candidates, their i! relative orders are uniform. Their greatest candidate occupies position i with probability 1/i, exactly the condition X_i=1. Therefore

$$E[X] = \sum_{i=1}^{n} P(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

No full distribution of X or independence of record events is needed. Integral comparison of 1/x gives $\ln(n+1) \le H_n \le 1 + \ln n$. Thus E[X]=Θ(log n), and expected fees are nc_i+$c_h H_n$; interviews do not become logarithmic.

Ranks [2,5,1,4,6,3] set records at 1,2,5, giving X=3. The uniform expectation at n=6 is H_6=49/20=2.45; one trace need not equal it. Exactly one hire occurs iff the best arrives first, with probability 1/n. Hiring everyone requires the sole increasing permutation, with probability 1/n!.

Indicators can count pairs

In a uniform permutation of distinct keys, the inversion indicator for i<j has probability 1/2: swapping those positions bijects the two relative orders. Thus $E[V] = \binom{n}{2}/2 = n(n-1)/4$. Shared positions make indicators dependent but do not invalidate linearity.

Uniformly return n distinct hats bijectively to n owners. Each owner gets the correct hat with probability 1/n, so expected fixed points equal 1 for n≥1. This does not ensure one match in every outcome: at n=2 there are either zero or two, each with probability 1/2. At n=0 there are zero, and 1/n is undefined.

First prove that the indicator sum equals the target count, then evaluate individual probabilities. Addition of disjoint-event probabilities differs from addition of expectations of dependent variables.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

5.3 Randomized algorithms

RANDOM(a,b) returns every integer in the inclusive range with probability 1/(b−a+1), independently across calls. A finite-state pseudorandom generator approximates this probability model; uniformity and independence remain explicit assumptions.

From independent fair bits, use rejection sampling for m=b−a+1 outputs. If m=1 return a. Otherwise draw k=ceil(lg m) bits as z in $0..2^k-1$. If z<m return a+z; otherwise redraw the whole group. Acceptance probability $m/2^k$ exceeds 1/2, and accepted legal z values are equiprobable. Expected rounds are less than 2 and expected bit cost O(log m).

Using z mod m is biased unless m divides 2^k. For three bits modulo 3, residues 0 and 1 have three preimages each, but residue 2 has two.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

An in-place uniform shuffle and its joint invariant

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap A[i],A[j]
```

Give elements distinct identities even when their keys agree. Before round i, any specified ordered sequence of i−1 distinct identities is the prefix with probability (n−i+1)!/n!. The initial empty sequence has probability 1. Conditional on that prefix, each remaining identity occupies one suffix position, and RANDOM selects the specified next identity with probability 1/(n−i+1).

The specified i-prefix therefore has probability (n−i)!/n!, preserving the invariant. At termination each complete permutation has probability 1/n!, proving joint uniformity. At n=0 there is one empty permutation and no iterations. Unit-cost random selection gives Θ(n) time and O(1) extra space; a bit-based implementation needs its separate random-bit cost.

Every round of one shuffle

Start A=[A,B,C,D] and choose j=3,4,3,4.

i	Available indices	j	Array after swap	Fixed prefix
1	1..4	3	[C,B,A,D]	C
2	2..4	4	[C,D,A,B]	C,D
3	3..4	3	[C,D,A,B]	C,D,A
4	4	4	[C,D,A,B]	C,D,A,B

This path has probability (1/4)(1/3)(1/2)・1=1/24. Every target permutation has exactly one legal j sequence: find its next identity in the remaining suffix, swap, and continue. Thus all 4!=24 paths are equiprobable.

Every round of one shuffle (continued)

Start [A,B,C,D]: select only from the unfixed suffix

	Pos 1	Pos 2	Pos 3	Pos 4	Round probability
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

Green cells stay fixed. This path has probability 1/4·1/3·1/2·1=1/24.

Uniform marginals do not suffice

A uniformly chosen cyclic shift puts every identity at each position with probability 1/n, yet for n≥3 has only n possible outputs rather than n!. The joint distribution is not uniform.

Another algorithm swaps each i=1..n with j chosen from all 1..n. At n=3 it has 3^3=27 equiprobable paths. Uniformity among six permutations would require 27/6 paths per output, not an integer. Later swaps can disturb already processed positions, invalidating the fixed-prefix invariant.

Stopping the correct shuffle after m rounds gives each ordered m-sample probability (n−m)!/n!. Each unordered subset has m! orders, hence probability $1/\binom{n}{m}$. This assumes random access to the suffix; any initial construction of an n-element array must still be charged.

Random priorities and collisions

Give each identity an independent uniform priority in {1,…,M}, then sort by priority. Conditional on all priorities being distinct, label symmetry makes all n! orders uniform. Comparison sorting takes Θ(n log n) and storing priorities O(n) space.

A pair collides with probability 1/M. The union bound gives $P(\text{some collision}) \le \binom{n}{2}/M$. For $M=n^3$,$n \ge 2$, this is at most $(n-1)/(2n^2)<1/(2n)$: rare, not impossible.

Stable tie breaking leaves unconditional bias. At n=2, the first element remains first with probability (M+1)/(2M)>1/2. Redrawing the entire group after any collision restores uniformity through label symmetry of acceptance; at this M the expected number of rounds is bounded. Uniformly shuffling each tie group also works. The in-place SHUFFLE avoids the collision condition from the start.

Randomization gives a guarantee for each fixed input

RANDOMIZED-HIRE shuffles, then runs HIRE. Every fixed distinct-rank input becomes uniform, giving $E[X]=H_n$ and expected fees $nc_i+c_hH_n$. Shuffling adds Θ(n) operations, measured separately from hiring fees.

When candidates must be interviewed in external arrival order and cannot be rearranged, this shuffle is unavailable. The H_n result then still needs an arrival-distribution assumption; internal randomness cannot silently control external arrivals.

Shuffling and random-pivot sorting preserve answer correctness on every random path; only cost varies. Such algorithms are generally called Las Vegas. Monte Carlo algorithms allow errors and need separate error probabilities and quantifiers. Expected time alone supplies no correctness probability.

5.4 Probabilistic analysis and further uses of indicator random variables

Birthday models count colliding pairs or collision existence; balls and bins count loads or waiting times; streak models count windows or longest length; online selection measures the probability of choosing the best. Expectations, probabilities, and maxima are different quantities.

Three tools follow directly. A union's indicator is at most the sum of event indicators, giving the union bound. For nonnegative X, $X \geq tI(X \geq t)$, giving Markov's inequality $P(X \geq t) \leq E[X]/t$. For nonnegative integer L, $L = \sum_{j \geq 1} I(L \geq j)$, giving $E[L] = \sum_{j \geq 1} P(L \geq j)$. Finite sums can be exchanged directly; infinite geometric waiting sums use monotone limits of nonnegative partial sums.

Whenever independence is needed, identify the independent trials. Overlapping adjacent windows cannot be multiplied as if independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact birthday collision probability

Let k people draw independently and uniformly from n≥1 dates. For k≤n, conditional on the first j being distinct, person j+1 has n−j unused dates. Hence

$$P(\text{all distinct}) = \prod_{j=0}^{k-1}(1 - j/n), \qquad P(\text{collision}) = 1 - \prod_{j=0}^{k-1}(1 - j/n).$$

At k=0,1 collisions are impossible. For k>n the pigeonhole principle gives probability 1; do not extend the product into negative factors. For 2≤k≤n, 1−x≤e^(−x) bounds the no-collision probability by exp(−k(k−1)/(2n)). Thus k(k−1)≥2n ln 2 is sufficient for collision probability at least 1/2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expected pairs differ from an event probability

Let I_{ij} indicate equal dates. Independence and uniformity give $P(I_{ij}=1)=\sum_r (1/n)^2=1/n$, hence $E[X]=\binom{k}{2}/n$ colliding pairs. Independence among these indicators is unnecessary.

A collision means $X \geq 1$. Markov gives only $P(X \geq 1) \leq E[X]$. A large expectation alone gives no probability lower bound: $X=M^2$ with probability $1/M$ and zero otherwise has expectation M but a small chance of being positive.

For independent uniform dates, use both the exact complement and expectation. If $k=o(\sqrt{n})$, the union bound tends to zero. For $k \geq C\sqrt{n}$ with sufficiently large C, the exponential complement bound gives a fixed positive collision probability. These bounds establish the $\sqrt{n}$ threshold scale.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sixteen dates: exact numbers

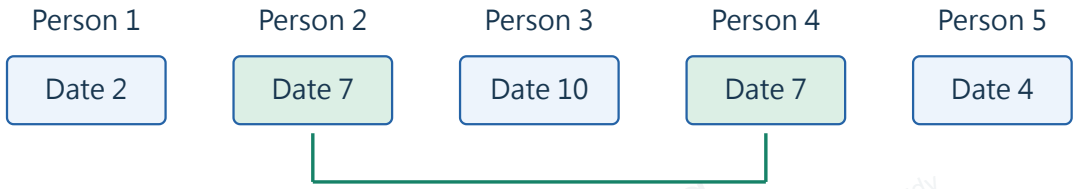
For $n=16,k=5$, no-collision probability is $15 \cdot 14 \cdot 13 \cdot 12/16^4=4095/8192$. Collision probability is $4097/8192 \approx .500122$, while expected colliding pairs equal $10/16=5/8$. They answer different questions.

At $k=5$ the exponential upper bound on the complement is $e^{(-10/16)}>1/2$, so it does not prove probability above one half. At $k=6$ it gives $e^{(-15/16)}<1/2$. A sufficient bound need not locate the exact threshold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sixteen dates: exact numbers (continued)

One date sample and its model: distinguish counts from expectations



Only colliding pair in this sample: (2,4)

Model: 16 dates, 5 people. P(collision)=4097/8192≈0.500122.

E[colliding pairs]=10/16=5/8, distinct from the event probability above.

Uniform marginals alone could make every date the same random Z, giving collision probability 1, not pair probability 1/n. Pairwise independence suffices for the pair expectation but not the full no-collision product.

Pairwise versus mutual independence

Let U,V be independent uniform values in {0,1,2}; dates are $B_1=U, B_2=V, B_3=2U-V \bmod 3$. Any pair uniquely determines U,V, so each pair is uniform over nine ordered choices and is independent.

When U=V, all three dates agree, giving three cases. When $U \neq V$, all dates differ, giving six cases. Collision probability is therefore 1/3, unlike 7/9 for three mutually independent dates. Both models have expected colliding pairs $3 \cdot (1/3)=1$. The first produces three pairs with probability 1/3; the second has another distribution.

Pairwise independence suffices for this expectation. The complement product requires stronger joint information. Equal expectations do not determine equal distributions.

Loads, empty bins, and singleton bins

Throw t balls independently and uniformly into b≥1 bins. A fixed bin's load N sums t hit indicators, so E[N]=t/b and

$$P(N = r) = \binom{t}{r}(1/b)^r(1 - 1/b)^{t-r}.$$

Let Z_0 count empty bins and Z_1 singleton bins. Then $E[Z_0]=b(1-1/b)^t$ and $E[Z_1]=t(1-1/b)^{(t-1)}$, the latter for t≥1. At t=0, $Z_0=b, Z_1=0$. For b=1 use the direct event or the finite-product convention $0^0=1$.

At t=b=4, E[N]=1, $E[Z_0]=81/64$, $E[Z_1]=27/16$. Loads are dependent because their sum is fixed at t. Average load one does not mean one ball per bin.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Geometric waits and collecting every bin

A fixed bin is hit with probability 1/b on each independent throw. Its waiting time satisfies $P(T>t)=(1-1/b)^t$, so $E[T]=\Sigma_{t\geq 0}P(T>t)=b$.

After seeing i different bins, a new bin is hit with probability (b−i)/b. The next stage has mean b/(b−i). Regardless of dependence between stage durations, linearity gives

$$E[C] = \sum_{i=0}^{b-1} E[T_i] = b\sum_{j=1}^{b} 1/j = bH_b = \Theta(b \ln b) \quad (b \geq 2).$$

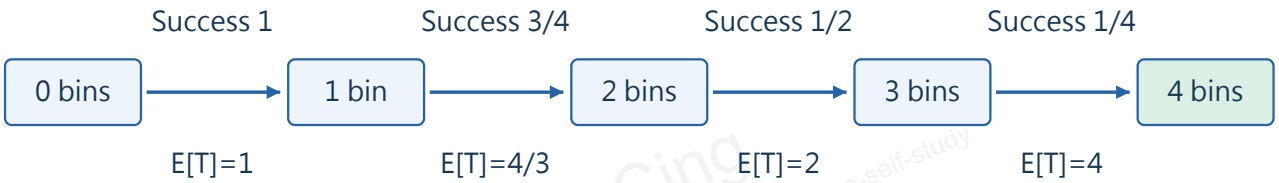
At b=1, C=1. For b=4 the stage expectations are 1,4/3,2,4, totaling 25/3. This is an expected waiting time, not a deterministic completion at a fractional ball count.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Geometric waits and collecting every bin (continued)

Collecting four bins: a repeated hit leaves the state unchanged

State = distinct bins seen. Advance only on a new bin.



Sample throws: 2,2,4,1,4,3. New bins appear at throws 1,3,4,6.
Stage means sum to 1+4/3+2+4=25/3, not a fixed stopping time.

Coverage and first collision have different scales

After t throws, a specified bin is empty with probability $(1-1/b)^t \le e^{(-t/b)}$. The union bound gives $P(C > t) \le be^{-t/b}$. Thus $t \ge b(\ln b + c)$ makes failure probability at most e^{-c}, without independent empty-bin events.

For first repeated-bin time D, P(D>k) is exactly the birthday no-collision event, at most $\exp(-k(k-1)/(2b))$. For $k \ge 2$, $k(k-1) \ge k^2/2$. The tail sum gives

$$E[D] \le 2 + \sum_{k \ge 2} e^{-k^2/(4b)} \le 2 + \int_0^\infty e^{-x^2/(4b)} dx = 2 + \sqrt{\pi b}.$$

For $k \le \sqrt{b}/2$, the union bound makes collision probability at most $k(k-1)/(2b) \le 1/8$. The first $\Theta(\sqrt{b})$ tails are at least 7/8, giving $\Omega(\sqrt{b})$.

First collision therefore has mean $\Theta(\sqrt{b})$, whereas collecting all bins has mean $\Theta(b \log b)$. Different stopping conditions produce different scales.

Streak windows and longest length

Flip n independent fair coins. Let L be the longest H run, zero if no H occurs. For $1 \le k \le n$, let A_{ik} mean all H in the k-window starting at i, with $1 \le i \le n-k+1$. Its probability is $2^{(-k)}$, so the expected number of such windows is $(n-k+1)/2^k$.

Overlapping windows are dependent: $A_{1,3}$ and $A_{2,3}$ both require four initial Hs, probability 1/16 rather than $(1/8)^2$. Expectations still add. The event $L \ge k$ is their union and needs union or joint analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Streak windows and longest length (continued)

Four consecutive Hs contain two dependent length-3 windows

Three length-3 all-H windows start at 1,5,6.

Longest run: 5..8, length 4.

Overlapping windows can both occur; starts 5 and 6 share two flips.

The illustrated sequence has length-3 windows starting at 1,5,6, totaling three. Four consecutive Hs in the second run contribute two overlapping windows, not two disjoint runs. Longest length L=4 differs from window count 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General upper and tail bounds for longest streaks

The union bound gives P(L≥k)≤min(1,n2^(−k)). For n≥1 set k_0=ceil(lg n). The integer tail identity gives

$$E[L] = \sum_{k=1}^{n} P(L \geq k) \leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1.$$

For r≥1 and k=ceil(r lg n), the tail is at most n^(1−r). For n=1000,k=20, the sharper window count gives 981/2^{20}<1/1000; k=30 gives at most 1000/2^{30}<10^{-6}. Tail guarantees describe individual-run risk beyond the expectation.

At n=0 set L=0 directly instead of taking log 0. For k>n the event is impossible.

A lower bound from independent disjoint blocks

For sufficiently large n let s=floor((lg n)/2)≥1,m=floor(n/s). Partition into m disjoint length-s blocks, ignoring the remainder. Each all-H event has probability 2^(−s)≥1/$\sqrt{n}$. Distinct blocks use disjoint independent coins, making these events mutually independent.

$$P(L < s) \leq (1 - 2^{-s})^m \leq e^{-m2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0,$$

Here m=Θ(n/log n), so m/$\sqrt{n}$→∞. Consequently E[L]≥sP(L≥s)=Ω(log n), matching the upper bound. Merely having many expected overlapping windows would not prove this lower bound; independent blocks establish high probability of at least one success.

For n=16,s=2,m=8, at least one successful block has probability 1−(3/4)^8=58975/65536. Hence E[L]≥2・58975/65536, a finite lower bound rather than an exact expectation.

Complete online-selection model

Let n≥2 distinct-quality candidates arrive in uniform random order. Only seen candidates can be compared; rejected candidates cannot be recalled, and exactly one hire is allowed. Fix an integer 1≤k<n. Reject the first k while recording their maximum M, then select the first later score exceeding M. If none does, select the last candidate.

```
ONLINE-SELECT(n,k):
    M = -infinity
    for i = 1 to k: M = max(M, score(i))
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

M need not change in the second loop. Before the first exceedance all new scores are at most M; at the first exceedance the procedure returns. Known n, uniform arrivals, distinct scores, and no recall are premises. Arbitrary arrival distributions do not inherit a 1/e guarantee.

Success probability by conditioning

Suppose the best candidate is at i. If i≤k the strategy fails. For i>k it succeeds iff the maximum of the first i−1 lies among the first k. Otherwise an earlier later-stage record already triggered selection; conversely, when the maximum is in the sample no earlier candidate exceeds M.

The best position has probability 1/n. Conditional on that position, the earlier relative order remains uniform, giving success probability k/(i−1). The i events are disjoint, so

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

Conditioning proves the formula without asserting unproved independence. At n=1 select the sole candidate, with probability 1. Define k=0 separately as selecting the first candidate, probability 1/n; do not multiply zero by an undefined harmonic term.

Finite thresholds and the 1/e limit

Let $q_k = \sum_{j=k}^{n-1} 1/j$. The exact difference is $P_{k+1} - P_k = (q_{k+1} - 1)/n$. Since q decreases strictly, an optimal threshold has $q_k \geq 1, q_{k+1} \leq 1$. Equality can give adjacent tied optima. The finite answer need not equal n/e.

Integral bounds give $\ln(n/k) \leq q_k \leq \ln(n/k)+1/k$, hence

$$\frac{k}{n} \ln \frac{n}{k} \leq P_k \leq \frac{k}{n} \ln \frac{n}{k} + \frac{1}{n}.$$

For x=k/n, f(x)=x ln(1/x) has derivative ln(1/x)−1 and second derivative $-1/x<0$. Its maximum is at x=1/e. Integer k near n/e gives f(k/n)=1/e−O(1/n), so success is at least 1/e−O(1/n). The optimum is at most 1/e+1/n. Therefore the limit is 1/e, without an extra 1/k factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six-candidate thresholds and trace

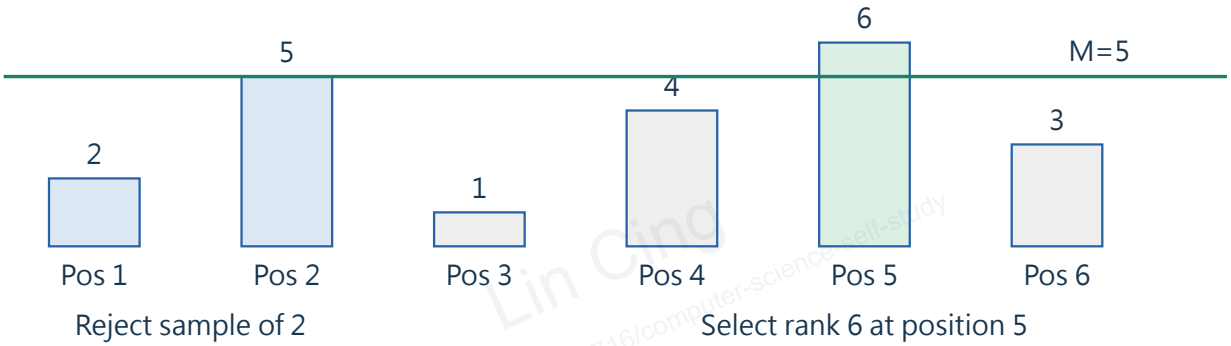
For n=6, probabilities for k=1,2,3,4,5 are respectively 137/360,77/180,47/120,3/10,1/6. Since $q_2 = 77/60 > 1$ and $q_3 = 47/60 < 1$, k=2 is optimal.

On ranks [2,5,1,4,6,3], rejecting the first two gives M=5. Scores 1 and 4 at positions 3 and 4 fail to exceed it. Score 6 at position 5 triggers selection; position 6 need not be interviewed. This trace chooses the global best, though other permutations fail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six-candidate thresholds and trace (continued)

Ranks [2,5,1,4,6,3]: sample maximum 5; first exceedance at position 5



On [5,6,1,2,3,4], the global best is rejected in the sample. No later score exceeds M=6, so fallback selects 4. Correctly executing the strategy does not guarantee success on every permutation.

Extension: a random recursive split

For size $n \geq 2$, print n stars, choose p uniformly from $1..n-1$, and recurse on p and $n-p$. A size-1 call prints one exclamation mark and stops. Let L_n, S_n be expected exclamation and star counts, with $L_1=1, S_1=0$.

Condition first on p and add child costs, then average:

$$L_n = \frac{2}{n-1}\sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1}\sum_{j=1}^{n-1} S_j.$$

Every split preserves total size, and every leaf has size 1, so every execution tree has exactly n leaves. $L_n=n$ is stronger than an expectation. Height and star count still depend on p; extreme splits can create chains without changing the leaf count.

Closed form for random splits

For L_n, strong induction substitutes L_j=j and gives $2\sum_{j<n} j/(n-1) = n$. For stars, multiply by n−1 to obtain $(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j$. Subtract this from the corresponding n+1 equation to get

$$S_{n+1} = 2 + \frac{n+1}{n} S_n.$$

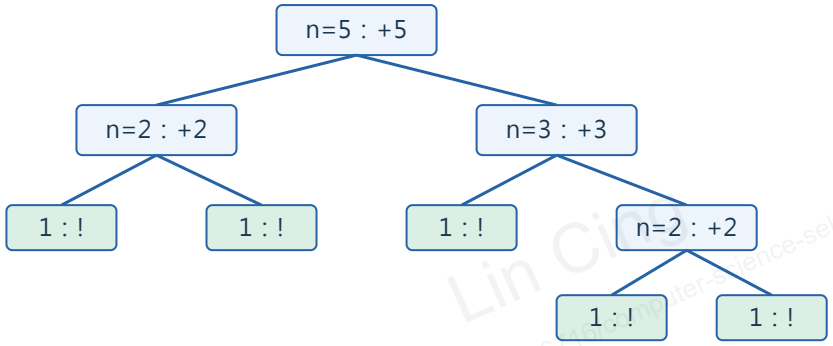
With $Q_n = S_n/n$, $Q_{n+1} = Q_n + 2/(n+1)$. Since $Q_1 = 0$, summing gives $Q_n = 2H_n - 2$, hence $S_n = 2n(H_n - 1) = \Theta(n \log n)$. It is expected cost, not the cost of every path.

n	Leaves in every execution	Expected stars
2	2	2
3	3	5
4	4	26/3
5	5	77/6
6	6	87/5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closed form for random splits (continued)

Five unit leaves: leaf count is conserved; star cost varies



Leaves here=5

Stars here=12

E[stars]=77/6

+number: stars at this call

!: one exclamation mark

Shown: p=2, then 3=1+2. All splits give 5 leaves; total stars depend on splits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extension: expected Quicksort comparisons

For distinct keys, independently choose a uniform pivot in every subproblem of size at least 2. Compare each other key once to the pivot, partition into smaller and larger groups, recursively sort, and concatenate. Sizes 0 and 1 return directly. Group ordering and induction prove correctness. Count only key comparisons, separately from copying, allocation, and randomness.

Write sorted keys $z_1<\ldots<z_n$, and let I_{ij} indicate a direct comparison. A pair compares at most once, only when one endpoint becomes pivot and is then removed. Equivalently assign a uniform random priority order to all keys and use each subproblem's earliest key as pivot. Remaining relative orders stay uniform, matching recursive uniform pivot choices. The first pivot among $z_i..z_j$ is therefore uniform in that interval. Outside pivots leave the entire interval together in one child.

If the first interval pivot is an endpoint, the pair compares. An interior pivot permanently separates them. Thus $P(I_{ij}=1)=2/(j-i+1)$, and total $X=\Sigma_{i<j}I_{ij}$.

Quicksort sums, numbers, and boundaries

Group by distance $d=j-i$, with $n-d$ pairs of that distance.

$$E[X]=2\sum_{d=1}^{n-1}\frac{n-d}{d+1}=2\sum_{r=2}^{n}\left(\frac{n+1}{r}-1\right)=2(n+1)H_n-4n.$$

At $n=5$ this is $2(4/2+3/3+2/4+1/5)=37/5$. For $n=2,3,4,6$, values are $1,8/3,29/6,103/10$. Constant per-element partition work gives time $\Theta(n+X)$, hence expected $\Theta(n\log n)$. Extreme pivot sequences still require $n(n-1)/2$ comparisons. A high-probability bound needs a separate proof and does not follow automatically from expectation.

Distinctness or an explicit duplicate-key policy is necessary. Two-way Lomuto partitioning that sends all equal keys to one side stays quadratic on all-equal input even with random pivots. Chapters 7 and 9 address three-way equal regions. The distinct-key pair probability cannot be transferred unchanged.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 5, § § 5.1–5.4 and § 7.1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Amortized analysis

Part IV Advanced Design and Analysis Techniques

Scope

One operation can be expensive while an entire sequence costs only linear time. Stacks, counters, and dynamic tables develop three amortized methods, with attention to initial credit, every prefix's balance, and payment for copies at growth/shrink boundaries.

Prerequisites

Operation sequences and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Distinguish amortized from average-case cost.
- Apply aggregate, accounting, and potential analysis.
- Prove O(1) amortized cost for doubling tables.
- 17.1 Aggregate analysis
- 17.2 The accounting method
- 17.3 The potential method
- 17.4 Dynamic tables

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Aggregate analysis

Amortized analysis bounds total cost for **every** legal sequence. From a fixed initial state, if every length-m sequence satisfies $\Sigma c_i \leq Am+B$, each operation has $O(1)$ amortized cost. Individual c_i need not be constant, and no random-input averaging is involved.

For an initially empty stack, PUSH costs1, a successful POP costs1, and MULTIPOP(k) removes $q=\min(k,\text{size})$ elements at cost q. Fixed call and empty-test overhead adds $O(1)$ separately.

Each element is pushed once and popped at most once, so total popping work is at most the number of pushes. m operations contain at most m pushes; total element work$\leq 2m$. A single MULTIPOP may cost $\Theta(m)$, yet any complete sequence costs $O(m)$.

Total binary-counter flips

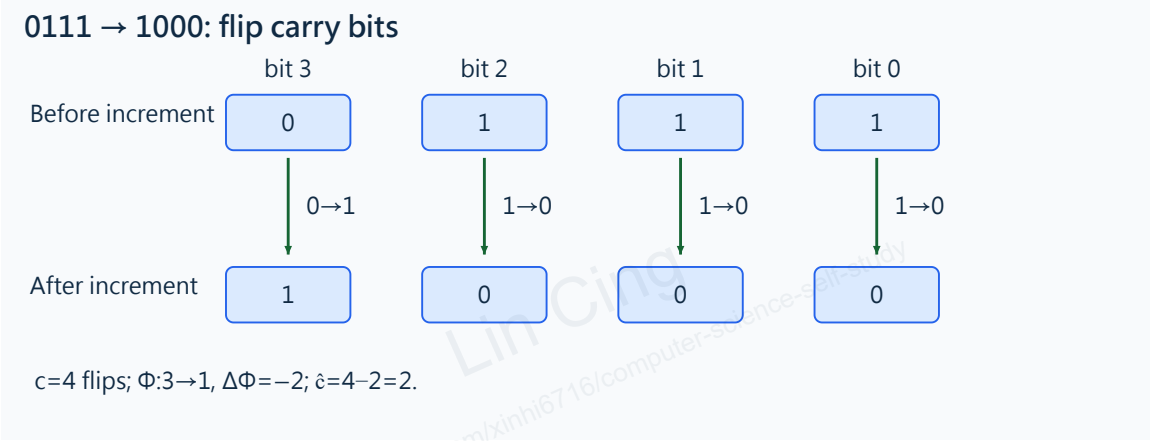
A fixed $b \geq 1$-bit counter starts at all zeros. INCREMENT changes trailing ones to zeros, then the first zero to one. All ones overflows to all zeros. Count bit flips.

Bit j flips once per2^j increments. Over m increments, total flips are

$\Sigma_{j=0}^{b-1} \lfloor m/2^j \rfloor < 2m$.

No independence assumption is needed. Bit0 flips m times, giving $\Omega(m)$, hence total $\Theta(m)$. An implementation scanning all b bits even when unchanged uses a different cost model.

Total binary-counter flips (continued)



c=4 flips; Φ:3→1, ΔΦ=−2; ĉ=4−2=2.

A geometric bound for doubling tables

First allow insertion only, starting at(n,M)=(0,0). Allocate one slot initially, then double when full. Cost is one insertion write plus the number of old elements copied; fixed allocation/release overhead is constant.

After m≥2 insertions, the last copy size L is the largest power of two smaller than m. Copy sizes1,2,4,…,L sum to2L−1<2m, so

$T(m)=m+2L-1<3m$.

Total copies are not generally≤m. At m=5, copies1+2+4=7>5 and total cost12. At m=8, copies7 and total15. Only selected endpoints have fewer copies than insertions.

If every new slot must be initialized, count that work too. Its geometric sum remains O(m), but the precise constant3 changes.

The accounting method

Charge an amortized fee and retain income minus actual spending as credit. Prove that credit is nonnegative for **every prefix** before using total fees to bound actual cost.

For an empty stack, charge PUSH2 and POP/MULTIPOP0. One unit pays insertion and one stays with the element for its eventual removal. A nonexistent element cannot be popped, so no prefix overspends. Add a fixed fee if calls have fixed overhead.

Charge each counter increment2. A0→1 flip spends1 and stores1 on that bit. A1→0 flip uses its stored credit. All-ones overflow is paid entirely from old credit. All-zero initialization needs none; a nonzero initial state requires enough initial credit for its one bits.

Doubling-table fees and the actual sequence

Charge3 per insertion. The aggregate prefix bound T(m)<3m gives B_m=3m−T(m)≥0. Starting empty:

Operation	Final(n,M)	Copies	c_i	Total cost	Credit B_i
1	(1,1)	0	1	1	2
2	(2,2)	1	2	3	3
3	(3,4)	2	3	6	3
4	(4,4)	0	1	7	5
5	(5,8)	4	5	12	3
6	(6,8)	0	1	13	5
7	(7,8)	0	1	14	7
8	(8,8)	0	1	15	9

Doubling-table fees and the actual sequence (continued)

Capacity2→4 occurs at operation3, not4. Copies cannot be shifted to a later operation. The next section's potential inequalities also prove the credit bound step by step.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The potential method

For state D_i, define potential Φ(D_i) and amortized cost

$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{(i-1)})$.

Summing m operations telescopes:

$\Sigma c_i = \Sigma \hat{c}_i + \Phi(D_0) - \Phi(D_m)$.

If every reachable final state has $\Phi(D_m) \geq \Phi(D_0)$, total actual cost≤total amortized cost. A common sufficient condition is $\Phi(D_0)=0$ and $\Phi(D_i) \geq 0$. Positive initial potential instead generally yields $\Sigma c_i \leq \Sigma \hat{c}_i + \Phi(D_0)$; do not discard that initial term.

Adding a constant to all potentials changes no $\hat{c}_i$. The key is the final potential's lower bound relative to the initial one, not absolute nonnegativity everywhere. Individual $\hat{c}_i$ can be negative when stored potential subsidizes the operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack and counter potential proofs

For an empty stack use Φ=size. PUSH has $c=1, \Delta\Phi=1, \hat{c}=2$. Removing q elements has $c=q, \Delta\Phi=-q, \hat{c}=0$. Nonnegative potential and initial0 give element work$\leq$2m for every m-operation sequence.

For a counter use Φ=number of one bits. Flipping t trailing ones and then a zero gives $c=t+1, \Delta\Phi=1-t, \hat{c}=2$. All-ones overflow gives $c=b, \Delta\Phi=-b, \hat{c}=0$. All-zero initialization therefore gives total flips$\leq$2m.

For arbitrary b-bit initialization, $\Phi(D_0)\leq b$, so flips$\leq$2m+b. Starting at all ones makes the first increment cost b, consistent with this bound. Arbitrary initial states do not retain the2m prefix bound without initial credit.

Dynamic tables

Now support insertion and deletion at the end. Capacities remain powers of two under these exact rules.

```
insert:
    if M==0: allocate M=1
    elif n==M: allocate 2M; copy n items; M=2M
    write new item; n++
delete-last:
    require n>0
    n--
    if n==0: free storage; M=0
    elif M>=4 and n<=M/4:
        allocate M/2; copy n items; M=M/2
```

We shrink at post-deletion $n\leq M/4$, slightly different from the textbook's strict-below-quarter trigger. The exact values below are proved for our rule, not borrowed across boundary conventions. Successful deletion has basic cost1 plus n copied elements if shrinking.

Shrinking needs a buffer

Shrinking as soon as half full causes thrashing. Start full at(k,k), with sufficiently large power-of-two k. Insert once, copying k to reach(k+1,2k). Delete once and immediately shrink back to capacity k, copying k again. Repeating costs Θ(k) per pair and Θ(mk) over m operations.

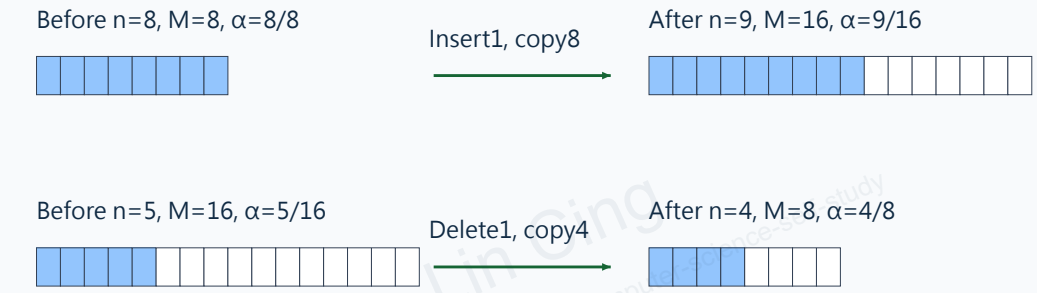
A quarter-full shrink instead leaves approximately half-full tables after both growth and shrink. Θ(M) intervening deletions are needed to shrink after growth, or insertions to grow after shrink. This allows credit accumulation; the following piecewise potential proves all mixed sequences rather than relying on intuition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shrinking needs a buffer (continued)

Double when full, halve at quarter load

Blue: occupied. White: empty.



Growth leaves about half full; shrink leaves half full. Reverse triggers remain distant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Piecewise potential

Set $\Phi(0,0)=0$. For nonempty tables with load $\alpha=n/M$,

```
Φ(n,M) = 2n−M       if n>=M/2
         M/2−n      if n<M/2
```

Both pieces are nonnegative and meet at zero when half full. A full table has $\Phi=M$, enough for its next M copies. A quarter-full table has $\Phi=M/4$, comparable to its shrink copy count.

Using only$2n-M$ fails for deletion: reachable$(n,M)=(3,8)$ gives-2 and cannot use initial0/final-nonnegative reasoning. The piecewise potential gives1 there. Exact before/after resize states still need case analysis.

Insertion cases

First insertion:$(0,0)\to(1,1)$, $c=1,\Phi 0\to 1,\hat{c}=2$.

Without growth, starting at least half full gives $\Delta\Phi=2$ and$\hat{c}=3$. Starting below half full can reach at most exactly half full; continuity of the low-load branch gives $\Delta\Phi=-1$ and$\hat{c}=0$.

A full-table growth goes from(M,M) to$(M+1,2M)$. Actual cost is $M+1$ and potential $M\to 2$, so

$\hat{c}=(M+1)+(2-M)=3$.

Every insertion has $\hat{c}\leq 3$. The argument includes switching branches at half full rather than assuming both states use the same formula.

Deletion cases

Without shrink, remaining at least half full gives $\Delta\Phi=-2$ and$\hat{c}=-1$. Dropping one element from exactly half full gives $\Phi 0\to 1$ and$\hat{c}=2$. Starting below half full without shrink gives $\Delta\Phi=1$ and$\hat{c}=2$.

A shrink must start with n_old=M/4+1, then n_new=M/4 and capacity M/2. Old $\Phi=M/4-1$; the new table is half full with $\Phi=0$. Actual cost1+M/4 gives

$\hat{c}=1+M/4-(M/4-1)=2$.

Deleting to empty releases storage, costs1, and drops nonnegative potential to0, so$\hat{c}\leq 1$. All operations have$\hat{c}\leq 3$. Initial potential0 and final potential≥ 0 imply actual cost$\leq 3m$ for every legal m-operation sequence, deterministically and without probability assumptions.

Full mixed trace and model boundaries

Insert to n=5, reaching$(n,M,\Phi)=(5,8,2)$, then delete:

Operation	Before(n,M,Φ)	After(n,M,Φ)	Copies	c	$\hat{c}$
delete	(5,8,2)	(4,8,0)	0	1	−1
delete	(4,8,0)	(3,8,1)	0	1	2
delete	(3,8,1)	(2,4,0)	2	3	2
delete	(2,4,0)	(1,2,0)	1	2	2
delete	(1,2,0)	(0,0,0)	0	1	1

Five insertions cost12 and five deletions8, total20. Total amortized cost also20 because both endpoint potentials are0. Negative amortized cost does not mean negative actual time.

Only end deletion is covered. Deleting in the middle while preserving contiguous order adds $\Theta(n)$ suffix movement, so $O(1)$ amortized cost does not carry over. Different growth/shrink factors or thresholds require rechecking slopes and every boundary case.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter17.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heapsort

Part II Sorting and Order Statistics

Scope and prerequisites

A binary heap represents a nearly complete binary tree by array indices, supporting extreme-key extraction and priority updates. Align parent/child indices, heap size, and every swap, prove linear bottom-up building, and distinguish sorting from priority-queue operations. Prerequisites are complete binary trees, array indexing, and sorting invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Operate a max-priority queue using the max-heap property.
- Prove heapify costs and linear-time heap construction.
- Sort by repeatedly selecting the heap maximum and analyze O(n log n).

Sections 6.1–6.5 cover heaps, maintenance, building, heapsort, and priority queues.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps

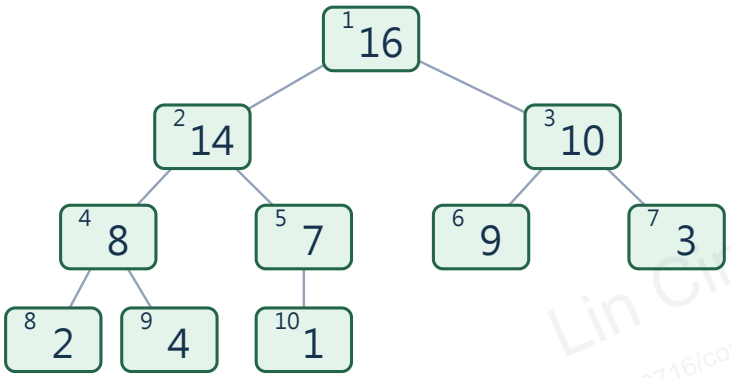
A binary heap is nearly complete: all but the last level are full, and the last fills left to right. In a 1-based array, root=1, parent(i)=floor(i/2), left(i)=2i, and right(i)=2i+1. Only indices at most heap-size exist.

A max-heap requires $A[\text{parent}(i)] \geq A[i]$ for every nonroot i. Repeated ancestor inequalities prove that the root is maximum. Siblings and the two subtrees have no mutual sorting order, so arbitrary-key binary search does not apply.

Array length and heap size differ. Heapsort excludes its fixed sorted suffix from heap size; heapify must not revisit it. For $n \geq 1$, depth h's leftmost index is 2^h, giving height $\text{floor}(\log_2 n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps (continued)



Node: small index, large key

Fixed indices; moving keys

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

Every edge: parent key ≥ child key
Nodes on one level need not be sortec

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 Maintaining the heap property

MAX-HEAPIFY(A,i,s) assumes both child subtrees are already max-heaps, with only i possibly smaller than a child. Here s is heap size. An interface must reject i outside 1..s, or skip a call for an empty heap.

```
while true:
    largest=i
    if 2i<=s and A[2i]>A[largest]: largest=2i
    if 2i+1<=s and A[2i+1]>A[largest]: largest=2i+1
    if largest==i: return
    swap(A[i],A[largest])
    i=largest
```

Equivalently choose the larger child and compare it to the parent. Swapping merely because one child exceeds the other, without comparing the parent, is incorrect.

Heapify correctness and height cost

If the parent is largest, both parent-child edges and the already legal child subtrees form a heap. Otherwise the greatest child root is at least all its descendants. Moving it up makes both top edges legal. Only the old parent moved down can remain invalid.

Induct on remaining height to repair that subtree; the other subtree is unchanged. Each new i is 2i or 2i+1, one level lower, proving termination. A height-h subtree costs O(h+1) in the worst case; root repair costs O(log(n+1)). An already large parent may stop in O(1).

For A=[16,4,10,8,7,9,3],s=7,i=2, choose 8 over 7 and exchange 4↔8, obtaining [16,8,10,4,7,9,3]. At i=4, children 8 and 9 exceed s, so stop. The value 3 at index 7 is a child of index 3, not of 4.

6.3 Building a heap

```
s=length(A)
for i=floor(s/2) down to1:
    MAX-HEAPIFY(A,i,s)
```

Before an iteration, every index greater than i roots a legal heap. Initially floor(n/2)+1..n are leaves. Children of i have larger indices, so heapify's preconditions hold; afterward i also roots a legal heap. This maintains the next iteration. At i=0 the root is legal, so the whole array is a heap.

Empty and singleton arrays need no iterations. Index floor(n/2) need not be a leaf: with n=10, index 5 still has child 10 and their keys must be compared.

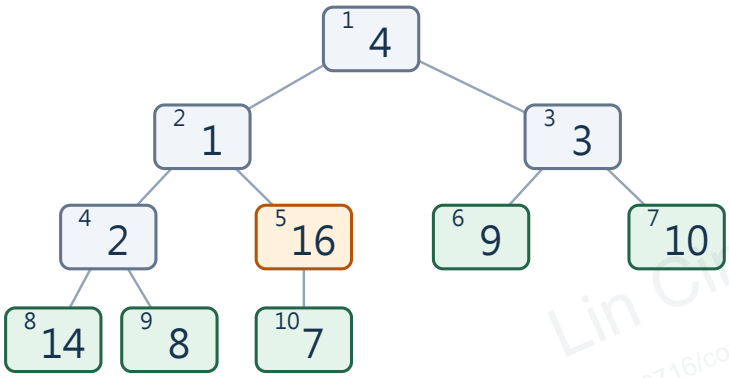
Five complete build rounds

Input A=[4,1,3,2,16,9,10,14,8,7],s=10.

i	Comparisons and swapped indices	Full result
5	16≥7 at child 10; no swap	[4,1,3,2,16,9,10,14,8,7]
4	choose 14 over 8; swap 4↔8	[4,1,3,14,16,9,10,2,8,7]
3	choose 10 over 9; swap 3↔7	[4,1,10,14,16,9,3,2,8,7]
2	choose 16 over 14; swap 2↔5, then 5↔10	[4,16,10,14,7,9,3,2,8,1]
1	choose 16, then 14, then 8; swap 1↔2,2↔4,4↔9	[16,14,10,8,7,9,3,2,4,1]

The next five figures use exactly these records. Orange marks the processed root, green roots already guaranteed to be heaps, and gray pending roots. Indices remain fixed during swaps; keys at those positions change.

Build at i=5: compare its only child



Node: small index, large key

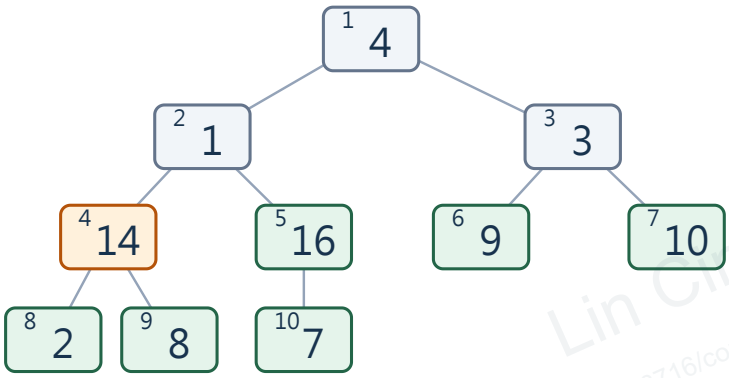
Fixed indices; moving keys

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

Orange: root just repaired
Green: legal; gray: pending

Value 16 at position 5 exceeds value 7 at 10, so no swap occurs. Positions 6..10 are leaves; the subtree rooted at 5 is now legal too.

Build at i=4: promote 14



Node: small index, large key

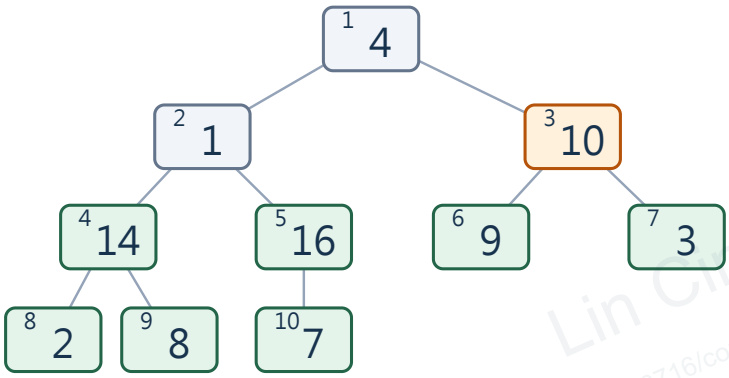
Fixed indices; moving keys

1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

Orange: root just repaired
Green: legal; gray: pending

Original position 4 holds 2, with children 14 and 8. Choose 14 and exchange indices 4,8. Old 2 stops at leaf 8. Value 8 at index 9 is unchanged.

Build at i=3: promote 10



Node: small index, large key

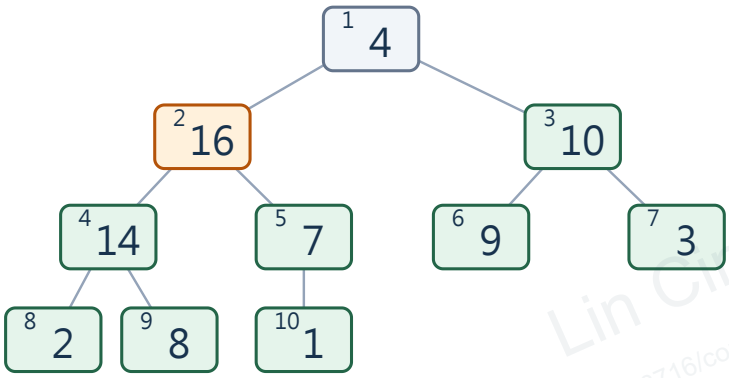
Fixed indices; moving keys

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

Orange: root just repaired
Green: legal; gray: pending

Original position 3 holds 3. Its children are 9 at index 6 and 10 at index 7. Swap 3,7; old 3 stops at leaf 7. Value 9 at 6 remains unchanged.

Build at i=2: descend through 5 to 10



Node: small index, large key

Fixed indices; moving keys

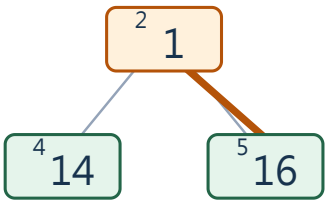
1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

Orange: root just repaired
Green: legal; gray: pending

Old 1 at position 2 first swaps with 16 at 5. It then has child 7 at 10, requiring another swap. Value 14 at index 4 is not moved by this round.

i=2: how does the old key 1 descend?

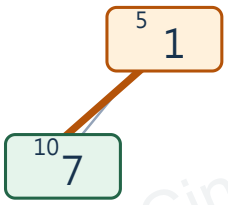
Comparison 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

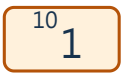
Comparison 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

Leaf: stop



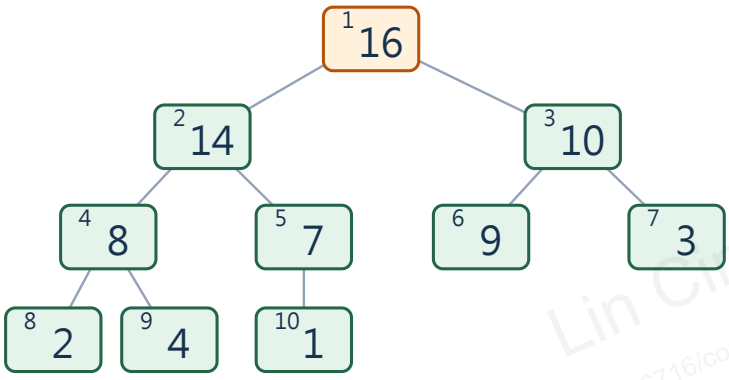
20, 21 > heap-size 10

No child access

Old key 1 ends at index 10

Orange follows the same old key 1. The thick edge selects the larger child. Small node labels are indices; large labels are keys. Omitted subtrees are unchanged.

Build at i=1: descend through 2,4,9



Node: small index, large key

Fixed indices; moving keys

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

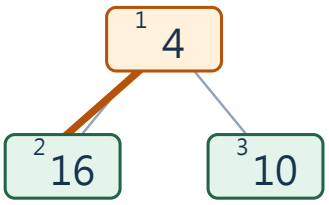
Orange: root just repaired

Green: legal; gray: pending

Old root 4 exchanges with 16,14,8 and stops at index 9. Final edges give 16≥14,10; 14≥8,7; 10≥9,3; 8≥2,4; 7≥1. All ten keys are preserved.

i=1: three comparisons for the old root 4

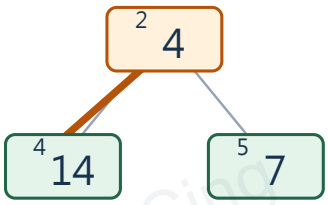
Comparison 1



16 > 4 → swap(1,2)

A[1]=16; A[2]=4

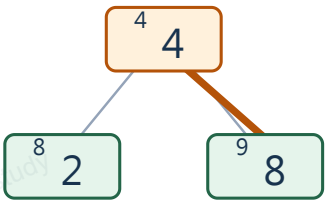
Comparison 2



14 > 4 → swap(2,4)

A[2]=14; A[4]=4

Comparison 3



8 > 4 → swap(4,9)

A[4]=8; A[9]=4

Each panel is the local state before a swap; its bottom line gives the changed keys afterward. Key 4 ends at leaf index 9: indices 18,19 exceed heap-size=10. The path is not a scan of final array values.

Check the full array after every swap

Indices stay 1..10. Continue from the completed i=3 round and match each row to the preceding local diagrams.

Round root	Swapped indices	A[1..10] after the swap
2	2 ↔ 5	[4,16,10,14,1,9,3,2,8,7]
2	5 ↔ 10	[4,16,10,14,7,9,3,2,8,1]
1	1 ↔ 2	[16,4,10,14,7,9,3,2,8,1]
1	2 ↔ 4	[16,14,10,4,7,9,3,2,8,1]
1	4 ↔ 9	[16,14,10,8,7,9,3,2,4,1]

Check yourself: hide the results, select the larger child, then compare it with the parent. Each swap changes only two positions; the other eight entries and the key multiset stay unchanged.

Why heap construction is linear

Position i has height at least h iff its leftmost h-generation descendant i • $2^h \le n$. Exactly floor(n/2^h) positions have height at least h. Count each node's height one level at a time:

$$\sum_i height(i) = \sum_{h \ge 1} \lfloor n/2^h \rfloor < n.$$

Each heapify costs O(height(i)+1), and there are floor(n/2) calls, proving O(n). For $n \ge 2$ the loop itself processes floor(n/2) roots, giving Ω(n), hence Θ(n). At n=15 the height sum is 7+3+1=11.

Repeated INSERT operations can instead have Θ(n log n) total upward travel. They are a different operation sequence and cannot use this bottom-up height sum.

6.4 The heapsort algorithm

```
BUILD-MAX-HEAP(A)
for end=length(A) down to2:
    swap(A[1],A[end])
    MAX-HEAPIFY(A,1,end-1)
```

Before each iteration, A[1..end] is a max-heap. The suffix A[end+1..n] is sorted increasingly and every suffix key is at least every heap key. Swapping fixes the current maximum at A[end]. Shrinking s and repairing only the prefix preserves the invariant. At end=1 all records are ordered. Swaps preserve their multiset.

Time is O(n log(n+1)); the comparison-sorting information bound gives worst-case Θ(n log n). With equal-key heapify stopping immediately, all-equal input can take Θ(n), so the worst bound does not apply equally to every input. Iterative heapify uses O(1) extra space; recursive heapify adds O(log n) stack.

6.4 The heapsort algorithm (continued)

For the lower bound, distinct keys have n! possible input orders. A correct comparison decision tree needs at least n! leaves; a binary tree of depth d has at most 2^d leaves. Thus $d \geq \log_2(n!)$. The final floor(n/2) product factors are each at least n/2, giving $\log_2(n!) \geq \text{floor}(n/2)\log_2(n/2) = \Omega(n \log n)$. At least one input needs that many comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every ten-element heapsort round

Each round swaps the root with the last heap position, then repairs the remaining prefix.

New heap size	Repaired heap	Sorted suffix
9	[14,8,10,4,7,9,3,2,1]	[16]
8	[10,8,9,4,7,1,3,2]	[14,16]
7	[9,8,3,4,7,1,2]	[10,14,16]
6	[8,7,3,4,2,1]	[9,10,14,16]
5	[7,4,3,1,2]	[8,9,10,14,16]
4	[4,2,3,1]	[7,8,9,10,14,16]
3	[3,2,1]	[4,7,8,9,10,14,16]
2	[2,1]	[3,4,7,8,9,10,14,16]
1	[1]	[2,3,4,7,8,9,10,14,16]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every ten-element heapsort round (continued)

In round two, replacement root 1 follows only indices 1→3→6, exchanging with 10 and 9, not with 8 or 2. A swap path is not a list of all values in the final array.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5 Priority queues

MAXIMUM reads A[1] in O(1). EXTRACT-MAX saves it, replaces it with the final key, decreases s, and heapifies, taking worst-case O(log(n+1)). Empty input returns EMPTY instead of reading A[1].

INCREASE-KEY(A,i,k) requires k at least the old key. Change it, then swap upward while the parent is smaller. Stop at a sufficient parent or the root. Only ancestor edges can fail; each upward step repairs one, giving correctness and worst-case O(log n).

INSERT appends a position with a formal negative-infinity key, then increases it. Equivalently write the new key directly and use the same upward repair without a representable infinity. Dynamic-array growth needs separate amortized analysis; fixed capacity first checks FULL.

For A=[20,15,18,7,12,9,17,2], increase position 8 to 16. Swap 8↔4 to obtain [20,15,18,16,12,9,17,7], then 4↔2 to obtain [20,16,18,15,12,9,17,7]. Parent 20 stops the ascent.

Decreases, deletion, and stability

A max-heap key decrease can violate only child edges, so repair downward; an increase repairs upward. To delete position i, fill from the final position and decrease s. If the replacement exceeds its parent, move upward; otherwise heapify downward. Deleting the last position needs only shortening. Each repair costs O(log n), but searching for an arbitrary key to obtain i can still cost Θ(n).

If clients identify records by handles, every swap must update the handle-to-index map. An array index does not permanently identify one record.

Heapsort is generally unstable. With (2,a),(2,b),(1,c) and no equal-key heapify swap, output can be (1,c),(2,b),(2,a), reversing a,b. Adding original positions to comparison keys can enforce stability but changes comparison and storage requirements.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 6.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sorting in linear time

Part II Sorting and Order Statistics

Scope and prerequisites

Linear sorting exchanges restrictions on key representations or input distributions for speed. Distinguish the comparison-model lower bound, then derive counting-sort blocks, LSD stability, and squared bucket-size costs. State ranges, bit lengths, distributions, and space requirements. Prerequisites are comparison trees, integer ranges, and stable sorting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Prove the Ω(n log n) comparison-sorting lower bound.
- Analyze counting-sort assumptions and stability.
- Identify the cost premises of radix and bucket sort.

Sections 8.1–8.4 cover lower bounds, counting sort, radix sort, and bucket sort.

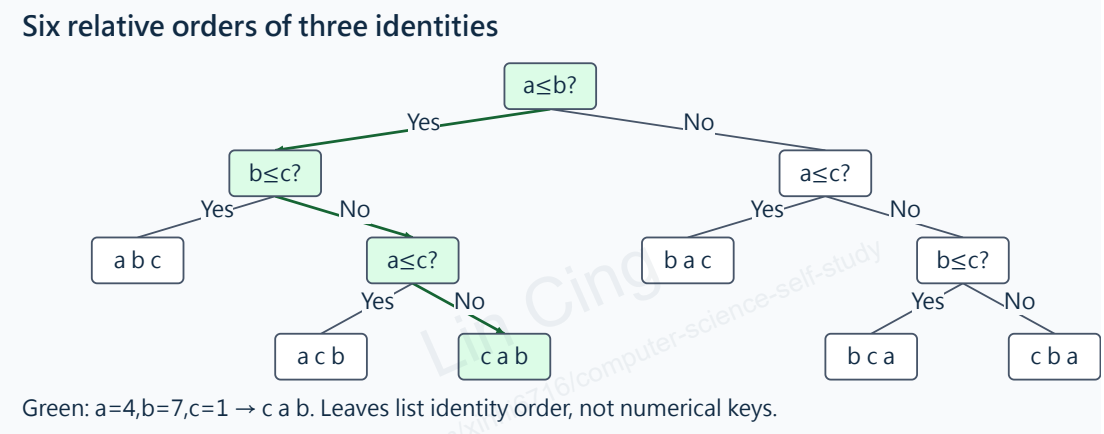
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1 Lower bounds for sorting

A comparison sort obtains order information only through key comparisons, not by using keys as indices. Restrict first to n distinct keys, making each $\leq$ comparison binary. For a fixed n and algorithm, an internal node names two original identities to compare, a root-to-leaf path records an execution, and a leaf determines the entire identity order.

The n! relative orders need distinguishable leaves. If two different orders reached the same leaf, the same result sequence would force the same output identity order, making at least one input wrong. Several leaves may represent the same output, but at least n! reachable leaves are necessary.

8.1 Lower bounds for sorting (continued)



A leaf such as “c a b” means c<a<b, not six arbitrary numerical outputs. The input a=4,b=7,c=1 takes left,right,right to reach c a b.

Quantifiers and model of the worst-case bound

A binary tree of height h has at most 2^h leaves, so $h \geq \text{ceil}(\log_2(n!))$. Without Stirling, $n! \leq n^n$, while its final ceil(n/2) factors are each at least n/2. Hence

$$(n/2)^{n/2} \leq n! \leq n^n, \qquad \log_2(n!) = \Theta(n \log n).$$

For every correct comparison sort and each n, **some** length-n input therefore requires Ω(n log n) comparisons. Not every input does: sorted insertion-sort input uses n−1 comparisons.

Distinct keys form a hard subset even when duplicate inputs are allowed, so allowing duplicates does not remove the general worst-case bound. Counting and radix sorts read integer representations and index arrays, leaving the comparison model. They incur range and bit-length costs instead of contradicting the theorem.

Average comparisons require a proof too

Suppose all N=n! orders are uniform. Choose a reachable leaf for each, with depths $d_1,\ldots,d_N$. Distinct leaf paths form prefix-free binary strings. Kraft's inequality $\Sigma 2^{(-d_i)} \leq 1$ follows by assigning each depth-d leaf a disjoint binary interval of length $2^{(-d)}$ in [0,1).

Arithmetic mean at least geometric mean gives

$$2^{-\sum_i d_i/N} \leq \frac{\sum_i 2^{-d_i}}{N} \leq 1/N.$$

Taking logs proves average depth at least $\log_2 N = \Omega(n \log n)$. A deepest-leaf bound alone would not prove this average.

For an always-correct randomized comparison sort, fixing random bits gives a correct tree. Its uniform-input average obeys the same bound. Averaging over random bits preserves it, so some input also has expected comparisons Ω(n log n). This argument does not cover error-permitting Monte Carlo sorting.

8.2 Counting sort

Keys are integers 0..k. Satellite data travels with its record. Input/output positions are 1-based; count-table keys are 0-based.

```
C[0..k]=0
for j=1..n: C[key(A[j])]++
for v=1..k: C[v]+=C[v-1]
for j=n downto 1:
    v=key(A[j])
    B[C[v]]=A[j]
    C[v]--
return B
```

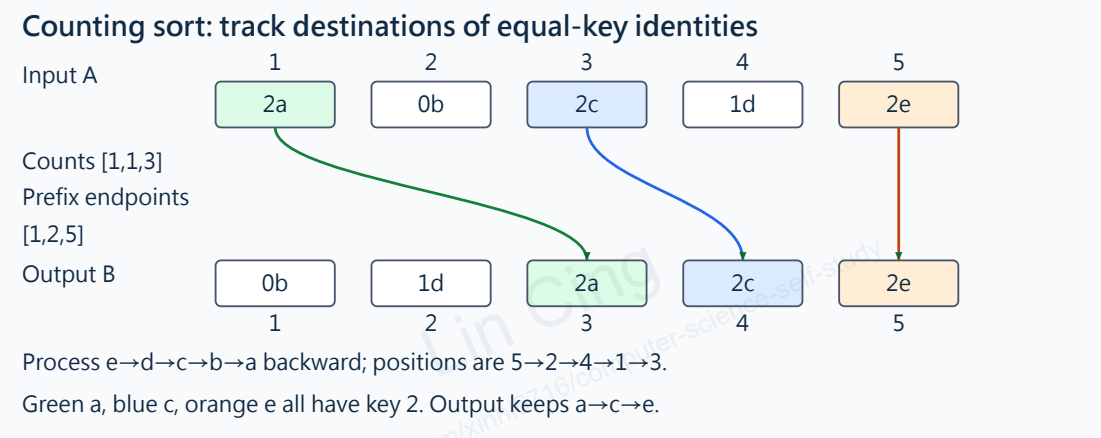
After counting, C[v] is key v's frequency. After prefix summing, it is the number of records with keys $\leq$v, the final position of v's block. Placement consumes C. Save the original prefix table P first if range queries are needed.

From frequencies to output positions

Use A=[2a,0b,2c,1d,2e], with identities a,b,c,d,e. Frequencies are [1,1,3], cumulative block endpoints [1,2,5]. Key 0 occupies position 1; key 1 occupies 2; key 2 occupies 3..5.

j	Record	Output position	Updated C
5	2e	5	[1,2,4]
4	1d	2	[1,1,4]
3	2c	4	[1,1,3]
2	0b	1	[0,1,3]
1	2a	3	[0,1,2]

From frequencies to output positions (continued)



Output is [0b,1d,2a,2c,2e]. Stability requires checking a,c,e, not just bare keys.

Placement invariant, stability, and costs

For key v, set L=P[v−1], or zero at v=0, and R=P[v]. After processing a suffix, its v-records occupy the right end of [L+1,R]. C[v] points to that block's rightmost unfilled position. The next backward-scanned v fills C[v], then decrements it, without duplication or leaving the block. Finally every block contains exactly its frequency, with blocks ordered by key, proving correctness.

If x precedes y with the same key, backward scanning puts y farther right, then x farther left, preserving their order. Forward scanning with the same decrementing endpoints reverses equal identities. Forward scanning can instead be stable with incrementing **starting** positions; do not mix these specifications.

Time and extra space are $\Theta(n+k+1)$, including output B, so this is not in-place. Negative keys can shift by v−min, but range k=max−min may remain huge. Reading whole keys and indexing are unit-cost word-RAM assumptions.

Saved P answers a closed range $0 \le a \le b \le k$ by P[b]−P[a−1], with P[−1]=0. For [1,2], the example gives P[2]−P[0]=5−1=4 records. Queries cost $O(1)$ after $\Theta(n+k+1)$ preprocessing.

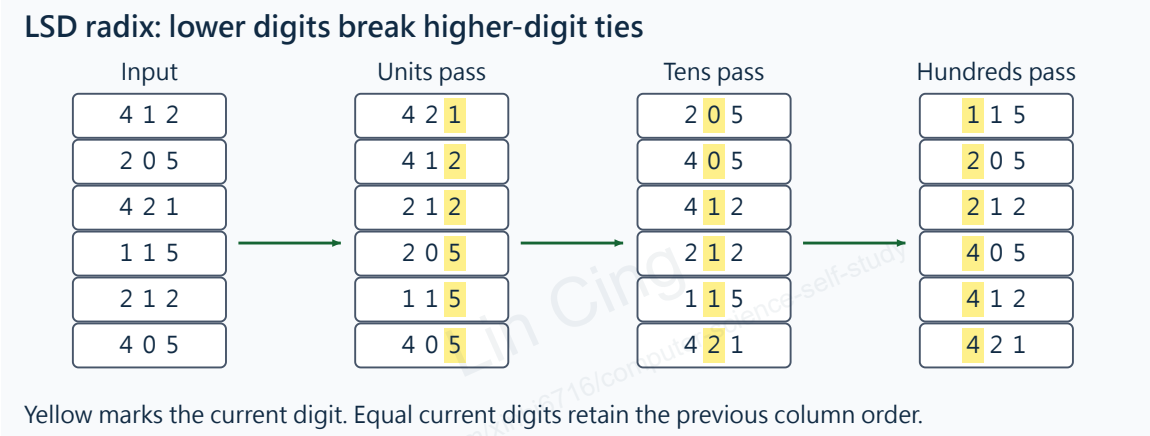
8.3 Radix sort

For fixed d-digit nonnegative integers with each digit 0..r−1, LSD radix sort processes digits from least to most significant, using a stable sort each pass.

After t passes the sequence is sorted by its lowest t digits. The t=0 condition is empty. Pass t groups by that digit; different groups have the correct order, while stability within each group preserves the previous lower-digit order. This proves the invariant and complete sorting at t=d.

Lower-digit order is retained **within a common higher-digit group**, not globally. Unstable digit sorting breaks the induction: [12,11] becomes [11,12] after units, but a tie reversal on tens yields the incorrect [12,11].

Three complete radix passes



Yellow marks the current digit. Equal current digits retain the previous column order.

Three complete radix passes (continued)

Stage	Entire sequence
Input	[412,205,421,115,212,405]
Units	[421,412,212,205,115,405]
Tens	[205,405,412,212,115,421]
Hundreds	[115,205,212,405,412,421]

In the tens pass, 205,405 both have digit 0 and retain that order. Records 412,212,115 all have tens digit 1 and retain their previous order. In the hundreds pass, 205 precedes 212 because the lower pair 05<12 is already ordered.

Sorting the entire array hundreds→tens→units, even stably, can make units the final primary key. Correct MSD sorting instead recurses within equal higher-digit groups; it is a different algorithm.

Bit length, radix, and the cost model

Stable counting costs $\Theta(n+r)$ per digit, giving $\Theta(d(n+r))$ time and $\Theta(n+r)$ extra space. Constant d and $r=O(n)$ directly give $\Theta(n)$. Long keys requiring nonconstant digit extraction need additional access costs.

For b-bit keys with s bits per pass, $1 \le s \le b$, $r=2^s$ and $d=\lceil b/s \rceil$. Cost is $\Theta(\lceil b/s \rceil (n+2^s))$. Zero-pad a shorter highest digit. For $n \ge 2$ choose $s=\min(b,\lfloor \log_2 n \rfloor)$, so $2^s \le n$ and time is $O(n \lceil b/\min(b,\lfloor \log_2 n \rfloor) \rceil)$. If $b=O(\log n)$, this is $O(n)$.

For 32-bit keys in the simplified equal-unit model, s=8 costs 4(n+256), while s=16 costs 2(n+65536). The latter wins iff n>65024, with equality at 65024. These are modeled coefficients, not a measured crossover; cache, allocation, and bandwidth can change implementation choices.

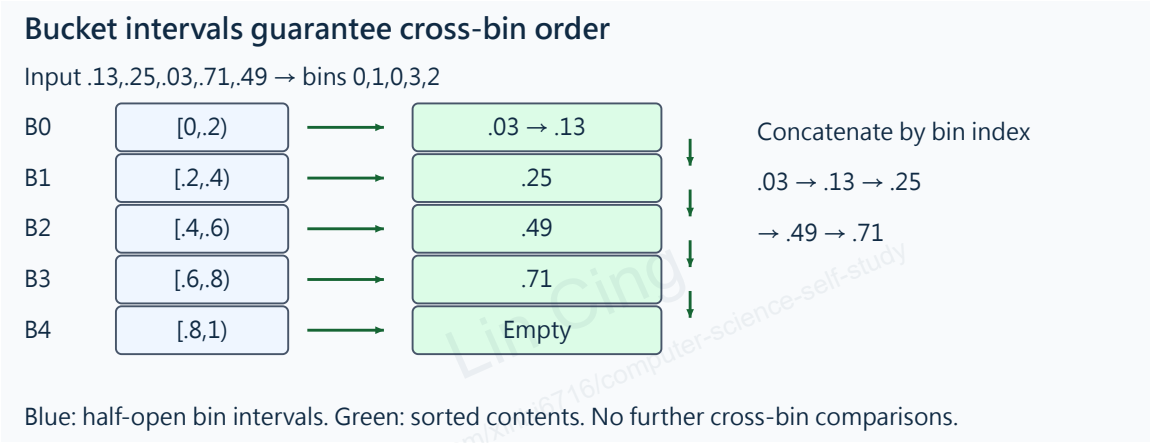
8.4 Bucket sort

For $n \geq 1$ real values x in $[0,1)$, create n bins $B[0..n-1]$, append x to $B[\text{floor}(nx)]$, insertion-sort each bin, and concatenate in index order. Empty input returns immediately. The value 1 is excluded; it would address nonexistent $B[n]$.

Correctness needs no distribution: $x \leq y$ implies $\text{floor}(nx) \leq \text{floor}(ny)$. Distinct bins are already ordered by their intervals, and sorting within bins completes the order. Stability additionally requires input-order append and stable insertion sort; head insertion would first reverse equal records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4 Bucket sort (continued)



For [.13,.25,.03,.71,.49], indices are [0,1,0,3,2], not [0,0,0,1,2]. Sorted bins are B_0=[.03,.13],B_1=[.25],B_2=[.49],B_3=[.71],B_4 empty, giving [.03,.13,.25,.49,.71].

Linear expectation requires squared bin sizes

Let N_i be bin size. Initialization, distribution, and concatenation cost $\Theta(n)$; internal insertion sorts total $O(\Sigma N_i^2)$. Thus $T=O(n+\Sigma N_i^2)$. Knowing $E[N_i]=1$ does not make $E[N_i^2]=1$.

Assume independent uniform inputs in $[0,1)$. Let X_{ij} indicate record j in bin i. Then $N_i=\Sigma_j X_{ij}$, $X_{ij}^2=X_{ij}$, and $E[X_{ij}]=1/n$. Independence of distinct records gives $E[X_{ij}X_{ik}]=1/n^2$ for $j\neq k$. Consequently

$$E[N_i^2] = \sum_j 1/n + \sum_{j\neq k} 1/n^2 = 2 - 1/n.$$

The sum has expectation $2n-1$, giving $O(n)$; reading n records gives $\Omega(n)$, hence $\Theta(n)$. Different bin sizes are dependent because their sum is n. Their independence is unnecessary for linearity.

Distribution, dependence, and worst cases

For independent records with bin probabilities p_i summing to 1, the same expansion gives

$$E[\sum_i N_i^2] = n + n(n-1)\sum_i p_i^2.$$

Uniform probabilities give sum of squares $1/n$. More generally $O(1/n)$ suffices for linear expected work. If one bin has probability 1, squared size is n^2. Concentration increases pair-collision probability Σp_i^2.

Uniform marginals alone fail: choose J uniformly from $0..n-1$ and set every record to $(J+1/2)/n$. Each record has uniform bin marginals and $E[N_i]=1$, but every execution has $\Sigma N_i^2=n^2$. This disproves the squared-work inference, not a claim that equal keys make stable insertion sort quadratic.

An actual worst-case family is $x_j = (n-j+1)/(2n^2)$,$j=1..n$,$n\geq 2$. All values occupy the first bin in strictly decreasing order, forcing $n(n-1)/2$ shifts and $\Theta(n^2)$ time. Replacing internal sorting with worst-case $O(m \log(m+1))$ yields total worst-case $O(n \log n)$ while preserving linear expectation under independent uniform inputs, since $m \log(m+1)=O(m^2)$.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary data structures

Part III Data Structures

Scope

The difficult part of elementary data structures is often the representation and the operation's preconditions. Physical array order is not queue order, a node handle is not a key, and a free slot is not necessarily cleared. Concrete before-and-after states establish the link invariants and costs of queues, lists, allocators, and tree representations.

Prerequisites

Arrays, pointers, abstract data types, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Represent stacks and queues with arrays or links.
- Maintain linked-list invariants and boundary conditions.
- Represent objects with index links when native pointers are unavailable.
- 10.1 Stacks and queues
- 10.2 Linked lists
- 10.3 Implementing pointers and objects
- 10.4 Representing rooted trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stacks and queues

An abstract data type (ADT) specifies operation semantics before choosing a representation. A stack's pop removes the most recently inserted element that remains, giving LIFO order. A queue's dequeue removes the earliest inserted element that remains, giving FIFO order. The rule determines the element to delete; neither operation accepts an arbitrary key or position.

For an array stack of fixed capacity $m \geq 1$, use zero-based indices and let top be the number of live elements. The data occupy $S[0..top-1]$, with $0 \leq top \leq m$.

```
push(x): if top==m: error overflow
         S[top]=x; top++
pop():   if top==0: error underflow
         top--; return S[top]
```

Checking before push prevents an out-of-bounds write; checking before pop prevents top from becoming −1. By induction, each operation extends or shortens the live prefix, whose last value is the latest push, so LIFO holds. Old bits may remain after pop, but they are outside the logical stack. Clearing references is a separate memory-management concern.

Circular-queue representation invariant

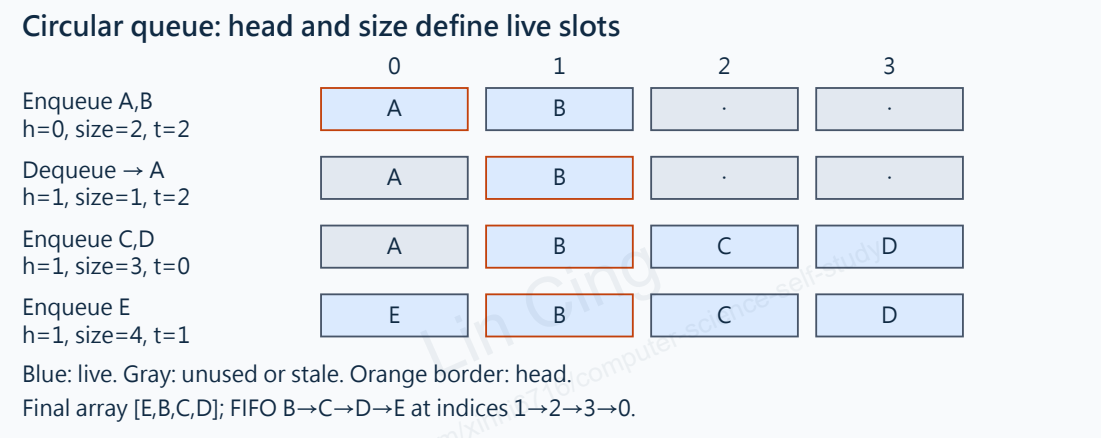
Use $Q[0..m-1]$, head, and size. Define tail as the next write position, $tail=(head+size) \bmod m$. Logical element j is at $Q[(head+j) \bmod m]$, for $0 \leq j < size$.

```
enqueue(x): if size==m: error overflow
            Q[(head+size) mod m]=x; size++
dequeue():  if size==0: error underflow
            x=Q[head]; head=(head+1) mod m; size--; return x
```

Enqueue appends to the old logical sequence. Dequeue returns its old element 0 and advances head once. Reduction modulo m handles wraparound, preserving FIFO order and the indexing invariant. Both size=0 and size=m may have head=tail; size distinguishes them.

The textbook also presents a version that reserves one slot. With only head and tail, empty means head=tail and full means $(tail+1) \bmod m=head$, so usable capacity is $m-1$. Our size-based version holds m elements. Do not mix their fullness tests.

Wraparound preserves FIFO order



At capacity 4, enqueue A,B; dequeue returns A; enqueue C,D; then enqueue E. The final physical array is [E,B,C,D], with head=1, size=4, tail=1. Logical order is B→C→D→E at indices 1→2→3→0. Another enqueue overflows; the next dequeue returns B.

Wraparound preserves FIFO order (continued)

Fixed-array push, pop, enqueue, and dequeue each take worst-case O(1), assuming an element can be moved in O(1). Creating an m-slot array requires Θ(m) space and the corresponding allocation. Automatic growth may move Θ(m) elements in one operation; Chapter 17's amortized analysis is then needed instead of the fixed-capacity per-operation bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linked lists

Array indices determine adjacency in an array; next links determine logical order in a linked list. Adjacent nodes need not occupy adjacent memory addresses. Singly linked nodes store next; doubly linked nodes also store prev. Searching an unsorted list may inspect all n records, taking Θ(n). A local splice with a valid node handle takes O(1).

Without a sentinel, the first node has prev=NIL and the last has next=NIL; deletion must update head or tail separately. A circular doubly linked list has a real dummy sentinel N. N.next points to the first node, N.prev to the last, and both ends link back to N. An empty list has N.next=N.prev=N.

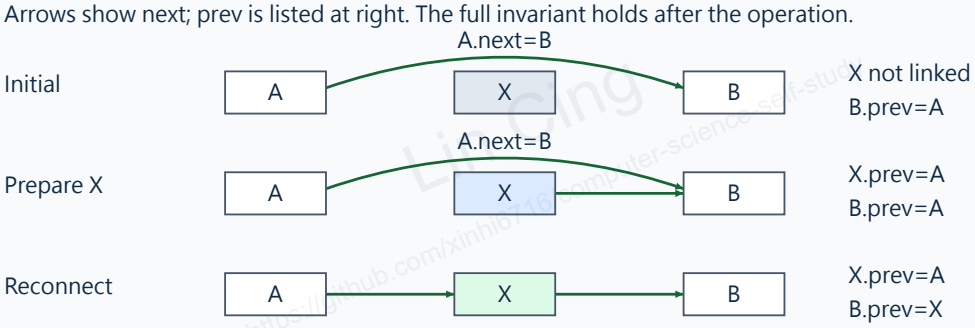
The representation invariant says that following next from N back to N visits every data node exactly once, and every edge u.next=v has v.prev=u. N represents no data key. Search fails on reaching N, and delete must never remove N.

Save the old neighbor before inserting

To insert a new X, currently in no list, between adjacent A and B, first save B=A.next. Write X.prev=A and X.next=B, then A.next=X and B.prev=X. Read an old neighbor before overwriting the link used to find it.

Insertion: save B, then prepare both links of X

Arrows show next; prev is listed at right. The full invariant holds after the operation.



Gray X: uninitialized. Blue: links prepared. Green: reachable from A. A/B may be sentinel.

Save the old neighbor before inserting (continued)

Intermediate writes need not preserve the full doubly linked invariant. This specification assumes a single thread and restores the invariant at the end of the operation. Concurrent readers or exceptions during an operation require additional synchronization or transaction rules; these four writes are not atomic.

The final links are A.next=X, X.prev=A, X.next=B, and B.prev=X. All other edges are unchanged, so the global cycle and reverse-link consistency remain valid. A or B may be the sentinel. The same code handles insertion at either end and insertion into an empty list, where A=B=N.

Deletion preconditions, boundaries, and cost

For a valid data node x, save a=x.prev and b=x.next, then write a.next=b and b.prev=a. The path a→x→b becomes a→b, with the reverse link updated and every other edge unchanged. With one data node, a=b=N, and the two writes restore N's self-loop.

Deleting A from N↔A↔B↔N leaves N↔B↔N. B remains, so the list is not empty. Setting x.next and x.prev to NIL can help debugging, but it does not invalidate external stale handles or replace correct lifetime rules.

O(1) deletion assumes that the caller supplies a valid handle to a node currently in this list and not already deleted. A key alone requires search first, with worst-case Θ(n). Scanning from the head to validate membership on every call also takes Θ(n); that cost cannot be omitted from a claim about the complete operation.

A singly linked list can delete a known predecessor's successor in O(1). Knowing x alone is generally insufficient, particularly at the tail, where copying a successor is impossible. Copying a successor also changes identity and does not implement arbitrary-node deletion under ordinary handle semantics.

Implementing pointers and objects

Slot indices can serve as handles without native pointers. Parallel arrays key[0..m−1], next, and prev use the same index i for one object. Let NIL=−1 so that it cannot be confused with valid slot 0.

For example, an active list may be slot0→slot2 with key[0]=10, key[2]=30, next[0]=2, and prev[2]=0. The logical successor is next[0]=2, not physically adjacent slot1.

In a single-array layout, a fixed three-field object has base=3i, key=A[base], next=A[base+1], and prev=A[base+2]. Specify whether stored pointers are slot indices i or base addresses. Here fields still contain slot indices, so accessing the next object requires multiplication by 3 again. A representation storing base addresses directly must not multiply again.

The two layouts preserve the same list semantics. Variable-length objects do not fit the simple 3i mapping. They require additional block-allocation and length information, so the following fixed-slot free list cannot be applied unchanged.

A free list is a stack of unallocated slots

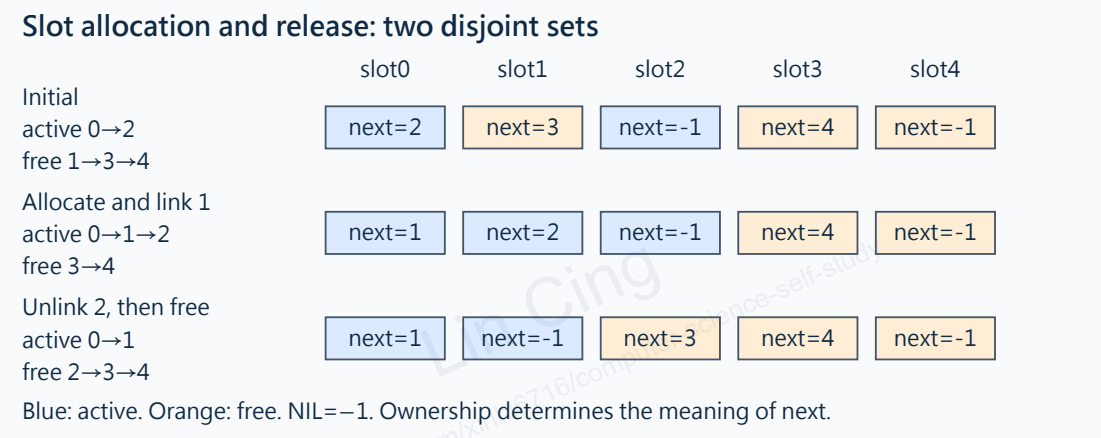
free points to the first unallocated slot, or NIL when none remains. A free slot's next field is reused as its free-list link. Allocation removes the first slot; release prepends a slot that has already been unlinked.

```
allocate(): if free==NIL: error out-of-space
            x=free; free=next[x]; return x
free-object(x): next[x]=free; free=x
```

Allocated and free slots form disjoint sets whose union is all m slots. The free list has no cycle or duplicate. Allocation moves one slot from free to allocated; the caller initializes key, prev, and next before linking it into a data structure. Before release, x must be removed from every structure holding it and must not already be free. Otherwise an active link may be corrupted or a free-list self-loop created.

The free list reads only next, so a free slot's prev and key need not be zeroed. On reuse, initialize every needed field before reading it. This allocator takes O(1), excluding search, destruction of large satellite data, or removal of all external references.

One allocation, insertion, unlinking, and release trace



Initially active is 0→2 and free is 1→3→4. Allocate returns 1 and changes free to 3. Initialize slot1 and insert it between 0 and 2, giving active 0→1→2. Unlink 2 from active before calling free-object(2). The result is active 0→1 and free 2→3→4.

One allocation, insertion, unlinking, and release trace (continued)

If free(2) changes next[2] while active still points to 2, the next traversal enters free slots. Reallocating the slot to another object does not make old handle 2 distinguish the old object from the new one. Do not use a handle after release. Detecting stale handles can use (slot,generation) with version validation, but this adds a representation requirement.

Compacting objects can improve locality but changes index handles. Movement is valid only if all references are updated or a stable indirection table resolves locations. Identical object contents do not make links to its old position valid.

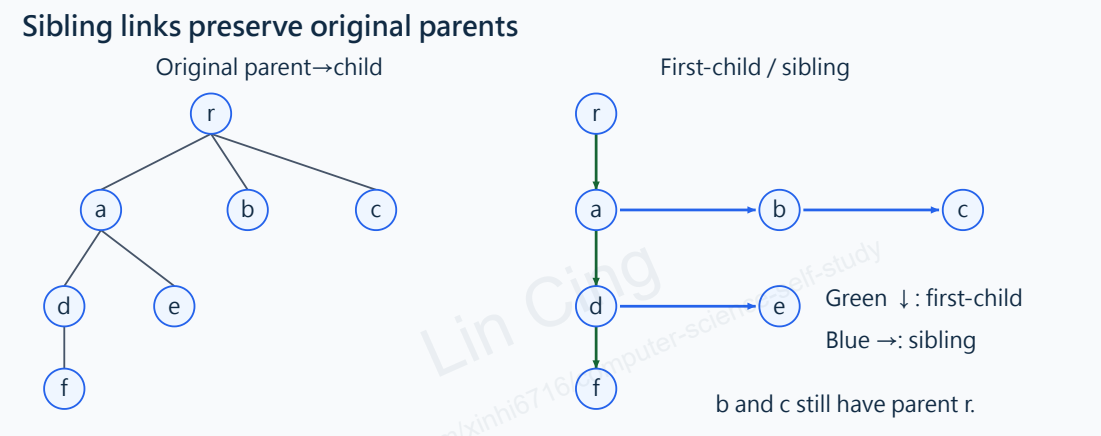
Representing rooted trees

A binary-tree node may store parent, left, and right. The root has parent=NIL, missing children are NIL, and an empty tree has root=NIL. In a general rooted tree, every nonroot still has exactly one parent. The difference is an unrestricted number of children, not permission to have multiple parents.

The first-child/right-sibling representation stores first, sibling, and optionally parent per node. first[x] is x's first child; sibling[x] is the next child of the same parent. The last sibling has sibling=NIL, a leaf has first=NIL, and the root has no sibling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Representing rooted trees (continued)



Here r has a,b,c; a has d,e; and d has f. Thus first[r]=a; sib[a]=b; sib[b]=c; first[a]=d; sib[d]=e; first[d]=f. A sibling edge is not an original parent-child edge. In particular, b's original parent remains r, not a.

Recovering traversal and operation costs

```
visit-tree(x):
    if x==NIL: return
    output key[x]
    c=first[x]
    while c!=NIL:
        visit-tree(c)
        c=sibling[c]
```

Each node is output once. Each parent's loop lists each of its children once, and the whole tree has n−1 children counted across parents. Time is Θ(n), not n nodes multiplied by an independent worst-case n. Stack depth depends on the original tree height h and is O(h+1). A chain needs Θ(n), so O(log n) is not unconditional. This example's preorder is r,a,d,f,e,b,c.

Enumerating all children of x takes Θ(deg(x)+1); a stored parent field gives O(1) ascent. With only parent links, finding all children may require scanning every node. Match representation to operations. A heap uses complete-tree array indices to avoid pointers, whereas a disjoint-set structure can store only parents because it walks toward roots.

Recovering traversal and operation costs (continued)

Mechanically treating first and sibling as binary left and right links also visits the starting node's siblings. That traverses the sibling forest starting at x, which need not equal the original subtree rooted at x. Explicitly enumerating children as above preserves the correct original subtree boundary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash tables

Part III Data Structures

Scope

Hash tables map keys to save space in a sparse universe, but speed depends on collisions and load. Separate arguments establish the costs of chaining, universal families, probe permutations, and two-level perfect hashing. Tombstones, duplicate prevention, and unsuccessful-query traces make the conditions behind expected and worst-case guarantees explicit.

Prerequisites

Probability, modular arithmetic, and average-case analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Derive expected hashing costs from direct addressing.
- Evaluate the assumptions of division, multiplication, and universal hashing.
- Analyze open-address probing and perfect hashing.
- 11.1 Direct-address tables
- 11.2 Hash tables
- 11.3 Hash functions
- 11.4 Open addressing
- 11.5 Perfect hashing

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Direct-address tables

A dictionary maintains a key→record mapping. Here each key has at most one record, and inserting an existing key updates its record. If the universe U={0,…,M−1} is small enough, use T[key] directly. EMPTY marks an unused slot and is distinct from any legal record.

Search reads T[k], insert writes T[k], and delete clears it. Each takes worst-case O(1), with Θ(M) space. Initializing all slots to EMPTY usually takes Θ(M); constant query time does not imply constant construction time. A bit set uses one bit per key but stores no satellite data.

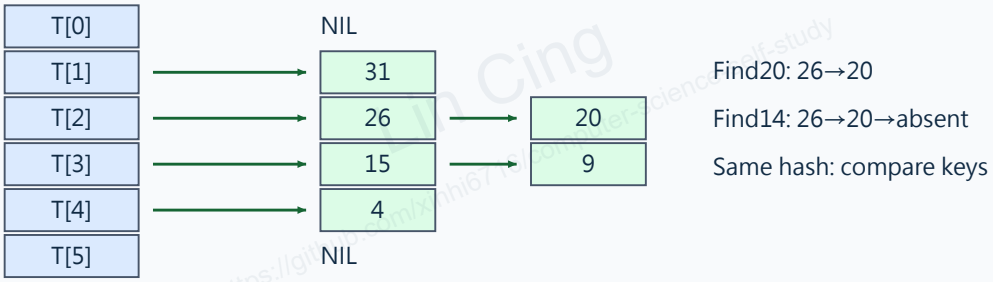
For U=0..9 with keys 2,5,8, only T[2], T[5], and T[8] point to records. The key's numerical value supplies the index, with no collisions. Reading and converting a long string key costs time proportional to its length. Subsequent O(1) hashing and equality claims assume keys handled in a fixed number of word operations.

Chaining

When |U| greatly exceeds the n stored records, use m slots and h:U→{0,…,m−1}. If |U|>m, the pigeonhole principle guarantees a collision between distinct keys for every fixed h. Chaining links records in the same slot. Search must compare original keys: equal hashes do not imply equal keys.

Chaining: buckets and linked records

Insert 4,9,15,20,26,31 at heads; m=6, h(k)=k mod6.



Blue: buckets. Green: records. Absent14 still scans bucket2; a hash does not prove membership.

Chaining (continued)

Let m=6 and h(k)=k mod 6. Insert 4,9,15,20,26,31 in that order at chain heads. Bucket 2 is 26→20, bucket 3 is 15→9, bucket 1 contains 31, and bucket 4 contains 4. Finding 20 checks two records. An unsuccessful search for 14 also scans bucket 2's two records. This specific table is not the expected-cost formula over random h.

Head insertion takes O(1) if the key is already known to be new. Our dictionary's update semantics require search to avoid duplicates, so complete insert is not unconditionally worst-case O(1). Unlinking a valid handle in a doubly linked chain takes O(1); deletion by key still pays for search.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two expected chaining-search costs

Let α=n/m, which may exceed 1. Simple uniform hashing assumes independent uniform slots for distinct fixed keys and O(1) hash evaluation. For an absent query k and each stored x, the indicator I_x=[h(x)=h(k)] has expectation 1/m. Thus the expected scan length is E[ΣI_x]=n/m=α; fixed indexing overhead gives Θ(1+α).

Successful search additionally assumes that every stored record is equally likely to be queried. Fix insertion order $x_1,\ldots,x_n$, use head insertion, and perform no deletions. Searching for x_i checks itself and later keys in the same bucket, so

```
E[successful record checks] = (1/n)Σ_i(1+(n−i)/m) = 1+(n−1)/(2m).
```

This precise expression uses the query distribution and insertion order. It does not follow from equally likely successful queries to each bucket: a long chain contains more keys and is more likely to be selected. For only an O(1+α) upper bound, bound the entire chain containing the key without requiring uniform successful queries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two expected chaining-search costs (continued)

Constant expected cost requires $\alpha=O(1)$. A fixed table may still have a chain of length n and a worst-case $\Theta(n)$ operation. Resizing and choosing a new h require moving and rehashing records; Chapter 17 addresses amortization of dynamic-table costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and multiplication methods

The division method is $h(k)=k \bmod m$. If $m=2^p$ it keeps only the lowest p bits. At m=8, 0 collides with 8 and 6 with 14. A prime modulus sometimes avoids a particular pattern, but this is no uniformity theorem for arbitrary key sets, nor does it remove the pigeonhole limitation.

Horner evaluation hashes a long string incrementally: $H_0=0$ and $H_{i+1}=(B \cdot H_i+c_i) \bmod m$. This computes the complete base-B integer modulo m without constructing a huge integer, in Θ(number of characters). If $B \equiv 1 \bmod m$, all character permutations have the same sum and collide. The formula alone guarantees no useful distribution.

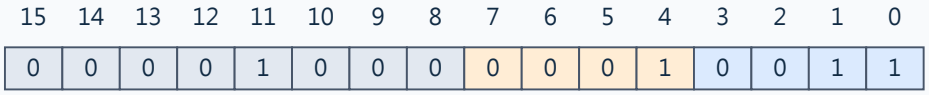
The multiplication method uses $0<A<1$ and $h(k)=\lfloor m \cdot \operatorname{frac}(kA) \rfloor$. With $A=s/2^w$ and $m=2^p$, compute the low w-bit word r_0 of $k \cdot s$, then take its highest p bits. Do not take the highest p bits of the entire 2w-bit product.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and multiplication methods (continued)

Multiplication hashing: high bits of the low word

w=8, s=159, k=13; product2067=8·256+19; m=16 uses p=4 bits.



High word 00001000 = 8 Low word 00010011 = 19

Orange: high four bits of low word, 0001 → h=1.

Equivalently: (2067 mod256) >>4 = 19 >>4 = 1.

Gray: discarded high word. Orange: selected bits. Blue: remaining low bits.

For w=8, s=159, k=13, the product is 2067=8 • 256+19. The low word is 00010011. With m=16, its high four bits are 0001, giving h=1; ⌊16 • 19/256⌋ also equals 1.

Universality is a probability guarantee

A family H is universal if, for every fixed pair of distinct keys x,y, uniformly choosing h from H gives $\Pr[h(x)=h(y)] \leq 1/m$. The key set and queries must be fixed independently of this choice of h. Keys need not themselves be naturally uniform.

For a fixed query k, sum collision-pair indicators. If k is absent, $E[\text{chain length}] \leq n/m$. If present, the bound is $1+(n-1)/m$. This needs only pairwise collision bounds, neither independent collision events nor uniform marginal output at every slot.

After initializing m slots, a fixed sequence of operations with at most O(m) simultaneous records gives expected O(1) per search and duplicate-checking insert, hence expected O(t) for t operations. Initialization still costs Θ(m) separately. An adversary that sees h or adaptively chooses keys from responses does not automatically satisfy the fixed-pair premise. Universality is also not cryptographic unpredictability; it does not require h to remain permanently secret.

Collision proof for a prime affine family

Choose a prime P larger than every legal key. Choose a uniformly from 1..P−1 and b from 0..P−1, and define h_(a,b)(k)=((ak+b) mod P) mod m.

For fixed x≠y, set r=(ax+b) mod P and s=(ay+b) mod P. Both a and x−y are nonzero in the field F_P, so r−s=a(x−y)≠0. Conversely, each r≠s uniquely determines a=(r−s)(x−y)^(−1) mod P and b=r−ax mod P. Uniform (a,b) therefore corresponds to the P(P−1) uniformly chosen ordered distinct pairs (r,s).

For each r, at most ⌈P/m⌉−1≤(P−1)/m of the other P−1 possible s share its residue modulo m. Collision probability is at most 1/m, proving universality. If m≥P, the final modulo operation merges no distinct r,s and collision probability is zero.

Primality makes every nonzero x−y invertible. The proof does not transfer to an arbitrary composite modulus. O(1) arithmetic assumes P, modular multiplication, and keys fit the specified word-RAM model; arbitrary-precision integer costs must be counted separately.

Open addressing

All records occupy the m slots themselves, so n≤m. Each key has a fixed probe sequence h(k,0),…,h(k,m−1), reproduced by both insertion and search. To find a slot whenever one is empty, the sequence must cover every slot, usually as a permutation.

Slots have three states: EMPTY, never used since construction; DELETED, a tombstone left by deletion; or OCCUPIED(record). Search returns on an equal key, rejects at EMPTY, and continues past DELETED. It makes at most m probes. Even with no EMPTY slot, it must terminate rather than wait forever for an empty slot.

Stopping at EMPTY is correct because a key inserted farther along could not have skipped an EMPTY slot at insertion time. Later deletion changes slots to DELETED, never EMPTY, so it cannot create a new stopping gap along a surviving key's path.

Tombstones and duplicate-free insertion

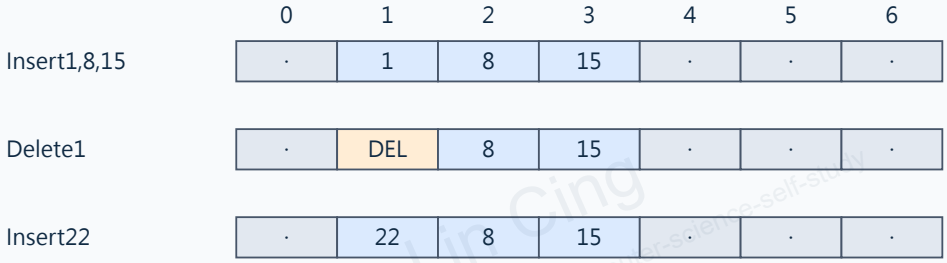
```
insert(k,value):
    first_deleted=NIL
    for i=0..m-1:
        j=h(k,i)
        if T[j] is OCCUPIED and T[j].key==k:
            update value; return j
        if T[j] is DELETED and first_deleted==NIL:
            first_deleted=j
        if T[j] is EMPTY:
            place at first_deleted if set, otherwise j; return
    if first_deleted!=NIL: place there; return
    error full
```

Do not insert immediately at the first DELETED slot: the same key may appear later, creating a duplicate record. The later duplicate search may be omitted only if the caller already guarantees a new key. Delete searches first, marks a found slot DELETED, and changes nothing if absent.

Tombstones and duplicate-free insertion (continued)

Tombstones preserve probe paths and unique keys

m=7, home=k mod7, linear probing. Find15 follows1→2→3 across DEL.



Insert8 updates slot2. Insert22 saves DEL1, probes to EMPTY4, then writes slot1.

Gray EMPTY: stop. Orange DELETED: continue. Blue OCCUPIED: compare key.

Preserving search on the same probe path

Let m=7 and home(k)=k mod 7. Inserting 1,8,15 places them in slots 1,2,3. Deleting 1 makes slot 1 DELETED. Searching for 15 still probes 1→2→3 and returns slot 3; it cannot stop at 1.

insert(8,new-value) remembers reusable slot 1 but continues to slot 2, finds 8, and updates it without creating another 8 at slot 1. Inserting new key 22 probes 1(DELETED), 2(8), 3(15), and 4(EMPTY). Only after confirming absence does it place 22 at the first DELETED slot, slot 1.

Lower active load α=n/m does not guarantee shorter probing. Many tombstones can leave no EMPTY slot even with few live keys, so an unsuccessful search takes m probes. The clean expected bounds below assume no deletions or tombstones. Dynamic workloads must separately limit their fraction or rebuild and account for that cost.

Probe formulas and coverage

Linear probing, $(h_0(k)+i) \bmod m$, always covers the table. Adjacent clusters absorb later keys whose homes lie there, producing primary clustering. Equal homes imply equal sequences.

Quadratic probing, $(h_0(k)+c_1 i+c_2 i^2) \bmod m$, does not cover the table for arbitrary parameters. With m=7, offsets i^2 are 0,1,4,2,2,4,1, so insertion can fail despite empty slots. Equal homes still imply equal sequences, called secondary clustering.

A full-coverage version uses m=2^w and f(i)=i(i+1)/2, dividing as integers before reducing modulo m. If f(i)=f(j) mod 2^w, then 2^(w+1) divides (i−j)(i+j+1). The factors have opposite parity, so the even factor must be divisible by 2^(w+1). For $0 \le i,j < 2^w$, this is possible only with i−j=0. Thus i=j and the sequence is a permutation. At m=8, offsets are 0,1,3,6,2,7,5,4.

Double hashing uses $(h_1(k)+i \cdot h_2(k)) \bmod m$, with period $m/\gcd(h_2(k),m)$. Write $m=gm'$ and $h_2=gd'$, with $\gcd(m',d')=1$. Returning home is equivalent to m' dividing i, so the least positive period is m'. Full coverage requires a coprime step: a prime m with nonzero step, or a power-of-two m with odd step, suffices.

Unsuccessful search under ideal uniform probing

Uniform hashing here means that each key's probe permutation is uniform over all m! permutations, and a fresh query remains an unbiased uniform permutation relative to the current table. This is stronger than uniform single-bucket hashing for chaining. Linear, quadratic, and double hashing cannot simply be assumed to satisfy it.

With no tombstones and n<m, let X count probes through the first EMPTY in an unsuccessful search. The probability that its first i−1 slots are occupied is

$\Pr(X \geq i) = \prod_{j=0}^{i-2} (n-j)/(m-j) \leq (n/m)^{i-1} = \alpha^{i-1}$ for $i \leq n+1$; later tails are zero.

For a nonnegative integer X, $X = \sum_{i \geq 1} I[X \geq i]$. Taking expectations gives $E[X] = \sum \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = 1/(1-\alpha)$.

The product samples without replacement; it does not assume independent probes. Insertion of a new key through the first EMPTY has the same cost. For m=10,n=8 the bound is 5, not exactly five probes per query and not a guarantee for actual linear probing.

Successful search averages insertion loads

Fix an insertion sequence of n distinct keys with no deletion, and query a stored key uniformly. At insertion of key i+1, only i records are present, so expected probes are at most m/(m−i). Later operations neither move this key nor delete the occupied slots on its path. Search reproduces its insertion path.

$$E[\text{successful probes}] \leq (1/n) \sum_{i=0}^{n-1} m/(m-i) = (1/\alpha) \sum_{k=m-n+1}^{m} 1/k$$

$$\leq (1/\alpha) \int_{m-n}^{m} dx/x = (1/\alpha) \ln(1/(1-\alpha)).$$

Because 1/x decreases, its integral on [k−1,k] is at least 1/k. At α=0 there is no successful query; do not substitute zero into 1/α. The bound tends to 1 as α→0. Both expected bounds diverge as α→1, motivating spare capacity. The threshold depends on the implementation and cost model.

Perfect hashing

A static dictionary's key set remains fixed after construction. The FKS two-level scheme gives worst-case constant memory accesses per query after construction, with O(n) space. Random retries occur during construction, whose time is not worst-case O(n).

For the empty set, return absent without modulo zero. Assume $n \geq 1$. The first level has $m=n$ buckets and a universal random h; bucket j has n_j keys. If Y counts unordered colliding key pairs, $\Sigma n_j^2 = n+2Y$. Each pair collides with probability at most $1/n$, so

$$E[\Sigma n_j^2] \leq n+2 \cdot \binom{n}{2}/n = 2n-1.$$

Markov gives $\Pr(\Sigma n_j^2 \geq 4n) < 1/2$. Retry until $\Sigma n_j^2 < 4n$, requiring fewer than two trials in expectation, each taking $O(n)$ to distribute keys. Expected linear space alone does not establish a definite linear bound for an accepted table; actually check the acceptance condition.

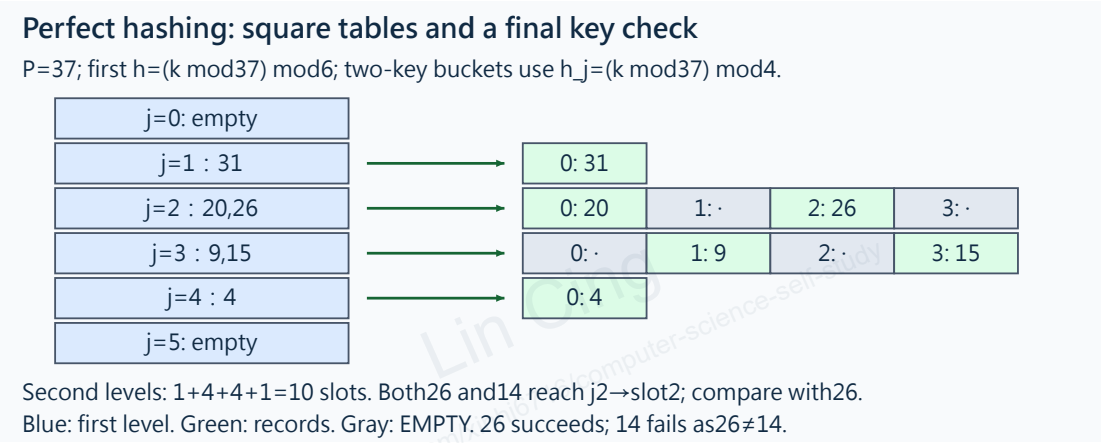
Why square-size second levels succeed quickly

For a bucket of $s=n_j \geq 2$ keys, allocate s^2 slots and independently choose a universal h_j. Its collision-pair count Z satisfies $E[Z] \leq \binom{s}{2}/s^2 = (s-1)/(2s) < 1/2$. Thus $\Pr(Z \geq 1) \leq E[Z] < 1/2$. Accept only after checking zero collisions; fewer than two trials are needed in expectation.

If each attempt allocates or clears s^2 slots, it costs $O(s^2)$. The accepted first level has $\Sigma s^2 < 4n$, so all second-level construction costs total expected $O(n)$, still $O(n)$ including the first level. Account separately for first-level retries and second-level retries conditional on the accepted buckets. An average for one bucket does not replace the total-cost argument.

Empty buckets need no second level; singleton buckets can store the record directly, without modulo zero. Lookup computes $j=h(k)$, then $h_j(k)$ if nonempty, reads the slot, and compares the full key. An absent query may reach an occupied slot; collision freedom among stored keys does not prove membership.

Actual two-level layout and an absent query



Use K={4,9,15,20,26,31}, P=37, and h(k)=(k mod 37) mod 6. Bucket sizes are [0,1,2,2,1,0], and total square-size space is 0+1+4+4+1+0=10<24. Both two-key buckets use h_j(k)=(k mod 37) mod 4, producing [20,EMPTY,26,EMPTY] and [EMPTY,9,EMPTY,15], with no collisions.

Actual two-level layout and an absent query (continued)

Query 26 follows first-level 2→second-level 2→record 26 and succeeds by equality. Query 14 follows the same 2→2 but reads 26≠14 and must reject. Each fixed query takes a constant number of word operations and memory accesses, a worst-case guarantee after successful construction. Adding keys or changing hash parameters may break collision freedom and requires rebuilding or verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 11.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search trees

Part III Data Structures

Scope

A BST turns global key order into a root-to-leaf query path, but an ordinary BST does not balance itself. The same ten-key tree aligns search, successor, and every parent update in two-child deletion. A direct height-moment proof for random insertion distinguishes complete traversal, average depth, and height.

Prerequisites

Recursion, linked nodes, and random permutations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Build and query binary search trees.
- Implement insertion and all three deletion cases.
- Analyze the expected height of a randomly built tree.
- 12.1 What is a binary search tree?
- 12.2 Querying a binary search tree
- 12.3 Insertion and deletion
- 12.4 Randomly built binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The global search-tree invariant

Each node stores key, left, right, and parent. The main algorithms use distinct keys: every key in x's left subtree is less than x.key, and every key in its right subtree is greater. The root has parent=NIL, missing children are NIL, and an empty tree has root=NIL. The structure must be connected and acyclic, with exactly one parent per nonroot node.

The condition constrains entire subtrees, not just direct children. A root 5 with left child 3 and 3's right child 6 satisfies 3<5 and 6>3, but 6 violates the root's left-subtree bound. Validate by recursively passing an allowed interval (lo,hi), setting hi=x.key on the left and lo=x.key on the right. Checking every node takes Θ(n), without rescanning subtrees at each node.

Duplicates can use a count per distinct key or a total order on (key,identity). With the latter, adjacent identities can have the same key, so successor no longer means a strictly larger key. All numerical cases here use distinct keys, as does the random-height analysis.

Sorted inorder and complete traversal cost

```
inorder(x):
    if x==NIL: return
    inorder(x.left)
    output x.key
    inorder(x.right)
```

Inductively, each subtree's inorder is sorted, and every left key<x.key<every right key. Concatenating left, root, right is sorted. Conversely, a valid binary tree with distinct keys and strictly increasing inorder satisfies the global BST condition, since all left keys precede the root and all right keys follow it.

Each of n real nodes calls both children once, giving exactly 2n+1 calls including NIL. Constant work per call gives Θ(n+1), conventionally Θ(n) for a nonempty tree, not O(h). Producing n outputs also gives Ω(n). Recursion depth is h+1 and can be Θ(n) on a chain. We use edge height throughout: a singleton has h=0 and an empty tree h=−1. Nonempty path operations take O(h+1); empty-tree search and first insertion take O(1).

Sorted inorder and complete traversal cost (continued)

A heap orders parents against children but does not place all left keys below the root and all right keys above it. Its inorder need not be sorted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

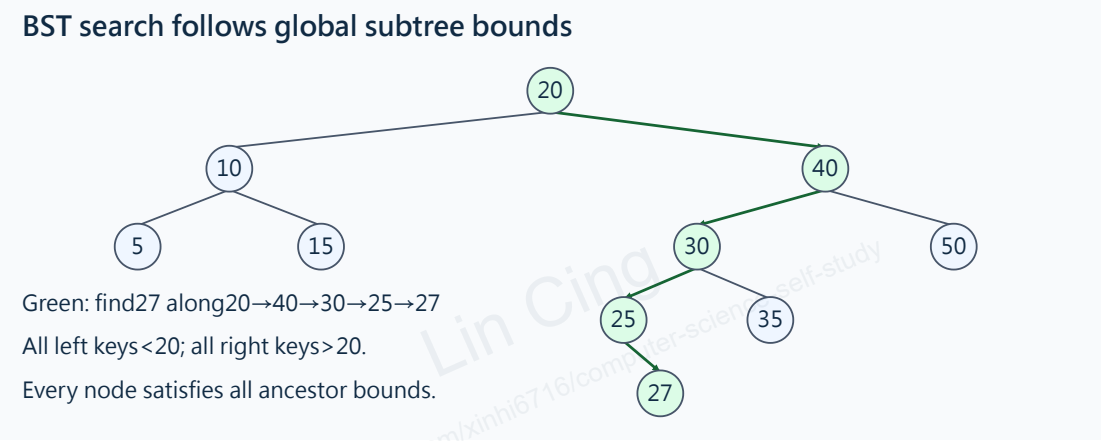
Querying a binary search tree

Search compares at the root: return on equality, go left for a smaller query and right for a larger one, and reject at NIL. Its loop invariant is that any existing answer lies in the current subtree. Global BST bounds justify discarding the other side. Each step descends one level, giving O(h+1) time and O(1) iterative extra space.

Minimum follows left links in a nonempty subtree until none remains. A left child supplies smaller keys; without one, the root is no greater than anything in its right subtree. Maximum reverses left and right.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Querying a binary search tree (continued)



search27 follows 20→40→30→25→27. search26 reaches 27, goes left to NIL, and fails. Minimum is 5 and maximum is 50. A query follows a path; complete inorder is 5,10,15,20,25,27,30,35,40,50. Their costs differ.

The two successor branches

If x.right is nonempty, successor is min(x.right): all right-subtree keys exceed x, and its minimum precedes every larger candidate there. Otherwise climb parents, skipping edges where the current node is a right child. The first parent entered from its left child is the answer; NIL means none exists.

```
successor(x):
    if x.right!=NIL: return minimum(x.right)
    y=x.parent
    while y!=NIL and x==y.right:
        x=y; y=y.parent
    return y
```

Skipped parents are smaller than the original x and cannot be successors. At the first left ascent, y has original x in its left subtree and is larger. No unexcluded key after x remains in y's left subtree: x had no right child, and the intervening ascents followed final right branches. Thus y is the smallest larger key. Predecessor exchanges left and right.

The two successor branches (continued)

successor20 follows the left chain 40→30→25 and returns 25. successor27 skips 25 on a right ascent, then enters 30 from the left and returns 30. successor50 skips 40 and 20 and reaches NIL. No new root search or sorting is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Successive successors share traversal work

Starting at minimum, repeated successor lists all n keys in $\Theta(n)$ total time. Over the complete inorder, each child edge is traversed downward at most once and upward at most once. Add n outputs for $O(n)$. One $\Theta(h)$ successor does not imply paying that path again at every node.

For k successive calls from any starting node, stopping at NIL, count upward steps U and downward steps D. Each call has at most one ascent from a left child to its parent, which is that call's answer, for at most k such edges. Every other ascent is from a right child. Its parent was output before entering its right subtree; after climbing above it, traversal never returns to that subtree, so each such parent is charged once.

Previously output parents from before these k answers must lie on the starting node's ancestor chain, at most h+1 nodes; other charged parents number at most k. Hence $U \leq 2k+h+O(1)$. Downward steps increase depth and upward steps decrease it. $D-U$ is final depth minus initial depth, at most $h+O(1)$, giving $D \leq U+h+O(1)$. Total path work plus k fixed call costs is $O(k+h+1)$, without assuming constant work per ascent or descent.

Successive successors share traversal work (continued)

This refines, but does not change, the single-call O(h+1) bound and explains why complete traversal is not O(nh). Reimplementing each successor as a new root search is a different algorithm and cannot use this shared-edge accounting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion

Follow the search path to its final NIL, retaining parent p. Initialize z.left=z.right=NIL and z.parent=p, then attach z to p.left or p.right according to comparison. If p=NIL, the tree was empty and root=z.

The path's accumulated interval (lo,hi) is exactly the allowed range for z.key. Reaching NIL means no equal key exists. Adding the leaf leaves other keys unchanged and satisfies every ancestor bound, preserving the BST invariant. On equality, update the existing dictionary record rather than adding a duplicate.

Inserting 26 into our tree follows 20→40→30→25→27 and reaches 27.left=NIL. Set 27.left=z26 and z26.parent=27. It must satisfy 25<26<27. Merely knowing 26>25 does not justify overwriting 25's right subtree rooted at 27.

Insertion takes worst-case O(h+1). Inserting 1,2,…,n into an empty tree creates a right chain with $0+1+\dots+(n-1)=\Theta(n^2)$ comparisons. A later O(n) inorder does not make this a O(n log n) sorting algorithm.

Transplant replaces an incoming link

```
transplant(T,u,v):
    p=u.parent
    if p==NIL: T.root=v
    elif u==p.left: p.left=v
    else: p.right=v
    if v!=NIL: v.parent=p
```

Transplant gives v u's former position under its parent in O(1). It does not set v.left/v.right or dispose of u's satellite data. The caller must prove that v's subtree fits the incoming link's global key interval; an arbitrary subtree cannot be transplanted safely.

If z has no left child, delete with transplant(z,z.right), allowing right=NIL and covering leaves and right-only nodes. If z has no right child, use transplant(z,z.left). The surviving subtree already satisfies z's ancestor bounds, so promotion preserves order and updates parent links. The same helper handles root deletion.

Complete pointer specification for two-child deletion

```
delete(T,z):
    if z.left==NIL: transplant(T,z,z.right)
    elif z.right==NIL: transplant(T,z,z.left)
    else:
        y=minimum(z.right)
        if y.parent!=z:
            transplant(T,y,y.right)
            y.right=z.right
            y.right.parent=y
        transplant(T,z,y)
        y.left=z.left
        y.left.parent=y
```

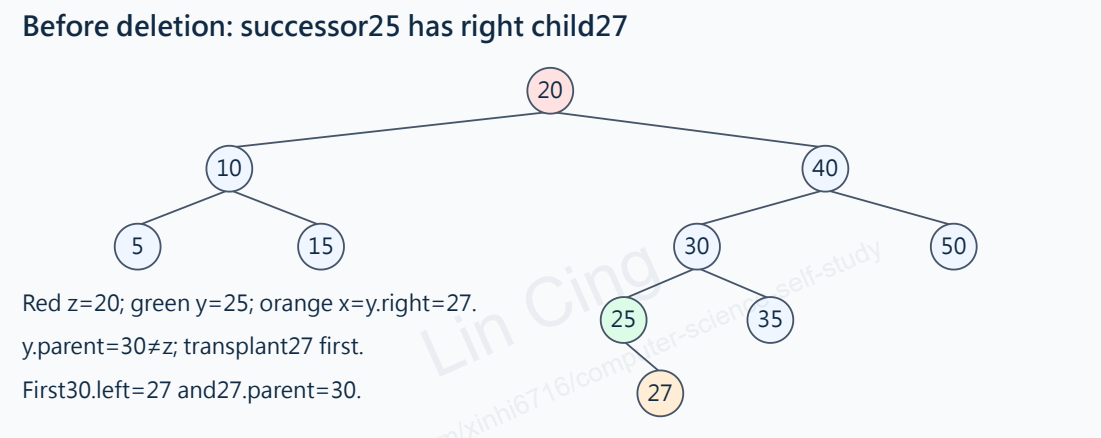
As the right-subtree minimum, y has no left child, but may have a right child that must survive. If y is not z.right, promote y.right to its old position, then give y the original right subtree. If y is z.right, skip that branch: y.right=z.right would create a self-cycle, and y's existing right child must remain intact.

Complete pointer specification for two-child deletion (continued)

Finally y replaces z under its old parent, or becomes root, and receives the original left subtree. Every new left key<z<y. The new right subtree is the old right subtree minus y, with all keys>y. Since y belonged to z's subtree, it also satisfies all of z's ancestor bounds. Thus global BST order, reachable node identities, and parent directions are correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserve a successor's right child



Here z=20 and y=25 is 30.left, not direct right child40. y.right=27. Apply these updates in order.

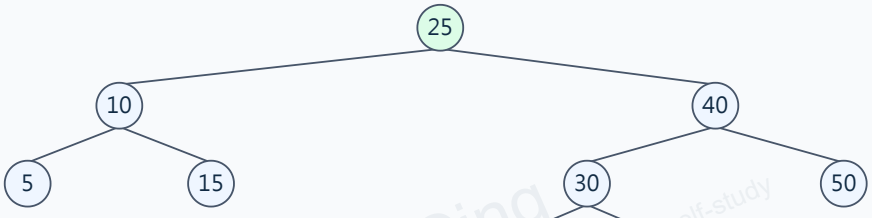
Preserve a successor's right child (continued)

Stage	Links actually changed
Replace y's old position	30.left=27, 27.parent=30
Attach original right subtree	25.right=40, 40.parent=25
Replace original root	root=25, 25.parent=NIL
Attach original left subtree	25.left=10, 10.parent=25

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserve a successor's right child (continued)

After deletion: reconnect both positions



Green25 becomes root; parent=NIL.

Orange27 becomes30.left; parent=30.

Only20 is removed; all nine other identities remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Only node20 is removed. All other nine identities remain reachable, with inorder 5,10,15,25,27,30,35,40,50. This moves node25; it does not mix in the different handle semantics of copying a key and deleting another node.

Handles and deletion cost

z must be a currently valid node in this tree. After deletion, external callers may not treat z as a member. Other node identities remain, including y25, whose handle still denotes the same record at a new position. Copying y's key/value into z and deleting y gives external handles different observations and needs a separate API specification.

Deletion has one O(h+1) minimum path and only a fixed number of transplants and pointer writes, so takes O(h+1). Starting from a key also requires search, of the same order, not O(1). Search, insertion, and deletion depend on height; ordinary BSTs have no automatic balancing guarantee.

A fixed sorted array can be converted into a balanced tree by recursively choosing midpoints in Θ(n). For arbitrary unsorted comparison inputs, constructing a complete BST and reading inorder would solve sorting, giving a general worst-case construction lower bound Ω(n log n). These are different input contracts.

Randomly built binary search trees

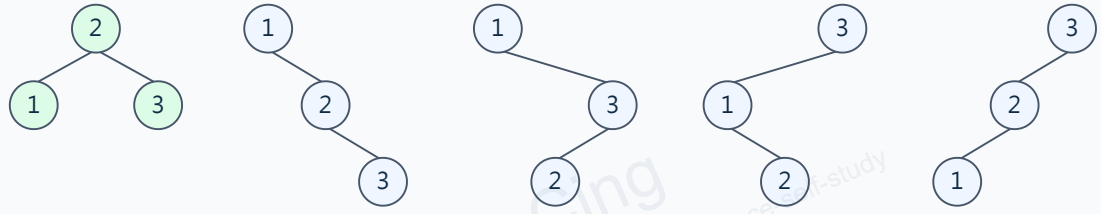
Fix n distinct keys, start empty, choose each of n! insertion permutations equally, and perform insertions only. This differs from assigning equal probability to every valid BST shape. The first key becomes root, so root rank R is uniform on 1..n.

Conditioned on R=r, the left side contains r−1 smaller keys and the right n−r larger keys. Each specified pair of internal relative orders has binom(n−1,r−1) interleavings. Thus, conditional on the root, both relative insertion orders are uniform and mutually independent.

Randomly built binary search trees (continued)

Uniform permutations weight tree shapes

Six permutations are uniform; five shapes are not. Singleton edge height0.



213 / 231	123	132	312	321
2/6	1/6	1/6	1/6	1/6

Green height1; others height2. E[H3]=(2×1+4×2)/6=5/3.

For n=3, [2,1,3] and [2,3,1] yield the balanced shape rooted at2, with probability 2/6. Each of four chain shapes has probability 1/6. Heights are 1 or 2, so $E[H_3]=(2\times1+4\times2)/6=5/3$, not 9/5 from uniform shapes.

Control exponential height directly

O(log n) expected depth of one node does not directly imply O(log n) expected maximum depth. Instead bound an exponential of height.

Let H_n be edge height, $H_0=-1$, and Z_n=2^(H_n). Then $Z_0=1/2$ and $Z_1=1$. Unlike the textbook's convenient separate convention $Z_0=0$, we keep Z=2^H valid even for the empty tree and use correspondingly looser constants.

Conditioned on left-subtree size i, H_n=1+max(H_i,H_(n−1−i)), hence `Z_n=2 max(Z_i,Z_(n−1−i))≤2(Z_i+Z_(n−1−i))`.

Write a_n=E[Z_n]. Conditional subtrees are uniform for their sizes, so averaging over i gives

`a_n ≤ (2/n)Σ_(i=0)^(n−1)(a_i+a_(n−1−i)) = (4/n)Σ_(i=0)^(n−1)a_i.`

This uses max≤sum followed by linearity, not the false equality E[max(X,Y)]=max(E[X],E[Y]).

A polynomial moment bound gives logarithmic height

Set b_n=(1/2)binom(n+3,3), with $b_0=1/2=a_0$. The hockey-stick identity $\Sigma_{i=0}^{n-1}\binom{i+3}{3}=\binom{n+3}{4}$ yields

$$(4/n)\Sigma_{i=0}^{n-1}b_i = (2/n)\binom{n+3}{4} = (1/2)\binom{n+3}{3}=b_n.$$

The identity telescopes Pascal's relation $\binom{i+4}{4}-\binom{i+3}{4}=\binom{i+3}{3}$. Induction from $a_i\le b_i$ gives $a_n\le b_n=O(n^3)$, including $a_1=1\le b_1=2$.

Since $\log_2$ is concave, Jensen gives $E[H_n]=E[\log_2 Z_n]\le\log_2 E[Z_n]\le\log_2 b_n=O(\log n)$. Conversely, a height-h binary tree holds at most $2^{h+1}-1$ nodes. Every tree therefore has $h\ge\log_2(n+1)-1$, giving $\Omega(\log n)$ as n grows. Together, $E[H_n]=\Theta(\log n)$.

Tail bounds and the scope of balancing guarantees

For $c>3$, nonnegative Z_n and Markov give

$$\Pr(H_n\ge c\log_2 n)=\Pr(Z_n\ge n^c)\le E[Z_n]/n^c\le b_n/n^c=O(n^{3-c}).$$

$c=4$ gives an $O(1/n)$ tail; choose $c\ge k+3$ for failure probability $O(n^{-k})$. This is a high-probability bound under random permutations, not a fixed height$\le 4\log n$ for every ordinary-BST input.

Input 1..n still deterministically creates a height n−1 chain. Deletion followed by insertion need not preserve the random-construction distribution, so an insertion-only theorem does not cover arbitrary dynamic sequences. Worst-case $O(\log n)$ per operation requires an explicitly balanced invariant, such as the next chapter's red-black tree; vaguely random data is no proof.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 12.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black trees

Part III Data Structures

Scope

Black height and local color constraints guarantee logarithmic height. Exact NIL and parent-child wiring supports proofs of the three insertion cases, four deletion cases, and their mirrors. Before-and-after figures use the same keys to check extra black, root cases, and record identity.

Prerequisites

Binary search trees, rotations, and invariant proofs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define a nearly balanced BST using red-black properties.
- Prove that rotations preserve inorder.
- Trace insertion and deletion repair and verify black height.
- 13.1 Properties of red-black trees
- 13.2 Rotations
- 13.3 Insertion
- 13.4 Deletion

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Read the roles and path counts first

Dark nodes are black B; pale red nodes are red R. B/R after each key also identifies its color. An orange outline marks the current repair position; it is not a third node color. Rotation diagrams leave colors unspecified.

Operation	Roles in the figures
Rotation	x: old local root; y: right child; p: old parent of x; α, β, γ: subtrees preserving in-order intervals
Insertion	z: current red node; p: parent; g: grandparent; u: uncle
Deletion	x: deficit position; p: parent; w: sibling; near: nephew toward x; far: nephew away from x

x +1 adds one black only for the proof. Figure path counts **include the starting node and terminal NIL**; $bh(x)$ excludes x. Omitted empty children are black NIL. A NIL near/far nephew is shown explicitly.

Properties of red-black trees

Color augments a BST to bound height

Internal nodes store key, data, left, right, parent, and color. Examples use distinct keys; duplicate insertion returns the existing record. Keeping duplicate keys instead requires a strict total order on (key,unique ID); the BST arguments remain valid.

The five properties are: (1) every node is red or black; (2) the root is black; (3) every external NIL leaf is black; (4) both children of a red node are black; (5) all paths from any node to descendant NIL leaves contain the same number of black nodes. Property5 must hold in every subtree, not merely for two selected whole-tree paths.

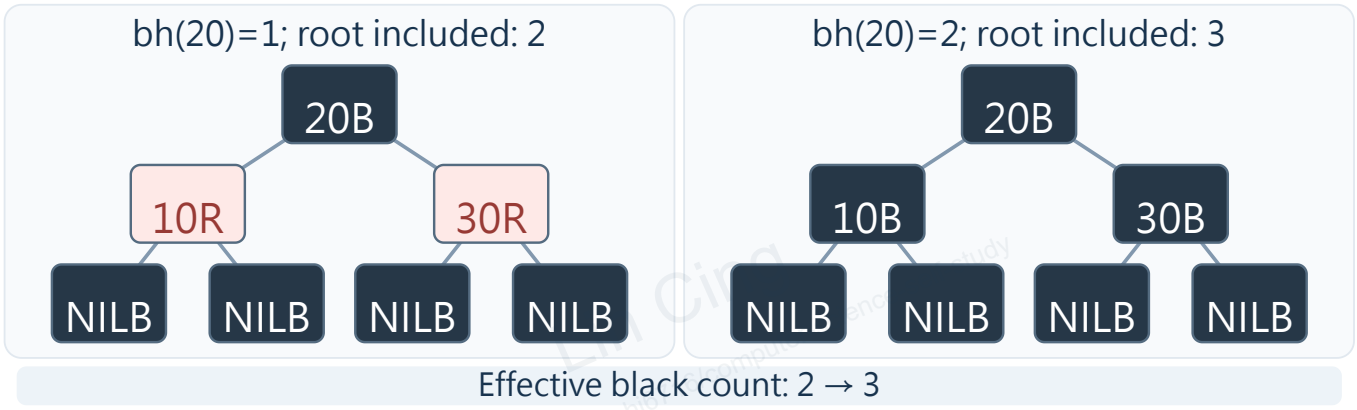
One black sentinel NIL represents all missing children and the root's parent. An empty tree has root=NIL. NIL has no valid key/data; NIL.left=NIL.right=NIL makes color reads safe. Non-NIL child links must point back to their correct parent. All empty links share one object, so they cannot each have an independent NIL.parent.

Black height excludes the start and includes terminal NIL

$bh(x)$ counts black nodes on any path from x to a descendant NIL, **excluding x and including terminal NIL**; $bh(NIL) = 0$. For a red child, $bh(child) = bh(x)$; for a black child, $bh(child) = bh(x) - 1$. The parent's count includes the child only when black.

With root20B and children10R,30R, all other links NIL, bh(20)=1 and bh(10)=bh(30)=1, while bh(NIL)=0. Making both children black gives bh(20)=2 and bh(10)=bh(30)=1. Complete black counts including the root rise from2 to3, but each bh still excludes its own starting node.

Black height excludes the start and includes terminal NIL (continued)



Here h is the longest root-to-NIL edge count, with empty h=0 and singleton black-root h=1. Counting edges only between internal nodes instead gives one less for a nonempty tree. Do not exchange conventions within a formula.

Minimum node count and the full height induction

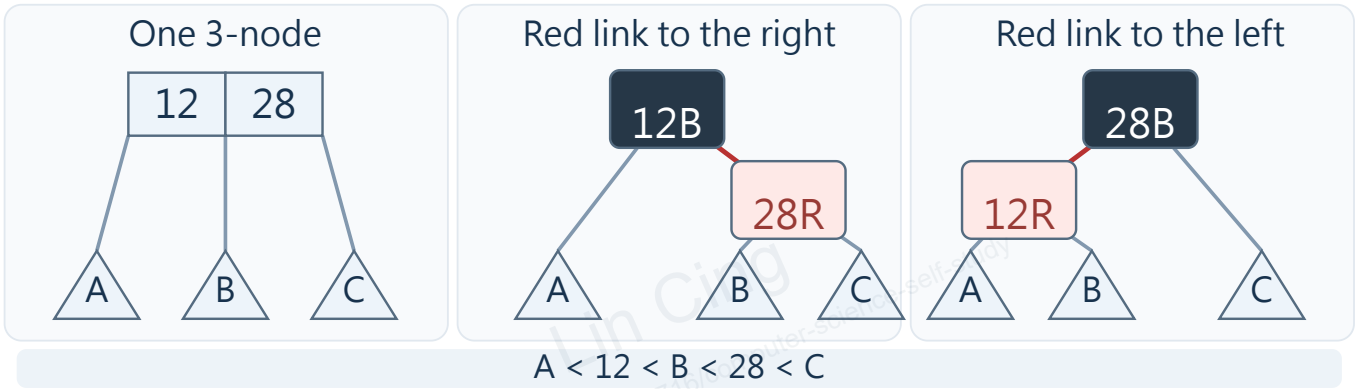
Lemma: a subtree rooted at x has at least $2^{bh(x)} - 1$ internal nodes. Induct on height. At NIL, 0=2⁰−1. Both children of an internal x have smaller height and black height at least bh(x)−1. Since bh(x)≥1, each has at least $2^{bh(x)-1} - 1$ internal nodes. Adding x gives

$$n_x \geq 2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1.$$

A longest root-NIL path has B black and R red nodes. Root and NIL are black and reds are nonadjacent. Pair each red with a distinct preceding black, giving R≤B−1. The path has h+1 nodes, so h+1=B+R≤2B−1 and h≤2(B−1)=2bh(root). Combining the lemma gives $n \geq 2^{h/2} - 1$, hence $h \leq 2\log_2(n+1)$. The empty tree also satisfies0≤0.

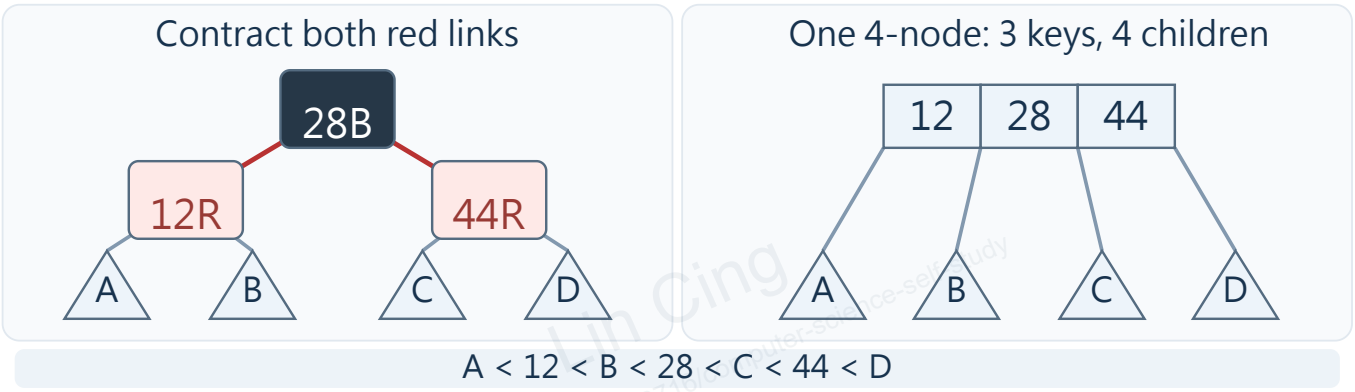
Search, min, max, successor, and predecessor follow paths in O(log(n+1)). Insertion and deletion still require repair-cost proofs. Complete inorder, preorder, and postorder output all n nodes in Θ(n). Red-black trees do not require child heights to differ by at most1; that is not their definition.

Why can one 3-node have two red-black encodings?



Merge a red child with its black parent into one multi-key node. A lone black node is a 2-node. A,B,C have black roots (possibly NIL) and equal black height; the inequalities apply to every key in each subtree. Both encodings preserve in-order sequence. **General red-black trees need not lean left.**

Equal black counts become equal external-leaf depths



No consecutive red links limits each merged node to three keys: a 2-, 3-, or 4-node. Boundary intervals stay ordered. Subtracting terminal NIL's black, each path's black count equals its multi-key-node count after contraction. Equal counts give equal external-leaf depths. Incoming links take the child's color; the root is separately black.

Rotations

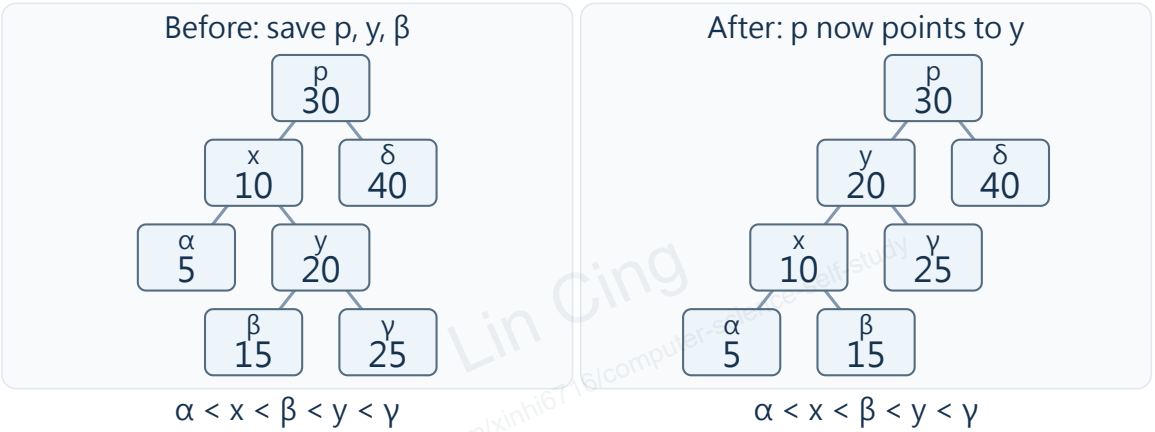
Save the old parent before reconnecting the local root

Assume x≠NIL and y=x.right≠NIL. Save p=x.parent and β=y.left. Internal links in α=x.left and γ=y.right remain unchanged.

```
LEFT-ROTATE(T,x):
    p=x.parent; y=x.right; beta=y.left
    x.right=beta
    if beta!=NIL: beta.parent=x
    y.parent=p
    if p==NIL: T.root=y
    elif x==p.left: p.left=y
    else: p.right=y
    y.left=x
    x.parent=y
```

Update **x's old parent p to point to y**. y's old parent is x; do not confuse them. Do not write beta.parent when beta=NIL, because that could overwrite the shared sentinel context used by deletion repair. Rotation changes no color, key, or data.

Rotations (continued)



Pointer table and the complete right mirror

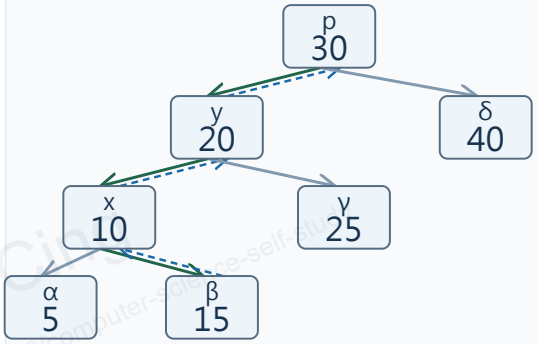
For p30,x10,y20,β15 with x=p.left, write10.right=15,15.parent=10,20.parent=30,30.left=20,20.left=10,10.parent=20. If p=NIL, replace the30.left write with root=20 and set20.parent=NIL.

```
RIGHT-ROTATE(T,y):
    p=y.parent; x=y.left; beta=x.right
    y.left=beta
    if beta!=NIL: beta.parent=y
    x.parent=p
    if p==NIL: T.root=x
    elif y==p.right: p.right=x
    else: p.left=x
    x.right=y
    y.parent=x
```

Pointer table and the complete right mirror (continued)

1. 10.right = 15
2. 15.parent = 10
3. 20.parent = 30
4. 30.left = 20
5. 20.left = 10
6. 10.parent = 20

Solid ↓ : child
Dashed ↑ : parent



If p=NIL: step 4 sets root=20; step 3 sets 20.parent=NIL.

Rotation preserves order, not arbitrary color invariants

Before and after left rotation, inorder is α,x,β,y,γ. Keys in α<x, keys in β lie between x and y, and γ>y, so β remains valid as x.right. Outside the local region, p's same-side link changes from x to y and all bounds remain. Non-NIL β.parent, x.parent, and y.parent become x,y,p respectively, keeping structure consistent. Right rotation is symmetric.

Only constantly many fields change, giving O(1) time. Finding the repair position still takes O(log n), so a constant-time rotation does not imply constant-time insertion or deletion. Rotation preserves BST order and record identities, **not automatically black height or the no-red-red rule**. Recoloring needs the local black-count arguments below.

Insertion

A red leaf can violate only root color or red adjacency

Use BST search to find an empty position. Initialize z.left=z.right=NIL and z.color=R, then attach its parent. Replacing a black NIL with a red leaf retains the same terminal black NIL, so path black counts do not change. Inserting black instead would add one black only on paths through that position, violating property5.

The first inserted root is temporarily red and becomes black at the end. Otherwise only red z and red parent p may conflict. The repair-loop invariant says z is red; if its parent is root, that parent is black; at most one violation exists, either a red root or z-parent red adjacency; all other color properties and BST order hold. Entering with p red implies p is not root, so grandparent g exists and is black, or another red-red violation would already exist.

Complete insertion repair

The first branch has p=g.left. The other exchanges left/right and LEFT/RIGHT-ROTATE everywhere.

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:              // I2, followed by I3
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                   // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: execute the fully mirrored three branches
T.root.color=B
```

Complete insertion repair (continued)

I2 and I3 are not mutually exclusive. I2 must be followed by I3. Use the updated z.parent and grandparent, not a stale pre-rotation p.

When p=g.right	Explicit action
I1: uncle g.left is red	Make p/uncle black and g red; z=g
I2: black uncle, z=p.left	z=p; RIGHT-ROTATE(z), then I3
I3: black uncle, outer child	New z.parent black, grandparent red; LEFT-ROTATE(grandparent)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1 preserves complete black counts

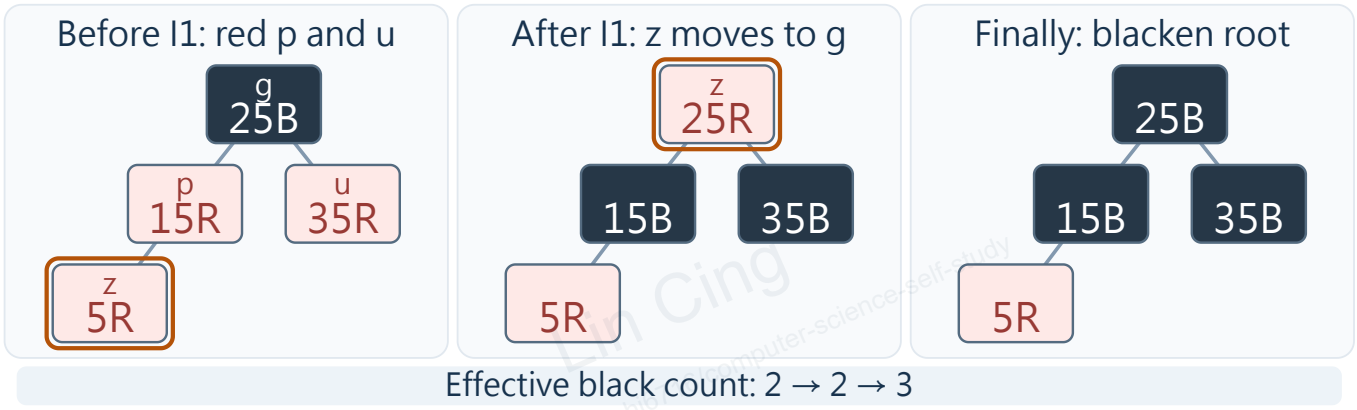
Suppose every path below p and uncle u contributes b black nodes, including terminal NIL. Initially g is black and p/u red, giving1+b on every complete path through g. Making g red and p/u black retains1+b. Although bh(g), which excludes g, changes from b to b+1, **complete ancestor paths through g gain no black node**.

Set new z=g. Red adjacency can only move up to g and its parent; the original z-p conflict is removed by making p black. If g becomes root, only the root is temporarily red. Blackening it at the end adds one black consistently to every whole-tree path.

Insert25,15,35,5. New5R has parent15R, uncle35R, grandparent25B. I1 makes15/35 black and25 red, then z=25. Its parent NIL is black, so terminate and blacken root25. A nonroot g does not receive this final whole-tree increase.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1 preserves complete black counts (continued)

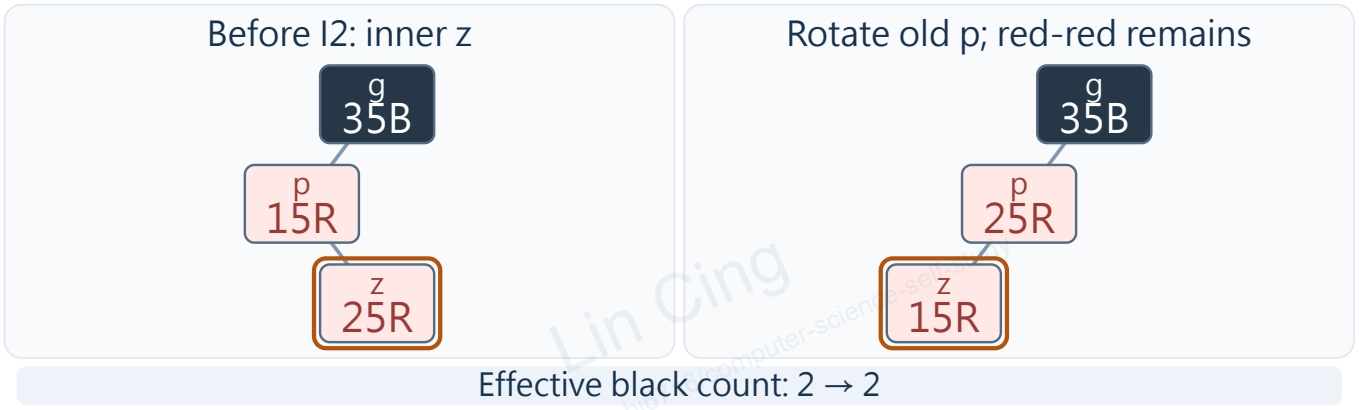


I2 converts an inner bend to an outer chain

With black uncle, p=g.left, and z=p.right, first set z=p and left-rotate the old p. Both rotated internal nodes are red, so rotation changes no black contribution to boundary paths and preserves inorder. Old p becomes a left red child under the new local root, producing an outer chain that still requires I3.

Insert35,15,25. Before repair, root35B has left15R with right25R. Left-rotating15 leaves root35B,35.left=25R,25.left=15R, and the algorithm's z now points to15. The15/25 red conflict remains; do not return here.

I2 converts an inner bend to an outer chain (continued)

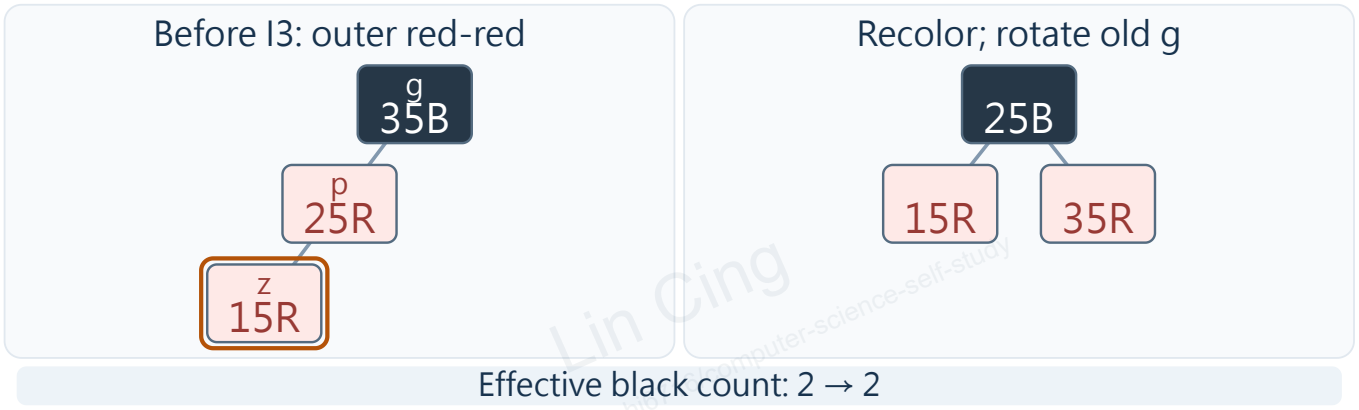


I3 recolors and rotates to restore every local path

The outer chain is gB-left pR-left zR, with black uncle. Make p black and g red, then right-rotate g to make p the local root. Let α,β be z's children, γ the old p.right, and δ the old g.right. All four boundary subtrees have black roots, possibly NIL, and contribute b black nodes per complete downward path. Each original path through g contributes1+b; after repair, each path through new p also contributes1+b. z remains red under black p, while red g has black γ,δ children. New local root p is black and cannot conflict with its external parent.

Continue the example: make25 black and35 red, right-rotate35, and obtain25B with15R,35R children. Inorder15,25,35 is preserved, bh(root)=1, and every root-NIL path contains2 blacks. Mirror insertion15,35,25 first right-rotates35, then left-rotates15, giving the same tree.

I3 recolors and rotates to restore every local path (continued)



Each I1 moves z up two levels without rotation. I2+I3 use at most two rotations and terminate. Recoloring is O(log(n+1)), rotations number at most2, and total insertion including BST search is O(log(n+1)).

Deletion

Distinguish z, y, x, and y's original color

z is the record to remove. If z has fewer than two children, y=z; otherwise y is the minimum in z.right and moves as a whole object into z's position. x is the sole possible child filling **y's old position**, namely y.right for a successor with no left child. Save yOriginalColor before giving moved y z.color. Missing black depends on yOriginalColor, not y's new color or merely z.color.

```
RB-TRANSPLANT(u,v):
    if u.parent==NIL: root=v
    elif u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent              // write even when v==NIL
```

NIL.parent now records the parent of the current empty x position so repair can find its sibling. It is not a universal parent for all empty links. The root's parent is NIL. Rotations skip beta.parent for beta=NIL to preserve this context. After a stable operation, NIL.parent may be reset to NIL; NIL.color must always remain black.

Complete object relocation in RB-DELETE

```
RB-DELETE(z):                  // z is a found, valid record
    y=z; yOriginalColor=y.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(z,z.right)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(z,z.left)
    else:
        y=MINIMUM(z.right); yOriginalColor=y.color; x=y.right
        if y.parent==z:
            x.parent=y          // including x==NIL
        else:
            RB-TRANSPLANT(y,y.right)
            y.right=z.right; y.right.parent=y
        RB-TRANSPLANT(z,y)
        y.left=z.left; y.left.parent=y
        y.color=z.color
    if yOriginalColor==B: RB-DELETE-FIXUP(x)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete object relocation in RB-DELETE (continued)

Move the entire y object, retaining its key, data, and handle; do not give it z's data. An API may first SEARCH(key), returning NOT_FOUND without modification if absent. This procedure assumes an existing z and never treats NIL as an ordinary deletable record.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

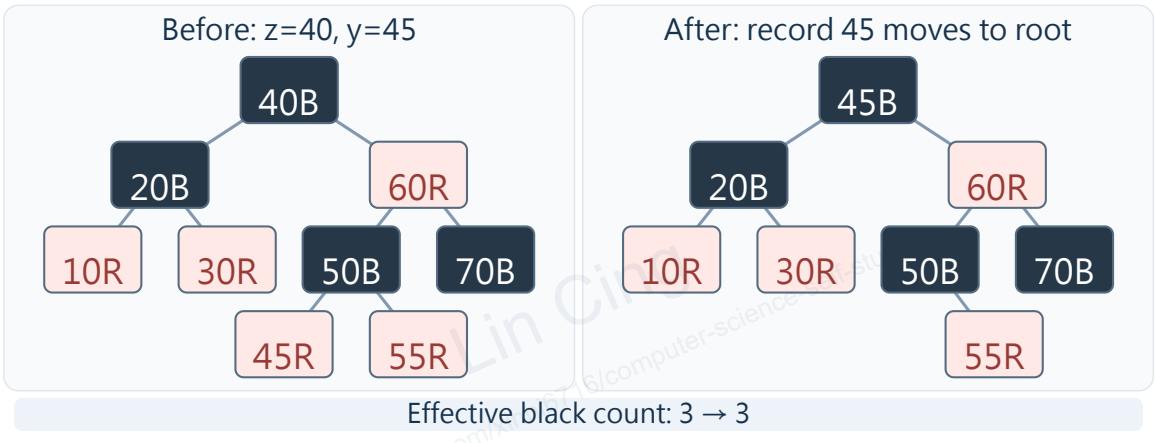
Successor relocation and NIL parent

Insert40,20,60,10,30,50,70,45,55. The tree has root40B, left20B(10R,30R), and right60R with left50B(45R,55R) and right70B. Deleting z40 chooses y45R and x=NIL, with y.parent50≠z.

Remove45 from50.left, giving50.left=NIL and NIL.parent=50. Attach60 as y.right and set60.parent=45. Replace root40 with y, then attach20 as y.left and set20.parent=45. Give y color B. Since y's original color was R, no black repair runs. Root is45B, original45's data moves with45, and only record40 is removed.

If y=z.right, x.parent must instead be set to y, never left pointing to deleted z. Both branches apply whether x is real or NIL.

Successor relocation and NIL parent (continued)



Track a missing black at x without a third color

Removing an originally red y reduces no path black count. Giving moved y z's old color preserves the other local relationships, so no fix is needed. If y was black, temporarily assign its missing black to x's position. Count one extra black on each path through x to match the pre-deletion effective counts.

This proof-only extra black does not change the actual R/B color domain. A red x absorbs it by becoming black. At the root, drop it from all tree paths together. Otherwise the loop has a black nonroot x. Its sibling w cannot be NIL: w would contribute only one ordinary NIL black while x's side contributes at least NIL plus extra, contradicting equal effective black counts.

Below, x=p.left, w=p.right, near=w.left, and far=w.right. A right-side deficit exchanges left and right, giving near=w.right and far=w.left, and reverses rotations.

Complete left-side deletion fixup

```
while x!=root and x.color==B:
    if x==x.parent.left:
        w=x.parent.right
        if w.color==R:                        // D1
            w.color=B; x.parent.color=R
            LEFT-ROTATE(T,x.parent)
            w=x.parent.right
        if w.left.color==B and w.right.color==B: // D2
            w.color=R
            x=x.parent
        else:
```

Complete left-side deletion fixup (continued)

```
            if w.right.color==B:              // D3, followed by D4
                w.left.color=B; w.color=R
                RIGHT-ROTATE(T,w)
                w=x.parent.right
            w.color=x.parent.color            // D4
            x.parent.color=B; w.right.color=B
            LEFT-ROTATE(T,x.parent)
            x=root
    else: execute the fully mirrored four branches
x.color=B
```

Refresh w after D1 and after D3. The four cases are not mutually exclusive elif branches. One iteration may execute D1→D2 or D1→D3→D4; the latter and direct D4 terminate.

Complete left-side deletion fixup (continued)

When x=p.right	w=p.left, near=w.right, far=w.left
D1: red w	w black, p red; RIGHT-ROTATE(p); refresh w
D2: both w children black	w red; x=p, move up or absorb by final blackening
D3: far black, near red	w.right black, w red; LEFT-ROTATE(w); refresh w
D4: far red	w takes p's old color; p and w.left black; RIGHT-ROTATE(p); x=root

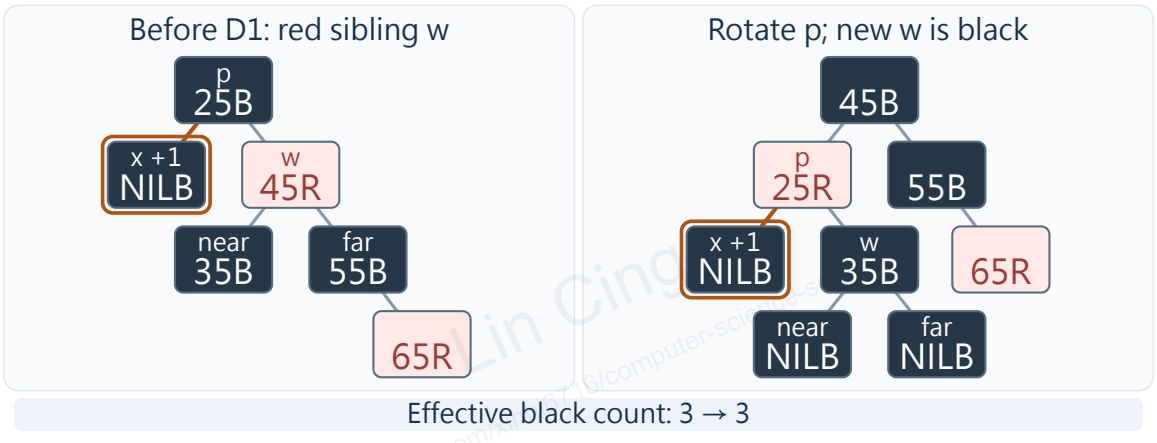
D1 converts a red sibling to a black sibling

Let x's **actual** complete black count be b, including x, or NIL if x=NIL. Extra makes it b+1. Red w forces p and both w children black. Each w-child path contributes b+1, so every effective complete path including p contributes b+2.

Make w black and p red, then left-rotate p. New black local root w has a red left child p whose x-side effective count b+1 equals the old near-child count b+1. Its right side, the old far child, also contributes b+1. Total remains b+2. Extra remains at x; its new sibling is w's old black near child. Proceed to D2, D3, or D4.

Insert15,25,35,45,55,65 and delete15. After removal x=NIL is25B.left and w45R. D1 makes45 black and25 red, then left-rotates25. New root45B has left25R(right35B) and right55B(right65R); x remains25.left. Complete black counts through ancestors are unchanged.

D1 converts a red sibling to a black sibling (continued)



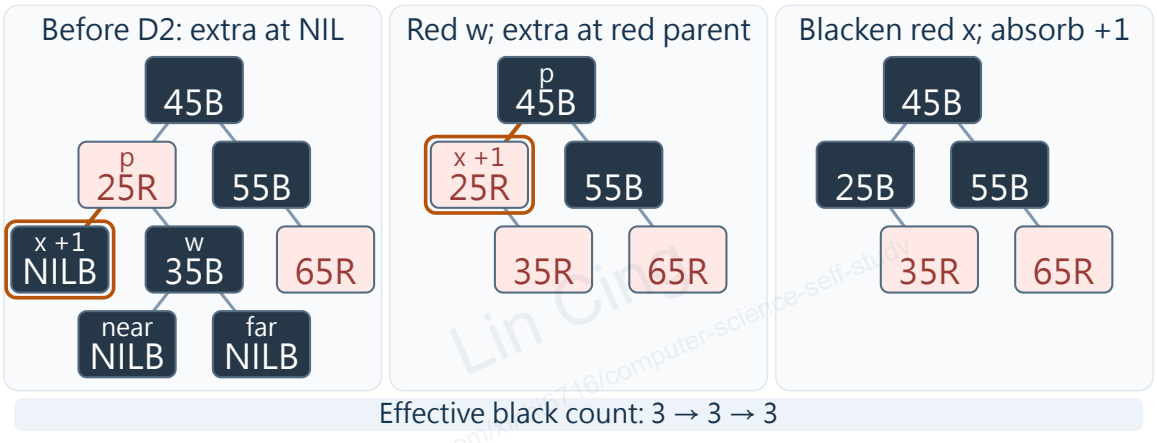
D2 moves the deficit upward or absorbs it at a red parent

Now w and both its children are black. x contributes actual b and effective b+1; w contributes b+1 and each child b. Making w red reduces its actual contribution to b. Both branches now have equal **actual** counts, but the whole p-subtree is one black short relative to its ancestors. Move extra to p by setting x=p.

If p was red, final blackening absorbs the extra and stops. If p was black and nonroot, iterate upward. At the root, drop extra consistently from all paths. D1 just made p red, so D1 followed by D2 absorbs in this round and cannot climb to another D1.

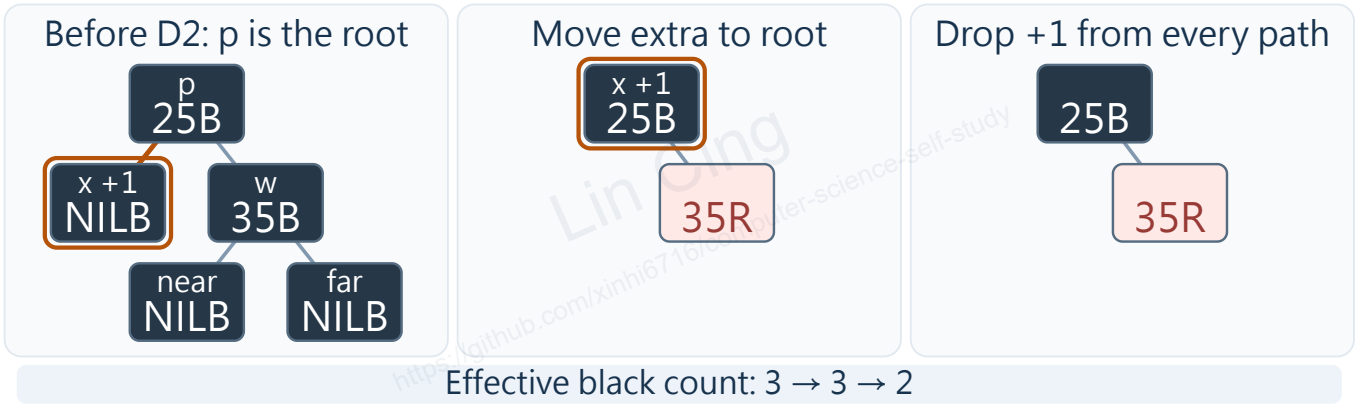
Continue the example: w35B has NIL children. Make35 red and move x from NIL to25R. The loop exits and25 becomes black. Finally45B has left25B(right35R) and right55B(right65R).

D2 moves the deficit upward or absorbs it at a red parent (continued)



D2 at the root reduces every path by one black

Start with25B and black leaves15B,35B. Deleting15 leads D2 to make35 red and move x to root25. Dropping extra from all root-leaf paths leaves25B(right35R). Root bh decreases from2 to1, with equal path counts. This differs from absorption by blackening a red parent.



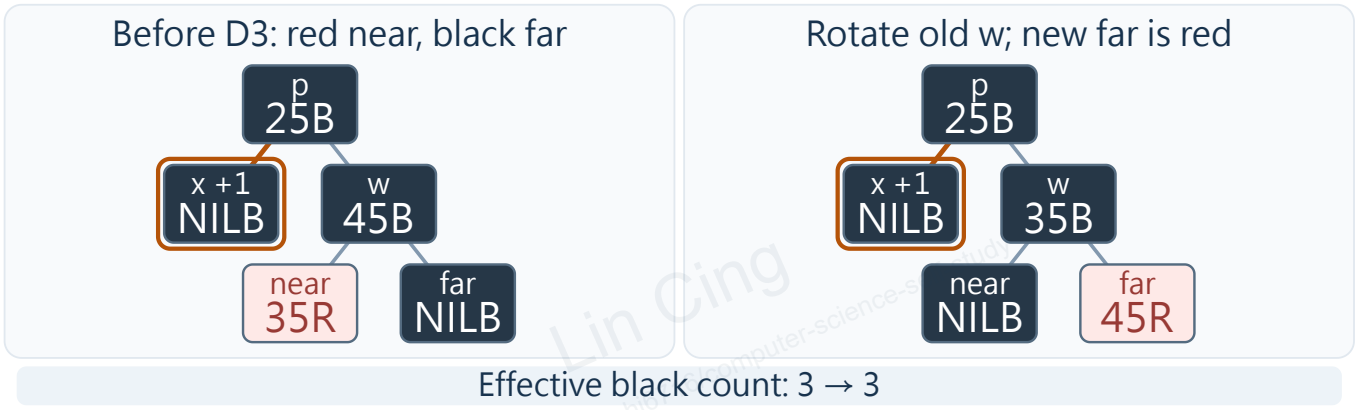
D3 moves an available red node to the far side

w is black, near N is red, and far is black. Let x's actual count be b. Both w children contribute b; red N's children each contribute b, as does far. Make N black and w red, then right-rotate w. New black sibling N has old N.left contributing b and old red w whose old N.right and far also contribute b. Its total remains b+1, preserving everything above p.

New sibling's far child is old red w, so execute D4 immediately. Extra remains at x; this case neither moves the deficit upward nor terminates.

Insert15,25,45,35 to obtain25B with left15B and right45B(left35R). Delete15, leaving x=NIL, w45B, near35R, and black far=NIL. Make35 black and45 red, right-rotate45, and obtain25B with right35B(right45R). D4 must follow.

D3 moves an available red node to the far side (continued)



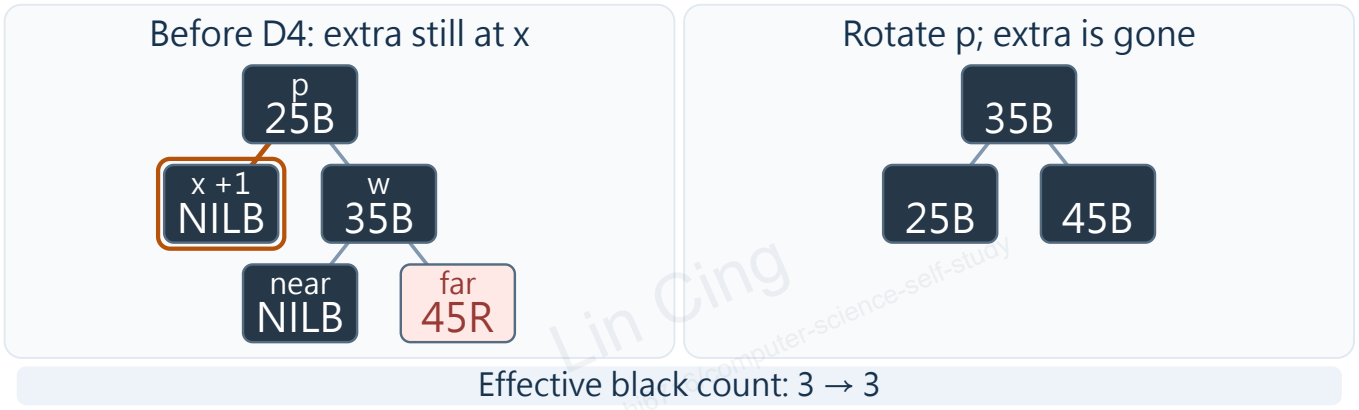
D4 repairs both sides by recoloring and rotation

Let p's original black contribution be c∈{0,1}, x's actual count b, and both children of black w contribute b; far is red. Give w p's old color, make p and far black, and left-rotate p. New root w has black left child p with equal actual b on x and near, so its complete left count is b+1. Far changes red to black and also contributes b+1. New local total is c+b+1, equal to original p's effective c+(b+1).

If new w is red, its children p and far are black. Making p black creates no red-red conflict, and w retains p's external color relationship. Extra is eliminated. Set x=root to stop and ensure the root is black.

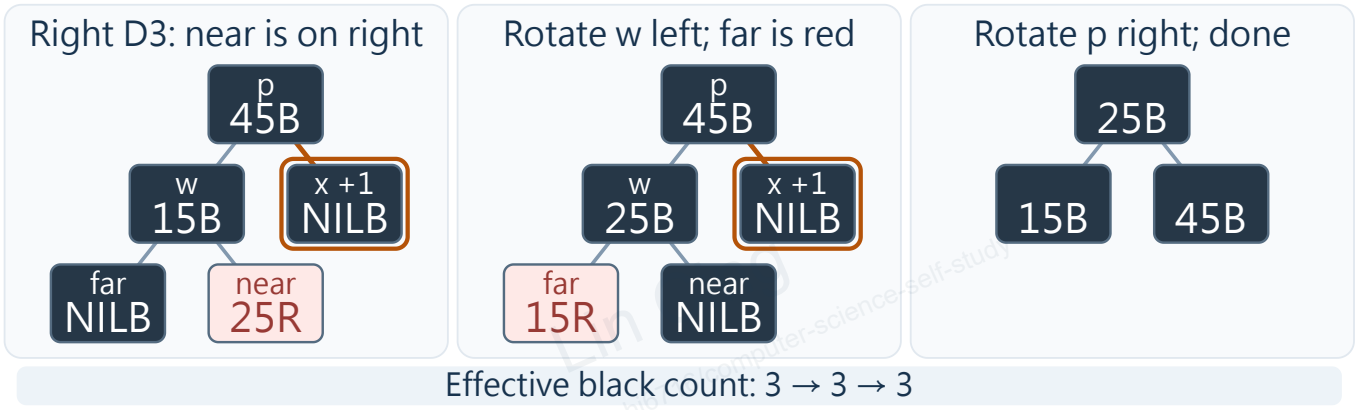
Continue D3:35 takes25's old black color;25 and far45 become black. Left-rotate25 to obtain35B with25B,45B children. Inorder is25,35,45, and every root-NIL path contains3 blacks with no extra.

D4 repairs both sides by recoloring and rotation (continued)



Mirror example: insert15,35,45,25, then delete35. Direct successor45B becomes root; x=NIL is45.right, so NIL.parent must be45. Sibling15B has near25R and far=NIL. Right-side D3 left-rotates15, then right-side D4 right-rotates45, yielding25B with15B,45B children.

D4 repairs both sides by recoloring and rotation (continued)



Count the black nodes along the same D3→D4 example

Every row **includes its starting node and NIL**. D3 leaves x=NIL in place; D4 removes the extra black.

Stage and path	Actual black	Extra	Effective total
Before D3: 25B → NIL(x)	2	1	3
Before D3: 25B → 45B → 35R → NIL	3	0	3
After D3: 25B → 35B → 45R → NIL	3	0	3
After D4: 35B → 25B → NIL	3	0	3
After D4: 35B → 45B → NIL	3	0	3

Each black node adds 1 and each red node adds 0. Add the extra only on paths through x. The other NIL paths also total 3 by the same rule; check them in the figures. After D4 both sides have the needed black without an extra.

From local conservation to the general proof and cost

BST relocation preserves order and every undeleted record. Removing red directly leaves a valid tree. Removing black assigns extra to x so effective ancestor-to-leaf counts remain balanced. D1 and D3 preserve its position and effective counts; D2 moves the sole extra upward; D4 eliminates it. A red x is repaired by final blackening. In D2, newly red w has black children; rotations and recolorings satisfy the checked local color conditions. At termination all five properties hold.

D2 ascends one level with no rotation, at most O(log(n+1)) times. After D1, p is red, so D2 absorbs, or D3/D4 terminates. Therefore D1, D3, and D4 occur at most once each, for at most3 rotations. Searching for z and its successor plus relocation and repair totals O(log(n+1)). NIL, root, and red-child absorption are valid base cases; deleting the last node leaves black root=NIL.

This proof covers every legal red-black tree. Finite example programs additionally check global key order, reverse parent links, black children of red nodes, equal NIL-path black counts, root color, and undeleted record identities. They cover both sides of all insertion and deletion cases. Recoloring may follow the height; at most3 rotations does not make complete deletion O(1).

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 13.

Lin Cing, personal handwritten data-structures notes on red-black and2-3-4 trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures

Part III Data Structures

Scope

Augmentation speeds up new queries while accounting for summary maintenance. Subtree size supports dynamic rank/select. Local-update proofs and exact maxima with closed endpoints establish why an interval tree can find overlap along one path without missing an answer.

Prerequisites

BSTs, size fields, intervals, and invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Support dynamic ranks with subtree sizes.
- Choose reliable auxiliary information systematically.
- Design an interval tree for overlap queries.
- 14.1 Dynamic order statistics
- 14.2 How to augment a data structure
- 14.3 Interval trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic order statistics

Chapter9 finds one kth smallest element of a static unsorted array in linear time. Repeated rank/select queries amid insertion and deletion can instead use subtree summaries in a balanced BST. An order-statistic tree augments each red-black node x with its number of real subtree nodes, including x, with NIL.size=0:

`size(x)=size(x.left)+size(x.right)+1`.

Rank is the one-based inorder position of a node identity. If duplicates occupy separate nodes, order by (key,identity) or explicitly use current inorder node order; a duplicate value has no unique rank by itself. Our examples have distinct keys.

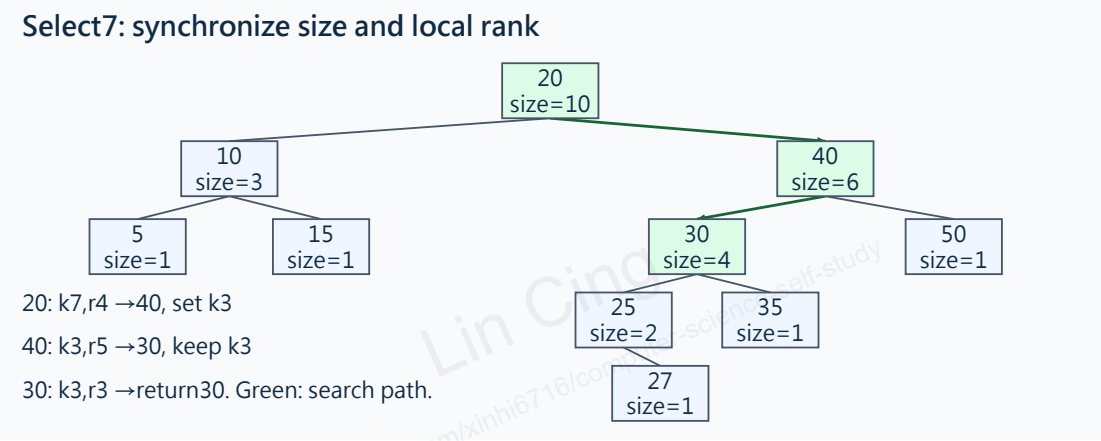
Query time is initially O(height+1). It becomes worst-case O(log(n+1)) only if the underlying tree preserves Chapter13's balancing properties. Adding size to an ordinary BST does not balance it.

Select's local-rank invariant

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

Invariant: the answer is local identity k in the current x-subtree. The left subtree has r−1 nodes and x is position r. Going left retains k; going right subtracts the entire left subtree plus x, r nodes. A valid k guarantees the chosen side is nonempty and its new k valid, so descent never incorrectly enters NIL. Each iteration descends one level and returns the correct node.

Select's local-rank invariant (continued)



At root20, left.size=3 and r=4, so select7 goes to40 with k=3. At40, left.size=4 and r=5, so it goes to30 with k still3. At30, left.size=2 and r=3, so return30. Do not compare global rank7 unchanged at every level.

Rank adds only an uncounted prefix

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

Invariant: r is original x's inorder rank inside current u's subtree. If u is a left child, the parent and right side follow x, so add nothing. If u is a right child, its parent's left subtree and parent precede x and lie outside the previously counted u-subtree. Add exactly left.size+1 without duplication. At root, r is the global rank.

For rank27, start r=1.27 is25.right, so add0+1 to get2.25 is30.left and30 is40.left, so add nothing.40 is20.right, so add3+1 to get6. Inorder5,10,15,20,25,27,30,35,40,50 confirms position6.

Update size according to the final structure

A new leaf has size1; only its ancestors gain one node. Count along the search path or recompute from its parent to root after linking. Each ancestor costs O(1), totaling O(height+1). Updating an existing duplicate record must not increment size.

Deletion cannot simply decrement ancestors of requested node z. With two children, successor y moves from its old position to z's, changing y's own subtree. A safe approach finishes Chapter12's transplants, then recomputes affected ancestors from deepest to shallowest.

Deleting20 and moving25 starts at25's old parent30: size30 changes4→3, size40 changes6→5, and new root25 becomes1+3+5=9. It cannot retain old size2. For a direct successor, start at moved y; for a leaf/one-child case, start at z's old parent.

Affected nodes lie on a constant number of ancestor paths, totaling O(height+1). Red-black repair rotations must also maintain size; the next section states when only two nodes need updates.

How to augment a data structure

Answer four questions together: what basic operations does the underlying structure support, what summary is stored, how does every mutation maintain it, and how does the new query use it to locate or prune? A formula for f alone proves neither maintenance nor query correctness.

A useful sufficient condition is that each node's f is computed in O(1) from its own key/data, unaffected by recoloring, and its children's f, with a constant NIL base case. Changed local data or child links can affect only that node and ancestors. Recompute descendants before ancestors.

The initial leaf attachment and a constant number of deletion transplants affect a constant number of O(log n) paths. For a rotation, repair the two pivot nodes first and propagate to ancestors if necessary, at most O(log n) extra work. Chapter13 bounds insertion to2 and deletion to3 rotations, so total augmentation work remains O(log n).

This general argument allows O(log n) propagation per rotation. It does not claim that every f needs only two local writes. If f depends on color, recoloring's dependency effects must also be maintained; unchanged structure alone is insufficient.

Size, maximum, and general summaries

In left-rotate(x), y=x.right rises and β=y.left moves to x. Finish pointer changes, then pull(x) before pull(y), because new y depends on new x. Reversing the order reads stale x information.

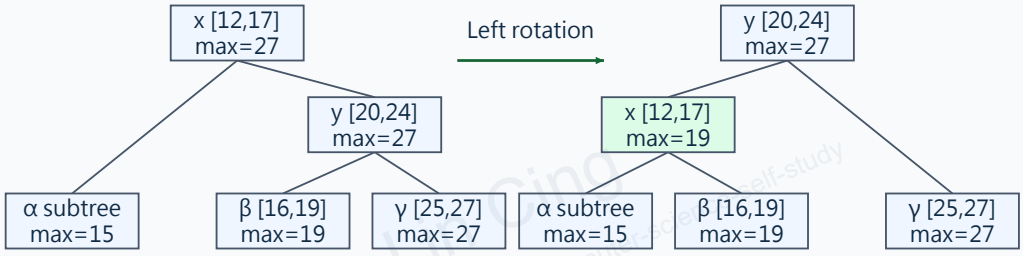
Size and maximum interval high depend only on subtree contents. New y contains exactly old x's entire subtree after rotation, so its summary equals old x's summary and other ancestors do not change. These two summaries need O(1) local recomputation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size, maximum, and general summaries (continued)

Local rotation: update the descending node first

Old x=[12,17], y=[20,24]; α.max15, β=[16,19], γ=[25,27].



Pull new x: max(17,15,19)=19; then new y: max(24,19,27)=27.

Green: descending x changes summary. Blue: other summaries stay unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size, maximum, and general summaries (continued)

Height is also child-computable, yet rotation can change the whole local subtree's height and require ancestor propagation. Explicitly storing global rank at every node requires $\Theta(n)$ writes when a new minimum raises all old ranks. Stored depth can similarly change across a large subtree after one rotation. Subtree size supports local maintenance and pathwise combination at query time, which is why it is useful.

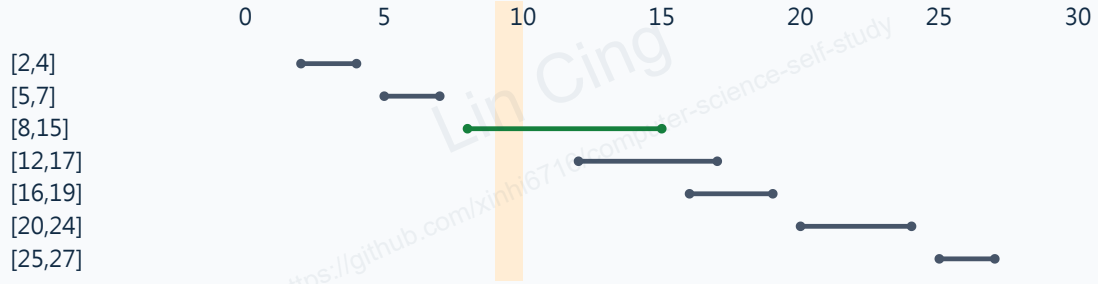
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interval trees

Intervals here are closed $[l,h]$, with $l \le h$, including singletons $[l,l]$. Intervals i,q overlap exactly when $i.low \le q.high$ and $q.low \le i.high$. Equivalently, neither lies completely left of the other: $i.high < q.low$ and $q.high < i.low$ are the two rejection cases.

Closed intervals and the same overlap query

Orange band: closed query[9,10]. Filled endpoint dots mean closed intervals.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Green[8,15]: overlaps. Gray: no overlap. [4,5] and[5,7] overlap at5.

Interval trees (continued)

Order the BST primarily by low; use identity to break equal-low ties, retaining left.low≤x.low≤right.low. Store max(x)=max(x.high,left.max,right.max), with NIL.max=−∞. This is the highest high in the **whole subtree**, not the node's high or the largest low.

Root[12,17] has high17 but max27. Left[5,7] has max15 from right descendant[8,15]. Looking only at the current interval can miss a farther-reaching descendant.

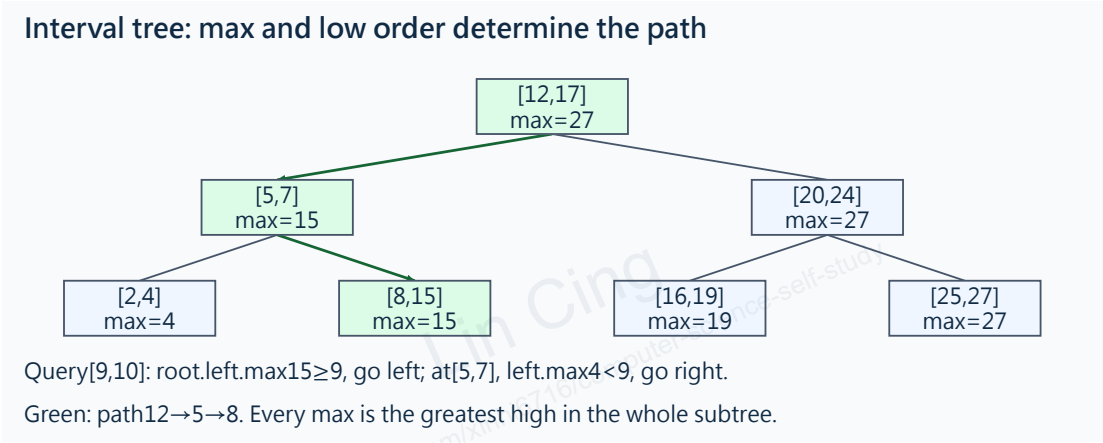
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One-path overlap search

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One-path overlap search (continued)



Query[9,10] does not overlap root[12,17], but left.max15≥9 leads to[5,7]. That also misses; left[2,4].max4<9 leads right to overlapping[8,15].

Query[18,18] goes from nonoverlapping root right because left.max15<18. At[20,24], left.max19≥18 leads to matching[16,19]. Query[28,29] follows[12,17]→[20,24]→[25,27]→NIL and fails.

Why rightward pruning is safe

Maintain: if an overlap exists anywhere in the tree, the current x-subtree contains one. This holds initially at root. Return if x overlaps. Otherwise prove that the chosen direction cannot discard all answers.

If left=NIL, it has no candidate. If left.max<q.low, every left interval j has j.high≤left.max<q.low and lies completely left of the query. Exclude the whole left side. Since x also misses, any answer must be right, preserving the invariant.

This uses a bound on every left descendant's high. The single condition x.high<q.low does not imply every left interval lies too far left. Change only the example root to[12,13] and query[14,14]: root.high13<14, yet left descendant[8,15] overlaps, so the left side cannot be pruned on that basis.

Why choosing left needs no later right search

When left.max≥q.low, choose a left interval j attaining that maximum. If left has any overlap, descending left is safe. If it has none, j.high≥q.low while j misses, so j.low>q.high; it cannot have j.high<q.low.

For every right descendant k, BST order gives `k.low≥x.low≥j.low>q.high`. The whole right side also lies too far right. With x itself nonoverlapping, this means the x-subtree had no answer. Thus either way, if an answer existed, the chosen left side retains one.

This proves the one-path algorithm. It does not assert that left.max≥q.low guarantees a left overlap. At NIL the chosen subtree is empty, so the invariant implies global absence. Each step descends once, giving O(height+1), or O(log(n+1)) with a red-black foundation.

Interval updates must allow maxima to decrease

After insertion/deletion, pull max along affected ancestors; after rotation, pull new x before new y. Merely increasing maxima on insertion is insufficient. Removing the greatest high or shortening an interval can decrease max, requiring recomputation from own high and both children.

Deleting[25,27] changes right[20,24].max27→24 and root.max27→24. This demonstrates a necessary decrease, but these two stale nodes are not the queried left-child summaries, so this deletion alone is not a concrete missed-query counterexample.

A real stale-left counterexample first changes[8,15] to[8,27], preserving low, making parent[5,7].max27. Delete[8,27]. The left subtree now has only[5,7] and[2,4], with correct max7; suppose it incorrectly stays27. Query[22,22] misses root[12,17] but follows stale left.max27≥22 to[5,7], then follows[2,4].max4<22 to empty right and returns NIL. It misses the actual overlap right[20,24]. An underestimated max can instead prune a true answer directly. Maintain **exact** max, not an arbitrary upper bound.

Interval updates must allow maxima to decrease (continued)

Closed intervals[5,7] and[4,5] overlap at5. Half-open[5,7) and[4,5) do not. Nonempty half-open intervals require l<h, strict overlap inequalities, and left.max>q.low for possible left overlap. Changing brackets alone while retaining comparisons is incorrect.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finding one, the leftmost, or all overlaps are different queries

The algorithm promises any overlapping record, not minimum low or all matches. To enumerate all, recurse left only if left.max≥q.low, test x, and recurse right only if x.low≤q.high and right.max≥q.low. Each visited node costs O(1), with worst-case O(n). The one-path O(log n) bound cannot be transferred to enumeration.

Another provable enumeration method repeats overlap-search, temporarily deletes each returned record, and records it in a list. k matches plus one final failure cost O((k+1)log(n+1)); reinsertion has the same order. This temporarily mutates the tree and must fit the caller's contract. Output alone costs Ω(k). Achieving exactly O(log n+k) requires another analysis or representation; saying “output-sensitive” is no proof.

Size and max can be stored together, each pulled in O(1), with the same maintenance order. Rank then follows BST order on (low,identity), not high order or number of overlaps, which need other summaries or algorithms.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter14.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-trees

Part V Advanced Data Structures

Scope

Packing multiple keys into a page reduces disk-page search paths. Insertion and deletion must move child pointers with keys to preserve global order, occupancy bounds, and equal leaf depth. Page I/O and within-page CPU work are counted separately.

Prerequisites

Multiway search trees, disk I/O, and height balance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define node capacity using minimum degree t.
- Analyze search, splits, and merges in page I/O.
- Check root and key-count boundaries.
- 18.1 Definition of B-trees
- 18.2 Basic operations on B-trees
- 18.3 Deleting a key from a B-tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Definition of B-trees

A node stores multiple sorted keys and child pointers, typically in one disk page. Keys are distinct; associated data or record pointers move with each key. Duplicate insertion reports EXISTS. This B-tree stores data in internal nodes, unlike a B+ tree with leaf-only data and copied parent separators.

Minimum degree t is an integer≥2. Every nonroot holds t−1..2t−1 keys; a nonempty root holds1..2t−1. An internal node with k keys has exactly k+1 children. All leaves have depth h, with root depth0.

Keys a0<…<a_(k−1) partition global child ranges: child0 keys<a0, child i keys in(a_(i−1),a_i), child k keys>a_(k−1). Sorted keys within each node alone do not prove these subtree bounds.

An empty tree is a zero-key leaf root at height0. The height theorem below assumes n≥1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Upper and lower height bounds

A minimum-occupancy nonempty height-h tree has one root key and two children; other nodes have t−1 keys. Depth i≥1 contains at least2t^(i−1) nodes, so

$$n \geq 1+\Sigma_{i=1}^{h} 2t^{i-1}(t-1)=2t^h-1,$$

hence $h \leq \log_t((n+1)/2)$. At h=0 the sum is empty and the formula still holds.

Conversely, at most2t−1 keys and2t children per node give

$$n \leq (2t-1)\Sigma_{i=0}^{h}(2t)^i=(2t)^{(h+1)}-1,$$

hence $h \geq \log_{(2t)}(n+1)-1$. These describe sparsest and densest legal trees; they do not predict a typical occupancy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Page bounds for a billion keys

For t=100,n=10^9, height2 holds at most200^3-1=7,999,999 keys, insufficient. Height3 holds up to200^4-1=1,599,999,999, so minimum possible height is3.

Height4 needs at least2 • 100^4-1=199,999,999 keys and remains possible. Height5 needs19,999,999,999,too many. Thus h∈{3,4},giving4 or5 visited nodes.

With a resident root and no other cached pages, this is3 or4 page reads; without cached root,4 or5. Linear within-page search needs O(t) comparisons,CPU O(th). Binary search reduces search comparisons to O(h log t),but insertion/deletion array movement may still cost O(t) per level.

Page size,key/record/child-pointer sizes,and metadata determine t. I/O counts are not measured disk latencies and cannot be replaced by comparison counts.

Basic search

Use zero-based node arrays. Find the first key≥k,or use key count if none.

```
search(x,k):
    i = first index with x.keys[i]>=k, or len(x.keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x is leaf: return NOT_FOUND
    read page x.children[i]
    return search(x.children[i],k)
```

Without an equal key,the separators leave at most one child that can contain k. All others are excluded by global order. Remaining height decreases at each step,ending at a successful key or unsuccessful leaf. Duplicate checking before insertion adds at most one path and leaves the asymptotic I/O bound unchanged.

Complete relocation when splitting a full child

Assume parent p is not full and y=p.children[i] has2t−1 keys. Save median=y.keys[t−1] before splitting.

```
z.keys = y.keys[t .. 2t-2]
y.keys = y.keys[0 .. t-2]
if y is internal:
    z.children = old y.children[t .. 2t-1]
    y.children = old y.children[0 .. t-1]
p.keys.insert(i,median)
p.children.insert(i+1,z)
write y,z,p
```

Each side has t−1 keys and,if internal,t children. The median moves out of the child into the parent as a real key; it is not duplicated. Original child ranges already lie on the correct sides of the median,so order and leaf depth remain. The nonfull parent can accept one key without exceeding2t−1.

Split before descending to keep the current node nonfull

```
insert(k):
    if search(root,k) succeeds: return EXISTS
    if root is full:
        root = new empty internal node with old root as child0
        split(root,0)
    x=root
    while x is internal:
        i = child index selected by k
        if x.children[i] is full:
            split(x,i)
            if k>x.keys[i]: i++
        x=x.children[i]
    insert k into x.keys in sorted position
```

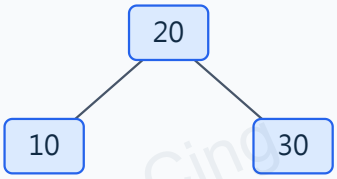
Invariant:x is nonfull. Splitting a full child before descent yields a chosen t−1-key child,so the next x is also nonfull. Leaf insertion adds at most one key without overflow. At most one split occurs per level. Only root splitting raises height,and it raises every leaf equally.

Split before descending to keep the current node nonfull (continued)

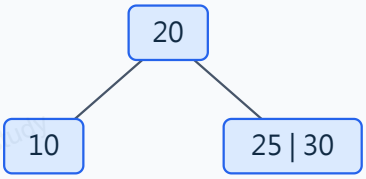
Split a full root, then insert25

1 Before insertion, t=2

2 Split, before insert25



3 Insert25 on the right



Median20 moves up and leaves the old leaf. Since25>20, insert25 into the right leaf.

Numerical root-split check

At t=2,insert25 into full leaf root[10,20,30].

Stage	Root	Children	Inorder
Before	[10,20,30],leaf	None	10,20,30
Split first	[20]	[10],[30]	10,20,30
Insert25	[20]	[10],[25,30]	10,20,25,30

20 appears only at root.25>20 chooses child1; that leaf grows from1 to2 keys,within maximum3. Both leaves have depth1.

A split writes old child,new child,and parent,a constant number of pages. State whether parent/old child are already cached. O(h) means linear in height,not an identical fixed read/write count for every insertion.

Deleting a key

Use a stronger descent condition: every recursively entered nonroot starts with at least t keys,so losing one leaves at least t−1. At the end,a zero-key internal root may be replaced by its sole child.

When k is found:

1. In a leaf,erase it directly.
2. In an internal node,if its left child has at least t keys,replace k by that subtree's maximum predecessor and delete it there. Otherwise use the successor if the right child has at least t keys.
3. If both children have t−1 keys,merge left child,parent key k,and right child into2t−1 keys,then delete k in the merged node.

Erasing an internal key without adjusting children breaks key/child counts and separators. Predecessor/successor replacement also moves the associated record data.

Repair the target child before descending

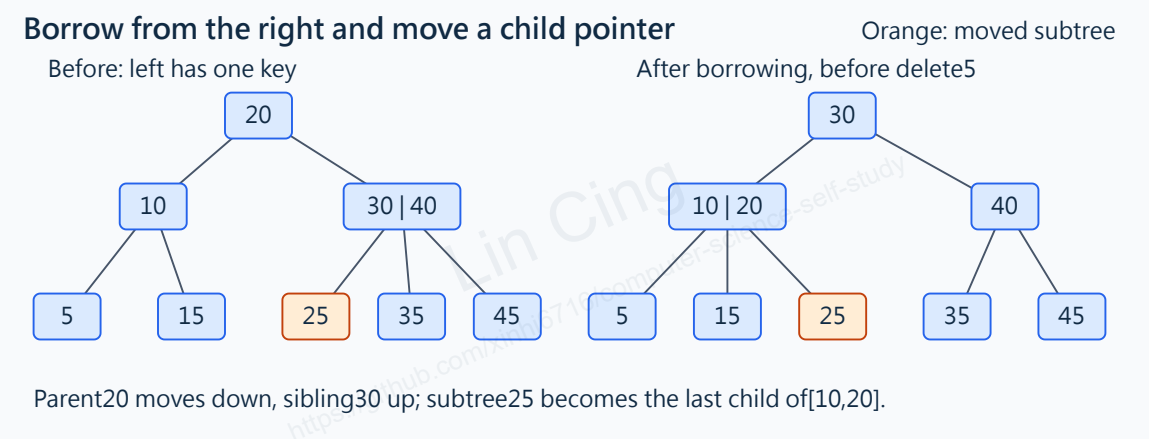
If x lacks k,search order selects child y. Descend directly if y has≥t keys. Otherwise y has t−1:

- With a left sibling holding≥t,move the parent separator to y's front and the sibling's maximum into the parent. For internal nodes,move that sibling's rightmost child to y's first child position.
- Otherwise,with a rich right sibling,move the separator to y's end and that sibling's minimum into the parent. Move its leftmost child to y's last child position.
- Otherwise merge y with an adjacent sibling and their parent separator,yielding2t−1 keys. If merging left,reduce the target child index by1.

Borrowing preserves both children’ s lower bounds without changing parent key count. Merging removes one parent key and one child while preserving ranges and leaf depth. An absent k may still trigger legal restructuring before NOT_FOUND,but the key set must remain unchanged.

Borrowing from the right also moves a child pointer

At t=2,deleting5 starts with a one-key left root child,too small for direct descent,and a two-key right child able to lend.



After borrowing,root30 has left child[10,20] with leaves[5],[15],[25],and right child[40] with[35],[45]. Subtree25 must move:left of new root30 and greater than20,it belongs as[10,20]'s last child.

Borrowing from the right also moves a child pointer (continued)

This is pre-descent repair;5 has not yet been removed. Next,[5] and adjacent[15] both have one key,so merge them with parent10 to[5,10,15],then delete5 to get[10,15]. Left internal node becomes[20],children[10,15],[25]. Root30 and right side remain. All leaves still have depth2.

Internal replacement and root contraction

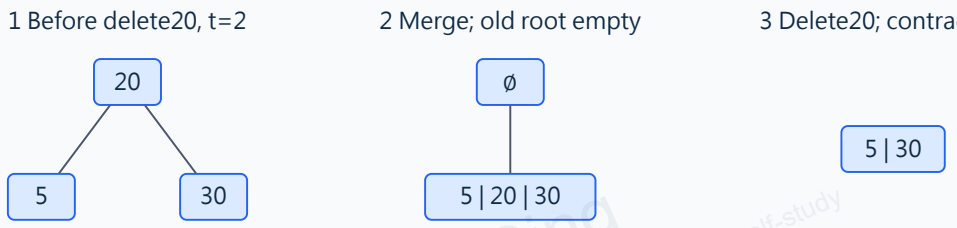
For root[20],left leaf[5,10],right leaf[30],delete20 via predecessor10. Result root[10],left[5],right[30],inorder5,10,30. Do not copy10 upward and forget to remove its old leaf occurrence.

For root[20] with leaves[5],[30],both are minimum-size. Merge to[5,20,30],leaving the old root temporarily empty with one child. Delete20 in the merged leaf to obtain[5,30],then replace the empty internal root by that child.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Internal replacement and root contraction (continued)

Merge, delete, and contract the root



Merge leaves and separator20; delete20 there; promote the sole child of the empty root.

Stable height1→0; only20 is removed from{5,20,30}.

Only an empty root may contract this way. Bypassing a general internal node would violate equal leaf depth. Deleting the last key retains a zero-key leaf root as the empty tree.

Deletion pseudocode and induction

```
delete(x,k):
    i = first key >=k, or key count
    if key i equals k:
        if leaf: erase key i; return FOUND
        if child i has >=t keys: replace k by pred; delete(child i,pred)
        elif child i+1 has >=t keys: replace k by succ; delete(child i+1,succ)
        else: y=merge(child i,key i,child i+1); delete(y,k)
        return FOUND
    if leaf: return NOT_FOUND
    y=child i
    if y has t-1 keys: borrow from a rich neighbor, else merge as above
    return delete(the repaired target child,k)
after top call: if root has0keys and1child: root=its child
```

Induct on remaining height. The descent condition lets a leaf lose one key without falling below t−1. Internal replacement preserves order while removing exactly target k from the total set. Borrow/merge gives a legal smaller-height subproblem. A nonroot x starts with≥t keys,so losing one separator in a merge remains legal. Final root contraction handles the exception. This proves termination,key-set correctness,and structural invariants.

Operation costs and implementation checks

Each descent handles constantly many adjacent nodes,with O(1) page reads/writes per level. Search,insertion,and deletion use O(h+1)=O(1+log_t(n+1)) page accesses. Within-node key/child movement costs O(t),giving CPU O(t(h+1)). Finding predecessor/successor separately before deletion adds only a constant number of paths in that subtree; these can also be combined into one descent.

Assume each node fits a page and a fixed root-cache model. Concurrency locks,transaction logs,compression,and fragmentation are additional implementation contracts,not consequences of abstract B-trees.

After each operation,check global key ranges,nonroot occupancy,child count=keys+1,equal leaf depth,no repeated child pointers,and inorder agreement with a reference set. Finite programs catch implementation errors; the induction proves arbitrary legal operations.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps

Part V Advanced Data Structures

Scope

Deferred consolidation gives meld and decrease-key low amortized costs. Root/child rings, loss marks, cascading cuts, and degree collisions must fit one potential argument. A Fibonacci subtree bound limits the consolidation array.

Prerequisites

Mergeable heaps, amortized analysis, and cut marks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Represent a relaxed heap with root and child lists.
- Amortize meld, insertion, and decrease-key.
- Prove consolidation cost using a degree bound.
- 19.1 Structure of Fibonacci heaps
- 19.2 Mergeable-heap operations
- 19.3 Decreasing a key and deleting a node
- 19.4 Bounding the maximum degree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Structure of Fibonacci heaps

A Fibonacci heap is a collection of min-heap-ordered trees. Every nonroot x satisfies parent.key≤x.key. The root list contains all roots, H.min points to a minimum-key root, and H.n counts all nodes. Decrease-key and erase accept valid node handles; their bounds exclude searching for an arbitrary key.

Each node stores key, degree, parent, child, left, right, and mark. Degree is its current child count; child points to one child. Siblings form circular doubly linked lists. We clear every root's mark. A marked nonroot has lost one child since it was last linked to its current parent.

Keys may repeat while handle identities differ. Deletion invalidates its handle. Meld consumes ownership of both input heaps; callers cannot keep mutating the same nodes through separate inputs. These premises define the objects covered by pointer and cost claims.

Constant-time ring pointer operations

Before removing x, save next=x.right. For a singleton set the ring head to NIL; otherwise reconnect x.left.right=x.right and x.right.left=x.left, updating head=next if needed. Finally set x.left=x.right=x and update its container's head/degree.

To concatenate nonempty rings A,B, choose a∈A,b∈B and save an=a.right,bp=b.left, then write

```
a.right=b; b.left=a
bp.right=an; an.left=bp
```

Only two adjacency edges change; no complete traversal is needed. An empty ring returns the other. Before adding x to roots, detach it from its child ring and decrement the old parent's degree, then set x.parent=NIL and mark=false. Overwriting x.right before traversing its original ring can lose the successor.

Mergeable-heap operations

MAKE-HEAP creates an empty root list, min=NIL, n=0. INSERT creates a degree-zero singleton ring with parent/child=NIL and mark=false, joins it to roots, updates min by comparison, and increments n. MINIMUM returns min, or NIL if empty.

MELD concatenates root rings, chooses the smaller input minimum, and adds node counts. It neither links trees nor scans nodes. Since each input already tracks its smallest root, one comparison suffices. These operations have O(1) actual cost.

Use $\Psi=C(t+2m)$, for a sufficiently large fixed C, root count t, and marked-node count m. Across multiple heaps, sum potentials. It starts at0 and remains nonnegative. INSERT adds one root and C potential, hence O(1) amortized. MELD preserves total roots and marks, giving $\Delta\Psi=0$ and O(1) amortized cost.

Complete extract-min order

```
extract-min(H):
    z=H.min
    if z==NIL: return NIL
    while z.child ring is nonempty:
        detach one child x from z
        x.parent=NIL; x.mark=false
        append x to root list
    remove z from root list; H.n--
    if H.n==0: H.min=NIL
    else: consolidate roots by degree; recompute H.min
    invalidate z's heap handle; return z
```

Children become roots and retain internal heap order. Other roots were already no smaller than z. Recompute min among the surviving consolidated roots. Do not leave min pointing to old z or remove z's entire subtree.

Consolidation collision loop

```
A[0..D] = NIL
while old root list is nonempty:
    x=detach one root; d=x.degree
    while A[d]!=NIL:
        y=A[d]; A[d]=NIL
        if y.key<x.key: swap(x,y)
        make y a child of x; y.parent=x; y.mark=false
        x.degree++; d=x.degree
    A[d]=x
rebuild root list from nonempty A slots
H.min = smallest-key root
```

Each link combines equal-degree roots under the smaller key, preserving heap order. After incrementing d, check A again. The result has at most one root per degree. D must be at least floor(log_ϕ n), or the array can grow on demand. floor($\log_2 n$) alone need not fit legal degrees.

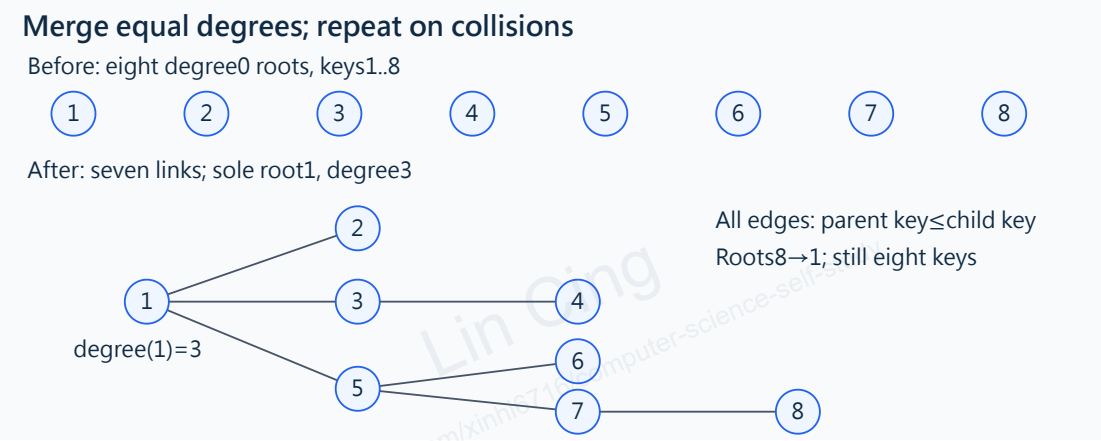
Consuming the old ring directly needs O(log n) auxiliary array space. A snapshot used to avoid mutation during traversal may contain Θ(n) roots, so its complete auxiliary space cannot still be called O(log n).

Numerical consolidation of eight roots

Insert0..8 and extract0, leaving singleton roots1..8. Process them in that order.

Root read	Links this round	Remaining A, degree:key
1	None	0:1
2	2 under1	1:1
3	None	0:3,1:1
4	4 under3;3 under1	2:1
5	None	0:5,2:1
6	6 under5	1:5,2:1
7	None	0:7,1:5,2:1
8	8 under7;7 under5;5 under1	3:1

Numerical consolidation of eight roots (continued)



Seven links reduce eight roots to one and preserve keys1..8. The tree need not be complete; every parent-child heap comparison and actual degree establish correctness.

Decreasing keys and cascading cuts

```
decrease-key(H,x,new):
    require new<=x.key and x is a valid handle
    x.key=new; p=x.parent
    if p!=NIL and x.key<p.key:
        cut x from p to root list
        cascading-cut(p)
    if x.key<H.min.key: H.min=x
cascading-cut(y):
    p=y.parent
    if p==NIL: return
    if y.mark==false: y.mark=true; return
    cut y from p to root list
    cascading-cut(p)
```

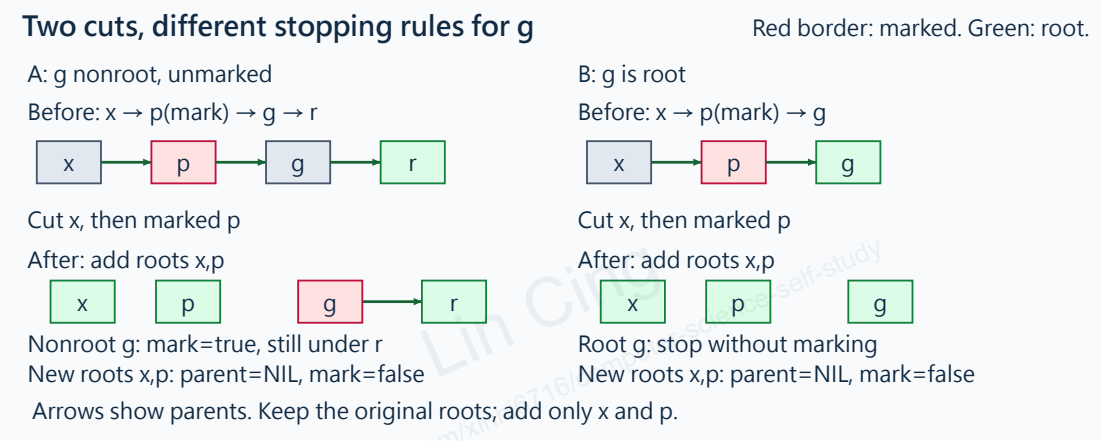
Cut clears the moved node's mark, decrements its old parent's degree, and preserves both ring directions. Lowering x can violate only its parent edge; its child comparisons become easier. Cutting the violated edge restores heap order. Marks separately protect the degree bound, rather than locating the minimum.

The two cascade stopping branches

If p was already marked, losing second child x cuts p to roots too. Inspect its old parent g next. If g is a nonroot and unmarked, mark it and stop. If g is root, stop without marking it for this loss.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The two cascade stopping branches (continued)



The root list **adds** x,p while retaining existing roots. Both have parent=NIL and mark=false. A nonroot g stays under its old parent and changes only its mark. If g was already marked, continue cutting rather than applying the unmarked branch.

Actual nodes from a legal history

Insert0..64 and extract0 to produce a consolidated64-node heap. Node IDs initially equal keys and stay fixed after decreases.

Branch	First operation	Second operation	Final state
g root	Lower id6 to−10;cut6,mark p=id5	Lower id7 to−11;cut7 then5	roots={1,6,7,5};g=id1 unmarked
g nonroot	Lower id14 to−10;cut14,mark p=id13	Lower id15 to−11;cut15 then13	roots={1,14,15,13};g=id9 marked,still under id1

After the first operation, both have t=2,m=1 and t+2m=4. After the second, the root branch has t=4,m=0 and scale4; the nonroot branch t=4,m=1 and scale6. These parents arise from legal links and cuts, rather than assuming an arbitrary shape is reachable.

Complete decrease-key potential accounting

Without a cut, actual cost and potential change are O(1). If c≥1 nodes are cut, roots increase by c. Apart from the first x cut because of its key, all c−1 cascading nodes were marked and lose their marks. At most one final unmarked nonroot gains a mark.

Thus Δm≤−(c−1)+1=2−c and

$$\Delta(t+2m)\le c+2(2-c)=4-c.$$

If actual cost≤ac+b, fix C≥a in the same Ψ=C(t+2m). Then

$$\hat{c}\le ac+b+C(4-c)\le b+4C=O(1).$$

Scaling explicitly pays the actual pointer-operation constants. One decrease-key can traverse a long ancestor chain; the constant is an amortized bound for operation sequences.

Deletion cost and a special minimum key

Abstract CLRS DELETE(x) first decreases x to a special $-\infty$ strictly below every ordinary key, then extracts minimum. Its amortized cost is $O(1)+O(\log n)=O(\log n)$, not $O(1)$.

That $-\infty$ is an extra key-domain element, not necessarily an integer type's minimum. If an ordinary node already has that value, ties can cause extract-min to remove another handle.

Alternatively erase the handle without changing its key. If x is nonroot, cut x and cascade from its old parent. Remove that specific root x, promote its children, consolidate, and recompute min. The cut stage is $O(1)$ amortized and root removal $O(\log n)$, not constant-time deletion that omits consolidation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bounding maximum degree

For fixed x, order its current k children $y_1..y_k$ by when they were linked to x. y_1 has degree ≥ 0. When y_i joined for $i \geq 2$, the earlier $i-1$ children that still survive were already present, so x then had degree $\geq i-1$.

Links combine equal-degree roots, so y_i then also had degree $\geq i-1$. While still a child of x, y_i can have lost at most one child since then; the second loss would have cut it. Therefore

$\text{degree}(y_i) \geq i-2$.

This uses insertion order of **currently surviving** children. It assumes neither that x never lost a child nor that all current child degrees are equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci lower bound on subtree size

Let F0=0,F1=1,F_(j+2)=F_(j+1)+F_j. Prove size(x)≥F_(degree(x)+2) by strong induction on subtree size N.

N=1 gives degree0 and F2=1. For N>1 every child subtree is smaller, so apply induction to its actual degree; no assumption that a child's degree is below its parent's is needed. Child degree bounds and monotonicity give, for k≥1,

`size(x)≥1+1+Σ_(i=2)^k F_i=F_(k+2).`

The identity follows from Σ_(i=0)^kF_i=F_(k+2)−1 with F0=0,F1=1. At k=1 the empty sum gives2=F3.

Set ϕ=(1+√5)/2. F2=1=ϕ⁰,F3=2≥ϕ. Adding previous bounds and using $\phi^2=\phi+1$ inductively gives F_(k+2)≥ϕ^k. Thus n≥size(x)≥ϕ^k and k≤log_ϕ n. This bounds degree, not tree height.

Extract-min's total potential account

Before removal there are t roots and at most D children under the minimum. Removal/promotion leaves at most t−1+D roots. Each link reduces root count by1, so actual consolidation work is O(t+D), not O(tD) despite the nested collision loop.

Afterward distinct degrees give at most D+1 roots and marks do not increase, so Δ(t+2m)≤D+1−t. With the same sufficiently large C,

`ĉ≤a(t+D)+b+C(D+1−t)=O(D+1).`

The degree proof gives D=O(log(n+1)). Extract-min and delete are O(log(n+1)) amortized. Minimum,insert,meld,decrease-key are O(1) amortized; the first three also have O(1) actual cost. All conclusions telescope the same nonnegative potential. Finite operation tests check diagrams,numerical cases,and implementation states rather than replacing these proofs.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter19.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

van Emde Boas trees

Part V Advanced Data Structures

Scope

Splitting integer high/low bits supports fast predecessors and successors. Cached min/max makes the doubly logarithmic bound possible by allowing only one expensive recursion per level. Independently stored minima and linear-universe allocation determine update rules and applicability.

Prerequisites

Integer universes, bit splitting, and recursive summaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Compare bitmaps, summaries, and clustering.
- Define recursive cluster and summary universes.
- Derive insertion, successor, and min/max costs.
- 20.1 Preliminary approaches
- 20.2 A recursive structure
- 20.3 The van Emde Boas tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preliminary approaches

The integer universe is U={0,…,u−1}, u=2^b, b≥1. successor(x) is the smallest stored value **strictly greater** than x; predecessor is strictly smaller. x need not be stored. Return NIL when absent.

Assume word-RAM with word length w≥b. Legal keys, shifts, masks, and array indexing each cost O(1). This is not bit complexity; keys spanning multiple machine words do not inherit the same constant cost.

A u-bit bitmap supports direct membership. Successor may scan forward; even with a fast first-one operation per word, worst case is Θ(u/w) words. O(1) requires the entire universe to fit one word and the model to provide the necessary bit scan. A red-black tree instead uses O(n) nodes and O(log(n+1)) time on arbitrary comparison keys without allocating u.

Clustering alone does not imply double logarithms

Split u=256 into16 groups of16 keys. Summary bit i indicates nonempty group i. Successor checks for a larger value in the current group, then finds the next nonempty group and its minimum.

Bit-by-bit scans still cost up to16 within-group steps plus16 summary steps. Group width m gives O(m+u/m), minimized near$\sqrt{u}$ at O($\sqrt{u}$), not O(log log u).

Recursing over many levels also requires counting expensive calls per level. The proto structure below shows why taking square roots of the universe alone is insufficient.

A recursive structure

Split b bits into upper ceil(b/2) and lower floor(b/2) bits:

```
upper=2^ceil(b/2); lower=2^floor(b/2)
high(x)=floor(x/lower); low(x)=x mod lower
index(h,l)=h*lower+l
```

There are upper clusters, each with universe lower. The summary has universe upper and stores nonempty cluster indices. These two sizes cannot always both be written as$\sqrt{u}$.

At u=32, b=5, upper=8, lower=4. Key27 has high6, low3, and recombines as6 • 4+3=27. Rounding$\sqrt{32}$ for both parts would lose a complete nonoverlapping bit partition.

Why proto-vEB's two recursions are slower

First let b be a power of two so splitting is exact. Without cached extrema, proto-minimum recursively finds the first nonempty group in summary and then its local minimum:

$T_{min}(u)=2T_{min}(\sqrt{u})+\Theta(1)$.

With S(b)=T(2^b), S(b)=2S(b/2)+Θ(1)=Θ(b), giving Θ(log u). Insertion that always recurses into both cluster and summary has the same bound.

Proto-successor may search its own cluster unsuccessfully, recurse into summary, then find the next group's minimum:

$T_{succ}(u)=2T_{succ}(\sqrt{u})+\Theta(\log u)$.

Substitution gives S(b)=2S(b/2)+Θ(b)=Θ(b log b), or Θ(log u log log u). Improvement must remove expensive branches, not merely count recursion depth.

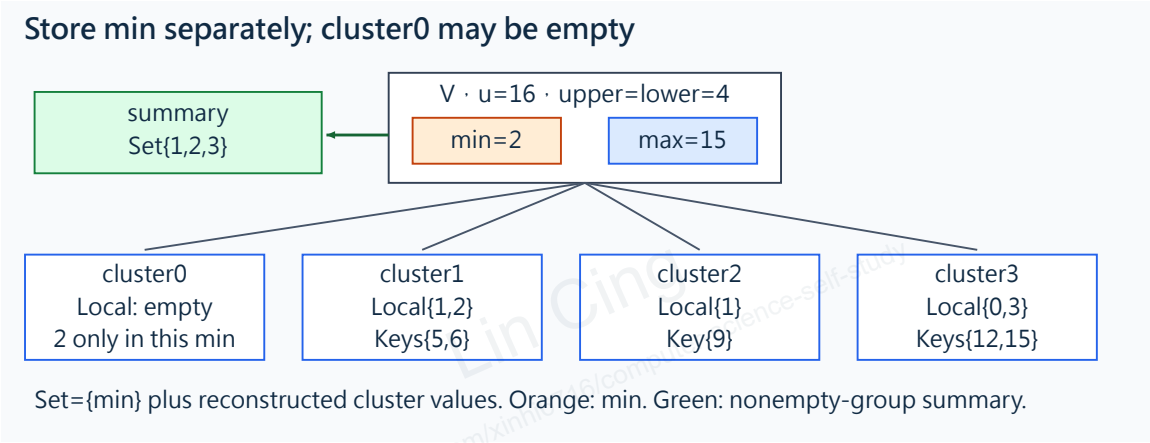
The van Emde Boas representation

Each node stores min,max, both NIL when empty. **min is excluded from all clusters.** Every other key occurs uniquely by high/low in a cluster. Summary contains exactly indices of those nonempty clusters.

A singleton has min=max with no nonempty clusters. With at least two values, max is also stored in a cluster. Base universe u=2 uses only min/max, without further clusters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The van Emde Boas representation (continued)



At u=16 and S={2,5,6,9,12,15}, min2,max15; cluster0 empty,cluster1 local{1,2},cluster2{1},cluster3{0,3},summary{1,2,3}. high(2)=0 does not force summary to contain0 because2 exists only in this node's min field.

Empty, singleton, and membership

```
member(V,x):
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return member(V.cluster[high(x)],low(x))
minimum(V): return V.min
maximum(V): return V.max
empty-insert(V,x): V.min=V.max=x
```

Membership enters one cluster per level with strictly smaller universe. Extrema and empty/singleton tests cost O(1). empty-insert does not recursively populate the whole structure, since a new min belongs to no cluster. This constant-time base also leaves only one expensive update recursion.

The public interface validates integer $x \in 0..u-1$. It may detect duplicate insertion or absent deletion using member, return false, and leave the set unchanged. Adding one same-order query does not change asymptotic time. Internal update pseudocode assumes a new key or a present key respectively.

Mutually exclusive successor branches

```
successor(V,x):
    if V.min==NIL or x>=V.max: return NIL
    if x<V.min: return V.min
    if V.u==2: return 1
    h=high(x); l=low(x); C=V.cluster[h]
    if C.max!=NIL and l<C.max:
        return index(h,successor(C,l))
    j=successor(V.summary,h)
    if j==NIL: return NIL
    return index(j,V.cluster[j].min)
```

Reading C.max determines whether a larger same-cluster value exists. If so, recurse only there. Otherwise recurse directly into summary without first searching a cluster known to fail. Reading the next cluster's min costs O(1), so there is at most one expensive call per level.

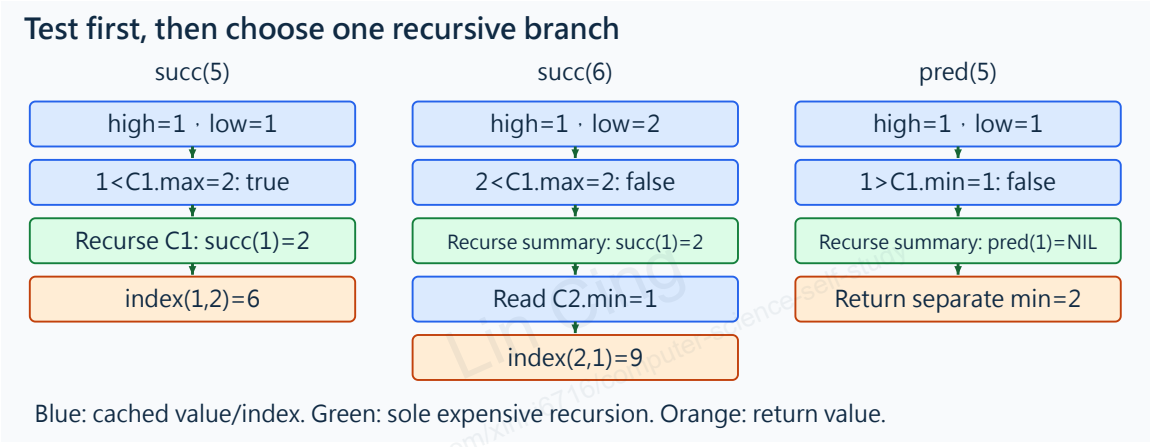
Predecessor must restore the separate minimum

```
predecessor(V,x):
    if V.min==NIL or x<=V.min: return NIL
    if x>V.max: return V.max
    if V.u==2: return 0
    h=high(x); l=low(x); C=V.cluster[h]
    if C.min!=NIL and l>C.min:
        return index(h,predecessor(C,l))
    j=predecessor(V.summary,h)
    if j==NIL: return V.min
    return index(j,V.cluster[j].max)
```

At j=NIL, earlier tests established x>V.min, so global min is a valid answer. Do not mirror successor's NIL branch mechanically: min was deliberately excluded from clusters.

For S={2,5,6,9,12,15}, pred5 has no smaller local key in cluster1 and no preceding summary group, but must still return2.

Actual same-group and cross-group queries



succ5: high1,low1,cluster1.max2. Since1<2, recurse only into cluster1, get local2, and return1・4+2=6.

succ6: high1,low2 is not below cluster1.max2, so skip that cluster. summary.succ1=2; read cluster2.min1 and return2・4+1=9.

Actual same-group and cross-group queries (continued)

Queries need not belong to S. succ3 crosses from empty cluster0 to cluster1.min1 and returns5. succ15=NIL and pred2=NIL.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion swaps min and treats empty groups specially

```
insert-new(V,x):
    if V.min==NIL: empty-insert(V,x); return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        h=high(x); l=low(x); C=V.cluster[h]
        if C.min==NIL:
            insert-new(V.summary,h)
            empty-insert(C,l)
        else: insert-new(C,l)
    if x>V.max: V.max=x
```

Lin Cing

A smaller new key stays in min; the old minimum becomes the value inserted into a cluster. Empty-group insertion updates two places, but cluster empty-insert is O(1), leaving only summary expensive. A nonempty group recurses only into its cluster without summary changes.

Insert1 into the example: min becomes1, old min2 enters cluster0 as local2, and summary gains0. Final max remains15. Do not also put new min1 into cluster0.

Deletion bases and promotion of a new minimum

```
delete-present(V,x):
    if V.min==V.max: V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=(1 if x==0 else 0); return
    if x==V.min:
        h=V.summary.min
        x=index(h,V.cluster[h].min)
        V.min=x
    h=high(x); l=low(x); C=V.cluster[h]
    delete-present(C,l)
```

Continue with the next slide's summary/max cleanup. In a multi-element set, deleting min promotes the smallest clustered key, then removes that promoted value from its old cluster to preserve exclusion. Variable x now denotes the promoted value, not the caller's original deleted minimum.

Complete summary and maximum cleanup

After the preceding deletion in C:

```
if C.min==NIL:
    delete-present(V.summary,h)
    if x==V.max:
        j=V.summary.max
        if j==NIL: V.max=V.min
        else: V.max=index(j,V.cluster[j].max)
elif x==V.max:
    V.max=index(h,C.max)
```

An emptied C had only one element before deletion, so its deletion used O(1) base work before summary recursion. If cluster deletion really recursed through several levels, C cannot become empty and no summary recursion follows. The two expensive branches remain exclusive.

A stale summary can direct later queries into empty groups. A stale max can make successor choose an incorrect branch. These fields are required invariants, not optional hints.

Recompute updates from the same data

Starting at S={2,5,6,9,12,15}:

After	min	max	cluster0	cluster1	cluster2	cluster3	summary
Initial	2	15	Empty	{1,2}	{1}	{0,3}	{1,2,3}
insert1	1	15	{2}	{1,2}	{1}	{0,3}	{0,1,2,3}
delete1	2	15	Empty	{1,2}	{1}	{0,3}	{1,2,3}
delete15	2	12	Empty	{1,2}	{1}	{0}	{1,2,3}

Deleting1 promotes2 from cluster0, then deletes local2 and removes newly empty group0 from summary. Deleting15 leaves local0 in cluster3, so summary retains3 and new max=3・4+0=12. Combining min with all clusters reconstructs every row's complete set without lost or duplicate promoted values.

Correctness and doubly logarithmic time

Induct on b. At u=2, min/max represent empty, singleton, or{0,1} exactly. Assume operations correct on smaller universes. Distinct high ranges partition values, index inverts high/low, and min is stored separately.

Membership enters its only possible group. Successor/predecessor use local extrema to test same-group existence, otherwise summary finds the nearest group, with separate min handling. Insertion swaps and deletion promotions transfer exactly one representative; summary stays equivalent to nonempty clusters. Thus representation and set semantics are preserved.

Each operation has at most one nonconstant subcall on at most ceil(b/2) bits:

$T(2^b) \le T(2^{\lceil b/2 \rceil}) + O(1)$.

For $b \ge 2$,$\lceil b/2 \rceil \le 2b/3$, so reaching b=1 needs O(log b) levels, giving O(1+log log u). This depends on the proved exclusive branches, not only structural depth.

Preallocated space recurrence

The original version builds every summary/cluster in advance. Even an empty set uses Θ(u) words and Θ(u) initialization time. Including node fields and cluster-pointer arrays, for u>2,

$$P(2^b)=2^{\lceil b/2\rceil}P(2^{\lfloor b/2\rfloor})+P(2^{\lceil b/2\rceil})+\Theta(2^{\lceil b/2\rceil}).$$

Let F(b)=P(2^b)/2^b and ε_b=2^(−floor(b/2)). For a fixed c,

$$F(b)\le F(\lfloor b/2\rfloor)+\varepsilon_b\ F(\lceil b/2\rceil)+c\ \varepsilon_b.$$

With G=F+c,

$$G(b)\le(1+\varepsilon_b)\max(G(\lfloor b/2\rfloor),G(\lceil b/2\rceil)).$$

Follow the larger term down to b=1 and bound the accumulated finite product, retaining summary cost rather than ignoring it.

Linear space and allocation limitations

Counting r levels backward along that halving path, bit sizes are at least2^(r−1)+1. Corresponding ε values are bounded by1/2,1/2,1/4,1/16,…,whose sum is bounded independently of b. Since1+x≤e^x,

$$\Pi(1+\varepsilon)\le\exp(\Sigma\varepsilon)=O(1).$$

Thus F(b)=O(1),P(u)=O(u). Cluster branches alone partition the original universe into u/2 base nodes at u=2, giving Ω(u), hence Θ(u). Initializing these fields also costs Θ(u).

For huge u and small n, preallocation can be unattractive. On-demand clusters and hash dictionaries require separate proofs of construction,dictionary time,and space. Calling allocation lazy does not establish O(n) space and worst-case O(log log u) operations simultaneously. The proved version here uses preallocated arrays and direct indexing on word-RAM.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter20.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide and conquer

Part I Foundations

Scope and prerequisites

Complete case analysis and recursive contracts establish correctness. Maximum subarrays and Strassen's seven products provide the algorithms. Fixed-constant induction, level sums, and a full master-theorem proof establish costs, with explicit rounding, logarithmic boundaries, and unequal-split assumptions. Prerequisites are recursion, array intervals, matrix-product indexing, and induction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Compute maximum contiguous subarrays by divide and conquer.
- Explain Strassen's block construction and subcubic cost.
- Analyze recurrences by substitution, recursion trees, and the master theorem.
- Identify the external Akra–Bazzi assumptions for unequal subproblem sizes and evaluate its integral.

Sections 4.1–4.6 cover maximum subarrays, Strassen, substitution, recursion trees, the master method, and its proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate recursive correctness from cost

Divide and conquer splits an input into smaller instances of the same problem, solves them recursively, and combines their answers. Correctness requires that child answers cover the original problem, followed by induction. Costs include division, combination, allocation, and copying at every call.

Use 1-based arrays and inclusive intervals. Time uses unit-cost exact real or integer arithmetic. If integer bit lengths grow, multiplication and addition cannot unconditionally remain O(1). Exact matrix algebra does not itself guarantee floating-point error bounds.

An equation $T(n) = \sum_i T(n_i) + f(n)$ describes complete cost. An inequality with $\leq$ normally supplies only an upper bound. Specify bases, strict shrinking $n_i < n$, and actual split costs.

4.1 The maximum-subarray problem

For $n \geq 1$, return a nonempty contiguous interval [i,j] maximizing $\sum_{k=i}^{j} A[k]$, together with (i,j,sum). Ties may return any maximizing interval unless another rule is specified.

For prices $P_0,\ldots,P_n$ with buying before selling, define $A_i = P_i - P_{i-1}$. Then $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$, so i..j means buying on day i−1 and selling on day j. Allowing no trade additionally permits the empty interval with profit 0. These specifications differ on all-negative inputs.

Enumerating starts and extending endpoints with accumulated sums costs $\Theta(n^2)$ time and O(1) extra space. Recomputing each interval sum from scratch instead costs $\Theta(n^3)$. Prefix sums make interval queries O(1), but leave $\Theta(n^2)$ candidates.

Crossing means suffix plus prefix

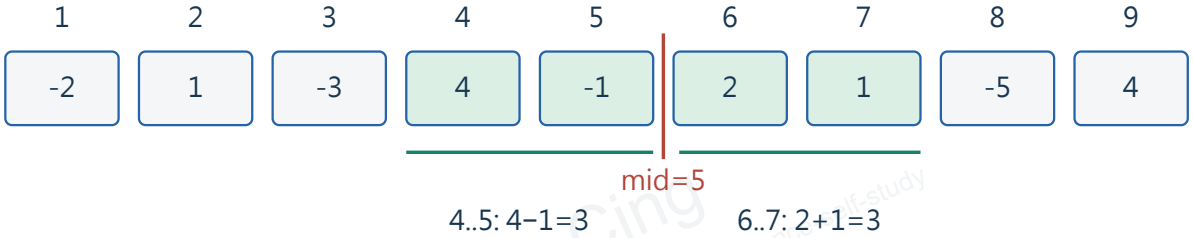
For low<high, let mid=floor((low+high)/2). Every nonempty interval belongs to exactly one class: wholly in low..mid, wholly in mid+1..high, or crossing with i≤mid<j. A crossing interval uniquely consists of left suffix i..mid and right prefix mid+1..j.

The endpoints can be chosen independently, so the best crossing sum is the best nonempty left suffix plus the best nonempty right prefix. Scan each side instead of enumerating endpoint pairs. Combining the halves' maximum subarrays is invalid because they need not reach the midpoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Crossing means suffix plus prefix (continued)

Indices and nonempty crossing interval: this call returns A[4..7]



Crossing sum 3+3=6. The left suffix and right prefix must meet at the midpoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete crossing scan

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

After a left-loop update, total equals the current i..mid sum, and left is the largest suffix sum examined. Initially there is no candidate; negative infinity prevents selecting an empty interval. Each iteration adds exactly one candidate and updates the maximum. At exit every left suffix has been considered. The right proof is identical. Total iterations are (mid−low+1)+(high−mid)=high−low+1.

Recursive maximum subarray and proof

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = floor((low+high)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Use strong induction on interval length. A singleton has only itself. For longer intervals, both nonempty children are smaller, so induction gives the first two class maxima. The crossing proof gives the third. These classes are disjoint and exhaustive; their maximum is globally optimal. Shrinking lengths also prove termination.

The recurrence is $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n)$. Sequential calls use O(log n) stack with constant indices and sums per frame. Copying subarrays requires separate allocation and live-space accounting.

Nine-element crossing trace

Let A=[−2,1,−3,4,−1,2,1,−5,4], low=1, high=9, mid=5.

Left i	Sum A[i..5]	Best sum and start
5	−1	−1, i=5
4	3	3, i=4
3	0	3, i=4
2	1	3, i=4
1	−1	3, i=4

Right prefixes ending at 6,7,8,9 have sums 2,3,−2,2, so the best is 6..7 with sum 3. Crossing 4..7 sums to 6. Left and right optima are 4..4 and 9..9, each with sum 4. Return (4,7,6). For [−8,−2,−5], return (2,2,−2), not zero.

A linear algorithm's state and invariant

Let end_i be the best nonempty interval sum required to end at i. Such an interval is A_i alone or a best interval ending at i−1 extended by A_i. Thus $end_i = \max(A_i, end_{i-1} + A_i)$ and $best_i = \max(best_{i-1}, end_i)$, with $end_1 = best_1 = A_1$. Choosing A_i resets the start to i; extension keeps the earlier start. Whenever best improves, save its start and endpoint i. Induction proves both state definitions, giving Θ(n) time and O(1) extra space including interval recovery.

For the nine-element input, end=[−2,1,−2,4,3,5,6,1,5] and best=[−2,1,1,4,4,5,6,6,6], recovering 4..7. Divide and conquer is not fastest here; it illustrates correctness through complete classes and a combination contract.

4.2 Strassen’ s algorithm for matrix multiplication

For n×n matrices, $C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}$. Three loops initialize each output to zero and add products by k. The inner invariant is the sum of the first k products. The n^2 outputs each require n multiply-adds, giving $\Theta(n^3)$ for this procedure, not a lower bound for every matrix-multiplication algorithm.

Write A=[[a,b],[c,d]], B=[[e,f],[g,h]], where letters denote n/2-square blocks. Then C_{11}=ae+bg, C_{12}=af+bh, C_{21}=ce+dg, C_{22}=cf+dh. Preserve factor order; matrix blocks generally do not commute.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full cost of eight recursive products

Pad equal-order square inputs to a power of two, then retain the original result size. Each call reads its own order from the input view's row count rather than keeping the original n.

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

A view stores an origin, stride, and order, making division O(1). Actual block copies cost $\Theta(n^2)$, still matching four block additions. The recurrence $8T(n/2)+\Theta(n^2)$, with constant base cost, gives $\Theta(n^3)$.

A constant number of additions combines into $\Theta(n^2)$, but eight recursive calls cannot become one T(n/2). The branching factor multiplies at every level and determines n^3 leaves. Absorbing local constants differs from changing tree branching.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All seven Strassen products

Compute seven products recursively; preparation uses elementwise block addition and subtraction.

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Recover $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$, and $C_{22} = M_1 - M_2 + M_3 + M_6$. At n=1 multiply scalars directly. Otherwise divide, compute these products, and assemble the outputs. This M numbering is defined here and must not be mixed with another source's P numbering.

Correctness of all four output blocks

Distributivity and additive cancellation suffice; no multiplication factors change order.

$$\begin{aligned} C_{11} &= (ae + ah + de + dh) + (dg - de) \\ &\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\ C_{12} &= (af - ah) + (ah + bh) = af + bh, \\ C_{21} &= (ce + de) + (dg - de) = ce + dg, \\ C_{22} &= (ae + ah + de + dh) - (ce + de) \\ &\quad + (af - ah) + (ce + cf - ae - af) = cf + dh. \end{aligned}$$

The scalar base is correct. If the seven smaller products are correct by induction, these identities recover all four blocks of AB. The construction needs addition, subtraction, and distributivity; it does not directly apply to a general semiring without subtraction.

Numerical products and reconstruction

Let A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. Thus a=2,b=−1,c=3,d=4,e=5,f=2,g=−2,h=1.

Product	Substitution	Value
M1	(2+4)(5+1)	36
M2	(3+4)5	35
M3	2(2−1)	2
M4	4(−2−5)	−28
M5	(2−1)1	1
M6	(3−2)(5+2)	7
M7	(−1−4)(−2+1)	5

Outputs are 36−28−1+5=12, 2+1=3, 35−28=7, and 36−35+2+7=10. Direct multiplication gives [[10+2,4−1],[15−8,6+4]]=[[12,3],[7,10]].

Numerical products and reconstruction (continued)

Seven products on the same input: preserve all four output positions

A

2	-1
3	4

B

5	2
-2	1

$M_1=36$; $M_2=35$

$M_3=2$; $M_4=-28$

$M_5=1$; $M_6=7$

$M_7=5$

C

12	3
7	10

$C_{11}=36-28-1+5=12$; $C_{12}=2+1=3$

$C_{21}=35-28=7$; $C_{22}=36-35+2+7=10$

Time, padding, and memory

View splitting is O(1). Preparing product inputs uses 10 block additions/subtractions; reconstructing outputs uses 8. Thus $T(n) = 7T(n/2) + \Theta(n^2)$, T(1)=Θ(1), giving $\Theta(n^{\lg 7})$, lg 7≈2.80735. Larger addition constants can make small matrices slower. A fixed threshold switching to ordinary multiplication preserves the exponent.

Pad arbitrary positive n to $m = 2^{\lceil \lg n \rceil}$, n≤m<2n. The upper-left product remains AB; restrict back to n×n. Since $m^{\lg 7} < 7n^{\lg 7}$, the order is unchanged. Return an empty matrix directly at n=0.

With depth-first execution, each call holds a constant number of current-level blocks, runs seven children sequentially, and releases temporaries promptly. Peak extra space satisfies $S(n) \leq S(n/2) + O(n^2) = O(n^2)$, plus O(log n) stack. Simultaneously retaining all child workspaces gives a different recurrence.

4.3 The substitution method for solving recurrences

Guess a bound, then use strong induction with the same fixed constants at every size. Increasing a constant at each level is invalid.

For $T(n) = 2T(\lfloor n/2 \rfloor) + n, T(1) = 1$, try T(n)≤cn lg n. For n≥4 put m=floor(n/2)≥2. Substitution gives

$$2cm \lg m + n \leq cn \lg(n/2) + n = cn \lg n - (c-1)n \leq cn \lg n$$

for c≥1. But n=1 gives zero on the right and cannot be a base. Instead use n=2,3: T_2=4,T_3=5 are covered by c≥2. At n≥4 the child is at least 2, closing induction. Alternatively cn(lg n+1) covers n=1. Every reached small input needs an induction or explicit base case.

Lower bounds and stronger hypotheses

For power-of-two n in T(n)=2T(n/2)+n,T_1=1, the hypothesis T(n)≥dn lg n yields dn lg n+(1−d)n. Choose 0<d≤1 and cover bases to prove Ω(n log n). Rounded trees below extend beyond powers of two.

For $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1, T_1 = 1$, the guess cn gives cn+1 and fails, although linear growth can still be correct. Strengthen to cn−d. Substitution yields cn−2d+1≤cn−d when d≥1; c≥d+1 covers the base. Indeed c=2,d=1 gives exact 2n−1.

Claiming “cn+n is still O(n)” silently replaces c by c+1 instead of proving the same hypothesis. This would incorrectly prove an n log n recurrence linear.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Variable changes and strict shrinking

For $T(n) = 2T(\sqrt{n}) + \lg n$, start on $n = 2^{2^k}$ with T_2=Θ(1). Let m=lg n,S(m)=T(2^m). Then S(m)=2S(m/2)+m, giving Θ(m log m), hence T(n)=Θ(log n log log n). New m differs from original n.

In T(n)=2T(n/2+17)+n, children shrink only for n>34. Put n=34+x,U(x)=T(34+x), obtaining U(x)=2U(x/2)+x+34. A sufficiently large constant base interval gives Θ(x log x)=Θ(n log n). Integer versions retain rounding errors. Giving only T_1 would let n=34 call itself.

The domains are explicit. Extending these proofs to all n requires rounding analysis, monotone comparison, or suitable bases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.4 The recursion-tree method for solving recurrences

A node stores only its nonrecursive f(size); children account for recursive work. Storing full T(size) at each node and adding descendants double-counts. Count depth, nodes per level, and leaves separately.

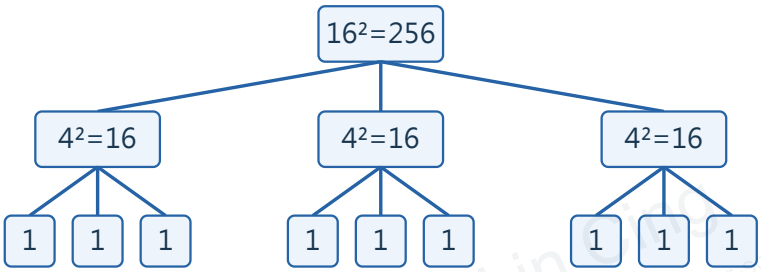
For T(n)=3T(n/4)+n^2,T_1=1,n=4^L, depth j has 3^j nodes of cost $(n/4^j)^2$, hence $n^2(3/16)^j$. Internal depths are 0..L−1; leaves number 3^L=n^($\log_4 3$).

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3} = \Theta(n^2).$$

The geometric sum is at most 16/13 and leaf cost at most n^2, giving the upper bound; the root gives lower bound n^2. At n=16, level costs 256,48,9 total 313. The nine leaves already carry base cost, so do not add f(1) again.

4.4 The recursion-tree method for solving recurrences (continued)

Three n/4 children: level-cost ratio 3/16, not 3/4



Depth 0: 1×256=256

Depth 1: 3×16=48

Depth 2: 9×1=9

T(16)=256+48+9=313. Bottom nodes are base cases; charge them once.

Unequal splits: depth and mass per level

For sufficiently large integer n put a=floor(n/3),b=n−a, and T(n)=T(a)+T(b)+cn. Handle fixed small inputs directly. Child sizes sum to n, so surviving mass at each level is at most n, giving cost at most cn.

The longest path shrinks to at most 2n/3+1 each step and has O(log n) depth; there are O(n) leaves. This gives O(n log n). A lower bound cannot assume mass n through the deepest level, because shorter paths stop earlier. For a sufficiently large fixed K absorbing rounding, no path reaches a base in the first floor($\log_3(n/K)$) levels. These complete levels each cost cn, proving Ω(n log n).

If the nonrecursive term is only O(n), the upper bound survives but the lower bound need not. Zero merge cost can leave only Θ(n) leaf work.

4.5 The master method for solving recurrences

Let fixed a≥1,b>1, positive bounded constant-size bases, and nonnegative eventually positive f define T(n)=aT(n/b)+f(n). Put p=log_b a. Start with exact powers; the next section gives rounding conditions.

Case	Required conditions	Result
1	fixed ε>0; f=O(n^(p−ε))	$\Theta(n^p)$
2	$f=\Theta(n^p)$	$\Theta(n^p \log n)$
3	fixed ε>0; f=Ω(n^(p+ε)); af(n/b)≤cf(n) eventually for fixed c<1	Θ(f(n))

Merely smaller or larger than n^p is insufficient; ε must not depend on n. Case 3 regularity makes lower levels payable by root work. Real a can be a recurrence weight; an actual count of a calls is integral.

Complete case classification examples

(a,b,f)	Exponent and check	Result
(9,3,n)	p=2; ε=1; case 1	Θ(n^2)
(1,3/2,1)	p=0; case 2	Θ(log n)
(3,4,n log n)	p≈.7925; ε=.1; 3f(n/4)/f(n)≤3/4	Θ(n log n)
(8,2,n^2)	p=3; ε=1	Θ(n^3)
(7,2,n^2)	p=lg 7; 0<ε≤lg 7−2	Θ(n^(lg 7))

The recurrence 2T(n/2)+n log n fits none of the classical cases. Its log factor grows slower than every fixed n^ε, so polynomial separation fails. Level sums still solve it; an inapplicable theorem does not mean an unsolvable recurrence.

Sum logarithmic boundary cases

For exact powers n=b^L and fixed constant-size bases, let $f(n) = \Theta(n^p(\lg n)^k)$ for fixed real k. Renumber from the leaf end by r=L−j, giving level cost Θ($n^p r^k$).

$$T(n) = \begin{cases} \Theta(n^p(\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg\lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

For k>−1, integral comparison gives $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$. At k=−1 the harmonic sum is Θ(log L). For k<−1 it converges to a positive constant; leaves still give Θ(n^p). The standard k≥0 extension is a subset. Use logs above the base interval to avoid negative powers of log 1.

Thus 2T(n/2)+n/log n gives Θ(n log log n), not Θ(n); 9T(n/3)+n^2log^{2}n gives Θ(n^2log^{3}n).

A counterexample without regularity

For $n=2^L$,$T_1=1$, let $f(n)=n^2$ for even L and n^3 for odd L. With a=b=2,p=1, $f \geq n^2$ gives polynomial separation, but T need not be $\Theta(f)$.

For even L,$n \geq 4$, the child's exponent is odd. Then $T(n) \geq 2f(n/2) = n^3/4$, while $f(n)=n^2$, giving unbounded $T/f \geq n/4$. The regularity ratio itself is $2f(n/2)/f(n) = n/4$.

This shows failure after omitting a hypothesis, not failure of the theorem. A one-unit rounding error also need not preserve arbitrary oscillatory f values; conditions must cover actual rounded arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6 Proof of the master theorem

For $n=b^L$,$T_1=t>0$, expand L times or induct on tree depth.

$$T(n) = a^L T(1) + \sum_{j=0}^{L-1} a^j f(n/b^j) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

The positive leaf term tn^p gives a lower bound in every case. In case 1, $f(x) \leq Cx^{(p-\varepsilon)}$ bounds depth-j work by $Cn^{(p-\varepsilon)}(b^\varepsilon)^j$. Its ratio $b^\varepsilon > 1$, so the sum is $O(n^{(p-\varepsilon)}(b^\varepsilon)^L)=O(n^p)$. The fixed final levels where the bound may fail also cost $O(n^p)$, absorbing finitely many f values. Adding leaves gives $\Theta(n^p)$.

The geometric sum increases toward leaves. For a=b=2,ε=1/2, the ratio is $\sqrt{2}>1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal-level and root-dominated proofs

Case 2 uses $a=b^p$, making each large-size level cost $a^j\Theta((n/b^j)^p)=\Theta(n^p)$. There are $L-O(1)=\Theta(\log n)$ such levels. Their two-sided sum gives $\Theta(n^p \log n)$, covering leaf work.

In case 3, repeated regularity gives $a^j f(n/b^j) \le c^j f(n)$ while arguments remain sufficiently large. Their sum is at most $f(n)/(1-c)$. Remaining fixed-size levels and leaves cost $O(n^p)$; polynomial separation makes this $O(f(n))$. The root gives lower bound $f(n)$, proving $\Theta(f(n))$.

Small inputs do not cost $O(1)$ across the whole tree: $\Theta(n^p)$ such nodes may exist. Charging them to leaf order explicitly prevents omission. Every case gives both bounds.

Rounding errors remain bounded

Along a path, $q_0=n$ and q_{j+1} is $\text{floor}(q_j/b)$ or $\text{ceil}(q_j/b)$, so $|q_{j+1}-q_j/b| \le 1$. With $C \ge b/(b-1)$, induction gives

$$|q_j - n/b^j| \le C,$$

because the next error is at most $C/b+1 \le C$. Choose base interval $q \le n_0$ with $n_0 > C+2$ and large enough to ensure strict shrinking. Every path reaches a base at depth $\log_b n+O(1)$. More than a constant number of levels above leaves, $n/b^j \ge 2C$, so q_j and n/b^j differ by fixed factors.

With a actual children per node, path depths differ by a constant, giving $\Theta(a^{\log_b n})=\Theta(n^p)$ leaves. Fixed-size bottom levels cost $O(n^p)$. For $a=1$ this is one path with a constant number of bottom nodes, consistent with the formula.

Conditions for all integer sizes

Case 1 uses $q_j=\Theta(n/b^j)$ above fixed final levels. Powers remain within constant factors even when $p-\varepsilon<0$. Applying f's polynomial upper bound gives the same increasing geometric sum. Add $O(n^p)$ bottom work.

Case 2 similarly has $f(q_j)=\Theta((n/b^j)^p)$. There are $\Theta(\log n)$ complete large-size levels of cost $\Theta(n^p)$; fixed incomplete bottom levels do not change the bound. For logarithmic extensions, $\log q_j$ is comparable to distance r from the bottom, yielding the same three sums.

Case 3 needs regularity on actual children. A sufficient rounded condition is

$$a\max\{f(\lfloor x/b\rfloor),f(\lceil x/b\rceil)\}\le cf(x),\quad c<1.$$

Then total work contracts by c at each level. Add $O(n^p)$ bottom work and use polynomial separation. Bounded argument errors alone do not imply this condition for arbitrary f.

Regularity can imply a polynomial gap

On exact powers, suppose $af(x/b)\le cf(x)$,$0<c<1$, with fixed bottom $f(x_0)>0$. Reversing the inequality k times gives $f(b^kx_0)\ge(a/c)^kf(x_0)$. Since $\log_b(a/c)=p+\log_b(1/c)$, this gives $f(n)=\Omega(n^{(p+\varepsilon)})$, $\varepsilon=\log_b(1/c)>0$.

This explains formulations that omit a separate gap under positive well-behaved bases. Here the full standard hypotheses remain explicit. The derivation is on exact powers; other domains need control of the terminal interval and its positive lower bound.

The master theorem and integer extension have now been proved. The unequal-ratio tool below is a cited external theorem.

Akra–Bazzi for unequal subproblems

The master theorem uses a equal-size n/b subproblems. Akra–Bazzi permits finitely many fixed ratios. This extends beyond CLRS third-edition Chapter 4. Use the continuous form from MIT 6.042J, with positive bounded bases as sufficient assumptions.

For $0 \le x \le x_0$ require fixed $0 < d_1 \le T(x) \le d_2$. For $x > x_0$ define

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x).$$

Require all of the following.

- Fixed finite k, fixed $a_i > 0$, fixed $0 < b_i < 1$.
- Nonnegative differentiable g, with $|g'(x)| \le D x^C$ eventually for fixed C,D. Examples also have g continuous on the base interval.
- $|h_i(x)| = O(x / \log^2 x)$ for every i.
- A sufficiently large x_0 puts every child in $[0, \rho x]$ for fixed $\rho < 1$, ensuring defined recursion and eventual base entry.

Akra–Bazzi for unequal subproblems (continued)

For the unique real p satisfying $\sum_i a_i b_i^p = 1$, the theorem gives

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

Positive bases pay for leaves. Allowing $T \equiv 0, g \equiv 0$ would invalidate the positive Θ lower bound. This theorem is cited, not proved here. The following arguments derive its root, integral, and applicability.

Rounding versus changed ratios

Floor and ceiling differ from $b_i x$ by at most 1, satisfying the perturbation bound. Fixed additive constants do too. The base interval must still ensure shrinking.

A perturbation εx for fixed $\varepsilon > 0$ is not $O(x/\log^2 x)$. It changes the ratio and requires a new root. The exponent depends on actual sizes, not merely two recursive calls.

Sources using $x/c_i, c_i > 1$ correspond to $b_i = 1/c_i$. Their equations $\sum_i a_i c_i^{-p} = 1$ and $\sum_i a_i b_i^p = 1$ are equivalent. Keep parameter definitions consistent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unique exponent and integral

Let $F(p) = \sum_i a_i b_i^p$. Each positive term decreases strictly in p. F tends to infinity at negative infinity and zero at positive infinity. Continuity gives exactly one real root of F=1. $F(0) = \Sigma a_i$ determines whether the root is positive, zero, or negative. It can be fractional; the theorem gives an order, not an exact closed form.

Set $T(x) = 1$ throughout $0 \le x \le 1$ and $T(x) = T(x/3) + T(2x/3) + x$ above 1. Then $a_1 = a_2 = 1, b_1 = 1/3, b_2 = 2/3, g = x$. Substitution gives the unique root $p = 1$ of $(1/3)^p + (2/3)^p = 1$. Thus

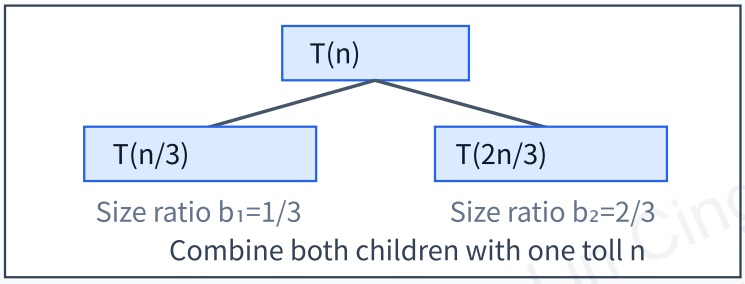
$$T(n) = \Theta\left(n\left[1 + \int_1^n u/u^2\,du\right]\right) = \Theta(n(1 + \ln n)) = \Theta(n \log n).$$

The general theorem supplies both bounds without finite-instance enumeration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unique exponent and integral (continued)

Unequal children: T(n)=T(n/3)+T(2n/3)+n



Characteristic equation

$(1/3)^p+(2/3)^p=1$

Strictly decreasing; unique root p=1

$\int_1^n u/u^2\,du = \ln n$

$T(n)=\Theta(n \log n)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Oversized perturbations change the exponent

For T(n)=T(n/2+n/4)+T(n/2)+n, treating both ratios as 1/2 would predict p=1 and Θ(n log n). But n/4 is not an O(n/log^(1+ε)n) perturbation.

Rewrite as T(n)=T(3n/4)+T(n/2)+n. The equation $(3/4)^p + (1/2)^p = 1$ has numerical root p≈1.507126591639. The integral yields $\Theta(n^p)$, not Θ(n log n). Check actual children and perturbation bounds before choosing ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Choosing a recurrence tool

1. For a children of common size n/b, check master-theorem cases and regularity.
2. For finitely many fixed ratios such as n/3 and 2n/3, check Akra–Bazzi, solve p, then integrate.
3. Floor, ceiling, and $O(n/\log^2 n)$ perturbations can retain ratios. Fixed-proportion perturbations require true new ratios.
4. Negative g, missing smoothness, or undefined bases leave theorem hypotheses unmet. Use expansion or direct induction.

The unequal children in T(n/3)+T(2n/3)+n are precisely what the ordinary master theorem cannot directly express.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 4.

M. Akra and L. Bazzi. "On the Solution of Linear Recurrence Equations." *Computational Optimization and Applications* 10(2), 1998. doi:10.1023/A:1018353700639.

K. Mehlhorn. *A Master Theorem for Recurrences*. Max Planck Institute for Informatics, 2008, Theorems 1–3.

MIT 6.042J, *Mathematics for Computer Science*, Fall 2010, § 10.4.3, Theorem 10.4.1. Official notes. This chapter uses positive bounded bases as sufficient assumptions.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort

Part II Sorting and Order Statistics

Scope and prerequisites

Partitioning establishes relations between regions and a pivot; recursion is still needed to sort. Follow Lomuto's invariant through every numerical state, then distinguish worst-case time, expected comparisons over internal randomness, and stack depth, including duplicate-key degeneration. Prerequisites are array partitioning, recursion, and expectation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Implement partition with its invariant.
- Distinguish worst, best, and expected Quicksort costs.
- Analyze random pivots and adversarial input order.

Sections 7.1–7.4 cover the algorithm, performance, randomization, and analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Choose a pivot, separate other keys by their relation to it, and recursively sort both sides. Partition alone does not order keys within a side. Use 1-based Lomuto partition with pivot at the final position r.

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Empty and singleton intervals are bases. Both child intervals exclude q and strictly shrink. Including q again can cause infinite recursion when the pivot reaches an endpoint.

Loop boundaries: i ends the small region; j is next

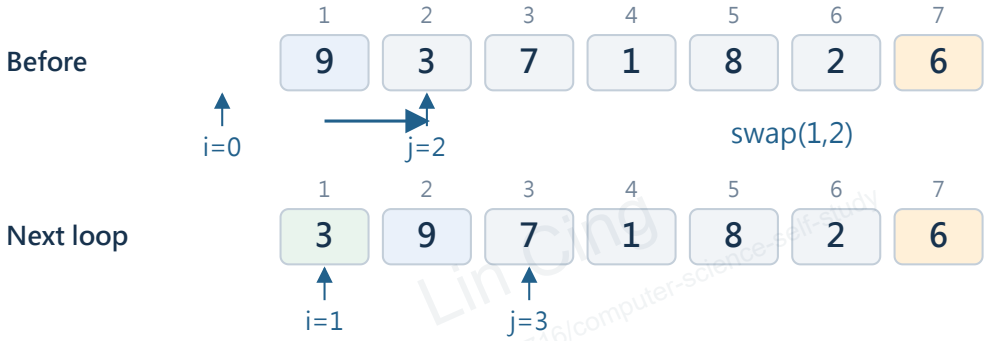
The pivot stays at A[r]. Use **1-based indices and the inclusive interval A[p..r]**. The boundary i=p−1 means that the ≤ region is empty.

Region	Known relation	Initially i=p−1, j=p
A[p..i]	≤ pivot	Empty
A[i+1..j−1]	> pivot	Empty
A[j..r−1]	Unclassified	All non-pivot keys
A[r]	Pivot	Fixed until the final exchange

Diagram key: green ≤, blue >, gray unknown, orange pivot. Both transition diagrams use [9,3,7,1,8,2,6] with pivot 6.

A small key: exchange, then advance both boundaries

3 ≤ 6: advance i, then exchange A[1] and A[2]



3 joins ≤; displaced 9 stays in >. Pivot 6 remains at index 7.

For A[2]=3, advance i to 1 and exchange A[1],A[2]. The key 3 joins ≤; displaced 9 stays in >. Finally advance j. In general i may equal j, giving a harmless self-exchange.

A large key: leave values fixed and advance j

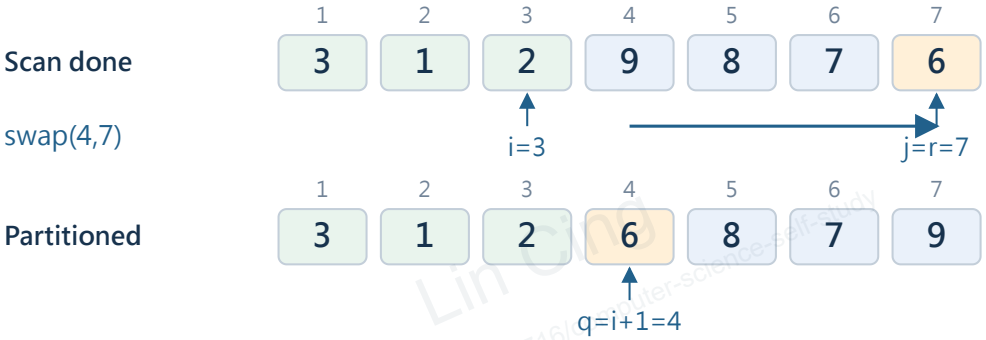
7 > 6: no exchange; keep i and advance j

7 moves from unknown to >. Values stay fixed; the boundary moves.

Since A[3]=7>6, leave i and the array unchanged. Advancing j adds 7 to the > region. **The values stay fixed; our classification changes.**

The final exchange establishes the pivot position q

Only after scanning: place the pivot between the two regions



≤6: [3,1,2] | pivot 6 | >6: [8,7,9]; the sides remain unsorted.

The unknown length r−j decreases once per loop, proving termination. At j=r, exchange A[i+1],A[r] and return q=i+1. The left region is ≤pivot and the right region >pivot; neither side is necessarily sorted.

From the partition invariant to sorting correctness

1. **Initialization:** i=p−1,j=p makes both classified regions empty.
2. **Maintenance:** a small key expands ≤ after an exchange; a large key expands >. Either branch removes one unknown key and leaves the pivot fixed.
3. **Partition termination:** no unknown keys remain. The final exchange places the pivot at q. Exchanges preserve the input multiset.
4. **Recursive induction:** empty and singleton intervals are sorted. Both children exclude q and are strictly shorter. Sorting them and adjoining the pivot therefore sorts the whole interval.

The ordering relation alone is insufficient: the proof also needs **multiset preservation, strictly smaller recursive intervals**, and sorted children upon return.

Complete partition trace

Let A=[9,3,7,1,8,2,6],p=1,r=7,pivot=6, initially i=0.

j	Test	New i	Array
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6], swap 1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6], swap 2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6], swap 3,6
End	place pivot at i+1	q=4	[3,1,2,6,8,7,9], swap 4,7

Left [3,1,2] is ≤6 and right [8,7,9] is >6; neither is sorted internally. Partition performs exactly r−p comparisons, Θ(r−p+1) time, and O(1) extra space.

Worst-case time: a family attaining quadratic work

Sorted distinct input with a fixed final pivot always chooses the maximum. The two child lengths are n−1 and 0.

Current length	Key comparisons	Next nonempty length
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

The sum is $n(n-1)/2$, proving a worst-case Ω(n^2) lower bound for an entire input family, rather than extrapolating from one measurement.

The quadratic upper bound must cover every split

Let C(n) be the worst comparison count. Child sizes satisfy a+b=n−1. Induction on smaller sizes gives

$$C(a) + C(b) + (n-1) \le \frac{a(a-1)}{2} + \frac{b(b-1)}{2} + n - 1 = \frac{n(n-1)}{2} - ab \le \frac{n(n-1)}{2}.$$

With C(0)=C(1)=0, this covers all splits. Equality holds when a=0 or b=0. Each pivot leaves recursion, so there are O(n) calls. Total time is O(n+C); the matching family proves **worst-case Θ(n²)**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Balanced splits: count the comparison work by level

Take n=15 and always choose the true median. Count only Lomuto key comparisons; singletons do not call partition.

Depth d	Subproblems	Size each	Comparisons at level
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

Total: 34 comparisons, versus 105 for endpoint splits on 15 keys. Partition still scans linearly; the difference is how much work survives at each depth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed split ratio also gives logarithmic depth

If the larger child always has size at most ρ times the parent size, for a fixed ρ<1, the longest branch has O(log n) levels. Intervals at one depth are disjoint, giving at most n scanned keys per level and O(n log n) total work. A fixed 1:9 split suffices.

For perfectly balanced $n = 2^k - 1$, depth d has 2^d subproblems of length $2^{k-d} - 1$. Its comparison work is

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

The first roughly k/2 levels each require Θ(n) comparisons, proving Ω(n log n) for that execution and hence Θ(n log n) overall. This lower bound counts actual recursive work; a comparison-sorting worst-input lower bound does not apply to every individual input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicate keys defeat this two-way partition

On [5,5,5,5,5], every ≤ test succeeds and q=r, leaving n−1 keys on the left. Randomly exchanging a pivot to the end still exchanges a 5, giving ten comparisons here and n(n−1)/2 generally.

The following O(n log n) random-pivot analysis therefore assumes distinct keys. For many duplicates, use <,=,> regions and never recurse on the equal region. Chapter 9 gives full interval boundaries and rank conversions.

Random pivots do not remove this duplicate-key premise. Stability is separate: long-distance swaps can change equal-key identity order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion space and the smaller-side transformation

Two recursive calls can produce n stack levels. In-place partitioning does not make the whole algorithm unconditionally constant-space.

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

The actual recursive call has size at most (n−1)/2. The larger side continues in the current frame through a while loop. Thus sizes halve along a frame chain, giving worst-case O(log(n+1)) stack. Every partition still executes, and worst time remains $\Theta(n^2)$; reducing stack does not prove O(n log n) time.

7.3 A randomized version of quicksort

On current A[p..r], choose uniform k in p..r, swap A[k],A[r], then apply the same partition. Repeat uniform selection on every current child. With distinct keys, pivot rank is uniform in 1..n, so every fixed input order has the same expected analysis.

Expectation concerns internal random choices, not a uniform user-data assumption. An execution can keep selecting endpoints and cost $\Theta(n^2)$. The next proof establishes an expectation, not a per-execution or an unproved tail bound.

The ideal model requires uniform sampling conditional on current state. A fixed public pseudorandom sequence is a reproducible implementation choice whose adversarial guarantees need separate justification. A random API call by itself is not a proof of mathematical uniformity.

Which pair compares? Watch the first pivot within its rank interval

Name distinct keys by sorted order $z_1<\ldots<z_n$. Let $X_{ij}=1$ when z_i and z_j compare in any partition. A pair compares at most once: one endpoint is the pivot and then leaves recursion.

Consider $Z_{ij}=\{z_i,\ldots,z_j\}$. An outside pivot keeps this entire interval in one child. Until the first pivot from the interval appears, both endpoints therefore remain together.

Sorted ranks for analysis: $Z_{2,5}$ = {2,3,4,5}; not an array snapshot

Pivot 7 first: ranks 2..5 remain together in its left child; $X_{2,5}$ is undecided.

First within interval: endpoint 2 (or 5)

This partition compares 2 and 5 → $X_{2,5}=1$

First within interval: interior 3 (or 4)

2 left, 5 right → never compare, $X_{2,5}=0$

Four interval keys are equally likely to be first; two are endpoints. $P(X_{2,5}=1)=2/4=1/2$.

Endpoint first and interior first give different outcomes

For z_2,z_5, ignore outside pivots and inspect the first chosen key from {2,3,4,5}.

First interval pivot	What happens to 2 and 5?	$X_{2,5}$
2	This partition directly compares them	1
3	They enter separate children and never meet again	0
4	They enter separate children and never meet again	0
5	This partition directly compares them	1

Each current subproblem chooses a uniform pivot; outside choices do not distinguish the four interval keys. Each is therefore equally likely to be first within the interval. In general,

$$P(X_{ij}=1)=\frac{1}{j-i+1}+\frac{1}{j-i+1}=\frac{2}{j-i+1}.$$

The denominator is the interval length j−i+1, **not the full input length n**. The endpoint need not be the first pivot of the entire execution.

Expected comparisons: sum by rank-interval length

The total is $C = \sum_{i<j} X_{ij}$. Linearity of expectation needs no independence. Set k=j−i+1; exactly n+1−k rank intervals have length k. Thus

$$\begin{aligned} E[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)(H_n - 1) - 2(n-1) \\ &= 2(n+1)H_n - 4n, \quad H_n = \sum_{k=1}^{n} \frac{1}{k}. \end{aligned}$$

The bounds $\ln(n+1) \le H_n \le 1 + \ln n$ give Θ(n log n) expected comparisons. Loop work has the same order; adding O(n) call overhead gives Θ(n log n) expected time.

This uses linearity alone. Comparison events are dependent, so variance or tail formulas for independent events do not automatically apply.

The same name partition can hide a different contract

Compare the contracts specified in these two lessons:

Item	Here: Lomuto	Data Structures Chapter 7: three-way
Indices and interval	1-based inclusive A[p..r]	0-based half-open A[l:r]
Return value	One pivot position q	Equal interval [lt,gt)
Recursive children	A[p..q−1], A[q+1..r]	A[l:lt], A[gt:r]
Equal keys	Non-pivot equal keys may recurse	The whole equal region leaves recursion

Our exact formula counts only `A[j] <= pivot`, excluding index tests, exchanges, and random generation. A three-way implementation testing < and then > may use two Boolean comparisons per classification. **Derive its exact constants separately** instead of transferring a count based on a function name.

With distinct keys and uniform pivots, both versions take expected Θ(n log n) time. All-equal input still costs $\Theta(n^2)$ with two-way Lomuto, but one three-way partition finishes in Θ(n).

All comparison probabilities for four ranks

Pair	Interval size	Probability
(1,2),(2,3),(3,4)	2	1 each
(1,3),(2,4)	3	2/3 each
(1,4)	4	1/2

The sum is 3+4/3+1/2=29/6, matching $2 \cdot 5 \cdot H_4 - 16$. Endpoints 1,4 separate if pivot 2 or 3 comes first and compare if 1 or 4 comes first. Each first choice has probability 1/4.

Adjacent ranks always comparing does not imply every pair compares. Nor is 29/6 the cost of one execution; four keys can require six comparisons in the worst case. It is an average over internal random choices.

Further reading: CLRS Chapter 7, § 7.1 for the partition invariant and § 7.4.2 for the random-pivot comparison analysis.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Medians and order statistics

Part II Sorting and Order Statistics

Scope and prerequisites

Selection finds one rank without sorting everything. Begin with exact extreme-value comparison counts, unify three-way regions and local ranks, prove expected linear random selection, and derive worst-case linear time from actual five-element exclusion witnesses. Prerequisites are selection, partitioning, and probabilistic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Find both extremes with ceil(3n/2)−2 comparisons and prove a matching worst-case lower bound.
- Prove expected linear randomized selection.
- Use grouped medians for a worst-case linear guarantee.

Sections 9.1–9.3 cover extremes, expected linear selection, and worst-case linear selection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

9.1 Minimum and maximum

The k-th smallest value is position k of the sorted multiset, with $1 \leq k \leq n$. Duplicate occurrences occupy separate positions but may return the same value. The lower median for even n uses k=ceil(n/2), not necessarily the average of the central values. Keys must form a total order. Reject empty inputs and invalid k at entry.

A minimum scan maintains the smallest value of the read prefix and uses n−1 comparisons. For a lower bound, every identity except the actual minimum must be shown greater than something. One comparison can first eliminate at most one minimum candidate, requiring at least n−1 comparisons. Maximum is symmetric.

Both extremes need not cost 2n−2. Compare pairs first; only the smaller challenges min and only the larger challenges max. For even n initialize from the first pair with one comparison. For odd n initialize both from the first record.

Complete paired comparison count

For [14,3,9,1,8,5,12,6], the first comparison gives min=3,max=14. Each further pair uses one internal, one min, and one max comparison.

Pair	Smaller, larger	New min	New max	Total comparisons
14,3	3,14	3	14	1
9,1	1,9	1	14	4
8,5	5,8	1	14	7
12,6	6,12	1	14	10

Even n gives 1+3(n−2)/2=3n/2−2; odd n gives 3(n−1)/2. Both equal ceil(3n/2)−2, including zero at n=1. After each pair the extrema cover a longer prefix, proving initialization, preservation, and exit just as for a scan, while sharing comparison information between the two tasks.

Matching worst-case lower bound

For distinct keys, use an adversary whose answers remain compatible with a total order. Each identity initially has both min and max candidacy. At exit only the true min's min candidacy and true max's max candidacy remain, so 2n−2 candidacies must disappear.

Comparing two never-compared keys can eliminate two: the larger loses min and the smaller loses max. At most floor(n/2) such comparisons exist. In every other comparison the adversary can eliminate at most one new candidacy.

Before comparison	Answer rule
Both max-only candidates	Choose a greater one; eliminate the other's max status
Both min-only candidates	Choose a smaller one; eliminate the other's min status
One max-only, one min-only	Declare max-only greater; eliminate neither
One never compared	Eliminate one of its statuses without another new loss

Matching worst-case lower bound (continued)

Keys with neither status cannot cause double elimination. The rules preserve the comparison partial order: remaining max candidates have never lost; min candidates have never won. Placing the former above the latter creates no directed cycle. Uncompared keys have no previous constraints.

With c comparisons, at most c+floor(n/2) statuses disappear. Thus $c \geq 2n-2-\text{floor}(n/2)=\text{ceil}(3n/2)-2$. The paired method attains this bound.

A tournament retains second-smallest candidates

Organize minimum comparisons as a balanced knockout tournament and save each winner's direct opponents. The second smallest must have lost directly to the minimum. Anyone else lost to another nonminimum key that is smaller, precluding second place.

For the eight-key example, first-round winners are 3,1,5,6; second-round winners 1,5; final winner 1. Its direct opponents are 9,3,5, whose minimum is 3. Seven tournament comparisons plus two list comparisons total nine.

For general $n \geq 2$, balanced scheduling with byes, never comparisons to fictitious values, uses $n-1$ comparisons. The champion has $d \leq \text{ceil}(\log_2 n)$ actual opponents; another $d-1$ comparisons suffice. Worst total is at most $n+\text{ceil}(\log_2 n)-2$. Assume distinct keys, or use (key,identity) to define the second identity under ties.

9.2 Selection in expected linear time

Select recurses on only the answer side; Quicksort uses both. To handle duplicates, use three-way partition throughout. Array interval p..r is **0-based**, but k is always the **1-based rank within the current interval**.

```
partition3(A,p,r,x):
    lt=p; i=p; gt=r
    while i<=gt:
        if A[i]<x: swap(A[lt],A[i]); lt++; i++
        elif A[i]>x: swap(A[i],A[gt]); gt--
        else: i++
    return lt,gt
```

Before a test, A[p..lt−1]<x,A[lt..i−1]=x,A[i..gt] is unknown,A[gt+1..r]>x. A small-key swap expands the smaller and equal regions. A greater-key swap brings an unknown value from gt, so i must not advance. Equality only expands the equal region. Unknown length gt−i+1 decreases each iteration, proving termination. Swaps preserve the multiset, giving complete regions at exit, without internal sorting.

Subtract two regions only when going right

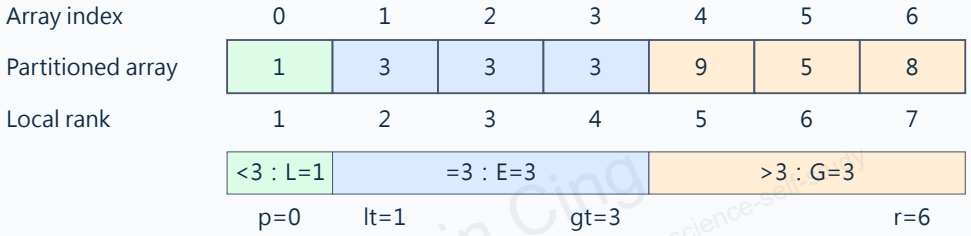
```
select-random(A,p,r,k):
    assert 1<=k<=r-p+1
    while p<r:
        x=A[uniform-integer(p,r)]
        lt,gt=partition3(A,p,r,x)
        L=lt-p; E=gt-lt+1
        if k<=L: r=lt-1
        elif k<=L+E: return x
        else: k-=L+E; p=gt+1
    return A[p]
```

The pivot is a current record, so $E \geq 1$. A left branch has $k \leq L$; a right branch has $1 \leq k-L-E \leq r-gt$. Neither can select an empty interval, and size drops by at least E. Sorted local ranks L+1..L+E all equal x.

Subtract two regions only when going right (continued)

Three-way partition: distinguish boundaries from local ranks

Input [8,3,5,3,9,1,3], pivot 3. Indices start at 0; local ranks at 1.



k=1: rank 1 left. k=2..4: return 3. k=6: rank 6−L−E=2 right.

Green <, blue =, orange >; interiors unsorted. For nonzero p, L=lt−p; lt is not a rank.

For A=[8,3,5,3,9,1,3],x=3, lengths are L=1,E=3,G=3. Rank 4 returns 3; rank 1 stays rank 1 left; rank 6 becomes rank 6−1−3=2 right. Absolute positions and local ranks differ. Another input [3,3,1,3,2,3] can become [1,2,3,3,3,3], with inclusive equal interval (lt,gt)=(2,5).

A selection that continues right

For A=[7,2,9,4,5],k=3, choosing x=2 gives empty smaller region, equal [2], and greater records [7,9,4,5]. With L=0,E=1, continue right looking for local rank 2, not 3.

If the next pivot is 5, the retained region divides into [4],[5],[7,9]. Its L=1,E=1 put k=2 in the equal region, returning 5. Discarded region interiors need not be sorted.

All-equal [5,5,5,5,5] ends after one $\Theta(n)$ scan. Two-way $\leq$ Lomuto selection for the minimum instead repeatedly has q=r, making 4+3+2+1=10 pivot comparisons and $\Theta(n^2)$ generally. Random choice among identical 5s cannot change that degeneration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete expected-linear substitution proof

Conditional on the current array, each pivot identity is uniform; future selections remain conditionally uniform. For distinct keys the rank R is uniform in 1..n. The retained length is at most m_R=max(R−1,n−R).

Let F(A,k) be expected remaining time. Assume every shorter state obeys $F(B,j) \leq a|B|$, and current scanning plus fixed overhead costs at most cn. Condition on each pivot and apply induction:

$$F(A,k) \leq cn + \frac{a}{n}\sum_{r=1}^{n}\max(r-1,n-r).$$

No monotonicity of an unknown T or independence of dependent costs is assumed. For n=2h, average m is $(3h^2-h)/(2h)=3n/4-1/2$. For n=2h+1 it is $(3h^2+2h)/(2h+1)\leq 3n/4$. Thus $F\leq(c+3a/4)n\leq an$ when $a\geq 4c$, also enlarged to cover n=1. The first scan gives $\Omega(n)$, proving expected $\Theta(n)$.

Replacing this argument by $E[T(n)]\leq E[T(3n/4)]+cn$ would be unjustified: an individual pivot can be extreme. The proof averages all conditional continuation bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicates and the worst-case boundary

The same proof handles duplicates by assigning equal keys a fixed identity order **only for analysis**, defining a unique pivot rank R. Actual partition still compares keys alone. The <x region has length at most R−1; the >x region at most n−R. Landing in =x ends immediately. Every pivot identity therefore has the same m_R bound and the same average induction; identities need not actually be sorted.

Worst execution remains $\Theta(n^2)$: with distinct keys, seek the minimum and happen to choose the maximum every time. Sizes n,n−1,… give the bound. A small but possible event cannot be excluded by expected linearity. Iterative selection avoids recursive frames; in-place partition plus fixed variables uses O(1) extra space.

Sorting first is a valid O(n log n) approach to medians or Top-k, but is not necessary. Returning one order statistic does not also produce a full sorting, so the sorting lower bound does not apply.

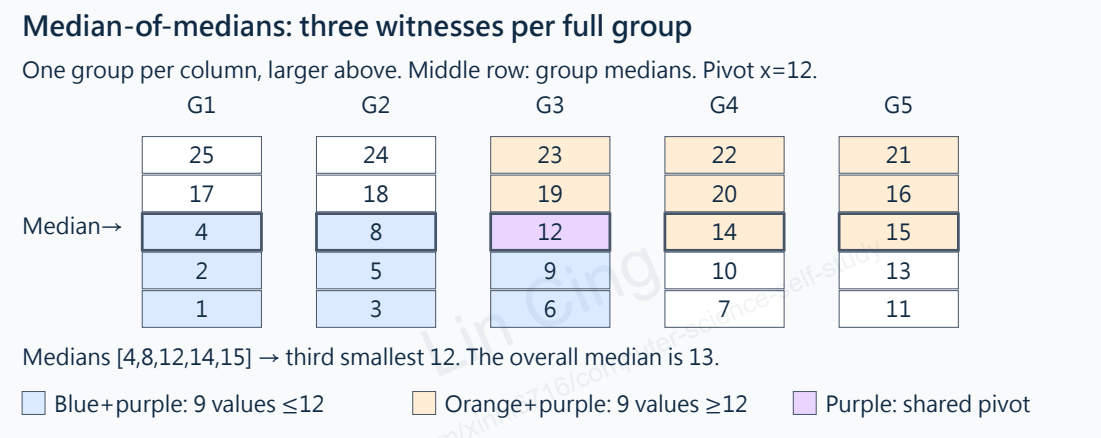
9.3 Selection in worst-case linear time

Deterministic select uses median-of-medians. For $n \leq 5$ sort directly and return rank k. Otherwise group current positions into sets of five, allowing one shorter final group. Sort each and take its lower median. Recursively select rank ceil(g/2) among these g=ceil(n/5) medians to choose x.

Use the same three-way partition. Return x if k is equal-region rank; otherwise recurse into the strict smaller or greater side with the same local-rank conversion. Both the pivot-selection recursion and answer recursion contribute to cost.

Each constant-size group sorts in O(1), totaling O(n). The pivot is an original value, so $E \geq 1$. Partition invariants and induction establish correctness and termination. The following exclusion count establishes efficiency separately.

Five-element exclusion witnesses



The five sorted groups have medians [4,8,12,14,15]. Their median is 12, whereas the entire input's true median is 13. The pivot needs a sufficiently balanced split, not an immediate answer.

Five-element exclusion witnesses (continued)

The first three groups have medians ≤ 12, each providing three values ≤ 12, giving nine witnesses. The last three have medians ≥ 12, each providing three values ≥ 12. The two witness sets may overlap at the pivot; they bound separate alternative recursive branches.

With duplicates the guarantees are nonstrict $\leq x$ and $\geq x$. A strict-side recursion excludes the full equal region, so these witnesses still bound its size.

Rounding, short groups, and linear induction

At least ceil(g/2) group medians are ≥x and at least ceil(g/2) are ≤x, even with a lower median at even g. Conservatively remove the possible short group and the chosen pivot's group. At least ceil(g/2)−2 full groups remain, each supplying three nonstrict witnesses.

Each strict recursion therefore excludes at least 3(ceil(g/2)−2)≥3n/10−6 records, leaving at most 7n/10+6. A negative exclusion lower bound for small n is merely uninformative; it does not mean a growing subproblem.

With nonrecursive work ≤cn and induction T(m)≤am for smaller sizes, n≥140 gives

$$T(n) \le a\lceil n/5\rceil + a(7n/10+6) + cn \le (9a/10+c)n + 7a.$$

Choose a≥20c. Then a/10−c≥a/20, and n≥140 pays 7a, giving T≤an. Increase a to cover finitely many sizes below 140. This is a proof threshold, not a requirement to sort 140-record bases in the implementation. Both recursive sizes are smaller, making induction valid. Processing n records gives Ω(n), hence worst-case Θ(n).

Complete path to the sixteenth smallest

Use stable scan-order lists L/E/G for readability. In-place swaps may reorder within regions but preserve lengths and rank conversions.

Current n,k	Group medians → pivot	L,E,G sizes	Next step
25,16	[4,8,12,14,15]→12	11,1,13	rank 4 in greater region
13,4	[19,20,16]→19	6,1,6	rank 4 in smaller region
6,4	[15,16]→lower median 15	2,1,3	rank 1 in greater region
3,1	base sort [16,17,18]	—	return 16

The first greater list is [17,25,18,24,19,23,14,20,22,13,15,16,21]. Groups of lengths 5,5,3 give medians 19,20,16. The subsequent smaller list is [17,18,14,13,15,16]; its first five have median 15 and its singleton median 16. Pivot 15 leaves greater list [17,18,16]. The answer 16 agrees with position 16 of a full sorting used only as a check.

Different group sizes require new proofs

For fixed odd group size s=2t+1, an idealized large-n half of group medians supplies t+1 excluded records per group. The two recursive size fractions sum approximately to

$$1/s + 1 - (t+1)/(2s) = 1 + (1-t)/(2s).$$

For s=5 this is 9/10<1, leaving induction slack. For s=7 it is 6/7<1, also permitting linearity after recomputing constants and short groups. At s=3 it is 1. The coarse recurrence T(n)≤T(n/3)+T(2n/3+O(1))+O(n) proves O(n log n), but does not alone prove an actual Ω(n log n) runtime. A nonlinearity claim needs a matching family sustaining bad splits across recursive levels.

A straightforward list implementation stores groups, medians, and L/E/G with O(n) peak extra space, since active recursive path sizes shrink geometrically. In-place O(1) space would require additional median movement and stack design; it is not part of the stated time theorem.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 9.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomials and FFT

Part VII Selected Topics

Polynomial multiplication becomes pointwise multiplication after evaluation at distinct points. We derive the DFT and its inverse from root orthogonality, then check the even/odd split, bit reversal, butterfly indices, and zero padding. Exact arithmetic and floating-point arithmetic require different guarantees.

Prerequisites are complex roots, value representations, and recursive divide and conquer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

- Compare multiplication in coefficient and value representations.
- Derive the DFT and inverse DFT.
- Implement radix-2 FFT and state the conditions on the roots of unity.

The sections cover polynomial representations, the DFT and FFT, and efficient FFT implementations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coefficient representation

A polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ has **degree bound** n, meaning $\deg A < n$. Its leading coefficients may be zero, so its actual degree may be smaller. Code may represent the zero polynomial by `[0]`. The degree bound is not the location of the first nonzero entry. Coefficients here run from low degree to high degree.

Addition of equal-length coefficient vectors takes $\Theta(n)$. Horner evaluation at z starts with $v = a_{n-1}$ and updates $v = a_j + zv$ for $j = n-2, \ldots, 0$. After the update at j, the invariant is $v = \sum_{r=j}^{n-1} a_r z^{r-j}$. The result is $A(z)$, using $n-1$ multiplications and additions. Independent Horner evaluations at n arbitrary points cost $\Theta(n^2)$.

For input lengths r, s, convolution gives $c_k = \sum_{i+j=k} a_i b_j$ and output length $r + s - 1$. Direct convolution uses rs coefficient products, or $\Theta(rs)$ operations. FFT improves the entire conversion-and-multiplication process.

Interpolation exists and is unique

Given distinct $x_0, \ldots, x_{n-1}$ and values $y_0, \ldots, y_{n-1}$, define

$$\ell_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}, \qquad A(x) = \sum_k y_k \ell_k(x).$$

Every denominator is nonzero. Each basis polynomial has degree less than n and satisfies $\ell_k(x_i) = \delta_{ik}$, proving existence.

If two polynomials of degree less than n match the values, their difference has n distinct roots. A nonzero polynomial of degree d has at most d roots. To prove this, factor out $x - \alpha$ at a root α. Every other root is a root of the quotient, so induction on degree applies. The difference must be zero. With fewer than n points, adding any multiple of $\prod_j (x - x_j)$ over the known points usually gives distinct solutions within the degree bound.

Vandermonde matrices and interpolation cost

Write $V[i, j] = x_i^j$ and $Va = y$. The Vandermonde determinant is $\det V = \prod_{j<k}(x_k - x_j)$. Equal points give equal rows, so every difference is a factor of the determinant polynomial. Its total degree is exactly $n(n-1)/2$. The coefficient of $x_1 x_2^2 \cdots x_{n-1}^{n-1}$ is 1, fixing the constant factor. Distinct points therefore make V invertible. General LUP interpolation costs $\Theta(n^3)$.

Lagrange interpolation can be converted to coefficients in $O(n^2)$ arithmetic operations. Build $Q(x) = \prod_j (x - x_j)$ one factor at a time in total quadratic time. For each k, use linear-time synthetic division to obtain $Q/(x - x_k)$, evaluate its denominator in $O(n)$, and accumulate coefficients. Rebuilding every basis polynomial from scratch does not justify that quadratic bound.

A product needs enough values

At the **same N distinct points**, multiplication needs only N pointwise products. Unique reconstruction of lengths r, s requires $N \geq r + s - 1$. Pad both coefficient arrays to N. Radix-2 FFT chooses a power of two at least this large.

For $A = 1 + 2x$ and $B = 1 - x$, the product is $1 + x - 2x^2$, which needs three coefficients. Choose $N = 4$, even though each input has length 2.

At the Nth roots of unity, $x^N = 1$. With too few points, the inverse returns convolution modulo $x^N - 1$. Position k receives $c_k + c_{k+N} + c_{k+2N} + \cdots$. For $(1 + x)^2$, $N = 2$ gives `[2,2]` because the x^2 coefficient wraps into the constant coefficient. Ordinary multiplication gives `[1,2,1]`. Zero padding prevents this aliasing.

Roots of unity and the forward DFT

Use the **positive-exponent forward convention** throughout:

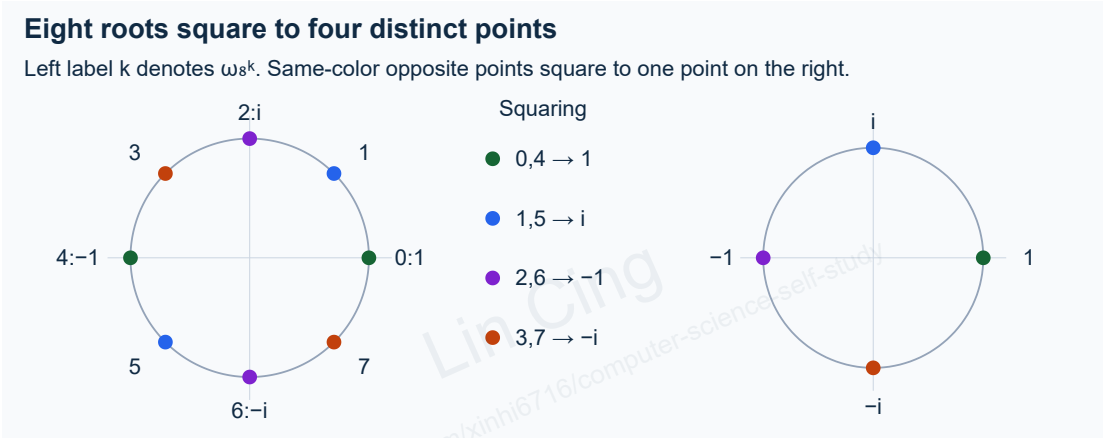
$$\omega_n = e^{2\pi i/n}, \qquad y_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk} = A(\omega_n^k).$$

Some fields use a negative forward exponent. Either paired convention works, but mixing them does not.

The root is primitive, so $\omega_n^k = 1$ exactly when $n \mid k$. Its n powers are distinct interpolation points. Cancellation gives $\omega_{dn}^{dk} = \omega_n^k$ for positive integers d. For even n, $\omega_n^{n/2} = -1$ and $(\omega_n^k)^2 = \omega_{n/2}^k$. Opposite roots square to the same value, leaving $n/2$ distinct roots, each twice.

The geometric sum $\sum_{j=0}^{n-1} \omega_n^{jt}$ is n if $n \mid t$. Otherwise it is $(1 - (\omega_n^t)^n)/(1 - \omega_n^t) = 0$. Primitivity makes the denominator nonzero in the latter case. The value 1 cannot replace a primitive root for arbitrary n.

Roots of unity and the forward DFT (continued)



Orthogonality gives the inverse

Let $F[k, j] = \omega_n^{jk}$ and $G[j, k] = \omega_n^{-jk}/n$. Then

$$(GF)[j, \ell] = \frac{1}{n} \sum_k \omega_n^{k(\ell-j)} = \delta_{j\ell}.$$

For distinct $j, \ell \in [0, n-1]$, $n \nmid \ell - j$, so orthogonality applies. Thus $GF = I$, $G = F^{-1}$, and

$$a_j = \frac{1}{n} \sum_k y_k \omega_n^{-jk}.$$

Use negative twiddles and divide by n once at the end. Normalization may instead be distributed across levels, but dividing by n at every level is wrong.

Applying the forward transform twice and dividing by n gives $(F^2 a/n)_j = a_{(-j) \bmod n}$. This is cyclic index reversal, not conjugation for arbitrary complex input. For $a = [2, 5+i, 7, -1]$, the result is `[2,-1,7,5+i]`.

Even and odd subproblems

For a positive power of two n, set $A_e(z) = a_0 + a_2 z + \cdots$ and $A_o(z) = a_1 + a_3 z + \cdots$. Then $A(x) = A_e(x^2) + x A_o(x^2)$. Root halving reduces evaluation at all n roots to two $n/2$-point DFTs, E and O.

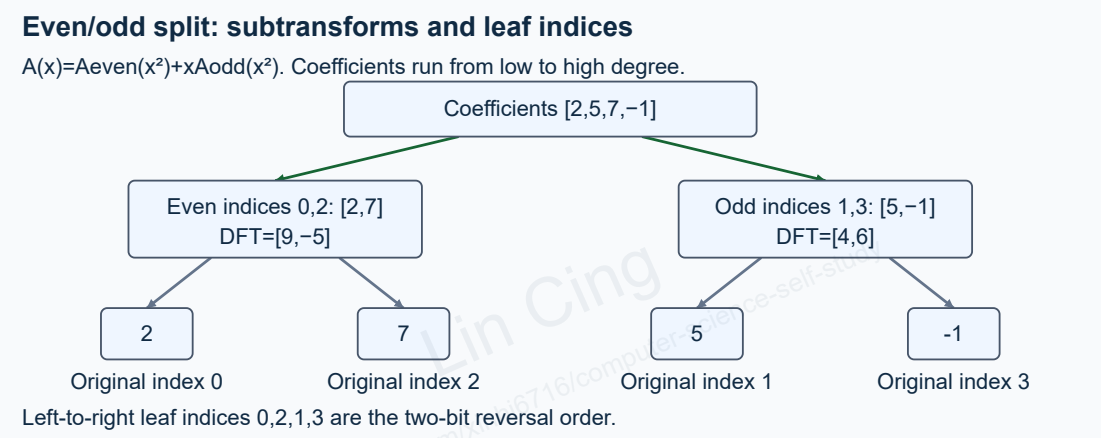
For $0 \le k < n/2$,

$$y_k = E_k + \omega_n^k O_k, \qquad y_{k+n/2} = E_k - \omega_n^k O_k.$$

Compute $t = \omega_n^k O_k$ once, then add and subtract. This is a butterfly. The base case $n = 1$ returns `[a[0]]`. Assuming the two recursive transforms are correct, the displayed identities produce every required value, proving correctness by induction.

For `[2,5,7,-1]`, the even coefficients `[2,7]` give `[9,-5]`, and the odd coefficients `[5,-1]` give `[4,6]`. Twiddle 1 gives $y_0 = 13, y_2 = 5$. Twiddle i gives $y_1 = -5 + 6i, y_3 = -5 - 6i$. Distinct intermediate values expose incorrect wiring.

Even and odd subproblems (continued)



Recursive FFT and its cost

```
FFT(a, sign):
    n = len(a)                  # require a positive power of two
    if n == 1: return [a[0]]
    E = FFT([a[0], a[2], ...], sign)
    O = FFT([a[1], a[3], ...], sign)
    w = 1; wm = exp(sign * 2*pi*i/n)
    for k = 0 .. n/2-1:
        t = w * O[k]
        y[k] = E[k] + t
        y[k+n/2] = E[k] - t
        w = w * wm
    return y
```

Forward uses `sign=+1`. Inverse uses `sign=-1` and divides the outer result by n. Splitting and combining cost $\Theta(n)$ per level. Thus $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ complex arithmetic operations. There are $n/2$ butterflies per level and $(n/2)\log_2 n$ in total.

Recursive FFT and its cost (continued)

A sequential implementation that copies subarrays and promptly releases child results needs $O(n)$ peak extra space and $O(\log n)$ stack space. Retaining all levels takes $O(n \log n)$. Recursion depth alone does not bound the arrays to logarithmic space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution

For $A = [1, 2, 0, 0]$, $B = [1, -1, 0, 0]$, $N = 4$, and $\omega_4 = i$:

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	$1 + 2i$	$1 - i$	$3 + i$
2	-1	2	-2
3	$1 - 2i$	$1 + i$	$3 - i$

The negative-exponent DFT of `[0,3+i,-2,3-i]` is `[4,4,-8,0]` before normalization. Dividing by 4 gives `[1,1,-2,0]`. Keep the first three coefficients to recover $1 + x - 2x^2$.

Two forward transforms, one inverse, and $\Theta(N)$ pointwise products cost $\Theta(N \log N)$. Because the output degree is below N, cyclic convolution equals ordinary convolution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution (continued)

Convolution: frequency values and inverse

A=1+2x, B=1−x, padded to N=4. Positive forward exponent, $\omega_4=i$.

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

Negative DFT: [4,4,−8,0]. Divide by 4: [1,1,−2,0].

Green cells are product values. Keep three coefficients: $1+x-2x^2$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1

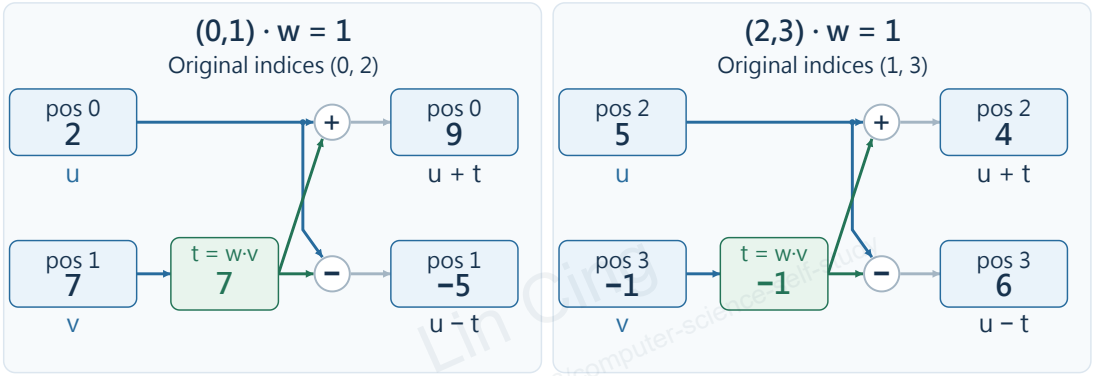
Recursive splitting reads the lowest index bit first, then the next bit. For $n = 2^b$, the leaf order reverses the b bits of each original index. Bit reversal is its own inverse, so both `A[rev(j)]=a[j]` and `A[j]=a[rev(j)]` describe the permutation.

For $n = 4$, `00,01,10,11` become `00,10,01,11`. The input `[2,5,7,-1]` becomes `[2,7,5,-1]`. Stage 1 pairs **physical positions** `(0,1)` and `(2,3)`, which hold original indices `(0,2)` and `(1,3)`. These are different kinds of index.

Both stage-1 twiddles equal 1. The array becomes `[9,-5,4,6]`. Stage 2 uses $m = 4$, pairs `(0,2)` and `(1,3)`, and uses twiddles 1 and i. Pairing adjacent positions again would perform a different computation.

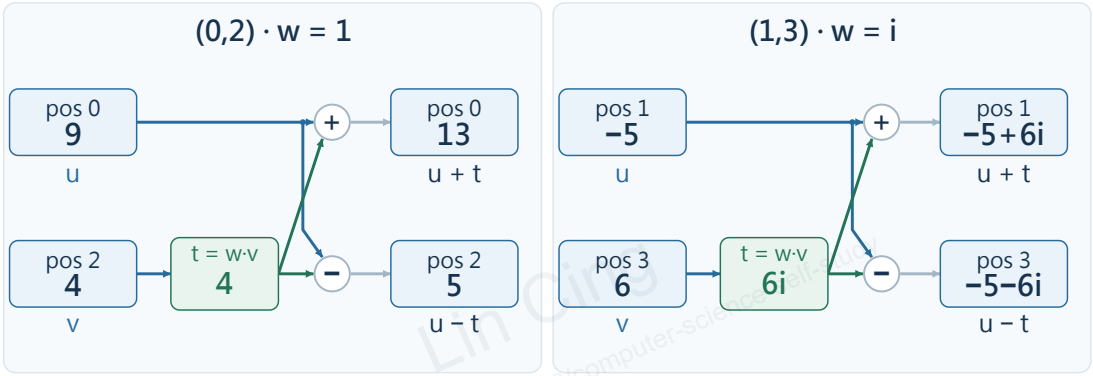
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1 (continued)



Stage 1: physical array [2,7,5,−1] → [9,−5,4,6]

Stage 2 combines across half-blocks



Stage 2: read positions 0,1,2,3 as [13,−5+6i,5,−5−6i]

j	$u = A[j]$	$t = \omega_4^j A[j+2]$	**New** $A[j]$	**New** $A[j+2]$
0	9	$1 \cdot 4 = 4$	13	5
1	-5	$i \cdot 6 = 6i$	$-5 + 6i$	$-5 - 6i$

Stage 2 combines across half-blocks (continued)

In physical output order, the result is `[13,-5+6i,5,-5-6i]`.

The same rules take `[1,2,0,-1]` through bit reversal `[1,0,2,-1]`, stage 1 `[1,1,1,3]`, and stage 2 `[2,1+3i,0,1-3i]`. Repeated intermediate 1s can conceal wiring mistakes. The earlier example makes the sources of each operation distinct.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Iterative FFT index contract

```
place a into A in bit-reversed order
for s = 1 .. log2(n):
    m = 2^s; wm = exp(sign * 2*pi*i/m)
    for base = 0, m, 2*m, ..., n-m:
        w = 1
        for j = 0 .. m/2-1:
            u = A[base+j]
            t = w * A[base+j+m/2]
            A[base+j] = u+t
            A[base+j+m/2] = u-t
            w = w*wm
for inverse only, divide every final entry by n
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

At the start of stage s, each length-m block has the DFT of its even subproblem in its left half and the DFT of its odd subproblem in its right half. Butterflies perform the recursive combination, leaving a complete m-point DFT in each block. Singletons establish the stage-1 base case. Induction reaches $m = n$. Save `u` before overwriting the upper cell, or the lower subtraction will use a new value.

Iterative FFT index contract (continued)

Each stage has $n/2$ butterflies, so arithmetic work is $\Theta(n \log n)$. Reversing b bits separately for each index takes $O(n \log n)$ bit steps. Reorder with an extra length-n array or in-place swaps of the permutation's two-cycles. Twiddles depend on m, j, not `base`. All blocks in a stage can share a table. Tables for all stages contain $\Theta(n)$ values and do not change the main arithmetic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit work and depth

With fixed twiddles and bit reversal treated as wires, a bounded-fan-in circuit runs all $n/2$ disjoint butterflies in a stage at once. Only the next stage reads their outputs. For $n = 8$, there are 3 stages and 12 butterflies. Work is $\Theta(n \log n)$, while depth is $\Theta(\log n)$, since every input-to-output path crosses $\log_2 n$ combinations.

This model assumes constant-depth complex addition and multiplication. It does not include serial twiddle generation, processor communication, or physical wire lengths. Software threads sharing an array need a barrier after each stage. Within a stage each butterfly writes a different pair of cells. With only P processors, the lower bound W/P still applies. A logarithmic-depth drawing does not give logarithmic running time on arbitrary hardware.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact arithmetic and floating point

The $\Theta(n \log n)$ bound counts complex arithmetic operations. Floating-point roots and arithmetic introduce errors. Rounding an integer convolution is valid only with an absolute error below $1/2$ in every output cell. A few successful round trips cannot guarantee all lengths and coefficient magnitudes.

An exact alternative is an NTT over a prime field $\mathbb{F}_p$. Require $N \mid p - 1$ and a root of order exactly N. Then N is invertible and the same geometric-sum and inverse proofs apply. Over a composite modulus, a primitive root alone does not ensure that the required denominators are units. The statement here is restricted to prime fields.

For the same inputs, use $p = 17, N = 4, \omega = 13$. We have $\omega^2 = 16 \neq 1, \omega^4 = 1$, and $4^{-1} = 13$. The transforms are `[3,10,16,9]` and `[0,5,2,14]`. Pointwise multiplication modulo 17 gives `[0,16,15,7]`, and the inverse gives `[1,1,15,0]`. Decode 15 as -2.

Signed reconstruction needs a magnitude bound

If all integer input coefficients have absolute value at most B, with lengths r, s, every convolution coefficient has absolute value at most $\min(r, s)B^2$. Choose $p > 2\min(r, s)B^2$ for unique signed reconstruction from one modulus. Here $r = s = 2, B = 2$, and $17 > 16$, so reconstruction is unique. Correct modular arithmetic alone cannot rule out wraparound of larger integers.

The NTT operation bound counts arithmetic modulo p. If p grows with coefficient magnitude, bit complexity must include multiplication and addition of $\log p$-bit integers. Arbitrarily large integer multiplication is not one constant-time operation.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 30.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic programming

Part IV Advanced Design and Analysis Techniques

Scope

Dynamic programming merges repeated recursive calls into states. The challenge is proving that a state retains everything needed for future decisions and reconstructing an actual solution from optimal values. Cutting, matrix parenthesization, common subsequences, and search trees develop this method.

Prerequisites

Recursion, optimization, and overlapping subproblems.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove optimal substructure for rod cutting and LCS.
- Design state dimensions and evaluation orders.
- Derive interval DP for matrix chains and optimal BSTs.
- Reconstruct choices and reassess DP assumptions in unfamiliar variants.
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Rod cutting

Given nonnegative integer length n, p_i is the price of an uncut piece of length i. All pieces have positive integer lengths. Leaving the rod uncut is permitted, but the whole rod must be sold; cuts cost nothing. Maximize revenue.

The first decision cuts off length i and optimally handles n−i. With $r_0=0$,

`r_n=max_{1<=i<=n}(p_i+r_{n-i})`.

Necessity. In an optimal cutting with value r_n, let the first piece have length i. If the remaining n−i were not optimal, replacing it with a better r_(n−i) cutting would be legal and improve the total, a contradiction.

Sufficiency. Each p_i+r_(n−i) is a legal cutting and cannot exceed the true optimum. The two inequalities establish the recurrence.

Uncached recursion makes $T(0)=1$ and $T(n)=1+\sum_{j=0}^{n-1}T(j)=2^n$ calls. Memoization or bottom-up evaluation computes each length once, with $\sum_{j=1}^{n}j=\Theta(n^2)$ transitions. T counts calls, not prices or revenue.

Memoization and bottom-up evaluation

Use one-based price array p and evaluate smaller remainders first. Even with negative prices, the whole rod must be sold, so nonempty states start at −∞ rather than silently allowing disposal.

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

At entry to length, all r[0..length−1] are optimal. The inner loop tests every legal first piece, so the recurrence makes r[length] optimal. Reconstruction reduces the remainder by at least1 per step and outputs at most n pieces.

Complete rod table and reconstruction

Let p=[1,5,8,9]. choice[n] stores an optimal first-piece length.

n	Candidate revenues	r[n]	choice[n]
0	Empty	0	0
1	1	1	1
2	5,1+1=2	5	2
3	8,1+5=6,5+1=6	8	3
4	9,1+8=9,5+5=10,8+1=9	10	2

From n=4, choice[4]=2 earns p_2=5 and leaves2. choice[2]=2 earns another5 and leaves0. The optimal cutting is 2+2, revenue10. Reconstruct after computing the table; do not guess cuts while modifying settled optimal values.

Complete rod table and reconstruction (continued)

A fixed cost c per cut can be subtracted from a candidate only if a remainder remains; one length dimension still suffices. A limit of k pieces requires a piece-count dimension. Add state only when the new restriction affects future feasible decisions, not merely because the specification changed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-chain multiplication

A_i has dimensions $p_{i-1} \times p_i$. If the final multiplication for A_i..A_j splits after k, its cost is

`m[i,k]+m[k+1,j]+p_{i-1}p_k p_j`.

Minimize over i≤k<j, with m[i,i]=0.

Both optimal subchains must be optimal. Replacing the left by cheaper parentheses leaves the final multiplication cost unchanged and improves the whole chain, contradicting optimality; likewise on the right. Every full parenthesization has a unique final split, so enumerating k=i..j−1 is complete.

Fill by increasing length l=j−i+1, since both dependencies are shorter. Store the best k in split[i,j]. There are Θ(n^2) states and O(n) candidates per state, totaling Θ(n^3).

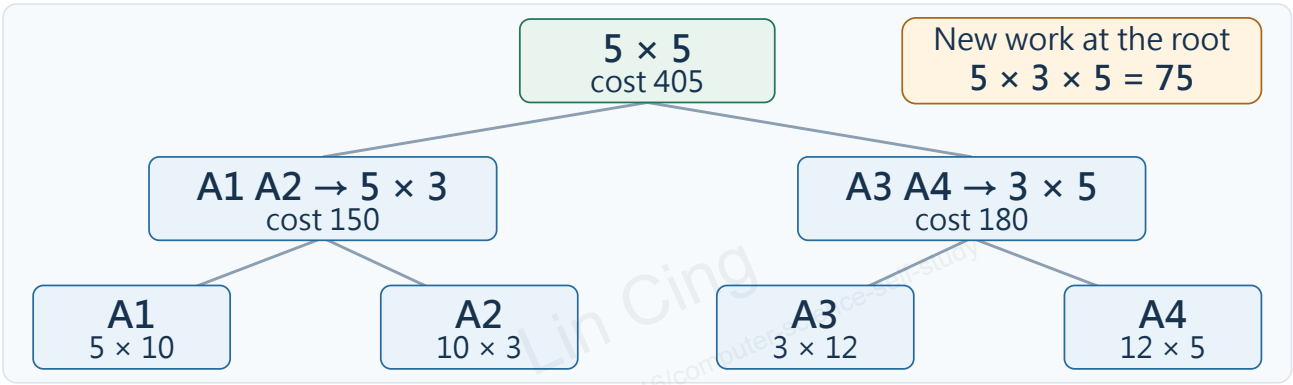
More precisely, the number of candidates is `Σ_(l=2)^n (n−l+1)(l−1)=n(n²−1)/6`. Each uses constantly many reads and arithmetic operations. This proves both bounds; merely multiplying two O bounds would not establish Θ.

Matrix-chain table and parenthesis output

```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

Every dependency is a strictly shorter interval, justifying length order. The reconstruction tree has n leaves and n−1 internal nodes, so emitting parentheses takes Θ(n) node operations, excluding actual matrix-entry multiplication.

Matrix-chain table and parenthesis output (continued)



150 + 180 + 75 = 405; cumulative cost differs from new work

Complete matrix-chain values and reconstruction

For p=[5,10,3,12,5], the upper-triangular table for A1..A4 is:

Length	Values
1	m11=m22=m33=m44=0
2	m12=150,m23=360,m34=180
3	m13=330,m24=330
4	m14=405

`m24=min(0+180+10*3*5,360+0+10*12*5)=330`, with split[2,4]=2.

`m14=min(0+330+5*10*5,150+180+5*3*5,330+0+5*12*5)=405`, with split[1,4]=2.

That split gives(A1A2) and(A3A4). Their splits1 and 3 produce final parenthesization `((A1A2)(A3A4))`.

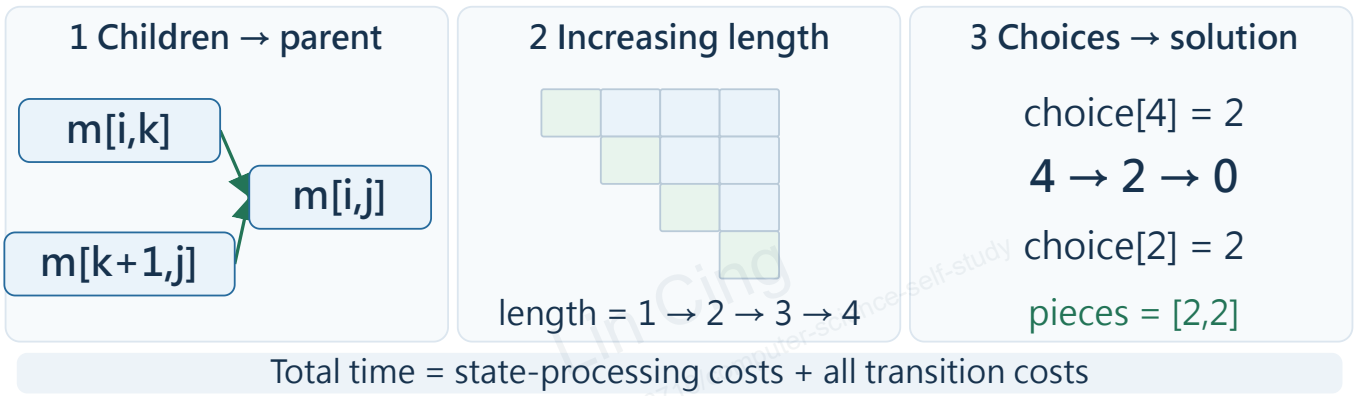
Elements of dynamic programming

A state must describe the remaining work, not merely rename the original problem. Transitions depend on smaller states. Establish four facts:

1. Removing the first decision from an optimum leaves an optimum for the corresponding subproblem.
2. Defined substates compose every candidate decision without omitting legal solutions.
3. Base cases are correct and the dependency graph is acyclic.
4. A topological evaluation order completes all prerequisites before a state is filled.

Together these prove recurrence correctness and valid evaluation. Overlap explains why caching saves repeated work; it alone does not prove optimal substructure. Cyclic dependencies require another justified solver rather than one table pass. Reconstruction can store choices or recheck transitions in a full value table; one final scalar optimum usually does not suffice.

Elements of dynamic programming (continued)



The state graph determines memoization cost

Rod cutting has n+1 states. Length j depends on0..j−1, totaling n(n+1)/2 edges. Constant work per state/edge gives Θ(V+E)=Θ(n^2).

Top-down memo expands all candidates the first time a state is evaluated and immediately returns its stored value afterward. Keep visited status separate from values, for example None for uncomputed, since a negative revenue can be valid. Bottom-up has no recursion stack; memo may have n stack levels. Both use Θ(n) table space.

The longest-simple-path counterexample does not mean every cycle forbids DP. Endpoint-only states fail to record used vertices. Individually longest simple u→v and v→w paths may both use z, so concatenation repeats z and is not simple. Adding a visited set S permits exponential-state DP; a DAG instead allows longest paths directly in topological order.

Longest common subsequence

Let c[i,j] be the LCS length of prefixes X_i,Y_j. Empty-prefix length is 0.

If x_i=y_j, an LCS not ending in that character could append it and become longer. Thus an optimum can end with the common character; removing it gives a candidate for X_(i−1),Y_(j−1). A better prefix subsequence could also append x_i and exceed the original optimum, so the prefix must itself be optimal.

If x_i≠y_j, no common subsequence can depend on both distinct terminal characters as its last character. It omits at least one terminal position, giving

`c[i,j]=max(c[i-1,j],c[i,j-1])`.

The two branches cover every possibility. Each transition costs O(1) and the full table O(mn).

Both LCS inequalities and the full algorithm

When terminal characters match, an LCS not ending with that character uses neither terminal position and can append it, contradicting maximality. Removing the final character bounds its length by c[i−1,j−1]+1. Conversely, any prefix LCS can append the common character legally. Hence c[i,j]=c[i−1,j−1]+1.

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both LCS inequalities and the full algorithm (continued)

The full c table already determines backtracking; a separate direction table is optional. Length alone needs two rows along the shorter string, Θ(min(m,n)) space. This backtracking still requires the full table and cannot run unchanged on only two rows.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking

X=ABCBDAB,Y=BDCABA gives c[7,6]=4. Including empty prefixes:

```
     ε B D C A B A
ε    0 0 0 0 0 0 0
A    0 0 0 0 1 1 1
B    0 1 1 1 1 2 2
C    0 1 1 2 2 2 2
B    0 1 1 2 2 3 3
D    0 1 2 2 2 3 3
A    0 1 2 2 3 3 4
B    0 1 2 2 3 4 4
```

Up decreases i; left decreases j.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LCS table and backtracking (continued)

Cell	Action	Next	Reverse characters
(7,6)	Up; both candidates4	(6,6)	Empty
(6,6)	A=A, diagonal	(5,5)	A
(5,5)	D≠B; upper3 larger	(4,5)	A
(4,5)	B=B, diagonal	(3,4)	AB
(3,4)	C≠A; left2 larger	(3,3)	AB
(3,3)	C=C, diagonal	(2,2)	ABC
(2,2)	B≠D; left1 larger	(2,1)	ABC
(2,1)	B=B, diagonal	(1,0)	ABCB

Reverse to get BCBA. Each step decreases at least one index, giving O(m+n) backtracking without scanning all mn cells.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

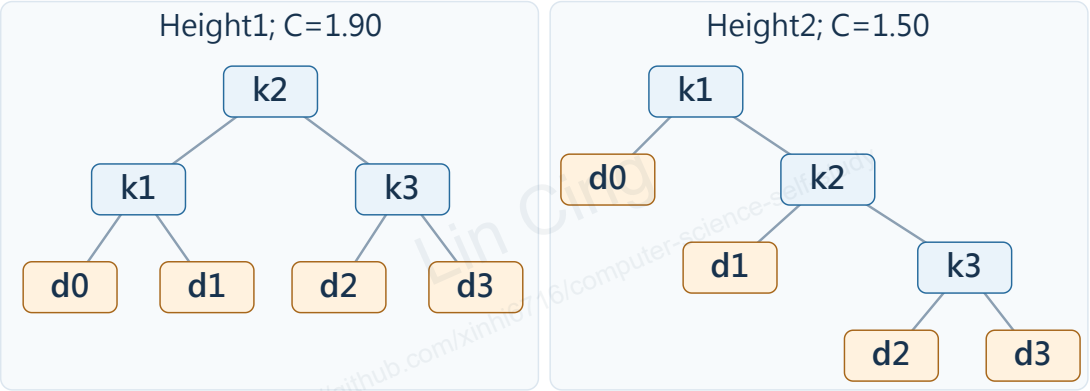
Complete LCS table and backtracking (continued)

i ↓ / j →	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	**4**

Green arrows: backtrack; amber: emit; reverse A,B,C,B → BCBA

Example A: lower height can cost more comparisons

Let p=(0.50,0.10,0.05), q=(0.15,0.10,0.05,0.05). Root depth is 0. Height counts real-key edges; C counts only real-key comparisons.



C counts only real-key comparisons; no extra dummy check

Example A: weight every successful and unsuccessful outcome

A successful search compares depth+1 keys. A failed search compares as many keys as its dummy depth, **without an extra dummy check**.

Query	Probability	Balanced: count × prob.	Right chain: count × prob.
k1	0.50	2 × 0.50 = 1.00	1 × 0.50 = 0.50
k2	0.10	1 × 0.10 = 0.10	2 × 0.10 = 0.20
k3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d0	0.15	2 × 0.15 = 0.30	1 × 0.15 = 0.15
d1	0.10	2 × 0.10 = 0.20	2 × 0.10 = 0.20
d2	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
Total	1.00	**C = 1.90**	**C = 1.50**

Success costs are 1.20/0.85; failure costs are 0.70/0.65. Frequent outcomes k1 and d0 finish earlier in the right chain, outweighing extra comparisons for the other outcomes.

Optimal binary search trees

Given distinct sorted keys k1<…<kn, choose a BST that minimizes expected search cost for a specified query distribution. Key order stays fixed; the root and subtree shapes are the decisions. Include both successful and unsuccessful searches.

A hit on key k_i has probability p_i. Failed searches end in gaps $d_0, \dots, d_n$, with probabilities $q_0, \dots, q_n$. For example, d_0 lies before the smallest key and d_i between adjacent keys k_i, k_{i+1}. A dummy leaf represents an empty stopping position, not a real record to compare. All probabilities sum to 1. Define the extended cost E, including a step at the dummy leaf:

$$E(T) = \sum_{r=1}^{n} p_r(\text{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r(\text{depth}_T(d_r) + 1).$$

e[i,j] is optimal cost for k_i..k_j; w[i,j] is all key and dummy probability mass in that interval. An empty interval has one dummy:

$$e[i, i-1] = w[i, i-1] = q_{i-1}.$$

Choosing root k_r increases every left/right-subtree search depth by1, adding its total weight. Thus

Optimal binary search trees (continued)

$$e[i,j] = \min_{i \le r \le j} \{e[i,r-1] + e[r+1,j] + w[i,j]\}.$$

Replacing a purportedly optimal subtree with a cheaper one would improve the whole tree. Every root is enumerated, establishing the recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two cost conventions differ by total failure probability

C(T) counts real-key comparisons. E(T) also counts the step reaching a dummy leaf after failure. These are cost conventions, not instructions to compare an imaginary record. The table e below uses E.

$$C(T) = \sum_{r=1}^{n} p_r(\operatorname{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r \operatorname{depth}_T(d_r).$$

$$E(T) = C(T) + \sum_{r=0}^{n} q_r.$$

For example A the difference is 0.35: E=2.25 for the balanced tree and 1.85 for the right chain. This shape-independent constant **preserves the optimal tree, but changes its reported cost**.

With no real key, C[i,i−1]=0, while e[i,i−1]=q[i−1]. C does not ignore failed searches: after attachment, a query still compares the parent key, and the parent interval accounts for that step through w. Initialization and later additions must use one consistent convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Across courses: search models and cost conventions

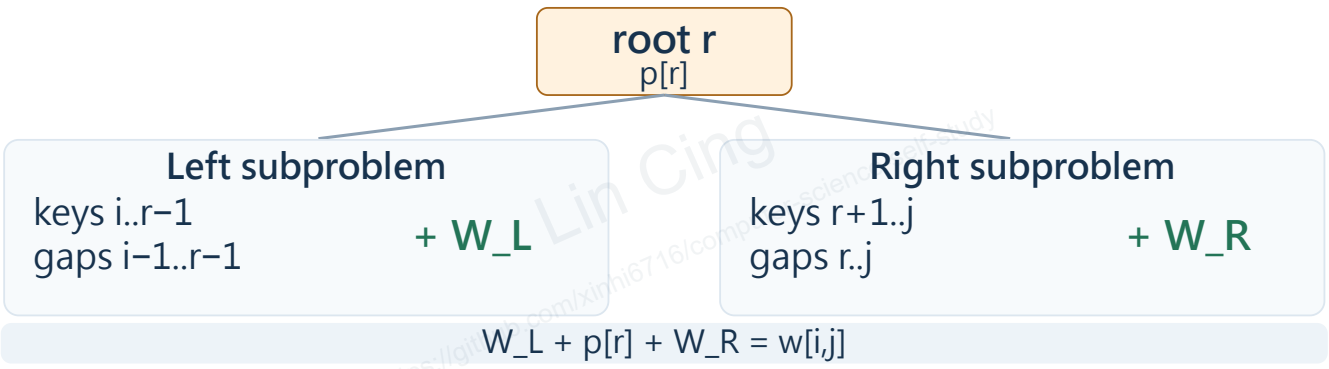
Data Structures Chapter 10 starts with successful searches only. Here, CLRS § 15.5 includes failed queries too. Compare answers only for the same distribution and cost convention.

Convention	Successful searches only	Full query model here
Input weights	p; total 1	p and q; total 1
Empty-interval cost	0	E: q; C: 0
Added interval weight	Successful-query mass	Successful and unsuccessful mass w

Dropping nonzero q and renormalizing p may change the optimal tree. For fixed p and q, E−C=Σq is constant, so optimal trees agree. Also check interval endpoints, empty-tree costs, and the meaning of root entries.

Why add all of w, rather than only the root probability?

$$w[i,j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t = W_L + p_r + W_R.$$



W_L, W_R are the two subtree probability masses. One extra level adds $W_L + W_R$; a root hit adds p_r. **Use global probabilities without renormalizing.**

Optimal-BST table and cost convention

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

This is CLRS's extended-search-tree cost: failed search includes one dummy check. Counting only real-key comparisons gives e[1,n]−Σq_i. This shape-independent constant leaves the optimal tree unchanged. Example B's optimum 2.10 becomes 2.10−0.40=1.70 under real-key comparisons.

Reconstruct build(i,j): if j<i, emit dummy d_(i−1); otherwise r=root[i,j], emit k_r and recursively build(i,r−1),build(r+1,j). Retain empty intervals to deliver the unsuccessful-search leaves correctly.

Example B: complete optimal-BST table and reconstruction

Take p=(0.20,0.30,0.10), q=(0.10,0.05,0.15,0.10).

Interval	w	e	Root
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

For e[1,2], root k2 gives e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45. root[1,3]=2; roots for[1,1] and[3,3] are 1,3. Thus k2 has children k1,k3 and dummies attach by BST intervals.

Example B: complete optimal-BST table and reconstruction (continued)

Increasing-length evaluation uses $\Theta(n^2)$ cells each for e,w,root and $\Theta(n^3)$ total candidate roots. This chapter uses that proved version.

Exactly $\Sigma_{l=1}^{n-l+1} l = n(n+1)(n+2)/6$ roots are tested. Each w updates in O(1) from $w[i,j-1]+p_j+q_j$. Rescanning probabilities inside every root candidate would add a factor n and invalidate this bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example B: compare every root and read the correct subproblems

e[i,j]: empty to longer intervals

i / j	0	1	2	3
1	0.10	0.50	1.45	2.10
2		0.05	0.70	1.35
3			0.15	0.60
4				0.10

Root candidates; w[1,3]=1

r=1
0.10 + 1.35 + 1 = 2.45

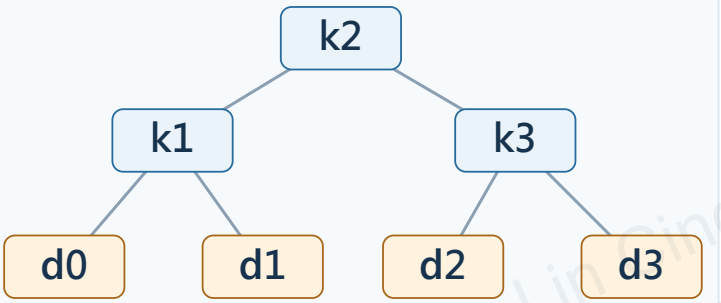
r=2
0.50 + 0.60 + 1 = 2.10

r=3
1.45 + 0.10 + 1 = 2.55

Green: subproblems read for r=2; amber: the result

Green cells are e[1,1] and e[3,3], read when r=2. Both are shorter intervals. Amber e[1,3] becomes 2.10 after all three candidates are compared. Retain the q cost of empty intervals.

Example B: reconstruct the actual search tree from root



Example B: includes dummy checks

k2: 0.30 × 1 = 0.30

k1,k3: 0.30 × 2 = 0.60

d0..d3: 0.40 × 3 = 1.20

E = 2.10

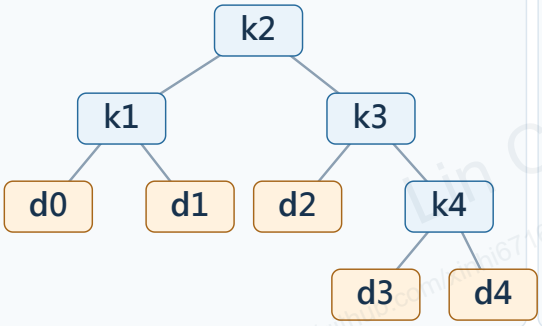
C = E − 0.40 = 1.70

d0 | k1 | d1 | k2 | d2 | k3 | d3

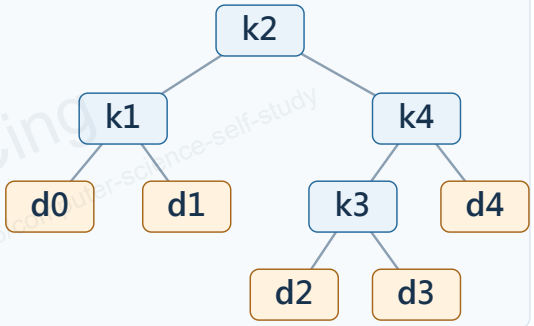
Example C: integer weights and two optimal trees

Weights P=(3,3,1,1), Q=(2,3,1,1,1) total 16; failure weights total 8. The right interval[3,4] has two optimal roots,3 and 4.

Choose root3 for interval[3,4]



Choose root4 for interval[3,4]



Both trees: S=32; divide by total weight16: C=2 and E=2.50

Sequence alignment: score matches, mismatches, and gaps

Global alignment allows match, mismatch, and gap. Its last column is one of:

- Paired(x_i,y_j), earning match/mismatch score.
- x_i paired with a gap.
- y_j paired with a gap.

For maximum prefix score F[i,j],

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j),F[i-1,j]+gap,F[i,j-1]+gap).`

This shares LCS's prefix states and backtracking skeleton but needs three score cases. For X=ACGTA,Y=AGCAC, match3,mismatch−2,gap−3, optimum is 1 with alignment `ACGTA- / A-GCAC`. Every gap costs its own fixed score. Comparing only equal-length positions misses displacement.

Unbounded knapsack uses capacity states

With repeatable item weights s_i and values v_i, let f[c] be maximum value at **exact** weight c:

`f[0]=0`, `f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i).`

For capacity at most C, maximize over0≤c≤C; exact fill takes only f[C]. Including f[c−1] in the recurrence instead defines value at capacity at most c. Distinguish the meanings.

For A=(3,5),B=(4,7),C=(7,12), capacity13,

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22].`

Optimum22 at weight13 uses A+A+A+B. Highest value/weight greedy chooses B×3 and earns21, disproving general ratio-greedy correctness.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter15, § § 15.1–15.5.

Lin Cing, personal handwritten data-structures notes on weighted OBST cost and interval recurrences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms

Part IV Advanced Design and Analysis Techniques

Scope

A greedy algorithm retains only one choice per step, avoiding other candidates. Activity exchanges, Huffman leaf exchanges, and matroid augmentation prove when a choice is safe. Weighted objectives and capacity restrictions show when an old rule fails.

Prerequisites

Sorting, exchange arguments, trees, and axiomatic selection properties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Prove activity selection by exchange.
- Distinguish greedy choice from optimal substructure.
- Build Huffman codes and prove matroid greedy correctness.
- Give counterexamples when safety fails and complete every exchange step.
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Activity selection

Given all start/finish times, maximize the number of pairwise nonoverlapping activities. Assume s_i<f_i, sorted nondecreasing finishes, and half-open intervals[s_i,f_i). Equal finish/start endpoints are compatible.

Theorem. For every nonempty subproblem, some maximum compatible set contains an earliest-finishing activity a_m.

Proof. Let A* be an optimum and a_j its earliest-finishing activity. Since f_m≤f_j, replace a_j with a_m.

- Every later activity in A* starts at or after f_j, hence also after f_m.
- There is no earlier activity in the subproblem's chosen schedule to disrupt.
- Cardinality is unchanged, so the new set remains optimal.

This establishes a safe greedy choice. After a_m, only activities starting at or after f_m remain. Repeating the same exchange inductively proves optimality.

Linear scan and boundaries

```
activity-select(activities sorted by finish):
    result=[]; last_finish=-∞
    for (s,f) in activities:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

Before processing activity j, result is the compatible set chosen by the rule from the first j−1 activities, and last_finish is its final endpoint. The exchange theorem makes each choice extendible to an optimum. One scan takes Θ(n) when sorted; otherwise add O(n log n) sorting.

Assume positive-duration half-open intervals; equal endpoints may touch. Zero-duration activities need separate same-point rules. Empty input returns an empty set. Earliest start is unsafe: [0,10), [1,2),[2,3) gives it one activity rather than two.

Shortest duration is also unsafe: [0,3),[3,6),[2,4). Choosing the middle length2 activity blocks both length3 activities and gives one rather than the optimum two.

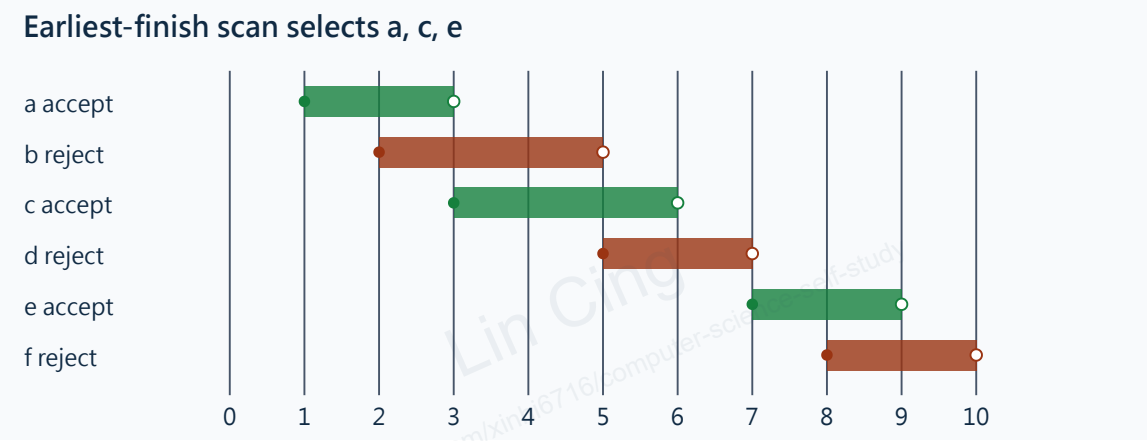
Complete activity scan

Activity	Start	Finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

Select a; reject b since2<3; select c since3≥3; reject d since5<6; select e since7≥6; reject f since8<9. Result{a,c,e}.

The optimum need not be unique: {b,d,e} also has size3. Exchange guarantees **some** optimum includes the earliest finish, not that every optimum must.

Complete activity scan (continued)



Elements of the greedy strategy

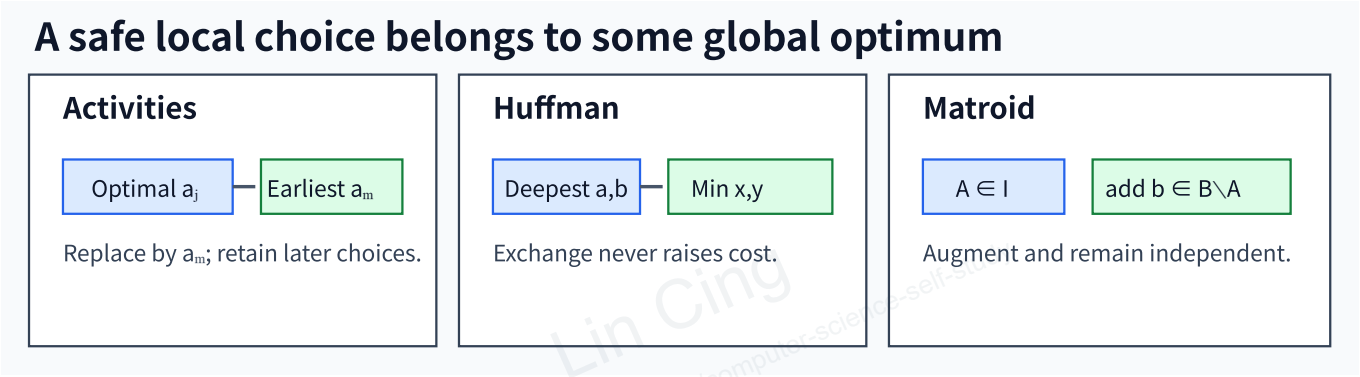
An exchange proof must establish:

1. **Safety.** For greedy g and an optimum A*, identify a differing choice x and form A′ by replacing x with g. Check feasibility and no loss of value.
2. **The residual problem.** After selecting g, the remaining solution belongs to a precisely defined subproblem that retains every relevant candidate.
3. **Termination by induction.** Empty/singleton bases are correct and each decision strictly shrinks the problem.

Only then do safe local choices imply global optimality. Immediate benefit, average performance, or successful examples alone do not suffice.

Largest-coin greedy fails for denominations{1,3,4}, amount6:4+1+1 uses three coins versus3+3 using two. This disproves the general rule; experience with familiar denominations does not prove a property for all denomination families.

Elements of the greedy strategy (continued)



Why weighted activities invalidate the same exchange

Maximizing total value changes the objective from cardinality.

Activity	Interval	Value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

Earliest finish selects short+tail, value2; optimum long+tail has101. Replacing long by short preserves compatibility but lowers value100→1, so it no longer preserves optimality.

The weighted problem still has optimal substructure. Sort by finish with one-based indices. Let p(j) be the largest earlier index finishing no later than s_j, or0 if none. D[j] is optimum value among the first j activities, with D[0]=0:

`D[j]=max(D[j-1],value_j+D[p(j)])`.

Why weighted activities invalidate the same exchange (continued)

Excluding j leaves the first j−1 activities. Including it restricts all remaining choices to the first p(j), which can be optimized independently. These cases cover every solution. Here p=(0,0,2),D=(0,1,100,101). Reconstruct tail at D3 and long at D2. Finish-time greedy guarantees unweighted cardinality only.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fractional and 0–1 knapsack require different proofs

Assume positive weights/values and C>0. Fractional knapsack allows x_i∈[0,1], maximizing Σv_i x_i. Sort decreasing density ρ_i=v_i/w_i, taking highest densities fully and at most one partial item.

If total weight≤C, taking everything is optimal. Otherwise any optimum fills C: spare space plus a partially untaken positive-value item would allow improvement.

Suppose an optimum uses positive lower-density j weight while higher-density i is not full. Move δ=min(w_i(1−x_i),w_j x_j)>0 weight from j to i. Capacity is unchanged and value increases by δ(ρ_i−ρ_j)≥0. Repeat until the first greedy choice agrees, then induct on remaining capacity. Some optimum follows the whole density order; ties may be arbitrary.

In 0–1 knapsack, x_i∈{0,1}, so that δ exchange is usually illegal. For C=10 and(w,v)=(6,12),(5,9),(5,9), density greedy takes(6,12) and cannot fill the remaining4, earning12. The 0–1 optimum takes both(5,9), earning18. Fractional greedy takes(6,12) and4/5 of one(5,9), earning19.2.

For 0–1 integer-capacity DP, D[i,c] is optimum for the first i items and capacity at most c. Bases D[0,c]=D[i,0]=0. Take max of D[i−1,c] and v_i+D[i−1,c−w_i], the latter only if w_i≤c. Time Θ(nC) is pseudopolynomial because C occupies only Θ(log C) input bits.

Huffman codes

A binary prefix tree's leaf depth is code length. For frequency f_c, total bits are

$B(T)=\Sigma_c f_c d_T(c)$.

Repeatedly extract the two minimum-weight trees x,y and insert their parent of weight f_x+f_y. n symbols need n−1 rounds, each two extract-min and one insert, or3(n−1) heap operations. With linear heap construction, time is O(n log n).

Swapping left/right changes codeword0/1 values but not lengths or total cost. Equal frequencies may yield different optimal trees.

Huffman's safe choice places minimum frequencies at deepest siblings

Assume at least two symbols and positive frequencies. An optimal prefix tree may be full: removing a one-child internal node shortens every code beneath it and lowers cost. A deepest leaf has a leaf sibling at the same depth, since an internal sibling would have deeper descendants.

Let x,y have the two lowest frequencies. First move x to one of these deepest positions. If one holds y, use the other, keeping y available for the second exchange or already in its desired sibling position.

Swapping x with deepest leaf a changes only their contributions:

$B(T')-B(T)=(f_a-f_x)(d_x-d_a)\leq 0$.

Here $f_a \geq f_x$ and $d_x \leq d_a$. With x fixed, all remaining frequencies are at least f_y. Move y to x's deepest sibling without raising cost or moving x. A symbol already in place gives a zero exchange; equal frequencies also permit ties.

Thus some optimal prefix code includes this greedy choice. The argument requires both the minimum-frequency and maximum-depth inequalities, not intuition alone.

Huffman contraction and optimal substructure

Replace x,y by virtual symbol z of frequency $f_z=f_x+f_y$. Expand z in an optimum contracted tree T′ into leaves x,y to obtain T.

Other leaf depths stay unchanged, while x,y have depth $d_{T'}(z)+1$. Hence

$B(T)=B(T')+f_x+f_y$.

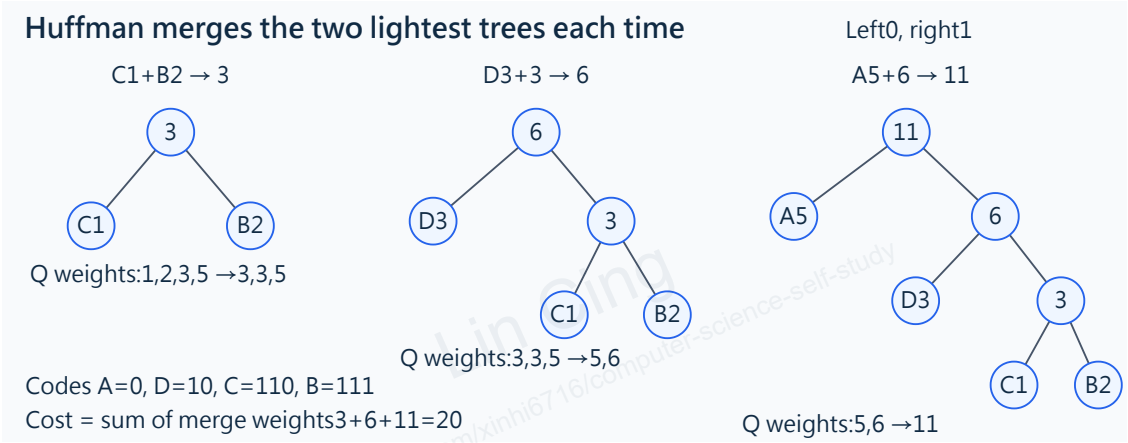
If a better original tree T* existed, the safe-choice lemma permits deepest siblings x,y. Contracting them would give

$B(T^{**})=B(T^*)-f_x-f_y<B(T')$,

contradicting optimality of T′. Expansion is optimal, and induction on alphabet size proves Huffman.

For A5,B2,C1,D3, merge weights are3,6,11. One code is A=0,D=10,C=110,B=111, costing5+6+3+6=20=3+6+11.

Huffman contraction and optimal substructure (continued)



Huffman algorithm and singleton boundary

```
Q = min-heap of all positive-frequency singleton trees
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q's sole tree, or empty if Q is empty
```

Decode from root, following left/right for0/1. At a leaf, emit its symbol and restart at root. Prefix freedom means no complete word is a prefix of another, making boundaries unambiguous. Ending at an internal node means truncated input, not successful decoding.

An empty alphabet has no tree. A singleton may receive an empty code theoretically; practical streams often prescribe0 and transmit a symbol count separately. The equality between weighted length and merge-weight sum uses tree depths and, for the usual nontrivial coding case, at least two leaves. Do not substitute a singleton's forced one-bit implementation convention into merge cost.

Matroids and greedy methods

A matroid $M=(S,I)$ satisfies:

- Empty set belongs to I.
- Heredity: every subset of an independent set is independent.
- Augmentation: if $A,B\in I$ and $|A|<|B|$, some x in B excluding A makes $A\cup\{x\}$ independent.

Assume finite S and $w(x)>0$. Scan decreasing weights and accept if independence remains. Let x be the first independent singleton. If none exists, heredity excludes every nonempty independent set, and empty is optimal.

Take optimum B. If x is absent, start $A=\{x\}$. While $|A|<|B|$, augment using an element of B outside A. Only B elements are added. At equal size, exactly one $y\in B$ has been omitted, so $A=B-\{y\}+\{x\}$. Since $\{y\}$ is independent, first feasible x has $w(x)\geq w(y)$, and $w(A)=w(B)-w(y)+w(x)\geq w(B)$. Thus A is also optimal and contains x.

Do not assert that any arbitrary y can be exchanged. Augmentation first constructs feasible A, and equal cardinality identifies its one omitted y.

Complete proof that contraction remains a matroid

Fix feasible x, let S' be S excluding x, and $I'=\{A\subseteq S':A\cup\{x\}\in I\}$. S' may contain elements that are never independent in the new system; these cause no problem.

- Empty belongs to I' because$\{x\}\in I$.
- If $A\in I'$ and $C\subseteq A$, then $C\cup\{x\}\subseteq A\cup\{x\}$; heredity gives $C\in I'$.
- If $A,B\in I'$ with $|A|<|B|$, augment $A\cup\{x\}$ from $B\cup\{x\}$. The added y lies in B excluding A, since x is in both. Hence $A\cup\{y\}\in I'$.

Removing x maps every feasible solution containing x to I' and subtracts exactly w(x); adding it reverses this mapping. After choosing x, the original problem is exactly maximum-weight independence in the contraction. Induct on $|S|$; later greedy feasibility tests are precisely the I' oracle, proving optimality.

An element rejected now never becomes feasible later: heredity of a later larger feasible set containing it would imply feasibility at rejection, a contradiction. Sorting costs $O(n \log n)$; $O(f(n))$ per oracle gives $O(n \log n+n f(n))$ total. For arbitrary independent sets with negative weights, skip negative elements rather than unconditionally forming a full basis.

Graphic matroids and a nonmatroid counterexample

For fixed undirected G=(V,E), take S=E and I all acyclic edge sets, retaining isolated vertices in V. Heredity holds. Let A's components be V1..Vk. If every B edge stays within a component Vi, B's forest restriction there has at most|Vi|−1 edges. Summing gives|B|≤|V|−k=|A|, contradicting|B|>|A|. Thus B has an edge crossing A-components, and adding it creates no cycle. Augmentation holds.

In a connected graph, bases are n−1-edge spanning trees. To derive minimum-cost MST from the positive maximum-weight theorem, set w′(e)=K−cost(e)>0. Equal basis sizes make maximizing Σw′ equivalent to minimizing original cost. This precisely connects increasing-cost Kruskal to the theorem without confusing arbitrary minimum forests with bases.

Heredity alone does not imply augmentation. For S={a,b,c},weights10,9,9, take I={∅,{a},{b},{c},{b,c}}. A={a},B={b,c} have unequal sizes, yet neither b nor c augments A. Greedy gets10 at a; optimum{b,c} gets18.

Unit-time scheduling as a matroid

Job j takes one unit, has deadline d_j, and late penalty w_j. A set A can all finish on time exactly when sorted deadlines satisfy d_(k)≥k, equivalently for every t,

`N_t(A)=|{j in A:d_j<=t}|<=t.`

Necessity follows from capacity t in the first t slots. For sufficiency, schedule by increasing deadline: job k's deadline≥k makes it on time.

All jobs are ready at0, nonpreemptive, on one machine. Deadlines are positive integers and completion exactly at d_j is on time. n jobs need only n slots, so cap deadlines above n at n without changing feasibility.

Heredity holds. For augmentation with|A|<|B|, let k be the largest integer with N_k(A)=k, including0. Since|A|<|B|≤n, k<n. A has exactly k jobs with deadline≤k, while B has at most k. Thus B has at least|B|−k>|A|−k jobs with deadline>k. Some x∈B outside A has d_x>k.

Adding x leaves N_t unchanged for t≤k. For t>k, maximality implies N_t(A)<t, so integrality gives N_t(A)≤t−1. Adding at most1 preserves feasibility, proving augmentation.

Complete scheduling greedy trace

Job	Deadline	Penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

Scan decreasing penalty: accept a; accept b with deadlines1,3; accept c with1,3,3. Reject d:1,2,3,3 has fourth deadline3<4. Reject e because N1=2>1.

On-time set{a,b,c} is scheduled b,a,c at completion times1,2,3. Place d,e at4,5; both are indeed late. Penalty is6+5=11. Total penalty35 minus maximum saved24 gives11.

Complete scheduling greedy trace (continued)

A length-n deadline-frequency array yields all prefix N_t and checks independence in O(n). n tests cost $O(n^2)$, plus O(n log n) initial penalty sorting. Sorting separately for every oracle costs O(n log n) each and $O(n^2 \log n)$ overall; sorting is not linear.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter16, § § 16.1–16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementary graph algorithms

Part VI Graph Algorithms

Scope

Search order determines distances, recursion ancestry, and finish times. On one six-vertex directed graph, FIFO layers and the white-path theorem establish the correctness and linear cost of BFS, DFS, topological sorting, and SCC decomposition.

Prerequisites

Graph representations, colors, timestamps, and linear traversals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Choose adjacency matrices or lists.
- Derive the shortest-path property of BFS.
- Use DFS for topological sorting and SCCs.
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

Graphs here are finite and either directed or undirected, with stable vertex identifiers and explicitly specified edge directions. A path may have length zero, so every vertex reaches itself. Let $n=|V|$ and $m=|E|$, counting each undirected edge once. Unless stated otherwise, shortest distance counts **edges**, rather than arbitrary edge weights.

An adjacency list has n headers. Its lists contain m entries for a directed graph and 2m for an undirected simple graph. Space is $\Theta(n+m)$. Enumerating u's neighbors takes $\Theta(1+\text{outdegree}(u))$; testing (u,v) by scanning an ordinary unsorted list takes that same worst-case time. Hashing or sorting changes lookup guarantees and must not be assumed for an ordinary list.

In an adjacency matrix A, rows are sources and columns are destinations. $A[u,v]=1$ means $u \to v$. Space is $\Theta(n^2)$, edge lookup $\Theta(1)$, and enumerating u's neighbors generally scans n entries. An undirected graph satisfies $A=A^{\mathsf{T}}$. A weighted matrix needs a separate absence marker because zero can be a valid edge weight.

One six-vertex directed graph

BFS, DFS, and SCC below use **these same directed edges**, with outer vertex order and adjacency order as displayed.

Vertex	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

Here n=6,m=8. The list lengths sum to 8, as do the indegrees, rather than 16. To construct G^{T}, initialize n empty lists and scan all m entries, placing u in $\text{Adj}^{\mathsf{T}}[v]$ for every $u \to v$. Time and additional space are $\Theta(n+m)$; the original graph need not change.

One six-vertex directed graph (continued)

One graph as lists and a matrix

Adj[u]: row u → column v. Directed graph, m=8.

Vertex	Adjacency list
a	b,c
b	c
c	a
d	b,e
e	d
f	e

Columns of A

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

Green cells are directed edges: 8 cells correspond to 8 list entries.

22.2 Breadth-first search

BFS maintains distance layers with a FIFO queue. White means undiscovered, gray discovered but not fully scanned, and black fully scanned. The discovery predecessor is π[v]; ∞ means no known distance and NIL no predecessor. Neither can be replaced by zero.

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

22.2 Breadth-first search (continued)

Change color before enqueueing, so competing predecessors cannot enqueue a vertex twice. At the start of each while iteration, the queue contains exactly the gray vertices. While scanning u, however, u is gray and already dequeued. The invariant does not hold after every individual instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why FIFO processes nearer vertices first

List the queue from front to back as $q_1,\ldots,q_r$. Distances are nondecreasing and $d[q_r] \leq d[q_1]+1$; regard this condition as true for an empty queue.

Initially only s is queued. If u is removed with $d[u]=k$, the remaining distances, by induction, are k or k+1 in nondecreasing order. Every new vertex receives distance k+1 and is appended, preserving order. If layer k empties, the next front has distance k+1; only a later iteration can generate distance k+2. Dequeued distances are therefore nondecreasing.

Every finite d[v] describes an actual predecessor path. The source has length zero, and discovering v appends $u \to v$ to u's path, giving length $d[u]+1$. Thus true shortest distance $\delta(s,v) \leq d[v]$. Distances decrease by one along predecessor links, so those links cannot cycle.

An actual path gives only an upper bound. Shortest-path correctness still requires ruling out an earlier discovery by a longer route.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete proof of BFS shortest distances

Suppose a reachable v ends with $d[v] \neq \delta(s,v)$, and choose such a v with minimum δ. It is not s. Let u precede v on a shortest path. Then $\delta(s,u)=\delta(s,v)-1$ and, by minimality, $d[u]=\delta(s,u)$. Vertex u has finite distance, is enqueued, and is eventually dequeued.

When $u \to v$ is scanned, if v is white, it receives $d[v]=d[u]+1=\delta(s,v)$, a contradiction. Otherwise let w have first discovered v. Vertex w was dequeued no later than u. Queue order gives $d[w] \leq d[u]$, hence $d[v]=d[w]+1 \leq \delta(s,v)$. Together with $\delta \leq d$ from the predecessor path, this again gives equality. Both cases contradict the choice of v. All reachable distances are correct. Discovering an unreachable vertex would give an actual path to it, also a contradiction, so its distance remains ∞.

Every reachable nonsource vertex has one predecessor, and following predecessors reaches s after exactly d[v] edges. These links form a BFS tree. Adjacency order can change the selected predecessors and shortest paths, but not distances.

Initialization takes $\Theta(n)$. Each vertex enters and leaves the queue at most once; each reachable adjacency list is scanned once. Total time is $O(n+m)$, with $O(n)$ additional space for the queue and arrays. A matrix scans n positions for every dequeued vertex, giving worst-case $\Theta(n^2)$.

Complete queue trace from d

The queue shown is its content **after scanning finishes** in that iteration. Parenthesized numbers are distances; newly discovered vertices receive predecessors immediately.

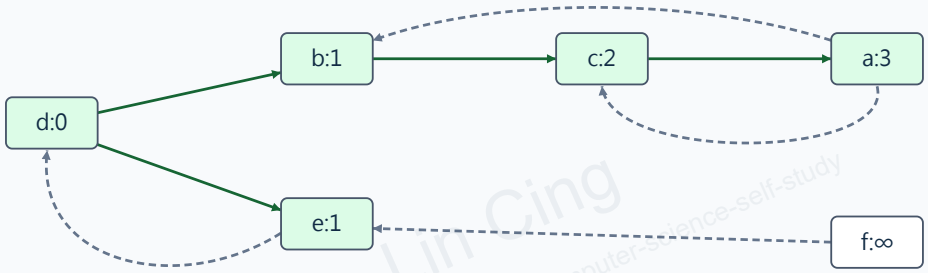
Removed u	Scan result	Queue at iteration end
Before the loop	Initialize d[d]=0	d(0)
d	b white: d[b]=1,π[b]=d; e white: d[e]=1,π[e]=d	b(1),e(1)
b	c white: d[c]=2,π[c]=b	e(1),c(2)
e	d already black; do not enqueue	c(2)
c	a white: d[a]=3,π[a]=c	a(3)
a	b,c already discovered; unchanged	empty

Finally $(d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,\infty)$. A shortest path to a is $d \to b \to c \to a$. Edge $f \to e$ does not permit travel from d backward to f.

Complete queue trace from d (continued)

BFS from d on the same six-vertex graph

Boxes: vertex:distance. Green: predecessor edges. Gray dashed: other edges.



Dequeue: d(0) → b(1) → e(1) → c(2) → a(3). Vertex f stays at ∞.

Directed distances, equal weights, and path counts

For every edge $u \to v$ with reachable u, $\delta(s,v) \leq \delta(s,u)+1$. A directed graph provides no reverse inequality. Here $a \to b$ goes from distance 3 to 1, a difference of 2. Only when both directions of an undirected edge exist do we obtain $|d[u]-d[v]| \leq 1$.

For equal positive weight w, shortest weight is $w \cdot d[v]$. BFS fails for general weights: $s \to t$ of weight 9 competes with $s \to x$ and $x \to t$ of weight 1 each. The one-edge route costs 9, whereas the two-edge route costs 2. With all weights zero, every reachable route costs zero. Negative weights or cycles require a separately specified problem.

To count shortest paths in a unit-weight graph, initialize ways[s]=1 and all other counts to zero. Discover and enqueue white v as usual. For **every** edge $u \to v$ satisfying $d[v]=d[u]+1$, add ways[u] to ways[v], including edges into an already discovered v. Count each simple-graph edge once. All distance-$(k-1)$ vertices finish before distance-k vertices are dequeued, so ways[u] is complete when u is dequeued.

Directed distances, equal weights, and path counts (continued)

For edges 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5, distances are (0,1,1,2,2,3), and ways= (1,1,1,2,2,4). This uses O(n+m) additions. Counts may occupy Θ(n) bits, so arbitrary-precision additions are not unconditionally constant-time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search

DFS completes the current recursive branch before advancing to another neighbor. The outer loop must cover all vertices, including those unreachable from the first root. Here time counts **discovery and finish events**, not scanned edges.

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search (continued)

Each vertex is visited once and each adjacency entry scanned once, taking Θ(n+m) time. Recursion depth is at most n and additional space O(n). An iterative implementation must retain the next neighbor or an explicit finish event in each stack frame. Pushing all neighbors at once with a visited flag does not automatically reproduce the same forest or finish times.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate

For distinct u,v, suppose tin[u]<tin[v]. If v is discovered before u finishes, Visit(u) remains on the stack. Vertex v is its descendant and must return first, giving tin[u]<tin[v]<tout[v]<tout[u]. If v is discovered after u finishes, tin[u]<tout[u]<tin[v]<tout[v], so the intervals are disjoint.

Thus tin[u]<tin[v]<tout[u]<tout[v] is impossible. Conversely, strict interval containment means u was still on the stack when v was discovered, so v is a proper descendant. This parenthesis theorem depends on last-in-first-out recursive returns.

DFS on the six-vertex graph in a,b,c,d,e,f order gives the table. Its parenthesis sequence is (a(b(c c)b)a)(d(e e)d)(f f); every timestamp from 1 through 12 appears exactly once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

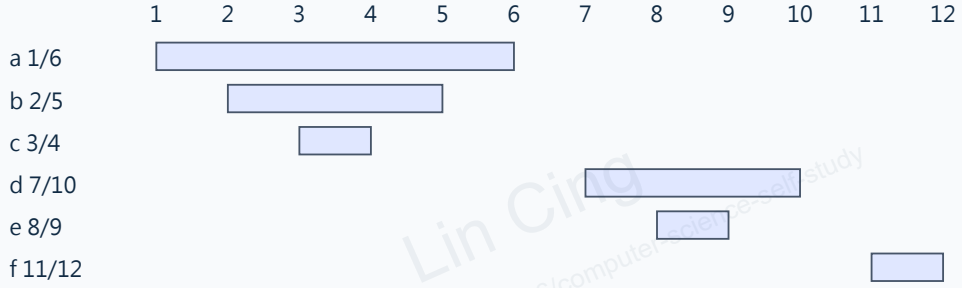
Vertex	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

DFS discovery and finish events

Bars run from tin to tout. Descendants nest; different root intervals are disjoint.



a contains b,c; d contains e; f is the third root. Vertex intervals never interleave.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The time condition in the white-path theorem

Immediately **before Visit(u) changes u from white to gray**, v is a descendant of u in this DFS, allowing v=u, if and only if there is a u-to-v path whose vertices are all white at that instant. This time convention includes the starting vertex u itself.

For the forward implication, the forest path from u to v uses original graph edges. All proper descendants are discovered after tin[u], so every vertex on this path is white at the specified instant.

Conversely, take the first vertex y on a white path that supposedly fails to become a descendant of u. Its predecessor x is already such a descendant, possibly u. When Visit(x) scans x→y, a still-white y is recursively visited and becomes a descendant. If y has been discovered already, it was white at the initial instant and was therefore discovered during u's unfinished recursive call, also within u's subtree. Both cases contradict the assumption.

The requirement **all white at that instant** is essential. In a→b,a→c,c→b, scan b first. By c's discovery, b is black. Although c reaches b, b will not become a descendant of c. Ordinary reachability cannot replace the white-path condition.

Four edge types in directed DFS

While scanning u→v, u is the deepest gray vertex, and all gray vertices form exactly the stack's ancestor chain. White v gives a tree edge. Gray v gives a back edge, including a self-loop. For black v, tin[u]<tin[v] gives a forward edge; otherwise it is a cross edge.

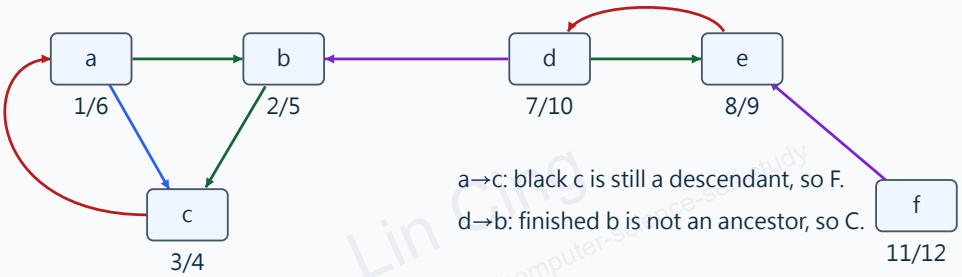
Type	Forest relationship	Timestamp check after DFS
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	Nontree edge to a proper descendant	Same inequality; timestamps alone do not distinguish tree/forward
back	v is an ancestor of u, or v=u	For a proper ancestor: tin[v]<tin[u]<tout[u]<tout[v]
cross	Neither is the other's ancestor	tout[v]<tin[u]; preserve the scanned direction

The actual scan order is a→b(T),b→c(T),c→a(B),a→c(F),d→b(C),d→e(T),e→d(B),f→e(C). Edge a→c is scanned after c finishes. A black destination need not mean a cross edge.

Four edge types in directed DFS (continued)

Four directed edge types in one DFS

Green T: tree. Red B: back. Blue F: forward. Purple C: cross.



Labels are tin/tout. All 8 edges remain; their directions determine their types.

Undirected edge types and cycle tests

An undirected edge occurs in two adjacency lists. Classify the edge using the direction of its **first scan**. Suppose tin[u]<tin[v]. Since u lists v, v is discovered before u finishes. If u→v is the first scan, v must still be white, or v→u would already have been scanned; it is a tree edge. If v→u is scanned first, u is gray and it is a back edge. Thus undirected DFS has only tree/back edges, with no genuine forward/cross edges.

In a simple undirected graph, the reverse adjacency entry of the parent edge is not an additional cycle. With parallel edges, skip the same parent edge by **edge identifier**, rather than skipping all edges to the parent. A second parallel edge can form a length-two cycle under the multigraph convention.

A directed graph has a cycle if and only if DFS has a back edge. A back edge u→v, together with the tree path v to u, gives a cycle. Conversely, choose the earliest-discovered vertex v on a cycle. All other cycle vertices are white then. The cycle path to v's predecessor u is a white path, so the white-path theorem makes u a descendant of v. Edge u→v is a back edge. A self-loop satisfies the same conclusion directly.

22.4 Topological sort

A topological order permutes all vertices so every edge u→v satisfies pos[u]<pos[v]. A directed cycle would require strict increases around a loop back to the starting position, which is impossible. Exactly DAGs admit such an order.

Run DFS on a DAG and reverse its finish list; comparison sorting is unnecessary. For every scanned u→v, v cannot be gray, since that would give a cycle. If white, v becomes a descendant and tout[v] <tout[u]. If black, v has finished already and the same inequality holds. Decreasing finish order therefore directs every edge forward. Time is Θ(n+m), with O(n) additional space.

An implementation can report a cycle upon scanning a gray destination, and deliver the order only after validation. A reverse-finish list always exists, but it is not automatically a topological order for an arbitrary directed graph.

For a→c,b→c,c→d and outer order a,b,c,d, DFS gives a=1/6,c=2/5,d=3/4,b=7/8. Reverse finish order is b,a,c,d. Order a,b,c,d is also valid. **Order a,c,b,d is invalid because it reverses b→c.**

Kahn's indegree updates and failure condition

Compute indegrees and enqueue all zero-indegree vertices. Output a removed u, decrement indegree[v] for every u→v, and enqueue v exactly when its value becomes zero. The counts equal indegrees in the subgraph induced by vertices not yet output. Thus u has no unprocessed predecessor when output, and every output prefix respects edge order.

If a nonempty remainder has no zero-indegree vertex, repeatedly follow a remaining incoming edge from any vertex. Finiteness forces a repeated vertex and hence a cycle. Conversely, every nonempty induced subgraph of a DAG has a zero-indegree vertex, so all n vertices are output. Fewer than n outputs means failure. **A remaining vertex can lie downstream from a cycle without lying on one.**

Kahn's indegree updates and failure condition (continued)

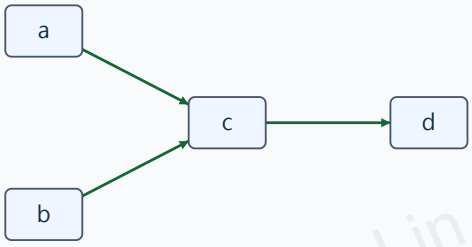
Output u	Remaining indegree of c	Remaining indegree of d	Queue at iteration end
Initial	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	empty

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kahn's indegree updates and failure condition (continued)

Kahn: indegrees and queue states

DAG: a→c, b→c, c→d. Remove a source, then decrement its destinations.



Operation	in(c)	in(d)	Queue
Initial	2	1	a,b
Remove a	1	1	b
Remove b	0	1	c
Remove c	0	0	d
Remove d	0	0	empty

Output a,b,c,d. Order a,c,b,d reverses b→c and is invalid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.5 Strongly connected components

Define u~v exactly when u reaches v and v reaches u. Zero-length paths give reflexivity; the definition gives symmetry; concatenating paths gives transitivity. This equivalence relation partitions vertices into SCCs, the inclusion-maximal mutually reachable sets, **not merely the class with the greatest cardinality**.

The condensation has one vertex per SCC and an edge for every original edge between distinct classes. It is a DAG: a cycle of distinct classes would make all those classes mutually reachable, contradicting their separation. Reversing every edge reverses paths in both directions, so G and G^T have identical SCCs.

For the six-vertex graph, A={a,b,c},D={d,e},F={f}, with condensation F→D→A. In G^T the direction is A→D→F. Starting arbitrarily at a in G^T can reach all three SCCs. **Transposition alone does not prevent crossing components; root order and visited marks are essential.**

Why finish times order different SCCs

Use timestamps from the **first DFS on G**. Define tin(C)=min tin[v] and F(C)=max tout[v] over v∈C. For distinct SCCs with C→D, F(C)>F(D). There are two discovery cases.

1. If tin(C)<tin(D), the first discovered x in C can follow white paths inside C to the outgoing edge, cross to D, and traverse D internally. The white-path theorem puts every other C vertex and every D vertex below x. Thus F(C)=tout[x]>F(D). Paths between C vertices can be chosen entirely inside C: any outside vertex on such a path would also be mutually reachable with C and therefore belong to it.
2. If tin(D)<tin(C), the first y in D reaches every white vertex of D, so F(D)=tout[y]. No path leads from D to C, or C→D would make them one SCC. Hence y's recursion cannot discover C. C is discovered after y finishes, implying F(C)>F(D).

Why finish times order different SCCs (continued)

Consequently, a cross-component edge C→D in G^T satisfies F(C)<F(D). This compares **maximum finish times per component**, not every pair of individual finish times. For a↔b,a→c, scanning a→b first gives tout[b]=3,tout[c]=5,tout[a]=6. Then F({a,b})=6>F({c})=5 even though b finishes before c. The actual cross-component edge a→c still has tout[a]>tout[c]. An edge between different SCCs cannot be a back edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two complete DFS passes and the boundary proof

```
SCC(G):
  first=DFS(G,a fixed order of all vertices)
  roots=reverse(first.finished)    // save the first finish order
  GT=transpose(G)
  reset every vertex to WHITE, π=NIL for the second pass
  consider vertices in roots order; call Visit(u) on GT only if WHITE
  output each new root's entire DFS tree as one component
```

Inductively, every previously output tree is exactly one SCC. The next white root u lies in a component C with the greatest F(C) among all unprocessed components. If C's maximum-finish vertex were already processed, the induction hypothesis would make all of C processed, contradicting white u.

Paths internal to C reach every C vertex from u. All remaining C vertices are white, so the second pass collects all of them. For a cross edge C→D in G^T, F(D)>F(C), so D must already be black. Visit does not recurse into it, and cannot cross it to another white class. The tree collects neither too few nor too many vertices, completing the induction.

Two complete DFS passes and the boundary proof (continued)

Each DFS takes Θ(n+m), as does transposition. Reversing the appended finish list takes Θ(n); no O(n log n) sort is needed. Total time is Θ(n+m), and the transpose plus arrays use O(n+m) additional space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph

The first finish list is c,b,a,e,d,f, giving second-pass candidates f,d,e,a,b,c. The transposed lists, alphabetized internally, are a:[c],b:[a,d],c:[a,b],d:[e],e:[d,f],f:[].

Candidate root	New discoveries in G^T	Cross-component edge encountered	Output
f	f	none	{f}
d	d,e	e→f; f already black	{d,e}
e	skip, already black	none	no new component
a	a,c,b	b→d; d already black	{a,b,c}
b,c	skip, already black	none	no new component

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

The values F(A)=6,F(D)=10,F(F)=12 decrease along the original condensation F→D→A. Reversing discovery order could place c ahead of a, but this particular example need not fail. Accidental success is not a correctness argument. The minimal counterexample is x→y: first-pass discoveries tin[x]=1,tin[y]=2 would start the second pass at y, whose transposed edge y→x incorrectly merges two SCCs.

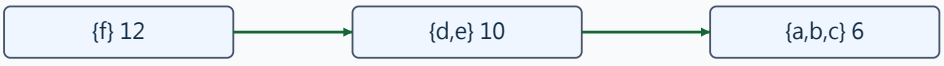
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

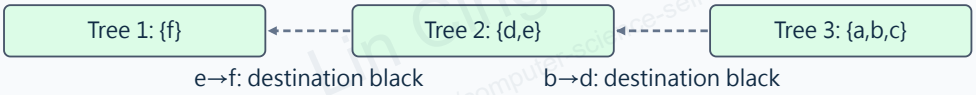
SCC: finish order prevents crossing in pass two

First-pass maxima in G: F({f})=12, F({d,e})=10, F({a,b,c})=6.

Original condensation DAG



Root order and black boundaries in GT



Green boxes collect one SCC each. Gray arrows cannot recurse into black vertices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 22.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disjoint sets

Part V Advanced Data Structures

Scope

Disjoint sets turn a partition that only merges into representative queries. We compare weighted lists and rank forests on the same merges, then prove the inverse-Ackermann sequence bound through complete level/iter potential accounting. Single-operation cost, amortized sequence cost, and implementation assumptions remain distinct.

Prerequisites

Representatives, forests, ranks, and path compression.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Compare linked-list and forest representations.
- Apply union by rank and path compression.
- Prove the inverse-Ackermann amortized bound.
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.1 Disjoint-set operations

The partition is the state; a representative is a handle

Every established universe element belongs to exactly one nonempty set, and these sets are pairwise disjoint. MAKE-SET(x) requires a new element and creates {x}. FIND-SET(x) requires an existing element and returns a representative of its set. UNION(x,y) merges two sets. The representative need not be the minimum key, and internal parent pointers need not be original graph edges.

Let n count MAKE-SET calls and m count all external MAKE-SET, FIND, and UNION calls, with $m \geq n$. Establish all n singletons first to fix the analysis parameters. There are at most $n-1$ successful merges, but arbitrarily many repeated UNION calls. Find both roots first; if they agree, return without LINK or a rank increase. LINK accepts **two distinct roots**.

FIND can compress parents without changing the partition. UNION only merges. Deletion, rollback, dynamic removal of graph edges, and enumeration of all members require additional structures or operations. Different sets have different representatives; all finds within one set return the same representative.

Why union maintains exactly the connected components

For an undirected graph, MAKE-SET every vertex and UNION(u,v) for each arriving edge. The invariant is that the DSU partition equals the connected components of the processed-edge subgraph. Initially both consist of singletons. An edge within a component changes neither structure; an edge between components joins exactly those two. Induction proves that at termination FIND(u)=FIND(v) exactly when an undirected u–v path exists.

On a..f, insert ab, cd, bc, ab, ef, de. The partitions are {ab},{c},{d},{e},{f}; {ab},{cd},{e},{f}; {abcd},{e},{f}; the same after repeated ab; {abcd},{ef}; and {abcdef}. Five merges succeed during six UNION calls.

Why union maintains exactly the connected components (continued)

Edge stream: each partition change

Each row shows the partition after an edge. Repeated edge 4 leaves the set count fixed.

			Set count
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

Five merges: 6→5→4→3→3→2→1. All six UNION calls still check representatives.

Static DFS or BFS already finds components in Θ(n+|E|). DSU additionally supports connectivity queries as edges arrive and Kruskal cycle tests. Ignoring directions in a directed graph gives weak connectivity, not strongly connected components.

21.2 Linked-list representation of disjoint sets

Set records and member back-pointers

Each set record stores head, tail, and size; each member stores next and a set pointer. The representative is record.head. An empty collection takes O(1) to initialize. MAKE-SET creates next=NIL, head=tail=x, size=1, and x.set pointing to the new record. FIND returns x.set.head in O(1).

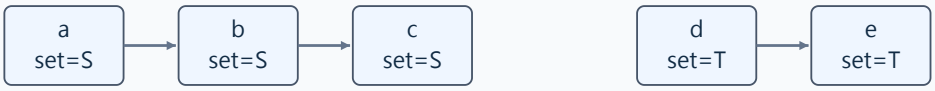
UNION reads S=x.set and T=y.set. If equal, return the head. Swap S,T if S.size<T.size; on ties keep the first S. Set S.tail.next=T.head, S.tail=T.tail, and S.size+=T.size. Traverse the old T list, redirecting each member's set pointer to S, then discard T's record. The traversal retains the original next links, so successors are not lost. Tail splicing alone does not update representatives.

One UNION costs Θ(1+min(|S|,|T|)). Merging a→b→c with d→e produces a→b→c→d→e. Only d,e change set records; the tail moves from c to e and size from 3 to 5. Their FIND calls now return a.

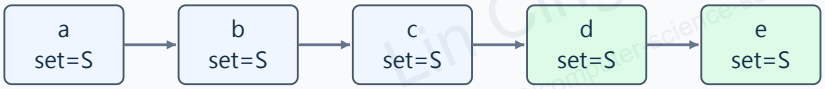
21.2 Linked-list representation of disjoint sets (continued)

Weighted lists: update set as well as next

Before: S head=a, tail=c, size 3; T head=d, tail=e, size 2.



Append short to long. Change only d,e set pointers to S, and set c.next=d.



After: S head=a, tail=e, size 5. Green members changed set; arrows show next.

A quadratic counterexample and the doubling proof

Suppose the second set is always appended to the first. Starting with n singletons, UNION(x_2,x_1), UNION(x_3,x_2), … makes each new singleton absorb the old long list. Pointer rewrites number 1,2,…,n−1, totaling n(n−1)/2. Thus m=2n−1 calls cost $\Theta(n^2)$, despite constant-time next splicing with a tail pointer.

Weighted union moves only the shorter set. Each rewrite of a fixed member x at least doubles its set size. After k moves, size is at least 2^k and at most n, so $k \leq \text{floor}(\log_2 n)$. Summing over n members gives at most n floor($\log_2 n$) rewrites, plus O(m) call overhead, for O(m+n log n). Use log(n+1) for small n. A single merge of two n/2-member lists still costs Θ(n); the sequence bound does not give worst-case O(log n) per union.

For $n=2^k$, repeatedly pair equal-sized sets. Each of k rounds rewrites exactly n/2 pointers, totaling $(n/2)\log_2 n$. Constant-time MAKE and FIND are consistent with this total union cost.

Accounting for eight elements

Keep the first set on equal sizes. Use this sequence again for the forest. Letters are handles, not priority keys.

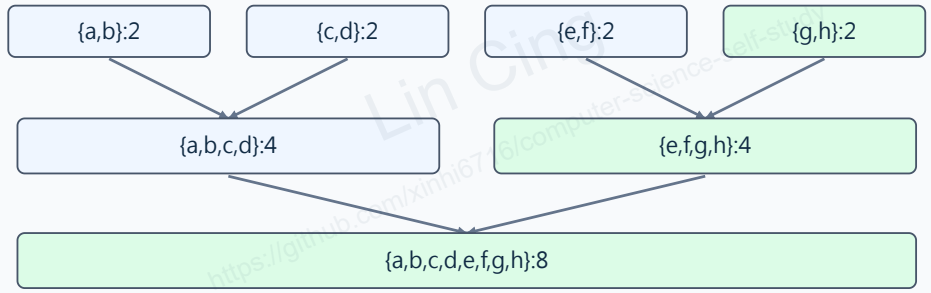
UNION	Old sizes	Members whose set changes	New representative/size	Cumulative rewrites
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

Accounting for eight elements (continued)

For h, the set pointer follows h→g→e→a, and the containing size grows 2→4→8. It changes three times; a never changes. Total rewrites are 12, not three for every element. In the earlier singleton-absorption sequence, weighting reduces the total to n−1 rewrites.

Equal-size pairwise merges: three rounds of rewrites

Each round rewrites four set pointers. Green marks h's containing set: 2→4→8.



Total rewrites 4+4+4=12; h changes 3 times, a changes 0 times. Sets never split.

21.3 Disjoint-set forests

Self-parent roots and the LINK precondition

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.3 Disjoint-set forests (continued)

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-pass FIND preserves the old parent before overwriting

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

The first pass finds the root without mutation. The second follows the old route and redirects every parent to r. Overwriting parent[x] before saving it jumps immediately to r and misses intermediate nodes. Recursive parent[x]=FIND(parent[x]) finds the root upward and redirects while returning. Its stack uses at most the path height; the two-pass iterative version uses O(1) extra space.

Compression reconnects nodes only to the root of the same tree, preserving roots and the partition. It never changes rank. Visiting s nodes costs Θ(s). Forest storage is O(n). Member lists can support enumeration, but parent pointers alone do not automatically provide O(set size) enumeration.

The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

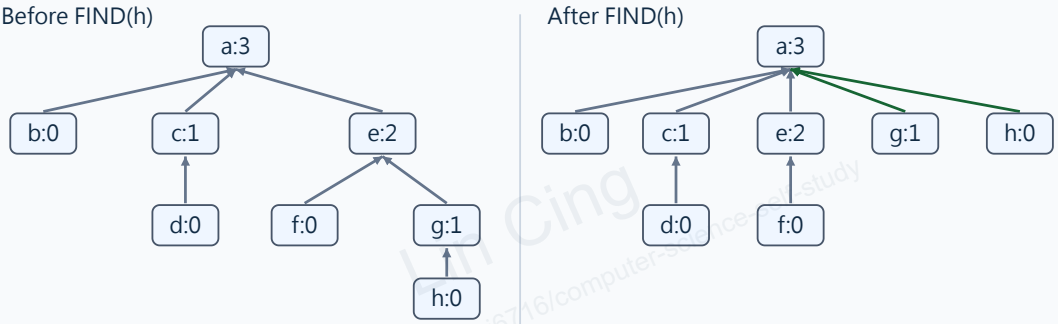
Vertex	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

The first pass takes h→g→e→a, three links. The second sets the parents of h,g,e to a; e already had that value. Nodes d→c and f→e are outside this path and do not change. All ranks remain fixed. A later FIND(h) takes one link, but FIND(d) still takes two. One find does not flatten the whole tree.

The same eight elements before and after compression (continued)

The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.



h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

Rank bounds height and historical membership

Induct on operations to prove that a root's current height is at most its rank. A singleton has height zero. For unequal ranks, the smaller height plus one is at most the larger rank; equal ranks allow one extra level. Compression only shortens paths. A rank-r root contains at least 2^r members: its rank rises only by merging two rank-(r−1) sets, each of size at least 2^{r-1}. Other merges only add members. A nonroot keeps its rank and the historical membership bound from when it was last a root. Thus every rank is at most floor($\log_2 n$).

Rank is not a lower bound on current height. After compressing every element in the example, a retains rank 3 but has height 1. A rank-2 root can likewise be the center of a four-node star. Rank is not current subtree size either, since compression can move descendants away without reducing rank.

Rank alone gives O(log n) per FIND and O(m log n) overall. Pairwise equal-rank merges create height $\log_2 n$, and repeated uncompressed finds on a deepest leaf attain that order. Without rank, arbitrary links can create a linear chain and a first FIND of Θ(n). That counterexample does not apply when both heuristics are present.

21.4 Fine-grained analysis of rank and compression

Ackermann iteration means composition

Fix the final number of elements n and set $\alpha = \alpha(n)$ throughout the analysis, even while MAKE-SET calls are still creating elements. For $j \geq 1$, define

$$f^{(0)}(j) = j, \quad f^{(i+1)}(j) = f(f^{(i)}(j)), \quad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

The superscript counts **repeated applications to the previous output**. Since $A_0^{(i)}(j) = j + i$, we get $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$, hence

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

Inductively, each A_k is strictly increasing in its input and satisfies $A_k(j) \geq j + 1$: a larger starting input undergoes more iterations, each increasing its input. Since $j + 1 \geq 2$, also $A_k(j) > A_{k-1}(j)$. Both monotonicities are used below.

The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4:

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	greater than 10^{80}

Thus $\alpha(2) = 0$, $\alpha(8) = 3$, and $\alpha(2048) = 4$. Other texts may use a two-parameter convention; its numerical values cannot be substituted here.

The complete bound is $O(m(1 + \alpha(n)))$ for m external operations, including n MAKE-SET calls. Only for $n \geq 3$ can it be shortened to $O(m\alpha(n))$. Small instances still require constant work per operation.

Which quantities can change?

Induction on operations shows that a root points to itself and every nonroot has smaller rank than its parent. Linking equal-rank roots raises the retained root's rank by one. Unequal ranks attach the smaller rank under the larger. Compression changes parents, never ranks.

Once a node becomes a nonroot, its rank is fixed. Its parent's rank may increase, or compression may replace the parent by a higher ancestor. Neither event decreases parent rank. This is why a node's level cannot decrease over time.

A rank-r root contains at least 2^r elements. A nonroot retains that historical bound from losing root status. Hence every rank is at most $\lfloor \log_2 n \rfloor < n$, regardless of the current compressed subtree size.

Define level and iter only for positive-rank nonroots. Rank-zero nodes and roots receive separate potential formulas.

Level selects the scale; iter measures progress within it

For a nonroot, write $r = \operatorname{rank}(x) \geq 1$, $p = \operatorname{rank}(\operatorname{parent}(x))$, and define

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

Here $\ell = \operatorname{level}$ and $i = \operatorname{iter}$. First find the highest scale reached by parent rank; then count iterations at that scale.

Since $p \geq r + 1 = A_0(r)$, we have $\ell \geq 0$. Also $A_\alpha(r) \geq A_\alpha(1) \geq n > p$, so $\ell < \alpha$. The definition gives $i \geq 1$. If $i \geq r + 1$, then

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

contradicting maximality of ℓ. Therefore $0 \leq \ell < \alpha$ **and** $1 \leq i \leq r$.

Nonnegative potential records work still available

Assign a potential to each node and sum over the forest:

$$\phi(x) = \begin{cases} \alpha r, & x \text{ is a root or } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ is a nonroot and } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

For nonroots, the lower bound is $(\alpha - (\alpha - 1))r - r = 0$, and the upper bound is $\alpha r - 1$. Roots have potential αr; rank-zero nodes have zero potential. Thus Φ is always nonnegative.

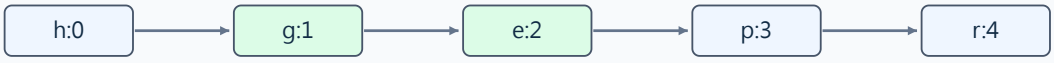
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting on a five-node path

Take $n = 16$, $\alpha = 3$, and compress a path of ranks 0, 1, 2, 3, 4. The root keeps its self-parent.

FIND values: progress through level and iter

n=16, α=3; after FIND(h), h,g,e,p all point to r. All ranks stay fixed.



Before (level,iter)				
—	(0,1)	(0,1)	(0,1)	root
After (level,iter)				
—	(1,1)	(0,2)	(0,1)	root
φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

Good nodes g,e: path potential 27→25. Exceptions: rank 0, root, last node at each level.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Each good node releases one unit: g gains a level; e increases iter within one level. The other three potentials stay fixed, so the path total falls from 27 to 25.

Why can a rising level reset iter?

Fix the rank r of a nonroot. Within one level, increasing parent rank leaves iter unchanged or increases it. Each additional iteration decreases ϕ by one.

If the level rises from k to $k' > k$, iter may reset from i to a smaller i'. Nevertheless,

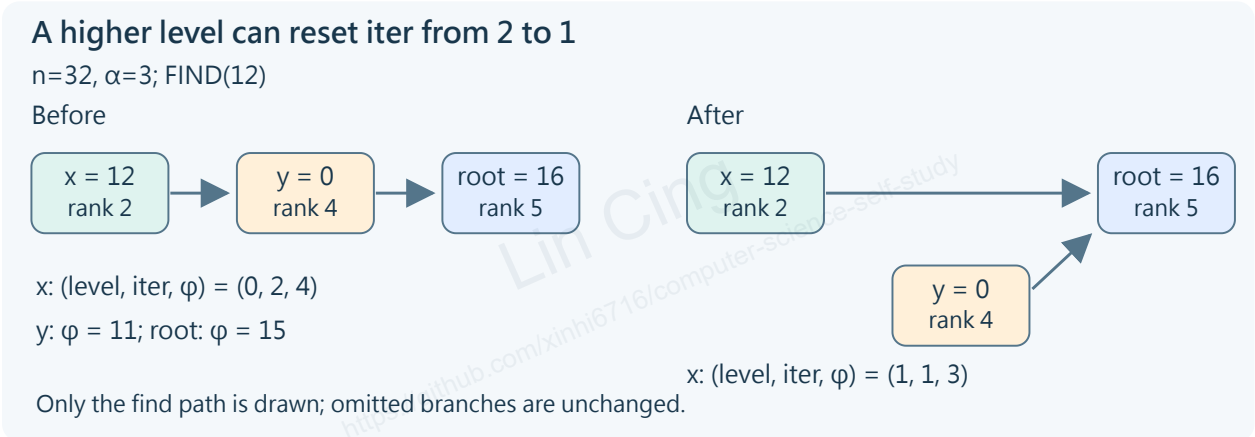
$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

This uses $k' - k \geq 1$, $i \leq r$, and $i' \geq 1$. **The gain from changing level pays for resetting iter.** The pair progresses in lexicographic order, comparing level first, then iter. Its two coordinates need not increase independently.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A legal FIND that really decreases iter

Take $n = 32$ and $\alpha = 3$. The find path is $12 \to 0 \to 16$, with ranks $2, 4, 5$. Unvisited branches are omitted and remain unchanged.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computing the reset example term by term

Node $x = 12$ keeps rank 2. Only its parent rank changes, from 4 to 5.

Quantity	Before compression	After compression
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$, so 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$, so 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$, so 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$, so 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

Node 0 keeps potential 11, and the root keeps potential 15. All other branches are unchanged. Thus the forest has $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$. The displayed totals sum only the path; the other 29 nodes need not have zero potential.

Constructing that path with legal operations

Create singleton keys 0 through 31. Equal-rank merges retain the first root.

Operation	Checkable result
Merge adjacent equal-size blocks of 0..15 in rounds	Root 0 has rank 4; node 12 has rank 2
FIND(12)	Parent of 12 becomes 0; ranks stay fixed
Build 16..31 by the same rounds	Root 16 has rank 4
UNION(16,0)	Root 16 rises to rank 5; parent of 0 becomes 16
FIND(12) again	Path $12 \to 0 \to 16$; parent of 12 becomes 16

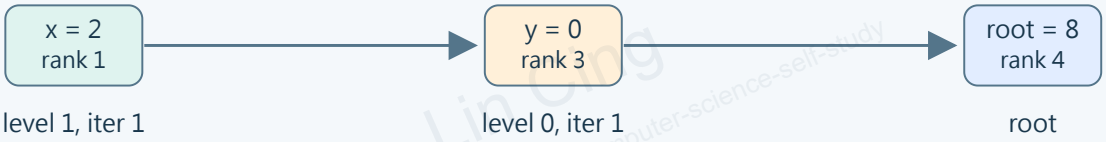
The first half starts with (0,1), (2,3), …, (14,15), then (0,2), (4,6), …, followed by (0,4), (8,12), and finally (0,8). These ranks come from a legal history rather than arbitrary assignments to a drawn chain.

Levels need not increase along a path

Take $n = 16$. Merge 0..7 into root 0 of rank 3, then call FIND(2). Build 8..15 into root 8 of rank 3 and call UNION(8,0). This creates the following legal path.

Increasing ranks do not imply increasing levels

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

Along this path: ranks 1 < 3 < 4, but levels 1 > 0.

Count the last node of each level; no sorted-level assumption is needed.

Comparing different nodes changes the input rank too. Temporal monotonicity concerns one fixed node.

Why does each good node release one unit?

A positive-rank nonroot x is good if a later nonroot y on the old path has $\ell(x) = \ell(y) = k$. The two nodes need not be adjacent.

Put $i = i(x)$. Strictly increasing path ranks, the level definition and monotonicity of A_k give

$$\text{rank}(\text{parent}(y)) \ge A_k(\text{rank}(y)) \ge A_k(\text{rank}(\text{parent}(x))) \ge A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r).$$

Compression attaches both nodes to the root, whose rank is at least the old rank of parent(y). Thus x reaches the next iteration threshold of its old level. If the level stays fixed, iter increases by at least one. If the level changes, the earlier cross-level inequality still yields $\Delta\phi(x) \le -1$.

Only the last node of each level is an exception

Group positive-rank nonroots of an s-node path by level. Except for the last node of each group along the path, every node has a later same-level node and is good. A group need not occupy consecutive positions.

There are at most α levels, numbered $0, \ldots, \alpha - 1$. Add the root and a possible rank-zero starting node: at most $\alpha + 2$ exceptions. Strictly increasing ranks rule out a second rank-zero node.

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

Here G is the actual number of good nodes, while g is only a lower bound. The three-node reset example has $s = 3$, $\alpha = 3$, hence $g = 0$. It still has one actual good node and releases one unit. The lower bound need not be tight.

Fixed constants pay for the entire scan

Suppose two-pass find costs at most cs. Fix $C > c$ and use scaled potential $\Psi = C\Phi$. The amortized cost satisfies

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \leq cs - Cg.$$

Path length	Bound after substituting g
$s \leq \alpha + 2$	$cs \leq c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \leq C(\alpha + 2)$

Thus each FIND has amortized cost $O(1 + \alpha)$. For $s = 7$, $\alpha = 3$, $c = 2$, $C = 3$, the lower bound is $g = 2$, so $\hat{c} \leq 14 - 6 = 8$. Keep the same C throughout the sequence.

Only the retained LINK root can gain potential

MAKE-SET adds a rank-zero node with zero potential and still has amortized cost $O(1)$. Suppose LINK attaches old root u under retained root v.

Possibly affected nodes	Reason for the potential change
u	Loses root status without changing rank; potential does not rise
Old children of v	Parent rank cannot fall; the earlier monotonicity argument applies
v	Unequal ranks leave it fixed; equal ranks add one, increasing potential by α

Every other node keeps its rank, parent rank and root status. Therefore $\Delta\Psi \le C\alpha$, and LINK costs $O(1) + C\alpha = O(1+\alpha)$ amortized. UNION performs two FIND calls and links only distinct roots, expanding into at most three operations.

Recovering actual sequence cost

Let c_j be actual cost and $a_j = c_j + \Psi_j - \Psi_{j-1}$ amortized cost. Summing cancels intermediate potentials:

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \le \sum_{j=1}^{m'} a_j.$$

The inequality uses the empty initial forest, $\Psi_0 = 0$, and nonnegative final potential. Expanding UNION gives $m' \le 3m$, so the original m operations cost $O(m(1+\alpha(n)))$ in total.

This is a worst-case bound on the entire sequence. With rank and compression, a single FIND can still cost $\Theta(\log n)$: build a balanced rank tree and first query a deepest leaf. A linear chain requires removing the rank rule and does not satisfy this theorem's hypotheses.

Potential analysis reference: CLRS, third edition, Section 21.4, printed pp. 576–581.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Minimum spanning trees

Part VI Graph Algorithms

Scope

Safe edges across cuts reduce the cost of connectivity. Exchange arguments establish Kruskal and Prim. One five-vertex graph tracks components, keys, and parents, leading to independently checkable certificates of optimality and uniqueness.

Prerequisites

Connected undirected graphs, cuts, and weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Derive MST safety from the cut property.
- Compare Kruskal and Prim, including their structures and costs.
- Identify assumptions about negative weights, disconnected graphs, and uniqueness.
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

Minimize total tree weight, not each path length

Let G=(V,E) be a finite undirected graph, with n=|V|,m=|E| and comparable finite real edge weights w(e). First assume n≥1 and G connected. A spanning tree T contains all vertices, is connected and acyclic, and has exactly n−1 edges. An MST minimizes $w(T) = \sum_{e \in T} w(e)$. There are finitely many candidate trees, so a minimum exists. Weights may be negative, zero, or repeated.

Negative weights do not invalidate MSTs, but acyclicity remains essential. In a triangle with all weights−1, a tree costs−2, whereas the general connected subgraph using all edges costs−3. An MST need not be a shortest-path tree. For ab:4,ac:5,bc:2, the MST {ab,bc} costs 6, and its a-to-c path has length6; the shortest graph path ac has length5.

For n=1, return no edges and cost 0; for n=0, use the empty-forest convention. A disconnected graph has no spanning tree. Its minimum spanning forest, MSF, consists of an MST in each original component. Self-loops belong to no tree. Parallel edges retain distinct edge IDs, so endpoints alone do not identify a removable edge.

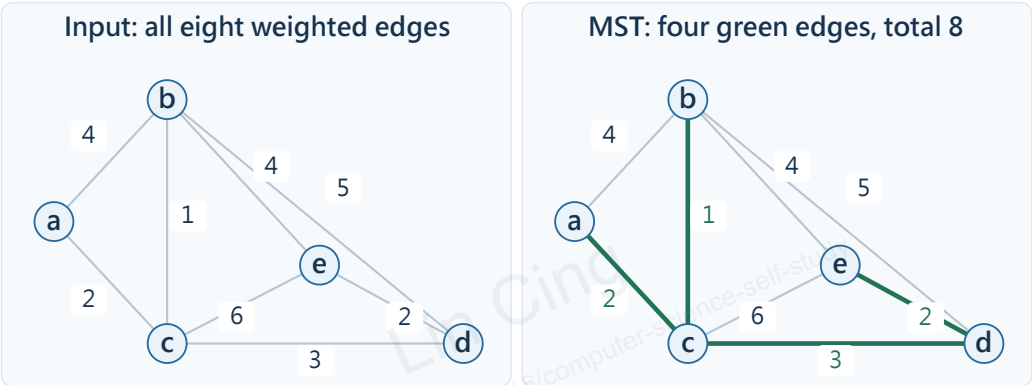
One five-vertex graph, two growth orders

Use the following fixed input throughout. Kruskal breaks weight ties by edge ID.

ID	0	1	2	3	4	5	6	7
Edge	ab	ac	bc	bd	cd	ce	de	be
Weight	4	2	1	5	3	6	2	4

Kruskal selects bc,ac,de,cd; Prim from a selects ac,cb,cd,de. Both end with {bc,ac,de,cd}, costing1+2+2+3=8, but their intermediate forests and grown vertex sets differ. Gray edges also belong to the input. The final green tree alone does not determine the selection process.

One five-vertex graph, two growth orders (continued)



Safe means extendible to an optimal tree

Maintain that $A \subseteq E$ is contained in at least one MST. An edge $e \notin A$ is safe for A if some MST contains $A \cup \{e\}$. This optimal-extension requirement is stronger than adding e without a cycle, which only preserves a forest.

A cut $(S, V-S)$ requires $\varnothing \neq S \neq V$. An edge crosses it when its endpoints lie on opposite sides. The cut respects A if no A edge crosses it. A light edge has minimum weight among the crossing edges, possibly tied.

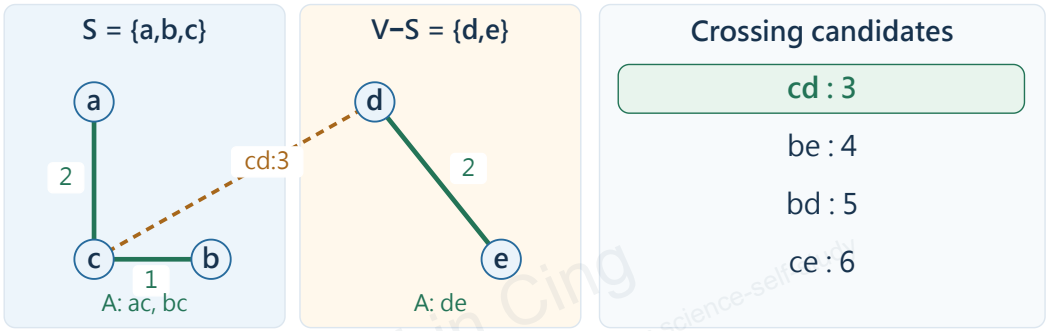
The generic algorithm starts with $A=\varnothing$ and repeatedly adds a safe edge until $|A|=n-1$. The invariant ensures acyclicity. A forest on n vertices with $n-1$ edges is connected, so the result is an MST. The algorithmic task is to establish safety efficiently; choosing a cheap edge is not itself a proof.

Safe-edge theorem and preservation of selected edges

Suppose A extends to an MST, cut (S,V−S) respects A, and e is a light edge of the cut. Choose an MST T containing A. If e∈T, the claim already holds. Otherwise T+e has exactly one cycle. Since e crosses the cut, the cycle must cross it again at some edge f to return to its starting side. Since the cut respects A, f∉A.

Removing f breaks the cycle while preserving connectivity, so $T' = T - \{f\} + \{e\}$ is a spanning tree. Because $w(e) \le w(f)$, its cost is at most w(T). Optimality of T makes T′ an MST containing $A \cup \{e\}$. The exchange proves structural validity, nonincreasing cost, and preservation of A.

Safe-edge theorem and preservation of selected edges (continued)



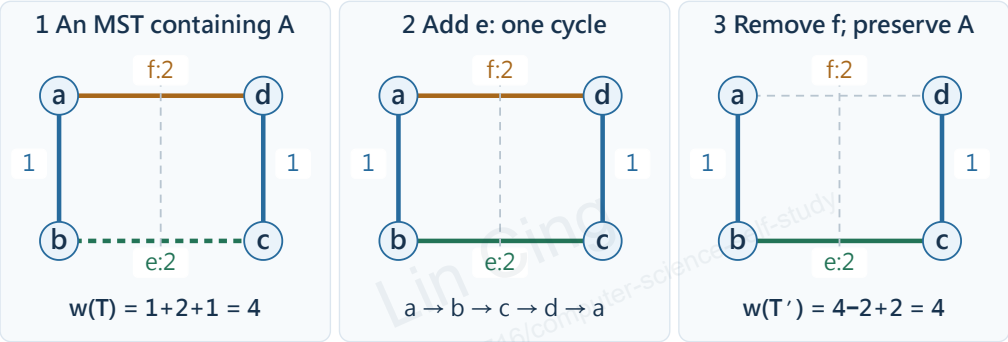
No selected edge crosses the cut; cd is the lightest crossing edge.

Respect A: preserve selected edges, then safely add cd

Here S={a,b,c},A={ac,bc,de}. Crossing edges are cd:3,be:4,bd:5,ce:6, so cd is safe. Edge de lies entirely on the other side.

An exchange in three stages

Four vertices: ab:1, cd:1, ad:2, bc:2, ac:4, bd:5. Fix A={ab,cd} and cut {a,b}|{c,d}.



Blue A={ab,cd} is preserved. f is outside A. Dashed green e is the candidate.

Equal-weight exchange: same optimum, now containing e

T attains the three-edge lower bound 1+1+2=4. Add e=bc, then remove f=ad from cycle a-b-c-d-a. Cost stays 4 and A is preserved. **Some MST contains A and e; the original T omits e.**

Quantifiers for ties, forced edges, and cycles

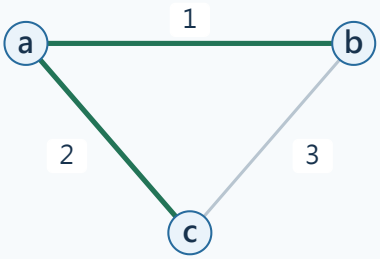
If e is the **unique** light edge of a cut, an MST omitting it could be strictly improved by the exchange. Thus every MST contains e. A tied light edge need only belong to some MST. A triangle with all weights 1 has three MSTs; every edge is light for some cut, but all three cannot be selected together.

Cycle property: a uniquely heaviest edge e on a cycle belongs to no MST. If an MST T contained e, removing it would define a cut. The remaining original cycle path supplies a crossing edge f with w(f)<w(e), giving a strict improvement. If e is only tied for heaviest, a nonincreasing exchange proves that some MST omits it, not that every MST must omit it.

Every component C of forest A defines a respecting cut (C,V−C). Its lightest outgoing edge is therefore safe. Kruskal and Prim share this argument; neither requires selecting the globally cheapest edge at every iteration.

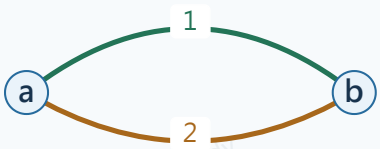
The second-lightest edge depends on the graph model

Simple graph: the first three edges



Second edge can occur; third need not

Parallel edges invalidate the claim



The unique MST uses only weight 1

The second-edge claim requires a connected simple graph

Let e be second by nondecreasing weight, with arbitrary ties. If an MST T omits e, the cycle in T+e has at least two other edges, but at most one edge is strictly lighter than e. Some cycle edge f has $w(f) \geq w(e)$; removing f gives an MST containing e. **Parallel edges invalidate the cycle-length premise. The third edge has no such guarantee.**

23.2 The algorithms of Kruskal and Prim

Kruskal merges components in global weight order

```
KRUSKAL(V,E):
    A=empty set
    for v in V: MAKE-SET(v)
    sort E nondecreasingly by (weight,edge-ID)
    for e=(u,v) in sorted E:
        ru=FIND(u); rv=FIND(v)
        if ru!=rv:
            A.add(e); UNION(ru,rv)
    return A
```

The DSU represents current components of (V,A), not final components of the original graph. Equal roots mean an existing A path, so adding e would cycle; this also rejects self-loops. Distinct roots join two trees with one edge and preserve acyclicity. For connected input, stop after accepting n−1 edges. For an MSF, scanning all input edges is also valid.

Kruskal's correctness and earlier cheap edges

Assume A extends to an MST before the next accepted edge e=(u,v). Let S be u's current DSU component, whose cut respects A. Any crossing edge f preceding e has already been processed. If accepted then, it joined its endpoints permanently, so it cannot cross the current S. If rejected then, its endpoints were already in one component and remain together, also impossible for a crossing edge. Thus no earlier edge crosses this cut.

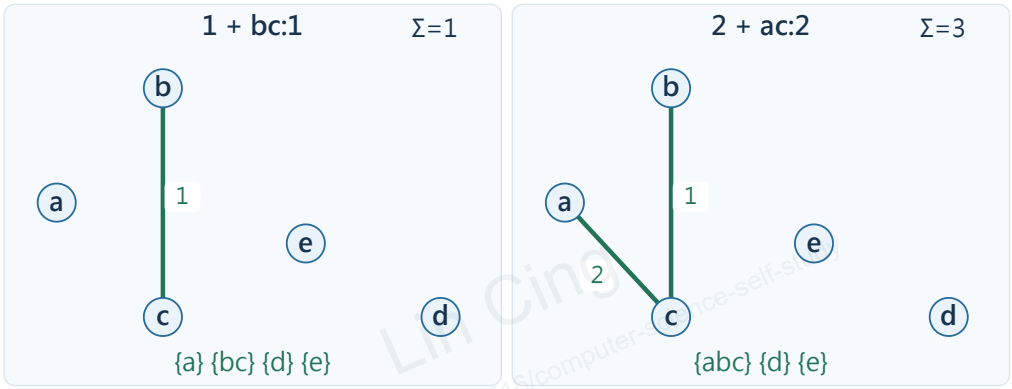
Unscanned edges have weight at least w(e), making e light for the cut. The safe-edge theorem preserves optimal extendibility. At termination, if an original component still consisted of multiple A components, an original connecting path would contain an edge crossing A components. That edge could not have been rejected when scanned, a contradiction. Thus every original component receives an optimal spanning tree, and the output is an MSF.

The proof requires neither distinct weights nor minimum-vertex DSU representatives. Root identities are implementation details.

Every Kruskal state

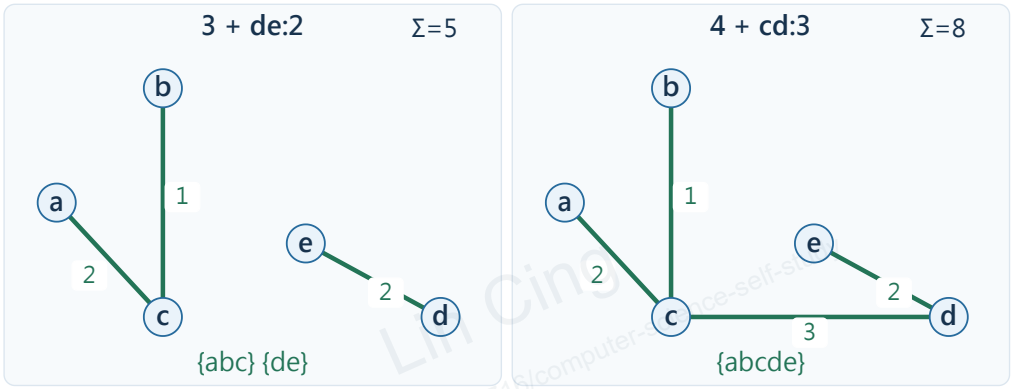
Scanned edge	Endpoint components before scan	Decision	Components afterward	Cumulative cost
bc:1	{b},{c}	accept	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	accept	{abc},{d},{e}	3
de:2	{d},{e}	accept	{abc},{de}	5
cd:3	{abc},{de}	accept	{abcde}	8
ab:4	same component	reject	unchanged	8
be:4	same component	reject	unchanged	8
bd:5	same component	reject	unchanged	8
ce:6	same component	reject	unchanged	8

Every Kruskal state (continued)



Only selected edges shown; component sets appear below

Grow two trees, then join them with one edge



Round 3 grows a separate de tree; Kruskal grows a forest

Step 4 already accepts n−1=4 edges and permits stopping. The final four rows check the full-scan version. At step 3, de need not touch {abc}: Kruskal grows a forest rather than Prim's single tree.

Kruskal's cost and graph assumptions

Comparison sorting costs O(m log(m+1)). Rank-and-compression DSU handles n MAKE-SET calls,2m FIND calls, and at most n−1 UNION calls in O((n+m)(1+α(n))). Under the Chapter 21 convention α(2)=0, so only n≥3 permits the simplification O((n+m)α(n)); retain the constant term for smaller n. Including input gives O(m log(m+1)+(n+m)(1+α(n))). An empty graph returns an empty forest directly. The sorted edge list uses O(m) space, DSU O(n), and output O(n).

For a connected **simple graph**, $n-1 \leq m < n^2$, so log m=O(log n), yielding the familiar O(m log n). Arbitrarily many parallel edges invalidate $m<n^2$; retain log m unless an additional algorithm removes redundant edges. Negative and equal weights do not affect comparison sorting.

For integer weights 0..W, counting sort can replace comparison sorting in O(m+W), but DSU cost remains. A large W can make this worse. This changes the representation assumption behind the comparison lower bound.

Prim's key is one crossing edge, not path distance

For the grown set S, every v∉S maintains $key[v] = \min_{u \in S, (u,v) \in E} w(u, v)$, or∞ when no such edge exists. Its parent records the endpoint in S and edge ID realizing the value. Each unselected vertex has one priority-queue item. Set the first root's key to 0 only to extract it first; this zero is not an output edge weight.

Extract a minimum key each iteration. Except at the initial root, its parent edge is a light edge across (S,V−S) and is safe. After extracting u, compare each new candidate w(u,v) directly with key[v], not key[u]+w(u,v).

For ab:4,ac:5,bc:2, extracting a gives keys b4,c5. Extracting b lowers c's key to 2, although the tree path from a costs 4+2=6. Dijkstra compares d[b]+2=6 and retains d[c]=5. Prim permits negative weights; Dijkstra's correctness has its own nonnegative-edge assumption.

Prim's complete queue contract

```
PRIM-CONNECTED(V,Adj,r):
    for v in V: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=empty set
    while Q is not empty:
        u=EXTRACT-MIN(Q); inQ[u]=false
        if key[u]==∞: report disconnected input
        if parent[u]!=NIL: A.add(the edge recorded in parent[u])
        for each edge e=(u,v) in Adj[u]:
            if inQ[v] and w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    return A
```

DECREASE-KEY synchronizes heap and key storage. Array inQ gives O(1) membership; scanning the heap for membership changes the cost. For an MSF, extracting∞ starts a new original component: make u a root, set key[u]=0, add no parent edge, and scan normally. Do not add∞ to the forest cost.

Prim's invariant, termination, and complete trace

At each iteration start S=V−Q; A is a tree on S, empty before the first iteration, contained in an MST. Each queued vertex's key and parent specify its lightest edge to S. Initialization handles the root separately. Extraction adds a safe edge by the cut theorem, and scanning u supplies exactly all newly available crossing candidates. Thus the invariant is restored. Each iteration removes one vertex, so n iterations produce an n−1-edge MST.

Extracted item	Remaining key(parent) values after scan	Accepted edges	Cost
a:0	b4(a),c2(a),d∞,e∞	none	0
c:2	b1(c),d3(c),e6(c)	ac	2
b:1	d3(c),e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	empty	ac,cb,cd,de	8

Prim's invariant, termination, and complete trace (continued)

Extracted keys0,2,1,3,2 are not nondecreasing. A new vertex can provide a cheaper crossing edge. Optimality follows from cuts, rather than key monotonicity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New candidates determine the next safe edge

Prim: adding c changes the next light edge

Parent in parentheses. Right arrows show replaced values.

Before c: S={a}, next c:2		⇒	After c: S={a,c}, next b:1	
b	4 (a)		b	1 (c) ← old 4(a)
c	2 (a)		c	extracted
d	∞		d	3 (c) ← old∞
e	∞		e	6 (c) ← old∞

A key is one lightest edge to S. Adding c lowers b from 4 to 1; extracted keys need not increase.

For S={a}, ac:2 is lightest. Extracting c lowers b from ab:4 to cb:1 and gives d,e their first finite keys. The next extraction must be b:1; having just extracted2 cannot exclude weight 1. Extracting b replaces ce:6 by be:4 for e, and extracting d later replaces that by de:2.

New candidates determine the next safe edge (continued)

Save every parent change together with the corresponding key. The final parent need not be the first encountered neighbor. Vertex extraction order alone cannot reconstruct these competing candidates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Keep Prim's implementation costs distinct

Queue and representation	Work	Total cost
Indexed binary heap and adjacency lists	n extracts, at most2m scanned entries and m decreases	O((n+m)log n)
Fibonacci heap and adjacency lists	O(n log n) amortized extraction, O(m) amortized decreases	O(m+n log n)
Linear minimum-key search and adjacency lists	n scans of at most n keys,2m edge entries	$O(n^2+m)$
Adjacency matrix and linear keys	n rounds scanning n keys and n neighbor positions	$\Theta(n^2)$

For a simple graph, $m=O(n^2)$, making the third row $O(n^2)$. Handle n=1 directly with an empty tree. Indexed handles locate a decrease in O(1). A lazy heap that pushes replacement entries and discards stale ones can contain O(m) items; the n-item proof does not apply unchanged.

Keep Prim's implementation costs distinct (continued)

For m=Θ(n), both heap variants give O(n log n). For m=Θ(n^2), binary and Fibonacci bounds are respectively O(n^2 log n) and O(n^2), and linear-key scanning also gives O(n^2). The executable example verifies choices using dense Prim; it does not claim a measured Fibonacci-heap speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Certify a candidate tree without rerunning the algorithm

Let T be a spanning tree. Quantify only over **non-self-loop nontree edges** e=(u,v),u≠v, and let M_T(u,v) be the maximum weight on the unique T path from u to v. Ignore self-loops; no maximum is assigned to an empty u-to-u path. Then

$$T \text{ is an MST} \quad \Longleftrightarrow \quad \forall e = (u, v) \in E \setminus T,\ u \neq v :\ w(e) \geq M_T(u, v).$$

Necessity follows by adding any e with w(e)<M and removing a maximum path edge, strictly improving T. For sufficiency, remove any f∈T to define a cut. Every other crossing edge e is outside T and its T path uses f, giving w(e)≥M_T≥w(f). Thus f is light across that cut. No other T edge crosses it, so the cut respects any previously selected prefix of T. Add T's edges in any order and repeatedly apply the safe-edge theorem to prove optimality.

For nontree edges ab,bd,ce,be in the fixed graph, pairs (w,M) are (4,2),(5,3),(6,3),(4,3). All inequalities are strict, proving uniqueness here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness and changing one edge weight

Tree T is unique exactly when every non-self-loop nontree edge has w(e)>M_T. Equality allows replacing a maximum path edge with e at equal cost. Conversely, strictness makes f the unique light edge of every T−f cut. Every MST must then contain every edge of T. This does not require **all input weights to differ**: the example has two weight 2 edges and a unique MST.

Lower only one non-self-loop nontree edge e to w′, keeping everything else fixed, and let M be its original T-path maximum. If w′≥M, T still passes every certificate. If w′<M, replace a maximum path edge f by e. The new optimum is w(T)+min(0,w′−M). For a lower bound, a tree omitting e retains original weight at least w(T). For any new tree R containing e, remove e to define a cut. The original T path between its endpoints contains a crossing edge g. Tree R−e+g exists in the original weighted graph, hence w(T)≤w(R)−w′+w(g)≤w(R)−w′+M. This matches the constructed bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

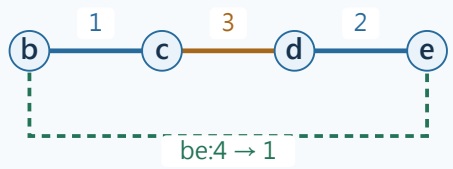
Uniqueness and changing one edge weight (continued)

Lowering be from 4 to 1 uses path b-c-d-e with weights 1,3,2, so remove cd:3 and get8−3+1=6. Lowering only to 3 leaves the old tree optimal but creates a tie. A certificate using a separate DFS per path costs O(nm). This update guarantee concerns one changed edge. No self-loop, however negative, can enter a tree; changing its weight leaves the MST unchanged and never triggers a tree-path exchange.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

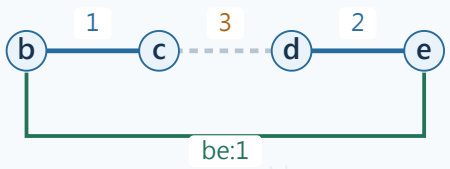
Uniqueness and changing one edge weight (continued)

1 Read the unique tree path b to e



$M_T(b,e)=3$

2 Lower be to 1; replace cd:3



$8-3+1=6$

Edge ac:2 is unchanged; this view enlarges only the affected b-c-d-e path.

Replace a path maximum only when the new weight is smaller: $w' < M_T$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Single-source shortest paths

Part VI Graph Algorithms

Scope

Relaxation maintains upper bounds and predecessors reconstruct paths. We prove the distinct contracts of Bellman–Ford, DAG shortest paths, and Dijkstra, then use negative-cycle certificates, difference constraints, and rooted-tree verification to distinguish finite distance, unreachability, and unbounded cost.

Prerequisites

Edge relaxation, negative weights, and reachability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Maintain shortest-path and triangle-inequality invariants.
- Analyze Bellman–Ford, DAG shortest paths, and Dijkstra separately.
- Convert difference constraints to weighted directed graphs.
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

Three distance meanings; negative edges differ from cycles

Let $G=(V, E)$ be finite and directed, with $n \geq 1$, $s \in V$ and finite real edge weights. Path cost sums edge weights; a walk may repeat vertices and contain cycles. Define $\delta(s, v)$ as the infimum of costs of all s-to-v walks. It is ∞ if unreachable, $-\infty$ if a negative cycle can be visited on the way to v, and otherwise a finite minimum. The source's zero-length walk costs 0, so $\delta(s, s) \leq 0$.

If a walk to v has no usable negative cycle, deleting positive cycles improves its cost and deleting zero cycles preserves it. A simple shortest path of at most $n-1$ edges therefore exists. A negative edge alone does not make distances undefined. A negative cycle affects exactly vertices **reachable onward from that cycle when the cycle is reachable from s**; other branches can retain finite distances.

A single-destination problem becomes single-source after reversing all edges. A single-pair answer is one entry of the single-source result. Representing a negative undirected edge by two opposite arcs creates a negative closed walk; intuition about an isolated directed negative edge no longer applies.

Relaxation only improves tentative upper bounds

```
INITIALIZE(s):
    for v in V: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u, v)):
    if d[u]!=∞ and d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u, e.ID)
        return true
    return false
```

Induction on relaxations shows every finite $d[v]$ is the cost of an actual s-to-v walk, hence $d[v] \geq \delta(s, v)$. Only strict improvements change parents. Parallel edges require IDs. Initialization is $\Theta(n)$, and each relaxation $O(1)$ under exact unit-cost arithmetic. Fixed-width implementations must avoid overflow and represent ∞ separately. Clamping negative values to an arbitrary floor can destroy negative-cycle detection.

Once d reaches a finite δ, it cannot increase or fall below δ and stays fixed. Finiteness matters: $-\infty$ is not the ordinary finite output of a single relaxation.

Bellman–Ford operations, early stopping, and cost

```
BELLMAN-FORD(G, s):
    INITIALIZE(s)
    repeat at most n−1 rounds:
        changed=false
        for each e in E in a fixed order:
            if RELAX(e): changed=true
        if not changed: return d, parent,"no reachable negative cycle"
    for each (u, v) in E:
        if d[u]!=∞ and d[v]>d[u]+w(u, v):
            return "reachable negative cycle exists"
    return d, parent,"no reachable negative cycle"
```

The last scan tests for a cycle; a still-decreasing finite estimate is not delivered as a shortest distance. After an unchanged round, every reachable edge satisfies $d[v] \leq d[u]+w$. Repeating the same scan cannot change anything, permitting early return. For $n=1$ the final test must still inspect self-loops and detect a negative one.

Bellman–Ford operations, early stopping, and cost (continued)

The full-round version takes $\Theta(n+nm)$, often written $\Theta(nm)$ when $m \geq 1$, $n \geq 2$. Early stopping has best case $\Theta(n+m)$ and the same worst case. Arrays d, parent use $O(n)$ extra space; input uses $O(n+m)$. Complete negative-cycle classification and certificates can use the DP below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

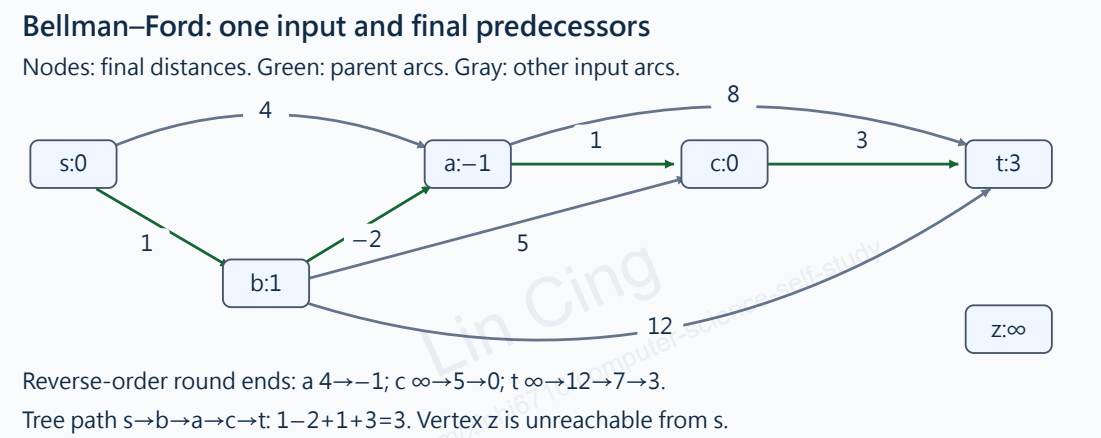
Reverse edge order forces several propagation rounds

Vertices are (s, a, b, c, t, z). Scan the **fixed ordered edges** ct:3, at:8, ac:1, bt:12, bc:5, ba:−2, sa:4, sb:1. Each edge points from its first letter to its second. Vertex z is isolated.

Round end	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

In round 2, old a=4 first gives t=12, c=5; only later does ba lower a to−1. In round 3, ct first gives 8, at lowers it to 7, then ac sets c=0. Only round 4 obtains t=3. Round 5 changes nothing. Final predecessors form s→b→a→c→t, cost 1−2+1+3=3.

Reverse edge order forces several propagation rounds (continued)



Why n−1 rounds suffice and detect negative cycles

Without a reachable negative cycle, every reachable v has a simple shortest path $s=v_0,\ldots, v_k=v$ with $k\le n-1$. Round i scans (v_{i-1}, v_i), after round i−1 has already made $d[v_{i-1}]=\delta$. Convergence makes $d[v_i]=\delta$. Other same-round relaxations cannot spoil a true value. Thus n−1 rounds suffice. This proves the in-place algorithm covers at least i-edge paths; its state need not equal the DP minimum over paths with at most i edges.

If a negative cycle is reachable, all its vertices have finite estimates after n−1 rounds: reachability itself has a simple path of at most n−1 edges, without needing a shortest path. If no cycle edge could improve, summing $d[v]\le d[u]+w$ around it would cancel all distances and give $0\le w(\text{cycle})<0$. Thus some reachable edge must improve. Conversely, without such a cycle all distances are correct and triangle inequalities forbid improvement. These are both directions of the detection proof.

Reconstructing a certificate with an edge-count DP

Let $D_i[v]$ be the minimum cost of a walk with at most i edges, with $D_0[s]=0$ and all other entries ∞.

$$D_i[v] = \min\left(D_{i-1}[v], \min_{(u,v)\in E}(D_{i-1}[u] + w(u,v))\right).$$

Each layer reads only the preceding layer. Record skip when retaining an old value, or the edge ID and preceding-layer vertex u when taking an edge. Every walk with at most i edges either has at most i−1, or loses its last edge to become one with at most i−1. Conversely every recurrence candidate is a legal walk, proving equality.

Compute through layer n. If $D_n[v]<D_{n-1}[v]$, backtracking n layers gives an **exactly n-edge** walk: fewer edges would already qualify at layer n−1. Decompose the walk into a simple path and cycles cut at repeated vertices. If every cycle were nonnegative, deleting them would produce an at-most-(n−1)-edge path of no greater cost, contradicting strict improvement. At least one extracted cycle is negative. Its ordered edge IDs and total weight form a checkable certificate.

Reconstructing a certificate with an edge-count DP (continued)

The full-history version takes $O(n+nm)$ time and $O(n^2+m)$ space. Two distance layers suffice for classification but do not retain this backtracking table. Do not claim the in-place algorithm's $O(n)$ auxiliary space for this version.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive $-\infty$

Let seeds $B=\{v:D_n[v]<D_{n-1}[v]\}$. Search from B in the **original edge direction**, marking every reached vertex $-\infty$. Other reachable vertices keep D_{n-1}; unreachable ones remain ∞.

A seed's n-edge walk contains a negative cycle, so the seed and its descendants are affected. Conversely, every reachable negative cycle has finite entries in D_{n-1}. If it had no seed, each of its edges would satisfy $D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$; summing gives a contradiction. Every such cycle therefore has a seed, whose forward search reaches all affected descendants. The classification covers exactly the affected set, rather than only cycle vertices.

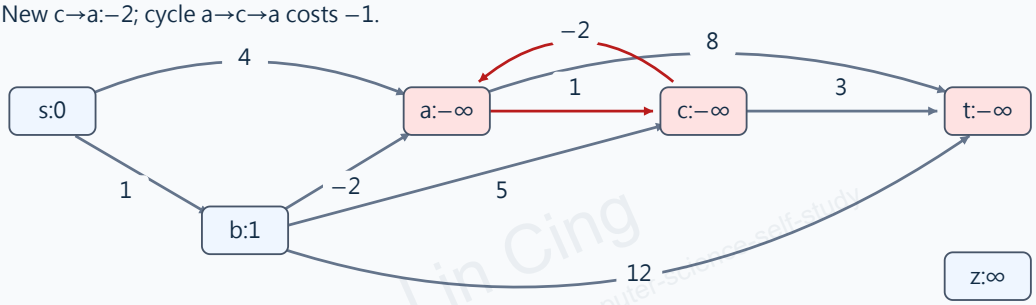
Add $c\to a:-2$ to the earlier graph. Cycle $a\to c\to a$ costs $1-2=-1$. Distances on (s, a, b, c, t, z) become $(0,-\infty,1,-\infty,-\infty,\infty)$. The cycle cannot return to s or b, which stay finite; z stays unreachable. An additional unreachable negative cycle must not cause a false single-source report.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive −∞ (continued)

A reachable negative cycle affects only forward descendants

New c→a:−2; cycle a→c→a costs −1.



Descendants a, c, t receive−∞; b=1, s=0. No cycle exit returns to s or b.

A negative cycle does not erase finite branches or unreachable z.

24.2 Single-source shortest paths in directed acyclic graphs

Topological order delivers every candidate in time

```
DAG-SHORTEST(G, s):
    order=TOPOLOGICAL-SORT(G)
    if a cycle is found: reject this algorithm's input
    INITIALIZE(s)
    for u in order:
        for each edge e in Adj[u]: RELAX(e)
    return d, parent
```

Every s-to-v path is relaxed in its path order, so path relaxation gives δ at its end. Equivalently, induct on topological position: all incoming sources have been finalized and scanned before v is processed, leaving no future better candidate. Unreachable vertices retain∞. Negative edges are allowed because no cycle means no negative cycle. Sorting and scanning together take Θ(n+m), with O(n) auxiliary space.

24.2 Single-source shortest paths in directed acyclic graphs (continued)

The earlier graph is already a DAG. In order s, b, a, c, t, z, scanning s gives a4, b1; b gives a−1, c6, t13; a gives c0, t7; c gives t3. One pass matches four Bellman–Ford rounds. The difference is **topological vertex order**, not the cost definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs

Negate every DAG weight and negate the resulting shortest cost to obtain its longest cost. Alternatively initialize L[s]=0, others−∞, and use maximum relaxation. A cyclic graph cannot use the DAG algorithm unchanged; positive cycles make longest walks unbounded.

For vertex jobs of duration[v]≥0, an edge u→v requires u to finish before v starts. In topological order compute F[v]=duration[v]+max({F[u]:u→v}∪{0}). Backtracking a maximizing predecessor gives a critical path.

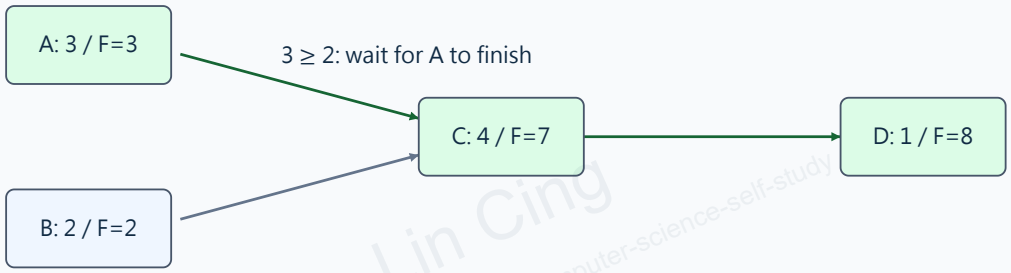
For A:3, B:2, C:4, D:1 with A→C, B→C, C→D, finishes are F[A]=3, F[B]=2, F[C]=4+max(3,2)=7, F[D]=1+7=8. Critical path A, C, D costs 8. With unlimited processors and only these dependencies, starting each eligible job immediately attains this lower bound. Finite processors add resource constraints, so the critical-path bound alone is not a full schedule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs (continued)

Job DAG: the slower predecessor determines completion

Boxes: job / duration / earliest finish F. Green arrows: critical path.



F[C]=4+max(3,2)=7；F[D]=1+7=8。

Unlimited processors finish at 8. Total work 10 requires at least 10 on one processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.3 Dijkstra’ s algorithm

Extract the minimum tentative distance and never reopen it

Assume **every edge weight is nonnegative**. An indexed min-heap holds one item per unextracted vertex with key=d. Initialize d[s]=0, others∞, parent=NIL, inQ=true.

```
while Q is not empty:
    u=EXTRACT-MIN(Q); inQ[u]=false
    if d[u]==∞: stop; all remaining vertices are unreachable
    for e=(u, v) in Adj[u]:
        if inQ[v] and d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u, e.ID)
            DECREASE-KEY(Q, v, d[v])
```

Extracted vertices S have true distances. Unlike Prim, the candidate is **total path cost d[u]+w**. An extracted vertex is neither updated nor reinserted. A single-pair query may stop when the target is actually extracted, not when first discovered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The precise use of nonnegative weights

Suppose the next finite extraction u is the first wrong one. On a true shortest path s→…→x→y→…→u, let y be the first unextracted vertex and x its extracted predecessor. The source is extracted first with d[s]=0. Otherwise x already has its true distance and has scanned xy, so convergence gives d[y]=δ(s, y).

The suffix y-to-u has nonnegative weight, giving δ(s, y)≤δ(s, u). Minimum extraction gives d[u]≤d[y]; the upper-bound property gives δ(s, u)≤d[u]. Thus

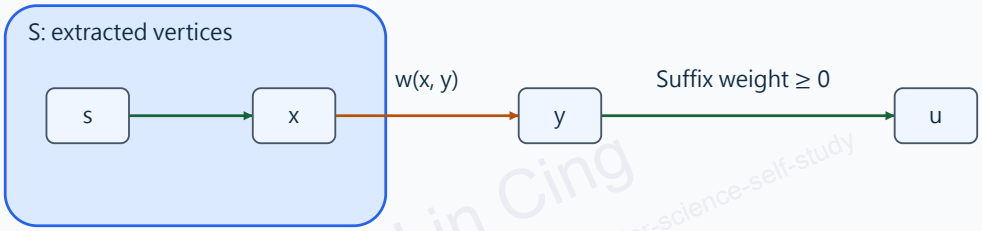
$$\delta(s, u) \le d[u] \le d[y] = \delta(s, y) \le \delta(s, u).$$

All inequalities are equalities, contradicting error. Zero edges and distance ties are allowed; strict increases are unnecessary. If an∞ item is extracted while a reachable vertex remains, the first unextracted vertex on its path would have a finite candidate, contradicting minimum extraction.

The precise use of nonnegative weights (continued)

Dijkstra: bound u using the first unextracted vertex

Follow one true shortest path. Orange: the first arc leaving S.



Vertex x is finalized and xy scanned ⇒ d[y]=δ(s, y).

Minimum extraction and a nonnegative suffix give d[u]≤d[y]=δ(s, y)≤δ(s, u).

Numerical trace and a genuine negative-edge failure

Change only ba in the earlier example to+2. States after scans are

Extraction	Unextracted finite d(parent)	Finalized distances
s0	a4(s), b1(s)	s0
b1	a3(b), c6(b), t13(b)	s0, b1
a3	c4(a), t11(a)	also a3
c4	t7(c)	also c4
t7	none; z∞	also t7

The shortest-path tree is s→b→a→c→t, cost 1+2+1+3=7. Target estimates13→11→7 demonstrate why first discovery is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical trace and a genuine negative-edge failure (continued)

For a failure, use s→a:2, s→b:5, b→a:−4, a→t:2. After s, extract a2 and assign t4; extract t4 before b5, which would give a1 and true t3. The standard no-reopening algorithm retains wrong answers. Updating an extracted a without rescanning its outgoing edges also leaves t wrong. This graph is a DAG with no negative cycle. The failed premise is δ(s, b)=5>δ(s, a)=1, contradicting the proof's nonnegative-suffix comparison.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queue, graph representation, and exact arithmetic costs

An indexed binary heap takes O(n) to build, O(n log n) for n extracts, and O(m log n) for at most one decrease per scanned edge, totaling O((n+m)log n), with n=1 handled directly. Fibonacci decreases take O(1) amortized, giving O(m+n log n). Handles locate heap items, and inQ is an O(1) array lookup.

Linear minimum-distance scanning with adjacency lists costs $O(n^2+m)$, simplifying to $O(n^2)$ for simple graphs. A lazy heap pushes all new candidates and can hold m items; it requires a separate O((n+m)log(m+1)) argument and O(n+m) space. The indexed n-item bound does not transfer automatically. The numerical example uses linear-key scanning and adjacency lists.

These bounds use a real-RAM model. With B-bit integer weights and no negative cycles, simple-path costs need O(B+log n) bits; exact addition and comparison costs must be included separately. Floating-point error or arbitrary epsilons can change equality and cycle decisions. Mathematical correctness alone does not establish numerical stability.

24.4 Difference constraints and shortest paths

Derive the edge direction algebraically

Constraint $x_j - x_i \le b$ corresponds to arc $i \to j:b$ because $x_j \le x_i + b$. Each coefficient row has one+1 and one−1, with unrestricted real variables. Rewrite $x_j - x_i \ge b$ as $x_i - x_j \le -b$. Split an equality into two opposite inequalities.

Add a super-source q with zero-weight arcs to every variable and no incoming arc, making all original negative cycles reachable. Without a negative cycle, set $x_i = \delta(q, i)$. Each edge's triangle inequality gives $x_j - x_i \le b$. Conversely, summing constraints around a negative cycle cancels every variable and gives $0 \le \Sigma b < 0$. Thus feasibility is equivalent to absence of negative cycles.

For n variables and m constraints, the augmented graph has n+1 vertices, m+n edges, giving $O(n^2+nm)$ Bellman–Ford time. Instead initialize all original distances to 0, as though q's arcs have already been processed, and scan the original edges for at most n−1 rounds plus a test. Cost becomes O(n+nm), including initialization when m=0.

Complete feasible and infeasible examples

Constraints $x_2-x_1\le -2$, $x_3-x_2\le -1$, $x_1-x_3\le 4$ give arcs$1\to 2:-2, 2\to 3:-1, 3\to 1:4$. From all zero, the first scan gives $(0,-2,-3)$; $-3+4=1\ge 0$ does not change x_1. The second round is unchanged.

Check each inequality: $-2-0=-2\le -2$; $-3-(-2)=-1\le -1$; $0-(-3)=3\le 4$. Adding 3 gives $(3,1,0)$, also feasible because differences are unchanged. The feasible region is invariant under translation along the all-ones vector and has no fixed origin.

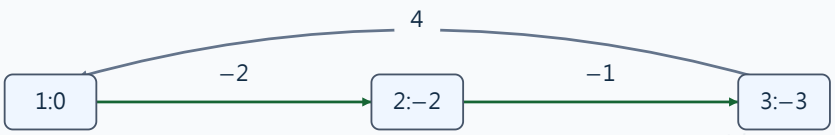
Changing the last constant to 2 makes the sum$0\le -2-1+2=-1$, proving infeasibility. The negative cycle is $1\to 2\to 3\to 1$. Failure of one proposed vector alone would not prove absence of every feasible vector.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete feasible and infeasible examples (continued)

Difference constraints: a feasible vector and a cycle contradiction

Arc $i\to j:b$ means $x_j-x_i\le b$. Node values are a feasible vector x.



Differences: $-2-0=-2$; $-3-(-2)=-1$; $0-(-3)=3\le 4$.

Change the last weight 4 to 2: summing gives $0\le -2-1+2=-1$, a contradiction.

Original cycle weight 1: feasible. New weight -1: infeasible. Translation preserves differences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extensions that retain the proof

Equality $x_j - x_i = b$ gives $i \to j : b$ and $j \to i : -b$. Together they force equality. To express $x_i \le c$ or $-x_i \le c$, introduce a reference r and respectively $r \to i : c$ or $i \to r : c$. After solving, translate all coordinates so $x_r = 0$ before treating them as absolute bounds. Translation must preserve this reference.

Integer right sides and integer-zero initialization make Bellman–Ford's additions and minima produce an integer feasible solution. If **all** variables must be integers but right sides are real, replace each b by $\text{floor}(b)$: integer differences make the constraints equivalent. Then test for negative cycles. This rowwise rounding is not automatically valid when only some variables must be integers.

Under the normalization $d \le 0$, shortest distances form the coordinatewise greatest feasible vector. Any feasible x satisfies $x_v \le$ the cost of every q-to-v path by repeated substitution, hence $x_v \le \delta(q, v)$. This conclusion includes $x_i \le 0$. With unrestricted translation in the original system, Σx_i can increase arbitrarily.

24.5 Proofs of shortest-paths properties

Optimal substructure, triangle inequality, and convergence

Assume no negative cycle reachable from s. A subpath of a finite shortest path must be shortest: replacing a more expensive subpath gives a cheaper s-to-v walk, a contradiction. Appending $u \to v$ to a shortest s-to-u path gives $\delta(s, v) \le \delta(s, u) + w$. An unreachable u makes the right side ∞ and imposes no additional restriction.

Initialization and the actual-walk interpretation prove the upper-bound property. An unreachable vertex cannot become finite without an actual walk. Immediately after RELAX(u, v), $d[v] \le d[u] + w$, but **only at that instant**: a later decrease in $d[u]$ can require rescanning this edge.

If u already has its true distance and $u \to v$ lies on a shortest path, relaxation gives $d[v] \le \delta(s, u) + w = \delta(s, v)$. The upper bound gives the reverse inequality, so v becomes permanently correct. This is convergence. Starting with s and applying convergence in shortest-path edge order fixes one vertex after another. Interspersed relaxations only lower upper bounds and cannot spoil correct values, establishing path relaxation.

Parent acyclicity follows from update times

Suppose a strict relaxation $u \to v$ first creates a parent cycle. Before it, the parent subgraph has a path $v \to \dots \to u$. For every old parent arc $x \to y$, the last assignment of parent[y] made $d[y] = d[x] + w$. Since y has not been reassigned and x can only decrease, immediately before this update $d[y] \geq d[x] + w$. Summing the old path gives $d[u] \geq d[v] + w(v \to \dots \to u)$.

The new strict update requires $d[v] > d[u] + w(u, v)$. Combining gives a negative total cycle weight. All its vertices are finite and reachable from s, contradicting the assumption. A negative self-loop follows directly from the same strict improvement. Thus the parent subgraph remains acyclic.

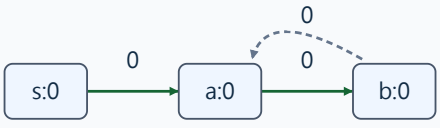
Every finite $v \neq s$ received a non-NIL parent by relaxation. The source retains $d[s] = \delta(s, s) = 0$ and parent NIL. Parent vertices of finite vertices are also finite. In a finite acyclic graph, backtracking must terminate at the only parentless finite vertex s. Each vertex has one parent, giving a rooted tree on all discovered finite vertices.

Parent acyclicity follows from update times (continued)

Zero edges: equal distances still need rooted parents

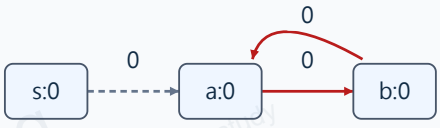
Original arcs s→a:0, a→b:0, b→a:0. All distances are 0 on both sides.

Valid parents: rooted paths from s



parent[a]=s · parent[b]=a

Invalid parents: a, b form a cycle



parent[a]=b · parent[b]=a

Green: valid tree. Red: invalid cycle. Gray dashed: unselected original arcs.

Tightness checks cost. Search parent children from s to verify connectivity and acyclicity.

Correct distances and valid parents give a shortest-path tree

Once finite d equals δ, the last-update argument for every current parent arc u→v gives d[v]≥d[u]+w. The triangle inequality gives the reverse after substituting d=δ, hence equality. Summing on the s-to-v tree path telescopes to path cost d[v]−d[s]=δ(s, v). Unreachable vertices retain∞ and NIL parents, completing all shortest-path-tree requirements.

Do not justify this by strictly decreasing predecessor distances. Zero edges permit equality and negative edges can decrease forward distances. Acyclicity depends on the strict-update-time argument. Choosing arbitrary tight predecessors afterward can form a zero cycle. In s→a:0, a→b:0, b→a:0 all distances are 0; parent[a]=b, parent[b]=a makes both edges tight without any root path.

A linear-time certificate verifies optimality too

Given d and parent edge IDs, compute the actual reachable set R from s. Require finite d in R, d[s]=0, and∞ with NIL parents outside R. For every v∈R−{s}, verify that its parent is an actual input arc u→v with d[v]=d[u]+w. Search the parent-children graph from s, visiting exactly R without repeats, and require parent[s]=NIL. Finally scan all input arcs from reachable u and check d[v]≤d[u]+w.

The tight rooted tree realizes a path of cost d[v], proving δ(s, v)≤d[v]. Summing edge inequalities along any s-to-v walk gives d[v]≤its cost, hence d[v]≤δ(s, v). Both directions establish equality. A reachable negative cycle would contradict the summed inequalities and is rejected. Verification takes Θ(n+m) time, O(n) space without rerunning Bellman–Ford or Dijkstra.

Edge inequalities alone allow unrealistically small values with no realizing paths. Tight parents alone allow zero cycles or overlook cheaper input arcs. Each certificate component proves a separate direction.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 24.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All-pairs shortest paths

Part VI Graph Algorithms

Scope

All-pairs shortest paths uses two distinct DP states, edge budgets and intermediate vertices. Min-plus products, Floyd–Warshall, and Johnson connect path reconstruction, zero cycles, affected pairs, and the signs and costs of reweighting.

Prerequisites

Min-plus matrices and predecessor representations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Connect min-plus multiplication to shortest paths.
- Derive Floyd–Warshall from intermediate-vertex states.
- Use Johnson reweighting on sparse graphs with negative edges.
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’ s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP takes a directed weighted graph on n vertices, allowing negative edges, and returns a distance for every pair(i, j). Unreachable pairs have$+\infty$. If i can reach a negative cycle that can reach j, repeating it makes walk cost unbounded below. Define that distance as$-\infty$; no finite shortest path exists for the pair.

First assume no negative cycle. Every repeated-vertex walk contains a nonnegative cycle, which can be removed without increasing cost. Repeating this leaves a simple path. For $i \neq j$, a shortest path therefore uses at most $n-1$ edges. For i=j, the zero-length path costs 0.

Matrix W[i, j] stores the lightest direct edge, or∞ if absent. Set W[i, i]=0 to allow no edge. For negative-self-loop detection, initialize to min(0, w(i, i)) rather than overwriting a negative loop with 0. All diagonals in the following cycle-free example are 0.

State as an upper bound on edge count

Let $L^{(m)}[i, j]$ be the minimum weight of a walk with **at most m edges**, not exactly m. Set $L^{(0)}=I$, with zero diagonal and∞ elsewhere. Splitting at the last edge gives

$L^{(m+1)}[i, j]=\min_k(L^{(m)}[i, k]+W[k, j])$.

Every nonempty candidate splits into an at-most-m-edge prefix and its last edge; every finite recurrence term conversely concatenates to a valid walk. Shorter old paths remain through k=j and W[j, j]=0. Both directions describe the same feasible walks. With negative cycles, a fixed finite edge budget still defines a meaningful finite optimum, but budget $n-1$ no longer gives unrestricted distances.

Define$(A \otimes B)[i, j]=\min_k(A[i, k]+B[k, j])$. Matrices for budgets a, b concatenate to budget a+b. Conversely, split any at-most-(a+b)-edge walk after min(a, its length) edges; the suffix uses at most b. Hence $L^{(a)} \otimes L^{(b)}=L^{(a+b)}$.

Min-plus algebra and two algorithms

Associativity expands both sides to the same finite minimum:$((A\otimes B)\otimes C)[i, j]=\min_{p,\varphi}(A[i, p]+B[p, q]+C[q, j])=(A\otimes(B\otimes C))[i, j]$. Matrix I is the identity;∞ is neutral for min and$\infty+$a finite value$=\infty$. Do not introduce$-\infty$ at this stage, since$\infty+(-\infty)$ is undefined.

```
Extend(A, B):
  C=new n×n matrix filled with∞
  for i, j, k: C[i, j]=min(C[i, j], A[i, k]+B[k, j])
  return C
Slow(W): L=I; repeat n−1 times: L=Extend(L, W); return L
Square(W): L=W; m=1
  while m<n−1: L=Extend(L, L); m=2m
  return L
```

Slow performs $\Theta(n)$ products of $\Theta(n^3)$, totaling $\Theta(n^4)$ for $n\geq 2$. Square performs $\operatorname{ceil}(\log_2(n-1))$ squarings, taking $O(n^3 \log n)$. Two input matrices and a fresh output need $O(n^2)$ working space. Overwriting inputs during a product invalidates its fixed old-budget interpretation. For n=0 return an empty matrix; handle n=1 directly through its diagonal.

Min-plus algebra and two algorithms (continued)

Ordinary multiplication replaces min by addition and addition by multiplication, but Strassen's subtraction formulas do not directly apply to min-plus. The analogy does not transfer fast multiplication bounds for rings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One four-vertex, six-edge example

All matrices, Floyd stages, and Johnson states below use1→2:4,1→3:7,2→3:−2,2→4:6,3→4:3,4→2:1. Vertex 1 has no incoming edge. Cycle 2→3→4→2 costs 2, so there is no negative cycle.

Rows of W are[0,4,7,∞],[∞,0,−2,6],[∞,∞,0,3],[∞,1,∞,0].

Rows of $L^{(2)}$ are[0,4,2,10],[∞,0,−2,1],[∞,4,0,3],[∞,1,−1,0].

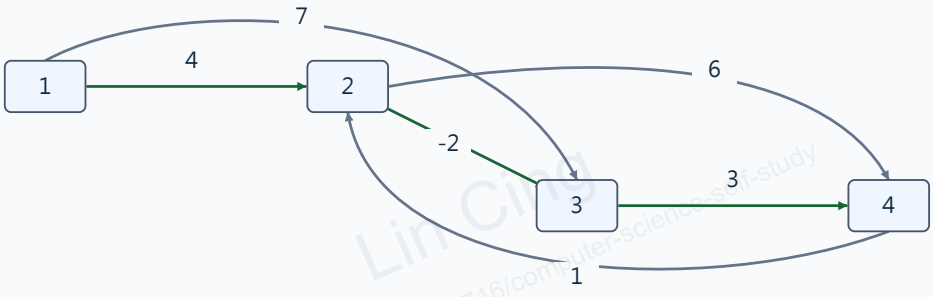
For example $L^{(2)}[1,4]=\min(0+\infty,4+6,7+3,\infty+0)=10$, with two tied two-edge paths. Also $L^{(2)}[2,4]=-2+3=1$. Only the third extension permits 1→2→3→4 and gives $L^{(3)}[1,4]=5$; all other entries stay unchanged. Since n=4, $L^{(3)}$ is the answer, as is the squared $L^{(4)}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One four-vertex, six-edge example (continued)

Six directed edges and edge-budgeted paths

Green: best 3-edge route. Gray: other input edges. Labels: original weights.



1→2→4=10 (2 edges); 1→3→4=10 (2 edges); 1→2→3→4=5 (3 edges).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.2 The Floyd-Warshall algorithm

Here k bounds the **identifiers of allowed intermediate vertices**, not edge count. State $D^{(k)}[i, j]$ is the minimum walk cost when all intermediate vertices belong to$\{1,\ldots, k\}$; endpoints i, j are unrestricted. Without negative cycles, choose a simple path, so k occurs at most once.

An optimum avoiding k belongs to old state $D^{(k-1)}$. An optimum using k splits there into segments with no internal k, giving $D^{(k-1)}[i, k]+D^{(k-1)}[k, j]$. For optimal substructure, replacing a nonoptimal segment with a cheaper legal segment improves the concatenation; deleting any nonnegative cycles cannot worsen it, contradicting optimality. Conversely, concatenating two old-state paths introduces only intermediate k and cannot give an illegally low candidate.

Thus $D^{(0)}=W$ and $D^{(k)}[i, j]=\min(D^{(k-1)}[i, j], D^{(k-1)}[i, k]+D^{(k-1)}[k, j])$. At k=n all intermediates are allowed. New matrices per stage take $\Theta(n^3)$ time. Saving all stages uses $\Theta(n^3)$ space; retaining only old and new uses $\Theta(n^2)$.

In-place updates work, arbitrary loop orders do not

Without negative cycles, D[k, k]=0 at every stage. Updating row k compares D[k, j] with D[k, k]+D[k, j], leaving it unchanged; column k is likewise unchanged. All stage-k inputs D[i, k], D[k, j] therefore remain old-stage values. Updating each D[i, j] once exactly implements the recurrence in one matrix.

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i, k]!=∞ and D[k, j]!=∞:
        D[i, j]=min(D[i, j], D[i, k]+D[k, j])
```

Time is $\Theta(n^3)$, space $\Theta(n^2)$. Fixed-width implementations must prevent an INF sentinel plus a negative number from becoming falsely reachable and must avoid overflow. Check reachability before addition; the mathematical theorem assumes exact arithmetic.

In-place updates work, arbitrary loop orders do not (continued)

For a counterexample, use unit-weight chain1→2→4→3. A single i, j, k pass considers i=1, j=3 before D[1,4] or D[2,3] is finite, so D[1,3] remains∞. Later j=4 computes D[1,4]=2 but never revisits j=3. Correct k, i, j order gives D[1,3]=3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete matrices at every k

Rows and columns use1,2,3,4. $D^{(0)}=W$; vertex 1 has no incoming edge, so $D^{(1)}=W$ too.

Stage	Row1	Row2	Row3	Row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

Stage 2 changes(1,3) to 4−2=2,(1,4) to 4+6=10, and(4,3) to 1−2=−1. Stage 3 changes(1,4) to 2+3=5 and(2,4) to−2+3=1. Stage 4 changes(3,2) to 3+1=4. Every other old value is at most its candidate and remains.

Unlike $D^{(2)}$, $L^{(2)}$ already permits 2→3→4 and 3→4→2. Intermediate vertices 3,4 are not allowed in $D^{(2)}$, whose corresponding entries remain6 and∞. A second step in each algorithm means a different state.

Complete matrices at every k (continued)

Six Floyd updates: two old inputs give one new value

Strict updates only. Inputs use the previous stage; unlisted entries stay unchanged.

k	(i, j)	Old	D[i, k]	D[k, j]	New
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

Green: new minimum. No updates at k=1; three at k=2, two at k=3, one at k=4.

A predecessor is not an intermediate vertex

Entry Π[i, j] is the vertex **immediately preceding j** on a selected shortest i-to-j path, not arbitrary intermediate k. Initialize Π[i, j]=i for direct edges; use NIL for diagonals and unreachable entries. With two stage buffers, a strict improvement through k assigns Πnew[i, j]=Πold[k, j], because the final edge belongs to the k-to-j suffix. Otherwise preserve the old predecessor. When k=j, no strict improvement occurs, so a zero-length suffix cannot overwrite a valid predecessor with NIL.

Preserve the old route on equal costs; changing $<$ to $\leq$ is unsafe. With only edge 1→2 of weight 1, stage k=2 has equal candidate1+0=1. Assigning Π[1,2]←Π[2,2]=NIL then destroys the existing route. Zero cycles likewise prevent arbitrary tight-edge parent choices without an acyclicity argument.

In the example, Π[1,4] first improves at k=2 to Π[2,4]=2, giving 1→2→4. At k=3 it becomes Π[3,4]=3. With Π[1,3]=2 and Π[1,2]=1, the final reverse chain is 4←3←2←1, cost 4−2+3=5. The next tie rule provides a directly decreasing integer measure for a complete predecessor tree.

Provably acyclic predecessors even with zero cycles

Store pair(cost, hops), minimizing cost first and then hops. Concatenation adds both coordinates. Initialize diagonals(0,0), direct edges(w,1), and absent paths(∞,∞), then apply the same Floyd recurrence. Without negative cycles, positive cycles worsen cost and zero cycles worsen hops. A lexicographic optimum has a simple representative, so the decomposition and in-place proofs still hold. The algorithm uses $\Theta(n^3)$ operations on fixed-size pairs.

For each fixed source s and finite $v \neq s$, choose an original incoming edge $u \to v$ satisfying

$\text{cost}[s, v] = \text{cost}[s, u] + w(u, v)$, $\text{hops}[s, v] = \text{hops}[s, u] + 1$.

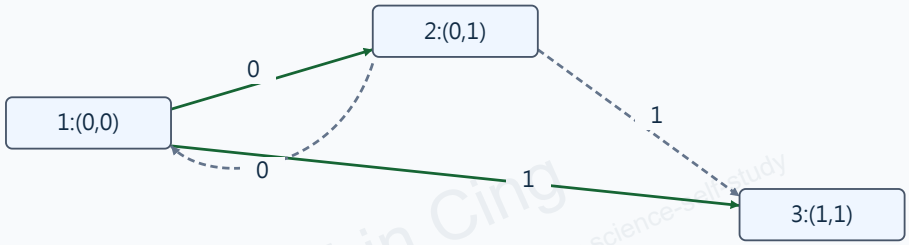
Such a u exists on the final edge of a lexicographically optimal path. A nonoptimal prefix could be replaced to improve the whole path, a contradiction. Set $\Pi[s, v] = u$. Every parent step decreases nonnegative integer hops by one, excluding cycles and infinite descent. All finite nonsource vertices have parents, so chains terminate at s and reconstruct actual shortest paths.

Scanning all m edges for every source costs $O(nm)$, no more than $\Theta(n^3)$ on a simple graph with $m \leq n^2$. Saved cost, hops,Π need $\Theta(n^2)$ space. For zero edges $1 \leftrightarrow 2$ and weight 1 edges $1 \to 3, 2 \to 3$, source 1 has hops=(0,1,1). It cannot acquire parent2 and no$1 \leftrightarrow 2$ parent cycle arises.

Provably acyclic predecessors even with zero cycles (continued)

Safe backtracking through zero cycles: hops decrease

Boxes: vertex:(cost, hops). Green: selected edges; backtrack against their arrows.



$\Pi[1,2] = 1$, $\Pi[1,3] = 1$. Source 1 has no parent.

Route $2 \to 3$ also gives cost 1 but needs 2 hops, losing to direct$1 \to 3$ with 1 hop.

Gray dashed: unselected. Every parent step lowers hops by 1, excluding zero-cycle parents.

Negative-cycle detection and affected pairs

If final Floyd has D[k, k]<0, its finite value concatenates actual edges into a negative closed walk. Decomposing this into simple cycles finds at least one negative cycle. Conversely, take a negative simple cycle and its largest-numbered vertex k. All other vertices are allowed intermediates of its k-to-k walk, so Floyd detects a negative closed walk and some negative diagonal. This proves both detection directions.

With negative cycles, fixed stages produce some finite walk values, which are not shortest distances for affected pairs. The earlier in-place equivalence proof relying on D[k, k]=0 no longer applies. Standard in-place k, i, j still detects cycles: finite values have actual-walk witnesses and are no greater than the corresponding buffered candidates, so negative diagonals cannot be missed. Different update details can produce different temporary negative values.

Negative-cycle detection and affected pairs (continued)

Compute reachability R separately. Mark(i, j) as−∞ exactly when some k satisfies D[k, k]<0, R[i, k], R[k, j]. Sufficiency follows by entering a negative cycle, repeating it arbitrarily, and leaving toward j. For necessity, if the i-to-j relevant region has no negative cycle, deleting all nonnegative cycles leaves one of finitely many simple paths with a finite lower bound. Its distance cannot be−∞. Unreachable pairs remain∞; other Floyd entries remain valid.

Mark affected pairs before returning results, and do not feed−∞ back into ordinary∞ arithmetic. Johnson may reject the whole graph on any negative cycle; partial finite results require these pairwise semantics.

A negative cycle need not affect the whole graph

Use1→2:2,2→3:−3,3→2:1,3→4:4, with isolated5. Cycle 2→3→2 costs−2.

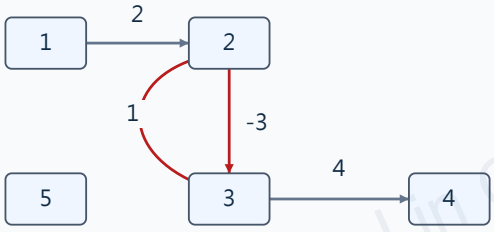
Source / destination	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter the cycle and leave for 4, giving $\delta(1,4)=-\infty$. There is no return to 1, so $\delta(1,1)=0$. Vertices 4,5 cannot enter the cycle and retain zero self-distances. A negative diagonal does not justify marking every matrix cell as erroneous.

A negative cycle need not affect the whole graph (continued)

A negative cycle affects 9 pairs, not the whole matrix

Red cycle 2→3→2 costs−2. Red cells require both entry to and exit from the cycle.



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter but not return, so $\delta(1,1)=0$. Vertices 4,5 cannot enter.

Transitive closure uses the same split with Boolean algebra

Initialize $R^{(0)}[i, j]=(i=j)$ or an edge $i \to j$, including zero-length paths and true diagonals. Replace min/+ by OR/AND:

$R^{(k)}[i, j]=R^{(k-1)}[i, j]$ OR$(R^{(k-1)}[i, k]$ AND $R^{(k-1)}[k, j])$.

Remove repeated vertices from a path regardless of cycle weights and split by whether intermediate k appears. Both directions cover exactly reachability. This proof does not require absence of negative cycles. In-place updates are safe: R[k, k]=true and R[i, k] OR(R[i, k] AND true)=R[i, k], with the same argument for row k.

For the common example, final rows are[1,1,1,1],[0,1,1,1],[0,1,1,1],[0,1,1,1]. Time is $\Theta(n^3)$ Boolean operations, space $\Theta(n^2)$ Boolean values. Bit packing can save storage and exploit word parallelism, but requires an explicit word-size and implementation assumption; arbitrary-length bit strings are not single constant-time words.

25.3 Johnson’ s algorithm for sparse graphs

Johnson makes all edges nonnegative while preserving relative costs of routes with fixed endpoints. For any finite potential $h:V \to R$, define $\hat{w}(u, v)=w(u, v)+h(u)-h(v)$. Summing along $v_0, \ldots, v_r$ cancels internal potentials:

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$.

Every route with the same endpoints receives the same adjustment, preserving strict comparisons and ties, and therefore the **set** of shortest paths. Different tie-breaking can still select different predecessor trees. Cycles receive adjustment0, preserving all negative cycles. Reweighting cannot repair a negative-cycle problem.

Adding the same constant C per edge instead adds rC to an r-edge route and can change the optimum. For direct$1 \to 4$:6 and two-edge $1 \to 2$:2,$2 \to 4$:2, the original two-edge cost 4 beats6. Adding 3 per edge gives direct9 versus two-edge 10, reversing the result.

Constructing a potential and rejecting negative cycles

Add a super-source s with zero edges to all original vertices and no incoming edges. If the original graph has no negative cycle, neither does the augmented graph. Every vertex is reachable, so Bellman–Ford gives finite $h(v)=\delta(s, v)$. For each edge $h(v) \leq h(u)+w(u, v)$, equivalent to $\hat{w}(u, v) \geq 0$. This is the needed Dijkstra contract.

The n source edges need not be stored. Initialize all $h[v]=0$, as after relaxing them, and synchronously scan original edges for $n-1$ rounds, then test whether $h[v]>h[u]+w(u, v)$ still holds. This gives zero-cost starts at any original vertex and an $n-1$-edge original budget. Without negative cycles a simple shortest path suffices. If any negative cycle exists, its potential inequalities cannot all hold, since summing gives $0 \leq$ a negative number. The final scan must reject it.

In the common example, start $h=(0,0,0,0)$. Round 1 lowers only $h[3]$ to -2; rounds 2,3 do nothing. Six reweighted edges in input order are 4,9,0,6,1,1, all nonnegative. This h is not distance from original vertex 1: $h[2]=0$ whereas $\delta(1,2)=4$.

Constructing a potential and rejecting negative cycles (continued)

Johnson potentials and six reweighted edges

	h[1]=0		h[2]=0	h[3]=-2	h[4]=0
Original edge		w	h[u]	h[v]	$\hat{w}=w+h[u]-h[v]$
1→2		4	0	0	4+(0)−(0)=4
1→3		7	0	-2	7+(0)−(-2)=9
2→3		-2	0	-2	-2+(0)−(-2)=0
2→4		6	0	0	6+(0)−(0)=6
3→4		3	-2	0	3+(-2)−(0)=1
4→2		1	0	0	1+(0)−(0)=1

Green weights are nonnegative. The potential uses a new zero-edge source, not original vertex 1.

Run Dijkstra per source, then restore original distances

```
Johnson(G):
  h=Bellman-Ford potential with all-zero initialization
  if any edge can still relax: report a negative cycle and stop
  for each original edge: ŵ(u, v)=w(u, v)+h[u]-h[v]
  for source u in V:
    run Dijkstra on original vertices with nonnegative ŵ, giving distHat
    for v in V:
      D[u, v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

From 1, distHat begins(0,∞,∞,∞). Extract 1 to get(0,4,9,∞); extract 2 to lower3 from 9 to 4 and assign4 value 10; extract 3 lowers4 to 5. Extract 4 cannot improve2. Extracted items are 1:0,2:4,3:4,4:5, using smaller IDs on distance ties. Final(0,4,4,5) restores to(0,4,2,5).

Other distHat rows are[∞,0,0,1],[∞,2,0,1],[∞,1,1,0]. Subtract source potential and add destination potential to recover the final Floyd matrix. For 3→2, this gives 2−(−2)+0=4; for 4→3, it gives 1−0+(−2)=−1. Unreachable entries stay∞; do not apply finite arithmetic to a sentinel.

Run Dijkstra per source, then restore original distances (continued)

Dijkstra on nonnegative weights and distance restoration

All reweighted edges are nonnegative. Cells show distHat; smaller IDs break ties.

Action	To 1	To 2	To 3	To 4
Initialize	0	∞	∞	∞
Extract 1	0	4	9	∞
Extract 2	0	4	4	10
Extract 3	0	4	4	5
Extract 4	0	4	4	5

Green: strict improvement. Original distance = distHat − h[1] + h[v].
Final (0,4,4,5) restores to (0,4,2,5), matching Floyd row1.

Sparsity, queues, and the unavoidable output cost

All-zero potential initialization costs $O(n)$, plus $n-1$ scans and a final check of m original edges, giving $O(nm+n)$. Generic Bellman–Ford on the explicitly augmented graph costs $O((n+1)(m+n))$; include its n added edges.

Fibonacci Dijkstra takes $O(m+n \log(n+1))$ amortized. Running it n times and filling n^2 outputs gives $O(nm+n^2 \log(n+1))$. Binary heaps give $O(n(m+n)\log(n+1))$; their log factor on m does not disappear. Graph and per-source work use $O(n+m)$, plus $\Theta(n^2)$ if the full output is retained. Row streaming can reduce resident output space but does not reduce the n^2-entry output itself.

For $n=1000$, $m=5000$, Floyd performs10^9 triple-index updates. Potential computation scans at most about 5×10^6 original edges. Binary repeated-Dijkstra's leading expression is $1000 \cdot 6000 \cdot \log_2 1000 \approx 5.98\times10^7$; Fibonacci's is $5\times10^6+10^6 \cdot \log_2 1000 \approx 1.50\times10^7$. These count different modeled operations and are **not measured execution times or directly divisible speedups**. For dense $m=\Theta(n^2)$, Floyd remains simple and $\Theta(n^3)$, and Johnson's sparse-graph advantage disappears.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 25.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum flow

Part VI Graph Algorithms

Scope

Residual networks, cut certificates, and valid heights prove maximum flow. Complete traces reconstruct augmentation, matching, and relabel-to-front.

Prerequisites

Directed graphs, BFS, and summation notation. This chapter develops conservation, residual networks, and cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define flow networks and net flow value.
- Use residual edges to revise Ford–Fulkerson choices.
- Prove max-flow min-cut and connect bipartite matching.
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

A capacity network has finite V, distinct source s and sink t, and capacities $c(u, v) \geq 0$. There are no self-loops; nonedges have capacity 0, and same-direction parallel capacities may be combined. Write $n = |V| \geq 2$ and m for the number of positive-capacity directed edges. Isolated vertices and vertices outside all s-to-t paths are allowed.

Use antisymmetric net flow $f(u, v) = -f(v, u)$ with $f(u, v) \leq c(u, v)$. Its full range is $-c(v, u) \leq f(u, v) \leq c(u, v)$. Internal $u \neq s, t$ obeys $\Sigma_v f(u, v) = 0$. Flow value $|f| = \Sigma_v f(s, v)$ is net source outflow, not a sum of absolute edge flows.

Requiring nonnegativity for all ordered net-flow pairs would force every value to 0 by antisymmetry. Capacities may be real. First count exact arithmetic operations; integer and rational bit costs are separate.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Nonnegative arc flow differs from net flow

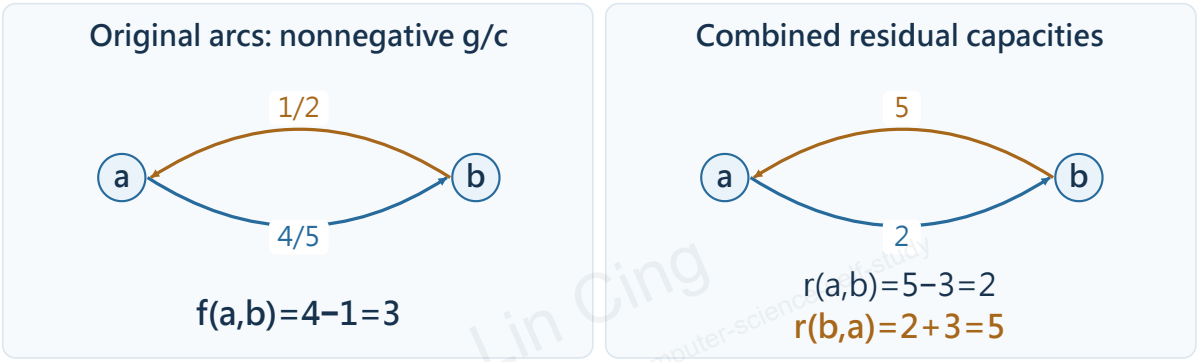
An alternative stores $0 \leq g(u, v) \leq c(u, v)$ on original arcs, without antisymmetry. Define $f(u, v) = g(u, v) - g(v, u)$, preserving conservation and net value. Opposite arc flows can cancel $\min(g(u, v), g(v, u))$ without changing any vertex's balance.

Conversely, for net f define $g(u, v) = \max(f(u, v), 0)$. Capacity bounds give $g \leq c$, and $g(u, v) - g(v, u) = f(u, v)$. Thus the models have the same maximum net value.

With $c(a, b) = 5$, $c(b, a) = 2$ and $g(a, b) = 4$, $g(b, a) = 1$, net values are 3 and -3. Canceling 1 gives arc flows 3 and 0. Forward residual capacity is $5 - 3 = 2$; reverse residual is $2 - (-3) = 5$, combining withdrawal of 3 with 2 units of original reverse capacity.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Original reverse capacity and cancellation are distinct



Reverse capacity 5 = original reverse capacity 2 + cancellation of net flow 3

For this pair of arcs, capacities are 5 and 2, original arc flows are 4 and 1, and net flow is 3. Residual b→a allows 5: undo 3 units of net flow, then use 2 units of original reverse capacity. **A residual arc need not be an original arc; reverse residual capacity need not consist only of cancellation.**

Internal cancellation gives the cut upper bound

A cut(S, T) partitions V with s∈S, t∈T. Its capacity c(S, T)=Σ_(u∈S, v∈T)c(u, v) counts only S-to-T arcs. Sum net outflows over S: source contributes |f| and other vertices contribute0. Internal antisymmetric pairs cancel, giving

$$|f| = \sum_{u \in S, v \in T} f(u,v) \leq \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

Count net crossing flow, not merely positive contributions. A feasible flow certifies a lower bound on the optimum; any cut certifies an upper bound. Equality is a checkable optimality certificate.

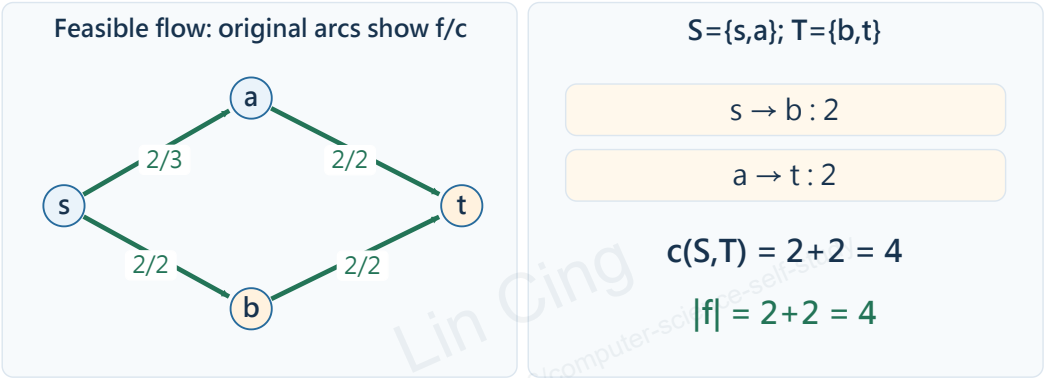
Flow and cut certificates on the same network

Capacities are s→a=3, a→t=2, s→b=2, b→t=2. Send2 along each s-a-t and s-b-t. Conservation and capacity bounds hold, with value 4.

For S={s, a}, T={b, t}, crossing capacities s→b:2 and a→t:2 sum to 4. Thus the maximum is both at least and at most4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow and cut certificates on the same network (continued)



Flow gives a lower bound; cut gives an upper bound; equality proves optimality

Source outcapacity or sink incapacity alone does not establish internal connectivity. The full network, feasible flow, and equal cut establish optimality here.

Multiple sources and sinks with finite added capacities

For disjoint supply set A and demand set B without individual bounds, add $s' \to a$ for each $a \in A$ and $b \to t'$ for each $b \in B$. Use total original capacity C for added capacities instead of an implemented ∞; no useful transport exceeds that total. If individual endpoint limits exist, use those actual limits.

Attach original endpoint net supplies to the new arcs to obtain conservation. Removing those arcs recovers the original supplies and demands at unchanged total value. This establishes both directions of the reduction.

To eliminate antiparallel original arcs in a nonnegative-arc model, replace $u \to v$ of capacity c by $u \to x \to v$ with both capacities c. Conservation at x forces equal segment flow, giving an exact correspondence. Setting both capacities to c/2 would change the problem.

26.2 The Ford-Fulkerson method

Residual capacity is $c_f(u, v) = c(u, v) - f(u, v) \geq 0$. The residual graph contains exactly positive-residual arcs. Neighbor candidates include original reverse directions even when their original capacity is 0.

For a simple residual s-to-t path p, let $\delta = \min_{(uv \in p)} c_f(u, v) > 0$. Update $f(u, v) \mathrel{+}= \delta$ and $f(v, u) \mathrel{-}= \delta$ together. Forward capacity is respected; reverse decrease cannot violate its upper bound, and its lower bound is equivalent to the forward upper bound. Path-internal balance changes cancel. Source net outflow increases by δ, preserving feasibility and strictly increasing value.

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u, v) for (u, v) in p)
    for (u, v) in p:
        f(u, v)+=delta; f(v, u)-=delta
return f
```

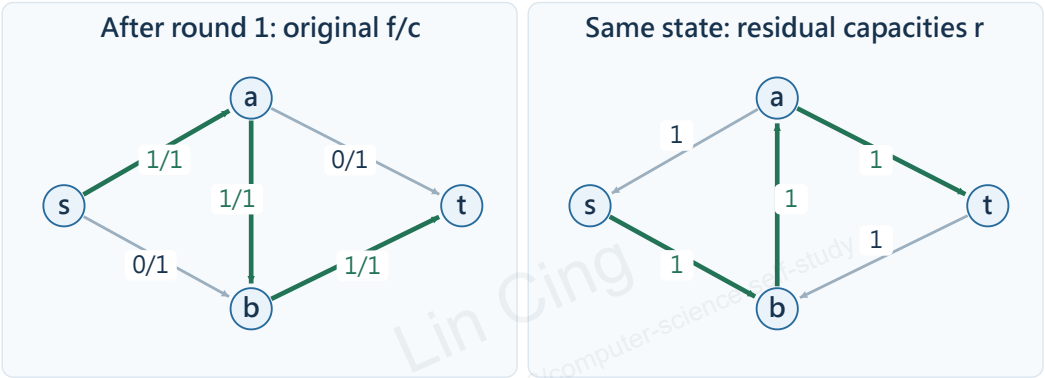
Two augmentations that withdraw an earlier choice

All original capacities s→a, s→b, a→b, a→t, b→t are 1.

Stage	Path and δ	(f_sa, f_sb, f_ab, f_at, f_bt)	Value
Initial	none	0,0,0,0,0	0
First	s→a→b→t; 1	1,0,1,0,1	1
Second	s→b→a→t; 1	1,1,0,1,1	2

Residual b→a cancels old a→b flow from 1 to 0. The final flow sends 1 on s-a-t and 1 on s-b-t. Restricting search to original directions would incorrectly stop after the first augmentation and return 1.

Two augmentations that withdraw an earlier choice (continued)

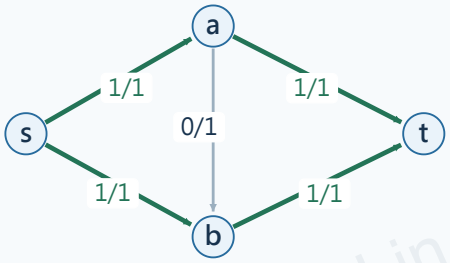


Green residual path s→b→a→t has bottleneck 1; b→a can undo old flow

When storing residual capacities directly, equivalent updates are r(u, v)−=δ, r(v, u)+=δ. Decreasing only the forward entry loses withdrawal capability.

After cancellation, check original arcs and optimality

After round 2: feasible value 2



Check every original arc

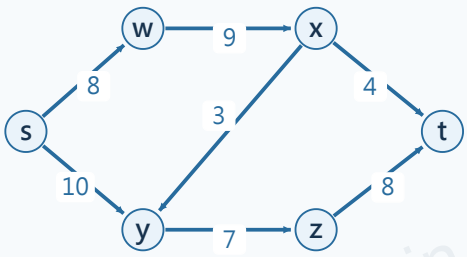
s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b falls from 1 to 0; both source arcs are saturated; cut {s} has capacity 2

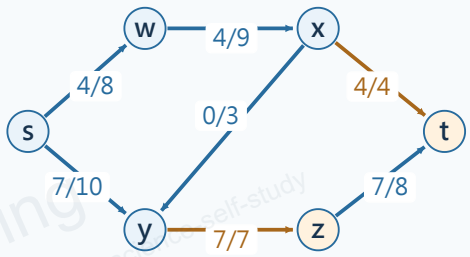
The second augmentation updates s→b, b→a, a→t. Using b→a decreases net f(a,b), without removing its original capacity. Vertices a and b each receive and send 1. Flow value 2 equals cut {s} capacity 2. Residual a→b still has capacity 1, but s cannot reach a, so it permits no further augmentation.

Six vertices: how do we reach the optimum?

Six vertices: original capacities c



After four rounds: original f/c



S={s,w,x,y}: cut arcs x→t:4 and y→z:7 total 11, equal to the flow

The original graph has x→y:3, but no y→x. The right panel shows original f/c after four augmentations, not residual capacities. Use vector order (sw,wx,xy,xt,sy,yz,zt) throughout the trace.

First send x→y, then undo it along y→x

Round	Path; residual capacities before augmentation	δ	Original flows afterward
Start	none	none	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t; (8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t; (10,3,4)	3	(3,3,0,3,3,3,3)

Round 1 creates residual y→x:3. Round 2 uses it, reducing original f(x,y) from 3 to 0. It does not add an original reverse arc. Now x receives 3 through wx and sends 3 through xt; y receives 3 through sy and sends 3 through yz. Source net outflow is 3+3=6.

Finish the trace and verify the cut

Vectors remain ordered (sw,wx,xy,xt,sy,yz,zt).

Round	Path; residual capacities before augmentation	δ	Original flows afterward
3	s-y-z-t; (7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t; (5,6,1)	1	(4,4,0,4,7,7,7)

Value rises 6→10→11. Vertices w,x each receive and send 4; y,z each receive and send 7. Residual reachability gives S={s,w,x,y}. The only original S→T arcs are x→t:4 and y→z:7. **Feasible value 11 and cut capacity 11 prove optimality.** Source outcapacity 8+10=18 alone cannot determine the answer.

Three equivalent max-flow min-cut conditions

For feasible f, these are equivalent:(i) f is maximum;(ii) no residual s-to-t path exists;(iii) some cut satisfies $|f|=c(S, T)$.

(i)⇒(ii): an existing path would augment to a larger feasible value.

(ii)⇒(iii): let S be residual-reachable from s and T its complement. Since $t \notin S$, this is a cut. Every $u \in S$, $v \in T$ has $c_f(u, v)=0$, or v would be reachable. Thus $f(u, v)=c(u, v)$, and the net cut identity gives equality.

(iii)⇒(i): every feasible g has $|g| \leq c(S, T)=|f|$.

Neither integer capacities nor a specific path rule is required. Integrality governs termination and cost, not this equivalence.

Integrality and pseudopolynomial time

With integer capacities and initial zero flow, residuals and δ remain integers. Each augmentation increases value by at least 1. A finite cut bounds the value, so the algorithm terminates with an integral maximum net flow after at most optimum F* augmentations.

Use adjacency lists, O(n+m) initial allocation, and timestamped visited marks that initialize only touched vertices. Each newly seen nonsource vertex corresponds to a scanned edge, so one path search costs O(m+1). Total time is O(n+m+(m+1)F*). Since F* can be exponential in binary input length, this is pseudopolynomial.

Rational capacities can be scaled by the product D of denominators. Its bit length is at most the sum of denominator bit lengths, but its numeric value can be enormous, so augmentation count may still be huge. Irrational capacities do not ensure improvement by at least 1; this argument does not prove termination for arbitrary paths. BFS supplies a capacity-independent bound below.

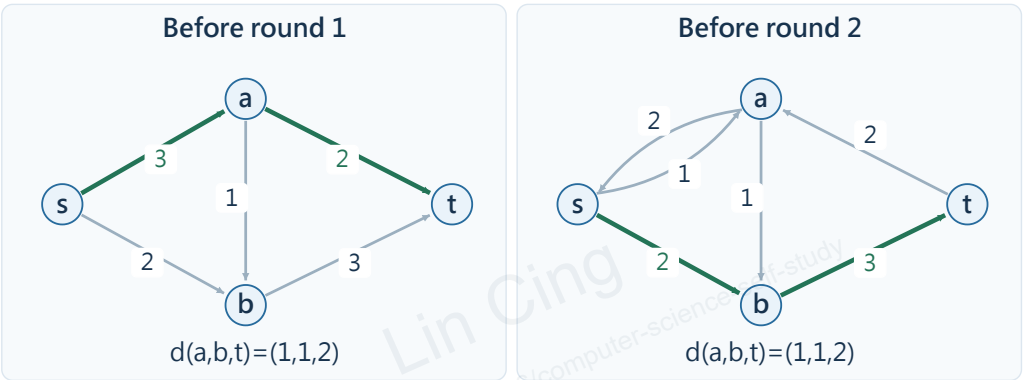
Edmonds–Karp minimizes residual edge count

Use BFS to choose a fewest-edge residual path, then augment through parents. Let d_f(v) be shortest edge count from s, or∞. For s→a=3, s→b=2, a→b=1, a→t=2, b→t=3 and neighbor a before b:

Before augmentation	d(a), d(b), d(t)	Path and δ	Flow tuple afterward
Round 1	1,1,2	s→a→t; 2	2,0,0,2,0
Round 2	1,1,2	s→b→t; 2	2,2,0,2,2
Round 3	1,2,3	s→a→b→t; 1	3,2,1,2,3
Final check	∞,∞,∞	no path	value 5; cut{s} has capacity 5

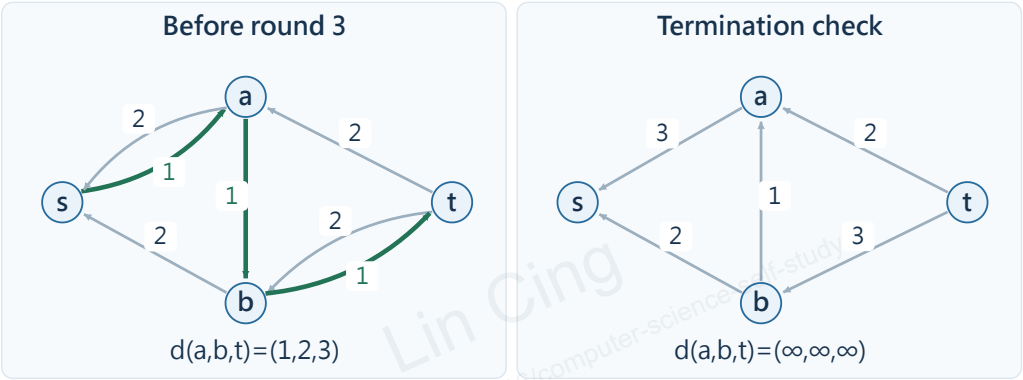
The tuple order is(f_sa, f_sb, f_ab, f_at, f_bt).

Edmonds–Karp minimizes residual edge count (continued)



All positive residual arcs shown; green marks the BFS path; bottlenecks 2, 2

Round 3 and termination: read the cut from residual reachability



Round 3 has δ=1; finally s has no outgoing arc: flow 5 equals cut {s} capacity 5

Path s→a→b→t has bottleneck min(1,1,1)=1, increasing value 4→5. The final residual arcs are a→s:3, b→s:2, b→a:1, t→a:2, t→b:3. Only s is reachable from s, so cut {s} has original capacity 3+2=5, equal to the flow. Distance ∞ means unreachable, not infinite capacity.

Residual BFS distances never decrease

Write d before and d′ after augmentation. Suppose $d'(v)<d(v)$, choosing such a v with smallest $d'(v)$. Its predecessor u on a new shortest path has $d'(u)=d'(v)-1$ and cannot be a smaller decrease counterexample, so $d(u)\le d'(u)$.

If uv was already residual, $d(v)\le d(u)+1\le d'(v)$, a contradiction. Therefore uv is a newly created reverse edge, and vu lies on the last BFS augmentation path. A prefix of that shortest path is shortest, so $d(u)=d(v)+1$.

Combining gives $d(v)=d(u)-1\le d'(u)-1=d'(v)-2$, contradicting $d'(v)<d(v)$. The old-unreachable/new-reachable∞ case also leads to finite old predecessors and hence old reachability, a contradiction. Every distance is nondecreasing.

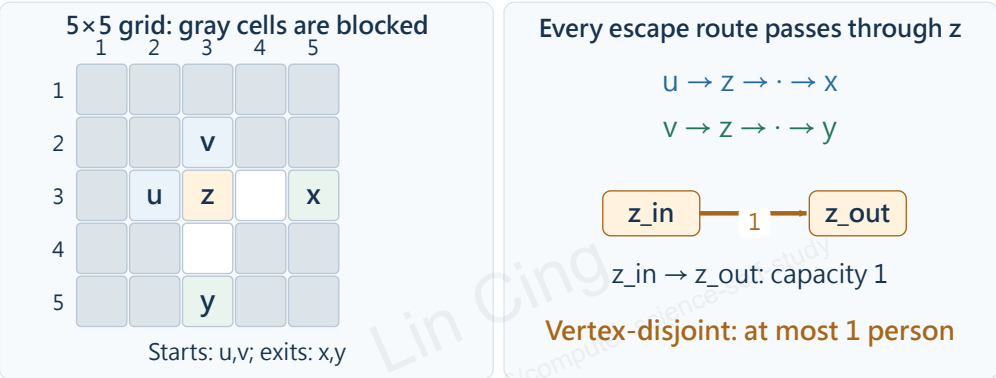
A critical edge's tail rises by at least two before reuse

Every augmentation saturates at least one critical uv at bottleneck δ, with $d(v)=d(u)+1$. For that direction to reappear, a later augmentation must traverse vu. Then $d'(u)=d'(v)+1 \geq d(v)+1=d(u)+2$, using distance monotonicity.

Thus between successive critical uses of one direction, its tail distance increases by at least 2. On an augmenting simple path the tail distance is at most $n-2$, so each direction is critical $O(n)$ times. At most$2m$ candidate directions give $O(nm)$ augmentations.

Each BFS and update takes $O(m+1)$, giving $O(n+m+nm^2)$ for $m \geq 1$, usually written $O(nm^2)$ after initialization. For $m=0$, return zero flow in $O(n)$. Common-denominator scaling of rational capacities keeps all value bit lengths polynomial in input length, and the number of rounds is independent of their magnitude. Thus the algorithm is also polynomial in bit cost; $O(nm^2)$ first counts arithmetic operations.

Escape routes: edge-disjoint does not mean vertex-disjoint



Edge capacities alone give 2; vertex capacities correctly limit escape to 1

Coordinates are (row,column). Open cells are {(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}; all other cells are blocked. Starts are u=(3,2), v=(2,3); boundary exits are x=(3,5), y=(5,3). Unit edge capacities allow two routes sharing z=(3,3). **If no cell may be shared, flow value 2 in that model does not prove both people can escape.**

Split every cell into an entry and an exit

For a $k \times k$ grid, let q be the open-cell count, p the number of distinct occupied starts, and b the open-boundary count. To forbid all shared cells:

Constraint	Network representation
At most one person per cell	$v_{in} \to v_{out}$, capacity 1
Move between adjacent open cells	$u_{out} \to v_{in}$ and $v_{out} \to u_{in}$, each capacity 1
One person per start	source $\to$ start_in, capacity 1
Leave through an open boundary	boundary_out $\to$ sink, capacity 1

Omit blocked cells. A boundary start still traverses its internal edge. The network has $N = 2q + 2$ vertices and $M \le q + 4k(k-1) + p + b$ arcs. For a full open grid, $b = 4k - 4$ requires $k \ge 2$; $b = 1$ when $k = 1$.

Why flow p is equivalent to everyone's escape

Given p vertex-disjoint escape routes, send one unit along each corresponding network path. Conversely, integer capacities admit an integral maximum flow. Decompose it into source–sink paths and cycles; remove cycles without changing value. Value p saturates all p source arcs. Every internal capacity 1 prevents two paths from using the same cell. Mapping back gives one escape route per person. This proves both directions.

Every route in the example passes through z. Edge $z_{in} \to z_{out}$ forms a capacity-1 cut, so at most one person escapes; route $u \to z \to (3,4) \to x$ attains 1. The edge-only model admits two edge-disjoint routes and has value 2. **The answers differ because the constraints differ.**

26.3 Maximum bipartite matching

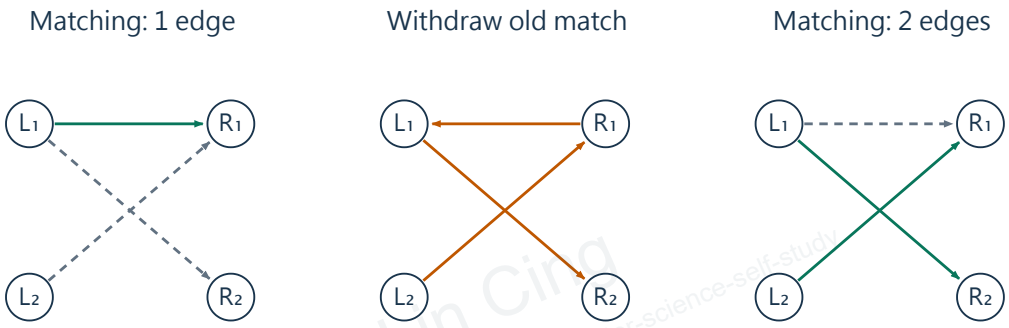
For bipartition $L \cup R$, a matching has no shared endpoints. Build unit arcs $s \to L$, original edges $l \to r$, and $R \to t$.

A matching M sends 1 along each s-l-r-t; disjoint endpoints ensure capacities, and value is $|M|$. Conversely, an integral flow gives each left vertex at most one unit total incoming, so conservation and integrality allow at most one original matching arc with flow 1. The right-side argument is symmetric. Selected unit-flow original arcs form a matching of size equal to the flow value.

Integer augmentation guarantees existence of an integral maximum flow, so maximum matching size equals maximum flow value. It does not make every optimal flow integral: in $K_{2,2}$, put1/2 on all four left-to-right arcs and 1 on source/sink arcs. This optimal value 2 flow does not make all four positive arcs a matching.

Maximum size is at most $\min(|L|,|R|)$, and each augmentation adds 1. With $m'=|E|+|L|+|R|$, direct Ford–Fulkerson costs $O(|V|+m'+m'\min(|L|,|R|))$. Isolated vertices prevent assuming $m'=\Theta(|E|)$ unconditionally.

The actual edges flipped by an alternating path



$L_2 \to R_1 \to L_1 \to R_2$: unmatched, matched backward, unmatched

Green: matching. Gray dashed: unselected original edges. Orange: residual alternating path.

Edges are L_1R_1, L_1R_2, L_2R_1; initially match L_1R_1. Augment along $s \to L_2 \to R_1 \to L_1 \to R_2 \to t$, replacing L_1R_1 by L_2R_1 and L_1R_2. Internal vertices keep one matching edge; both free endpoints become matched, raising size $1 \to 2$.

A constructive proof of Kőnig's equality

For a maximum matching, let Z be residual-reachable from s, and $C=(L \setminus Z) \cup (R \cap Z)$. Consider original lr with $l \in Z$, $r \notin Z$. If unmatched, its forward residual arc reaches r. If matched, the source-to-l arc is saturated; the last arc reaching l must come backward from its matched r, also making r reachable. Both contradict $r \notin Z$. Thus C covers every original edge.

Both endpoints of a matched edge are either reachable or unreachable together: a reachable right endpoint reaches its left mate backward, and a reachable matched left endpoint must have been entered from its right mate. Therefore C contains exactly one endpoint of every matched edge. Unmatched left vertices are source-reachable and outside C. Unmatched right vertices cannot be reachable, or an augmentation would exist, and also lie outside C. Hence $|C|=|M|$.

Any vertex cover must choose a distinct endpoint for every matching edge. Minimum bipartite cover size therefore equals maximum matching size. General graphs need not satisfy this: a triangle has minimum cover2 and maximum matching1.

Sufficiency of Hall's condition

A matching covering L requires $|N(A)| \geq |A|$ for every $A \subseteq L$, because distinct left vertices need distinct neighbors.

If a maximum matching leaves some left vertex unmatched, search alternating paths from unmatched left vertices. Let A, B be reached left and right sets. The preceding forward/backward argument gives $B=N(A)$. Every B vertex is matched, or the search yields an augmentation. Its matched left mate lies in A, which additionally contains an unmatched starting vertex. Hence $|A|>|B|=|N(A)|$, violating Hall's condition.

The condition is sufficient as well as necessary. If L is empty, the empty matching already covers it. Covering all of L is perfect on both sides only when the two side sizes also agree.

26.4 Push-relabel algorithms

A preflow retains antisymmetry and capacity bounds but allows excess $e(u)=\Sigma_v f(v, u)\geq 0$ for $u\neq s$. Active means $u\neq s$, t and $e(u)>0$. The sink accumulates excess but is never discharged.

Valid heights satisfy $h(s)=n$, $h(t)=0$ and $h(u)\leq h(v)+1$ on every residual uv. Initialize other heights and all f, e to 0, then saturate source arcs:$f(s, v)=c(s, v)$, $f(v, s)=-c(s, v)$, $e(v)+=c(s, v)$, $e(s)-=c(s, v)$.

Nonsource excess is nonnegative. No positive residual arc leaves s; other residual arcs start at height0 and satisfy validity. Heights are valid labels, not necessarily current BFS distances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete push and relabel preconditions

```
push(u, v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u], c(u, v)-f(u, v))
    f(u, v)+=d; f(v, u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u, v)>0)
```

Admissible means positive residual and a height drop of exactly 1. A push is saturating if residual becomes 0; otherwise it is nonsaturating and empties e(u). Count simultaneous saturation and emptying as saturating.

An active vertex must have a positive-residual outgoing arc. Otherwise every $f(u, v)=c(u, v)\geq 0$, giving $e(u)=-\Sigma f(u, v)\leq 0$, a contradiction. Thus relabel minimizes a nonempty set. With no admissible arc, integer validity implies $h(u)\leq$every residual neighbor height, so relabel raises h(u) by at least 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operations preserve preflows and valid heights

Push keeps e(u) nonnegative and only increases other nonsource excess. Capacity and antisymmetry follow as for augmentation. Heights remain unchanged. The only possibly new reverse vu has h(v)=h(u)−1, so it remains valid but is not admissible.

Relabel leaves f, e fixed. The new outgoing bound follows from 1+the minimum neighbor height. Incoming wu remains valid because h(w)≤old_h(u)+1≤new_h(u), and cannot be admissible. At least one outgoing arc attaining the minimum becomes admissible.

The admissible network is a DAG: a k-edge cycle would imply h=h−k. Push creates no admissible edge, and relabel can create only its own outgoing admissible edges. These facts support the later list invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Active vertices reach the source, bounding their heights

Let R be residual-reachable from active u, and suppose s∉R. All R vertices have nonnegative excess and u positive, so Σ_(v∈R)e(v)>0. No positive-residual arc leaves R, so f(v, w)=c(v, w)≥0 across its outgoing cut. Internal antisymmetric terms cancel, giving

Σ_(v∈R)e(v)=−Σ_(v∈R, w∉R)f(v, w)≤0,

a contradiction. Thus u reaches s along a simple residual path of at most n−1 edges. Summing valid-label inequalities gives h(u)≤h(s)+n−1=2n−1. Relabeling leaves u active, so its new label has the same bound; at other times its height is unchanged. Each internal vertex is relabeled at most2n−1 times, for $O(n^2)$ total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness at termination still needs a termination bound

If all internal excesses vanish, the preflow is a feasible flow. A residual s-to-t simple path of $k \leq n-1$ edges would give $n=h(s) \leq h(t)+k=k \leq n-1$, a contradiction. Therefore no augmenting path exists, and max-flow min-cut proves optimality.

Sink excess equals the flow value. Some initial excess can return to the source instead of reaching the sink.

Correctness on termination alone does not prove termination. Bound relabels, saturating pushes, and nonsaturating pushes separately to obtain a finite capacity-independent count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

There are O(nm) saturating pushes

Fix direction uv. At its first saturation, $h(u)=h(v)+1$ and its residual becomes 0. Another positive uv push requires a reverse vu push restoring residual. At that step $h(v)=h(u)+1$; height monotonicity makes v at least 2 higher than at the prior forward saturation.

At the next uv saturation, u must again be one higher than v, and therefore at least 2 higher than its own previous saturation height. Heights are bounded by $2n-1$, so each direction saturates $O(n)$ times. At most$2m$ directions give $O(nm)$ total.

The bound uses height differences and monotonicity, not push quantity, and remains valid for fractional capacities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting for nonsaturating pushes

Let $\Phi=\Sigma_(\text{active } u)h(u)$. Initially it is 0 and always nonnegative. Relabel increases it exactly by its height increase. Across all vertices, total height increase is at most$(n-2)(2n-1)=O(n^2)$.

A saturating push can activate only its destination, adding at most$2n-1$ to Φ. The $O(nm)$ such pushes contribute $O(n^2m)$ total positive increase. A nonsaturating push empties u. A newly activated v adds $h(v)=h(u)-1$; an already-active v or terminal adds nothing. Thus Φ falls by at least 1.

Nonnegativity bounds these decreases by all positive increases, giving $O(n^2+n^2m)$, or $O(n^2m)$ when $m\geq 1$. Combining all basic operations gives $O(n^2m)$. With $m=0$, initialization leaves no active vertex and terminates directly.

26.5 The relabel-to-front algorithm

The linked list L always contains **all** vertices except s, t, including zero-excess vertices. It is not an active-only list. Fix $N(u)=\{v:c(u, v)>0 \text{ or } c(v, u)>0\}$, including reverse candidates. Preserve current across discharges:

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u, v)>0 and h[u]==h[v]+1:
            push(u, v) # do not advance current here
        else: current[u]=next-neighbor(v)
```

A saturated edge is skipped only on the next loop iteration. If push empties u, return immediately with current still pointing to that neighbor. A different implementation that advances after every push cannot share this trace unchanged.

Why skipped current arcs need no immediate revisit

Fix an interval with unchanged h(u). If uv is skipped for zero residual, only a reverse vu push can restore it. That requires h(v)=h(u)+1, so restored uv points uphill and remains inadmissible. If uv is residual but inadmissible, valid integer heights imply $h(u) \leq h(v)$. Vertex v can only rise, never creating a one-level descent from fixed u.

Equivalently, push creates no admissible edges, and a neighbor's relabel leaves no admissible incoming edge. Only u's own relabel can create new admissible outgoing arcs; that is exactly when current resets.

Reaching NIL therefore means all skipped arcs are still unusable and relabel is legal. Previously inspecting an edge alone would not justify ignoring later state changes.

Read next from the vertex's new position

```
initialize_preflow()
L=all vertices except s, t, in a fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # after any move
```

Move only after the whole discharge, not after each internal relabel. After moving an interior u to the head, its next is the old head, so earlier vertices that may have gained excess are revisited. Saving old next would incorrectly skip them.

Two branches of the list invariant

At every outer-loop test:(1) L is a topological order of the admissible network restricted to nonterminals;(2) all vertices before u have zero excess. Initially nonsource heights are 0, so no admissible arc connects list vertices. Any L works, and u begins at its head.

If discharge does not relabel, push creates no admissible edge, so topological order remains. It sends excess only to later list vertices or terminals, preserving the earlier zero-excess prefix. The discharged u is empty, restoring the prefix condition at next(u).

If discharge relabels, the last relabel leaves u without admissible incoming edges, and subsequent pushes create none. Moving u to the head preserves topological order. Only empty u precedes the new next, and other relative positions stay unchanged.

When u=NIL, the entire list is a zero-excess prefix. Conservation and the generic correctness theorem give maximum flow. This proof does not transfer directly to an active-only scheduling list.

Five-vertex initialization and the complete discharge of a

Capacities are s→a=4, s→b=3, s→c=2, a→b=2, a→t=1, b→c=2, b→t=4, c→t=5. Fix N(a)=[b, t, s], N(b)=[c, t, a, s], N(c)=[t, b, s], L=[a, b, c].

Set h(s)=5, other heights0, and saturate source arcs. Record each push/relabel immediately after it finishes, before subsequent neighbor scanning.

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)
Initialize	4,3,2,0	0,0,0	b, c, t
a scans b, t, s to NIL; relabel	4,3,2,0	1,0,0	b, c, t
Push a→b by 2	2,5,2,0	1,0,0	b, c, t
Skip b to t; push a→t by 1	1,5,2,1	1,0,0	t, c, t
Scan t, s to NIL; relabel again	1,5,2,1	6,0,0	b, c, t
Skip b, t to s; return 1 along a→s	0,5,2,1	6,0,0	s, c, t

Five-vertex initialization and the complete discharge of a (continued)

Height of a rises from 0 to 6. Move it to the front after discharge; it was already the head, so L remains unchanged. Next process b.

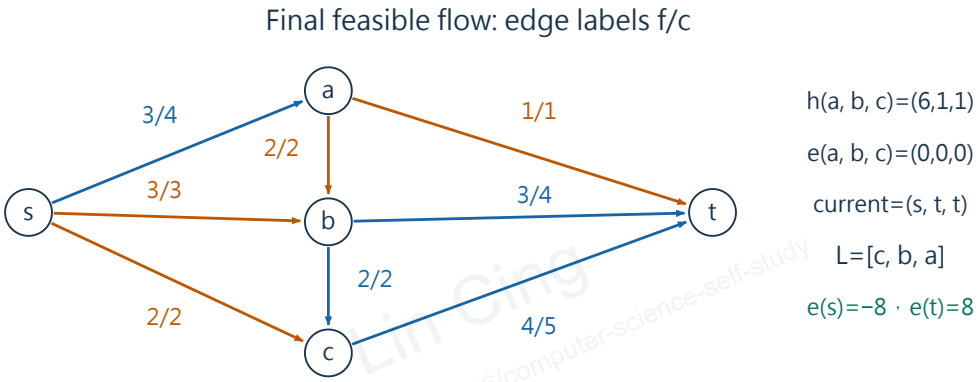
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discharging b, c and the exact move times

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)	L
b scans c, t, a, s to NIL; relabel	0,5,2,1	6,1,0	s, c, t	a, b, c
Push b→c by 2	0,3,4,1	6,1,0	s, c, t	a, b, c
Skip c to t; push b→t by 3	0,0,4,4	6,1,0	s, t, t	a, b, c
Move b to front after discharge	0,0,4,4	6,1,0	s, t, t	b, a, c
Revisit a; excess0, no operation	0,0,4,4	6,1,0	s, t, t	b, a, c
c scans t, b, s to NIL; relabel	0,0,4,4	6,1,1	s, t, t	b, a, c
Push c→t by 4; move c after discharge	0,0,0,8	6,1,1	s, t, t	c, b, a

The new next is b, then a, both empty, and finally NIL. Sink excess stays 1 after b→c and becomes 4 only after b→t. Final source net outflow 3+3+2=8 equals sink inflow1+3+4=8.

Independently verify the final flow by its cut



Vertex a receives 3 and sends 2+1; b receives 3+2 and sends 2+3; c receives 2+2 and sends 4. Residual reachability gives S={s,a}; cut capacity 3+2+2+1=8 equals feasible value 8.

One of the initial 9 units returns from a to s, leaving net value 8. No flow is lost. The cut certificate is independent of heights and scheduling.

Complete $O(n^3)$ accounting

There are $O(n^2)$ relabels. A phase lies between consecutive relabels. Without relabeling, the outer cursor advances through at most n list vertices; a relabeling discharge starts a new phase. Thus there are $O(n^3)$ discharge calls. With valid doubly linked handles, move-front costs $O(1)$.

Between its own relabels, a current pointer scans its neighbors once. Each vertex relabels $O(n)$ times, so advances and relabel minimum scans total $O(n\Sigma deg(u))=O(nm)$. Saturating pushes total $O(nm)$. A nonsaturating push empties u and ends discharge, so at most one occurs per call, for $O(n^3)$.

Total $O(n^3+nm)=O(n^3)$, since the simple capacity network has $m \leq n(n-1)$. Sparse adjacency and reverse records, heights, excesses, current pointers, and list need $O(n+m)$ space. These are arithmetic counts; capacity arithmetic and comparisons also pay their actual bit costs.

Choosing a generic operation is not free

Scanning all vertices for an active one and all neighbors at every step does not realize the $O(n^2m)$ basic-operation bound automatically. Maintain an active set instead: push changes only two excesses, and handles plus membership marks insert or remove in $O(1)$.

Use the same current rule for neighbors. All pointer advances and relabel scans total $O(nm)$, with $O(1)$ per push, realizing $O(n^2m)$ arithmetic time after initialization. Relabel-to-front uses a fixed all-vertex list and discharges to get $O(n^3)$; it is not merely another name for the active set.

Integer Ford–Fulkerson depends on F^*, Edmonds–Karp on nm^2, generic push-relabel on n^2m, and relabel-to-front on n^3. These arise from different worst-case mechanisms and do not directly predict measured machine speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 26; § 26.5.

Algorithm_wjungle, handwritten notes on flow networks, escape routes, and vertex capacities, physical PDF pages 37–42.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computational geometry

Part VII Selected Topics

Geometric predicates, degenerate inputs, and data-structure order must agree. Orientation, intersection detection, convex hulls, and closest pairs show how local decisions produce correct global results. Prerequisites are vectors, cross products, sorting, and sweep lines.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover segment properties, detection of any intersecting pair, convex hulls, and closest pairs. The goals are to use cross products for turns, prove a sweep-line intersection detector, and analyze hull and closest-pair algorithms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closed segments and orientation

A point is $p = (x, y)$. The closed segment $ab = \{a + t(b - a) : 0 \le t \le 1\}$ includes endpoints. Intersection includes endpoint contact, collinear overlap, and segments reduced to a point, as well as proper crossings.

$$\operatorname{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

With the x-axis pointing right and y-axis pointing up, a positive value means a counterclockwise left turn from a through b to c. Negative means right turn, and zero means collinear. This is the signed parallelogram area. The triangle area is half its absolute value. Reversing cross-product order reverses the signs and requires reversing the turn convention.

Exact signs and arithmetic limits

If integer coordinates have absolute value at most B, differences are at most $2B$, and the determinant has absolute value at most $8B^2$. The coordinate type may be too narrow for products. Widen **before** subtraction and multiplication or use arbitrary precision. A predicate on b-bit coordinates multiplies $O(b)$-bit integers and is not always constant machine time.

Floating-point cancellation can turn a small nonzero determinant into zero. Fixed epsilon cannot guarantee correct signs at every scale. Scaling coordinates by λ multiplies orientation by λ^2, while the turn stays the same. Treating approximate collinearity as exact can also destroy transitivity of a sorting comparator.

Proofs here use exact real predicates. Reproducible examples use exact integers or rationals. Floating implementations need error-bounded filters followed by increased precision when the filter is inconclusive. An unproved epsilon does not replace a sign test.

Complete segment-intersection test

Let $o_1 = \text{orient}(a,b,c), o_2 = \text{orient}(a,b,d), o_3 = \text{orient}(c,d,a), o_4 = \text{orient}(c,d,b)$. The on test requires both collinearity and closed bounds on both axes.

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

Compare signs rather than multiplying large determinant values. Vertical segments need no slope, and on handles point segments. The bounding box alone is sufficient only after collinearity is already known.

Why both sides must straddle

When no orientation is zero, opposite signs for o_1, o_2 place c, d on opposite sides of the line through ab. Their unique line intersection lies inside cd. Opposite signs for o_3, o_4 place that same intersection inside ab, so the segments meet.

Conversely, an interior crossing puts each segment's endpoints on opposite sides of the other's line. Zero orientations cover an endpoint on another segment or collinearity. Two overlapping closed collinear intervals have an endpoint in the other interval, so the four on tests are complete. A point segment also reduces to testing its point on the other segment.

One straddling pair alone is insufficient. A segment may cross the extension of ab beyond its endpoints.

Crossings, contacts, overlap, and separation

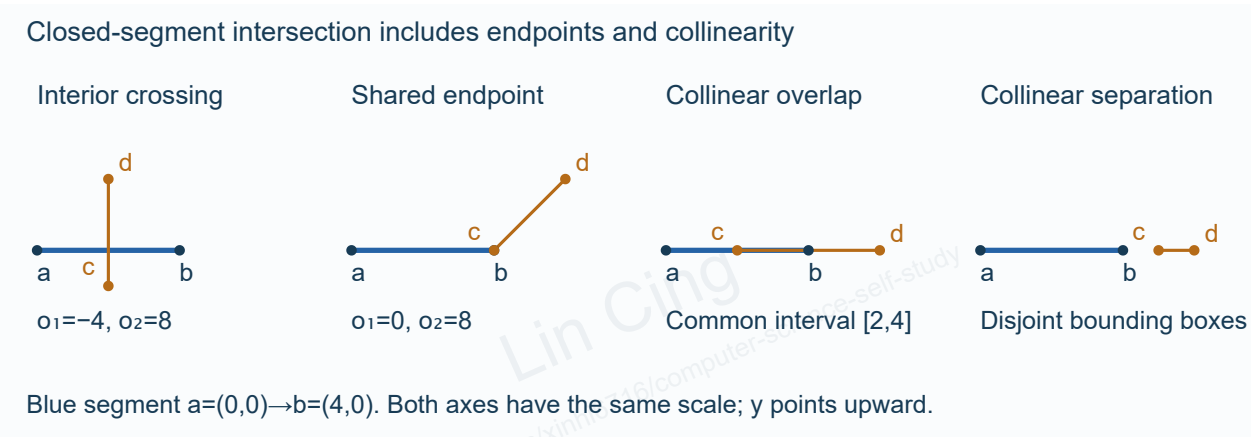
Fix $a = (0, 0), b = (4, 0)$.

c,d	o1,o2	o3,o4	Result
(2,-1),(2,2)	-4,8	6,-6	Interior crossing (2,0)
(4,0),(6,2)	0,8	8,0	Shared endpoint b
(2,0),(6,0)	0,0	0,0	Overlap [2,4]
(5,0),(6,0)	0,0	0,0	Disjoint bounding boxes

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Crossings, contacts, overlap, and separation (continued)

Closed-segment intersection includes endpoints and collinearity



Blue segment a=(0,0)→b=(4,0). Both axes have the same scale; y points upward.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Detect whether any pair intersects

For n closed segments, return whether some pair intersects. Listing all intersections is a different task. All-pairs testing costs $\Theta(n^2)$, and an input with $\Theta(n^2)$ intersections already needs quadratic output for full enumeration.

First consider nonvertical segments. Order each segment's endpoints as left l, right r, with $l_x < r_x$. At sweep position x, active segments satisfy $l_x \le x \le r_x$. The status orders them by height $y_s(x)$. Events are endpoints. On insertion, test the new segment against its upper and lower neighbors. Before deletion, test the two neighbors that will become adjacent.

The key is current geometric height, not segment ID:

$$y_s(x) = \frac{l_y(r_x - l_x) + (r_y - l_y)(x - l_x)}{r_x - l_x}.$$

Denominators are positive, so compare heights by cross multiplication. With constant-cost exact predicates, each balanced-tree operation costs $O(\log n)$.

Endpoint-only sweep detector

```
process endpoints in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

Skip missing neighbors. Tests use entire segments, so the detector may return before the sweep reaches the intersection. It stops at the first detected pair. Replacing `return` by `print` and continuing loses correctness because status order changes after crossings.

Earliest-intersection proof

Every true return follows an exact segment test, so there are no false positives. Suppose an intersection exists but the detector returns false. Choose an intersection p of smallest x-coordinate. Before it, segments do not cross, so their height order cannot exchange and the status remains valid.

If other segments lie between two segments meeting at p, continuity forces those intermediate segments through p too. Among this finite group, some intersecting pair is adjacent just before p. If contact starts at an endpoint, insertion connects the same-height group. Processing left endpoints before right endpoints retains segments ending at the same point.

The pair first becomes adjacent either when one is inserted or when the last segment between them is deleted. Both events test their entire segments and would return true, a contradiction. Collinear overlap likewise creates an intersecting adjacent pair when overlap starts at an endpoint.

No re-sort at every x is needed. Before detection, no order exchange occurs. If one is about to occur, the earlier neighbor test has already stopped the algorithm.

Deleting a middle segment creates neighbors

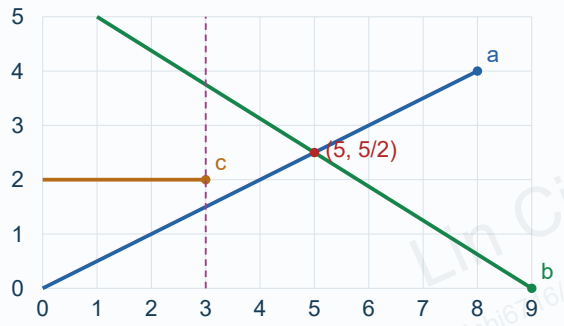
Let $a : (0, 0) \to (8, 4)$, $b : (1, 5) \to (9, 0)$, $c : (0, 2) \to (3, 2)$. Segments a, b meet at $(5, 5/2)$. Segment c meets neither.

Event	Status from low to high	New test
x=0 insert a	a	None
x=0 insert c	a,c	a,c do not meet
x=1 insert b	a,c,b	c,b do not meet
x=3 before deleting c	a,c,b	a,b meet, return true

At x=3, heights are $3/2, 2, 15/4$. Only removing c makes a, b adjacent. Testing insertions alone would miss this event.

Deleting a middle segment creates neighbors (continued)

Deleting c exposes the new neighbor pair a,b



Heights at x=3:

a: 3/2

c: 2, about to be deleted

b: 15/4

Test a,b before deleting c

Accept even if the intersection lies ahead

Vertical segments, endpoints, and cost

For general closed segments, batch equal-x events. Insert nonvertical segments starting there, process all vertical segments at that x, then delete nonvertical segments ending there. Endpoint contact remains active.

For a vertical segment with y-range $[L, H]$, find the first status segment of height at least L. It intersects exactly when that height is at most H. This is a tree `lower_bound` in $O(\log n)$, not insertion of the vertical segment as one height key. Sort same-x vertical intervals by L and maintain the largest previous H. If the next L is at most that maximum, intervals overlap. Otherwise update the maximum. Point segments belong to this closed-interval group too.

Sorting costs $O(n \log n)$. There are at most $2n$ nonvertical updates and n range queries, each logarithmic. Total sorting across vertical groups is also $O(n \log n)$. The general detector uses $O(n \log n)$ time and $O(n)$ space under constant-cost exact geometric arithmetic. Cross products on b-bit coordinates require the corresponding additional bit cost.

Convex hull and output convention

A convex set contains the segment between every pair of its points. For finite Q, $CH(Q)$ consists of convex combinations $\sum_i \lambda_i p_i$, with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. It is also the smallest convex set containing Q.

Deduplicate coordinates first. Output only extreme vertices in counterclockwise order, without repeating the first at the end. Points in the interior of a hull edge are omitted. Empty input gives no vertices. One unique point gives that point. An all-collinear set with at least two points gives just its two endpoints.

Graham scan sorts by polar angle and maintains a stack. Jarvis march finds a supporting edge at each step and depends on hull size h. Neither needs floating-point `atan2`. Exact orientation and squared distance comparisons suffice for ordering and collinear ties.

Graham scan and equal-angle points

Choose p_0 with smallest y-coordinate, breaking ties by smallest x. Other points lie in the upper half-plane or along positive x. Compare directions u, v using $\text{orient}(p_0, u, v) > 0$. Break collinear ties by squared distance from p_0. Keep only the farthest point on each ray. All others lie between it and p_0, so removing them preserves the hull.

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

The size test avoids empty-stack access. Handle all-collinear input by returning endpoints first. For noncollinear input, the result closes to a convex polygon. Using `<=0` removes flat intermediate vertices, matching the extreme-vertex convention. Listing every boundary point needs different same-ray and final-collinear-chain handling.

Why a popped vertex is no longer extreme

For a new point p_i, consider the final two stack points r, t. Their polar angles increase from p_0. If $\text{orient}(r, t, p_i) \leq 0$, then t is not a new hull vertex.

If $r = p_0$, strictly increasing angles spanning less than π give a positive turn, so no pop occurs. Otherwise translate p_0 to zero. Rays r, t, p_i occur in order within a span below π, so t is in the cone of r, p_i. Write $t = \alpha r + \beta p_i$, $\alpha, \beta \geq 0$. With $D = \text{orient}(0, r, p_i) > 0$,

$$\text{orient}(r, t, p_i) = (\alpha + \beta - 1)D.$$

A nonleft turn gives $\alpha + \beta \leq 1$. Thus $t = \alpha r + \beta p_i + (1 - \alpha - \beta)p_0$ is a convex combination in the triangle. Removing it preserves the new hull. Equality puts it on edge rp_i, still not extreme. This containment argument justifies repeated pops.

Stack invariant and correctness

Before processing p_i, the stack is exactly the counterclockwise hull vertices of the processed points. One or two points establish the degenerate base case. The first noncollinear point forms a triangle.

The new point has the largest polar angle, so $p_0 \to p_i$ is a supporting edge. All earlier points lie on its right or on its line. Looking backward from the old hull's end, edges not visible from p_i remain unchanged. A contiguous visible end chain is replaced by new edges. Each pop preserves the hull by the triangle proof. When (r, t, p_i) becomes a left turn, $t \to p_i$ is the other tangent. The remaining convex edges and $p_i \to p_0$ bound a convex polygon.

That polygon contains the old hull and new point, while all its vertices are input points. It both contains and is contained in the new convex hull, proving equality. Induction completes correctness. Each point is pushed once and popped at most once, so the scan takes $O(n)$. Sorting dominates at $O(n \log n)$, with $O(n)$ space.

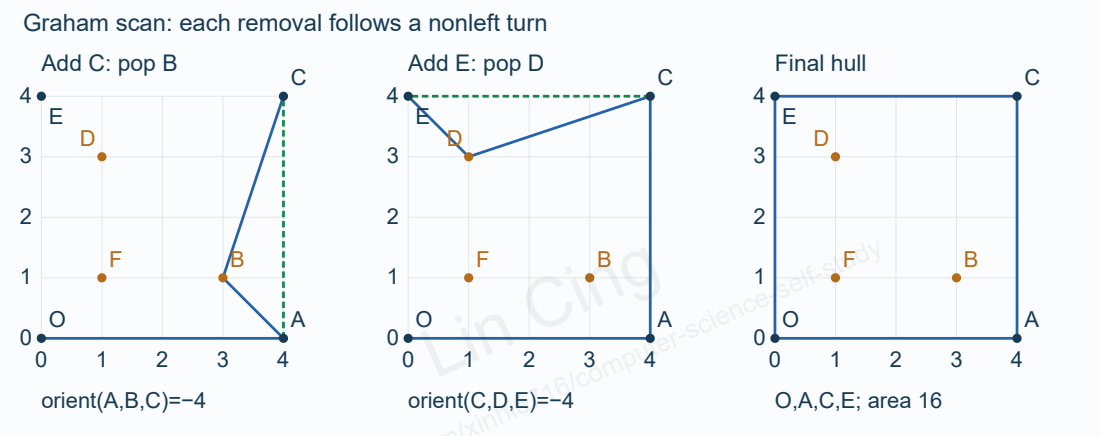
Two numerical pop checks

Use $O(0,0), A(4,0), B(3,1), C(4,4), D(1,3), E(0,4)$ in that angle order. Point $F(1,1)$ shares C's ray and is removed first.

New point	Predicate	Stack after processing
O,A,B	orient(O,A,B)=4	O,A,B
C	orient(A,B,C)=-4, pop B; orient(O,A,C)=16	O,A,C
D	orient(A,C,D)=12	O,A,C,D
E	orient(C,D,E)=-4, pop D; orient(A,C,E)=16	O,A,C,E

Indeed, $B = O/4 + A/2 + C/4$ and $D = O/4 + C/4 + E/2$. The popped points are convex combinations of other points. The hull is the square O,A,C,E, with area 16.

Two numerical pop checks (continued)



Jarvis march finds supporting edges

Start at the lexicographically smallest point p. Choose any $q \neq p$. Scan all $r \neq p$. If $\text{orient}(p, q, r) < 0$, set $q = r$. For collinear points, retain the farther one. At the end, all points lie on or left of $p \to q$, so q is the next counterclockwise extreme vertex. Output p, set $p = q$, and stop upon returning to the start.

Why does one scan suffice? At a hull vertex, all directions to other points lie in a supporting half-plane spanning at most π. Orientation linearly orders the rays there. Each update retains the clockwise-most ray seen so far, and same-ray ties retain the farthest point. The final ray is a supporting edge. Repeated steps go between extreme vertices and do not enter the interior.

Each of h hull edges scans $n - 1$ points, giving $O(nh)$ time and $O(h)$ space including output. Since $h \leq n$, worst-case time is $O(n^2)$. The square example selects A,C,E,O and has $h = 4$. Jarvis can help when h is small, but is not always faster than Graham.

Closest pair and balanced groups

For $n \geq 2$ points with unique IDs, find one pair minimizing Euclidean distance. Repeated coordinates are legal and may give zero. For fewer than two points, report no pair. Compare squared distances instead of computing roots for every candidate.

Sort X by `(x,y,ID)` and Y by `(y,x,ID)`. Split X by rank, assigning its first $\lceil n/2 \rceil$ IDs to L and the rest to R. Choose dividing x-coordinate ℓ between their x-ranges, allowing that same x if tied. Stably partition Y by **ID membership**. Using only $x < \ell$ would fail for equal-x inputs and could destroy balance.

Recurse for δ_L, δ_R, and set $\delta = \min(\delta_L, \delta_R)$. If zero, return immediately. Any improving cross-group pair has both endpoints within δ of the dividing line and vertical separation below δ. Only the width-2δ strip matters.

Closest-pair pseudocode

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; choose dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

The strip uses the fixed δ from the recursive results. The best distance can shrink during comparisons, but at most seven successors are inspected per point. Since only one minimum pair is requested, equal-distance candidates need no update. Strict strip membership and >= termination are valid. Listing all minimum pairs needs explicit equality and output rules.

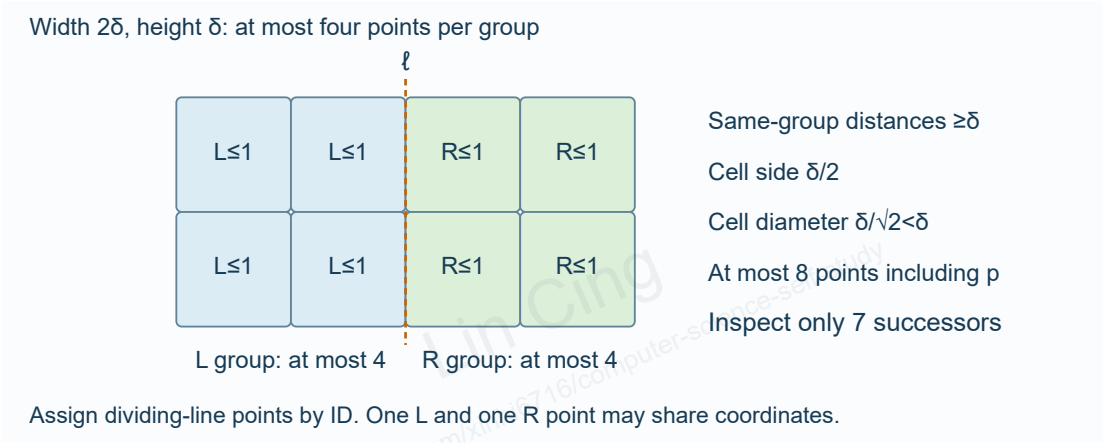
Packing proves seven successors suffice

Fix a strip point p. Any later point q that improves the result has $0 \leq q_y - p_y < \delta$, placing it in a width-$2\delta$, height-$\delta$ rectangle. L points lie in its left δ-square and R points in its right square. Within each group, pairwise distances are at least δ, by recursion.

Divide each half-square into four $\delta/2$-squares. Assign boundary points using fixed half-open conventions. A cell's diameter is $\delta/\sqrt{2} < \delta$, so it contains at most one point from that group. There are at most four points per group and eight total. Excluding p, at most seven remain. Every possible improving q is among the next seven positions in y-order.

Points on the dividing line belong to one group by ID and are counted in that group's half-square. Do not treat points from both groups on that line as if they share one within-group spacing guarantee. Cross-group duplicate coordinates are found by these comparisons. Within-group duplicates already make recursion return zero.

Packing proves seven successors suffice (continued)



Assign dividing-line points by ID. One L and one R point may share coordinates.

A complete merge example

X order is $A(0,0), B(1,4), C(2,1), D(3,7), E(4,1), F(5,5), G(6,0), H(7,8)$. Split A-D from E-H and choose $\ell = 7/2$.

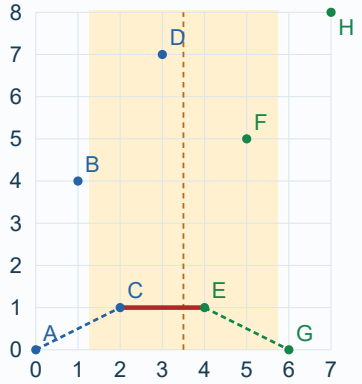
The left best pair A,C has squared distance 5. The right best pair E,G also has squared distance 5. Thus $\delta = \sqrt{5}$. The strip in y-order is C(2,1), E(4,1), F(5,5), D(3,7).

Point	Successor checks	Result
C	E has squared distance 4; next F has y-gap 4	Set best=C,E; 4≥2, stop this row
E	F has y-gap 4	Cannot improve distance 2, stop
F	D has y-gap 2	Equality cannot improve, stop
D	No successor	Done

The cross-group pair C,E has distance 2, improving both subproblems' $\sqrt{5}$. The figure uses the fixed $\sqrt{5}$ strip even after the best distance shrinks.

A complete merge example (continued)

Both sides have distance √5; cross-group C,E improve it to 2



ℓ=7/2, fixed δ=√5

L group A-D; R group E-H

Strip in y-order: C,E,F,D

$|C-E|^2=(4-2)^2+0^2=4$

Both sides have minimum squared distance 5

After cross-group checks: best²=4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable partitioning determines the bound

Rank splitting X, stable ID partitioning of Y, strip construction, and seven comparisons per point take $O(n)$ nonrecursive work. Thus

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = O(n \log n).$$

The initial two sorts also take $O(n \log n)$. Sorting Y again in every subproblem instead gives $O(n \log n)$ work per merge level contribution and overall $O(n \log^2 n)$. Mark each ID's group at the current level. Searching the whole L array separately for every Y point would not be a linear partition.

With depth-first recursion and prompt release of temporary arrays, array lengths along one recursive chain sum to $n + n/2 + \cdots = O(n)$. Sibling arrays and return records preserve the $O(n)$ workspace bound, plus $O(\log n)$ stack. Saving every array in the entire recursion tree instead takes $O(n \log n)$ space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 33.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms

Part VII Selected Topics

Scope

Dynamic multithreading separates total work, critical paths, and scheduling through a dependency DAG. Spawn/sync semantics and race repairs lead to complete matrix-product and parallel-merge/sort algorithms, proofs, and space assumptions. Average parallelism does not define a processor-benefit cutoff.

Prerequisites

DAG scheduling, parallel cost models, and recurrence analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Express dynamic multithreading with spawn and sync.
- Distinguish work, span, and parallelism.
- Analyze parallel limits of matrix multiplication and merge sort.
- 27.1 The basics of dynamic multithreading
- 27.2 Multithreaded matrix multiplication
- 27.3 Multithreaded merge sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27.1 The basics of dynamic multithreading

Programs specify computations allowed to run concurrently; the scheduler decides when and on which processor they run. A logical task is not a permanently bound OS thread, and spawning does not promise an immediate second core.

Assume P identical ideal processors and sequentially consistent shared memory. Instructions interleave in an order respecting dependencies; each load/store is one step, but a combined read-add-write is not automatically atomic. Arithmetic, indexing, and spawn/sync have constant unit work. Scheduler overhead, bandwidth, caches, and communication are outside this ideal bound.

A nonempty finite computation is a DAG. Nodes can be strands, sequential instruction sequences without parallel control, with positive execution time. Remove empty strands; split positive integral strands into unit-instruction chains for unit-node proofs. Edges express control and data dependencies. Nodes merely drawn at the same level need not be independent.

Exact semantics of spawn, call, and sync

`x = spawn F(...)` allows F to run with the caller's continuation. An ordinary `y = G(...)` requires the caller to await G's return. A `sync` waits for all children previously spawned by this procedure; it is not a system-wide barrier.

Spawn arguments are evaluated in the parent first. Each child owns its call-local variables, but shared-array references can still lead several tasks to the same cell. Different variable names do not prove independence. Reading a spawned result requires a dependency, usually an explicit sync.

Procedure return has an implicit sync here. However, if `return x+y` evaluates its expression before returning, that implicit wait does not permit an earlier read of x. Explicitly sync before computing the return expression. Removing spawn/sync and replacing parallel for by ordinary for gives a serialization. Correct serialization alone does not prove every parallel interleaving correct.

Fibonacci code and correctness

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

Children read their own n and return separate results, with no shared writes. Decreasing n ensures termination. Bases F0=0, F1=1 and induction, followed by synchronized addition, prove Fn. Spawning both children still requires waiting before addition and changes constants rather than work/span orders.

This version deliberately omits memoization and recomputes parameters. High parallelism does not imply work efficiency: sequential iteration uses O(n) arithmetic operations, whereas occupying more cores does not excuse exponential work.

Work and span in one explicit strand model

Split each nonbase call into three unit strands:pre through spawning, cont before the second ordinary call, and join for addition/return after sync. A base call is one unit strand. Pre precedes the left child and cont; cont precedes the right child; both child finishes precede join.

With total work W and longest dependency-path cost D,

$$W_n = W_{n-1} + W_{n-2} + 3, \qquad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \qquad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

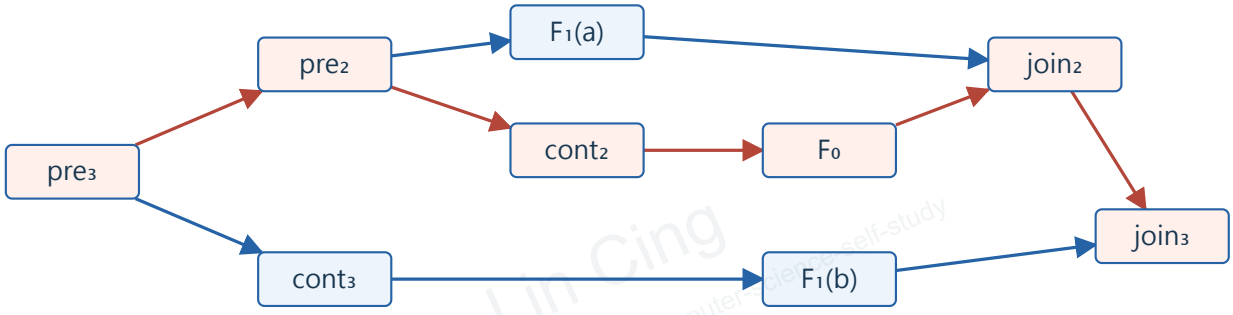
Work and span in one explicit strand model (continued)

Induction verifies Wn=4F(n+1)−3 and Dn=2n for n≥2. Let $\varphi = (1+\sqrt{5})/2$. Since ϕ²=ϕ+1, induction gives ϕ^(n−2)≤Fn≤ϕ^(n−1) for n≥2:check2,3, and add adjacent bounds to obtain the next power. Thus W=Θ(ϕ^n), D=Θ(n). A complete recursive call is not one O(1) node in this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace all dependencies among nine strands

All nine strands of P-FIB(3): arrows express dependencies



Each box is one unit strand: W=9. The red six-box critical path gives D=6.

Pre3 forks to pre2 and cont3; cont3 calls the right F1. Pre2 forks to its F1 and cont2; cont2 calls F0. Both bases precede join2, and join2 plus the other F1 precede join3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace all dependencies among nine strands (continued)

The red path pre3→pre2→cont2→F0→join2→join3 has 6 nodes, so W=9, D=6. Path pre3→cont3→F1→join3 has only 4. Arrows express dependencies, not a single sequential chain through every node of a call.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Work law, span law, and achievable speedup

Write $W = T_1$,$D = T_\infty$, and actual P-processor time T_P. At most P units of work finish per time unit, and a critical path cannot parallelize, so

$$T_P \geq \max\{W/P, D\}.$$

The discrete unit-node model also gives $T_P \geq \lceil W/P \rceil$. Speedup $S_P = W/T_P \leq \min(P, W/D)$, and efficiency S_P/P≤1. Average parallelism A=W/D limits speedup. P>A precludes perfect P-fold acceleration, but does not imply processor P+1 has no benefit.

Sequential composition adds both W and D. Independent fork/join branches add work and take the maximum branch span, plus explicit fork/join costs. An empty computation has W=D=0 and time 0;0/0 is not defined parallelism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete greedy-scheduling upper-bound proof

At each step execute P ready nodes if possible, otherwise execute all ready nodes. Let C be complete steps and I incomplete steps, T=C+I. Every incomplete step before completion executes at least one node, so W≥PC+I.

An incomplete step removes all sources of the remaining DAG. Any longest path remaining afterward can be extended backward in the previous DAG to at least one removed source. If nothing remains, the earlier span was at least 1. Thus each incomplete step decreases remaining span by at least 1, and I≤D. Consequently

$$T \leq (W - I)/P + I \leq (W - D)/P + D \leq W/P + D.$$

The substitution uses coefficient1−1/P≥0. Positive integral strand durations can be split into chains for this proof. It does not give the same guarantee in a different model with arbitrary indivisible tasks of unknown lengths.

Two-approximation and quantified slackness

An optimal scheduler also obeys $T_P^* \geq \max(W/P, D)$, so greedy T≤W/P+D≤2T*. The sharper bound is(2−1/P)T*. Constant-factor performance requires no future-DAG knowledge.

Define slackness $\sigma = W/(PD) = A/P$. For D>0, the bounds give

$$1 \leq T_P/(W/P) \leq 1 + (1 - 1/P)/\sigma \leq 1 + 1/\sigma.$$

Large σ brings ideal greedy time near W/P. This is a mathematical guarantee, not measured bandwidth or scheduler overhead. An optimal schedule can ignore excess processors, so its time cannot increase with P. Measured implementations also depend on overhead and scheduling choices.

Parallelism30 still benefits from processor 31

Complete697 independent unit tasks before starting a23-unit dependency chain. Then W=697+23=720, D=1+23=24, A=30.

P	Independent-stage allocation	Stage time	Chain time	Exact optimum
30	23 rounds of 30, then7	24	23	47
31	22 rounds of 31, then15	23	23	46

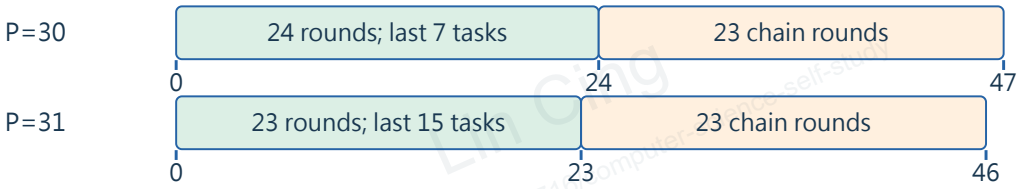
The chain cannot start until every predecessor finishes. Thus ceil(697/P)+23 is both an attainable schedule and a lower bound. Speedups720/47≈15.32 and 720/46≈15.65 both stay below30, yet the added processor helps. Only P≥697 makes the first stage one round and attains span24.

Parallelism30 still benefits from processor 31 (continued)

Same DAG: processor 31 still improves on 30

W=697+23=720
D=1+23=24

Average parallelism W/D=30. Both rows use the same time scale.



$T_{30}=47$, $T_{31}=46$. Speedup≤30 does not preclude benefit from processor 31.

Less work but greater span can reverse the winner

Two correct versions have(W, D)=(1024,4) and(768,16), both with ideal greedy scheduling.

At P=8, the first takes at least 1024/8=128; the second takes at most768/8+16=112, so the second is definitely faster. At P=128, the first takes at most1024/128+4=12; the second takes at least 16, so the first is faster.

Disjoint upper/lower bounds justify these conclusions. W/P+D is an upper bound, not exact time. Overlapping feasible-time intervals may leave the ordering unresolved from W, D alone. A single small-core measurement also cannot identify separate work and span changes.

Implementing parallel for by balanced fork/join

Use zero-based half-open intervals[l, r), with l=r empty, here and in merge algorithms.

```
PAR-FOR(l, r, body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l, m, body)
    PAR-FOR(m, r, body)
    sync
```

The n nonempty iterations give n leaves and n−1 internal nodes. Constant control work per internal node adds O(n), not n log n. The longest call/join chain has depth Θ(log n), so span is Θ(log n) plus the largest iteration span, provided iterations are independent.

An ordinary loop spawning n tasks serially has a Θ(n) issuing chain in its parent and cannot claim the same logarithmic span. The final sync also makes all results ready before the next phase reads them.

Matrix-vector multiplication safely parallelizes rows

```
MAT-VEC(A, x):
    n=A.rows
    allocate y[n]
    parallel for i=0 to n-1:
        y[i]=0
        for j=0 to n-1: y[i]+=A[i, j]*x[j]
    return y
```

Iteration i exclusively owns y[i], with A, x read-only. After inner step j, the entry is the sum of the first j+1 products, proving correctness. Work is $\Theta(n^2)$, span $\Theta(\log n)+\Theta(n)=\Theta(n)$, parallelism $\Theta(n)$. For n=0 return an empty vector.

Making the inner loop parallel as well causes unsynchronized read-modify-writes to one y[i], a determinacy race. Two logically parallel accesses conflict when they touch one cell and at least one writes. Even equal written values or one successful run does not prove independence.

Sequential consistency still permits lost updates

Lost updates: loads, local registers, and final stores

Initially x=0. Each task preserves load → add → store.

Step	Task 1	Task 2	Shared x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1+1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2+1 = 1$	1
6		store $x \leftarrow r_2$	1

After sync both read 1. Waiting cannot make three instructions one atomic operation.

The interleaving respects each task's load→add→store but ends at x=1. Running one task completely before the other gives 2. Sequential consistency orders individual instructions; it does not make the three-instruction compound operation atomic.

Sequential consistency still permits lost updates (continued)

Use disjoint ownership or reduction to remove the race. Atomic additions or locks can impose dependencies, but their serialization, work, and span need separate accounting. They do not retain an analysis assuming all additions complete simultaneously.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A reduction parallelizes the dot product correctly

```
DOT(Arow, x, l, r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m=floor((l+r)/2)
    left=spawn DOT(Arow, x, l, m)
    right=DOT(Arow, x, m, r)
    sync
    return left+right
```

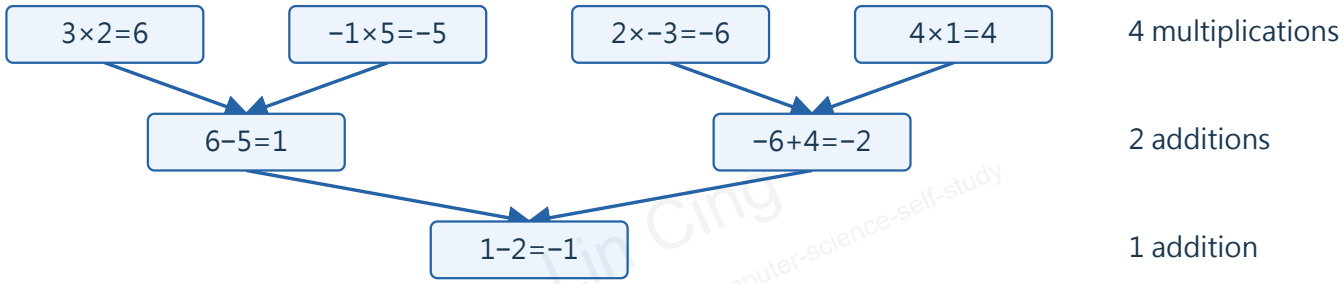
Children return separate values rather than sharing an accumulator. Associativity and induction give the interval dot product. Work is $\Theta(n)$, span $\Theta(\log(n+1))$. Replacing each MAT-VEC row by DOT gives $\Theta(n^2)$ work and $\Theta(\log n)$ span for $n \geq 2$.

For row=[3,−1,2,4], x=[2,5,−3,1], products are[6,−5,−6,4], first-level sums[1,−2], and result−1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A reduction parallelizes the dot product correctly (continued)

Dot product: four private products combine in a tree



Arithmetic DAG: work=7, span=3. Fork/join adds Θ(log n) control depth.

Floating-point tree addition may round differently from sequential accumulation. The proof assumes exact associative arithmetic and does not promise bitwise-identical floating results.

Granularity and the allocation model

Group n constant-cost iterations into batches of at most g,$1 \leq g \leq n$, and issue batches with balanced PAR-FOR. Work is Θ(n), span $\Theta(\log\lceil n/g \rceil + g)$. Larger g reduces task overhead constants but increases leaf serial time; the tradeoff depends on P and slackness.

Here allocate obtains a descriptor for uninitialized, nonoverlapping storage at constant abstract span. If actual allocation initializes each cell, count that work and parallelize initialization to retain logarithmic span. Serial zeroing of n^2 cells is not hidden inside O(1).

Matrix blocks use views(row origin, column origin, stride, order), without hidden full-block copies. Task descriptors, allocators, caches, and memory bandwidth have implementation costs beyond the stated ideal model.

27.2 Multithreaded matrix multiplication

```
P-MULTIPLY(A, B):
    n=A.rows                // both n by n
    allocate C[n, n]
    parallel for i=0 to n-1:
        parallel for j=0 to n-1:
            C[i, j]=0
            for k=0 to n-1: C[i, j]+=A[i, k]*B[k, j]
    return C
```

Each(i, j) owns one output cell and accumulates its inner loop sequentially. Work is $\Theta(n^3)$. The two issuing trees contribute $\Theta(\log n)+\Theta(\log n)$, then the inner loop contributes $\Theta(n)$, so span $\Theta(n)$, parallelism $\Theta(n^2)$. Two parallel keywords do not remove each cell's n-term dependency.

Using a DOT-style reduction for each Cij preserves $\Theta(n^3)$ work but reduces span to $\Theta(\log n)$, giving $\Theta(n^3/\log n)$ parallelism. Concurrent writes to Cij are not this reduction algorithm.

Eight subproducts need eight distinct destinations

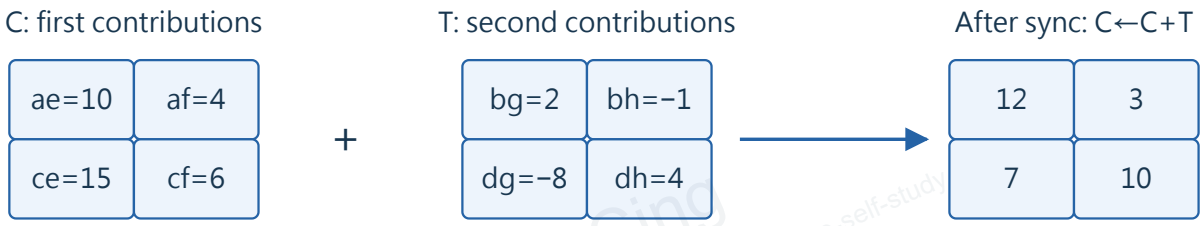
Write A=[[a, b],[c, d]], B=[[e, f],[g, h]], where letters are equal-order blocks. Compute C=[[ae, af],[ce, cf]] and temporary T=[[bg, bh],[dg, dh]]. Every one of eight products has its own destination. Sync before C=C+T.

For A=[[2,−1],[3,4]], B=[[5,2],[−2,1]], temporary C=[[10,4],[15,6]], T=[[2,−1],[−8,4]], giving[[12,3],[7,10]].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eight subproducts need eight distinct destinations (continued)

Eight independent products: separate destinations before addition



Each product writes one C or T cell. Await all products, then add cellwise.
A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. No simultaneous += contributions to one cell.

Two tasks executing C11+=ae and C11+=bg conflict. The temporary storage removes a shared-write dependency; spawning alone cannot do that.

Complete recursive matrix-product procedure

```
P-REC(C, A, B):
    n=A.rows
    if n == 1: C[0,0]=A[0,0]*B[0,0]; return
    allocate T[n, n]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    view C, T as four n/2 by n/2 blocks
    spawn P-REC(C11, a, e); spawn P-REC(C12, a, f)
    spawn P-REC(C21, c, e); spawn P-REC(C22, c, f)
    spawn P-REC(T11, b, g); spawn P-REC(T12, b, h)
    spawn P-REC(T21, d, g); P-REC(T22, d, h)
    sync
    parallel for i=0 to n-1:
        parallel for j=0 to n-1: C[i, j]+=T[i, j]
```

Inputs are equal-order square matrices of power-of-two order, A, B read-only, and C disjoint from them. Pad any other positive order with zeros to the next power of two and extract the original result. Return an empty matrix for n=0. T and child workspaces are independent. Eight spawn/call operations are a fixed constant, not an n-long issuing chain.

Inductive correctness and eight-way costs

The base writes the correct scalar product. Inductively, eight half-size products are correct and race-free through distinct destinations. Sync makes them available before cellwise addition. Block multiplication identities then yield AB. Shared reads of A, B are harmless.

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Span follows only the longest child, not the sum of eight. Each level's nested parallel combination has Θ(log n) issuing/join depth, not O(1). For n=2^L, the sum1+…+L=Θ(L^2) gives both bounds. Order4 has 8^2=64 scalar-product leaves; parallelization has not removed multiplications.

Three dependency stages for parallel Strassen

Use Chapter 4's M numbering. First prepare block sums/differences(a+d),(e+h),(c+d),(f−h),(g−e),(a+b),(c−a),(e+f),(b−d),(g+h) with cellwise parallel operations. Once inputs are ready, run seven products:

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Sync after all seven, then combine in parallel:C11=M1+M4−M5+M7, C12=M3+M5, C21=M2+M4, C22=M1−M2+M3+M6. Products cannot consume unfinished sums, and combination cannot consume unfinished products.

Strassen's work, span, and parallel workspace

Seven half-size children and a constant number of matrix sums give

$$W_S(n) = 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}),$$
$$D_S(n) = D_S(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Correctness follows from Chapter 4's expanded block identities and recursive induction, with the three additional parallel dependency stages. Parallelism Θ(n^(log₂7)/log²n) is smaller than for eight-way multiplication but work is lower. Maximizing a parallelism ratio alone is not the objective.

Sequential depth-first O(n²) temporaries do not transfer to simultaneous full child expansion. If each call keeps its own Θ(n²) workspace through join, eight-way space satisfies Q(n)=8Q(n/2)+Θ(n²)=Θ(n³); seven-way space is Θ(n^(log₂7)). Finite P and reuse can lower peaks, but require a consistent schedule/span analysis rather than an unimplemented space claim.

Executable specification of parallel Strassen

PADD(X, Y, s) allocates independent R and uses nested parallel loops to write R[i, j]=X[i, j]+s・Y[i, j], waiting before return. For s=±1, work is Θ(n²), span Θ(log n). Pad equal-order A, B to powers of two; the outer wrapper handles n=0.

```
P-STRASSEN(A, B):
    n=A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    S1=PADD(a, d,+1); S2=PADD(e, h,+1)
    S3=PADD(c, d,+1); S4=PADD(f, h,-1)
    S5=PADD(g, e,-1); S6=PADD(a, b,+1)
    S7=PADD(c, a,-1); S8=PADD(e, f,+1)
    S9=PADD(b, d,-1); S10=PADD(g, h,+1)
    M1=spawn P-STRASSEN(S1, S2)
    M2=spawn P-STRASSEN(S3, e)
    M3=spawn P-STRASSEN(a, S4)
    M4=spawn P-STRASSEN(d, S5)
```

Executable specification of parallel Strassen (continued)

```
    M5=spawn P-STRASSEN(S6, h)
    M6=spawn P-STRASSEN(S7, S8)
    M7=P-STRASSEN(S9, S10)
    sync
    allocate C[n, n], viewed as four blocks
    parallel for i=0 to n/2-1:
        parallel for j=0 to n/2-1:
            C11[i, j]=M1[i, j]+M4[i, j]-M5[i, j]+M7[i, j]
            C12[i, j]=M3[i, j]+M5[i, j]
            C21[i, j]=M2[i, j]+M4[i, j]
            C22[i, j]=M1[i, j]-M2[i, j]+M3[i, j]+M6[i, j]
    return C
```

Ten PADD calls are sequential here, but their fixed count retains Θ(log n) preparation span. Finish all before spawning products, consistent with the three-stage contract. No arbitrarily long serial issuing chain is ignored.

27.3 Multithreaded merge sort

Parallel recursive sorts with a serial merge are insufficient:

```
PAR-SORT-SERIAL-MERGE(A, l, r):
    if r-l <= 1: return
    m=floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A, l, m)
    PAR-SORT-SERIAL-MERGE(A, m, r)
    sync
    serial MERGE the sorted halves using an auxiliary buffer
```

Work satisfies2W(n/2)+Θ(n)=Θ(n log n), but span D(n/2)+Θ(n)=Θ(n), giving only Θ(log n) parallelism. Sync before merge prevents reading unfinished runs. Reducing span further requires parallel merge itself.

Define half-open cuts and tie order first

Read-only T contains runs[l1, r1),[l2, r2). Output O[o, o+N), where N is total length, must not overwrite T. Compare records by(key, original position), a total order with unique original positions, so equal keys retain original order.

```
LOWER-BOUND(T, l, r, x):
    while l < r:
        m=floor((l+r)/2)
        if T[m] < x: l=m+1
        else: r=m
    return l
```

The original run's boundary between values<x and≥x remains in current[l, r]. Positions before current l are<x; positions at and beyond current r within the original run are≥x. Each comparison preserves a possible cut and strictly reduces interval length. An empty run returns l without reading nonexistent T[l]. Work and span are O(1+log(n2+1)), including constant empty handling.

Complete parallel merge, including empty runs

```
P-MERGE(T, l1, r1, l2, r2, O, o):
    n1=r1-l1; n2=r2-l2
    if n1 < n2: swap (l1, r1),(l2, r2); swap n1, n2
    if n1 == 0: return
    m=floor((l1+r1)/2)
    j=LOWER-BOUND(T, l2, r2, T[m])
    q=o+(m-l1)+(j-l2)
    O[q]=T[m]
    spawn P-MERGE(T, l1, m, l2, j, O, o)
    P-MERGE(T, m+1, r1, j, r2, O, q+1)
    sync
```

The longer run occupies the first descriptor. Swapping views neither moves nor relabels records. Empty first means both empty. Empty second still permits recursive median placement from the first without reading an empty interval. Each call removes one pivot, leaving total child size N−1, so recursion terminates.

General proof of position, correctness, and stability

There are m−l1 first-run and j−l2 second-run records smaller than the pivot, total q−o. Every remaining record is larger in the total order. The pivot therefore belongs at q. Child output ranges[o, q),[q+1, o+N) are disjoint from one another and from the pivot.

Induct on N. Empty merge is correct. Both children are smaller and correctly merge their designated records by induction. Concatenating left result, pivot, right result is sorted and contains every input record exactly once. T is read-only and output ranges have exclusive writers, so no determinacy race occurs. Sync makes the complete answer available.

Original positions order equal keys, proving stability. A key-only comparison with the same lower_bound rule after swapping longer-run roles can place an equal right-run record first. Correct key sorting alone does not prove stability.

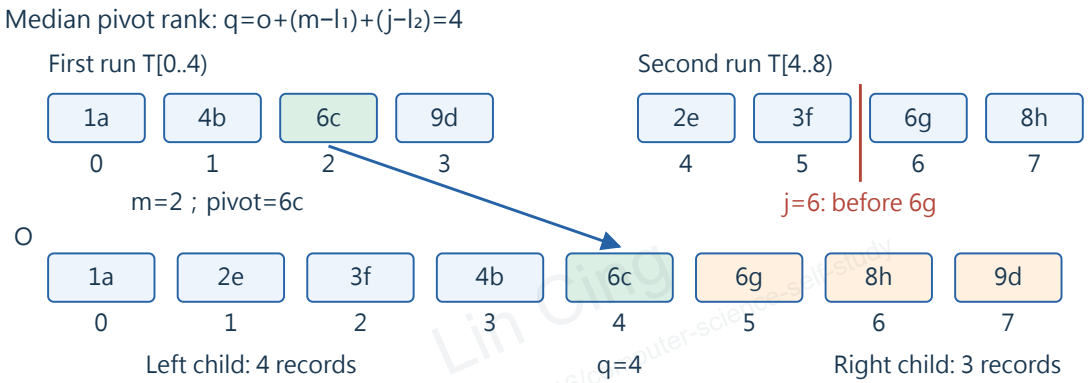
Root cut and binary-search steps for eight records

Use T=[1a,4b,6c,9d | 2e,3f,6g,8h], where letters a..h identify original positions0..7. Compare keys first, then positions. Set l1=0, r1=4, l2=4, r2=8, o=0. The first length 4 run has m=2, pivot6c.

Search[l, r)	mid	Comparison with 6c	Update
[4,8)	6	6g>6c: equal key, later position	r=6
[4,6)	5	3f<6c	l=6
[6,6)	none	empty interval	return j=6

Position q=0+(2−0)+(6−4)=4. Left child contains 4 records, right child3.

Root cut and binary-search steps for eight records (continued)



Compare(key, original position): 6c<6g. Left, pivot, right outputs are disjoint.

Every output write on the same example

One legal depth-first serialization writes pivots in the following order. Parallel interleavings may differ, but destinations stay fixed and each is written once.

Write	Pivot	q, zero-based	Meaning
1	6c	4	Root division
2	4b	3	Last record of root's left region
3	3f	2	Median after swapping longer run
4	2e	1	Small-run cut
5	1a	0	Other run empty
6	8h	6	Median after right-region swap
7	6g	5	Right region's left child
8	9d	7	Right region's right child

Every output write on the same example (continued)

Final O=[1a,2e,3f,4b,6c,6g,8h,9d], with c still before g. Running the right child first gives the same answer because T is read-only and every destination unique. Sortedness does not rely on a global sequential write order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Upper and lower bounds on worst-case merge span

For n1≥n2, either child has size at most⌊n1/2⌋+n2≤(N+n2)/2≤3N/4, even if the other cut lands at an endpoint. Binary search costs O(log(N+1)), so

$$D_M(N) \le D_M(3N/4) + O(\log(N+1)) = O(\log^2(N+1)).$$

Constant-factor shrinkage gives O(log N) levels with logarithmic sizes O(log N−cr); their arithmetic sum gives the square.

For a lower bound, merge interleaved runs[0,2,…,2m−2] and[1,3,…,2m−1], with power-of-two m. The left child retains the same equal-length interleaving structure, halving length each level. Its other-run median search takes Θ(log m) comparisons. Summing log m,(log m−1),… gives Ω(log²m), hence worst-case Θ(log²N). If the other run stays empty, search costs constant and span is only Θ(log N). Worst-case order is not exact cost for every input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Writing each record once does not pay for searches

Every nonempty call removes a distinct pivot, so there are N nonempty calls. Each creates two children, for 2N+1 total calls including empty returns. The N output writes give Ω(N), but an O(N) upper bound must also pay each binary search.

A size-m node has nonrecursive cost at most c log(m+1). Divide it among its m input records, charging each c log(m+1)/m. A fixed record x is charged along one ancestor chain until chosen as pivot.

Count r levels upward from the last call containing x. The 3/4 child bound makes that ancestor's size at least(4/3)^r. Sum these charges below, including empty children, without taking log0 or falsely assuming both small children have size at least N/4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Each record's total search charge is bounded

Function $g(u) = \log_2(u+1)/u$ decreases for u>0. The sign of its derivative numerator is u/(u+1)−ln(u+1)<0, using $\int_0^u (1+t)^{-1}dt > u/(1+u)$. Thus the charge r levels upward is at most

$$c\frac{\log_2((4/3)^r+1)}{(4/3)^r} \le c[1 + r\log_2(4/3)](3/4)^r.$$

Both Σq^r=1/(1−q) and Σr q^r=q/(1−q)² converge at q=3/4; the latter follows by differentiating finite sums and taking limits. Each record pays O(1), for O(N) total. The N+1 empty calls add constant work each. With the output lower bound, W_M(N)=Θ(N).

This proof covers arbitrary legal cuts, empty runs, and unbalanced small children. A finite example is not the upper-bound proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete two-buffer parallel merge sort

Attach original positions to records and initialize separate arrays X, Y of length n to the same input, using one parallel copy. Call SORT(X, Y,0, n), then read Y.

```
SORT(src, dst, l, r):
    if l == r: return
    if r-l == 1: dst[l]=src[l]; return
    m=floor((l+r)/2)
    spawn SORT(dst, src, l, m)
    SORT(dst, src, m, r)
    sync
    P-MERGE(src, l, m, m, r, dst, l)
```

Children swap buffer roles, leaving their sorted runs in the parent's src. Sync before merging into dst. P-MERGE input and output must be different physical buffers. Two global buffers avoid allocating a new length-n temporary per call and avoid overwriting unread input.

Buffer-swap correctness and race checks

At each SORT entry, both buffers' current intervals still contain that interval's original input. The root copy establishes this. Before launching children, the parent has not altered its interval, and each child touches only its disjoint half, so the entry property holds recursively.

A singleton copies to dst. By induction, children leave two sorted unchanged-multiset runs in parent src. After sync, P-MERGE stably merges them to dst. The parent need not promise src equals dst on exit; the equality condition is needed only on entry.

Even with role swaps, children write disjoint index ranges of the same two arrays. Parent merging waits for both. Data workspace is O(n), peak task descriptors O(n), and each call chain O(log n). Initial copying takes Θ(n) work,Θ(log n) span, without changing sorting orders.

A tight sort bound needs one compatible worst-case input

Every sort level has Θ(n) merge work and there are Θ(log n) levels, so W_S(n)=Θ(n log n). Span obeys

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\log^2 n) = O(\log^3 n).$$

For the lower bound, the same input must force expensive merges at all relevant levels. For n=2^L, place keys in L-bit-reversed index order; for n=8, use[0,4,2,6,1,5,3,7]. Root halves sort into interleaved even and odd runs. Each child's relative ranks have the same construction. Every level therefore has an Ω(log²(size)) merge, and parent merging starts only after both children finish.

Along a child chain,Σ_(r=1..L)r²=Θ(L³), giving worst-case Θ(log³n) for powers of two. Average parallelism is Θ(n/log²n). Merge's squared-log span is not the sort's span.

Compatible expensive levels for nonpowers of two

For any n, recursively assign relative ranks. If n is even, put even ranks in the first half and odd in the second. If odd, put odd ranks in the shorter first half and even in the longer second. Recursively arrange each half's ordered key set the same way. Every sorted pair of runs interleaves with length difference at most1. For powers of two, this is bit reversal.

At each such merge, the longer-run median has m same-run records on its left, and m or m+1 other-run records. The left child remains interleaved with length difference at most1 and total size between N/2−1 and N/2+1. For N≥8, both runs have Θ(N) size. The selected path has Θ(log N) levels and logarithmic search cost at each shrinking scale, giving Θ(log²N) span.

The same recursive rank arrangement gives this property at every sort node. Follow the larger child, whose size roughly halves. Its completion precedes parent merge, so critical paths concatenate and give Ω(log³n). This supplies the tight lower bound for arbitrary n, without choosing incompatible worst inputs separately at different levels.

Compare work and worst span in one model

Algorithm	Work	Worst span	Average parallelism
Recursive P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
Row-parallel matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Reduction matrix-vector	$\Theta(n^2)$	$\Theta(\log n)$	$\Theta(n^2/\log n)$
Two outer parallel loops for matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
Eight-way recursive matrix product	$\Theta(n^3)$	$\Theta(\log^2 n)$	$\Theta(n^3/\log^2 n)$
Parallel Strassen	$\Theta(n^{\log_2 7})$	$\Theta(\log^2 n)$	$\Theta(n^{\log_2 7}/\log^2 n)$
DOT-reduction matrix product	$\Theta(n^3)$	$\Theta(\log n)$	$\Theta(n^3/\log n)$
Parallel merge	$\Theta(n)$	$\Theta(\log^2 n)$	$\Theta(n/\log^2 n)$
Parallel merge sort	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n/\log^2 n)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare work and worst span in one model (continued)

Asymptotic entries assume sufficiently large n; empty and singleton cases use explicit bases. Work efficiency, small span, and low space are distinct objectives. For fixed P, compare W/P, D, and memory together rather than substituting one parallelism ratio for the full analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 27; §§27.1–27.3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix operations

Part VII Selected Topics

Scope

Elimination must preserve permutations and earlier multipliers together. Complete LUP leads to inversion, Schur complements, and multiplication reductions. SPD matrices and orthogonal residuals connect Cholesky to least squares while separating exact arithmetic, rank, and numerical sensitivity.

Prerequisites

Linear algebra, Gaussian elimination, and least squares.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Solve systems by LUP and explain permutations.
- Analyze inversion through blocks and LU.
- Handle SPD matrices and least-squares normal equations.
- 28.1 Solving systems of linear equations
- 28.2 Inverting matrices
- 28.3 Symmetric positive-definite matrices and least-squares approximation

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.1 Solving systems of linear equations

First use real exact arithmetic. A nonsingular $n \times n$ matrix A has a unique solution to $Ax=b$ for every b. A singular A can give no solution or infinitely many, so an elimination zero does not imply every b is inconsistent. Nonsingularity does not guarantee elimination without row swaps: $A=[0\ 1;1\ 0]$ is invertible but its first diagonal pivot is 0.

The LUP contract is **PA=LU**. Matrix P permutes rows, L is unit lower triangular, and U upper triangular with nonzero diagonal. Array p means current row i came from original row p[i], so(Pb)[i]=b[p[i]]. This permutes equations, not columns or unknown x.

From PAx=Pb and PA=LU, solve Ly=Pb then Ux=y. Substitution gives $Ax=P^{-1}Ly=P^{-1}Pb=b$. Nonzero triangular diagonals and invertible P establish uniqueness. Only P=I permits A=LU; assuming that equality while replacing b by Pb without explanation is inconsistent.

Forward and backward substitution dependencies

For unit lower L, process increasing i with $y[i]=b[p[i]]-\Sigma_{j<i}L[i, j]y[j]$. All required earlier y[j] are known, and each step satisfies the first i equations of Ly=Pb.

Process decreasing i with $x[i]=(y[i]-\Sigma_{j>i}U[i, j]x[j])/U[i, i]$. Later unknowns have already been solved, and the last n−i+1 equations of Ux=y now hold. Nonzero diagonals justify division. These invariants complete every row rather than guessing all unknowns simultaneously.

```
Solve(L, U, p, b):
  for i=1..n:
    y[i]=b[p[i]]-sum(j=1..i-1, L[i, j]*y[j])
  for i=n..1:
    x[i]=(y[i]-sum(j=i+1..n, U[i, j]*x[j]))/U[i, i]
  return x
```

Each substitution has n(n−1)/2 summation products; with additions and n divisions, total time is $\Theta(n^2)$, extra vector space O(n). Reuse one factorization for r right sides in $\Theta(n^3+rn^2)$. Refactoring the same A for every b discards reusable work.

Elimination, Schur complements, and existence

For nonzero first pivot a, write the active matrix$[a\ w^T; v\ B]$. Elimination uses

$[a\ w^T; v\ B]=[1\ 0; v/a\ I]\cdot[a\ w^T;0\ B-vw^T/a]$.

The lower-right $S=B-vw^T/a$ is the Schur complement. The left factor is invertible, so a nonsingular original gives a nonsingular right factor. If nonzero z satisfied $Sz=0$, vector$(-w^Tz/a, z)$ would lie in that factor's nullspace. Thus S is nonsingular and recursion can continue.

An all-zero pivot column makes the active S singular. Together with previously nonzero pivots, this implies original A singular. Conversely, a nonsingular active S has a nonzero entry in its first column. Swap that entry to the pivot and continue to completion. Hence **every nonsingular A has LUP**, without proving existence of unpermuted LU for every A.

In-place LUP swaps earlier multipliers too

Store U on and above the diagonal of work, and L's multipliers below it; unit L diagonal is implicit. Swap whole rows, including columns 1..k−1 holding previous multipliers.

```
work=copy(A); p=[1,.., n]
for k=1..n:
  r=argmax(i=k..n, abs(work[i, k]))
  if work[r, k]==0: report singular
  swap(work[k,1..n], work[r,1..n]); swap(p[k], p[r])
  for i=k+1..n:
    work[i, k]=work[i, k]/work[k, k]
    for j=k+1..n:
      work[i, j]=work[i, j]-work[i, k]*work[k, j]
```

Before step k, the block invariant is $PA=[L_{11}\ 0; L_{21}\ I]\cdot[U_{11}\ U_{12};0\ S]$. An active-row permutation R changes S to RS and must also change L_{21} to RL_{21}. Otherwise the lower product rows refer to wrong original equations. New elimination multipliers then extend L, preserving PA=LU.

There are $\Sigma_k(n-k)^2=\Theta(n^3)$ trailing-block updates and $\Theta(n^2)$ pivot-search work. These count arithmetic operations, not bit costs for arbitrarily long rationals.

Complete example with two row swaps

Take $A=[0\ 2\ 1;1\ 1\ 0;2\ 3\ 4]$, $b=(0,0,7)^T$. The largest first-column magnitude is 2. Swap rows 1,3, so $p=(3,2,1)$. Multipliers $m_{21}=1/2$, $m_{31}=0$ leave trailing block$[-1/2\ -2;2\ 1]$.

The second column again selects magnitude 2. Swap current rows 2,3, so $p=(3,1,2)$. **Earlier L multiplier 1/2 moves from row 2 to row 3.** New $m_{32}=(-1/2)/2=-1/4$ gives final pivot$-2-(-1/4)\cdot 1=-7/4$.

Thus $L=[1\ 0\ 0;0\ 1\ 0;1/2\ -1/4\ 1]$, $U=[2\ 3\ 4;0\ 2\ 1;0\ 0\ -7/4]$.

Multiplying gives $LU=[2\ 3\ 4;0\ 2\ 1;1\ 1\ 0]$, exactly PA in original-row order 3,1,2. Swapping only the trailing block leaves an erroneous row 2 multiplier 1/2 and breaks $LU=PA$.

Complete example with two row swaps (continued)

Two pivots: swap trailing blocks and saved L together

Green: saved L multipliers. Blue: U or the active trailing block. Unit L diagonal is implicit.

Original A

0	2	1
1	1	0
2	3	4

p=(1,2,3)

→ Swap rows 1/3, then eliminate

2	3	4
1/2	-1/2	-2
0	2	1

p=(3,2,1)

→ Swap rows 2/3, then eliminate

2	3	4
0	2	1
1/2	-1/4	-7/4

p=(3,1,2)

Swap two moves first-column 1/2 from current row 2 to row 3.

Final m31=1/2, m32=−1/4, U33=−7/4, satisfying PA=LU.

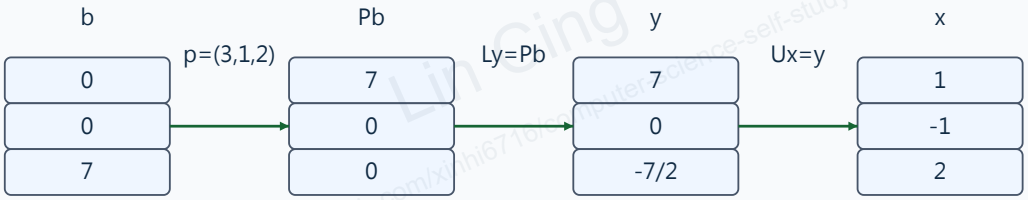
Substitution on the example and numerical boundaries

Here $Pb=(7,0,0)^T$. Forward substitution gives $y_1=7$, $y_2=0$, $y_3=0-(1/2)\cdot 7-(-1/4)\cdot 0=-7/2$.

Backward substitution gives $x_3=(-7/2)/(-7/4)=2$, $x_2=(0-1\cdot 2)/2=-1$, $x_1=(7-3\cdot(-1)-4\cdot 2)/2=1$. Original row checks are $0-2+2=0$, $1-1=0$, $2-3+8=7$.

From b to Pb, y, x: two triangular systems

Same A, b. Row permutation reorders right-side equations, not unknown x.



Forward: $y_3=0-(1/2)\cdot 7=-7/2$.

Backward: $x_3=2$, $x_2=-1$, $x_1=1$. Original A gives $(0,0,7)$.

Substitution on the example and numerical boundaries (continued)

Partial pivoting selects the largest absolute entry only in the current column. Exact arithmetic guarantees $|m_{ik}|\leq 1$ for this step, limiting column multipliers. It **does not guarantee small floating-point error for every matrix**. Element growth can occur, and ill-conditioning can amplify small input perturbations. Mathematical nonzero pivots differ from implementation tolerance thresholds; near-singularity decisions need scale and error assumptions.

28.2 Inverting matrices

Column j of A^{-1} solves $Ax=e_j$. One LUP plus n substitutions costs $\Theta(n^3)+n\Theta(n^2)=\Theta(n^3)$. Check $AA^{-1}=I$; for square matrices in exact arithmetic, a right inverse is the unique inverse. Solving one $Ax=b$ normally uses substitution directly rather than computing an inverse first.

For A=[1 1;2 5], L=[1 0;2 1], U=[1 1;0 3]. For e_1, forward y=(1,−2), then $x_2=-2/3$, $x_1=5/3$. For e_2, y=(0,1), then $x_2=1/3$, $x_1=-1/3$. Thus A^{-1}=[5/3 −1/3;−2/3 1/3], not entrywise reciprocals.

Multiplication checks 5/3−2/3=1,−1/3+1/3=0,10/3−10/3=0,−2/3+5/3=1. A singular A can still solve selected right sides, but not all unit vectors uniquely, and has no inverse under this definition.

Block-inverse order and assumptions

Write A=[B C; D E], require invertible B, and define $S=E-DB^{-1}C$. Factor A=[I 0; DB^{-1} I] · [B C;0 S]. If A is invertible, so is S. Invert the factors in **reverse order**:

```
A⁻¹ = [ B⁻¹+B⁻¹CS⁻¹DB⁻¹    −B⁻¹CS⁻¹ ]
      [ −S⁻¹DB⁻¹              S⁻¹     ]
```

Matrix factors do not commute. Do not exchange C, D, or inverse positions. In the scalar example B=1, C=1, D=2, E=5, S=3; the top-left term 1+1 · 1 · (1/3) · 2 · 1=5/3 matches columnwise inversion.

This block recursion does not directly cover every invertible A. Matrix[0 1;1 0] is invertible but leading B=0 cannot be inverted. SPD properties below make leading blocks and Schur complements invertible. General matrices require permutations or the following theoretical reduction.

Block-inverse order and assumptions (continued)

Schur complement: the exact block after elimination

Left-multiply by $[I\ 0;-DB^{-1}\ I]$. Matrix B must be invertible; factor order matters.

A

B	C
D	E

After elimination

B	C
0	$E-DB^{-1}C$

$S=E-DB^{-1}C$; $A=[I\ 0;\ DB^{-1}\ I]\cdot[B\ C;0\ S]$.

Example $B=1$, $C=1$, $D=2$, $E=5$: $S=3$, $A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$.

For $B=0$, B^{-1} is undefined. Even invertible A can have singular B.

Reductions between multiplication and inversion

For $n\times n$ B, C, construct $3n\times 3n$ $T=[I\ B\ 0;0\ I\ C;0\ 0\ I]$. Its inverse is$[I\ -B\ BC;0\ I\ -C;0\ 0\ I]$; multiplication gives upper-right $BC-BC=0$. If inversion cost $I(n)=\Omega(n^2)$ and $I(3n)=O(I(n)),\Theta(n^2)$ construction plus one inverse extracts BC in $O(I(n))$. Dimension and regularity assumptions are essential.

Conversely, first invert SPD blocks. Leading B and Schur S are SPD, as proved next, so invert two half-size blocks and perform a constant number of products:$T(n)\leq 2T(n/2)+O(M(n))$. Assume $M(n)=\Omega(n^2)$, padding below 2n changes only constants, and $M(n/2)\leq cM(n)$ for $c<1/2$. Recurrence levelℓ costs at most$(2c)^\ell O(M(n))$; the geometric sum gives $T(n)=O(M(n))$. Powers $M(n)=\Theta(n^\omega),\omega\geq 2$ satisfy this.

For general real nonsingular A, Gram $G=A^TA$ is SPD because $z^TGz=||Az||^2>0$. Formula $A^{-1}=G^{-1}A^T$ follows by multiplying it by A to get I, and retains $O(M(n))$ cost. Pad nonpowers of two as diag(A, I), whose original inverse block is unchanged. This is an **exact-arithmetic complexity reduction**. Forming A^TA solely for this proof is unsuitable as a default floating-point strategy because it squares the condition number. Complex matrices require conjugate transpose A^*, not ordinary transpose.

28.3 Symmetric positive-definite matrices and least-squares approximation

Real SPD means $A=A^T$ and $z^TAz>0$ for **every** nonzero z. Singularity would give a nonzero $Az=0$ and zero quadratic form, so SPD implies nonsingularity. For any leading block B, pad its nonzero vector z to(z,0); positivity of the full quadratic form proves B SPD.

For SPD $A=[B\ C;\ C^T\ E]$, $S=E-C^TB^{-1}C$ is symmetric. For nonzero y, set $z=-B^{-1}Cy$. Then

$(z, y)^TA(z, y)=z^TBz+2z^TCy+y^TEy=y^T(E-C^TB^{-1}C)y=y^TSy>0$.

Thus S is SPD. The first pivot $e_1^TAe_1=A_{11}$ is positive, and elimination recursively handles SPD Schur complements, making every pivot positive. This proves no-pivot elimination avoids zero. Positive original diagonals alone are insufficient:[1 2;2 1] has second pivot−3.

Computing LDL^T and Cholesky

Symmetric elimination gives positive pivots d_j and unit-lower multipliers. Induction gives $A=LDL^T$ with positive diagonal D. Set $C=LD^{1/2}$ to obtain Cholesky $A=CC^T$, with lower C and positive diagonal. A direct column algorithm is

$C[j, j]=\mathrm{sqrt}(A[j, j]-\Sigma_{k<j}C[j, k]^2)$,

$C[i, j]=(A[i, j]-\Sigma_{k<j}C[i, k]C[j, k])/C[j, j]$, for $i>j$.

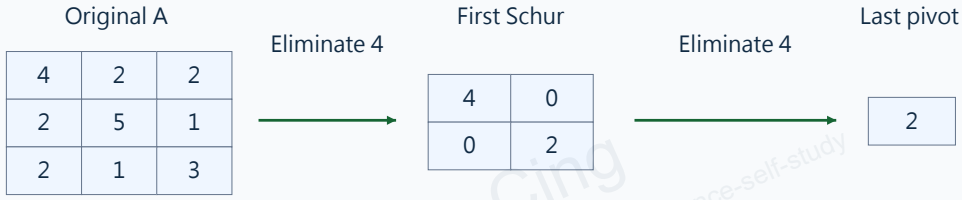
SPD Schur complements keep every square-root argument positive. Triple summations take $\Theta(n^3)$, storing only one triangle. A common flop convention gives leading $n^3/3$, versus about $2n^3/3$ for general LU. This does not promise exactly twice the speed on every machine.

For $A=[4\ 2\ 2;2\ 5\ 1;2\ 1\ 3]$, $C_{11}=2$, $C_{21}=C_{31}=1$, $C_{22}=\mathrm{sqrt}(5-1)=2$, $C_{32}=(1-1\cdot 1)/2=0$, $C_{33}=\mathrm{sqrt}(3-1-0)=\sqrt{2}$. Hence $C=[2\ 0\ 0;1\ 2\ 0;1\ 0\ \sqrt{2}]$. Its product has diagonals 4,5,3 and off-diagonals 2,2,1, matching A.

Computing LDL^T and Cholesky (continued)

SPD Schur complements make Cholesky legal

Every SPD elimination pivot is positive; here 4,4,2.



C rows=[2,0,0]、[1,2,0]、[1,0,√2].

Product CC^T has diagonals 4,5,3 and off-diagonals 2,2,1, matching every original entry.

Least squares: necessity and global sufficiency

For $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, minimize $f(x)=||Ax-b||_2^2$. Linearity is in coefficients. Basis functions $A[i, j]=f_j(t_i)$ may be nonlinear in t; for example 1, t, t^2 still give a linear coefficient model.

For any direction z, expand $f(x+tz)=f(x)+2t\, z^T A^T(Ax-b)+t^2||Az||^2$. At a minimum, each directional linear term must vanish; otherwise a small t of the opposite sign decreases f. Therefore $A^TAx=A^Tb$. Conversely, if these normal equations hold, setting t=1 gives $f(x+z)=f(x)+||Az||^2 \geq f(x)$. The stationary point is a global minimum.

Gram A^TA is symmetric positive semidefinite, with $z^TA^TAz=||Az||^2$. It is SPD **if and only if the columns are linearly independent**. Full column rank gives unique coefficients. Rank deficiency permits multiple coefficients but one prediction Ax:two minimizers differ by z with Az=0. At least one minimizer always exists. Choose independent columns as a basis, solve their full-rank SPD normal equations, and observe that residual orthogonality to the basis gives orthogonality to all columns. For all-zero A, take x=0, p=0, residual b.

Full-column-rank pseudoinverse and projection

For full column rank, define $n \times m$ $A^{+}=(A^{T}A)^{-1}A^{T}$, so $\hat{x}=A^{+}b$. Although $A^{+}A=I_n$, generally $AA^{+} \neq I_m$. Instead $Q=A(A^{T}A)^{-1}A^{T}$ satisfies $Q^{T}=Q$, $Q^{2}=Q$, and $QAz=Az$ for every column-space vector Az. It projects b onto Col(A), with $A^{T}(I-Q)b=0$.

For the next 3×2 design, A^{+} rows are$(5/6,1/3,-1/6),(-1/2,0,1/2)$, mapping $b=(0,2,1)$ to$(1/2,1/2)$. Q rows are$(5/6,1/3,-1/6),(1/3,1/3,1/3),(-1/6,1/3,5/6)$. This is not I_3; it maps b to$(1/2,1,3/2)$, not b itself.

For rank-deficient A, the inverse$(A^{T}A)^{-1}$ is undefined and this formula cannot be applied. The general Moore–Penrose inverse can be constructed by SVD. The formula and coefficient uniqueness here require full column rank.

Complete fit to three noncollinear data points

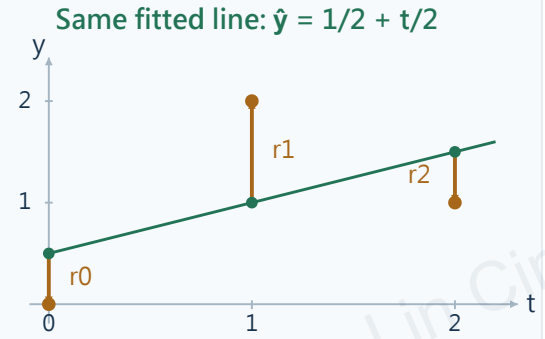
Use$(t, y)=(0,0),(1,2),(2,1)$, coefficients $x=(c, m)^{T}$, and model $\hat{y}=c+mt$.

$A=[1\ 0;1\ 1;1\ 2]$, $b=(0,2,1)^{T}$, $A^{T}A=[3\ 3;3\ 5]$, $A^{T}b=(3,4)^{T}$. Subtract $3c+3m=3$ from $3c+5m=4$ to get $m=1/2$, $c=1/2$.

t	Observed y	Prediction $\hat{y}$	$r=y-\hat{y}$
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

The components of $A^{T}r$ are$-1/2+1-1/2=0$ and $0 \cdot (-1/2)+1 \cdot 1+2 \cdot (-1/2)=0$. Squared residual norm is 3/2. Independent columns$(1,1,1),(0,1,2)$ establish unique coefficients. **Collinearity of data points does not determine this design's column rank.** Keeping t and replacing y by$(1,2,3)$ makes the data collinear without changing A; only the residual becomes 0.

Complete fit to three noncollinear data points (continued)

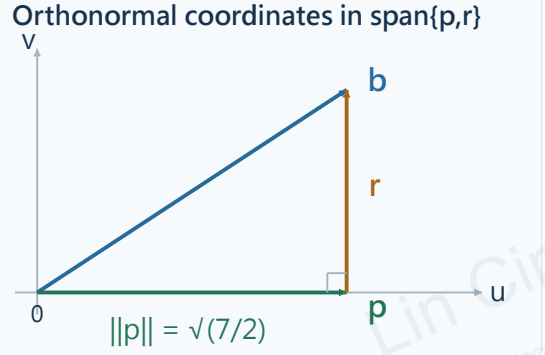


t	y	ŷ	r
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$r^T r = 1/4 + 1 + 1/4 = 3/2$

Amber: vertical error at fixed t. Green: predicted values and fitted line.

Treat the three observations as one vector in R^3



Coordinates in R^3

b = (0, 2, 1)
p = (1/2, 1, 3/2)
r = (−1/2, 1, −1/2)

u = p / ||p||
v = r / ||r||

$A^T r = (0, 0)$
$||r||^2 = 3/2$

Section E=span{p,r}; its horizontal line is E∩Col(A)=span{p}

For these nonzero orthogonal p,r, use orthonormal axes u=p/||p|| and v=r/||r||. The listed b,p,r retain their original R^3 coordinates. Since $A^T r=0$, r is perpendicular to all of Col(A). Section E=span{p,r} meets Col(A) only along span{p}; the horizontal line denotes this intersection. Col(A) remains a two-dimensional plane in R^3.

Orthogonal residuals, conditioning, and rank deficiency

Let $p=A\hat{x}$, $r=b-p$. Equation $A^{T}r=0$ makes r orthogonal to all Col(A), with $b=p+r$. For coefficients $\hat{x}+z$, the residual is $r-Az$, with orthogonal terms. Squared error is $||r||^{2}+||Az||^{2}$, connecting normal equations to minimum-distance geometry.

In the(t, y) scatterplot, r_{i} is the **vertical y-error at fixed t_{i}**, not the shortest point-to-line distance. The other picture treats all b as one vector in R^{3}; there r is the shortest displacement from b to subspace Col(A). These pictures use different spaces.

With repeated columns A=[1 1;1 1;1 1], b=(0,2,1), only $c_{1}+c_{2}=1$ is fixed. Every(s,1−s) is optimal, while prediction(1,1,1) and residual(−1,1,0) remain unique.

For full column rank,$\kappa_{2}(A^{T}A)=\kappa_{2}(A)^{2}$ because singular values are squared. Matrix diag(1,10^{-3}) has condition 10^{3}, its Gram 10^{6}. This measures sensitivity, not operation count. QR or SVD often solves floating-point least squares, especially near rank deficiency. Exact normal-equation correctness does not guarantee stability for every floating dataset.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 28.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear programming

Part VII Selected Topics

Scope

Equivalent models, exact pivots, and cycle prevention establish two-phase simplex. Dual solutions and improving rays certify its three possible outcomes.

Prerequisites

Convex polyhedra, standard form, and duality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Convert models to standard and slack forms.
- Run simplex from a basic feasible solution.
- Use weak/strong duality and feasibility initialization.
- 29.1 Standard and slack forms
- 29.2 Formulating problems as linear programs
- 29.3 The simplex algorithm
- 29.4 Duality
- 29.5 The initial basic feasible solution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.1 Standard and slack forms

Our standard form is $\max\ c^T x,\ Ax \le b,\ x \ge 0$, with A∈R^(m×n). Constraints are non-strict and variables continuous real numbers. Finite binary-input bit complexity assumes rational coefficients. Other texts call equality form standard; check conventions first.

A feasible point satisfies all constraints; an optimum maximizes the objective over feasible points. Infeasible means no feasible point. Unbounded means feasible with arbitrarily large objective, not merely an unbounded feasible region. For example, max(−x) over x≥0 has an unbounded region and optimum 0.

Intersections of finitely many linear half-spaces are convex polyhedra:Ax≤b, Ay≤b imply A(λx+(1−λ)y)≤b for 0≤λ≤1. Convexity neither guarantees integer solutions nor makes nonlinear products linear constraints.

Four conversions must recover original variables

Convert minimizing f to maximizing−f and negate the returned objective. Multiply $a^T x \ge b$ by−1. Split equality into opposite inequalities. Replace unrestricted u by $u^+ - u^-$ with both nonnegative. Every u has representation(max(u,0), max(−u,0)); every such pair recovers an original value by subtraction. Although not one-to-one, the optimum values correspond.

For $\min\ 2u - v;\ u + v = 3;\ u - 2v \ge -3;\ v \ge 0;\ u\ \text{free}$, maximize$-2u^+ + 2u^- + v$ subject to

$u^+ - u^- + v \le 3;\ -u^+ + u^- - v \le -3;\ -u^+ + u^- + 2v \le 3$.

Substituting u=3−v gives v≤2 and original objective 6−3v, so v=2, u=1 attains minimum 0, corresponding to maximum 0. Starting with n variables, m constraints, sign splitting uses at most 2n variables and equality splitting at most 2m constraints, adding only constant coefficient bit length.

Slack form, bases, and basic feasibility

Introduce $s=b-Ax\geq 0$. Let $M=[A\ I_m]$, $q=(x,s)\geq 0$, so $Mq=b$. Slacks are uniquely determined, not freely allocated independent decisions.

Choose m independent columns as basis B and let N be the others. Invertibility gives

$q_B=M_B^{-1}b-M_B^{-1}M_Nq_N;\ z=v+\rho_N^Tq_N$.

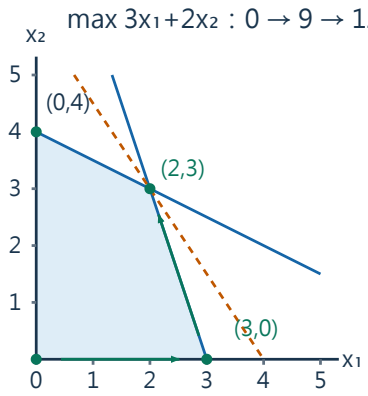
Set all nonbasic variables to 0 for a basic solution. It is feasible exactly when $M_B^{-1}b\geq 0$ componentwise. Initial slack basis I is feasible exactly when $b\geq 0$. Basic values need not be positive; zero basic variables cause degeneracy. A fixed basis determines a unique dictionary because the inverse is unique.

Geometry, level lines, and basis on one example

Use $\max\ 3x_1+2x_2;\ x_1+2x_2\leq 8;\ 3x_1+x_2\leq 9;\ x\geq 0$. Feasible vertices are(0,0),(3,0),(2,3),(0,4). The vertex path improves objective 0→9→12, ending at(2,3).

Initially $x_3=8-x_1-2x_2;\ x_4=9-3x_1-x_2;\ z=3x_1+2x_2$, with basis$\{x_3, x_4\}$ and slacks(8,9).

Geometry, level lines, and basis on one example (continued)



Blue: two resource constraints

$x_1+2x_2=8$

$3x_1+x_2=9$

Green: two simplex moves

Orange dashed: $3x_1+2x_2=12$

Intersection(2,3), objective 12

An entire edge can be optimal. For objective x_1+2x_2, the segment(0,4) to(2,3) has value 8. A vertex method returns one optimal basic solution without asserting uniqueness.

29.2 Formulating problems as linear programs

Specify each variable's meaning, unit, and domain. Map every model-feasible solution to an original plan and every original feasible plan back. Matching objectives while omitting capacity or conservation does not give equivalent models.

For shortest paths, use potentials d_v, fix d_s=0, and constrain d_v−d_u≤w_uv per edge. Maximize d_t. Potentials are unrestricted; negative paths can require d_t<0.

First assume all vertices are source-reachable, no negative cycle exists, and t is reachable. Summing inequalities along any s-to-t path gives d_t≤its length. Every feasible potential is therefore a lower bound on the shortest distance. Actual distances δ(s, v) satisfy all triangle inequalities and attain δ(s, t). Maximization, not minimization, finds the shortest distance.

Potential boundaries are not program Infinity values

Restricting to the s-reachable subgraph gives finite true distances when no negative cycle exists. Report unreachable t separately;∞ is not a real LP variable. Keeping an unreachable subgraph with a negative cycle instead makes the whole potential LP infeasible even if the source-to-target distance is ordinary. That assumption cannot be omitted.

If the entire graph has no negative cycle, a zero-edge super-source gives globally feasible potentials. For t unreachable from s, increase every unreachable vertex's potential by arbitrary $K>0$. No reachable-to-unreachable edge exists, so all differences stay feasible. Then d_t is unbounded, corresponding to unreachability, not a negative-length path.

For $s\to a:2$, $a\to t:-1$, $s\to t:4$, constraints are $d_a\le 2$, $d_t\le d_a-1$, $d_t\le 4$. Values(0,2,1) are feasible and the two-edge path bounds $d_t\le 1$, so optimum 1. Erroneously requiring all $d\ge 0$ would exclude correct solutions on other negative-total-path examples.

Flow models use nonnegative variables per original arc

Use arc flow g_{uv} rather than the preceding chapter's antisymmetric net flow. Constrain $0\le g_{uv}\le c_{uv}$, internal outflow minus inflow 0, and maximize net source outflow. Antiparallel original arcs each retain their own variable and capacity. Do not also impose $g_{uv}=-g_{vu}$.

Every original flow satisfies the model, and capacity/conservation make every model solution a flow. There are m variables and $O(n+m)$ constraints, without n^2 nonedge variables.

Minimum-cost flow additionally fixes source net outflow D and sink net inflow D, and minimizes $\Sigma a_{uv}g_{uv}$. Demand is exact; maximizing flow then reporting cost does not solve the same objective. Finite original capacities bound negative-cost circulation, which need not make the LP unbounded. Prohibiting circulation requires an additional model restriction.

Complete minimum-cost-flow example

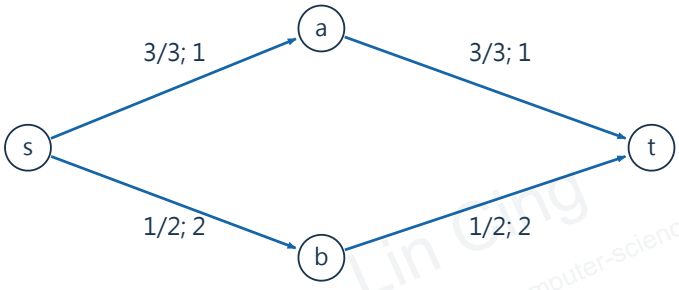
Arcs s→a, a→t each have capacity 3, unit cost 1. Arcs s→b, b→t each have capacity 2, unit cost 2. Demand D=4. Conservation makes route flows u, v, with u+v=4,0≤u≤3,0≤v≤2, and objective min 2u+4v.

Substitute v=4−u to get cost 16−2u and feasible 2≤u≤3. Thus u=3, v=1 costs 10. This specifies every arc flow, demand, feasible interval, and optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete minimum-cost-flow example (continued)

Demand D=4. Labels: flow / capacity; unit cost.



u+v=4

2≤u≤3

Cost=16−2u

Optimum u=3, v=1

Upper cost 3×(1+1)=6; lower cost 1×(2+2)=4; total 10

Using u+v≤4 would incorrectly make zero flow at cost 0 optimal. Omitting u≤3 would incorrectly select u=4 at cost 8.

Multicommodity flow shares capacities

Commodity k has(s_k, t_k, D_k) and per-arc g^k_uv≥0. Each commodity separately conserves flow and meets endpoint net supplies±D_k. Shared capacity is Σ_kg^k_uv≤c_uv. A feasibility objective may be0; costs can also be included.

Two commodities each uniquely requiring a capacity 1 bottleneck u→v with demand 3/4 are individually feasible but jointly need 3/2>1. Independent max-flow solutions miss this shared resource.

With k commodities, m arcs, n vertices, there are km variables and O(kn+m) constraints, a polynomial-size LP for rational data. This models splittable flow. One-path-per-commodity or integral-flow requirements create different discrete problems, without the same general LP guarantees.

29.3 The simplex algorithm

Fix dictionary signs as `x_i=b_i-Σ_(j∈N)a_ijx_j; z=v+Σ_(j∈N)c_jx_j`. Entry a is the coefficient being subtracted, opposite the displayed signed coefficient. All ratios and pivots use this contract.

From feasible b≥0, choose c_e>0 to enter, set x_e=θ≥0, and keep other nonbasics 0. Basic x_i=b_i−a_ieθ. Rows a_ie≤0 impose no upper bound; positive a_ie requires θ≤b_i/a_ie. Choose the minimum nonnegative ratio with a positive denominator,**including zero**.

If no denominator is positive, direction q_e=1, q_i=−a_ie≥0, others 0 gives feasible ray x(θ)=x(0)+θq with objective v+c_eθ→∞. Do not write x_e=∞; infinity is not a feasible real value.

Pivoting retains the leaving variable's coefficient

For leaving l and a_le≠0, solve its row exactly:

$x_e=b_l/a_{le}-x_l/a_{le}-\Sigma_{(j\in N\setminus\{e\})}(a_{lj}/a_{le})x_j$.

Substitute into every other row and objective, then exchange B, N. Variable x_l becomes nonbasic, not absent. Omitting−x_l/a_le changes the feasible set.

New entries are a_el=1/a_le, b_e=b_l/a_le, a_ej=a_lj/a_le. For i≠l, b′_i=b_i−a_ieb_e, a′_il=−a_ie/a_le, a′_ij=a_ij−a_iea_lj/a_le. The objective becomes v′=v+c_eb_e, c′_l=−c_e/a_le, c′_j=c_j−c_ea_lj/a_le.

This invertible substitution preserves every point satisfying the equations and its objective, not merely the current basic point. With nonnegativity, it defines the same LP.

General feasibility proof for the ratio test

The new entering value b′_e=θ=b_l/a_le≥0. Other b′_i=b_i−a_ieθ stay nonnegative:for positive a_ie, the minimum ratio bounds θ; for nonpositive a_ie, the new value is at least b_i≥0. With new nonbasics 0, the new basic solution is feasible.

Objective improvement is c_eθ≥0, strict for θ>0. At θ=0, the point and objective may stay unchanged while the basis changes. This proves both why negative-denominator rows do not restrict the step and why zero ratios cannot be excluded.

```
while some reduced cost c[e]>0:
    choose entering e
    candidates={i in B : a[i, e]>0}
    if candidates empty: return feasible point + improving ray
    l=minimum-ratio row using a specified anti-cycling rule
    pivot(l, e)
return current basic solution
```

Two pivots on the same two-dimensional example

Initially B={3,4}. Enter x_1:ratios 8/1=8,9/3=3 make x_4 leave with θ=3:

$x_1=3-x_2/3-x_4/3;\ x_3=5-5x_2/3+x_4/3;\ z=9+x_2-x_4.$

Enter x_2:ratios 3/(1/3)=9,5/(5/3)=3 make x_3 leave:

$x_2=3-3x_3/5+x_4/5;\ x_1=2+x_3/5-2x_4/5;\ z=12-3x_3/5-4x_4/5.$

Nonbasic $x_3=x_4=0$ gives(2,3), tight in both constraints. Every reduced cost is nonpositive, so every feasible point has objective≤ 12. No further numerical search of the polygon is needed.

Zero steps are legal pivots

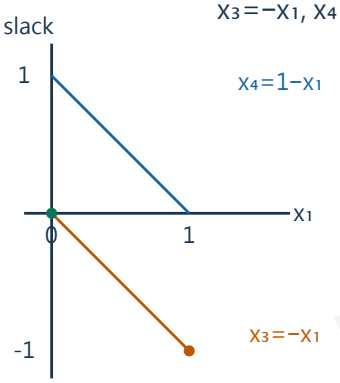
Use $\max\ z=x_1;\ x_3=-x_1;\ x_4=1-x_1$, all variables nonnegative. Initially B={3,4}, N={1}, basic values(0,1).

Basic row	Positive denominator	Ratio	At θ=0	Incorrect θ=1
x_3	1	0	0	−1, infeasible
x_4	1	1	1	0

Let x_3 leave. New $x_1=-x_3$, $x_4=1+x_3$, $z=-x_3$ has the same point(0,0,1), objective 0, but now nonpositive reduced costs prove optimality. Ignoring zero ratios immediately violates feasibility.

Zero steps are legal pivots (continued)

$x_3=-x_1$, $x_4=1-x_1$: do not skip a zero ratio



$x_1=0$: $x_3=0$, $x_4=1$, feasible

Ratios: 0/1=0 and 1/1=1

Legal step $\theta=0$; x_3 leaves

Wrong $\theta=1$: $x_3=-1$, infeasible

Only $x_1=0$ is feasible

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finitely many bases alone do not prevent cycling

Matrix M=[A I] has m rows, and a basis selects m independent columns. There are at most C(n+m, m) possibilities. A fixed basis determines one dictionary, so an infinite run repeats a basis. Degenerate steps may preserve objective, so mere nondecrease does not forbid repetition.

Use exact lexicographic perturbation for a self-contained anti-cycling proof. From an initial feasible dictionary $q_B=\beta-Dq_N$, replace its RHS by $\beta_i+\varepsilon^i$ for $i=1,\ldots, m$. Here ε is a positive infinitesimal,**not an arbitrary floating decimal**. Nonnegative constants and a positive first ε coefficient make the initial dictionary strictly feasible in the ordered field.

Represent comparisons by coefficient vectors:constant first, then $\varepsilon,\varepsilon^2,\ldots$; the first nonzero coefficient determines sign. Continue minimum-ratio tests with positive denominators, using later coefficients to resolve equal real ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the lexicographic rule terminates

Let the initial equation matrix be $M_0=[I\ D]$, with RHS $\beta+\Sigma e_i\varepsilon^i$. For any invertible basis K, its basic values are

$M_0,\ K^{-1}\beta+\Sigma(M_0,\ K^{-1}e_i)\varepsilon^i$.

No inverse row is entirely zero, so no basic value is the identically-zero polynomial. Feasibility therefore makes every basic value strictly positive. Two positive-denominator ratios cannot coincide identically, or two inverse rows would be proportional, contradicting invertibility. Every legal θ is strictly positive, and $c_e>0$ makes the lexicographic objective strictly increase.

A fixed basis has one dictionary/objective, so it cannot repeat. Finite bases imply termination. Taking constant coefficients returns an original-LP feasible basic point. Reduced costs do not depend on RHS; nonpositive costs establish its optimality. An improving ray likewise has ε-independent direction and certifies original unboundedness from the constant feasible point. No guessed sufficiently-small floating ε is required.

A reproducible lexicographic tie

Use $x_3=2-x_1;\ x_4=2-x_1-x_2;\ \max\ x_1+x_2$, with RHS $2+\varepsilon,2+\varepsilon^2$. Entering x_1 compares $2+\varepsilon$ against $2+\varepsilon^2$; the latter is smaller, so x_4 leaves.

New rows are $x_1=2+\varepsilon^2-x_2-x_4;\ x_3=\varepsilon-\varepsilon^2+x_2+x_4;\ z=2+\varepsilon^2-x_4$. No reduced cost is positive. The constant solution is $x_1=2$, other variables 0, objective 2. Although x_3 has zero constant,$\varepsilon-\varepsilon^2$ is strictly positive and separates degeneracy symbolically.

Another common anti-cycling choice is Bland's rule:smallest-index positive reduced-cost entering variable, and smallest-index leaving variable among minimum-ratio ties. Both parts matter. The complete termination proof here uses lexicographic perturbation and does not claim arbitrary choices terminate.

Per-pivot cost is not total simplex cost

A dense ordinary dictionary updates O(mn) coefficients per pivot. This does not prove polynomial total time; basis count C(n+m, m) can be exponential. Lexicographic RHS adds m+1 coefficients per row and a polynomial update/comparison factor, but retains only a finite/exponential general worst-case pivot guarantee.

Rational exact pivots use fractions. Basis solutions are ratios of subdeterminants. After clearing original denominators, a determinant has at most m! terms, giving bit length bounded polynomially by dimension and input bit length. A single basis thus needs no unrestricted representation growth, but pivot count need not be polynomial.

Floating implementations need separate feasibility/reduced-cost tolerances, near-zero ratio handling, and residual recomputation. Clamping negative b to 0 can change the problem. Ellipsoid and interior-point methods are different algorithms; their polynomial theory does not follow from these pivot counts.

29.4 Duality

Primal $\max\ c^{\mathsf{T}}x,\ Ax \le b,\ x \ge 0$ has dual $\min\ b^{\mathsf{T}}y,\ A^{\mathsf{T}}y \ge c,\ y \ge 0$. Its m resource rows give m prices and n primal variables give n price inequalities. Transpose A; merely exchanging max/min is insufficient.

For feasible x, y, nonnegativity and both constraint systems give

$c^{\mathsf{T}}x \le x^{\mathsf{T}}A^{\mathsf{T}}y = y^{\mathsf{T}}Ax \le y^{\mathsf{T}}b$.

This is weak duality. A dual feasible point bounds every primal objective above. Equality certifies optimality on both sides. Equal objective numbers without checking feasibility are not a certificate.

Constructing a dual from the terminal basis

Let M=[A I], d=(c,0), and terminal basis matrix M_B be invertible. Define row $y^T=d_B^T M_B^{-1}$. Substituting basic variables gives

$z=y^T b+\Sigma_(j\in N)(d_j-y^T M_j)q_j$.

For every column, reduced cost ρ_j=d_j−yᵀM_j, and basic costs are 0. Terminal $\rho\le 0$ gives $-y_i\le 0$ from slack columns e_i, hence $y\ge 0$. Decision columns give $c_j-y^T A_j\le 0$, hence $A^T y\ge c$.

The current feasible basic objective is yᵀb, attaining weak-duality equality. A finite optimal primal therefore has a same-value dual. Read y_i as the negative reduced cost of original slack i, or0 if that slack is basic. This follows from the full dictionary identity, not coefficient matching at only one point.

The same example's dual and complementary slackness

The two-dimensional example has dual $\min\ 8y_1+9y_2;\ y_1+3y_2\ge 3;\ 2y_1+y_2\ge 2;\ y\ge 0$. Terminal $z=12-3x_3/5-4x_4/5$ gives y=(3/5,4/5). Both dual rows are tight, and 24/5+36/5=12 matches x=(2,3).

The general gap decomposes exactly as

$b^T y-c^T x=y^T(b-Ax)+x^T(A^T y-c)$.

For feasible sides, every summand is nonnegative. Gap 0 is therefore equivalent to all products y_i(b_i−A_ix)=0 and x_j((Aᵀy)_j−c_j)=0. This proves necessity and sufficiency of complementary slackness. Positive primal slack forces its price 0; positive primal variable forces its dual row tight.

For $\max\ 2x_1+x_2;\ x_1+x_2\le 4;\ x_1\le 2;\ x\ge 0$, dual y=(1,1) gives upper bound 6. Point(2,2) has value 6 and all complementary products 0. Point(1,3) has value 5, with the sole positive product $y_2(2-x_1)=1$.

The same example's dual and complementary slackness (continued)

$$\text{gap} = y^T(b - Ax) + x^T(A^Ty - c) \geq 0$$

$x=(2,2)$	$x'=(1,3)$
$y=(1,1)$, dual value 6	$y=(1,1)$, dual value 6
primal 6	primal 5
$y_2(2-x_1) = 0$	$y_2(2-x_1) = 1$
Other three products are 0	Other three products are 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unboundedness and infeasibility are not interchangeable

An unbounded primal has no feasible dual, or one fixed dual value would bound all primal values. A downward-unbounded dual similarly makes primal infeasible. The converse need not hold:an infeasible primal need not have an unbounded dual, because both can be infeasible.

Primal `max-x; 0x≤-1; x≥0` is infeasible; dual `min-y; 0y≥-1; y≥0` is feasible and downward-unbounded. Changing primal objective to max x gives dual constraint $0 \geq 1$, making both sides infeasible.

For other sign conventions, convert to standard first. A primal equality has a free dual multiplier; an unrestricted primal variable gives a dual equality. These rules follow by combining split nonnegative variables or opposite inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.5 The initial basic feasible solution

Negative entries in initial b prevent starting a ratio loop that assumes feasibility. For $Ax \leq b$, $x \geq 0$, add $a \geq 0$ and solve auxiliary $\max -a;\ Ax - a1 \leq b$. It is always feasible:take x=0, a=max(0,−min_i b_i). Its objective is at most 0 and cannot be upward-unbounded.

Original feasibility allows the same x with a=0, attaining auxiliary optimum 0. Conversely, auxiliary optimum 0 gives a=0 and an original feasible x. A strictly negative optimum proves infeasibility. A currently positive artificial variable alone is insufficient; first establish auxiliary optimality.

When m=0, use the origin directly. A positive c_j gives an unbounded coordinate ray; otherwise the origin is optimal. No minimum over an empty row set is needed.

A special pivot initializes auxiliary feasibility

Write $s_i = b_i - A_i x + a$ and choose r with minimum $b_r < 0$. Let a enter, s_r leave. The pivot coefficient is −1; this is **not** the improving ratio step from an already-feasible basis.

Solving gives $a = -b_r + A_r x + s_r$; other rows become $s_i = b_i - b_r + (A_r - A_i)x + s_r$. Set new nonbasic $x = s_r = 0$. Then $a = -b_r > 0$ and $s_i = b_i - b_r \geq 0$, establishing a feasible basis. Only now use simplex and lexicographic anti-cycling.

Start symbolic RHS perturbation at this feasible dictionary and take constant parts afterward. Reduced costs ignore RHS, so the result is the original auxiliary optimum and feasible point, rather than a false infeasibility judgment from an ε objective.

Zero artificial value still requires basis cleanup

At auxiliary optimum 0, delete a's column if it is nonbasic. If a is basic with value 0, its row is $a=-\Sigma a_{ajx_j}$. A nonzero coefficient permits that x_j to enter and a to leave. This zero-step algebraic pivot leaves the point unchanged; its coefficient may have either sign and it is not an improving ratio step.

If every coefficient is 0, the equation a=0 is redundant. Delete its row and a. General equality or multi-artificial implementations need this branch. Full independent slack rows can sometimes exclude it by rank, but never justify division by 0.

Finally substitute every currently basic variable appearing in the **original objective** to recover a nonbasic-only objective. Changing a title or retaining auxiliary−a does not restore the original problem. This completes Phase I; Phase II begins with that feasible basis and restored objective.

Complete Phase I and original-objective example

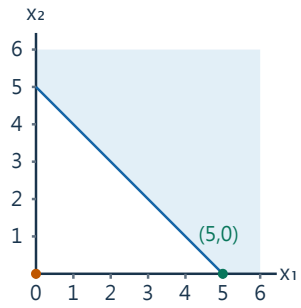
Original $\min\ z=2x_1+x_2;\ x_1+x_2\geq 5;\ x\geq 0$ uses surplus $x_3\geq 0$ with $x_1+x_2-x_3=5$. Add artificial $a=5-x_1-x_2+x_3$, minimizing w=a in Phase I, equivalent to maximizing−a.

State	Basic row	At nonbasics 0	Objective
Initial	$a=5-x_1-x_2+x_3$	a=5	w=5
x_1 enters, a leaves	$x_1=5-x_2+x_3-a$	$x_1=5$	w=a, min 0
Delete a; substitute original objective	$x_1=5-x_2+x_3$	$x_1=5$	$z=10-x_2+2x_3$
x_2 enters, x_1 leaves	$x_2=5-x_1+x_3$	$x_2=5$	$z=5+x_1+x_3$

Nonnegative final minimization coefficients prove optimum 5. Original origin(0,0) is infeasible; adding a=5 makes it feasible only for the auxiliary problem. This one-equation example uses one artificial column; the shared-a construction above covers arbitrary $Ax\leq b$ systems.

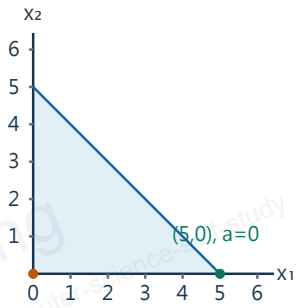
Complete Phase I and original-objective example (continued)

Original: $x_1+x_2\geq 5$



Origin infeasible

Auxiliary slice $x_3=0$: $a=5-x_1-x_2$



Origin with $a=5$ is feasible

Artificial variables change only the auxiliary problem. Setting $a=0$ restores original constraints.

Infeasibility needs a completed certificate

Original $x\leq 1$, $x\geq 2$, $x\geq 0$ becomes $x-a\leq 1$, $-x-a\leq -2$. Summing gives $-2a\leq -1$, so $a\geq 1/2$. Values $x=3/2$, $a=1/2$ satisfy both tightly, proving auxiliary max $-a=-1/2<0$ and original infeasibility.

The complete dictionary starts $s_1=1-x+a$, $s_2=-2+x+a$. Special a-entry, s_2-exit gives $a=2-x+s_2$, $s_1=3-2x+s_2$, $w=-2+x-s_2$. Enter x:ratios 2,3/2 make s_1 leave, producing

$x=3/2-s_1/2+s_2/2;\ a=1/2+s_1/2+s_2/2;\ w=-1/2-s_1/2-s_2/2.$

Both reduced costs are nonpositive, so this is truly optimal. A positive artificial amount alone would not suffice. The original inequalities also sum to $0\leq -1$, an independent contradiction certificate.

Three outcomes and the proof dependencies

Phase I starts from a feasible auxiliary basis. Lexicographic simplex terminates finitely, and auxiliary objective is bounded above. A negative optimum means infeasible; a zero optimum permits artificial cleanup and objective restoration.

Phase II also starts feasible and terminates under the lexicographic rule. No positive ratio denominator produces a feasible point plus improving ray, certifying unboundedness. Otherwise nonpositive terminal reduced costs produce a feasible basic point and same-value dual, certifying a finite attained optimum.

Thus every LP in this standard form is exactly one of infeasible, feasible with unbounded objective, or feasible with a finite attained optimum. Finite-dimensional structure and non-strict constraints matter. Problem $\max\ x;\ 0 \le x < 1$ has unattained supremum 1 and lies outside this trichotomy's assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 29.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms

Part VII Selected Topics

Divisibility, groups, and CRT support modular equations, powers, RSA, and randomized primality tests. A returned factor is a deterministic certificate. Pollard rho's time estimate needs an additional assumption. Prerequisites are divisibility, modular arithmetic, groups, and primes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover elementary number theory, greatest common divisors, modular arithmetic, modular linear equations, the Chinese remainder theorem, powers of an element, RSA, primality testing, and integer factorization.

The goals are to use Euclid and extended Euclid, solve modular equations and CRT systems, and distinguish RSA correctness, randomized testing guarantees, and factorization assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divisibility and input length

For integers d, a, $d \mid a$ means $a = kd$ for some integer k. A prime $p \geq 2$ has exactly the positive divisors 1 and p. The integer 1 is not prime. The gcd is nonnegative and positive unless both inputs are zero. Define $\gcd(0, 0) = 0$. Code first takes absolute values of negative inputs.

For $n \geq 1$, $a \equiv b \pmod{n}$ means $n \mid a - b$. The representative `a mod n` lies in $[0, n-1]$, even for negative inputs. For example, $-7 \bmod 5 = 3$. Congruences preserve sums, differences, and products because their differences remain multiples of n. Division needs additional conditions.

A β-bit integer can have value near 2^β. A loop up to $\sqrt{n}$ is exponential in input length. We distinguish arithmetic operation counts from bit costs. Schoolbook multiplication and long-division remainder on $O(\beta)$-bit integers each take $O(\beta^2)$ bit operations.

Bezout, cancellation, and factorization

For a, b not both zero, let g be the smallest positive integer of the form $ax + by$. Divide a by g. The remainder $a - qg$ is another integer linear combination. A positive remainder below g contradicts minimality, so $g \mid a$. Similarly $g \mid b$. Every common divisor divides g, proving $g = \gcd(a, b)$ and Bezout's identity $ax + by = g$.

If $\gcd(a, n) = 1$, write $au + nv = 1$. Multiplication by b gives $b = abu + nbv$, so $n \mid ab$ implies $n \mid b$. Coprimality permits cancellation. Modulo 6, $2 \cdot 1 \equiv 2 \cdot 4$, but $1 \not\equiv 4$.

If prime $p \nmid a$, then $\gcd(a, p) = 1$, so $p \mid ab$ implies $p \mid b$. Every $n \geq 2$ factors into primes by recursively factoring smaller divisors. Compare two factorizations. A prime in the first divides some prime in the second and therefore equals it. Cancel and induct to prove unique prime factorization.

Euclid's algorithm

For $a = qb + r$, $0 \le r < b$, a common divisor of a, b divides $r = a - qb$. A common divisor of b, r divides $a = qb + r$. Thus $\gcd(a, b) = \gcd(b, r)$.

```
euclid(a,b):
    a=abs(a); b=abs(b)
    while b != 0: (a,b)=(b,a mod b)
    return a
```

The second argument strictly decreases to zero. The current gcd equals the original gcd, and the final $\gcd(a, 0) = a$ proves correctness. If $0 < b \le a$, set $r = a \bmod b$. If $r \le b/2$, the second argument already halves. Otherwise the next remainder is $b \bmod r = b - r < b/2$. Every two divisions halve it, giving $O(\log b)$ divisions, or $O(\beta)$ for β-bit inputs.

Fibonacci chains and bit bounds

If $a > b \ge 1$ takes k divisions to reach zero, tracing backward gives each earlier remainder at least the sum of the next two. With $F_1 = F_2 = 1$, this yields $a \ge F_{k+2}, b \ge F_{k+1}$. Fibonacci numbers at least double every two positions, so $k = O(\log b)$. Consecutive Fibonacci inputs have quotient 1 except at the final division and attain the same order of chain length.

The coarse per-division bound $O(\beta^2)$ times $O(\beta)$ rounds gives a safe $O(\beta^3)$ bit bound. This is not optimal. Tracking shrinking operands gives the sharper classical $O(\beta^2)$ gcd bound, but we do not use that refinement to claim measured speed.

Keeping two remainders uses $O(\beta)$ bits. Saving every trace row has additional storage cost. Euclid needs no known prime factorization.

Extended Euclid

If recursion on (b, r) returns $g = bx' + ry'$, substituting $r = a - qb$ gives $g = ay' + b(x' - qy')$. Return $(x, y) = (y', x' - qy')$. The base case $b = 0$ returns $(g, x, y) = (a, 1, 0)$.

```
extended(a,b):                  # a,b >= 0
    if b == 0: return (a,1,0)
    (g,x1,y1)=extended(b,a mod b)
    return (g,y1,x1-floor(a/b)*y1)
```

For 252 and 198, the divisions are $252 = 198 + 54$, $198 = 3 \cdot 54 + 36$, $54 = 36 + 18$, $36 = 2 \cdot 18$. Back substitution gives $18 = 54 - 36 = 4 \cdot 54 - 198 = 4 \cdot 252 - 5 \cdot 198$. The gcd is 18 and coefficients are $(4, -5)$. Adjust corresponding coefficient signs for negative original inputs.

Extended Euclid (continued)

Euclid: every remainder is an integer linear combination

Remainder r	252 coefficient x	198 coefficient y	Check 252x+198y
252	1	0	252+(0)=252
198	0	1	0+(198)=198
54	1	-1	252+(-198)=54
36	-3	4	-756+(792)=36
18	4	-5	1008+(-990)=18

Quotients 1,3,1,2. Last nonzero remainder: 18 = 4×252 − 5×198.

Modular groups and units

A group operation is closed and associative, has an identity, and gives every element an inverse. An abelian group also has a commutative operation. Inverses are unique. If b, c are inverses of a, then $b = b(ac) = (ba)c = c$.

The additive group $(\mathbb{Z}_n, +)$ is finite and abelian, with identity 0 and inverse $-a$. Multiplication on all of $\mathbb{Z}_n$ is generally not a group, since 0 has no inverse. For $n \geq 2$, the units $U(n) = \{a \in [0, n-1] : \gcd(a, n) = 1\}$ form the multiplicative group.

Bezout gives $au + nv = 1$ when $\gcd(a, n) = 1$, so u is an inverse. Conversely, if $au \equiv 1$, every common divisor divides $au - nv = 1$, so the gcd is 1. Products of units remain units because $(ab)^{-1} = b^{-1}a^{-1}$. Division by a has a unique modular meaning precisely when a is a unit.

Lagrange's theorem through cosets

A subgroup $H \subseteq G$ contains the identity and is closed under products and inverses. A left coset gH has $|H|$ elements because multiplication by g is a bijection. If $gh_1 = kh_2$, then $k^{-1}g = h_2h_1^{-1} \in H$, so $gH = kH$. Otherwise the cosets are disjoint. They partition finite G, proving $|H| \mid |G|$. A proper subgroup has size at most $|G|/2$.

A nonempty product-closed subset of a finite group is already a subgroup. For each a, finiteness gives $a^i = a^j$, $i < j$, so $a^{j-i} = e$ belongs to the subset. If $j - i > 1$, a^{j-i-1} is an inverse in the subset. If $j - i = 1$, then $a = e$, whose inverse is itself.

The order of a is the least positive t with $a^t = e$. The elements $a^0, \ldots, a^{t-1}$ are distinct and powers repeat modulo t, so its generated subgroup has size t. Lagrange gives $t \mid |G|$, hence $a^{|G|} = e$. Finiteness matters. The additive group of integers has elements of infinite order.

Totients and distinct operations

Define $\varphi(n) = |U(n)|$. For prime p, the nonunits modulo p^α are exactly the $p^{\alpha-1}$ multiples of p. Hence $\varphi(p^\alpha) = p^{\alpha-1}(p-1)$. CRT below identifies units of a coprime product with the Cartesian product of component unit groups, giving $\varphi(n) = \prod_i p_i^{\alpha_i-1}(p_i-1)$.

For 15, $U(15) = \{1, 2, 4, 7, 8, 11, 13, 14\}$, so $\varphi(15) = 8$. Since $7 \cdot 13 \equiv 1$, the modular fraction $4/7$ means $4 \cdot 13 \equiv 7$, not integer quotient 0.

In additive $\mathbb{Z}_8$, 2 generates $\{0, 2, 4, 6\}$, with order 4. In multiplicative $U(7)$, 2 has powers 1,2,4 and order 3. The powers of 3 are 1,3,2,6,4,5 and its order is 6. The notation $\langle a \rangle$ depends on the operation.

All modular linear solutions

For $ax \equiv b \pmod{n}$, $n \geq 1$, set $g = \gcd(a, n)$. Necessarily $g \mid b$, since $ax + ny = b$. Extended Euclid gives $au + nv = g$. If $b = gh$, then $x_0 = uh$ is a solution, proving sufficiency.

Divide by g to get $(a/g)x \equiv b/g \pmod{n/g}$. Its coefficient is coprime to the new modulus, so it has one solution modulo n/g. Subtracting two solutions and cancelling gives $n/g \mid x - x_0$. All g distinct solutions modulo n are therefore

$$x = x_0 + k(n/g) \pmod{n}, \qquad k = 0, \ldots, g-1.$$

This proves existence, number, completeness, and an algorithm. If $a = 0$, then $g = n$. Every representative solves it when $n \mid b$, and none do otherwise. For $n = 1$, the only representative is 0. We do not assert general unit division for that modulus.

Equation example and output cost

For $18x \equiv 30 \pmod{42}$, $g = 6 = -2 \cdot 18 + 42$, so $x_0 = -10 \equiv 4 \pmod{7}$. The six solutions modulo 42 are 4,11,18,25,32,39. For example, $18 \cdot 4 \equiv 30$, and increasing x by 7 increases the product by $126 = 3 \cdot 42$. Replacing 30 with 31 gives no solution, since $6 \nmid 31$. The inverse of 18 modulo 42 does not exist.

The compressed triple $(x_0, n/g, g)$ avoids enumeration. Listing all solutions requires $\Omega(g)$ outputs. After normalizing $a \bmod n$, Euclid plus enumeration takes $O(\log n + g)$ arithmetic operations. Each output still has $O(\log n)$ bits, which must be counted.

Chinese remainder construction

For pairwise coprime $m_1, \ldots, m_k \geq 2$, let $M = \prod_i m_i$, $M_i = M/m_i$, $z_i = M_i^{-1} \pmod{m_i}$, $e_i = M_i z_i$, and $x = \sum_i a_i e_i \pmod{M}$.

For fixed j, every e_i with $i \neq j$ is zero modulo m_j, while $e_j \equiv 1$. Thus $x \equiv a_j \pmod{m_j}$, proving existence. If x, x' both solve the system, each m_i divides their difference. Pairwise coprimality and cancellation imply the product divides the difference, proving uniqueness modulo M.

The map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a bijection preserving addition and multiplication. A residue is a unit exactly when every component is a unit. Otherwise it shares a prime factor with some component modulus. This proves multiplicativity of the totient for coprime products.

CRT selectors reconstruct 29

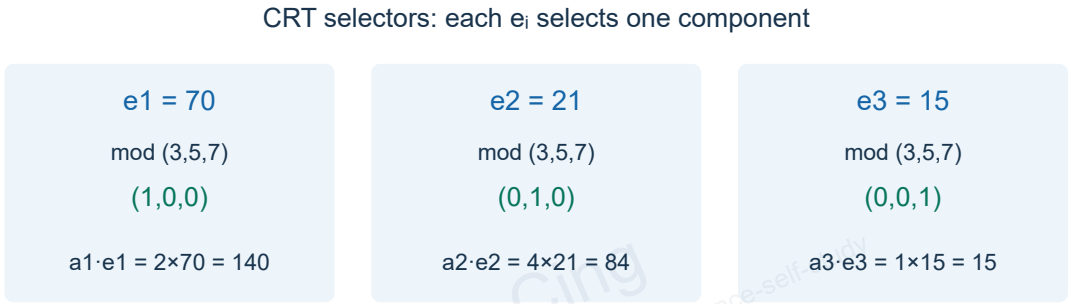
Solve $x \equiv 2 \pmod 3, x \equiv 4 \pmod 5, x \equiv 1 \pmod 7$. Here $M = 105$, $M_i = (35, 21, 15)$, inverses are $(2, 1, 1)$, and selectors are $(70, 21, 15)$.

$$x = 2 \cdot 70 + 4 \cdot 21 + 15 = 239 \equiv 29 \pmod{105}.$$

Indeed, 29 has residues 2,4,1. The selector coordinates are $(1, 0, 0), (0, 1, 0), (0, 0, 1)$. Modulo M, $e_i^2 = e_i$, $e_i e_j = 0$ for $i \neq j$, and $\sum_i e_i = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT selectors reconstruct 29 (continued)



Intermediate integers must not overflow. The bit length of M is of order $\sum_i \log m_i$. Use large integers and reduction as needed. The numerical value of M is not its memory cost in bits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Noncoprime compatibility

For $x \equiv a \pmod{m}, x \equiv b \pmod{n}$, let $g = \gcd(m, n)$. Substituting $x = a + mk$ gives $mk \equiv b - a \pmod{n}$. A solution exists exactly when $g \mid b - a$, and its period is $\operatorname{lcm}(m, n) = mn/g$, not mn.

For $x \equiv 2 \pmod{6}, x \equiv 5 \pmod{9}$, the difference 3 is divisible by $g = 3$. The equation $6k \equiv 3 \pmod{9}$ becomes $2k \equiv 1 \pmod{3}$, so $k \equiv 2$, giving $x \equiv 14 \pmod{18}$. Replacing 5 by 4 gives no solution.

Merge general systems one congruence at a time, checking compatibility and updating the lcm. The pairwise-coprime selector formula cannot be applied directly to 6 and 9, because the inverses do not exist.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Powers, orders, and logarithms

For $a \in U(n)$, Lagrange gives Euler's theorem $a^{\varphi(n)} \equiv 1 \pmod{n}$. For prime p, this becomes Fermat's theorem $a^{p-1} \equiv 1$. Every residue satisfies $a^p \equiv a \pmod{p}$, with the zero case holding directly.

An element g of order $\varphi(n)$ is a primitive root and generates all units. Each unit equals g^k for a unique $k \pmod{\varphi(n)}$, its discrete logarithm $\operatorname{ind}_{n,g}(a)$. By the order definition, $g^x = g^y$ exactly when $x \equiv y \pmod{\varphi(n)}$.

Modulo 7, base 3 maps exponents 0,1,2,3,4,5 to 1,3,2,6,4,5, so $\operatorname{ind}(4) = 4$. Base 2 reaches only 1,2,4 and has no logarithm for 3. A nongenerator need not represent every unit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite fields have multiplicative generators

In a finite abelian group, elements of coprime orders r, s have a product of order rs. If $(ab)^k = 1$, then $a^k = b^{-k}$ lies in both generated subgroups. Their intersection has order dividing both r, s, so it contains only the identity. Thus $r \mid k, s \mid k$. Conversely the rsth power is 1.

Let L be the lcm of element orders in $\mathbb{F}_p^\times$. For each prime ℓ, some element order supplies the highest power ℓ^v in L. A suitable power of that element has order exactly ℓ^v. Multiplying the elements for distinct primes gives an element of order L.

All $p-1$ nonzero elements are roots of $X^L - 1$. A nonzero degree-L polynomial over a field has at most L roots, by repeated factorization, so $p - 1 \leq L$. Lagrange makes each order divide $p-1$, giving $L \leq p-1$. Hence $L = p-1$, and a generator exists. Arbitrary finite groups need not be cyclic.

Generators for odd prime powers

For odd prime p, take a generator g modulo p. Modulo p^2,

$$(g+p)^{p-1} \equiv g^{p-1} + (p-1)g^{p-2}p.$$

The second term divided by p is nonzero modulo p, so at least one $h \in \{g, g+p\}$ satisfies $h^{p-1} = 1 + cp$, $p \nmid c$.

If $v_p(z-1) = j \geq 1$, the binomial expansion gives $v_p(z^p - 1) = j + 1$. The first term is $p(z-1)$. All later terms have at least one more factor of p, including the last because $p \geq 3$, so they cannot cancel the first. Thus $v_p(h^{(p-1)p^k} - 1) = k + 1$.

Reduction modulo p makes the order modulo p^α divisible by $p-1$. Lagrange makes it divide $p^{\alpha-1}(p-1)$. The valuation excludes every smaller power of p, so the order equals the totient. Hence $U(p^\alpha)$ is cyclic. Reusing a generator modulo p unchanged requires the p^2 check.

Exactly which unit groups are cyclic

For $n > 1$, the possibilities are exactly $2, 4, p^\alpha, 2p^\alpha$, with p odd prime. The previous proof covers odd prime powers. CRT gives $U(2p^\alpha) \cong U(2) \times U(p^\alpha)$, whose first component is trivial. The cases 2 and 4 are direct.

If $8 \mid n$, reduction maps $U(n)$ onto $U(8)$. Every element of $U(8)$ squares to 1, but the group has four elements and is not cyclic. A cyclic group's homomorphic image must be cyclic, a contradiction. Surjectivity follows by choosing an odd residue in the power-of-two component and unit residues in the odd prime-power components via CRT.

If n has two distinct odd prime factors, independent CRT signs give at least four units squaring to 1. If $4 \mid n$ and an odd prime divides n, its two components also give four roots. A cyclic group has at most two solutions to $g^{2k} = 1$ modulo its order. These cases exclude all remaining forms.

Nontrivial square roots expose factors

For an odd prime power p^α, $x^2 \equiv 1$ implies $p^\alpha \mid (x-1)(x+1)$. The gcd of the two factors divides 2, so the odd prime p cannot divide both. One factor contains all of p^α, giving $x \equiv \pm 1$.

If $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv \pm 1$, then n is composite and both $\gcd(x-1, n)$ and $\gcd(x+1, n)$ are proper nontrivial factors. Neither equals n, or x would equal a sign. Neither equals 1, since cancellation would force the other factor to be divisible by n.

For $n = 77$, $x = 34$, $34^2 = 1156 \equiv 1$, yet $34 \neq 1, 76$. The gcds are 11 and 7. CRT interprets 34 as +1 modulo 11 and -1 modulo 7, with unsynchronized signs.

Square and multiply

For $n \geq 1, b \geq 0$, normalize $a \bmod n$ and initialize $r = 1 \bmod n$. Read every exponent bit from highest to lowest.

```
r = 1 mod n
for bit in binary(b), highest first:     # empty string for b=0
    r = r*r mod n
    if bit == 1: r = r*a mod n
return r
```

After reading prefix v, the invariant is $r \equiv a^v \pmod{n}$. The next bit δ changes it to $2v + \delta$, exactly the effect of squaring and optionally multiplying by a. The empty prefix has value 0. The final prefix is b, proving correctness. Exponent 0 returns $1 \bmod n$, including 0 when $n = 1$. Coprimality is unnecessary. Negative exponents need an inverse and a separate input contract.

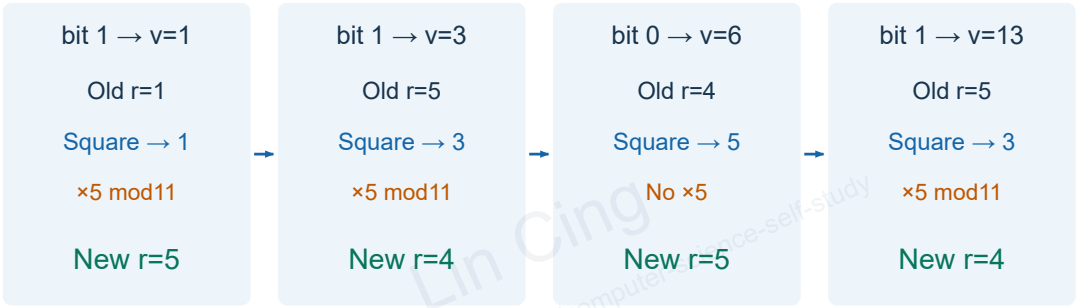
Every bit of 5 to the thirteenth

The binary representation of 13 is 1101. Starting with $r = 1$, scan all four bits modulo 11.

Bit	New prefix	Old r	Square modulo 11	Multiply by 5 or retain
1	1	1	1	5
1	3	5	3	4
0	6	4	5	5
1	13	5	3	4

Every bit of 5 to the thirteenth (continued)

$5^{13} \bmod 11$: scan all bits 1101, starting at r=1



Invariant $r=5^{v} \bmod 11$. Prefixes 1 → 3 → 6 → 13; return 4.

Initializing $r = a$ means the leading 1 was already read, so only 101 remains. Starting at a but scanning all of 1101 instead follows exponents 3,7,14,29 and values 4,3,9,9, incorrectly returning 9. A k-bit exponent uses at most $2k$ modular multiplications. With a β-bit modulus, ordinary bit cost is $O(k\beta^2)$, or $O(\beta^3)$ when $k = O(\beta)$. Initial reduction of an exceptionally long base has additional cost.

RSA on every residue

Choose distinct odd primes p, q, set $n = pq, \varphi = (p-1)(q-1)$, and choose $1 < e < \varphi$ with $\gcd(e, \varphi) = 1$. Extended Euclid gives $d = e^{-1} \pmod{\varphi}$, $1 \le d < \varphi$. Publish (n, e), retain d, p, q. For $0 \le m < n$, encrypt $c = m^e \bmod n$ and decrypt $m' = c^d \bmod n$.

Write $ed = 1 + k(p-1)(q-1)$. For any m, modulo p, if $p \mid m$, both m^{ed} and m are zero. Otherwise Fermat gives $m^{p-1} = 1$, so $m^{ed} = m$. The same applies modulo q. CRT uniqueness proves $m^{ed} \equiv m \pmod{n}$, including nonunits. Euler's theorem alone would omit those legal messages.

The private and public powers also undo each other in the opposite order, so they are inverse mathematical permutations. This correctness is separate from protocol security.

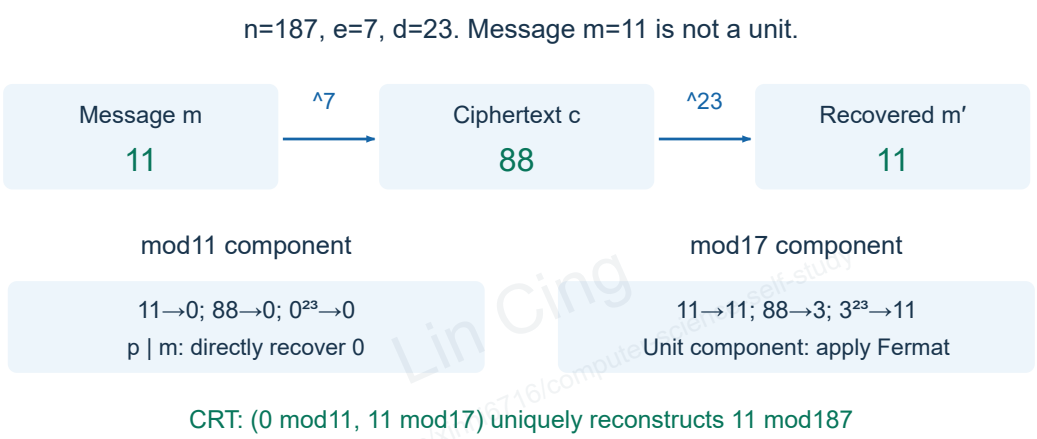
A nonunit RSA message

Take $p = 11, q = 17, n = 187, \varphi = 160, e = 7$. Since $1 = 23 \cdot 7 - 160$, $d = 23$. The message $m = 11$ is a legal nonunit.

Encryption uses $11^2 \equiv 121, 11^4 \equiv 55$, and $11^7 \equiv 55 \cdot 121 \cdot 11 \equiv 88 \pmod{187}$. Decryption uses $88^2 \equiv 77, 88^4 \equiv 132, 88^8 \equiv 33, 88^{16} \equiv 154$. Since $23 = 16 + 4 + 2 + 1$, multiply the corresponding values. The reductions are $154 \cdot 132 \equiv 132$, $132 \cdot 77 \equiv 66$, and $66 \cdot 88 \equiv 11$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A nonunit RSA message (continued)



A fixed-bit-length public exponent costs $O(\beta^2)$ with ordinary arithmetic. A private exponent of $O(\beta)$ bits costs $O(\beta^3)$. Factoring n reveals p, q, hence φ, d. This does not prove the converse claim that RSA inversion is equivalent to factoring.

Algebraic limits of textbook RSA

Without encoding, RSA is deterministic. Equal messages give equal ciphertexts, so a small message space can be enumerated. Also $E(m_1)E(m_2) \equiv E(m_1 m_2)$. Given $c = E(m)$ and $r \in U(n)$, the ciphertext cr^e represents mr. This is malleability, not a security proof.

Using m^d directly as a signature also lacks a complete security construction. Choose any σ, then set $m = \sigma^e$. This creates a pair that passes verification without an authorized signature on a meaningful message. Practical schemes separately specify randomized encodings, hashes, and verification formats. This chapter derives the permutation rather than deployment parameters.

These algebraic weaknesses can coexist with hard factorization of large integers. Decryption correctness cannot replace an attack model and security definition.

Primality tests

Trial division needs divisors only through $\lfloor \sqrt{n} \rfloor$. If $n = ab$ and both factors exceed $\sqrt{n}$, their product exceeds n. Absence of a small factor therefore proves primality. For β-bit n, $\sqrt{n}$ is about $2^{\beta/2}$, not polynomial in bit length.

The prime number theorem gives $\pi(X) \sim X / \ln X$. Uniform sampling in a comparable-magnitude interval has approximate prime density $1/\ln X$, or $2/\ln X$ among odd candidates. This asymptotic estimate does not guarantee a prime within $\ln X$ trials or prove the test's error bound.

A Fermat test finding $a^{n-1} \not\equiv 1$ proves compositeness. Equality gives only a probable prime. For $341 = 11 \cdot 31$, $2^{10} = 1024 \equiv 1$, so $2^{340} = 1$ despite compositeness. Carmichael numbers let every unit pass. For $561 = 3 \cdot 11 \cdot 17$, all of 2,10,16 divide 560, so Fermat and CRT give $a^{560} = 1$ for every unit.

Miller-Rabin boundaries and square chains

Handle $n < 2$ as nonprime, $n = 2, 3$ as prime, and all other even n as composite. For odd $n \geq 5$, write $n - 1 = 2^s d$, d odd, $s \geq 1$. Choose a uniformly among integers in $[2, n - 2]$.

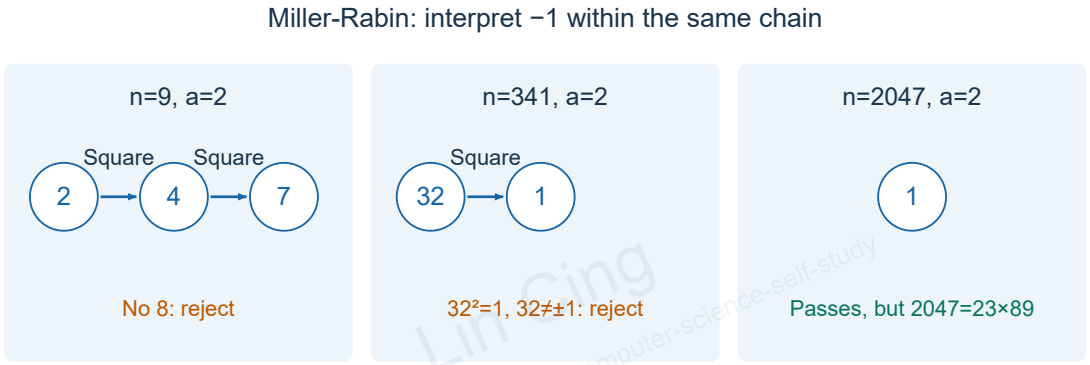
```
x = modular_power(a,d,n)
if x == 1 or x == n-1: pass this round
else:
    repeat s-1 times:
        x = x*x mod n
        if x == n-1: pass this round
    if no -1 occurred: return COMPOSITE
```

Repeat with independent bases and return probably prime only if every round passes. For prime n, Fermat gives $a^{n-1} = 1$. If the chain does not start at 1, the value before its first 1 must be -1, since a field has only square roots ± 1 of 1. Thus primes always pass, and every rejection proves compositeness. Without the special case for 3, sampling would use the empty interval $[2, 1]$.

Strong testing separates Fermat examples

n, a	Decomposition	Initial power	Later squares	Outcome
5,2	$4 = 2^2 \cdot 1$	2	$4 = -1$	Pass
9,2	$8 = 2^3 \cdot 1$	2	4,7	Reject, no -1
341,2	$340 = 2^2 \cdot 85$	32	1	Reject, nontrivial root 32
2047,2	$2046 = 2 \cdot 1023$	1	None needed	Pass, but $2047 = 23 \cdot 89$

Strong testing separates Fermat examples (continued)



Passing means probable prime. Repeating a fixed base does not amplify confidence.

A fixed single base cannot prove primality for general integers. The error is acceptance of a composite. Primes are never rejected. The proof below bounds random-base acceptance for each fixed composite.

Count roots in prime-power components

Fix odd composite $n = \prod_{i=1}^{r} p_i^{\alpha_i}$, with $n - 1 = 2^s d$. Write $p_i - 1 = 2^{t_i} u_i$, with odd u_i. Let $D_i = \gcd(d, u_i)$, $\tau = \min(s, t_1, \ldots, t_r)$. A passing base must be a unit. If $p_i \mid a$, every positive power in that component is zero, never either sign.

The group $\mathbb{F}_{p_i}^{\times}$ has a generator. Write $a = g^k$. The equation $a^d = 1$ has D_i roots. The equation $a^{2^j d} = -1$ becomes a linear congruence in k. The coefficient gcd is $2^{\min(j,t_i)} D_i$, which divides $(p_i - 1)/2$ exactly when $j < t_i$. In that case there are $2^j D_i$ roots.

Each root lifts uniquely to $p_i^{\alpha_i}$. If z solves $X^h - \eta = 0$ modulo p_i^a, try $z + cp_i^a$. At the next level solve

$$(z^h - \eta)/p_i^a + hz^{h-1}c \equiv 0 \pmod{p_i}.$$

For $h = 2^j d$, $p_i \nmid h$ because $p_i \nmid n - 1$. Also z is a unit. The derivative is nonzero, so c is unique. The root counts therefore hold at every prime-power component.

CRT counts synchronized strong liars

Passing cases are disjoint. Either $a^d = 1$, or a unique $j \in [0, s-1]$ satisfies $a^{2^j d} = -1$. Two levels cannot both be -1, because squaring the first gives 1 thereafter. CRT makes components independent, but every component must reach -1 at the **same level**.

Thus the exact strong-liar count in $[1, n-1]$ is

$$L = \left(1 + \sum_{j=0}^{\tau-1} 2^{jr}\right) \prod_i D_i.$$

The first term counts initial 1 in every component. The term for j counts synchronized -1 at that level. Combining components that reach -1 at different levels overcounts bases that the test rejects. Factorization is used for analysis. The algorithm does not need the factors.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

One-quarter bound with one or at least three factors

If $r = 1$, then $n = p^\alpha, \alpha \geq 2$. Since $p - 1 \mid p^\alpha - 1$, $\tau = t, D = u$, so $L = p - 1$. Therefore

$$\frac{L}{n-1} = \frac{p-1}{p^\alpha - 1} \leq \frac{1}{p+1} \leq \frac{1}{4}.$$

If $r \geq 3$, use $\varphi(n) = \prod_i p_i^{\alpha_i - 1} 2^{t_i} u_i$, $D_i \leq u_i$, and $t_i \geq \tau$ to get

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 2^{jr}}{2^{r\tau}} = \frac{1}{2^r - 1} + \frac{2^r - 2}{(2^r - 1)2^{r\tau}}.$$

This decreases for $\tau \geq 1$. Its maximum at 1 is $2^{1-r} \leq 1/4$. Since $\varphi(n) \leq n - 1$, $L \leq (n-1)/4$. For $n = 9$, $L = 2 = (n-1)/4$, so this constant cannot be lowered on the full representative domain.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The necessary two-factor case

For $r = 2$, if $t_1 \neq t_2$ or $s < \min(t_1, t_2)$, then $t_1 + t_2 \geq 2\tau + 1$. Thus

$$\frac{L}{\varphi(n)} \leq \frac{1 + \sum_{j=0}^{\tau-1} 4^j}{2^{2\tau+1}} = \frac{4^\tau + 2}{6 \cdot 4^\tau} \leq \frac{1}{4}.$$

Otherwise $t_1 = t_2 = \tau \leq s$. If some $\alpha_i \geq 2$, the extra prime-power factor in the totient is at least 3. If some $D_i < u_i$, the odd integer u_i/D_i is at least 3. At least one occurs. Otherwise $n = pq$ is squarefree and $D_i = u_i$, so both $p - 1, q - 1$ divide $n - 1$. Reducing $pq - 1$ modulo $p - 1$ gives $p - 1 \mid q - 1$. The reverse gives $q - 1 \mid p - 1$, forcing $p = q$, a contradiction.

The extra divisor of at least 3 gives $L/\varphi(n) \leq (4^\tau + 2)/(9 \cdot 4^\tau) \leq 1/6 < 1/4$. Every odd composite case is covered without an unspecified subgroup or Carmichael exception.

Sampling and posterior probability

Bases 1 and $n - 1$ always pass. Removing them from $L \leq (n - 1)/4$ gives $(L - 2)/(n - 3) \leq 1/4$ on $[2, n - 2]$. For each **fixed composite** n, R independent uniform rounds therefore accept with probability at most 4^{-R}. This is not directly the probability that an accepted candidate is composite.

If the candidate distribution has prior prime probability $\pi > 0$, prime pass probability is 1 and composite acceptance is at most $\epsilon = 4^{-R}$. Bayes gives

$$\Pr(\text{composite} \mid \text{pass}) \leq \frac{\epsilon(1 - \pi)}{\pi + \epsilon(1 - \pi)}.$$

The candidate distribution must supply π. An adversarial fixed input cannot be assigned a uniform prime density without evidence.

Each round uses $O(\log n)$ modular multiplications and ordinary bit cost $O(\beta^3)$. R rounds cost $O(R\beta^3)$, plus sampling. A fixed base list is deterministic only on a proved input range. Reusing one randomly selected base does not give independence. Independent sampling with replacement may happen to repeat a value and still satisfies the contract.

Factorization and Pollard rho

Proving compositeness need not reveal factors. Trial division finds them deterministically but can require $\sqrt{n}$ trials. Pollard rho seeks a collision modulo an unknown factor.

For $f(x) = x^2 + c \bmod n$ and $p \mid n$, reducing $f(x)$ modulo p equals applying the same polynomial to $x \bmod p$. If $x_i \equiv x_j \pmod{p}$, then $p \mid x_i - x_j$, so $d = \gcd(|x_i - x_j|, n) > 1$. If $x_i \not\equiv x_j \pmod{n}$, then $d < n$, yielding a proper factor.

This implication is deterministic. Verify a returned value by $1 < d < n, d \mid n$. The uncertain parts are the collision time and whether all components collide together. The latter gives $d = n$, which is not successful factorization.

Floyd and checkpoint schedules

```
x=y=x0
repeat until work budget exhausted:
    x=f(x); y=f(f(y))
    d=gcd(abs(x-y),n)
    if 1<d<n: return d
    if d==n: restart with another x0 or c
return no factor found within budget
```

Floyd advances the slow pointer by one and the fast pointer by two. A finite functional graph has tail length μ and cycle length λ. Once both are on the cycle, their relative position changes by one per round, so they meet within λ more rounds. Cost is $O(\mu + \lambda)$ function evaluations and a constant number of stored integers.

Another schedule fixes a checkpoint $y = x_{2^k}$ while x advances to the next power of two. Once the checkpoint is on the cycle and the interval is at least λ, the interval contains a matching component. This is a different schedule, so its trace must not be mixed with Floyd's. Both need a budget and a rule for $d = n$.

Three rounds factor 8051

Use $n = 8051, x_0 = 2, f(x) = x^2 + 1 \bmod 8051$, with Floyd's schedule.

Round	x	y	Absolute difference	gcd with 8051
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

In round 3, 677 and 871 both have residue 95 modulo 97, while their residues modulo 83 are 13 and 41. The collision yields 97 rather than 8051. Division gives 83. Trial division through their square roots confirms both are prime, completing $8051 = 83 \cdot 97$.

Three rounds factor 8051 (continued)

Floyd round 3: x=677, y=871, difference 194

mod97: collision

677 mod97 95

871 mod97 95

mod83: no collision yet

677 mod83 13

871 mod83 41

97 | 194, but 83 ∤ 194, so gcd(194,8051)=97

Check 8051 = 83×97. Success requires a proper divisor.

Birthday estimates and their assumptions

If iteration modulo unknown prime factor p behaves approximately as a uniform random mapping, it explores new points until the first collision. The probability of no collision in k steps is approximately

$$\prod_{j=0}^{k-1}(1 - j/p) \leq \exp(-k(k-1)/(2p)).$$

Typical collision scale is $\sqrt{p}$, and tail-cycle methods detect it in the same order of steps. A fixed polynomial $x^2 + c$ is not independent randomness. This assumption gives neither a worst-case time for that polynomial nor a success probability for each restart.

For balanced semiprimes, p is about $\sqrt{n}$, giving heuristic scale $n^{1/4} = 2^{\beta/4}$, still not polynomial in β. Every round also pays for a β-bit modular square and gcd. Using this chapter's safe coarse gcd bound gives $O(\beta^3)$, rather than constant cost.

Birthday estimates and their assumptions (continued)

After finding a factor, recursively factor both sides. Remove even factors first and specify the confidence of prime or probable-prime leaves. Exhausting a budget means no factor was found within it, not that the input is prime. The factor certificate is certain. Completion-time estimates remain conditional on the heuristic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 31.

M. O. Rabin, "Probabilistic Algorithm for Testing Primality," *Journal of Number Theory* 12(1), 1980, doi:10.1016/0022-314X(80)90084-0.

L. Monier, "Evaluation and Comparison of Two Efficient Probabilistic Primality Testing Algorithms," *Theoretical Computer Science* 12(1), 1980, doi:10.1016/0304-3975(80)90007-9.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String matching

Part VII Selected Topics

Matching reuses known equal prefixes and suffixes as state. We derive finite automata and KMP from character comparison and modular hashing, proving the failure chain, correctness, and total linear cost. Prerequisites are pattern prefixes, hashing, and finite automata.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover naive matching, Rabin-Karp, finite-automaton matching, and Knuth-Morris-Pratt. The goals are to define valid shifts and borders, compare these four methods, and prove the prefix function and KMP time bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Indices, shifts, and boundary cases

Text T has length n, pattern P has length m, and characters belong to finite alphabet Σ. Equality costs $O(1)$. Character indices start at 1, but shifts start at 0. A valid shift satisfies $0 \le s \le n - m$ and $P[j] = T[s + j]$ for **every** $j = 1, \ldots, m$. Output all shifts, including overlaps.

Write $P_k = P[1..k]$, with $P_0 = \epsilon$. A proper prefix or suffix cannot equal the whole string. A border is both a prefix and a suffix.

For $m = 0$, the specification outputs all $n + 1$ boundaries $0, \ldots, n$. For $m > n$, output nothing. Handle these before the main algorithms, which assume $1 \le m \le n$, so they never read a nonexistent $P[1]$.

Naive matching checks every shift

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

Before each inner comparison, $P[1..j-1] = T[s+1..s+j-1]$. Equality extends the invariant. A mismatch invalidates this shift. Reaching $j = m + 1$ means every character agrees. The outer loop enumerates exactly the possible shifts, proving both soundness and completeness.

There are $n - m + 1$ windows and at most m comparisons each. The worst case is $\Theta((n - m + 1)m)$, attained by $T = a^n, P = a^m$. Many true matches alone can force this cost. Extra workspace is $O(1)$, excluding output.

All windows of one text

For $T = \texttt{ababaca}$, $P = \texttt{aba}$, stop at the first mismatch.

Shift	Window	Comparisons in order	Count	Result
0	aba	a=a, b=b, a=a	3	Output 0
1	bab	a≠b	1	Reject
2	aba	a=a, b=b, a=a	3	Output 2
3	bac	a≠b	1	Reject
4	aca	a=a, b≠c	2	Reject

There are ten comparisons. Matches at shifts 0 and 2 share text position 3 and are both valid. Naive matching rechecks alignments. KMP uses known equal sections to rule out impossible shifts.

Rabin-Karp rolling identity

Encode characters injectively as $0, \ldots, d-1$. For a length-m string, $H(X) = \sum_{j=1}^{m} X[j] d^{m-j}$. Arithmetic on an arbitrary m-digit integer is not constant time. Store $h_q(X) = H(X) \bmod q$, $q \geq 2$, and verify characters whenever hashes agree.

Let $p = h_q(P), t_s = h_q(T[s+1..s+m]), h = d^{m-1} \bmod q$. Removing the highest digit, shifting, and adding a digit gives

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

The identity holds for every $q \geq 2$. Primality helps some randomized analyses but does not by itself give collision probability $1/q$ for fixed strings. Normalize negative remainders to $[0, q-1]$ and prevent overflow in intermediate products.

Hashes reject; characters accept

```
h=1; p=0; t=0
for j=1..m-1: h=d*h mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner evaluation and the rolling identity inductively maintain the current window hash. Equal strings have equal hashes, so no match is lost. Different hashes reject. Equal hashes trigger exact comparison, preventing false reports. Collisions affect running time rather than correctness in this verified version. Omitting verification creates a different algorithm that can report false matches.

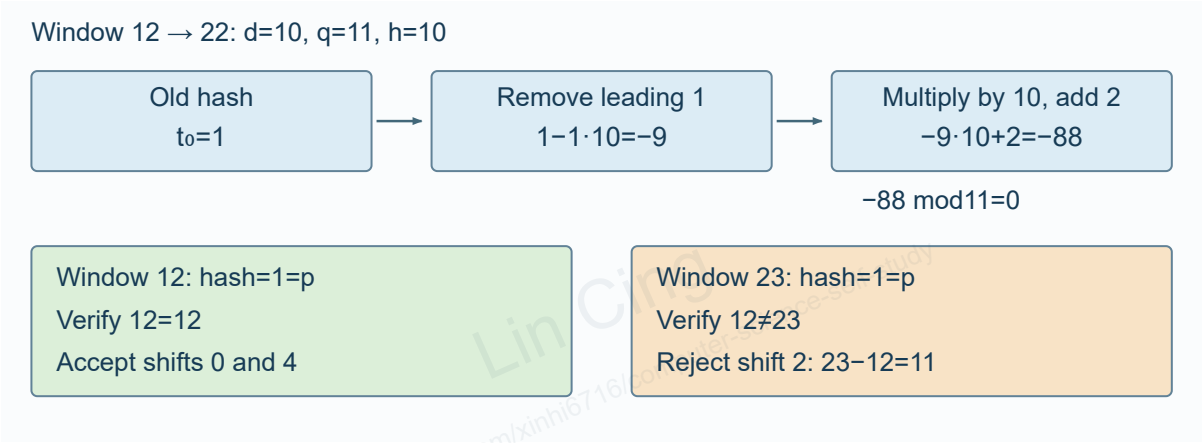
Match, collision, match

Take $T = 122312, P = 12, d = 10, q = 11$. Then $h = 10, p = 1$.

Shift	Window	Hash	Next hash calculation	Verification
0	12	1	$(10(1 - 1 \cdot 10) + 2) \bmod 11 = 0$	Match, output 0
1	22	0	$(10(0 - 2 \cdot 10) + 3) \bmod 11 = 1$	Different hash, skip
2	23	1	$(10(1 - 2 \cdot 10) + 1) \bmod 11 = 9$	$12 \neq 23$, reject collision
3	31	9	$(10(9 - 3 \cdot 10) + 2) \bmod 11 = 1$	Different hash, skip
4	12	1	Final window, read no new character	Match, output 4

A spurious hit has equal hashes but different strings. Here $23 - 12 = 11$ vanishes modulo the prime modulus. Prime moduli still permit collisions.

Match, collision, match (continued)



True and spurious hits both cost time

In the word-RAM model, characters, q, and modular products fit in constant-cost word operations, or constant-cost modular multiplication is available. The intermediate $d(t - \text{out} \cdot h)$ must also fit. Normalizing the difference before multiplication keeps that product below dq. Arbitrary-precision implementations must include bit costs.

Preprocessing takes $\Theta(m)$, and rolling takes $\Theta(n - m + 1)$. With z true matches and f spurious hits, verification takes at most m comparisons per hit, giving $O(n + m + m(z + f))$.

Either z or f can be $\Theta(n - m + 1)$, so worst-case time remains $\Theta((n - m + 1)m)$. All-a inputs have only true hits and still repeat full verification. Output alone costs $\Omega(z)$, while this version spends another m comparisons per true hit.

A proved random-base hash bound

Fix prime $q > d - 1$, so distinct character codes remain distinct in $\mathbb{F}_q$. **After fixing the input**, choose a uniform base $b \in \mathbb{F}_q$, independently of the input. Hash with $\sum_j X[j] b^{m-j} \bmod q$. Replace d with b in the rolling identity. The identity also handles $b = 0$, with $h = b^0 = 1$ when $m = 1$.

For fixed distinct strings X, Y, their difference polynomial is nonzero and has degree at most $m - 1$. A nonzero field polynomial has at most its degree many roots. At a root, factor out its linear factor and induct on degree. Thus

$$\Pr[h_b(X) = h_b(Y)] \le \min(1, (m-1)/q).$$

Sum the expectations of collision indicators over nonmatching windows. Independence between windows is unnecessary, and $E[f] \le (n - m + 1 - z)(m - 1)/q$. A loose expected-time bound is $O(n + m + mz + mnm/q)$. A large enough modulus reduces spurious-hit cost but not the mz cost of true matches. Its bit length and modular arithmetic costs must still be counted.

Finite-automaton state

A finite automaton consists of $(Q, q_0, A, \Sigma, \delta)$, a finite state set, initial state, accepting set, alphabet, and transition function for every state-character pair. For matching use $Q = \{0, \dots, m\}$, $q_0 = 0$, $A = \{m\}$.

Define $\sigma(x) = \max\{k \le m : P_k \text{ is a suffix of } x\}$. The empty string is always a suffix, so the maximum exists. The state $q = \sigma(x)$ is the longest pattern prefix currently available for extension, rather than the number of completed matches.

Set $\delta(q, a) = \sigma(P_q a)$. Read one character and take one transition. State m reports the shift ending at that position. Accepting states still have outgoing transitions, allowing overlapping matches.

Why the longest prefix is sufficient

We prove $\delta(\sigma(x), a) = \sigma(xa)$. Let $q = \sigma(x), r = \sigma(xa)$. If $r > 0$, remove the last character from its match. Then P_{r-1} is a suffix of x, so $r - 1 \leq q$. Since both are suffixes of x, the shorter P_{r-1} is a suffix of P_q. Appending a makes P_r a suffix of $P_q a$, hence $r \leq \sigma(P_q a)$. This inequality also holds for $r = 0$.

Conversely, $P_q a$ is a suffix of xa. Every suffix of $P_q a$ that is a pattern prefix is therefore a suffix of xa. Thus $\sigma(P_q a) \leq \sigma(xa) = r$, proving equality.

Starting with $\sigma(\epsilon) = 0$, induction gives $q = \sigma(T[1..i])$ after each transition. State m holds exactly when P is a suffix of that text prefix, equivalent to a match at shift $i - m$. This proves both directions of acceptance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete transition table and overlap

For $P = \texttt{abab}, \Sigma = \{a, b, c\}$:

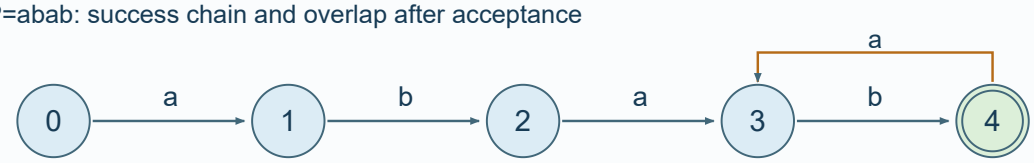
q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

Scanning `ababab` gives states 1,2,3,4,3,4, reporting shifts 0 and 2 at positions 4 and 6. The transition $\delta(4, a) = 3$ uses suffix `aba` of `ababa`. Resetting to zero after a match would lose the second occurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete transition table and overlap (continued)

P=abab: success chain and overlap after acceptance



The full transition table is in the notes. Only the success chain and δ(4,a)=3 are shown.

T[i]	a	b	a	b	a	b
State	1	2	3	4	3	4

Accept at positions 4 and 6: shifts 0 and 2. Do not reset after a match.

Table construction versus scanning

A direct construction tries candidate suffix lengths from $\min(m, q+1)$ downward for each q, a, comparing characters. It can take $O(m^2)$ per cell and $O(m^3|\Sigma|)$ overall. It is correct but slow. Scanning is still $\Theta(n)$.

Using the prefix function π below, construct in increasing q:

```
for a in Sigma: delta(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Sigma:
        if q<m and a==P[q+1]: delta(q,a)=q+1
        else: delta(q,a)=delta(pi[q],a)
```

After mismatch, the next candidate is a proper border of P_q. The longest has length $\pi[q]$, and all shorter candidates are suffixes of it. Reuse its transition on the same character. Since $\pi[q] < q$, dependencies are already computed. Computing π takes $\Theta(m)$. The table takes $\Theta(m|\Sigma|)$ time and space. Constant-time character lookup assumes constant-time alphabet indexing.

The prefix function

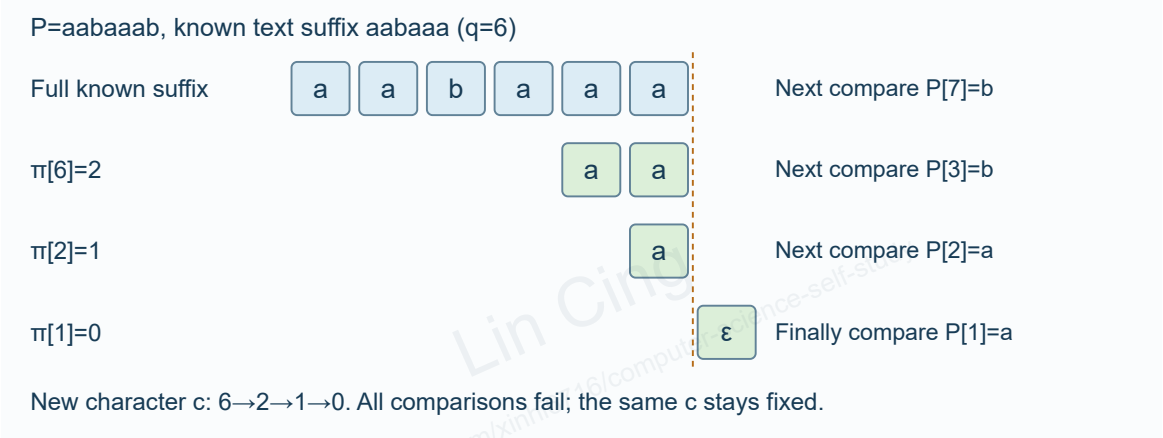
Define $\pi[q] = \max\{k < q : P_k \text{ is a suffix of } P_q\}$, for $q = 1, \ldots, m$. Thus $\pi[1] = 0$ and $\pi[q] < q$. It depends only on the pattern.

For aabaaab, $\pi = [0, 1, 0, 1, 2, 2, 3]$. The proper borders of $P_6 =$ **aabaaa** are aa, a, and the empty string. The chain is $6 \to 2 \to 1 \to 0$. Length 6 is the starting state, not a proper border.

KMP stores no full transition table. On a mismatch, follow π to a shorter candidate while keeping the new text character fixed. Previously read text need not be read again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The prefix function (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The failure chain contains every candidate

Let $B(q)$ be all proper border lengths of P_q, and let $b = \pi[q] > 0$. For every $k < b$,

$$k \in B(q) \quad \iff \quad k \in B(b).$$

Since P_b is a suffix of P_q, their length-k suffixes coincide. Their length-k prefixes also coincide with P_k. Thus the prefix-suffix equality is identical in both directions. No border has $b < k < q$, by maximality. If $b = 0$, no positive proper border exists.

Hence $B(q) = \{b\} \cup B(b)$. Induction on strictly decreasing lengths proves that $\pi[q], \pi[\pi[q]], \ldots, 0$ lists every proper border. This completeness justifies fallback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Construct the prefix function

```
pi[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=pi[k]
    if P[k+1]==P[i]: k++
    pi[i]=k
```

Before processing i, $k = \pi[i-1]$. If P_i has a positive proper border of length r, deleting its last character leaves a proper border of P_{i-1} of length $r-1$. Conversely, a border of that length extends when $P[r] = P[i]$.

Start at the largest candidate $\pi[i-1]$ and follow the complete decreasing failure chain. The first extendible candidate gives the largest legal r. At zero, unequal characters give answer zero. Only smaller, already-correct prefix values are used, proving induction. Since $\pi[i-1] \le i-2$, the entire P_i cannot be mistaken for its own proper border.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every prefix-table step

For `aabaaab`:

i	Character	Initial k	Comparison and fallback	pi[i]
1	a	None	A single character has no nonempty proper border	0
2	a	0	P[1]=a, extend	1
3	b	1	P[2]=a≠b; 1→0; P[1] also differs	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a; 2→1; P[2]=a	2
7	b	2	P[3]=b	3

The value $\pi[6] = 2$, not 3, because suffix `aaa` differs from prefix `aab`. Fallback keeps known possible borders rather than always subtracting one.

KMP scan and the state after a match

```
pi=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=pi[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=pi[m]
```

At the acceptance check, $q = \sigma(T[1..i])$, as in the automaton. If $q < m$, retain it. At $q = m$, output first, then retain $\pi[m]$, the longest proper pattern prefix that is a suffix of the read text. A new match only needs a proper border, since a full length-m prefix cannot extend to length $m + 1$.

Every shorter candidate after mismatch lies on the failure chain. The first that extends with $T[i]$ gives exactly the automaton transition. If none extends, use zero. This proves completeness and ensures $q < m$ before accessing $P[q + 1]$.

Overlap after a full match

For $P = \texttt{aabaaab}$, $\pi[7] = 3$. Scan $T = \texttt{aabaaabaaab}$.

i	Character	q before comparison	q after extension	q retained after output
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7, output 0	3
8	a	3	4	4
9	a	4	5	5

Overlap after a full match (continued)

i	Character	q before comparison	q after extension	q retained after output
10	a	5	6	6
11	b	6	7, output 4	3

The matches share `aab` at positions 5 through 7. Resetting to zero discards that known equality.

Many comparisons for one character, linear total

From state 6 on new character `c`, compare against $P[7] = b, P[3] = b, P[2] = a, P[1] = a$. All differ. Follow $6 \to 2 \to 1 \to 0$ and retain zero. The same `c` is compared repeatedly, but the text index never retreats.

During the scan, q starts at zero and increases by at most one per character, for total increase at most n. Every while-body iteration decreases it by at least one. Post-match fallback also only decreases it, and q stays nonnegative. Thus there are at most n while iterations, plus constant work per character and output. Scanning takes $\Theta(n)$. The same accounting gives at most $m - 1$ increases and while iterations in prefix construction, hence $\Theta(m)$.

Total time is $\Theta(n + m)$, with $\Theta(m)$ workspace excluding output. This is a sequence bound, not a worst-case constant bound for each character. A full automaton table provides that per-character bound at a higher space cost.

Assumptions of the four methods

Method	Preprocessing	Worst-case scan	Pattern workspace
Naive	None	$\Theta((n - m + 1)m)$	$O(1)$
Verified Rabin-Karp	$\Theta(m)$	$\Theta((n - m + 1)m)$	$O(1)$
Full automaton using pi	$\Theta(m\|\Sigma\|)$	$\Theta(n)$	$\Theta(m\|\Sigma\|)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(m)$

The table assumes $1 \leq m \leq n$, constant-cost characters, and word modular arithmetic. Preprocessing is separate. Fast expected Rabin-Karp requires explicit randomization and collision conditions. KMP requires neither random input nor a random pattern. Unicode implementations must choose bytes, code points, or grapheme clusters as the matching units. Different encoded character sequences are not automatically identical strings.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 32.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-completeness

Part VII Selected Topics

Bit encodings, quantifiers, and verifiable certificates define computational hardness. We construct the core reductions, prove both semantic directions, and bound their size and running time. Prerequisites are decision problems, Turing machines, reductions, and Boolean circuits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover polynomial time, polynomial-time verification, reducibility, NP-completeness proofs, and NP-complete problems. The goals are to define P, NP, NP-hardness, and NP-completeness, construct many-one reductions, and prove the SAT, clique, vertex-cover, Hamiltonian-cycle, TSP, and subset-sum connections.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision languages and encodings

An abstract problem maps instances to answers. A decision problem has yes/no answers, represented by a language $L = \{x : \text{the answer on } x \text{ is yes}\}$. One object can have different encodings. An integer t needs about $\log_2(t+1)$ bits in binary but t symbols in unary. Polynomial time is measured in actual input bits, so the encoding must be specified.

$$L \in \mathsf{P} \iff \exists \text{ deterministic TM } M, \exists \text{ polynomial } p : \forall x,\ M(x) \text{ halts within } p(|x|), \quad M(x) = 1 \iff x \in L.$$

The quantifier is every x, not some x. Malformed encodings must also be rejected or otherwise handled as specified in polynomial time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Polynomial composition and practical speed

If A produces a valid encoding within $p_A(n)$ steps and B solves the next problem within p_B steps, their composition takes at most $p_A(n) + p_B(p_A(n))$. Output length cannot exceed production time. Substituting a polynomial into another polynomial remains polynomial.

Thus a problem that reduces in polynomial time to a polynomial-time solvable problem is also in P. Multiple polynomial conversions compose, provided the semantic direction is correct and lengths stay bounded.

Running quickly on one computer is not a membership proof. $O(n^{100})$ is polynomial, while a low-constant exponential algorithm can be more useful for small n. Complexity classes and engineering speed answer different questions.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Pseudopolynomial is not input-bit polynomial

A cost $O(nt)$ may be exponential in the bit length of t. Subset-sum DP has $n(B+1)$ cells and costs $O(nB)$. With binary B, this depends on numerical magnitude rather than input length and is pseudopolynomial.

For a traceable family, let $n = m, B = 2^m, a_i = 2^{m-1} + i, i = 1, \ldots, m, m \geq 2$. Each number has $\Theta(m)$ bits, and B has m+1 bits. Total input length is $N = \Theta(m^2)$, but full-table work is

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

If bounded by CN^c for fixed c,C, then eventually $m2^m \leq C'm^{2c}$, contradicting $m2^m/m^{2c} \to \infty$. The algorithm can still be useful for small B or unary input. It does not prove binary SUBSET-SUM belongs to P.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP's complete quantifiers

Fast verification requires a bounded certificate and a deterministic verifier:

$$L \in \text{NP} \iff \exists V, \exists \text{ polynomials } p, q: \quad \forall x, \quad x \in L \iff \exists y \, (|y| \leq q(|x|) \wedge V(x, y) = 1).$$

The verifier halts on **all** x,y within $p(|x| + |y|)$ steps. The string y is a certificate or witness. Quantifier order matters. A yes instance has some witness. For a no instance, no witness can make the verifier accept. Nondeterministically guessing a witness represents this existential quantifier as an accepting computation path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completeness and soundness of a verifier

For relation R(x,y), completeness says every $x \in L$ has a polynomial-length y with R(x,y)=1. Soundness says that existence of an accepted y implies $x \in L$. Both are needed for the equivalence. Completeness alone may accept false instances. Soundness alone may miss true ones.

For Hamiltonian cycle, a certificate is a sequence of n vertices. Verify that its length is n, that it is a permutation of all vertices, that each consecutive pair is an original edge, and that the last vertex connects to the first. Checking only the length is unsound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P is contained in NP

If $L \in P$, let M be its deterministic polynomial-time decider. Define V(x,y) to ignore y and run M(x). Use the empty certificate. If x is in L, that certificate accepts. If x is outside L, every y is rejected. Thus soundness and completeness hold and $P \subseteq NP$.

The reverse containment remains open. Failure to find a polynomial algorithm is not a proof that none exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial many-one reductions

The notation $A \leq_p B$ means there is a function f computable in polynomial time on every source instance, producing a valid target instance, such that for every x,

$$x \in A \iff f(x) \in B.$$

The direction converts source A into target B. To prove a new B hard, reduce a known hard A to B. Reducing B to an easy problem cannot establish hardness.

A many-one reduction produces one B instance. It does not adapt the source after a solver response or make multiple oracle calls, unless those computations themselves fit inside the legitimate polynomial-time conversion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove both semantic directions

The forward direction $x \in A \Rightarrow f(x) \in B$ constructs a target yes witness from each source yes witness. The reverse direction $f(x) \in B \Rightarrow x \in A$ shows that any target witness decodes to a source yes instance. Authors sometimes swap the names completeness and soundness depending on viewpoint, so state the actual implication.

For a clique reduction, any target k-clique must carry enough consistent information to reconstruct a satisfying assignment. A gadget, edge rule, or threshold that permits contradictory assignments breaks the reverse implication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transitivity with length bounds

Suppose $A \leq_p B$ via f and $B \leq_p C$ via g. Define $h(x) = g(f(x))$. If x belongs to A, f(x) belongs to B and h(x) to C. Conversely, h(x) in C implies f(x) in B and then x in A. Thus $x \in A \iff h(x) \in C$.

If f takes $p_f(n)$ time, then $|f(x)| \leq p_f(n)$. Composition takes at most $p_f(n) + p_g(p_f(n))$, a polynomial. Therefore $A \leq_p C$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard and NP-complete

A language L is NP-hard if $A \leq_p L$ for every $A \in NP$. It is NP-complete if it is both NP-hard and in NP.

Given known NP-complete C, prove L is in NP and $C \leq_p L$. For any A in NP, hardness of C gives $A \leq_p C$, and transitivity gives $A \leq_p L$. Without membership, the conclusion is only NP-hard. Without the reduction in this direction, known hardness has not been transferred.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The first NP-complete problem

CIRCUIT-SAT asks whether a finite acyclic Boolean circuit with fan-in-two AND/OR gates and unary NOT gates has an input making its designated output 1. Original input bits form the certificate. Evaluate gates in topological order and check the output in linear gate-and-wire time, proving NP membership.

For hardness, fix any L in NP, its deterministic verifier V, certificate bound q(n), and time bound. Given x, the free inputs to circuit C_x must represent witness y. They are not an unspecified accepting state of one fixed computation.

Normalize witnesses to exactly $q(n) + b(n)$ bits, where $b(n) = 1 + \lceil \log_2(q(n) + 1) \rceil$. The first b bits encode actual length ℓ; the next q bits store y padded with zeros. The new verifier checks $0 \leq \ell \leq q(n)$, reads only those ℓ witness bits, then simulates V. If q(n)=0, the one-bit length field permits only zero. This preserves existence of a witness without conflating different string lengths.

Expand local machine transitions into gates

Fix a single-tape deterministic TM simulating that verifier, remaining in its terminal state after acceptance or rejection. A multitape-to-single-tape simulation costs at most a polynomial factor. Let T(n) be its polynomial step bound, including the normalized witness. Initial tape length is $N = O(n + q(n) + 1)$, including the nonempty length field.

A head moves at most one cell per step. Represent $O(N + T)$ cells and retain T+1 blank cells beyond both ends. A cell label contains its tape symbol and whether the head is there, with its state if present. Fixed machine states and alphabet use a constant b bits per label.

The label at time t+1 and position j depends only on labels j-1,j,j+1 at time t. Each output bit is a Boolean function of a fixed 3b inputs. Express its truth table as an OR of minterms, each an AND of inputs or negated inputs, then replace large gates by fan-in-two gates. Because the machine and b are fixed, every local update uses $O(1)$ gates.

Circuit size and the simulation equivalence

The initial layer hardwires x, blanks, head location, and initial state. Only normalized witness bits are free. For every time $0, \ldots, T-1$ and cell, copy the fixed local circuit and connect it to the next layer. An OR tree checks for an accepting-state label at time T.

Gate and wire counts are $O(T(N+T))$. Gate indices use $O(\log(T(N+T)))$ bits. Enumerating times, cells, and a fixed truth table produces the circuit in polynomial time, with no hidden exponential hardware.

For each y, induction on t proves every layer is the actual configuration of V(x,y). The initial layer is legal, and local transitions produce exactly the next configuration. Invalid local labels are never reached. Terminal acceptance persists, so $C_x(y) = 1$ iff V accepts.

If x is in L, an accepting witness satisfies the circuit. If the circuit is satisfiable, its free witness bits make V accept, so x is in L. Every NP language reduces this way to CIRCUIT-SAT, proving NP-completeness.

Circuit-to-SAT gate constraints

Give each wire value a variable, sharing the same gate-output variable where needed. Require output z_{out} by a unit clause. Encode every gate equivalently:

Relation	CNF constraints
z=u AND v	$(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$
z=u OR v	$(z \vee \neg u) \wedge (z \vee \neg v) \wedge (\neg z \vee u \vee v)$
z=NOT u	$(z \vee u) \wedge (\neg z \vee \neg u)$

For AND, the first two clauses force both inputs true if z is true; the last forces z true if both inputs are true. OR and NOT follow by the same two-direction truth cases. Fix constant wires with unit clauses.

Circuit-to-SAT gate constraints (continued)

An accepting circuit input extends to all computed wire values and satisfies the constraints. Conversely, gate equivalences force every wire to equal its actual value in topological order, and the output unit clause forces acceptance. At most three clauses per gate give linear total size in gates and wires. Repeatedly expanding shared subcircuits into formula trees could instead cause exponential growth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses

Parse a formula into unary NOT and binary AND/OR nodes. Replace implication by $\neg a \vee b$. Other fixed-arity connectives have fixed-size truth-table gadgets. Give each internal node a new variable, apply the gate clauses, and require the root true. Remove repeated literals within a clause. Delete clauses containing both a literal and its negation, since they are tautologies.

Clauses now have at most three literals. To require exactly three distinct variables per clause, use fresh padding variables for each clause:

- Replace $(a \vee b)$ by $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$. If the original is true, any p works. If false, the pair forces both p and its negation.
- Replace unit a by the four clauses $(a \vee p \vee q), (a \vee p \vee \neg q), (a \vee \neg p \vee q), (a \vee \neg p \vee \neg q)$. If a is false, every p,q assignment falsifies one clause. If true, all pass.
- An empty clause is false, so output the padded version of $z \wedge \neg z$. An empty conjunction is true, so output one fixed satisfiable three-variable clause.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses (continued)

Original assignments extend through parse-node values. Conversely, the tree constraints recover node semantics. Padding preserves each clause requirement. This is equisatisfiability, not literal equality of functions after adding variables. Constant clauses per node and at most four per padding step give polynomial size and construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splitting a long clause

For $l_1 \vee \cdots \vee l_r$, $r > 3$, a chain also works:

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

If all original literals are false, the first clause forces y1 true, each middle clause forces the next y true, and the last forces the final y false, a contradiction. If some literal is true, let yj indicate that the first j+1 literals are all false. Every chain clause then holds. This uses r-3 new variables and r-2 clauses. Unrestricted distributive expansion of a formula into CNF may be exponentially large.

For $F = (a \vee b) \wedge \neg a$, set $u = a \vee b, v = \neg a, z = u \wedge v$, add their gate clauses, and require z. Input a=false,b=true extends to u=v=z=true. If a=true, the NOT gadget forces v=false while z and AND require v=true, excluding a false acceptance.

The book's three-CNF definition uses three distinct literals. The main reductions here can further normalize to three distinct variables. Deduplication and padding also handle repeated literals before the occurrence construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to CLIQUE

For a formula F with k clauses, create one vertex per literal occurrence and set the clique threshold to k. For k=0, the empty conjunction is true and maps separately to `(empty graph,0)`. Otherwise k≥1.

Vertices within a clause have no edges. Vertices from different clauses are adjacent exactly when their literals are not complementary. With three occurrences per clause there are 3k vertices.

Let |F| be actual binary encoding length. Intern appearing variable names into n IDs, n≤3k. Checking vertex pairs costs $O(k^2)$ word comparisons. Bit accounting includes $O(\log(n+k))$ bits per ID comparison and edge endpoint. A safe bound is $O(|F|\log|F| + k^2\log(n+k))$. An adjacency matrix has $\Theta(k^2)$ bits. Comparing arbitrarily long variable names is not automatically constant time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A consistent occurrence clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Assignment $x_1 = false, x_2 = false, x_3 = true$ satisfies all clauses. Select x3 from the first, ¬x1 from the second, and x3 from the third. They come from different clauses and are pairwise noncomplementary, forming a size-3 clique.

This is not the only witness. Selecting x1 from the first and ¬x1 from the second would fail because they lack an edge. Under the chosen assignment, x1 is false anyway. The graph cannot combine contradictory requirements into a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

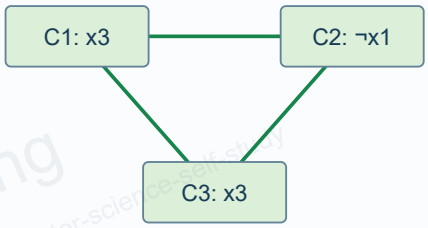
A consistent occurrence clique (continued)

Connect different clauses only for noncomplementary literals

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ Edge; × complementary pair, no edge

The three selected occurrences are consistent



x1=0, x2=0, x3=1: one true literal from each group

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both directions of the clique reduction

If F is satisfiable, fix a satisfying assignment and select one true occurrence from every clause. Two chosen literals cannot be complementary under one assignment. Every pair therefore has an edge, producing a k-clique.

Conversely, a k-clique has at most one vertex per clause since same-clause pairs lack edges. With k clauses, it selects exactly one from each. Every selected pair is noncomplementary, so no variable is required both true and false. Set each selected variable to satisfy its chosen literal and set unselected variables arbitrarily. These consistent choices satisfy every clause.

Thus F is satisfiable iff the constructed graph has a k-clique. A list of k vertices is a polynomial certificate checked by pairwise edges, so CLIQUE is NP-complete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Clique, independent set, and vertex cover

An independent set has no edge between its members. A vertex cover touches every edge. For a loopless undirected n-vertex graph, its complement has edge uv exactly when u≠v and G lacks that edge. Thus a set is a clique in G iff it is independent in its complement.

If S is independent in H, every H edge has an endpoint outside S, so $V \setminus S$ is a vertex cover. Conversely, a cover $V \setminus S$ forbids any edge with both endpoints in S, so S is independent. For $0 \leq k \leq n$,

$$\mathrm{CLIQUE}(G, k) \iff \mathrm{VC}(\overline{G}, n - k).$$

The reductions $(G, k) \mapsto (\overline{G}, n - k)$ and $(H, \ell) \mapsto (\overline{H}, n - \ell)$ both take $O(n^2)$ pair checks. A clique of at least k contains one of exactly k. A cover of at most n-k can be enlarged to exactly n-k. Handle out-of-range thresholds with fixed yes/no instances rather than negative certificate sizes.

A complement example

For vertices {1,2,3} and edges {12,23}, the set {1,2} is a size-2 clique. The complement has only edge 13. The same set is independent there, and its complement {3} covers the only edge, with size 3-2=1.

The transformation is to VC on the complement, with threshold n-k. Complementation is polynomial and reversible, giving reductions in both directions.

Vertex cover to Hamiltonian cycle

A Hamiltonian-cycle certificate is a permutation of all vertices with valid consecutive edges and closure. This proves NP membership. For hardness, build an undirected gadget for every original edge {u,v}. Crossings in its drawing are not vertices, and no planar crossing gadget is required.

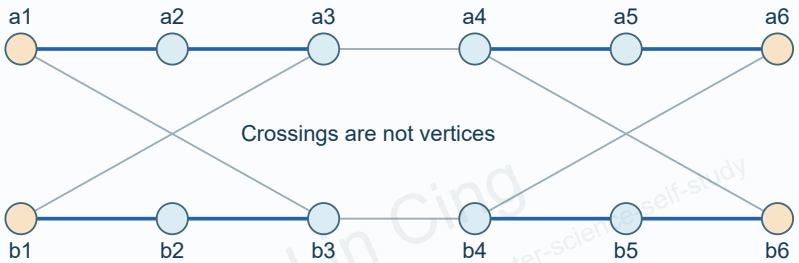
Create twelve vertices a1,...,a6 and b1,...,b6, where ai=[u,v,i] and bi=[v,u,i]. Add fourteen internal edges. Ten form the two chains a1-a2-a3-a4-a5-a6 and b1-b2-b3-b4-b5-b6. Four cross links are a1-b3, a3-b1, a4-b6, a6-b4.

Only a1,a6,b1,b6 may connect outside the gadget. The other eight vertices need two internal edges in any final cycle. In particular a2,a5,b2,b5 already have degree two in the gadget, forcing their eight adjacent chain edges.

Vertex cover to Hamiltonian cycle (continued)

HC gadget: only a1,a6,b1,b6 connect outside

a_i=[u,v,i]; b_i=[v,u,i]



Eight blue edges are forced by degree 2 at a2,a5,b2,b5. Six gray edges remain undecided.
Middle edges a3−a4 and b3−b4 force the four cross links. Orange marks the four ports.

Three legal gadget covers

The remaining choices are the middle edges a3-a4 and b3-b4. Once these are selected or omitted, degree two at internal vertices forces every cross link.

Middle edges selected	Cover of all twelve vertices
Both	Two disjoint six-vertex paths, a1 through a6 and b1 through b6
Only b3-b4	One path a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6
Only a3-a4	The symmetric full path from b1 to b6
Neither	Two closed six-vertex cycles with no way to join the external cycle, forbidden

These cases exhaust all internal degree configurations. A legal Hamiltonian cycle uses one of the first three. It cannot enter an a port and leave through a b external chain. It must cover every gadget vertex. Locally, at least one endpoint of the original edge must be selected.

Middle edges force external port use

Let x indicate selection of a3-a4 and y selection of b3-b4. Vertex a3 already uses forced edge a2-a3, so a3-b1 has indicator 1-x. Similarly a4-b6 has 1-x. Edges a1-b3 and a6-b4 each have 1-y.

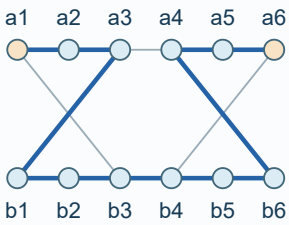
Thus a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x. Both ends on one original-vertex side connect externally together or neither does.

For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. Every vertex already has degree two, so neither cycle connects to a selector. They cannot form the whole graph's single Hamiltonian cycle. The three remaining full path covers are shown below.

Middle edges force external port use (continued)

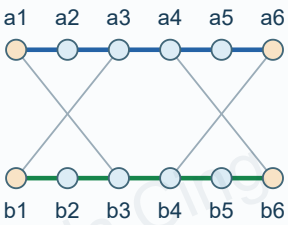
All three legal modes preserve the original-vertex side

u side only: 12 vertices



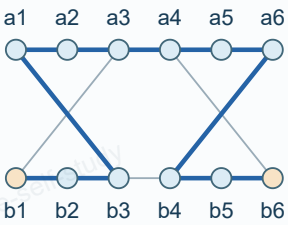
x=0, y=1

6 vertices on each side



x=1, y=1

v side only: 12 vertices



x=1, y=0

Gray edges are unused. Blue/green show paths. Each orange port needs one external edge for degree 2.

Incident chains and selectors

VERTEX-COVER asks for size at most k. Remove isolated vertices without changing the minimum cover. If k<0, or k=0 with an edge present, output a fixed no Hamiltonian-cycle instance. If no edges remain, or k is at least the remaining vertex count, output a fixed yes triangle. The main case has no isolated vertices and $1 \leq k < |V|$. A smaller cover can be enlarged to exactly k nonisolated vertices.

For each original u, order its neighbors u(1),...,u(d) arbitrarily. Connect its gadget sides by edges $[u, u(i), 6] - [u, u(i+1), 1]$, i=1,...,d-1. An external path selecting u must follow all of u's incident gadgets from one chain endpoint to the other.

Add k selectors s1,...,sk. Each selector connects to both ends of every u chain, with no other external edges. There are $12|E| + k$ vertices, $14|E|$ internal edges, at most $2|E|$ chain links, and $2k|V|$ selector links. The size is polynomial. Both source and target graphs are undirected.

Both directions of vertex cover to cycle

Given a cover $C = \{u_1, \ldots, u_k\}$, make one path through each selected vertex's incident chain. If only one endpoint of an original edge is selected, use that side's full twelve-vertex path. If both endpoints are selected, use the two disjoint six-vertex paths. Every edge has a selected endpoint, so every gadget vertex is visited exactly once. Join s1, the u1 chain, s2, the u2 chain, ..., sk, the uk chain, and back to s1. This visits every selector once and gives one Hamiltonian cycle.

Conversely, remove the k selectors from a Hamiltonian cycle. This leaves k disjoint paths. Local gadget modes forbid switching original-vertex identity, so each path follows one original u's entire incident chain. Two paths cannot select the same u, since they would share a chain endpoint or gadget vertex. Select these k original vertices. Every gadget is visited, and every legal mode uses at least one endpoint's chain. Hence every original edge is covered.

Therefore VC(G,k) holds iff the constructed graph has a Hamiltonian cycle. Construction, decoding, and size are polynomial. Fixed yes/no instances handle the preliminary cases without inventing empty chains for selectors.

A full twenty-five-vertex cycle

For original path u-v-w with k=1, choose cover {v}. Build U=W_uv and V=W_vw, each with twelve vertices, and one selector. The b side of U and a side of V represent v. Connect U.b6 to V.a1 and the selector to the two ends of the v chain.

Use these full paths, with every point prefixed by its gadget:

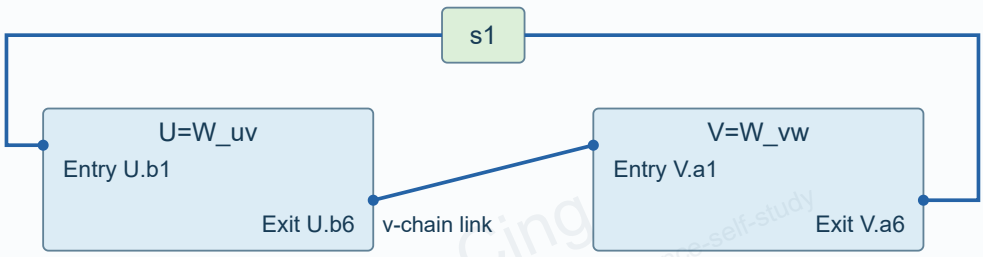
- U: b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6.
- V: a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6.

The complete cycle starts s1→U.b1, follows all of U, takes U.b6→V.a1, follows all of V, and returns V.a6→s1. Same-named vertices in different gadgets are distinct. Exactly 24 gadget vertices plus one selector are visited once. Every adjacency is an internal edge, chain link, or selector edge.

Both original edges have only v selected, so both gadgets use a full path on v's side. Choosing only u would leave W_vw uncovered and cannot create a cycle.

A full twenty-five-vertex cycle (continued)

Original path u−v−w, cover={v}: one selector closes a 25-vertex cycle



Each box contains the full 12-vertex path listed in the notes. U and V vertices are distinct.

This shows the selected HC. All external edges follow the selector and incident-chain rules.

Hamiltonian cycle to TSP decision

Given an undirected graph with $n \geq 3$, build complete graph K_n. Give original edges weight 1 and missing edges weight 2. Set threshold B=n. Construction takes $O(n^2)$.

A Hamiltonian cycle uses n original edges and becomes a tour of cost n. Conversely, every tour has n edges of weight at least 1. Total cost at most n forces every edge to have weight 1, so every tour edge belongs to the original graph. It is a Hamiltonian cycle there.

Thus HC(G) iff TSP(Kn,w,n). TSP decision is in NP. Its certificate is a vertex permutation, checked for uniqueness, closure, and total cost.

TSP trace and small graphs

For a triangle with edges 12,23,31, all three weights are 1. Tour 1→2→3→1 costs 3 and meets threshold B=3. Removing original edge 31 leaves that edge in the complete target graph with weight 2. Every three-point tour then costs 1+1+2=4>3, matching the absence of an original Hamiltonian cycle.

Simple undirected graphs with n<3 have no Hamiltonian cycle under this chapter's convention. Map them to a fixed no TSP instance. The n-edge argument above assumes n≥3.

Optimization TSP returns a minimum cost and, when requested, a tour. A polynomial exact optimizer could decide the threshold problem by comparison, so hardness of decision also makes exact optimization hard unless P=NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision, optimization, and search

Optimization counterparts include minimum-cost TSP, minimum SET-COVER, and maximum CLIQUE. An exact polynomial optimizer solves the decision version by comparing its value to the threshold. In the other direction, an exact decision oracle can find an integer optimum by binary search if upper and lower bounds have polynomial bit length and every threshold query is answered correctly.

Finding the optimum value does not itself output a witness. Reconstruction needs additional queries, for example fixing decisions step by step. A known value cannot be presented as an already constructed optimal solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision, optimization, and search (continued)

Classification	Meaning	Consequence	Does not imply
NP-hard language L	Every NP language reduces to L	If L is in P, P=NP	L is in NP
NP-complete language L	L is in NP and NP-hard	Short certificates and universal NP reductions	An optimization program is automatically a language
CLIQUE decision	Ask whether a k-clique exists	Membership and hardness proved here	Output of a maximum clique
MAX-CLIQUE optimization	Return maximum size, optionally a witness	An exact polynomial solver decides CLIQUE	Direct application of a language definition to an optimization output type

NP-hardness concerns reductions. NP-completeness also requires membership. Decision answers yes/no, optimization returns a value, and search returns a witness. Any claimed equivalence must explain how one output supplies the other's required answer.

Subset-sum membership and dynamic programming

Decision SUBSET-SUM asks whether some subset of given positive integers sums exactly to target t. A certificate lists selected indices. Check bounds, nonrepetition, and exact sum in polynomial time, proving membership in NP.

Let dp[i,s] mean that a subset of the first i integers reaches s. Then

$$dp[i, s] = dp[i - 1, s] \lor (a_i \leq s \land dp[i - 1, s - a_i]).$$

The $O(nt)$ algorithm is pseudopolynomial with binary t. The reduction below uses large values with polynomial bit length, consistently with that DP limitation.

Three-CNF-SAT to a digit table

Normalize to n appearing variables and k clauses, each with three distinct variables and no complementary tautology. Deduplication and padding above achieve this. Map constant true or false formulas separately to fixed yes/no integer instances.

Create n+k decimal columns, first for variables and then clauses. These columns compactly define integers. The final delivered encoding is binary.

- For each xi, create Ti and Fi. Both have 1 in variable column i and 0 in other variable columns. Ti has 1 in each clause containing xi; Fi has 1 in each clause containing ¬xi. Other clause digits are zero.
- For each clause j, add two slack numbers, with digits 1 and 2 respectively in that clause column and zero elsewhere.
- The target has 1 in each variable column and 4 in each clause column.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table (continued)

There are 2n+2k positive numbers, each with n+k decimal digits and $O(n+k)$ binary bits. Total output length is $O((n+k)^2)$, and construction is polynomial. If the SUBSET-SUM input format requires a set rather than a multiset, the nondegenerate premises ensure Ti and Fi differ: each variable appears, and positive/negative occurrences cannot coincide in a clause. Different variables have different high columns, and slack columns also distinguish the slack numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

No carry for every candidate subset

An arbitrary subset sums each variable column to at most 2. A clause has three distinct literals, so all truth-value numbers together contribute at most three 1s in its column. Slack adds at most 1+2=3. Thus every clause sum is at most 6<10. **No subset can carry**, not only a correctly constructed witness.

If the formula is satisfiable, choose exactly Ti or Fi according to each variable's value. Variable columns become 1. Each clause has r true literals, r=1,2,3. Fill its deficit 4-r by slack 1+2, 2, or 1, respectively. All columns reach the target.

Conversely, any subset reaching the target has exactly one of Ti,Fi per variable, because no carries can alter its variable column. These choices define a consistent assignment. Each clause target is 4 but its slack contributes at most 3, so some chosen truth-value number contributes 1 there. The corresponding literal is true. Every clause is satisfied.

The two implications and polynomial size prove NP-hardness. Together with NP membership, SUBSET-SUM is NP-complete.

A hand-checkable no-carry sum

Use $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Column order is x1,x2,x3,C1,C2.

Numbers	Decimal column representation
T1,F1	10010, 10001
T2,F2	01011, 01000
T3,F3	00110, 00101
S1,S1′	00010, 00020
S2,S2′	00001, 00002
Target	11144

Assignment (true,true,false) chooses T1+T2+F3=11122. Each clause has two true literals. Add S1′=20 and S2′=2 to reach 11144. No carries occur. Each high digit 1 confirms one truth-value number per variable.

A hand-checkable no-carry sum (continued)

Values can be of order 10^{n+k}, while binary length is only $O(n+k)$. The reduction has polynomial output even though exact $O(nt)$ DP is not polynomial in bit length.

T1+T2+F3=11122; add slack 00022 to reach target 11144

	x1	x2	x3	C1	C2	
T1	1	0	0	1	0	Each variable digit is 1
T2	0	1	0	1	1	Each clause has 2 true literals
F3	0	0	1	0	1	
slack	0	0	0	2	2	Slack 2 fills each to 4
target	1	1	1	4	4	

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why universal no-carry matters

Proving no carry only for satisfying-assignment selections would not establish the reverse direction. A wrong subset might otherwise forge target digits through carries. Here every subset has variable digits at most 2 and clause sums at most 6, so no carry can occur for any candidate.

Converting an n+k digit decimal number to binary by repeated multiplication by 10 and digit addition has a simple $O((n+k)^2)$ bit implementation. Performing it for all 2n+2k numbers is still polynomial. Verification sums at most $O(n+k)$ numbers of $O(n+k)$ bits, adding only $O(\log(n+k))$ bits to the sum.

For assignment (false,false,false) in the example, the two clauses have 0 and 2 true literals. Even both slack numbers give only 3 in the first clause column, below 4. No carry from another column can conceal the failure.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Encoding invariance, closure, and coNP

If two encodings convert to each other in polynomial time, P membership is unchanged. Convert then decide; conversion bounds output length, and polynomial composition applies. Binary and unary integers lack this two-way guarantee. Writing 2^{b-1} in unary from b binary bits requires exponential output.

P is closed under complement, union, and intersection by negating or combining polynomial deciders. For concatenation L1L2, test all n+1 split positions with the two deciders. For L*, use reachable[0]=true and test substrings x[i+1..j] for i<j as possible next pieces. $O(n^2)$ polynomial tests plus DP decide reachable[n]; accept the empty string directly. Both operations preserve P.

Define $coNP = \{L : \overline{L} \in NP\}$. It describes short certificates for no instances, not languages outside NP. Neither P=NP nor NP=coNP is known. If an NP-complete C belongs to coNP, take any A in NP with $A \leq_p C$. The same map gives $\overline{A} \leq_p \overline{C}$. Compose the reduction with the NP verifier for $\overline{C}$ to obtain $\overline{A} \in NP$, so NP⊆coNP. Taking complements yields equality. This is a conditional implication, not a proved separation.

Reduction chains and hypotheses

The main chain is CIRCUIT-SAT ≤p SAT ≤p 3-CNF-SAT ≤p CLIQUE. Complementation connects CLIQUE, INDEPENDENT-SET, and VERTEX-COVER. Another chain is VERTEX-COVER ≤p HAMILTONIAN-CYCLE ≤p TSP decision. The no-carry construction gives 3-CNF-SAT ≤p SUBSET-SUM.

Each arrow retains its own premises. HC here is undirected. The TSP complete graph has polynomial size. Subset-sum uses binary target encoding. Optimization TSP and its decision language have different output specifications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving a new NP-complete language

1. Specify the decision yes-set, instance parameters, and output.
2. Give a certificate, polynomial length bound, verifier, and acceptance conditions to prove NP membership.
3. Construct a reduction from a known NP-complete source to the new target.
4. Prove source yes implies target yes.
5. Prove target yes implies source yes.
6. Bound construction time, output size, and verification time by polynomials.
7. Handle empty graphs, k=0, repeated literals, disconnected graphs, and unreachable values as applicable.

A suggestive gadget without both implications is not a reduction proof. Converting the new problem back to a known hard problem also has the wrong direction for establishing hardness.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 34.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Approximation algorithms

Part VII Selected Topics

Approximation trades the cost of exact optimization for a proved bound. Covering, metric TSP, and subset-sum illustrate lower bounds, approximation errors, and bit costs. Constant ratios, logarithmic ratios, and FPTAS guarantees are distinct. Prerequisites are optimization, approximation ratios, and NP-hard problems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover vertex cover, traveling salesperson, set cover, randomization and linear programming, and subset sum. The goals are to prove approximation ratios and distinguish randomized guarantees, LP rounding, PTAS, and FPTAS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Objectives, feasibility, and denominators

Objectives are nonnegative. A minimization ρ-approximation returns a feasible solution on every input with $ALG \leq \rho OPT$. Maximization requires $ALG \geq OPT/\rho$, $\rho \geq 1$. When OPT=0, use the inequalities rather than 0/0. Minimization must return zero, while any nonnegative feasible maximization value satisfies its inequality.

A good result on one example is not an approximation guarantee. The bound must hold on every legal input, and time is measured in encoded input length. A minimization proof often finds $LB \leq OPT$ and proves $ALG \leq \rho LB$. Maximization can use $UB \geq OPT$ and prove $ALG \geq UB/\rho$.

A PTAS runs in polynomial input length N for every fixed $\epsilon > 0$. For example, $N^{1/\epsilon}$ can qualify. An FPTAS is polynomial jointly in N and the numerical value $1/\epsilon$, not merely its logarithm. Very small epsilon can still be costly.

Unweighted vertex cover

For a simple undirected graph, a cover touches every edge. Minimizing its size is NP-hard. This algorithm's guarantee is for unweighted size. With no edges, return the empty cover and OPT=0.

```
C=empty; matched[v]=false for every v
for each edge (u,v), in a fixed order:
    if not matched[u] and not matched[v]:
        add u,v to C
        matched[u]=matched[v]=true
return C
```

Equivalently, choose an uncovered edge, add both endpoints, and delete covered edges. Marking endpoints avoids repeatedly rebuilding the graph and takes $O(|V| + |E|)$ time and $O(|V|)$ workspace, excluding input storage.

A maximal matching is a lower bound

Let M be the selected edges. Marking both endpoints immediately ensures no selected edges share a vertex, so M is a matching. An unselected edge already had a marked endpoint when scanned, and is covered in the final C. Thus C is feasible.

The matching is maximal, meaning no edge can be added, but need not be maximum in cardinality. Every cover, including an optimum C*, must choose at least one endpoint for each M edge. Disjoint endpoints require distinct charged vertices, so

$$|C| = 2|M| \leq 2|C^*|.$$

The proof needs neither OPT nor a maximum matching. Arbitrary adjacent edges cannot be charged independently to obtain an |E| lower bound, since they may share one selected vertex. A matching prevents that double counting.

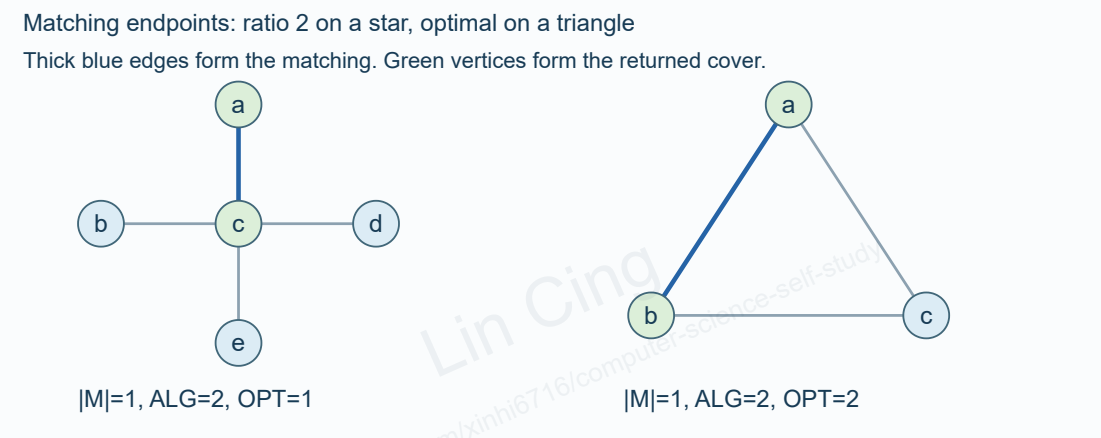
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality

For a four-leaf star centered at c, choose edge (c,a). The output is {c,a}, while {c} is optimal, attaining ratio 2. For a triangle, choosing ab gives {a,b}. Every one-vertex choice misses its opposite edge, so OPT=2 and this output is optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ratio two and exact optimality (continued)



Weights invalidate this proof for weighted cost. A single edge with endpoint weights 1 and 100 gives algorithm cost 101 and optimum 1. Weighted vertex cover needs a weight-sensitive bound, such as LP rounding below.

Metric TSP and double tree

The main input is a complete undirected graph with n≥3, symmetric nonnegative costs, and triangle inequality $c(u, w) \leq c(u, v) + c(v, w)$. This is metric TSP. Distinct vertices may have zero distance, a pseudometric; the proofs still hold.

A tour visits every vertex once and returns to its start. Double tree computes an MST T, makes a full DFS walk from any root, takes vertices in first-visit order, and closes the resulting tour. Completeness ensures each shortcut edge exists. This construction cannot be applied directly to an arbitrary sparse graph.

Simple Prim on a full cost matrix takes $O(n^2)$, with linear DFS and output, for $O(n^2)$ word arithmetic operations and comparisons. Large rational weights require additional bit costs.

Each inequality in the factor-two proof

Deleting any edge from an optimal tour leaves a spanning tree. Nonnegativity means deletion does not increase cost, so MST minimality gives $c(T) \leq OPT$.

The full DFS walk W traverses each tree edge twice, giving $c(W) = 2c(T)$. It repeats vertices and is not yet a TSP tour.

Replace a walk section $u \to v_1 \to \cdots \to v_k \to w$, whose interior vertices were already visited, by $u \to w$. Repeated triangle inequality makes the shortcut no more expensive. Keeping first visits and final closure yields a feasible tour H with

$$c(H) \leq c(W) = 2c(T) \leq 2OPT.$$

The lower bound uses nonnegativity, shortcutting uses the metric inequality, and feasibility uses completeness. These assumptions have different roles.

Every walk edge and its actual cost

Use Manhattan distances for A(0,0),B(2,0),C(2,1),D(0,1). The coordinatewise absolute-value triangle inequalities imply the metric inequality.

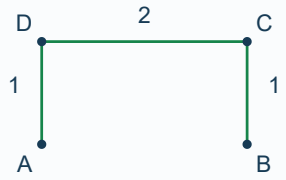
Take MST edges AD=1, DC=2, CB=1, totaling 4. The six complete-graph costs are 1,1,2,2,3,3. Every spanning tree uses three edges and therefore costs at least 1+1+2=4, proving this chain minimal.

Root at A. The full walk A,D,C,B,C,D,A costs 1+2+1+1+2+1=8. First-visit tour A,D,C,B,A costs 1+2+1+2=6. Any closed tour reaching both x-extremes has total x variation at least 4 and total y variation at least 2. Cost is at least 6, attained around the rectangle, so OPT=6. Walk cost 8 and tour cost 6 are different quantities.

Every walk edge and its actual cost (continued)

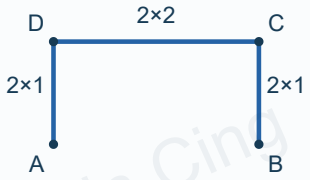
Same rectangle and Manhattan costs: MST 4 → walk 8 → tour 6

Minimum spanning tree



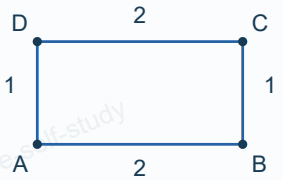
AD+DC+CB=1+2+1=4

Traverse each tree edge twice



A,D,C,B,C,D,A; cost 8

First visits, then return to A



A,D,C,B,A; cost 6

A(0,0), B(2,0), C(2,1), D(0,1). Axes have equal scale. Labels are Manhattan costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Christofides' three-halves guarantee

Under the same metric assumptions, let O be the odd-degree vertices of an MST. The degree sum is twice the edge count, so |O| is even. Find a minimum-weight perfect matching M on O and add it to T as a multigraph. Every odd degree becomes even, all other degrees remain even, and connectivity remains. An Euler closed walk exists; shortcut it to a tour.

Take an optimal tour and shortcut it to the vertices of O in visit order. Its closed cycle costs at most OPT. Alternating cycle edges form two perfect matchings. If |O|=2, view the cycle as two return edges. Their combined cost is at most OPT, so one costs at most OPT/2. Minimum matching therefore has $c(M) \leq OPT/2$. The final tour costs at most $c(T) + c(M) \leq 3OPT/2$.

This uses the subroutine theorem that general-graph minimum-weight perfect matching is polynomial-time solvable. An arbitrary greedy matching cannot replace the minimum matching in this proof. Euler traversal and shortcutting are polynomial too.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A constant ratio for nonmetric TSP would decide HC

Assume a deterministic polynomial-time algorithm A guarantees fixed ratio $\rho \geq 1$ on every nonnegative complete graph. Fix integer $R \geq \rho$. For an n-vertex HC input, give original edges weight 1 and nonedges weight $M = Rn + 1$.

With an HC, OPT=n, so A returns cost at most $\rho n \leq Rn$. Without an HC, every tour uses a nonedge, costing at least $M + n - 1 = Rn + n > Rn$. Comparing A's output to Rn decides HC in polynomial time, implying P=NP.

M has $O(\log n)$ binary bits, and the target graph has $O(n^2 \log n)$ bits. Large values conceal no superpolynomial output. These costs generally violate triangle inequality and do not contradict metric algorithms. The conclusion is conditional on P≠NP and applies to the stated deterministic guarantee. An expected-value randomized guarantee is not automatically a deterministic decider.

Weighted set cover

Let universe X have N elements and family $F = \{S_1, \ldots, S_M\}$, each a subset of X, with union X. Unweighted cover minimizes selected set count. Weighted cover has positive rational costs and minimizes total cost. Empty X returns the empty cover. If the union is insufficient, report infeasibility.

```
U=X; C=empty
while U is nonempty:
    choose S with |S intersect U|>0 minimizing w(S)/|S intersect U|
    C=C union {S}; U=U minus S
return C
```

For unit weights, this chooses maximum new coverage. Every round covers a new element, and previously chosen sets have zero new coverage. There are at most min(N,M) rounds. A direct incidence-matrix implementation computes counts in O(NM) per round and O(NM min(N,M)) operations overall, polynomial. Compare rational ratios exactly by cross multiplication.

Weighted set cover (continued)

Exact set cover is NP-hard. Map vertex cover to X=E, with one unit-cost set Sv of incident edges per vertex. At most k sets cover X iff at most k vertices cover the original edges. Incidence count is 2|E|, so size and construction are polynomial. Weighted set cover includes the unit-cost case. This does not claim every restricted family is hard.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Element charges yield a harmonic bound

When the greedy algorithm buys S of cost w covering k new elements, charge each new element w/k. Each element is charged exactly once, so total charge equals ALG.

Fix any candidate set A of size dA. Order its elements by first coverage as e1,...,edA, arbitrarily within one round. At the start of the round covering ej, A has at least dA-j+1 uncovered elements and is a valid alternative. Greedy's charge therefore satisfies

$$charge(e_j) \le \frac{w(A)}{d_A - j + 1}.$$

If other A elements were covered earlier in the same round, the number uncovered at the round's start is only larger, preserving the bound. Summing gives $\sum_{e \in A} charge(e) \le w(A) H_{d_A}$, where $H_d = \sum_{j=1}^{d} 1/j$ and H0=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summing over an optimum cover

Let C* be optimal and d=max|Sj|. Every element belongs to at least one optimum set and has nonnegative charge. Thus

$$ALG = \sum_{e \in X} charge(e) \leq \sum_{A \in C^*} \sum_{e \in A} charge(e) \leq \sum_{A \in C^*} w(A) H_{|A|} \leq H_d OPT.$$

Repeated elements on the right increase the upper bound, so the inequality direction is correct. For j≥2, $1/j \leq \int_{j-1}^{j} dx/x$, proving $H_d \leq 1 + \ln d$.

Weighted and unweighted cases share the proof but use different selection rules. Choosing only maximum new count with arbitrary weights does not minimize charge per element and has no guarantee from this argument.

Three greedy sets versus two optimal sets

Let X={1,2,3,4,5,6}, S1={1,2,3}, S2={4,5,6}, S3={1,2,4,5}, all cost 1. Break the later tie in favor of S1.

Round	U before selection	New counts S1,S2,S3	Choice	New charges
1	1,2,3,4,5,6	3,3,4	S3	1,2,4,5 each pay 1/4
2	3,6	1,1,0	S1	3 pays 1
3	6	0,1,0	S2	6 pays 1

ALG=3. Sets S1,S2 cover X, and no single set does, so OPT=2 and ratio=3/2. Charges sum to 3. Each optimum set receives total 3/2, at most $H_3 = 11/6$, as the general proof predicts.

Three greedy sets versus two optimal sets (continued)

S3 covers four first, but greedy needs three sets; S1+S2 need two

	1	2	3	4	5	6	
S1	1	1	1	0	0	0	Initially: 1,2,3,4,5,6
S2	0	0	0	1	1	1	After S3: 3,6
S3	1	1	0	1	1	0	After S1: 6
							After S2: ∅

charge(1,2,4,5)=1/4; charge(3)=charge(6)=1

Total charge 3 equals greedy cost. Optimal S1∪S2={1,…,6}; no single set covers X.

Randomized maximum three-CNF satisfaction

MAX-3-CNF-SAT maximizes satisfied clauses among m clauses. Require three distinct variables per clause and no complementary pair. Assign variables independently and fairly. Each clause has probability 1/8 of being entirely false and 7/8 of being true.

Exact polynomial optimization would decide three-CNF satisfiability by checking OPT=m, so exact optimization is NP-hard. It still admits the following approximation. Let Ij indicate satisfaction and Y=sum Ij. Clauses may share variables, so their indicators need not be independent. Linearity of expectation gives

$$E[Y] = \sum_j E[I_j] = 7m/8 \geq 7OPT/8.$$

An expected-objective 8/7 approximation does not promise that every run reaches 7OPT/8 or that $E[OPT/Y] \leq 8/7$. A single clause gives Y=0 with probability 1/8, preventing a finite expected-ratio statement. Repeated literals such as (a∨a∨a) have only 1/2 satisfaction probability. A clause containing a literal and its negation is always true and can be counted separately.

From expectation to a success probability

For m=0, any assignment gives Y=OPT=0 and succeeds with probability 1. For m>0, let $p = \Pr[Y \geq 3m/4]$. Since 0≤Y≤m and E[Y]=7m/8,

$$7m/8 \leq (1-p)3m/4 + pm,$$

so p≥1/2. Independently rerun r times and retain the best assignment. Failure to reach 3m/4 has probability at most 2^{-r}. This is a high-probability 4/3 threshold guarantee. It is not a high-probability 7/8 threshold guarantee.

For a deterministic 7/8 objective bound, fix variables by conditional expectation instead.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Conditional expectation never decreases

Given a partial assignment, a clause with a true literal has conditional satisfaction probability 1. A fully assigned false clause has probability 0. Otherwise, with r unassigned distinct variables, it has probability $1 - 2^{-r}$. Sum these probabilities to compute the conditional expectation exactly.

For the next variable x, the current expectation is the average of its values after fixing x=0 and x=1. Choose the larger branch. After all variables are fixed, expectation equals actual satisfaction count, at least $7m/8 \geq 7OPT/8$.

For clauses (a∨b∨c), (¬a∨b∨c), (a∨¬b∨c), (a∨b∨¬c), initial expectation is 7/2. Branch a=0 gives 13/4 and a=1 gives 15/4, so choose 1. Then b=0 gives 7/2 and b=1 gives 4, so choose 1. Either c works; choose 0 and all four clauses hold.

Recomputing m clauses at each of n variables takes O(nm) fixed-precision fraction operations. Denominators here are at most 8, with no hidden exponential precision.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Weighted vertex-cover LP and threshold rounding

With positive rational weights, minimize $\sum_v w_v x_v$, subject to $x_u + x_v \geq 1$ on every edge and integral x in {0,1}. Relax to 0≤xv≤1. Every integral cover remains feasible, so $LP^* \leq OPT$.

Find an optimal feasible LP solution x* and choose C={v:xv*≥1/2}. If both endpoints of an edge were omitted, both values would be below 1/2, violating the edge constraint. Thus C is a cover. For every vertex, its selection indicator is at most 2xv*. Multiply by nonnegative weights and sum:

$$w(C) \leq 2 \sum_v w_v x_v^* = 2LP^* \leq 2OPT.$$

Rational LP has polynomial-bit-complexity algorithms. Frequent practical success of simplex does not prove its polynomial worst case. The proof uses an actually feasible LP solution. Numerical tolerances need their own feasibility and cost analysis. The number 0.4999 is not exactly 1/2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LP optimum differs from integer optimum

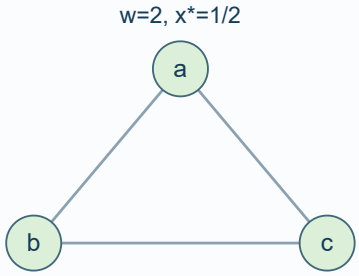
For a triangle with weights 2,3,4, the half-value solution costs 9/2. Multiply edge constraints ab,ac,bc by 1/2,3/2,5/2. Their left side becomes $2x_a + 3x_b + 4x_c$, with right side 9/2. Every LP solution therefore costs at least 9/2, proving the half solution optimal.

An integer cover needs at least two vertices. Their costs are 5,6,7, so OPT=5. Threshold rounding takes all three, giving ALG=9=2LP*≤2OPT=10. Its actual ratio is 9/5. The LP value 4.5 is not the original optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LP optimum differs from integer optimum (continued)

Weighted triangle: LP*=9/2, integer OPT=5, rounded cost=9



Edge multipliers: ab 1/2, ac 3/2, bc 5/2

Sum: 2x_a+3x_b+4x_c ≥9/2

Half values attain 9/2, proving LP optimality

Integer cover a,b costs 2+3=5

Threshold ≥1/2 selects all three: cost 9

Green vertices all pass the threshold. LP and integer optima are different.

Randomized set-cover rounding

For N≥2, use the LP constraints $\sum_{S \ni e} x_S \geq 1, x_S \geq 0$, with nonnegative costs. Set integer $k = 3\lceil \log_2 N \rceil \geq 3 \ln N$, and select each set independently with $p_S = \min(1, kx_S)$. Rational x gives rational probabilities, implementable by uniform integer sampling over their denominators. No irrational logarithmic probability is required.

If some set containing e has p=1, e is certainly covered. Otherwise, its omission probability is

$$\prod_{S \ni e} (1 - p_S) \leq e^{-\sum p_S} \leq e^{-k} \leq N^{-3}.$$

A union bound gives probability at most $N^{-2} \leq 1/4$ of any uncovered element. Expected cost is at most kLP*. By Markov, cost exceeds 2kLP* with probability at most 1/2. Adding the two failure probabilities gives at most 3/4, so each trial simultaneously covers X and meets the cost bound with probability at least 1/4. The ratio 2k is O(log N).

Randomized set-cover rounding (continued)

Check actual coverage and cost, and independently retry failures. Expected trials are at most four. This is an expected polynomial algorithm with a verifiable success condition, not a guaranteed single successful trial. For N=0 return empty. For N=1 choose the cheapest set containing its element. If LP*=0, nonnegative cost and zero expected cost imply cost zero with certainty. The cost check then automatically passes; only coverage needs retries, without dividing by zero in Markov's inequality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization subset sum

Given positive integers a1,...,an and integer t≥0, maximize a subset sum at most t. Items are distinguished by indices, even if values repeat. Remove ai>t first. For n=0 or t=0 return the empty subset with value zero.

The exact list algorithm starts L0={0}. At step i, merge and deduplicate Li-1 and Li-1+ai, then remove values exceeding t. Both lists are sorted, so two-pointer merging is linear without re-sorting.

Every subset of the first i items either excludes i or includes it, corresponding to those two lists. Positivity means an oversized sum can never decrease later, so removal is safe. Induction retains every feasible exact sum. There can be 2^i distinct sums, for example powers of two with a sufficiently large target. Linear merging per round does not make the overall algorithm polynomial.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TRIM retains smaller, actual sums

For a sorted list and δ>0:

```
R=[]
for y in L, in increasing order:
    if R is empty or y>(1+delta)*R[-1]: append y to R
return R
```

If y is omitted, the retained last value z satisfies $z \le y \le (1+\delta)z$. If retained, choose z=y. Every candidate therefore has a representative with $y/(1+\delta) \le z \le y$. Zero is kept, and the first positive value cannot be removed by zero.

Retained values are actual integer subset sums. TRIM does not divide or round their numerical values. Smaller representatives preserve capacity feasibility when later items are added. A larger representative can exceed the target even when the original y remains feasible.

Merge, trim, and remove oversized values

For $0 < \epsilon < 1, n \ge 1$, set $\delta = \epsilon/(2n)$.

```
L=[0]
for i=1..n:
    B=merge_unique(L, [z+a[i] for z in L])
    L=trim(B,delta)
    remove all values >t from L
return max(L)
```

Every retained value is an actual sum from the first i items, so the returned sum is feasible and at most t. To return its subset, retain immutable parent pointers and whether the current item was used. When equal values merge, choose any valid provenance. Backtracking takes at most n steps, with strictly decreasing item indices and no item reuse.

Value-only computation can roll its lists. Retaining reconstruction nodes for all rounds instead needs O(nK) nodes, where K bounds one list. It cannot still be claimed to use only O(K) space.

Induction includes the selected-item branch

For every exact feasible sum y≤t from the first i items, prove there is $z \in L_i$ with $y/(1+\delta)^i \le z \le y$. At i=0, only y=z=0 exists.

If item i is excluded, induction gives a prior representative u with $y/(1+\delta)^{i-1} \le u \le y$. Candidate B contains u. TRIM supplies z≥u/(1+δ), yielding the bound. Also z≤y≤t, so final removal does not discard it.

If item i is included, write y=y′+ai. Positivity ensures y′≤t. Induction gives u between $y'/(1+\delta)^{i-1}$ and y′. Candidate B contains u+ai, and

$$u + a_i \ge y'/(1+\delta)^{i-1} + a_i \ge (y' + a_i)/(1+\delta)^{i-1}.$$

TRIM loses at most one additional factor 1+δ. Its representative is at most u+ai≤y≤t, so survives capacity removal. The second inequality explicitly uses ai≥0. With negative items, both early removal and this proof need revision.

Accumulated error stays within epsilon

Apply the invariant to OPT. The maximum retained value is at least its representative, so $OPT/z_{max} \le (1+\delta)^n$ when OPT>0. For OPT=0, use the approximation inequality directly.

Since $\ln(1+x) \le x$,

$$n \ln(1 + \epsilon/(2n)) \le \epsilon/2.$$

For 0<ε<1, $\ln(1+\epsilon) = \int_0^\epsilon (1+x)^{-1} dx \ge \epsilon/(1+\epsilon) \ge \epsilon/2$. Therefore $(1+\epsilon/(2n))^n \le 1+\epsilon$, giving

$$z_{max} \ge OPT/(1+\epsilon) \ge (1-\epsilon)OPT.$$

The proof controls multiplication of per-round errors. Setting every round's δ equal to global ε would allow factor $(1+\epsilon)^n$ and would not establish the same global guarantee.

Four rounds with integer lists

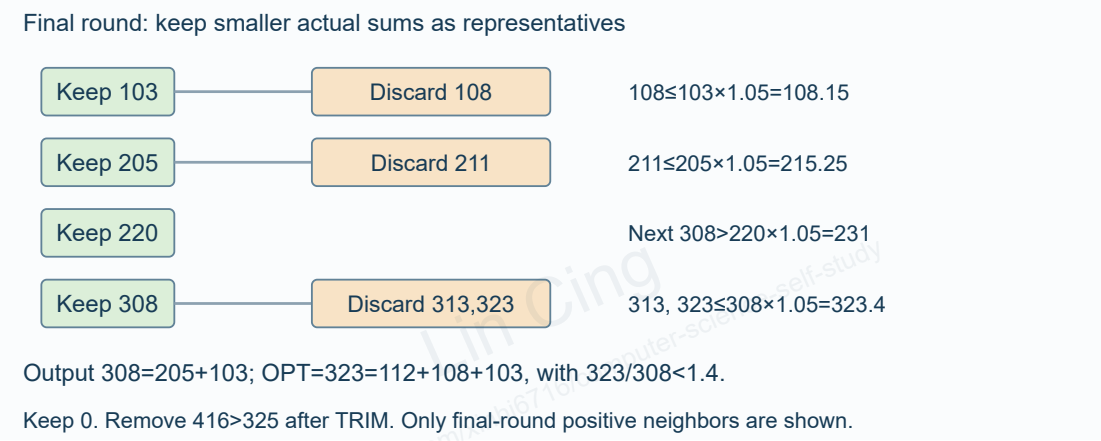
Use a=[112,108,205,103], t=325, ε=2/5, n=4, δ=1/20.

i	Merged unique B	After TRIM	After removing values above 325
1	0,112	0,112	0,112
2	0,108,112,220	0,108,220	0,108,220
3	0,108,205,220,313,425	Keep all	0,108,205,220,313
4	0,103,108,205,211,220,308,313,323,416	0,103,205,220,308,416	0,103,205,220,308

At round 2, 112≤108・1.05=113.4, so discard that candidate sum. At round 4, 108≤103・1.05=108.15, 211≤205・1.05=215.25, and both 313,323≤308・1.05=323.4. Each is represented by a smaller actual sum.

The output is 308=205+103. Exact OPT=323=112+108+103. The actual ratio is about 1.049≤1.4, and 308≤325. Discarding candidate sum 112 at one round does not delete that input item.

Four rounds with integer lists (continued)



List length, bit cost, and FPTAS

Adjacent positive trimmed values have ratio greater than 1+δ. The smallest is at least 1 and the largest at most t. There are at most $1 + \ln t / \ln(1+\delta)$ positive entries, plus zero. Since $\ln(1+\delta) \geq \delta/(1+\delta) \geq \delta/2$,

$$K = O(1 + n \log(t+1)/\epsilon).$$

Linear merging, trimming, and filtering over n rounds use $O(nK) = O(n + n^2 \log(t+1)/\epsilon)$ arithmetic operations and comparisons. Values alone need O(K) workspace. The logarithm is input bit length, unlike the numerical t in exact DP's O(nt).

For exact rational $\epsilon = \alpha/\beta$, test trimming by $2n\beta y > (2n\beta + \alpha)last$, avoiding floating relative error. Reading input and removing ai>t costs O(N) bit work, with original bit length N. Later candidates are at most 2t. Intermediate bit lengths are polynomially controlled by log(t+1), log n, and the encoding of α,β. Including each large-integer multiplication still gives time polynomial in input length and 1/ε, completing the FPTAS proof.

Guarantees that must remain distinct

Method	Feasibility	Objective guarantee	Conditions
Matching endpoints for unweighted VC	Every run	ALG≤2OPT	Does not cover arbitrary weights
Double-tree TSP	Every run	ALG≤2OPT	Complete, nonnegative, symmetric metric
Weighted set-cover greedy	Every run	ALG≤Hd OPT	Choose cost per newly covered element
Random MAX-3-CNF	Every run returns an assignment	E[ALG]≥7OPT/8	Three distinct variables per clause
LP threshold weighted VC	Every run	ALG≤2OPT	Optimal feasible LP and nonnegative weights
Subset-sum TRIM	Every run	ALG≥OPT/(1+ε)	Nonnegative items and δ=ε/(2n)

Guarantees that must remain distinct (continued)

Expectation, deterministic inequalities, and probability-of-failure statements answer different questions. An approximation scheme must account for precision, list length, encoded input size, and reconstruction cost. Removing nearby values alone is not a running-time proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 35.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summations

Part VIII Appendix

Scope

Derive arithmetic, power, geometric, telescoping, and product identities from finite sums. Analyze loops with fixed-constant induction, tail ratios, dyadic blocks, and integral bounds. State convergence assumptions before using infinite series or changing summation order.

Prerequisites

Arithmetic and geometric sequences, logarithms, and integral estimates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Use summation formulas and linearity.
- Bound sums by integration, splitting, and geometric series.
- Distinguish finite sums from infinite series.
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Summation formulas and properties

The finite sum $\sum_{k=1}^{n} a_k$ equals 0 when $n = 0$. The finite product $\prod_{k=1}^{n} a_k$ equals 1 when $n = 0$. These empty expressions use the additive and multiplicative identities, so formulas also cover loops with zero iterations. Finite terms may be reordered arbitrarily, and

$$\sum_{k=1}^{n}(ca_k + b_k) = c\sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k.$$

An infinite series $\sum_{k=1}^{\infty} a_k$ is the limit of its partial sums S_n. It is not obtained by substituting ∞ into a finite formula. If the partial sums have no finite limit, the series diverges. Convergence requires $a_k \to 0$, because $a_k = S_k - S_{k-1}$. That condition is insufficient. The terms $1/k$ tend to zero, but the harmonic series diverges.

Absolute convergence means that $\sum_{k=1}^{\infty} |a_k|$ converges. It permits arbitrary reordering without changing the sum. A conditionally convergent series, such as $\sum_{k=1}^{\infty} (-1)^{k+1}/k$, need not preserve its sum under reordering, which changes partial-sum cancellation. Termwise calculus below requires a convergence interval. Finite linearity does not justify arbitrary infinite operations.

Arithmetic sums by pairing and induction

Write $S_n = 1 + 2 + \cdots + n$ and reverse the order to obtain $n + (n-1) + \cdots + 1$. Adding the two expressions gives $2S_n = n(n+1)$, so

$$S_n = \frac{n(n+1)}{2}.$$

The formula includes $n = 0$. Induction gives a second verification. The base case is $S_0 = 0$, and the induction step is

$$S_{n+1} = \frac{n(n+1)}{2} + (n+1) = \frac{(n+1)(n+2)}{2}.$$

The induction coefficient stays fixed as n changes. More generally,

$$\sum_{j=0}^{n-1}(a + jd) = na + \frac{dn(n-1)}{2}.$$

A loop with $j = 3, \ldots, 7$ that performs $2j - 1$ units of work has total cost

Arithmetic sums by pairing and induction (continued)

$$\begin{aligned}\sum_{j=3}^{7}(2j-1) &= \left(2\sum_{j=1}^{7} j - 7\right) - \left(2\sum_{j=1}^{2} j - 2\right) \\ &= 49 - 4 = 45 \\ &= 5 + 7 + 9 + 11 + 13.\end{aligned}$$

Changing the starting index requires subtracting the correct prefix. The upper index 7 is not the number of terms, which is 5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square and cube sums from telescoping

Sum $(k+1)^3 - k^3 = 3k^2 + 3k + 1$ for $k = 0, \ldots, n$. The left side telescopes, giving

$$(n+1)^3 = 3\sum_{k=0}^{n} k^2 + \frac{3n(n+1)}{2} + (n+1).$$

Solving for the sum gives

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Similarly, sum $(k+1)^4 - k^4 = 4k^3 + 6k^2 + 4k + 1$ and substitute the preceding square and arithmetic sums:

$$\begin{aligned}(n+1)^4 &= 4\sum_{k=0}^{n} k^3 + 6\sum_{k=0}^{n} k^2 + 4\sum_{k=0}^{n} k + (n+1), \\ \sum_{k=0}^{n} k^3 &= \frac{n^2(n+1)^2}{4} = \left(\frac{n(n+1)}{2}\right)^2.\end{aligned}$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square and cube sums from telescoping (continued)

Both formulas hold at $n = 0$. At $n = 4$, the checks are

$$0 + 1 + 4 + 9 + 16 = 30 = \frac{4 \cdot 5 \cdot 9}{6},$$
$$0 + 1 + 8 + 27 + 64 = 100 = \left(\frac{4 \cdot 5}{2}\right)^2.$$

Telescoping proves the formulas for general n. The small cases check arithmetic and transcription.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite, infinite, and boundary geometric sums

Let $G_n(x) = \sum_{k=0}^{n} x^k$. Subtract G_n from xG_n. Interior terms cancel, so

$$(x - 1)G_n(x) = x^{n+1} - 1.$$

For $x \neq 1$, this gives $G_n(x) = (x^{n+1} - 1)/(x - 1)$. At $x = 1$, evaluate separately to obtain $G_n(1) = n + 1$; division by zero is invalid.

If $|x| < 1$, then $x^{n+1} \to 0$, and

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}.$$

At $x = 1$, partial sums are unbounded. At $x = -1$, they alternate between 1 and 0. If $|x| > 1$, the terms do not tend to zero.

For $x = -1/2$ and $n = 3$, the finite sum is $1 - 1/2 + 1/4 - 1/8 = 5/8$, whereas the infinite sum is $2/3$. When $|x| < 1$, the exact tail and its absolute bound are

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite, infinite, and boundary geometric sums (continued)

$$\sum_{k=n+1}^{\infty} x^k = \frac{x^{n+1}}{1-x}, \qquad \left|\frac{x^{n+1}}{1-x}\right| \le \frac{|x|^{n+1}}{1-|x|}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted geometric sums and valid calculus

Finite polynomials may be differentiated term by term. A direct derivation starts with $W_n = \sum_{k=0}^{n} kx^k$. Subtracting xW_n gives

$$(1-x)W_n = x + x^2 + \cdots + x^n - nx^{n+1}.$$

Substitute the finite geometric formula and rearrange:

$$W_n = \frac{x - (n+1)x^{n+1} + nx^{n+2}}{(1-x)^2}, \qquad x \neq 1.$$

At $x = 1$, $W_n = n(n+1)/2$. For $|x| < 1$, $n|x|^n \to 0$, because an exponential with a fixed base dominates a polynomial. Taking the limit gives

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}.$$

This derivation does not interchange differentiation with an infinite sum without proof. Power series permit termwise differentiation and integration inside their radius of convergence, which is 1 here. Endpoints require separate checks.

Weighted geometric sums and valid calculus (continued)

Apply $x\,d/dx$ once more inside that radius:

$$\sum_{k=0}^{\infty} k^2 x^k = \frac{x(1+x)}{(1-x)^3}.$$

Thus $x = 1/2$ gives $\sum_{k=0}^{\infty} k/2^k = 2$ and $\sum_{k=0}^{\infty} k^2/2^k = 6$. Terms tending to zero alone would not establish these convergent sums.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Telescoping leaves boundary terms

Each interior term occurs once with each sign, so

$$\sum_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0.$$

Align indices and endpoints before cancellation. For example,

$$\frac{1}{k(k+2)} = \frac{1}{2}\left(\frac{1}{k} - \frac{1}{k+2}\right),$$

which gives

$$\sum_{k=1}^{n} \frac{1}{k(k+2)} = \frac{1}{2}\left(1 + \frac{1}{2} - \frac{1}{n+1} - \frac{1}{n+2}\right).$$

At $n = 3$, $1/3 + 1/8 + 1/15 = 21/40$, matching $\frac{1}{2}(3/2 - 1/4 - 1/5)$. Letting $n \to \infty$ gives $3/4$, since the terminal denominators tend to infinity.

Products telescope as well. For $n \geq 2$,

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Telescoping leaves boundary terms (continued)

$$\prod_{k=2}^{n}\left(1-\frac{1}{k^2}\right)=\left(\prod_{k=2}^{n}\frac{k-1}{k}\right)\left(\prod_{k=2}^{n}\frac{k+1}{k}\right)$$
$$=\frac{1}{n}\frac{n+1}{2}=\frac{n+1}{2n}.$$

At $n=1$, the empty product is 1, as the formula also predicts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Products as sums and indispensable premises

If every $a_k>0$, then

$$\ln\left(\prod_{k=1}^{n}a_k\right)=\sum_{k=1}^{n}\ln a_k.$$

A zero or negative factor has no real logarithm. For example,

$$\prod_{k=1}^{n}2^{3k+1}=2^{\sum_{k=1}^{n}(3k+1)}=2^{3n(n+1)/2+n}.$$

For $0\le u_k<1$, logarithm bounds give

$$-\frac{u_k}{1-u_k}\le\ln(1-u_k)\le-u_k.$$

These bounds control products through sums. As u_k approaches 1, the lower-bound denominator cannot be treated as a fixed constant.

If fixed constants satisfy $0<c_1f(k)\le h(k)\le c_2f(k)$ for every $k\ge1$, summing gives

Products as sums and indispensable premises (continued)

$$c_1 \sum_{k=1}^{n} f(k) \le \sum_{k=1}^{n} h(k) \le c_2 \sum_{k=1}^{n} f(k).$$

If the inequalities hold only on a tail, retain the finite prefix separately. When $\sum_{k=1}^{n} f(k) \to \infty$ and the prefix is finite, the prefix does not change the Θ order. Constants must be uniform over k. An unbounded choice of a different constant at every index proves no common Θ bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bounding summations

Decide whether an exact value, an upper bound, or a two-sided order is needed. Induction, termwise bounds, fixed ratios, splitting, and integration serve different purposes. A rough upper bound alone does not establish tightness.

For $S_n = \sum_{k=0}^{n} 4^k$, seek $S_n \le c4^n$. The base case $n = 0$ requires $c \ge 1$. The induction step gives

$$S_{n+1} \le c4^n + 4^{n+1} = \left(\frac{c}{4} + 1\right) 4^{n+1}.$$

It closes when $c/4 + 1 \le c$, or $c \ge 4/3$. The fixed choice $c = 4/3$ works for all n. The last term gives $S_n \ge 4^n$, so $S_n = \Theta(4^n)$.

The false induction claiming that $\sum_{k=1}^{n} k = O(n)$ remains linear after adding $n + 1$ hides a growing constant. An assumption $\sum_{k=1}^{n} k \le cn$ would require

$$cn + n + 1 \le c(n + 1), \qquad \text{or} \qquad n + 1 \le c.$$

No fixed c satisfies this for every n.

Maximum-term upper bounds and tail lower bounds

For nonnegative a_k and any subset $J \subseteq \{1, \dots, n\}$,

$$\sum_{k=1}^{n} a_k \le n \max_{1 \le k \le n} a_k, \qquad \sum_{k=1}^{n} a_k \ge \sum_{k \in J} a_k.$$

Upper bounds may use all terms. Lower bounds often retain enough large terms. For example, $\sum_{k=1}^{n} k \le n^2$. The indices $\lceil n/2 \rceil, \dots, n$ contain at least $n/2$ terms, each at least $n/2$. Thus $\sum_{k=1}^{n} k \ge n^2/4$ for both odd and even n, proving $\Theta(n^2)$ without an exact formula.

For a fixed real $r \ge 0$, the same argument gives

$$2^{-r-1} n^{r+1} \le \sum_{k=1}^{n} k^r \le n^{r+1}.$$

Hence the sum is $\Theta(n^{r+1})$. Fixing r is essential. If $r = n$, the lower coefficient 2^{-r-1} depends on n, and this proof gives no uniform order.

A geometric tail requires a fixed ratio

Suppose $a_k \ge 0$ and a fixed $0 < \rho < 1$ satisfies $a_{k+1} \le \rho a_k$ for every $k \ge K$. Induction gives $a_{K+j} \le a_K \rho^j$, so

$$\sum_{k=K}^{\infty} a_k \le \frac{a_K}{1 - \rho}.$$

The multiplicative inequality also covers zero terms without division by zero.

For $a_k = k^2/3^k$ and $k \ge 2$,

$$\frac{a_{k+1}}{a_k} = \frac{1}{3}\left(1 + \frac{1}{k}\right)^2 \le \frac{3}{4}.$$

The prefix at $k = 0, 1$ contributes $1/3$. The tail starts at $a_2 = 4/9$, giving

$$\sum_{k=2}^{\infty} a_k \le \frac{4/9}{1/4} = \frac{16}{9}, \qquad \sum_{k=0}^{\infty} a_k \le \frac{19}{9}.$$

A geometric tail requires a fixed ratio (continued)

The exact weighted geometric formula gives $3/2$, a sharper finite bound. Harmonic ratios $k/(k+1) < 1$ tend to 1, so no fixed $\rho < 1$ bounds the whole tail. A prefix that depends on n, such as two terms each equal to n^2, is not $O(1)$ merely because it contains only two terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dyadic blocks reveal harmonic layers

Let $H_n = \sum_{k=1}^{n} 1/k$. The block $k = 2^j, \ldots, 2^{j+1} - 1$ has 2^j terms, each between $1/2^{j+1}$ and $1/2^j$. Each complete block contributes between $1/2$ and 1.

Complete the last partial block for an upper bound. There are $\lfloor \log_2 n \rfloor + 1$ blocks, each contributing at most 1. The first $\lfloor \log_2 n \rfloor$ complete blocks give a lower bound. For $n \geq 2$,

$$\frac{\lfloor \log_2 n \rfloor}{2} \leq H_n \leq \lfloor \log_2 n \rfloor + 1.$$

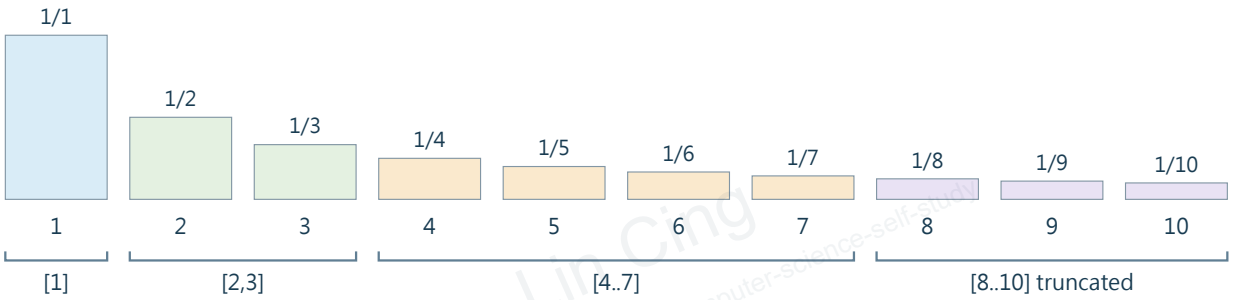
The lower bound is $\Omega(\log n)$, so $H_n = \Theta(\log n)$, also proving that the infinite series diverges.

At $n = 10$, the blocks $[1]$, $[2, 3]$, $[4, 5, 6, 7]$, and $[8, 9, 10]$ contribute 1, $5/6$, $319/420$, and $121/360$. Their total is $H_{10} = 7381/2520 \approx 2.928968$. The last block has three terms, not eight.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dyadic blocks reveal harmonic layers (continued)

n=10: each full dyadic block sums between1/2 and1



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integral bounds follow from unit intervals

If f is integrable and nondecreasing on $[m-1, n+1]$, each integer k satisfies

$$\int_{k-1}^{k} f(x)\,dx \le f(k) \le \int_{k}^{k+1} f(x)\,dx.$$

Summing gives

$$\int_{m-1}^{n} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f(x)\,dx.$$

For a nonincreasing f, the inequalities reverse:

$$\int_{m}^{n+1} f(x)\,dx \le \sum_{k=m}^{n} f(k) \le \int_{m-1}^{n} f(x)\,dx.$$

The endpoint shifts control the error. A sum is not simply replaced by an integral. Since $1/x$ decreases for $x > 0$,

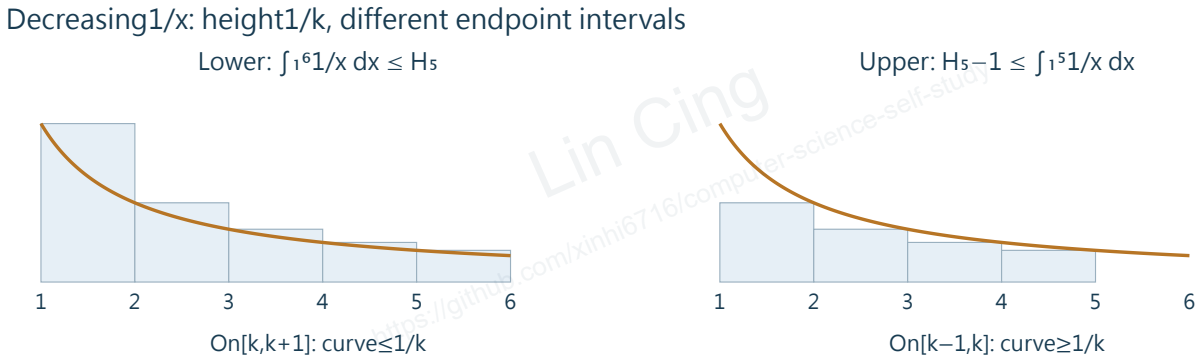
$$H_n \ge \int_{1}^{n+1} \frac{dx}{x} = \ln(n+1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integral bounds follow from unit intervals (continued)

Integrating from zero for an upper bound encounters a singularity. Remove the first term, then bound the terms $k = 2, \ldots, n$:

$$H_n \leq 1 + \int_1^n \frac{dx}{x} = 1 + \ln n.$$



Recomputable two-sided integral bounds

At $n = 5$, $H_5 = 137/60 \approx 2.283333$ lies between $\ln 6 \approx 1.791759$ and $1 + \ln 5 \approx 2.609438$. This bracket has an explicit width; it does not assert equality with $\ln n$. In general,

$$\ln(n+1) \leq H_n \leq \ln n + 1, \qquad H_n = \ln n + O(1).$$

For a fixed $p > 1$,

$$\sum_{k=1}^{n} \frac{1}{k^p} \leq 1 + \int_1^n x^{-p}\, dx \leq 1 + \frac{1}{p-1}.$$

The nonnegative partial sums increase and are bounded, so they converge. The case $p = 1$ diverges. For $0 \leq p < 1$, the integral lower bound grows as n^{1-p}, and the sum is $\Theta(n^{1-p})$. Vanishing terms alone do not imply convergence.

For a fixed $r \geq 0$, integration also gives

$$\frac{n^{r+1}}{r+1} \leq \sum_{k=1}^{n} k^r \leq \frac{(n+1)^{r+1} - 1}{r+1}.$$

Recomputable two-sided integral bounds (continued)

This again proves $\Theta(n^{r+1})$, with constants allowed to depend on the fixed parameter r.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nested loops and changing summation order

Loops with $i = 1, \dots, n$ and $j = i, \dots, n$ perform constant work

$$\sum_{i=1}^{n} (n - i + 1) = \frac{n(n+1)}{2}$$

times. Counting by j instead gives $i = 1, \dots, j$, hence $\sum_{j=1}^{n} j$. Both count the index set $\{(i, j) : 1 \leq i \leq j \leq n\}$.

If an iteration costs j, then

$$\sum_{i=1}^{n} \sum_{j=i}^{n} j = \sum_{j=1}^{n} \sum_{i=1}^{j} j = \sum_{j=1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6}.$$

At $n = 3$, the row sums are $6, 5, 3$, totaling 14. The column count $1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3$ also totals 14.

Finite reordering is safe. An infinite double sum with signed terms requires absolute convergence before arbitrary interchange of order. Finite identities alone do not justify that operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sets, relations, functions, graphs, and trees

Part VIII Appendix

Scope

Use elementwise logic and general proofs to connect sets, equivalence partitions, functions, graphs, and trees. State graph, path, direction, empty-set, and binary-slot conventions explicitly. Prove the six equivalent tree conditions and the bipartite criterion.

Prerequisites

Basic discrete-mathematics notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Apply definitions of sets, relations, and functions precisely.
- Recognize graph and tree structure.
- Connect Cartesian products, equivalence relations, and recursive structures.
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sets

A set is unordered and has no repetition, so $\{u, v, u\} = \{v, u\}$. A multiset retains multiplicity, while a sequence retains positions. These are different objects. Membership is written $x \in A$. The condition $A \subseteq B$ means that every $x \in A$ also lies in B; $A \subsetneq B$ additionally requires $A \neq B$.

The empty set is a subset of every set A, but need not be an element of A. The set $\{\varnothing\}$ has one element and is not empty. Equality requires both inclusions, $A \subseteq B$ and $B \subseteq A$.

Here $\mathbb{N} = \{0, 1, \ldots\}$, $\mathbb{Z}$ denotes the integers, and $\mathbb{R}$ denotes the reals. The expression $\{x \in \mathbb{Z} : 3 \mid x\}$ specifies a subset by a property, rather than an ordered list.

Set operations express elementwise logic. Membership in $A \cup B$ uses "or," membership in $A \cap B$ uses "and," and $x \in A \setminus B$ means $x \in A$ and $x \notin B$. A complement requires a specified universe U, with $A^c = U \setminus A$. Changing U changes the complement.

Set identities quantify over every element

Property	Union form	Intersection form
Identity and zero	$A \cup \varnothing = A$	$A \cap \varnothing = \varnothing$
Idempotence	$A \cup A = A$	$A \cap A = A$
Commutativity	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associativity	$A \cup (B \cup C) = (A \cup B) \cup C$	$A \cap (B \cap C) = (A \cap B) \cap C$
Distributivity	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$
De Morgan	$(B \cup C)^c = B^c \cap C^c$	$(B \cap C)^c = B^c \cup C^c$

For an arbitrary element x,

Set identities quantify over every element (continued)

$$\begin{aligned} x \in A \setminus (B \cap C) &\iff x \in A \ \wedge\ \neg(x \in B \wedge x \in C) \\ &\iff (x \in A \wedge x \notin B) \ \vee\ (x \in A \wedge x \notin C) \\ &\iff x \in (A \setminus B) \cup (A \setminus C). \end{aligned}$$

This two-way equivalence proves the identity for every x. A finite Venn example cannot replace that universal quantifier.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partitions, cardinality, and overlapping counts

A partition of S is a collection of nonempty, pairwise disjoint blocks whose union is S. Every $x \in S$ belongs to exactly one block. For $S = \varnothing$, the empty collection of blocks is allowed. Congruence modulo 3 partitions $\{0, \dots, 7\}$ into $\{0, 3, 6\}$, $\{1, 4, 7\}$, and $\{2, 5\}$.

For finite A, B, split their union into the disjoint parts $A \setminus B$, $A \cap B$, and $B \setminus A$. This gives

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For $A = \{0, 2, 4, 6\}$ and $B = \{2, 3, 6\}$, the union has size $4 + 3 - 2 = 5$. Simply adding cardinalities counts the intersection twice.

Three-set inclusion-exclusion adds the three singleton sizes, subtracts the three pair intersections, and adds the triple intersection. In general, an element that appears in $r \geq 1$ sets receives coefficient

$$\sum_{j=1}^{r} (-1)^{j+1} \binom{r}{j} = 1,$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partitions, cardinality, and overlapping counts (continued)

because $(1-1)^r = 0$. An element in no set receives coefficient 0. This proves inclusion-exclusion element by element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Power sets versus Cartesian products

The power set is $\mathcal{P}(S) = \{T : T \subseteq S\}$. For n distinguishable elements, each subset chooses whether each element is included. This is a bijection with the n-bit strings, so $|\mathcal{P}(S)| = 2^n$. In particular, $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element.

An ordered pair (a, b) retains positions and generally differs from (b, a). Define

$$A \times B = \{(a, b) : a \in A,\ b \in B\}.$$

For finite sets, each fixed a gives $|B|$ pairs. The $|A|$ disjoint rows give $|A \times B| = |A||B|$. An n-fold product counts ordered tuples and multiplies the finite factor sizes. The set A^0 contains the unique empty tuple, so it has size 1.

For $A = \{p, q\}$ and $B = \{0, 1, 2\}$, $A \times B$ has 6 elements. By contrast, $\mathcal{P}(A)$ has the four elements $\varnothing$, $\{p\}$, $\{q\}$, and $\{p, q\}$. A sequence is a function from positions to values. Equal values at distinct positions remain distinct entries.

Infinite cardinality requires bijections

Two sets have equal cardinality when a bijection exists. Being infinite alone does not establish equal cardinality. Define a map from $\mathbb{N}$ to $\mathbb{Z}$ by

$$f(0) = 0, \qquad f(2k-1) = k, \qquad f(2k) = -k \quad (k \geq 1).$$

Every integer has exactly one preimage, so $\mathbb{Z}$ is countable. The map $n \mapsto 2n$ is a bijection onto the even natural numbers. An infinite set can have the same cardinality as a proper subset.

To prove that $\mathbb{R}$ is uncountable, suppose $(0, 1)$ has a list $x_1, x_2, \ldots$. Choose decimal representations that do not end in infinitely many 9s. Define the ith digit of y to be 1 unless the ith digit of x_i is 1, in which case choose 2. Then $y \in (0, 1)$, its representation is unambiguous, and it differs from every x_i at digit i. This contradicts the claimed complete list.

For finite sets of equal size, injection implies surjection. For infinite sets, $f(n) = n + 1$ on $\mathbb{N}$ is injective but misses 0. The finite premise is essential.

Relations

A binary relation is a subset $R \subseteq A \times B$. A relation on one set has $R \subseteq A \times A$. Write aRb when $(a, b) \in R$. A relation need not assign a partner to every input, and a partner need not be unique.

Property	Requirement for all $a, b, c \in A$
Reflexive	aRa
Symmetric	$aRb \implies bRa$
Antisymmetric	$(aRb \land bRa) \implies a = b$
Transitive	$(aRb \land bRc) \implies aRc$

Symmetry and antisymmetry are not negations. Equality satisfies both. For distinct a, b, c, a relation containing $a \leftrightarrow b$ and $b \to c$, but lacking $c \to b$, can satisfy neither. On a nonempty A, the empty relation is vacuously symmetric, antisymmetric, and transitive, but is not reflexive. Symmetry and transitivity do not guarantee the existence of any related pair.

Equivalence relations induce partitions

An equivalence relation is reflexive, symmetric, and transitive. Define $[a] = \{b \in A : aRb\}$. Reflexivity makes each class nonempty and ensures that every a is covered.

Suppose $c \in [a] \cap [b]$. Then aRc and bRc imply aRb by symmetry and transitivity. For any $x \in [a]$, symmetry gives bRa, and transitivity with aRx gives bRx. Thus $[a] \subseteq [b]$. Exchanging a and b gives the reverse inclusion. Classes coincide or are disjoint, and removing duplicates yields a partition.

Conversely, given a partition, define aRb exactly when a and b share a block. Every element lies in one block, proving reflexivity. Sharing a block is symmetric. If a, b share one block and b, c share another, uniqueness of the block containing b makes all three share a block. This proves transitivity. The block containing a is exactly $[a]$, so the constructions correspond.

Congruence modulo 3 illustrates the axioms. The difference $a - b$ is a multiple of 3. Zero proves reflexivity, negation proves symmetry, and addition proves transitivity. The integer classes have residues $0, 1, 2$; restricting them to the earlier finite set gives its three blocks.

Partial orders, total orders, and extrema

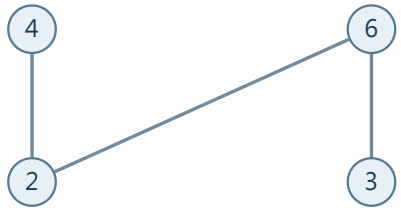
A partial order is reflexive, antisymmetric, and transitive. A total order also compares every pair. A total preorder requires reflexivity, transitivity, and comparability, but permits distinct elements tied in both directions.

A maximum M satisfies $x \leq M$ for every x. A maximal element m merely has no strictly larger element. Minimum and minimal are the dual notions. If M, N are both maxima, then $M \leq N$ and $N \leq M$, so antisymmetry gives $M = N$. A maximum is maximal, but a maximal element need not be a maximum.

On $S = \{2, 3, 4, 6\}$ ordered by divisibility, the minimal elements are $2, 3$ and the maximal elements are $4, 6$. Neither a minimum nor a maximum exists. A Hasse edge is a cover $x < y$ with no z satisfying $x < z < y$. Draw larger elements above smaller ones and omit reflexive and transitive edges.

Partial orders, total orders, and extrema (continued)

S={2,3,4,6}: divisibility Hasse diagram



Upward: larger divisible element

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

No minimum or maximum

Lines are covers, not numerical distances. Reflexive and transitive edges are omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Functions

A function $f : A \to B$ is a relation assigning exactly one $b \in B$ to every $a \in A$. The set A is the domain, B is the codomain, and the range $f(A)$ may be smaller than B. A partial function undefined at some a is not a total function under this definition.

An injection satisfies $f(a) = f(a') \implies a = a'$. A surjection gives every $b \in B$ at least one preimage. A bijection satisfies both properties. The map $f(n) = 2n$ from $\mathbb{N}$ to $\mathbb{N}$ is injective but not onto. Changing its codomain to the nonnegative even integers makes it bijective. The map $n \mapsto \lfloor n/2 \rfloor$ from $\mathbb{N}$ to $\mathbb{N}$ is onto but not injective, since both 0 and 1 map to 0.

For finite sets, an injection uses distinct codomain elements, so $|A| \leq |B|$. A surjection must supply every codomain element, so $|A| \geq |B|$. With equal finite sizes, either property implies the other. The infinite counterexample above prevents extending that inference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverses, preimages, and composition

For a bijection, $f^{-1}(b)$ is the unique a with $f(a) = b$, and

$$f^{-1} \circ f = \mathrm{id}_A, \qquad f \circ f^{-1} = \mathrm{id}_B\,.$$

An injection alone has an inverse only on its range. Without injection, the inverse relation generally is not a function.

For any function and $T \subseteq B$, the set preimage is

$$f^{-1}(T) = \{a \in A : f(a) \in T\}.$$

It is defined without invertibility. It shares notation with an inverse function but is a different object.

For $f : A \to B$ and $g : B \to C$, composition satisfies $(g \circ f)(a) = g(f(a))$. If both maps are injective, equality of composite values lets us cancel g, then f, proving equality of inputs. If both are onto, given $c \in C$, choose $b \in B$ with $g(b) = c$, then $a \in A$ with $f(a) = b$. This proves surjectivity of the composite. For bijections,

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1},$$

Inverses, preimages, and composition (continued)

with the order reversed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graph conventions and degree sums

A finite digraph $G = (V, E)$ has $E \subseteq V \times V$, permitting self-loops but no parallel edges. A simple undirected graph uses unordered pairs of distinct vertices and has no loops. A multigraph must preserve edge identities and specify how loops contribute to degree.

A directed edge (u, v) leaves u and enters v. A self-loop contributes one to each of indegree and outdegree. Undirected degree counts incident edges. Each undirected edge has two endpoints, while each directed edge has one tail and one head. Thus

$$\sum_{v \in V} \deg(v) = 2|E|,$$

$$\sum_{v \in V} \operatorname{outdeg}(v) = \sum_{v \in V} \operatorname{indeg}(v) = |E|.$$

The first identity implies that the number of odd-degree vertices is even. Moving vertices or bending edges does not change E. Crossing lines are not junctions unless marked as vertices. Legends or edge lists must determine directions, identities, and values.

Walks, simple paths, and cycles

A walk $v_0, \ldots, v_k$ follows an edge between each adjacent pair. Its length is k edges, rather than $k + 1$ vertices. CLRS permits repetition in a path; that object is called a walk here. A simple path has distinct vertices. Every vertex has a length-zero path to itself.

For undirected acyclicity, a cycle has length at least 3 and repeats only the start and end vertex. Traversing one edge from u to v and back is not a cycle. Directed simple cycles may have length 1 for a loop or 2 for opposite arcs, and must follow every direction. A general closed walk may repeat other vertices and differs from a simple cycle.

If a walk from u to v repeats $v_i = v_j$ with $i < j$, remove the closed segment between those occurrences. The remaining sequence is a shorter walk with the same endpoints. Repeated removal strictly decreases a finite length and eventually leaves a simple path. This proves equivalence of reachability by walks and by simple paths without listing infinitely many walks.

Components, SCCs, and direction

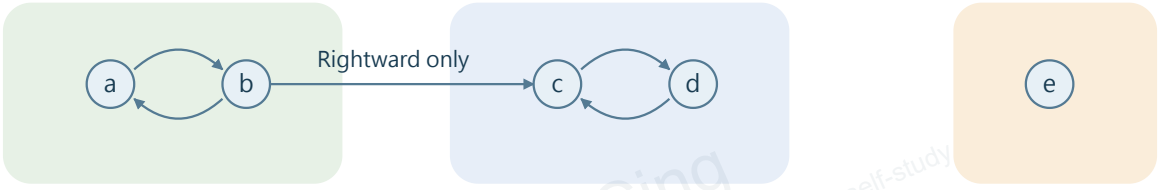
Undirected reachability is reflexive by zero-length paths, symmetric by reversal, and transitive by concatenation. Components are its equivalence classes. Directed reachability is reflexive and transitive, but need not be symmetric. Mutual directed reachability has all three properties; its classes are strongly connected components, or SCCs.

Take the arcs $a \to b$, $b \to a$, $b \to c$, $c \to d$, $d \to c$, and an isolated vertex e. The SCCs are $\{a, b\}$, $\{c, d\}$, and $\{e\}$. Vertex a reaches d, but d cannot reach a, so the first two SCCs remain distinct. Ignoring directions gives only the components $\{a, b, c, d\}$ and $\{e\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Components, SCCs, and direction (continued)

Reachability is directional; mutual reachability differs



a reaches d; d cannot reach a. Ignore direction: a/b/c/d together, e still isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subgraphs, isomorphism, and contraction

A subgraph selects vertices and some original edges, keeping both endpoints of every selected edge. An induced subgraph $G[W]$ keeps all original edges with both endpoints in W. For the preceding example, $W = \{a, b, c\}$ induces exactly $a \to b$, $b \to a$, and $b \to c$.

An isomorphism is a vertex bijection $f : V \to V'$ satisfying

$$(u, v) \in E \iff (f(u), f(v)) \in E'.$$

Preservation in one direction alone is not the general definition. Equal vertex counts, edge counts, and degree multisets are necessary but insufficient. The cycle C_6 and two disjoint triangles both have 6 vertices, 6 edges, and degree 2 everywhere, but only the cycle is connected.

Contracting an undirected edge $\{u, v\}$ merges its endpoints into x, redirects other incident edges, and removes the contracted edge. A simple representation merges parallel copies to a common neighbor and discards loops. Algorithms that need parallel edges, including some randomized contractions, must explicitly use a multigraph instead of silently deduplicating.

Graph classes and the bipartite criterion

The complete graph K_n joins every distinct pair and has $n(n-1)/2$ edges. A bipartite graph has a partition $V = L \sqcup R$ with every edge crossing the two sides, equivalently a proper two-coloring. A DAG is directed and acyclic; trees and forests here are undirected. Absence of directed cycles alone does not make a tree.

A bipartite graph has no odd cycle. Every edge switches sides, and an odd number of switches cannot return to the original side. Conversely, suppose there is no odd simple cycle. Root each component and color vertices by the parity of their shortest undirected distance from the root. An edge joining equal parities would create an odd closed walk by following a root-to-u path, the edge, and a reversed root-to-v path.

If that odd closed walk repeats an internal vertex, split there into two shorter closed walks. Their lengths sum to an odd number, so one is odd. Repeatedly keep the odd part until no internal repetition remains. This is an odd simple cycle, a contradiction. Hence every edge crosses colors, proving bipartiteness.

Graph classes and the bipartite criterion (continued)

A hyperedge may contain many vertices. Its incidence bipartite graph puts original vertices on one side and hyperedges on the other, with an edge for each membership. These are different node types; a hyperedge is not an ordinary vertex pair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trees and forests

A free tree here is a nonempty finite simple undirected graph that is connected and has no simple cycle. A forest is acyclic and may contain multiple components or isolated vertices. Empty forests are allowed. Empty graphs are outside the six-equivalence theorem below.

Every tree with $n \geq 2$ has a leaf. Take an endpoint v of a longest simple path. A neighbor outside the path would extend it. An additional neighbor elsewhere on the path would create a cycle. Thus v has only its adjacent path neighbor and has degree 1.

Removing a leaf and its edge preserves connectivity because simple paths between other vertices cannot use a leaf internally. It also preserves acyclicity. Starting with one vertex and zero edges, induction gives $n - 1$ edges in a tree. For a forest with c components, sum the component tree counts:

$$|E| = \sum_{i=1}^{c} (n_i - 1) = n - c.$$

The identity includes the empty forest, where $n = c = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six equivalent tree conditions

For a nonempty finite simple undirected graph with n vertices and m edges, these conditions are equivalent:

1. The graph is a tree.
2. Each pair of vertices has exactly one simple path between them.
3. The graph is connected, and deleting any edge disconnects it.
4. The graph is connected, with $m = n - 1$.
5. The graph is acyclic, with $m = n - 1$.
6. The graph is acyclic, and adding any missing edge creates a cycle.

$(1) \implies (2)$**.** Connectivity gives a path. If two distinct simple paths existed, their first divergence and subsequent first meeting would delimit internally disjoint segments forming a cycle. This contradicts acyclicity.

$(2) \implies (3)$**.** An edge $\{u, v\}$ is itself the unique simple u-to-v path. Deleting it destroys that reachability.

Six equivalent tree conditions (continued)

$(3) \implies (1)$**.** If a cycle existed, deleting one of its edges would leave the other arc as a replacement in any path using that edge. The graph would remain connected, a contradiction. The leaf-induction count gives $(1) \implies (4)$ and $(1) \implies (5)$. Existence of a path and uniqueness of that path are separate assertions.

Remaining directions and a spanning-tree construction

Every connected graph has a spanning tree. Start with one visited vertex, forming a set S. While $S \neq V$, connectivity supplies an edge from S to its complement. Add that edge and its one new endpoint. Each step attaches one new vertex and creates no cycle. After $n-1$ steps, the selected edges form a spanning tree with $n-1$ edges. In particular, every connected graph has $m \geq n-1$.

$(4) \implies (1)$**.** The constructed spanning tree uses all $n-1$ graph edges. There are no extra edges, so the graph equals that tree.

$(5) \implies (1)$**.** The forest identity $m = n - c$, together with $m = n-1$, gives $c = 1$, proving connectivity.

$(1) \implies (6)$**.** Every new edge $\{u, v\}$ closes the tree's unique u-to-v path into a cycle.

$(6) \implies (1)$**.** If two components existed, adding an edge between them could not create a cycle. This contradicts condition (6), so the graph is connected.

Remaining directions and a spanning-tree construction (continued)

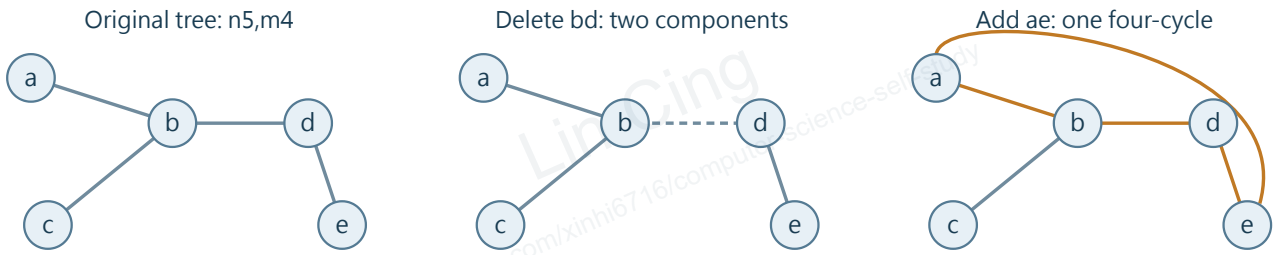
These implications establish all six equivalences. The equality $m = n-1$ alone is insufficient. A triangle plus an isolated vertex has $n = 4, m = 3$ but is not a tree. Acyclicity alone also does not ensure connectivity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same tree under deletion and cycle creation

Let $V = \{a, b, c, d, e\}$ and $E = \{ab, bc, bd, de\}$, so $n = 5, m = 4$. The unique simple path from a to e is a, b, d, e, of length 3. Deleting bd gives components $\{a, b, c\}$ and $\{d, e\}$. Adding ae creates the four-cycle a, b, d, e, a.

Same vertices: remove an edge to disconnect, add one to cycle



Middle dashed edge is deleted. Right orange cycle follows a–b–d–e–a via new edge ea.

A graph with a cycle can still have a spanning tree, although the graph itself is not a tree. Minimum-spanning-tree optimization adds a weight objective that is absent from the definition of a tree.

Rooted and ordered trees

Choose a root r in a free tree. Every nonroot vertex x has as its parent the preceding vertex on its unique simple path from r. Its other, downward neighbors are its children. Here ancestors and descendants include the node itself; proper ancestors and descendants exclude it. The subtree rooted at x contains all descendants of x.

Depth counts edges from the root to a node. A node's height counts edges to its deepest leaf. Leaves have height 0, the root has depth 0, and tree height is the maximum depth. Rooted degree counts children. Graph degree also counts the parent, so it is larger by 1 for each nonroot node.

Root the preceding example at b. Its children are a, c, d, and d has child e. Vertices a, c, d have depth 1, and e has depth 2. The heights are 2 for b, 1 for d, and 0 for the others. Vertex b has both graph degree and child count 3. Vertex d has graph degree 2 and one child.

An ordered tree also orders each child list. Replacing the order (a, c, d) at b by (c, a, d) keeps the rooted relationships but changes the ordered tree.

Binary and positional trees retain empty slots

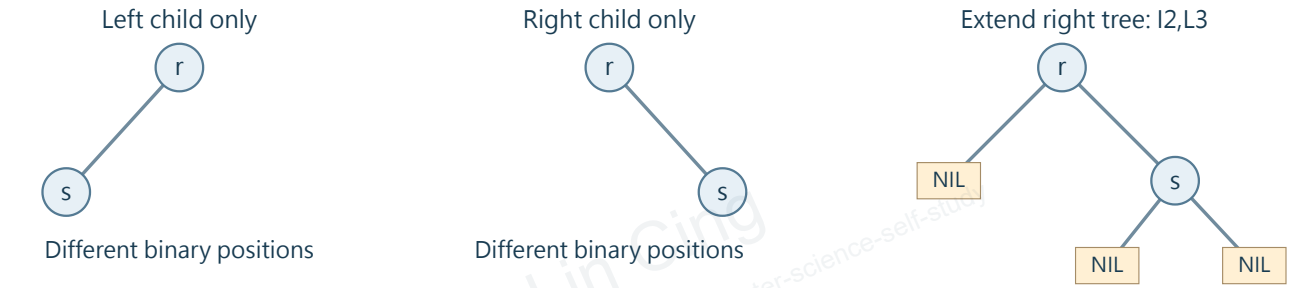
A binary tree is recursively either the empty tree `NIL` or a root with disjoint left and right binary subtrees. A single child in the left slot differs from a single child in the right slot. An ordered tree with one child has no such slot distinction, so an ordered tree of degree at most 2 does not fully specify a binary tree.

A positional k-ary tree has fixed slots $1, \ldots, k$, including meaningful empty positions. Replacing every missing binary child by an external `NIL` leaf gives a full binary tree. Each internal node has two children, and the left and right positions are preserved.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary and positional trees retain empty slots (continued)

Empty slots retain position: left and right differ



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For a nonempty binary tree, n real nodes have $2n$ child slots. The $n - 1$ real parent-child edges occupy that many slots, leaving

$$2n - (n - 1) = n + 1$$

missing positions. The extended tree has n internal nodes and $n + 1$ external leaves. Sharing one `NIL` sentinel object does not change the number of logical positions or require $n + 1$ actual allocations.

Full, complete, and height counts

A full k-ary tree gives each internal node exactly k children. Leaves may have different depths. With I internal nodes and L leaves, counting edges gives

$$kI = I + L - 1, \qquad L = (k-1)I + 1.$$

For binary trees, $L = I + 1$.

CLRS's appendix uses "complete k-ary" to require that all leaves also have depth h; this is often called "perfect" elsewhere. For $k \geq 2$, level d has k^d nodes, and

$$L = k^h, \qquad I = \sum_{d=0}^{h-1} k^d = \frac{k^h - 1}{k - 1}.$$

At $k = 3, h = 2$, the level sizes are $1, 3, 9$, so $I = 4, L = 9$, the total is 13, and there are $12 = 3I$ edges. For $k = 1$, a chain has $I = h, L = 1$; do not divide by $k - 1 = 0$.

Full, complete, and height counts (continued)

Heap chapters often use "complete binary" for a possibly partial final level filled from the left. Check which convention applies. A general nonempty binary tree of height h has at most $2^{h+1} - 1$ nodes, giving

$$h \geq \lceil \log_2(n+1) \rceil - 1.$$

A chain has height $n - 1$. Without balance, binary-tree height need not be $\Theta(\log n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting and probability

Part VIII Appendix

Scope

Define sample objects and common denominators before using counting, conditional probability, Bayes' rule, and indicators. Derive geometric and binomial moments and tail bounds. Distinguish levels of independence, zero-probability and support boundaries, a centered deviation r, and a total threshold t.

Prerequisites

Permutations and combinations, probability axioms, and random variables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Count by products, disjoint sums, and complements.
- Derive expectation and variance for geometric and binomial distributions.
- Bound binomial tails using exponential moments.
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting starts with the sample object

Define what one outcome records. The sum rule requires disjoint cases. The product rule requires a known branch count after each prefix. The available choices may depend on that prefix, provided each level's count is fixed. Variable branching requires summing branches rather than multiplying unspecified averages.

From six students, two distinct office holders give $6 \cdot 5 = 30$ ordered assignments. An unordered two-person committee is counted twice by those assignments, so there are 15 committees. Allowing one person to hold both offices gives $6^2 = 36$ assignments. These count three different objects.

There are q^m length-m strings over q symbols, retaining positions and allowing repetition. Substrings are contiguous; subsequences may skip positions. Specify whether identical contents at distinct positions count separately. For an n-character string and $k \geq 1$, the number of positional length-k substrings is $\max(0, n - k + 1)$. This need not equal the number of distinct contents.

Permutations, combinations, and uniform overcounting

There are $n!$ full permutations of n distinct records. Ordered selection of k records without replacement has

$$P(n, k) = \frac{n!}{(n-k)!}, \qquad 0 \leq k \leq n.$$

Every unordered k-subset has exactly $k!$ orderings, giving

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

The case $k = 0$ gives one empty choice. Extend the coefficient to 0 for $k < 0$ or $k > n$.

Distinct record identities still give $n!$ permutations when keys repeat. If only key strings matter and the multiplicities are $m_1, \ldots, m_s$, each string has exactly $\prod_{i=1}^{s} m_i!$ identity permutations. Thus the number of distinct strings is $n! / \prod_{i=1}^{s} m_i!$. Prove equal overcounting before dividing.

Complementation gives $\binom{n}{k} = \binom{n}{n-k}$. Partitioning subsets by whether they include a specified element gives Pascal's identity:

Permutations, combinations, and uniform overcounting (continued)

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial expansion and entropy bounds

In $(x+y)^n$, there are $\binom{n}{k}$ ways to choose the k factor positions contributing x. Hence

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

Setting $x = y = 1$ gives $\sum_{k=0}^{n} \binom{n}{k} = 2^n$, also counting subsets by size. For $1 \le k \le n$,

$$\binom{n}{k} = \prod_{j=0}^{k-1} \frac{n-j}{k-j} \ge \left(\frac{n}{k}\right)^k,$$

because each factor is at least n/k. Also,

$$\binom{n}{k} \le \frac{n^k}{k!} \le \left(\frac{en}{k}\right)^k,$$

using $k! \ge (k/e)^k$.

Let $\alpha = k/n \in (0,1)$. The nonnegative kth term in the expansion of $(\alpha + 1 - \alpha)^n = 1$ is at most 1. Therefore

Binomial expansion and entropy bounds (continued)

$$\binom{n}{k} \le \alpha^{-k}(1-\alpha)^{-(n-k)} = 2^{nH(\alpha)},$$

where $H(\alpha) = -\alpha \log_2 \alpha - (1-\alpha)\log_2(1-\alpha)$. Extend to the endpoints with $0 \log_2 0 = 0$, giving coefficient 1. Since $H'(\alpha) = \log_2((1-\alpha)/\alpha)$, the maximum is $H(1/2) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probability axioms and uniform outcomes

An elementary outcome records one complete distinguishable result. An event is a measurable subset of the sample space S. On the finite or countable discrete spaces here, every subset is an event. An uncountable space requires a specified event family closed under complements and countable unions; arbitrary subsets are not automatically measurable.

Probability is nonnegative, satisfies $P(S) = 1$, and is finitely or countably additive on pairwise disjoint events. It follows that

$$P(\varnothing) = 0, \qquad P(A^c) = 1 - P(A), \qquad A \subseteq B \implies P(A) \le P(B).$$

In a discrete space, $P(A) = \sum_{s \in A} P(\{s\})$.

The formula $P(A) = |A|/|S|$ requires finitely many equally likely elementary outcomes. Two independent fair bits have four ordered outcomes, each with probability $1/4$. Compressing them to the number of ones gives values $0, 1, 2$ with probabilities $1/4, 1/2, 1/4$, respectively. Those three values are not uniform.

Union bounds need no independence

Disjoint decomposition into the A-only part, the intersection, and the B-only part gives

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \leq P(A) + P(B).$$

For many events, define

$$B_1 = A_1, \qquad B_i = A_i \setminus \bigcup_{j<i} A_j.$$

The B_i are disjoint, have the same union as the A_i, and satisfy $B_i \subseteq A_i$. Additivity and monotonicity give

$$P\left(\bigcup_i A_i\right) = \sum_i P(B_i) \leq \sum_i P(A_i).$$

This also holds for countably many events and needs no independence. If the right side exceeds 1, cap it at 1. By contrast, factoring a general collection of intersection probabilities requires mutual independence.

Union bounds need no independence (continued)

If three failure events each have probability at most 0.01, their union has probability at most 0.03, regardless of dependence. If the three events are identical and each has probability 0.01, their union has probability 0.01. The expression $1 - 0.99^3$ requires independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditional probability and the chain rule

For $P(B) > 0$, conditional probability renormalizes the outcomes retained in B:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(A \cap B) = P(B)P(A \mid B).$$

Repeated application yields

$$P\left(\bigcap_{i=1}^{n} A_i\right) = P(A_1) \prod_{i=2}^{n} P\left(A_i \,\middle|\, \bigcap_{j=1}^{i-1} A_j\right).$$

Every conditioning prefix must have positive probability. A zero-probability prefix already makes the total intersection zero; subsequent undefined ratios must not be multiplied.

A bag contains three blue and two red balls. Drawing twice without replacement gives $P(\text{both blue}) = (3/5)(2/4) = 3/10$. The second draw's blue marginal remains $3/5$, but conditional on a first blue it is $1/2$. Thus the draws are dependent. Replacement with fresh uniform selection gives $(3/5)^2$ instead.

Conditional probability and the chain rule (continued)

Knowing that at least one occurrence happened usually differs from knowing that a specified occurrence happened. Observation and selection rules belong to the model. Two remaining verbal cases need not each have probability $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence, disjointness, and pairwise independence

Events A, B are independent when $P(A \cap B) = P(A)P(B)$. This includes probability-zero events. The equivalent expression $P(A \mid B) = P(A)$ additionally needs $P(B) > 0$. Disjoint events with positive probabilities are dependent. The empty event is independent of every event.

Mutual independence requires the intersection probability of every finite subcollection to factor into the product of its marginals. Pairwise independence checks only pairs and cannot justify factoring three or more events.

Let U, V be independent fair bits, and define $W = U \operatorname{XOR} V$. The outcomes $000, 011, 101, 110$ each have probability $1/4$. Each of the pairs (U, V), (U, W), and (V, W) has all four combinations equally likely, so every pair is independent. Yet

$$P(U = V = W = 1) = 0 \neq \frac{1}{8} = P(U = 1)P(V = 1)P(W = 1).$$

The three bits are not mutually independent.

Independence, disjointness, and pairwise independence (continued)

Pairwise independent, but not mutually independent

U	V	W=U XOR V	P(outcome)
0	0	0	1/4
0	1	1	1/4
1	0	1	1/4
1	1	0	1/4

Each pair:00/01/10/11, each1/4

All three P(=1) marginals are1/2

But111 is absent from sample space

$0 \neq (1/2)^3 = 1/8$

Pair checks do not factor triple intersections. Variances add here; triple products do not.

Total probability and Bayes denominators

Let the B_i form a countable partition of S, with $P(B_i) > 0$. The event A is the disjoint union of $A \cap B_i$, so

$$P(A) = \sum_i P(B_i) P(A \mid B_i).$$

When $P(A) > 0$, Bayes' rule gives

$$P(B_j \mid A) = \frac{P(B_j) P(A \mid B_j)}{P(A)}.$$

A monitor has fault prior 0.02, sensitivity 0.9, and false-positive rate 0.1. Its alarm probability and posterior fault probability are

$$P(\text{alarm}) = 0.02 \cdot 0.9 + 0.98 \cdot 0.1 = 0.116,$$

$$P(\text{fault} \mid \text{alarm}) = \frac{0.018}{0.116} = \frac{9}{58} \approx 0.155172.$$

Sensitivity 0.9 is not posterior probability 0.9. Normal cases contribute 0.098 false alarms to the denominator. If two tests are independent conditional on fault status, then

Total probability and Bayes denominators (continued)

$$P(\text{two alarms}) = 0.02 \cdot 0.9^2 + 0.98 \cdot 0.1^2 = 0.026,$$

$$P(\text{fault} \mid \text{two alarms}) = \frac{0.0162}{0.026} = \frac{81}{130} \approx 0.623077.$$

Squaring the likelihoods requires that conditional independence assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Continuous uniform probabilities

Let X be continuous uniform on $[a, b]$, where $a < b$. For $a \le c \le d \le b$,

$$P(c \le X \le d) = \frac{d - c}{b - a}.$$

Each singleton has probability 0, while the whole interval has probability 1. Countable additivity does not call for summing uncountably many singleton masses.

On $[2, 8]$, $P(3 \le X \le 5) = 2/6 = 1/3$, and removing endpoints changes nothing. The variable $Z = \lfloor X \rfloor$ has values $2, \ldots, 7$, each with probability $1/6$. The value $Z = 8$ occurs only when $X = 8$, with probability 0. Do not assign equal weights to seven integer labels.

The following sections concern discrete random variables and probability mass. A continuous density may exceed 1 even though every singleton has probability zero. Integrate a density over intervals rather than treating it as point probability.

Discrete random variables and joint masses

A random variable $X : S \to \mathbb{R}$ assigns a numerical value to an outcome. The event $\{X = x\}$ is a preimage. Its probability mass function is $p_X(x) = P(X = x)$, with nonnegative values satisfying $\sum_x p_X(x) = 1$. The word “mass” distinguishes discrete probabilities from continuous density.

The joint mass $p(x, y) = P(X = x, Y = y)$ gives marginals

$$p_X(x) = \sum_y p(x, y), \qquad p_Y(y) = \sum_x p(x, y).$$

For $p_Y(y) > 0$, the conditional mass is $p(x, y)/p_Y(y)$. Independence means $p(x, y) = p_X(x) p_Y(y)$ for every pair (x, y).

Let A, B be independent fair four-sided dice, and let $M = \max(A, B)$. Then

$$P(M \le m) = \frac{m^2}{16}, \qquad P(M = m) = \frac{m^2 - (m - 1)^2}{16} = \frac{2m - 1}{16}.$$

For $m = 1, 2, 3, 4$, the masses are $1, 3, 5, 7$ divided by 16. They are not uniform.

Expectation need not be a possible outcome

Expectation is $E[X] = \sum_x x p_X(x)$. Finite support always permits evaluation. On countable signed support, $E[|X|] < \infty$ ensures a finite expectation that may be reordered. A nonnegative X may have infinite expectation. The expression $\infty - \infty$ is not valid arithmetic.

For the dice maximum,

$$E[M] = \frac{1 + 6 + 15 + 28}{16} = \frac{50}{16} = \frac{25}{8},$$
$$E[M^2] = \frac{1 + 12 + 45 + 112}{16} = \frac{170}{16} = \frac{85}{8}.$$

The mean 3.125 is outside the integer support and is still correct.

For any g satisfying $E[|g(X)|] < \infty$, grouping outcomes by their X-preimages gives

$$E[g(X)] = \sum_x g(x) p_X(x).$$

There is no need to list the distinct values of $g(X)$ first. In general, $E[g(X)] \neq g(E[X])$; for the square function, the difference is variance.

Linearity and indicators

For integrable X, Y, expand using their joint mass:

$$\begin{aligned} E[X + Y] &= \sum_x \sum_y (x + y) p(x, y) \\ &= \sum_x x \sum_y p(x, y) + \sum_y y \sum_x p(x, y) \\ &= E[X] + E[Y]. \end{aligned}$$

Absolute integrability justifies splitting and reordering. Independence was not used. Also $E[aX + b] = aE[X] + b$, and repeated application gives linearity for finite sums.

The indicator I_A equals 1 on A and 0 otherwise, so $E[I_A] = P(A)$. If X counts events, then $X = \sum_i I_i$ gives $E[X] = \sum_i P(A_i)$, even with dependence. In a uniform permutation of n distinct keys, each inversion-pair indicator has expectation $1/2$, giving total expected inversions $n(n-1)/4$.

For independent integrable X, Y, joint factorization gives

Linearity and indicators (continued)

$$E[XY] = \left(\sum_x x p_X(x)\right)\left(\sum_y y p_Y(y)\right) = E[X]E[Y].$$

This step uses independence. If $X = Y$ is one fair bit, $E[XY] = 1/2 \neq 1/4$. Products of many variables generally require mutual independence, rather than pairwise independence alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail sums and Jensen

A nonnegative integer variable satisfies, outcome by outcome,

$$X = \sum_{i \geq 1} I_{\{X \geq i\}}.$$

Take expectations of finite truncations, then use monotone limits of nonnegative terms. This gives

$$E[X] = \sum_{i \geq 1} P(X \geq i),$$

possibly with value ∞. A signed variable cannot use this formula directly.

A convex function satisfies

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y), \qquad 0 \leq \lambda \leq 1.$$

For a finite support $x_1, \ldots, x_m$ with weights w_i, let $s = \sum_{i<m} w_i$. When $0 < s < 1$, combine the first $m-1$ points into $z = \sum_{i<m} w_i x_i / s$. Two-point convexity and induction give

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail sums and Jensen (continued)

$$\begin{aligned} f\left(\sum_{i=1}^{m} w_i x_i\right) &= f(sz + w_m x_m) \\ &\le s f(z) + w_m f(x_m) \\ &\le \sum_{i=1}^{m} w_i f(x_i). \end{aligned}$$

Zero weights may be omitted, and the single-point case is equality. Thus $f(E[X]) \le E[f(X)]$.

For the general integrable case, assume f is convex on an interval containing the support and $\mu = E[X]$. A supporting line at an interior μ has

$$f(x) \ge f(\mu) + a(x - \mu).$$

Taking expectations gives $E[f(X)] \ge f(\mu) + a(E[X] - \mu) = f(\mu)$. At a degenerate endpoint, X is concentrated there. The relevant expectations must be defined. Jensen is not permission to move an arbitrary function through expectation.

Variance, covariance, and dependence

For $E[X^2] < \infty$ and $\mu = E[X]$,

$$\operatorname{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2 \ge 0.$$

The standard deviation is $\sigma = \sqrt{\operatorname{Var}(X)}$. Scaling gives $\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$. For the dice maximum,

$$\operatorname{Var}(M) = \frac{85}{8} - \left(\frac{25}{8}\right)^2 = \frac{55}{64}, \qquad \sigma = \frac{\sqrt{55}}{8}.$$

Define $\operatorname{Cov}(X, Y) = E[XY] - E[X]E[Y]$. Expanding squares gives

$$\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X, Y),$$

$$\operatorname{Var}\left(\sum_i X_i\right) = \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j).$$

Pairwise independence and finite second moments suffice to make every covariance zero. Mutual independence is unnecessary for variance addition.

Variance, covariance, and dependence (continued)

Identical marginals alone do not suffice. If all n variables are the same fair bit, their sum equals 0 or n with equal probability and has variance $n^2/4$. Adding individual variances would give only $n/4$. Omitting covariance underestimates uncertainty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov and Chebyshev assumptions

Let $X \geq 0$ have finite mean, and let $t > 0$. The pointwise inequality

$$X \geq tI_{\{X \geq t\}}$$

gives $E[X] \geq tP(X \geq t)$ after taking expectations. Thus Markov's inequality is

$$P(X \geq t) \leq \frac{E[X]}{t}.$$

It requires no independence. It cannot be applied directly to signed variables, where positive and negative cancellation may hide a large positive tail.

For Y with finite second moment, set $X = (Y - \mu)^2$ and use the threshold a^2, where $a > 0$. Markov gives Chebyshev's inequality:

$$P(|Y - \mu| \geq a) = P((Y - \mu)^2 \geq a^2) \leq \frac{\operatorname{Var}(Y)}{a^2}.$$

This is a two-sided bound. It also bounds either one-sided event, although it may be loose.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov and Chebyshev assumptions (continued)

For the maximum M, use $\mu = 25/8$, variance $55/64$, and $a = 2$. The bound is $55/256 \approx 0.214844$. The actual event contains only $M = 1$, with probability $1/16$. A bound is not an equality; values above 1 may be capped at 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric and binomial distributions

A Bernoulli variable with parameter p equals 1 with probability p and 0 with probability $q = 1 - p$. Since $X^2 = X$, it has $E[X] = p$ and $\mathrm{Var}(X) = p - p^2 = pq$. The trial sequences below additionally assume mutual independence and a common p. Equal marginals alone are insufficient.

Here the geometric variable G counts trials including the first success. Its support is $1, 2, \ldots$, with $0 < p \leq 1$. The event $G = k$ requires $k - 1$ failures followed by success, so

$$P(G = k) = q^{k-1}p.$$

At $p = 0$, success never arrives and $G = \infty$, outside this finite-valued distribution.

Counting earlier failures gives $F = G - 1$, with support $0, 1, \ldots$, mass $P(F = f) = q^f p$, and mean q/p. Do not use the trial mean $1/p$ for both conventions. At $p = 1$, $G \equiv 1$, $F \equiv 0$, and both variances are zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric normalization and moments

The geometric masses normalize because

$$\sum_{k\ge 1} q^{k-1}p = \frac{p}{1-q} = 1.$$

For an integer $t \ge 0$, $P(G > t) = q^t$, hence $P(G \ge i) = q^{i-1}$. Tail summation gives

$$E[G] = \sum_{i\ge 1} q^{i-1} = \frac{1}{1-q} = \frac{1}{p}.$$

The weighted geometric identity yields

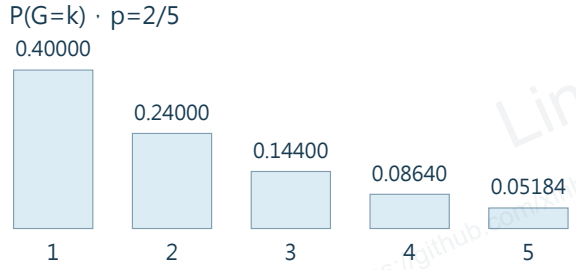
$$\sum_{k\ge 1} k^2 q^{k-1} = \frac{1+q}{(1-q)^3}, \qquad E[G^2] = \frac{1+q}{p^2}.$$

Subtracting the squared mean gives $\operatorname{Var}(G) = q/p^2$. Handle $q = 0$ directly as $G \equiv 1$, without dividing by q.

Geometric normalization and moments (continued)

At $p = 2/5$, the first five masses are $2/5$, $6/25$, $18/125$, $54/625$, and $162/3125$. Their sum is $2882/3125$, and the remaining probability is $P(G > 5) = 243/3125$. The mean is $5/2$ and the variance is $15/4$. A mean of 2.5 averages repeated complete experiments; it does not place a success on a fractional trial.

Fix the counting convention before masses and tails



One outcome: F → F → F → S

G=4: includes the success trial

F=G−1=3: failures only

No success in five: q⁵=243/3125

This tail remains beyond the first five bars

Axis counts trials including first success. E[G]=2.5,E[F]=1.5; the counts differ.

Memorylessness and retry-model boundaries

For $0 < p < 1$ and integers $s, t \geq 0$,

$$P(G > s + t \mid G > s) = \frac{q^{s+t}}{q^s} = q^t = P(G > t).$$

Past failures do not change the next success rate because the trials are independent with a common p. A long failure streak does not make success due.

Changing retry probabilities, persistent shared faults, and sampling without replacement violate that model. Independent trials with unequal p_i instead have tail $\prod_{i=1}^{t}(1 - p_i)$. Dependence may require conditional probabilities rather than q^t.

A fixed trial cost c gives expected total cost c/p. Different costs for failure and success give

$$\text{total cost} = c_{\text{fail}}(G - 1) + c_{\text{success}},$$
$$E[\text{total cost}] = c_{\text{fail}}\frac{q}{p} + c_{\text{success}}.$$

Specify how costs relate to trials before applying linearity.

Binomial mass, mean, and variance

For n mutually independent trials with a common success probability p, let $X = \sum_{i=1}^{n} I_i \in \{0, \ldots, n\}$. A specified set of k success positions has probability $p^k q^{n-k}$. There are $\binom{n}{k}$ disjoint such position sets, so

$$b(k; n, p) = \binom{n}{k} p^k q^{n-k}.$$

Normalization follows from $(p + q)^n = 1$.

Linearity gives $\mu = np$ without independence. Variance addition requires zero pairwise covariances, giving $\operatorname{Var}(X) = npq$. The complete binomial mass formula used mutual independence. Pairwise independence suffices for the mean and variance but does not imply the distribution.

At $n = 0$ or $p = 0$, X is concentrated at 0; at $p = 1$, it is concentrated at n. The endpoint convention $0^0 = 1$ represents an empty product.

Binomial mass, mean, and variance (continued)

Mutually independent trials with unequal p_i instead have a Poisson-binomial distribution, with mean $\sum_i p_i$ and variance $\sum_i p_i(1-p_i)$. Substituting the average p into a binomial mass formula does not make the distributions equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fully checkable binomial table

For $n=8, p=1/4, q=3/4$, the common denominator is $4^8=65536$. The numerators are $\binom{8}{k}3^{8-k}$.

k	0	1	2	3	4	5	6	7	8
Numerator	6561	17496	20412	13608	5670	1512	252	24	1

The numerators sum to 65536. Their k-weighted sum is 131072, giving $E[X]=2$. Their k^2-weighted sum is 360448, giving $E[X^2]=11/2$ and variance $11/2-4=3/2$.

The exact tail is

$$P(X \ge 5)=\frac{1512+252+24+1}{65536}=\frac{1789}{65536}\approx 0.027298.$$

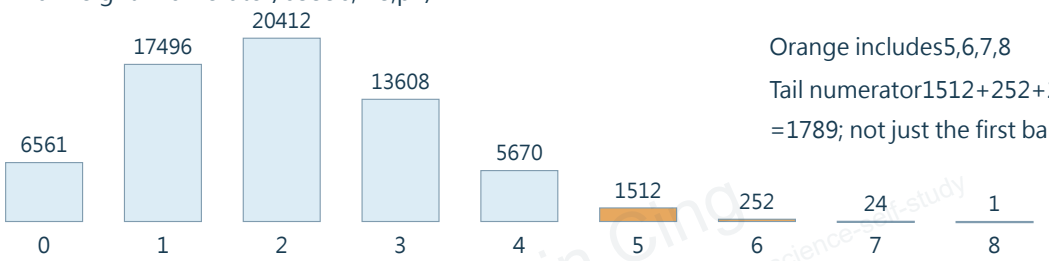
It includes every term through $k=8$, rather than only $b(5)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fully checkable binomial table (continued)

A single mass differs from a tail; threshold5 includes5

Bar height=numerator/65536; n8,p1/4



Orange includes5,6,7,8

Tail numerator1512+252+24+1

=1789; not just the first bar

Numerators sum65536; mean2,variance1.5; P(X≥5)=1789/65536≈.027298.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Modes follow adjacent ratios

For $0 < p < 1$ and $1 \le k \le n$,

$$\frac{b(k)}{b(k-1)} = \frac{(n-k+1)p}{kq}.$$

The ratio exceeds 1 exactly when $k < (n+1)p$, and equals 1 when $k = (n+1)p$.

If $(n+1)p$ is not an integer, the unique mode is $\lfloor (n+1)p \rfloor$. If it is an integer m, both $m-1$ and m are modes. For $n = 8, p = 1/4$, the mode is 2 and coincidentally equals the mean 2. For $n = 4, p = 1/2$, $(n+1)p = 2.5$ gives mode 2. For $n = 3, p = 1/2$, the modes are $1, 2$, while the mean 1.5 has no integer probability bar.

The coefficient and entropy bound gives

$$b(k) \le \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

Interpret endpoints by continuous extension or the 0^0 convention. This bounds one mass, rather than a complete tail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial tails and union bounds

For an integer-valued X, $P(X \ge a)$ sums from $\lceil a \rceil$, whereas $P(X > a)$ starts at $\lfloor a \rfloor + 1$. For an integer k, $P(X < k)$ ends at $k - 1$, unlike $P(X \le k)$.

For $X \sim \mathrm{Bin}(n, p)$, the event $X \ge k$ implies that some specified set of k trials all succeed. Each such set has probability p^k. Applying the union bound over the $\binom{n}{k}$ sets gives

$$P(X \ge k) \le \binom{n}{k} p^k, \qquad 0 \le k \le n.$$

The events need not be disjoint. At $k = 0$, the bound is 1; at $k > n$, the actual event is empty. Apply the same argument to failures $Y = n - X \sim \mathrm{Bin}(n, q)$ to get

$$P(X \le k) \le \binom{n}{k} q^{n-k}, \qquad 0 \le k \le n.$$

At $n = 8, p = 1/4, k = 5$, the upper bound is $56/1024 = 7/128 \approx 0.054688$, compared with the exact tail $1789/65536 \approx 0.027298$. Outcomes with six or more successes are counted repeatedly, which explains why the bound is usually strict.

Deriving a geometric left-tail ratio bound

Assume $0 < p < 1$ and an integer $0 < k < np$. For $1 \le i \le k$,

$$\frac{b(i-1)}{b(i)} = \frac{iq}{(n-i+1)p} < \frac{kq}{(n-k)p} = \rho < 1.$$

Working backward gives $b(k-j) < \rho^j b(k)$ for $j = 1, \dots, k$. Consequently,

$$\begin{aligned} P(X < k) &= \sum_{i=0}^{k-1} b(i) < b(k) \sum_{j \ge 1} \rho^j \\ &= \frac{\rho}{1-\rho} b(k) = \frac{kq}{np - k} b(k). \end{aligned}$$

The denominator is positive by assumption. Substituting $k = np$ is invalid.

If $0 < k \le np/2$, the coefficient is at most $q \le 1$, so $L_k = P(X < k) < b(k)$. Since $L_{k+1} = L_k + b(k)$,

$$\frac{L_k}{L_{k+1}} = \frac{L_k}{L_k + b(k)} < \frac{1}{2}.$$

Moving one integer threshold farther into the left tail at least halves its probability.

Right tails change both variables and thresholds

Let $Y = n - X \sim \mathrm{Bin}(n, q)$. For an integer $np < k < n$, we have $n - k < nq$. Apply the left-tail bound at threshold $n - k$:

$$\begin{aligned} P(X > k) &= P(Y < n - k) \\ &< \frac{(n-k)p}{nq - (n-k)} b(n-k; n, q) \\ &= \frac{(n-k)p}{k - np} b(k; n, p). \end{aligned}$$

If $k > (np + n)/2$, the left-tail halving result gives

$$P(X > k) < \frac{1}{2} P(X > k - 1).$$

This exchanges the roles of success and failure, including their thresholds.

At $n = 8, p = 1/4, k = 4$, the coefficient is

$$\frac{(8-4)(1/4)}{4-2} = \frac{1}{2}.$$

Right tails change both variables and thresholds (continued)

Thus $P(X > 4) < 2835/65536 \approx 0.043259$, compared with the exact tail $1789/65536$. This improves the union bound in this example, but need not do so for every parameter choice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exponential moments turn tails into nonnegative quantities

Let I_i be mutually independent Bernoulli indicators with parameters p_i. Write $X = \sum_i I_i$ and $\mu = \sum_i p_i$. For $\lambda > 0$, monotonicity and Markov give

$$P(X \geq t) \leq e^{-\lambda t} E[e^{\lambda X}].$$

Independence permits factorization. Using $1 + z \leq e^z$,

$$\begin{aligned} E[e^{\lambda X}] &= \prod_i E[e^{\lambda I_i}] \\ &= \prod_i [1 + p_i(e^\lambda - 1)] \\ &\leq \exp[\mu(e^\lambda - 1)]. \end{aligned}$$

Therefore $P(X \geq t) \leq \exp[\mu(e^\lambda - 1) - \lambda t]$. For $\mu > 0, t > \mu$, differentiating the exponent gives $\mu e^\lambda - t = 0$. The minimum occurs at $\lambda = \ln(t/\mu) > 0$, because the second derivative is $\mu e^\lambda > 0$. Substitution gives

$$P(X \geq t) \leq e^{-\mu} \left(\frac{e\mu}{t}\right)^t.$$

Exponential moments turn tails into nonnegative quantities (continued)

Unequal p_i are allowed; mutual independence is required. If $\mu = 0$, every indicator is zero almost surely. All positive-threshold tails are zero directly, without evaluating $\ln(t/0)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Centered r and total t are different thresholds

The looser CLRS form is

$$P(X-\mu \geq r) \leq \left(\frac{e\mu}{r}\right)^r, \qquad r > \mu > 0.$$

To derive it directly, for $\lambda > 0$ bound each centered exponential moment:

$$\begin{aligned} E[e^{\lambda(I_i - p_i)}] &= p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \\ &\leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda}). \end{aligned}$$

Factor the moments using mutual independence and apply Markov:

$$\begin{aligned} P(X-\mu \geq r) &\leq e^{-\lambda r} \prod_i E[e^{\lambda(I_i - p_i)}] \\ &\leq \exp(\mu e^{\lambda} - \lambda r). \end{aligned}$$

The exponent is minimized at $\lambda = \ln(r/\mu) > 0$, giving the stated bound.

Here r is the deviation above the mean, so the total threshold is $t = \mu + r$. In the preceding formula, t counts total successes. With $\mu = 10, r = 40$, the event is $X \geq 50$. The two bounds are

Centered r and total t are different thresholds (continued)

$$\begin{aligned} \left(\frac{e}{4}\right)^{40} &\approx 1.947 \times 10^{-7}, \\ e^{-10}\left(\frac{e}{5}\right)^{50} &\approx 2.650 \times 10^{-18}. \end{aligned}$$

Exchanging 40 and 50 changes the derivation. When r only slightly exceeds μ, the centered bound may exceed 1; cap it at 1. For an integer binomial variable, the exact tail starts at $\lceil \mu + r \rceil$, never a rounded-down threshold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare guarantees on one binomial tail

Let $X \sim \mathrm{Bin}(1000, 0.01)$, so $\mu = 10$, and consider $X \geq 50$.

Bound	Calculation	Upper bound
Markov	$10/50$	0.2
Chebyshev	$9.9/40^2$	0.0061875
Centered, $r = 40$	$(e/4)^{40}$	$\approx 1.947 \times 10^{-7}$
Total threshold, $t = 50$	$e^{-10}(e/5)^{50}$	$\approx 2.650 \times 10^{-18}$

The exact value remains

$$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare guarantees on one binomial tail (continued)

An adjacent-ratio recurrence or log masses avoid floating-point overflow from large factorials. Markov requires nonnegativity and the mean. Chebyshev requires the true variance, which is 9.9 here. Both exponential bounds require mutually independent indicators.

If all indicators instead share the same Bernoulli variable with parameter 0.01, then X is 0 or 1000, and $P(X \geq 50) = 0.01$. Markov still applies, but the binomial variance and exponential-product factorization do not. Equal marginals do not justify the binomial or Chernoff model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix C.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices

Part VIII Appendix

Scope

Entries, dimensions, and field assumptions define matrix arithmetic. Complete elimination, rank factorizations, and inverse traces support proofs of equal row and column rank, determinant laws, and Gram positive definiteness. Real, complex, and finite fields require different inner-product conventions.

Prerequisites

Vector spaces, linear maps, and matrix multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix route

Learning objectives

- Apply matrix arithmetic and transposition.
- Relate rank, determinant, and inverse.
- State the conditions for symmetric positive definiteness and spectral reasoning.
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices and matrix operations

Unless stated otherwise, dimensions are positive integers, entries are real, and vectors are columns. A rank-zero factorization allows zero intermediate columns and rows, with empty sums defined as 0. A matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns. The entry a_{ij} uses the row index first and the column index second. Equality requires equal dimensions and equality of every corresponding entry. Addition and scalar multiplication are defined entrywise.

For example,

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \qquad A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$$

have dimensions 2×3 and 3×2, respectively. Transposition means $(A^T)_{ji} = a_{ij}$. Row-major and column-major storage are memory conventions; the mathematical transpose is defined by this index correspondence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrices and matrix operations (continued)

The vector $x = (1, -1, 2)^T$ is 3×1, while x^T is 1×3. The standard basis vector e_i has a 1 in position i and zeros elsewhere, with dimension specified by context. Zero matrices also require dimensions. The scalar 0 and a 2×3 zero matrix are different types of object.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact zero positions in common square structures

Type	Entry condition for an $n \times n$ matrix
Diagonal	$i \neq j \Rightarrow a_{ij} = 0$
Identity I_n	Diagonal entries are 1; all others are 0.
Tridiagonal	$\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$
Upper triangular	$i > j \Rightarrow a_{ij} = 0$
Lower triangular	$i < j \Rightarrow a_{ij} = 0$
Unit triangular	Triangular, with every diagonal entry equal to 1.
Permutation	Exactly one 1 in each row and each column; all other entries are 0.
Real symmetric	$A = A^T$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact zero positions in common square structures (continued)

These classes can overlap. The identity is diagonal, upper and lower triangular, symmetric, and a permutation matrix. A tridiagonal matrix need not be symmetric because opposite diagonals may differ. Unit triangular does not mean that every entry is 1. A unit vector likewise need not be an all-ones vector.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check dimensions and the contracted index

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the product $C = AB$ belongs to $\mathbb{R}^{m \times p}$, with

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \le i \le m, \quad 1 \le j \le p.$$

The shared index k is summed out; the outer indices i, j remain. This is not entrywise multiplication.

Using the earlier A and $B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$ gives

Entry	Calculation
$(AB)_{11}$	$1 \cdot 2 + 2(-1) + 0 \cdot 4 = 0$
$(AB)_{12}$	$1 \cdot 1 + 2 \cdot 3 + 0 \cdot 0 = 7$
$(AB)_{21}$	$0 \cdot 2 + 1(-1) + 1 \cdot 4 = 3$
$(AB)_{22}$	$0 \cdot 1 + 1 \cdot 3 + 1 \cdot 0 = 3$

Check dimensions and the contracted index (continued)

Thus $AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$. The reverse product is also defined, but has size 3×3:

$$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}.$$

The products differ even in size. Square products also need not commute; scalar commutativity does not transfer to matrix multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check dimensions and the contracted index (continued)

Check sizes, then sum three aligned k terms

A : 2 rows × 3 columns

1	2	0
0	1	1

×

B : 3 rows × 2 columns

2	1
-1	3
4	0

=

C=AB : 2×2

0	7
3	3

C[1,2] = row1 of A · column2 of B = 1×1+2×3+0×0 = 7

Yellow row/column lengths3 share k. Output retains row1,column2; not entrywise multiplication.

Implementable multiplication and exact cost

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

For $n \geq 1$, this uses mnp scalar multiplications, $mp(n-1)$ additions, and mp output writes, plus loop control. Constant-cost scalar arithmetic in the RAM model gives $\Theta(mnp)$ time, or $\Theta(n^3)$ for square matrices. Initializing $s = 0$ and doing n additions instead adds mp additions. That is a different exact count with the same asymptotic order.

Output entries must not alias input entries that will be read later, since writes could corrupt those reads. Arbitrary-precision integers and field elements require separate bit costs. Performing n^3 scalar multiplications does not guarantee n^3 time for arbitrary bit lengths.

Associativity, distributivity, and compatible dimensions

Let A be $m \times n$, B be $n \times p$, and C be $p \times q$. For every valid output index (i, j),

$$\begin{aligned}((AB)C)_{ij} &= \sum_{k=1}^{p} \sum_{\ell=1}^{n} a_{i\ell} b_{\ell k} c_{kj} \\ &= \sum_{\ell=1}^{n} a_{i\ell} \left(\sum_{k=1}^{p} b_{\ell k} c_{kj} \right) = (A(BC))_{ij}.\end{aligned}$$

Reordering is valid because the sums are finite. Distributivity follows from scalar distributivity at each entry. Identity dimensions must match: $I_m A = A I_n = A$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transposition reverses factor order

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, compare position (j, i) on both sides of $(AB)^T = B^T A^T$:

$$\begin{aligned}((AB)^T)_{ji} &= \sum_{k=1}^{n} a_{ik} b_{kj}, \\ (B^T A^T)_{ji} &= \sum_{k=1}^{n} (B^T)_{jk} (A^T)_{ki} = \sum_{k=1}^{n} b_{kj} a_{ik}.\end{aligned}$$

The entries agree by **scalar** commutativity. The matrix product itself need not commute; transposition reverses the factor order.

For real symmetric matrices A, B of the same size, their sum and difference remain symmetric. Their product is symmetric if and only if $AB = BA$, since $(AB)^T = BA$. Two symmetric inputs alone do not imply a symmetric product.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-vector, inner, and outer products

Write the columns of $A \in \mathbb{R}^{m \times n}$ as $a_1, \dots, a_n$. For $x \in \mathbb{R}^n$, the product $Ax = \sum_{j=1}^{n} x_j a_j$ is an m-vector. The earlier example gives

$$Ax = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix},$$

which matches the row dot products.

For equal-length real vectors u, v, the inner product $u^T v$ is a scalar and the outer product uv^T is a matrix. With $u = (1, 2)^T$ and $v = (3, -1)^T$,

$$u^T v = 1, \qquad uv^T = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}.$$

Column j is $v_j u$, so the rank is at most one. If both vectors are nonzero, at least one column is a nonzero multiple of u and the rank is one. If either vector is zero, the rank is zero.

The real norm is $\|x\|_2 = \sqrt{x^T x}$. Complex vectors require $x^* x = \sum_i |x_i|^2$, where $*$ denotes conjugate transpose. For $x = [i]$, ordinary transpose gives $x^T x = -1$, which is not squared length.

Permutations act differently on rows and columns

For

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},$$

one has $Px = (x_3, x_1, x_2)^T$. If A has three rows, PA selects its old rows in the order $3, 1, 2$. In $(PA)_{ij} = \sum_{k=1}^{3} p_{ik} a_{kj}$, each row of P selects one term.

For a matrix M with three columns, MP selects its old columns in the order $2, 3, 1$, because the columns of P are e_2, e_3, e_1. Derive the direction from entries or basis columns, rather than memorizing one order for both sides.

Distinct basis columns give $P^T P = I_3 = PP^T$, so $P^{-1} = P^T$. Products of permutation matrices remain permutation matrices because composing two basis reorderings is a bijection.

Rank and null vectors

Vectors $v_1, \ldots, v_n$ are linearly independent if $\sum_{j=1}^{n} c_j v_j = 0$ forces every coefficient $c_j = 0$; otherwise they are dependent. Column rank is the largest number of independent columns, and row rank is defined analogously. Zero matrices have rank zero.

Since $Ax = \sum_j x_j a_j$, column independence is equivalent to $Ax = 0$ having only the solution $x = 0$. A nonzero solution is a nonzero null vector. The null space always contains 0; absence of a nonzero null vector does not mean the null space contains no vectors.

The matrix

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$$

has eight nonzero entries but rank two. Its third column is the sum of the first two, which are independent. A null vector is $(-1, -1, 1)^T$. Rank is neither the nonzero-entry count nor the number of nonzero entries on the original diagonal.

Elimination operations and intermediate states

Row swaps, nonzero row scaling, and adding a multiple of another row are invertible operations. Each is left multiplication by an invertible elementary matrix. They preserve column-dependence coefficients and row span.

For M, the complete elimination trace is

$$M \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \xrightarrow{R_2 \leftrightarrow R_3} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{R_1 \leftarrow R_1 - 2R_2} R = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Pivot columns $1, 2$ leave x_3 free, with $x_1 = x_2 = -x_3$. Hence the null space is $\{t(-1, -1, 1)^T : t \in \mathbb{R}\}$. The two nonzero rows have distinct leading positions and are independent. The nonpivot column is the sum of the pivot columns.

Elimination operations and intermediate states (continued)

M=BC: dimension2 from original pivot columns

Original M:3×3,rank2

1	2	3
2	4	6
0	1	1

=

B: original pivot columns

1	2
2	4
0	1

×

C: column coordinates

1	0	1
0	1	1

M.column3=column1+column2, so C.column3=$(1,1)^{T}$.

B retains original vectors. Reduced unit columns give coordinates and cannot replace B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Gauss–Jordan without assuming diagonal pivots

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

Before each step, the first r pivot columns are distinct unit columns, and unused rows are zero in those columns. A new pivot comes from a remaining row. Normalizing it and eliminating elsewhere preserves old pivot columns because its old pivot coordinates are zero. Skip a column only when every remaining row is zero there. The finite loops produce r pivots and zero rows in $O(mn\min(m,n))$ field operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Gauss–Jordan without assuming diagonal pivots (continued)

Exact comparison with zero follows the field arithmetic. Floating-point arithmetic needs pivoting, scale, and tolerance analysis. A fixed rule such as $|\text{pivot}| < 10^{-12}$ is not a universally correct mathematical definition of rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why row rank equals column rank

Let $R = EA$ be the reduced row echelon form, with E invertible. Applying the same reversible map to every column preserves exactly their dependence coefficients. The r pivot columns of R are independent, and every other column is their combination. Thus the column rank of A is r.

Invertible row operations preserve row span. The r nonzero rows of R are independent: their successive pivot coordinates force each coefficient in a dependence relation to vanish. Thus the row rank of A is also r. Write $\text{rank}(A)$ for this common value, between 0 and $\min(m, n)$.

The nullity is $n - r$. Free coordinates uniquely determine the pivot coordinates. Set one free variable at a time to 1 and the others to 0 to construct a null-space basis. These vectors are independent by their free coordinates and span every solution. This proves the dimension statement generally, beyond the example M.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank factorization and product bounds

Choose the r original pivot columns of A to form $B \in \mathbb{R}^{m \times r}$. Put the coordinates of every original column relative to B in $C \in \mathbb{R}^{r \times n}$. Then $A = BC$, the columns of B are independent, and C contains identity columns. Conversely, if $A = UV$ with s intermediate columns, every column of A lies in the span of U, so $\operatorname{rank}(A) \leq s$. Therefore r is the minimum possible intermediate dimension.

For M,

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \qquad M = BC.$$

The pivot vectors of R are not the original vectors of M; row operations changed their entries.

For compatible A, D, factor $A = B_r C_r$ to get $AD = B_r(C_r D)$, so $\operatorname{rank}(AD) \leq \operatorname{rank}(A)$. Factoring $D = U_s V_s$ similarly gives $AD = (AU_s)V_s$ and the bound by $\operatorname{rank}(D)$. Thus

$$\operatorname{rank}(AD) \leq \min\{\operatorname{rank}(A), \operatorname{rank}(D)\}.$$

Rank factorization and product bounds (continued)

For an invertible left factor P, the identity $A = P^{-1}(PA)$ proves the reverse inequality, giving $\operatorname{rank}(PA) = \operatorname{rank}(A)$. Invertible right multiplication is analogous.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverse existence and uniqueness

An inverse B of a square $n \times n$ matrix A satisfies $AB = I_n = BA$. If B, C are both inverses, $B = B(AC) = (BA)C = C$. For invertible A, B, the product inverse is $(AB)^{-1} = B^{-1}A^{-1}$, as multiplication on both sides verifies. Transposing the two inverse equations gives $(A^{-1})^T = (A^T)^{-1}$.

Square full rank is equivalent to no nonzero null vector, hence to injectivity of $x \mapsto Ax$. The n independent columns form a basis of n-space. Each e_j therefore has a unique solution $Ab_j = e_j$. Assemble these columns into B to obtain $AB = I_n$. Then $A(BA - I_n) = 0$ and injectivity imply $BA - I_n = 0$ columnwise. Thus full rank is equivalent to invertibility. Conversely, multiplying $Ax = 0$ by the inverse gives $x = 0$.

A rectangular one-sided inverse need not be two-sided. For $A = \begin{bmatrix} 1 & 0 \end{bmatrix}$ and $B = (1, 0)^T$, one has $AB = I_1$ but $BA = \operatorname{diag}(1, 0) \neq I_2$. The square premise cannot be dropped.

A fully checkable inverse trace

Let $H = \begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}$. Apply each row operation to both halves of $[H \mid I_2]$:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 3 & 2 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - (3/2)R_1} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1/2 & -3/2 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow 2R_2} \left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

Finish and verify the inverse

Subtract row two from row one, then normalize row one:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

$$\left[\begin{array}{cc|cc} 2 & 0 & 4 & -2 \\ 0 & 1 & -3 & 2 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1/2} \left[\begin{array}{cc|cc} 1 & 0 & 2 & -1 \\ 0 & 1 & -3 & 2 \end{array}\right].$$

The right half is $H^{-1} = \begin{bmatrix} 2 & -1 \\ -3 & 2 \end{bmatrix}$. Both checks give the identity:

$$HH^{-1} = \begin{bmatrix} 4-3 & -2+2 \\ 6-6 & -3+4 \end{bmatrix} = I_2, \qquad H^{-1}H = \begin{bmatrix} 4-3 & 2-2 \\ -6+6 & -3+4 \end{bmatrix} = I_2.$$

Elimination applies E to the left half while turning the right half into E. When $EH = I_2$, E is the inverse. Without all n left-side pivots for an $n \times n$ matrix, the right half cannot simply be declared an inverse.

Determinants, minors, and cofactors

For a square matrix A, the minor matrix $A[ij]$ removes row i and column j. The cofactor is $C_{ij} = (-1)^{i+j} \det A[ij]$. For $n = 1$, define $\det[a] = a$. The optional 0×0 convention $\det(\varnothing) = 1$ unifies cofactor formulas. Expansion along row i gives $\det A = \sum_{j=1}^{n} a_{ij} C_{ij}$.

For $N = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 2 \\ 0 & -1 & 1 \end{bmatrix}$, first-row signs $+, -, +$ give

$$\det N = 2\big(3 \cdot 1 - 2(-1)\big) - 1(1 \cdot 1 - 2 \cdot 0) + 0 = 10 - 1 = 9.$$

A minor is a matrix, its determinant is a scalar, and a cofactor adds a sign. These are distinct objects.

In general, with S_n the permutations of $\{1, \ldots, n\}$,

$$\det A = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma(i)}.$$

Each term selects one entry per row and per column, with a permutation-parity sign. Grouping terms by the selected column in a fixed row gives cofactor expansion.

Reasons for determinant laws

The permutation formula is linear separately in every column. With two equal columns, pair permutations that exchange those columns; the terms cancel, giving determinant zero. Swapping columns changes the sign. The same arguments apply to rows, and the bijection $\sigma \mapsto \sigma^{-1}$ gives $\det(A^T) = \det A$. Scaling a row by α scales the determinant by α. Adding a multiple of another row leaves it unchanged because the extra term from multilinearity has a repeated row.

For any alternating multilinear scalar function F of the columns, expand each column in the standard basis. Repeated basis indices vanish. The remaining permutation terms satisfy

$$F(e_{\sigma(1)}, \dots, e_{\sigma(n)}) = \operatorname{sgn}(\sigma) F(I_n).$$

Thus $F(B) = F(I_n) \det B$. Fix A and take $F(B) = \det(AB)$. Each product column is A times a column of B, so F is alternating multilinear and $F(I_n) = \det A$. This proves $\det(AB) = \det A \det B$ without assuming A invertible. If $\det A = 0$, the function is identically zero.

Determinants and singularity beyond the 2×2 formula

For an upper-triangular matrix, a nonzero permutation term requires $\sigma(i) \geq i$ for every i. Since $\sum_i \sigma(i) = \sum_i i$, every inequality is an equality. Only the diagonal term remains, so the determinant is the diagonal product. The lower-triangular case follows symmetrically or by transpose. A square echelon matrix with a zero row has determinant zero; with all n pivots nonzero, it has a nonzero diagonal product.

Swaps and nonzero scalings multiply the determinant by nonzero factors; row addition preserves it. Therefore

$$\begin{aligned} \det A = 0 &\iff \text{a pivot is missing} \\ &\iff \operatorname{rank}(A) < n \\ &\iff A \text{ is singular.} \end{aligned}$$

A nonzero determinant proves invertibility, without guaranteeing good numerical conditioning. For $A = \operatorname{diag}(1, \varepsilon)$ with $\varepsilon \neq 0$, exact full rank holds but the inverse contains $1/\varepsilon$. Small ε can amplify errors. Stable factorizations usually solve $Ax = b$ without explicitly forming the inverse. Mathematical existence and numerical reliability are different issues.

Real symmetric positive definiteness and Gram matrices

Here positive definite means a **real symmetric** matrix S with $x^T S x > 0$ for every nonzero real x. Positive semidefinite replaces $>$ with $\geq$. Without symmetry, a quadratic form sees only $(S + S^T)/2$; positivity alone does not imply $S = S^T$.

For real $A \in \mathbb{R}^{m \times n}$, the Gram matrix $G = A^T A$ is symmetric and

$$x^T G x = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0.$$

Equality holds exactly when $Ax = 0$. Thus G is positive definite exactly when A has no nonzero null vector, or equivalently has full column rank. This is impossible when $n > m$. A merely nonzero A is insufficient.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check Gram positivity on all vectors

For

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

the vector $x = (s, t)^T$ gives $x^T X^T X x = s^2 + t^2 + (s + t)^2 > 0$ unless $s = t = 0$.

For

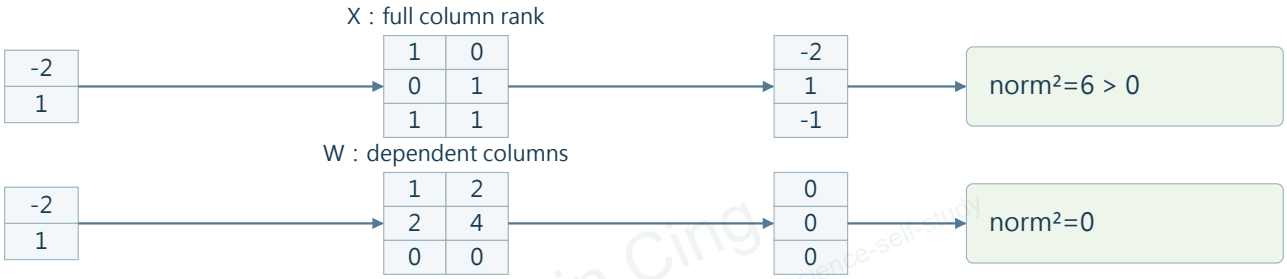
$$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}, \qquad W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix},$$

the quadratic form is $5(s + 2t)^2$. The nonzero vector $(-2, 1)^T$ has $Wx = 0$, so this Gram matrix is only positive semidefinite.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check Gram positivity on all vectors (continued)

Compare x^TA^TAx=norm(Ax)² at two ranks



Same nonzero $x=(-2,1)^T$. Positive definiteness needs all nonzero x; one null vector refutes W.

Complex and finite fields change some premises

Field arguments for multiplication, inverses, rank, and determinants extend to $\mathbb{C}$ and prime fields $\mathbb{F}_p$. Real order and Euclidean inner products do not transfer unchanged. Arithmetic modulo a composite number forms a ring in which nonzero elements may be noninvertible. For example, 2 modulo 6 has no inverse, so pivot normalization and field-rank arguments require reconsideration.

The complex conjugate transpose is A^*. A Hermitian matrix satisfies $S^* = S$. The Gram matrix A^*A is positive semidefinite and is positive definite for full column rank because

$$x^*A^*Ax = \sum_i |(Ax)_i|^2.$$

Ordinary A^TA need not be positive. For $A = [i]$, one has $A^TA = [-1]$ but $A^*A = [1]$.

For real or complex square A, the algebraic transpose identity is $\det(A^TA) = (\det A)^2$. The conjugate version is $\det(A^*A) = |\det A|^2 \geq 0$. The first expression implies nonnegativity only for real entries. Distinguish algebraic transposition from conjugation in inner products.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Appendix D.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study